

Microservices, System Design & Design Patterns

Knowledge Boost

Copyright © 2026 Titus. All rights reserved.

No part of this document may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without prior written permission of the author.

A brief about this eBook:

This is not just a list of questions and answers.

It is a clear learning path for Microservices, System Design, and Design Patterns.

The topics are arranged in a logical order, starting from basic concepts and slowly moving towards real world architecture decisions.

- It contains, Microservices conceptual questions and Scenarios.
- It covers System design conceptual topics and high-level designs like Instagram, YouTube. Real world designs like how platforms like WhatsApp, Hot Star, Uber work under the hood.
- SOLID Principles, Design patterns and Questions on design patterns.
- On top of that, it also includes:
 - **Reference articles** for in-depth learning
 - **Video links** especially for High level designs.
 - **Code examples** for Solid Principles and Design patterns.
 - **Real-time examples** for all the microservices concepts

Category	Questions	Page Number
Microservices Conceptual Questions	134	22
Microservices Scenarios	59	177
System Design Concepts	63	211
High Level Designs (Amazon, Insta, Hotstar etc)	22	271
Real world designs	21	395
SOLID Principles and Design patterns	35	40
Design patterns Questions	23	515

This document will be updated on the **15th** of every month. Keep the purchase email safe to access the updated version. Click on Access Purchase to get updated document.

Congratulations, your purchase is here! 🎉

Hi Abhi,

Thank you for buying Java and Spring Knowledge Boost 🙌

Super excited to see how it adds value to your life, I would wish to hear your feedback on the same 🙏

[Access Purchase](#)

If you like it, please give a testimonial [here](#)

Feel free to reach out if you need any help from me, [here!](#)

Microservices and system design are not about writing code. They are about understanding how systems behave, scale, and fail. They need strong basics of Java and Spring.

Java & Spring Knowledge boost eBook: <https://topmate.io/codinglyf/1746709> . This book focuses deeply on **Java and Spring**, explaining concepts in a clear and practical way.

Contents

1. Microservice: Content Outline.....	15
2. System Design: Content Outline	19
3. High level designs: Content Outline	20
4. Design Patterns: Content Outline	21
5. Microservice 1: What is Microservice?.....	22
6. Microservice 2: What is the difference between Monolithic and microservice?	23
7. Microservice 3: Is Microservices an Architectural Pattern?.....	24
8. Microservice 4: Difference between Architectural Pattern and Design pattern?	24
9. Microservice 5: What is architectural pattern?	25
10. Microservice 6: What is a Distributed System?	25
11. Microservice 7: What are the advantages of Microservice?	26
12. Microservice 8: What are the disadvantages or challenges of Microservice?	27
13. Microservice 9: What are the challenges you see while developing the microservices?	28
14. Microservice 10: When to choose monolithic and when to use microservice?	29

15.	Microservice 11: What is load balancer and how it plays important role in microservices?	30
16.	Microservice 12: What is are the types of load balancers?	31
17.	Microservice 13: How load balancing works internally or Algorithms of Load Balancer? ...	32
18.	Microservice 14: Explain Microservice architecture?	33
19.	Microservice 15: What are the design patterns of microservice.?	34
20.	Microservice 16: What is the Domain Driven Design Pattern?	37
21.	Microservice 17: What is Decomposition pattern or how you decompose a business into microservice?.....	38
22.	Microservice 18: What is Strangler Pattern?	38
23.	Microservice 19: What are benefits of using Strangler Pattern?.....	39
24.	Microservice 20: What are disadvantages or challenges of Strangler pattern?	40
25.	Microservice 21: What is Side Car Pattern?	40
26.	Microservice 22: What are the Database design patterns in microservices?	41
27.	Microservice 23: What is distributed transaction?	42
28.	Microservice 24: What is data consistency and how we achieve in Microservices?	43
29.	Microservice 25: What is Saga design pattern?	44
30.	Microservice 26: How do you implement saga pattern in Spring?.....	45
31.	Microservice 27: What are advantages and disadvantages of choreography pattern?	48
32.	Microservice 28: What are advantages and disadvantages of orchestration pattern?	48
33.	Microservice 29: Questions on Saga pattern.....	49
34.	Microservice 30: What is Two-Phase commit pattern?	50
35.	Microservice 31: What is difference between Saga and Two-Phase commit pattern?	52
36.	Microservice 32: When to use Saga and Two-Phase Commit?.....	53
37.	Microservice 33: What is CQRS Pattern?	53
38.	Microservice 34: When to use CQRS Pattern?.....	56
39.	Microservice 35: Questions on CQRS Pattern	56
40.	Microservice 36: What is Event-Driven database pattern?	57
41.	Microservice 37: What is Event Driven Architecture?	58
42.	Microservice 38: What is Eventual Consistency?	59
43.	Microservice 39: What is the recommended approach when two microservices using the same database?.....	60

44.	Microservice 40: What is communication pattern and its types?	61
45.	Microservice 41: How do you achieve Synchronous Communication in Microservices?	61
46.	Microservice 42: What do you know about GraphQL?	64
47.	Microservice 43: Questions on GraphQL?	67
48.	Microservice 44: When to use GraphQL?	70
49.	Microservice 45: What is gRPC and how it is useful?	71
50.	Microservice 46: What is asynchronous communication and how it is useful?	72
51.	Microservice 47: How can you implement Asynchronous communication in Microservice? 73	
52.	Microservice 48: When to use Asynchronous and when to use synchronous communication?	74
53.	Microservice 49: How does Apache Kafka works and why it is fast?	74
54.	Microservice 50: Explain the Kafka architecture?	77
55.	Microservice 51: Can you tell some real-world use cases for Kafka?	78
56.	Microservice 52: What are message queues?	79
57.	Microservice 53: What is message broker?	81
58.	Microservice 54: When to use message queues?	82
59.	Microservice 55: When to use Kafka, RabbitMQ and Pub/sub?	83
60.	Microservice 56: Implementations of Kafka and RabbitMQ in Spring boot?	84
61.	Microservice 57: Difference between Kafka and RabbitMQ?	84
62.	Microservice 58: What are the integration patterns used in Microservice?	85
63.	Microservice 59: What is API gateway pattern and why we use it?	86
64.	Microservice 60: What are the features of API gateway?	87
65.	Microservice 61: What are the ways to implement API gateway in Spring?	88
66.	Microservice 62: How API gateway works as load balancer?	91
67.	Microservice 63: What is difference between API gateway and load-balancer?	92
68.	Microservice 64: Can API Gateway replace load balancer completely?	92
69.	Microservice 65: Real time uses of API Gateway?	93
70.	Microservice 66: Some more Questions on API Gateway	93
71.	Microservice 67: If API gateway is single entry point, how does it handle millions of requests?	95
72.	Microservice 68: What is the Aggregator pattern?	96

73.	Microservice 69: What is the API Composition and how does it different from Aggregator?	98
74.	Microservice 70: How API Composition is different from GraphQL?	99
75.	Microservice 71: What is service discovery and service registry?	99
76.	Microservice 72: What is Server-side discovery and Client-side discovery.?	100
77.	Microservice 73: When to choose Server-side discovery and Client-side discovery?	101
78.	Microservice 74: How do you implement Service registry in Spring?	102
79.	Microservice 75: How Eureka server works internally?	106
80.	Microservice 76: If two or more instances of same service are running, how Eureka client decides which service to call?	107
81.	Microservice 77: What is the difference between API Gateway and Service Discovery?	108
82.	Microservice 78: How API Gateway and Eureka server works together?	109
83.	Microservice 79: Is Service discovery mandatory to have in distributed systems?	111
84.	Microservice 80: What is the deployments pattern we use Microservice?	112
85.	Microservice 81: What is End to End Deployment Steps we follow in Microservice?	112
86.	Microservice 82: What is containerization and why it useful?	116
87.	Microservice 83: How are Docker and Kubernetes related to containers??	119
88.	Microservice 84: What is Docker, how it useful and how it works?	120
89.	Microservice 85: Difference of Virtual machine, container and Docker?	122
90.	Microservice 86: List out some docker commands?	123
91.	Microservice 87: What is Kubernetes and how it is useful in Microservices?	124
92.	Microservice 88: Explain the architecture of Kubernetes?	125
93.	Microservice 89: What are the types of deployments?	127
94.	Microservice 90: What is the role of continuous integration (CI) and continuous deployment (CD) in Microservices??	128
95.	Microservice 91: Can we use embedded server in microservice deployment for production?	129
96.	Microservice 92: What observability design pattern?	130
97.	Microservice 93: What is monitoring in the microservices?	132
98.	Microservice 94: What is centralized logging in the microservices?	133
99.	Microservice 95: What is distributed tracing in the microservices?	133
100.	Microservice 96: What are the best practices we use for monitoring?	134

101. Microservice 97: What are tools and techniques we use for monitoring and logging in the microservices?	134
102. Microservice 98: What are the cross-cutting pattern in Microservices?	135
103. Microservice 99: What is Resilience or Resilient System?	137
104. Microservice 100: How do you achieve resilience?	138
105. Microservice 101: What is Fault Tolerance?	138
106. Microservice 102: What is the difference between fault tolerance and Resilience?	139
107. Microservice 103: What is circuit-breaker pattern?	139
108. Microservice 104: How do you implement Circuit breaker in Spring?	141
109. Microservice 105: When to use Circuit breaker in Spring?	143
110. Microservice 106: What is retry mechanism and how you achieve it?	144
111. Microservice 107: What is Rate limiting and how do you implement in Microservices? ..	146
112. Microservice 108: How can you implement Rate Limiter in Spring?	147
113. Microservice 109: What are the Rate limiting algorithms?	149
114. Microservice 110: Explain about Token bucket algorithm	150
115. Microservice 111: What is the Bulkhead pattern, and how does it prevent system-wide failures?	153
116. Microservice 112: How to implement bulkhead pattern using Spring?	154
117. Microservice 113: What is difference between Rate-limiting and bulkhead pattern?	155
118. Microservice 114: How do you handle failures in Microservices?	156
119. Microservice 115: Explain about security in microservices?	157
120. Microservice 116: How do you manage the configurations in distributed systems?	158
121. Microservice 117: What is Local-Cache and Distributed-Cache?	160
122. Microservice 118: What is In-memory caching and give any example?	161
123. Microservice 119: How do you achieve caching in distributed systems?	162
124. Microservice 120: What are pros and cons of caching?	163
125. Microservice 121: What is Cache Invalidation?	163
126. Microservice 122: Explain Cashe Aside, Write Through and Write-Behind?	165
127. Microservice 123: Where would you place caching and which is best (DB level, Service Level or API Gateway)?	166
128. Microservice 124: What is Spring cloud and how it is useful in Microservices?	167
129. Microservice 125: What do you know about feign-client?	168

130.	Microservice 126: How Feign-client work internally?.....	170
131.	Microservice 127: How does Feign Client know which service URL to call in a microservice system?	170
132.	Microservice 128: How Feign client works if Service registry is not used in the project?..	171
133.	Microservice 129: How feign-client is different from Rest -Template and Web Client?	171
134.	Microservice 130: When to use feign-client, Rest -Template and Web Client?	172
135.	Microservice 131: What are the advantages of Feign Client?	173
136.	Microservice 132: What is service mesh and how it is useful in microservices??	173
137.	Microservice 133: How service mesh is different from API Gateway?	175
138.	Microservice 134: Explain service mesh architecture?	176
139.	Scenario 1: Monolith to Microservice?	177
140.	Scenario 2: Monolith to Microservice: Billing system.....	179
141.	Scenario 3: Authentication Token Refresh - Streaming App.....	180
142.	Scenario 4: Shared data between multiple services of Database	181
143.	Scenario 5: Selection of Database	181
144.	Scenario 6: Book a Flight - Travel App	182
145.	Scenario 7: Bank Account Transfer Between Two Different Databases	182
146.	Scenario 8: Multiple instances of a Service.....	183
147.	Scenario 9: Communication between two services - 1	184
148.	Scenario 10: Communication between two services - 2	184
149.	Scenario 11: Invoke API from Frontend	184
150.	Scenario 12: Real-Time Data Requirements	185
151.	Scenario 13: API Gateway	186
152.	Scenario 14: Rate Limiting.....	186
153.	Scenario 15: API Aggregation or Composition	188
154.	Scenario 16: Traffic routing	190
155.	Scenario 17: Secure the microservices	190
156.	Scenario 18: Routing	191
157.	Scenario 19: Handling errors.....	191
158.	Scenario 20: Event Driven System	192
159.	Scenario 21: Synchronous API Processing	193
160.	Scenario 22: Real-time Event Streaming	194

161.	Scenario 23: Real-time Analytics vs Task Queue	195
162.	Scenario 24: Order Processing in E-Commerce	196
163.	Scenario 25: Microservices Communication	196
164.	Scenario 26: Payment Retry System	196
165.	Scenario 27: Real-Time Fraud Detection	197
166.	Scenario 28: Deploying 50 Microservices - Docker vs Kubernetes	197
167.	Scenario 29: Zero Downtime Deployment	197
168.	Scenario 30: Managing Dev, QA, Stage, and Production Environments	198
169.	Scenario 31: Handling Traffic Spikes	198
170.	Scenario 32: Ensuring Backward Compatibility when a breaking changes in a Microservice: 198	
171.	Scenario 33: Managing Dev, QA, Stage, and Production Environments	199
172.	Scenario 34: Choosing deployment strategy	199
173.	Scenario 35: Service Failure	199
174.	Scenario 36: Retry without causing overload	200
175.	Scenario 37: Fallback on failure	200
176.	Scenario 38: Limit the resources	201
177.	Scenario 39: Dynamic IPs in microservices	201
178.	Scenario 40: Load Balancing without an external Load Balancer	201
179.	Scenario 41: Handling service failures using service registry	202
180.	Scenario 42: Handling usage spikes	202
181.	Scenario 43: Load balancing inside microservices	202
182.	Scenario 44: Some servers are overloaded while others are idle	203
183.	Scenario 45: Rate-limiting	203
184.	Scenario 46: Debugging Failures in a Microservice Chain	203
185.	Scenario 47: Handling High Volume Logs	204
186.	Scenario 48: Detecting Problems Before Users Report Them	204
187.	Scenario 49: Debugging Slow or failing Requests	204
188.	Scenario 50: Partial Failure	204
189.	Scenario 51: Data consistency -1	206
190.	Scenario 52: Data consistency -2	206
191.	Scenario 53: Sending notifications / Broadcast an event	207

192.	Scenario 54: Scaling the microservice	208
193.	Scenario 55: Polling	208
194.	Scenario 56: Scheduler + Multiple Instances.....	209
195.	Scenario 57: API is failing	210
196.	Scenario 58: Security	210
197.	Scenario 60: Read heavy Service	211
198.	System design 1: What is system design?	211
199.	System design 2: What is Client-Server architecture?	212
200.	System design 3: Types of Hypertext protocols (HTTP vs HTTPS vs HTTP/2)?	213
201.	System design 4: What is TLS/SSL connection?.....	214
202.	System design 5: What is IP Address and DNS?	215
203.	System design 6: What will happen when you enter google.com OR How the Web works? 215	
204.	System design 7: What is Database and its types?	216
205.	System design 8: What is RDMS and when to use it?	217
206.	System design 9: What is Document Based DB and when to use it?	218
207.	System design 10: What is Columnar DB and when to use it?	219
208.	System design 11: What is key value pair database and when to use it?	220
209.	System design 12: What is data warehouse and when to use it?	221
210.	System design 13: What is CAP Theorem?	221
211.	System design 14: What is Database sharding?	222
212.	System design 15: What is partitioning in Database?	223
213.	System design 16: What are the types of partitioning methods?	223
214.	System design 17: What is Database indexing?	225
215.	System design 18: What is Master-Slave architecture in Database?.....	225
216.	System design 19: What is Event Driven System?	227
217.	System design 20: What is Message Queue?	228
218.	System design 21: What is publisher-subscriber model and how does it different from message queues?	229
219.	System design 22: What is caching and how it works in distributed systems	231
220.	System design 23: What is CDN, how they are useful?	231
221.	System design 24: How does CDN works?	232

222.	System design 25: How do you design an API, explain with example?.....	234
223.	System design 26: What is the difference between Synchronous API and Asynchronous API? 235	
224.	System design 27: What are best practices while designing the APIs?.....	235
225.	System design 28: What is polling, it types and when to use each?.....	236
226.	System design 29: What are Webhooks and how they are useful?	236
227.	System design 30: What is WebSocket and how they are useful?	237
228.	System design 31: What is the Server-Sent Event and how is it useful?.....	238
229.	System design 32: When to use Webhook, WebSocket and Server-sent events?	239
230.	System design 33: WebSocket implementation in Spring	240
231.	System design 34: Server-sent events Implementation (Optional).....	241
232.	System design 35: What is proxy server, proxy types and when to use each?	241
233.	System design 36: What are authentication mechanisms we need to consider while designing the systems.	243
234.	System design 37: What is scalability and why it is important?.....	244
235.	System design 38: What is Horizontal Scaling and Vertical Scaling.....	245
236.	System design 39: What are the advantages and disadvantages of Vertical Scaling.....	245
237.	System design 40: What are the advantages and disadvantages of Horizontal Scaling? ...	246
238.	System design 41: Latency vs Throughput.....	247
239.	System design 42: What is Extensibility and why it is important?	247
240.	System design 43: What is maintainability and why it is important?	247
241.	System design 44: What is consistent hashing?	248
242.	System design 45: Where Consistent hashing is used?.....	252
243.	System design 46: What is the problem of consistent hashing and how it solved??	253
244.	System design 47: What is Serverless architecture?	253
245.	System design 48: What is stateless and stateful architecture?	254
246.	System design 49: Common patterns in stateless architecture?	255
247.	System design 50: Common patterns in stateful architecture?	255
248.	System design 51: When to use stateless and stateful architecture?	257
249.	System design 52: What is Full text search?.....	257
250.	System design 53: What is Elastic Search and how does it different from Full text search? 258	

251.	System design 54: How elastic search stores data to make searching fast?	258
252.	System design 55: Will Full Text Search and Elasticsearch handle typos and how??	261
253.	System design 56: What are functional and non-functional requirements in HLD?.....	261
254.	System design 57: How do you ensure high availability while designing the system?	262
255.	System design 58: What are the key considerations while designing a scalable system in HLD?	264
256.	System design 58: How do you design for disaster recovery?.....	265
257.	System design 59: How do you design for data privacy and protection in HLD?	266
258.	System design 60: How do you design Caching in your application?	267
259.	System design 62: What are the key components in High Level Design?	268
260.	System design 63: How to approach a system design interview?	270
261.	HLD 0: Steps to Design a System	271
262.	HLD 1: Tiny URL Design	272
263.	HLD 2: Design Load balancer	275
264.	HLD 3: Design API Gateway	278
265.	HLD 4: Design the Rate limiter	281
266.	HLD 5: Design a search service with autocomplete / “type-ahead” feature	284
267.	HLD 6: Design ATM System	287
268.	HLD 7: Design Notification System	291
269.	HLD 8: Design Payment System	295
270.	HLD 9: Design Ticket booking system or Hotel Booking system	299
271.	HLD 10: Library Management system	304
272.	HLD 11: Recommendation system	307
273.	HLD 12: Banking System	310
274.	HLD 13: News feed service design (Facebook/Instagram/Twitter)	314
275.	HLD 14.1: Design eCommerce application like Amazon (Merchandise Flow).....	319
276.	HLD 14.2: Design eCommerce application like Amazon (Purchase Flow)	322
277.	HLD 14.3: Design eCommerce application like Amazon (Recommendations Flow)	326
278.	HLD 14.4: Design eCommerce application like Amazon (Reviews Flow).....	328
279.	HLD 15.1: Design OTT platform like Netflix / Prime / Hotstar (Video Upload)	331
280.	HLD 15.2: Design OTT platform like Netflix / Prime / Hotstar (Video streaming)	334

281.	HLD 15.3: Design OTT platform like Netflix / Prime / Hotstar (Billing and Subscription Flow)	338
282.	HLD 15.4: Design OTT platform like Netflix / Prime / Hotstar (Analytics Flow).....	341
283.	HLD 16: Design YouTube (Comments)	344
284.	HLD 17.1: Design Instagram (User Signup and Connection Flow)	347
285.	HLD 17.1: Design Instagram (Post and Feed System Design).....	351
286.	HLD 17.2: Design Instagram (Comment)	357
287.	HLD 18: Design Twitter	357
288.	HLD 19.1: Design Riding APP like Uber or Ola (Driver Perspective)	361
289.	HLD 19.2: Design Riding APP like Uber or Ola (Rider Perspective).....	366
290.	HLD 19.3: Design Riding APP like Uber or Ola (Payment flow).....	370
291.	HLD 19.3: Design Riding APP like Uber or Ola (Rating flow).....	374
292.	HLD 20: Design Google search Engine	377
293.	HLD 21: File sharing service like Google drive	382
294.	HLD 22: Chat service like Facebook Messenger or WhatsApp	388
295.	Real-World System Design Scenarios 1: If user clicks two times on pay now, how would you prevent double payments?	395
296.	Real-World System Design Scenarios 2: How Flipkart survives Big Billion Days without crashing?	396
297.	Real-World System Design Scenarios 3: How a Food Delivery System like Zomato handles millions of orders?	396
298.	Real-World System Design Scenarios 4: How a Food Delivery System like Zomato make sure order is accepted by only one rider?	397
299.	Real-World System Design Scenarios 5: If two users are booking at the same time, how BookMyShow handle this??	398
300.	Real-World System Design Scenarios 6: When two messages sent at same time in WhatsApp, how it maintains the order??	398
301.	Real-World System Design Scenarios 7: When you delete the message in group, how does it delete in every member in group?	399
302.	Real-World System Design Scenarios 8: Why WhatsApp took so much time to introduce the edit feature?.....	399
303.	Real-World System Design Scenarios 9: How WhatsApp show typing... when user is actually typing the message??	400

304. Real-World System Design Scenarios 10: How WhatsApp Delivers Messages Instantly to 2 Billion+ Users?	400
305. Real-World System Design Scenarios 11: How messages stay private even WhatsApp can't read them?	401
306. Real-World System Design Scenarios 12: How does Hotstar scale without crashing when a big match is happening like Ind vs Pak?	402
307. Real-World System Design Scenarios 13: How video streaming apps like Hotstar maintains logged in devices?	403
308. Real-World System Design Scenarios 14: How apps like Hotstar / Amazon keep us logged in without asking again?	403
309. Real-World System Design Scenarios 15: How YouTube handles millions of Uploads?	404
310. Real-World System Design Scenarios 16: How Google search bar shows instant suggestions when you are typing?	405
311. Real-World System Design Scenarios 17: How Uber updates the location without refreshing the app?	406
312. Real-World System Design Scenarios 18: How YouTube handles millions of User interactions?	406
313. Real-World System Design Scenarios 19: How Uber finds driver near to your location? ..	407
314. Real-World System Design Scenarios 20: How apps like Amazon or Instagram manages Billion of records in Database?	407
315. Real-World System Design Scenarios 21: How does Elastic search work efficiently in just milli seconds?	408
316. Solid Principles 1: What are SOLID Principles?	410
317. Solid Principles 2: What is Single Responsibility Principle?	410
318. Solid Principles 3: What is Open/Closed Principle?	412
319. Solid Principles 4: What is Liskov substitution principle ?	415
320. Solid Principles 5: What is Interface Segregation principle?	418
321. Solid Principles 6: What is Dependency Inversion pattern?	421
322. Design Patterns 1: What are design patterns?	424
323. Design Patterns 2: What are the types of design patterns?	424
324. Design Patterns 3: What are the benefits of Design patterns?	426
325. Design Patterns 4: What is singleton pattern and when to use it?	426
326. Design Patterns 5: How do you implement the singleton pattern?	428
327. Design Patterns 6: What are the different implementations of Singleton pattern?	429

328.	Design Patterns 7: How to break the Singleton pattern?	433
329.	Design Patterns 8: What is factory pattern and when to use it?	434
330.	Design Patterns 9: What is abstract factory pattern and when to use it?	439
331.	Design Patterns 10: Abstract Factory Pattern with another example?	443
332.	Design Patterns 11: What is builder pattern and when to use it?	449
333.	Design Patterns 12: What is prototype pattern and when to use it?	451
334.	Design Patterns 13: Differences between all the creational design patterns?	453
335.	Design Patterns 14: What is Adapter pattern and when to use it?	454
336.	Design Patterns 15: What is Composite pattern and when to use it?	458
337.	Design Patterns 15: What is Proxy pattern and when to use it?	461
338.	Design Patterns 16: What is Flyweight pattern and when to use it?	465
339.	Design Patterns 17: What is Facade pattern and when to use it?	471
340.	Design Patterns 18: What is Bridge pattern and when to use it?	475
341.	Design Patterns 19: What is Decorator pattern and when to use it?	479
342.	Design Patterns 20: Differences between all the structural design patterns?	483
343.	Design Patterns 21: What is Observer Pattern and when to use it?	484
344.	Design Patterns 22: What is Strategy Pattern and when to use it?	489
345.	Design Patterns 23: What is Command Pattern and when to use it?	493
346.	Design Patterns 24: What is State Pattern and when to use it?	497
347.	Design Patterns 25: What is Iterator Pattern and when to use it?	502
348.	Design Patterns 26: What is Chain of responsibility Pattern and when to use it?	506
349.	Design Patterns 27: What is Template Pattern and when to use it?	510
350.	Design Question 1	515
351.	Design Question 2	516
352.	Design Question 3	518
353.	Design Question 4	519
354.	Design Question 5	521
355.	Design Question 6	522
356.	Design Question 7	524
357.	Design Question 8	526
358.	Design Question 9	528
359.	Design Question 10	529

360.	Design Question 11	531
361.	Design Question 12	532
362.	Design Question 13	534
363.	Design Question 14	535
364.	Design Question 15	537
365.	Design Question 16	538
366.	Design Question 17	540
367.	Design Question 18	541
368.	Design Question 19	543
369.	Design Question 20	544
370.	Design Question 21	546
371.	Design Question 22	548
372.	Design Question 23	549
373.	Questions on Kafka.....	551

1. **Microservice:** Content Outline

1. Microservice Basics

- What is Microservice?
- Difference between Monolithic and Microservice
- Is Microservices an Architectural Pattern?
- Difference between Architectural Pattern and Design Pattern
- What is Architectural Pattern?
- What is a Distributed System?
- Advantages of Microservices
- Disadvantages of Microservices
- Challenges in Microservices
- When to Choose Monolithic and When to Use Microservices

2. Load Balancing

- What is Load Balancer
- Types of Load Balancers
- Load Balancing Algorithms

3. Microservice Architecture

- Explain Microservice Architecture

- Microservice Design Patterns Overview

4. Decomposition Design Patterns

- Domain Driven Design Pattern
- What is Decomposition Pattern
- What is Strangler Pattern
- Benefits of Strangler Pattern
- Challenges of Strangler Pattern
- What is Sidecar Pattern

5. Database Design Patterns

- What are Database Patterns in Microservices
- What is Distributed Transaction
- What is Data Consistency
- What is Saga Pattern
- Saga Design Pattern: Choreography and Orchestration
- Advantages of Choreography Pattern
- Disadvantages of Choreography Pattern
- Advantages of Orchestration Pattern
- Disadvantages of Orchestration Pattern
- Saga Pattern Interview Questions
- Saga Implementation using Spring.
- What is Two-Phase Commit
- Difference Between Saga and Two-Phase Commit
- When to Use Saga and When to Use Two-Phase Commit
- What is CQRS Pattern
- When to Use CQRS Pattern
- CQRS Pattern Interview Questions
- What is Event-Driven Database Pattern
- What is Eventual Consistency

6. Communication Patterns

- What is Communication Pattern
- Types of Communication in Microservices
- Synchronous Communication
- What is GraphQL
- GraphQL Technical Questions
- When to Use GraphQL
- What is gRPC
- Asynchronous Communication
- Implementing Asynchronous Communication
- When to Use Synchronous and Asynchronous Communication

7. Message Brokers & Streaming

- What is Kafka
- How Kafka Works
- Kafka Architecture
- Kafka Real-World Use Cases
- What are Message Queues
- What is Message Broker
- When to Use Message Queues
- When to Use Kafka, RabbitMQ, or Pub/Sub

8. API Gateway

- What is API Gateway
- Why Use API Gateway
- Features of API Gateway
- How API Gateway Works
- Implementation of API Gateway in Spring.
- API Gateway vs Load Balancer
- Zuul API Gateway
- Spring Cloud Gateway
- Rate Limiting in API Gateway
- Logging in API Gateway

9. Service Discovery

- What is Service Discovery
- Client-Side vs Server-Side Discovery
- Eureka Server
- Eureka Client
- How Eureka Works
- Heartbeats in Eureka
- Service Discovery Tools Comparison

10. Deployment patterns

- End to End deployment steps.
- Docket and its uses
- Virtual Machine vs Docker
- Kubernetes and its architecture
- Types of deployments

11. Logging and Monitoring

- What is Monitoring in Microservices
- Centralized Logging
- Log Correlation ID
- Distributed Tracing

- ELK Stack
- Zipkin
- Jaeger
- Sleuth
- Prometheus
- Grafana

12. Fault Tolerance and Resilience

- Why Fault Tolerance is Needed
- Retry Pattern
- Timeout Pattern
- Bulkhead Pattern
- Circuit Breaker Pattern
- Circuit Breaker States
- Rate Limiting Pattern
- Resilience4j Overview

13. Security in Microservices

- Security Challenges in Microservices
- OAuth2 in Microservices
- JWT Authentication
- Role-Based Access Control
- API Gateway Security
- Token Validation
- SSL/TLS Usage

13. Configuration Management

- Centralized Configuration
- Spring Cloud Config Server
- Config Repo with Git
- @RefreshScope Usage

14. Caching

- What is Caching
- Cache Strategies
- Local Cache vs Distributed Cache
- Redis in Microservices
- Cache Invalidation problems
- Write-Through, Write-Behind, Cache-Aside patterns

15. Spring cloud

- Use of Spring cloud
- library and its usage

- Feign client and its purpose
- Service Mesh

2. System Design: Content Outline

1. System Design Basics

- What is System Design?
- Client-Server architecture
- Http and Https
- TLS / SSL Connection
- IP Address and DNS
- What Happens when you enter google.com?

2. Database

- Type of databases
- What is data warehouse?
- Relational Database
- No SQL databases
- Database Sharding
- Indexing in the database
- Master-Slave architecture
- CAP theorem

3. Message Queues

- What is event-driven system
- What is Message queue
- What is the publisher-subscriber model

4. Caching

- Caching and its importance
- Distributed caching
- What are CDNs

5. Communication

- How to design APIs
- Sync and Async APIs
- Best practices
- Webhooks
- Websockets
- Server-Sent Events
- Proxy vs Reverse Proxy
- Polling, Long polling and short polling

6. Authentication Mechanisms

- Security: token based
- OAuth
- Sessions

- Cookies
 - TLS
7. Performance & Scalability
 - Scaling Horizontal vs Vertical
 - Latency vs Throughput
 - Extensibility
 - Maintainability
 - Scalability
 - Consistent Hashing
 - Resilient in nature
 8. Architectures
 - What is Serverless Architecture
 - Stateful vs Stateless Architecture
 9. Searching
 - Elastic Search
 - Full text Search
 10. HLD
 - Key components of HLD
 - Function and Non-functional requirements
 - High availability
 - Key considerations while designing
 - Disaster recovery
 - Data privacy
 - Approaching a system design interview
- 3. High level designs: Content Outline**
1. Tiny URL Design
 2. Load balancer design
 3. API Gateway design
 4. Rate Limiter design
 5. Autocomplete design
 6. ATM system design
 7. Notification system design
 8. Payment system design
 9. Ticket booking system
 10. Library management system.
 11. Recommendations system design.
 12. Banking system
 13. News Feed Design
 14. E-Commerce platform like Amazon/Flipkart
 15. OTT platform like Netflix/Prime
 16. YouTube Design
 17. Instagram Design

18. Twitter Design
19. Riding app like Uber / Ola
20. Google search engine Design
21. File sharing service (Google Drive)
22. Chat apps like WhatsApp Design.

4. **Design Patterns:** Content Outline

1. Solid Principles
 - Single Responsibility
 - Open/Closed
 - Liskov Substitution
 - Interface Segregation
 - Dependency Inversion
2. Creational patterns
 - Singleton pattern
 - Factory Pattern
 - Abstract Factory Pattern
 - Builder Pattern
 - Prototype Pattern
3. Structural patterns
 - Adapter pattern
 - Composite Pattern
 - Proxy Pattern
 - Flyweight Pattern
 - Facade Pattern
 - Bridge Pattern
 - Decorator Pattern
4. Behavioural patterns
 - Observer pattern
 - Strategy Pattern
 - Command Pattern
 - State Pattern
 - Iterator Pattern
 - Chain of Responsibility Pattern
 - Template Pattern

References videos for Microservices

<https://youtube.com/playlist?list=PLGRDMO4rOGcNCLJ38INfOR1ZZ-jeZLQ2z&si=4F6JEly2NnHoDJJ4>

https://youtube.com/playlist?list=PLq3uEqRnr_2EDsuxPboP9_WtVRR_TaMrF&si=sARuV3kAhI4PHoYI

Reference for Design patterns:

[27. All Creational Design Patterns | Prototype, Singleton, Factory, AbstractFactory, Builder Pattern](#)

[32. All Structural Design Patterns | Decorator, Proxy, Composite, Adapter, Bridge, Facade, FlyWeight](#)

[41. All Behavioral Design Patterns | Strategy, Observer, State, Template, Command, Visitor, Memento](#)

Reference videos for system design are there for each question**5. Microservice 1: What is Microservice?**

Think of a big application like a single large box. Everything is kept inside that one box. If something inside breaks, the whole box has to be opened and fixed. Even a small change means touching the entire thing. This is how a **monolithic** system works.

A microservice is a different way of building applications. Instead of putting everything in one big box, the system is divided into many small boxes. Each box does one clear job. These small boxes are called microservices. They work together but can run on their own.

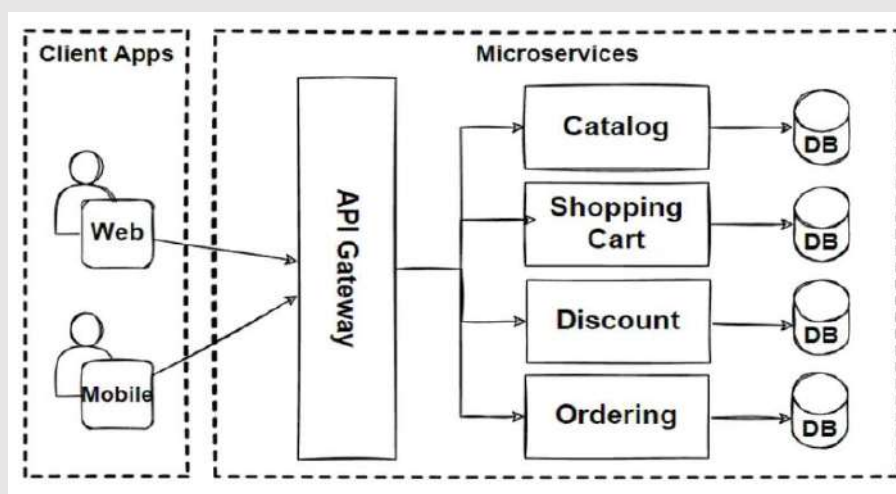
Imagine a simple food delivery app.

- One service takes care of Login and creating accounts.
- One service shows restaurants and menus.
- One service handles order.
- One service manages payments.

If the payment part has a problem, the rest of the app still works. Menus can be checked and orders can be placed. Once the payment service is ready, it joins back without disturbing others.

Definition:

A Microservice is a small, independent, and self-contained piece of software that does one specific job well. Instead of one big application, we break it into many smaller services. Each service runs on its own, can be developed and deployed separately, and communicates with others through lightweight APIs (usually HTTP or messaging queues).



6. **Microservice 2: What is the difference between Monolithic and microservice?**

Software can be built in two common ways: as one big system or as many small independent parts. These two styles are called **Monolithic architecture** and **Microservice architecture**.

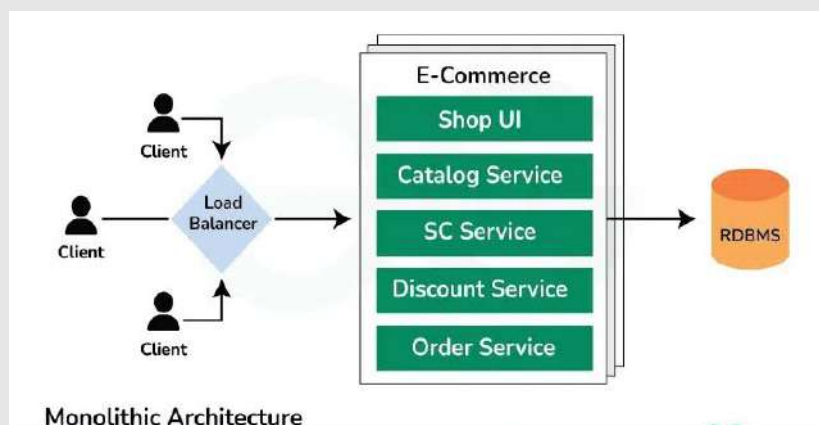
What is Monolithic?

In a monolithic application, everything lives in a single codebase. All features are developed, built, and deployed together.

For example, imagine a simple e-commerce application. It has user login, product listing, cart, payment, and order history. In a monolithic system, all these parts sit together in one project and use one database.

This is easy to build in the beginning. But as the application grows, problems start.

- A small change needs a full project build
- Deployment becomes slow and risky
- One bug can bring down the whole system
- It is hard for large teams to work together in the same codebase

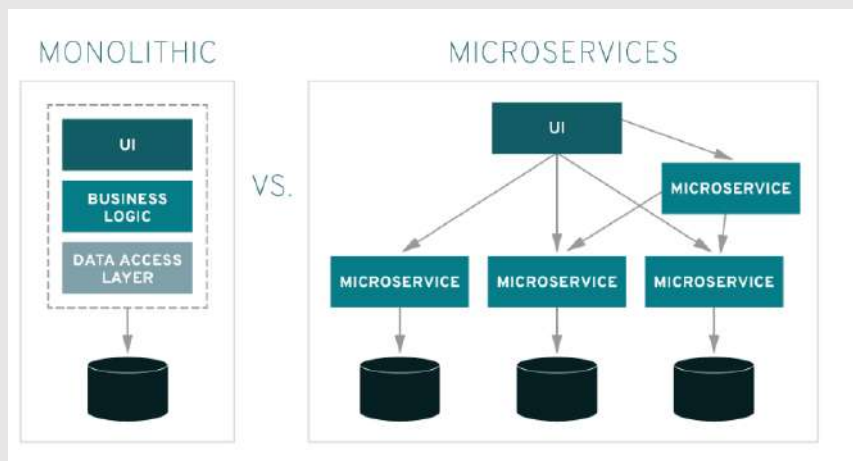


What is Microservice?

Microservice architecture divides a large system into small services. Each service handles only one business function and runs separately. For example, in the same e-commerce system:

Feature	Microservice
User login	User Service
Product listing	Product Service
Cart operations	Cart Service
Payments	Payment Service
Orders	Order Service

Each service has its own code, can use its own database, and is deployed on its own. If the payment service fails, the rest of the system still works.



Reference: [Microservices vs. monolithic architecture | Atlassian](#)

7. **Microservice 3:** Is Microservices an Architectural Pattern?

Yes, **microservices is an architectural style**. It is a way to design software by breaking a large system into small services that can run independently. Each service handles one business function, such as login, payment, product search, or order management.

Every service in microservices:

- Can be developed and deployed separately
- Can use its own database
- Can communicate with other services through APIs
- Can scale on its own based on load

8. **Microservice 4:** Difference between Architectural Pattern and Design pattern?

Architectural Pattern?

An architectural pattern defines the overall structure of a software system. It decides how different parts of the application are organized and how they communicate. It gives a big picture view.

Examples of architectural patterns:

- Layered Architecture
- Microservices Architecture
- Event-driven Architecture
- Client-Server Architecture

These patterns focus on high-level decisions. They guide how modules are arranged, how data flows, and how the system is deployed

Design Pattern?

A design pattern solves common coding problems inside a single module or class. It does not talk about the whole system. Instead, it focuses on how to write clean and reusable code.

Examples of design patterns:

- Singleton
- Factory Method
- Builder
- Strategy
- Observer

These patterns focus on class-level design and object interaction.

Simple Example

Think of constructing a building.

- The architectural pattern is like the building layout. It decides where the kitchen goes, how many floors there are, and where rooms connect.
- The design pattern is like the interior design. It decides how to arrange furniture inside a room and how switches and lights are placed.

9. **Microservice 5: What is architectural pattern?**

An architectural pattern is a way of designing the structure of a software system. It gives a clear guideline on how to organize system, how different parts of the system should interact, and how the application should grow in the future. It is not a programming language or a framework. It is simply a design approach that solves common problems in software structure.

Some common architectural patterns are:

Pattern	Idea
Layered Architecture	Code is divided into layers like controller, service, and repository
Event-Driven Architecture	System reacts to events
Microservices Architecture	System is split into small independent services
Client-Server Architecture	Frontend and backend are separated
Pipe and Filter Architecture	Data passes through processing steps

10. **Microservice 6: What is a Distributed System?**

A distributed system is a group of many computers that work together like a single system. These computers are often placed in different locations, but they talk to each other through a network to finish one common task.

Instead of putting all the work on one machine, we divide the work into smaller parts and give it to many machines. Each machine does its own job and then shares the result with others. This makes the system faster and more reliable.

Let's take a simple example. Think about an online shopping website. When we place an order, different things happen at the same time:

- One service checks if the product is available.
- Another service takes care of the payment.
- Another service handles the delivery.

Each of these services can run on a different machine. But to us, it looks like one single smooth process. All these machines talk to each other behind the scenes to make it happen.

The main goal of a distributed system is to share the load, work faster, and keep the system running even if one part fails. This is how large systems like online stores, banking apps, and streaming platforms work every day



Reference: [Distributed Systems: Basics. A distributed system is a system of... | by Naveenjain | Medium](#)

11. **Microservice 7:** What are the advantages of Microservice?

When an application is made of many small services instead of one big system, it gives us many benefits. Each service has its own job, and that makes work easier to build and manage.

1. Easy to build and change

Since every service is small, it is easier to understand what it does. If something needs to be updated, only that small part is touched. There is no need to touch the full system. This saves time and reduces mistakes.

Example: In a food delivery app, if the restaurant menu service needs a small change, only that part is updated. The order and payment parts stay safe and continue to work.

2. Faster development

Different teams can work on different services at the same time. One team can improve the login part while another team adds a new payment method. This makes development move faster.

Example: One team adds Google Pay to the payment service while another team improves the search feature in the restaurant service. Both changes can happen without waiting for each other.

3. Better fault control

If one-part stops working, the full application does not stop. Other services can still run. This helps reduce downtime and keeps the system stable.

Example: If the payment service has a problem, users can still browse restaurants and add food to their cart. Once the payment is fixed, everything connects back smoothly.

4. Easy to scale

Some parts of an application may get more traffic than others. Those parts can be scaled up without touching the rest of the system.

Example: During lunch time, the order service may get more traffic. Only that part can be scaled up to handle more orders, while the other services stay as they are.

5. Simple maintenance

Since each service is smaller and separate, finding and fixing bugs is easier. Testing becomes simple and updates become faster.

Example: If a bug is found in the restaurant listing service, it can be fixed and deployed quickly without affecting login, orders, or payment.

6. Polyglot

This is one of the significant aspects of microservices architecture. For *example*, if a team is working on one of the microservice using Java, Spring Boot, and MySQL, another team can work on another microservice using Python, Node JS, and NoSQL.

12. Microservice 8: What are the disadvantages or challenges of Microservice?

Microservices give many benefits, but they also bring a few challenges. When an application is broken into many small parts, managing everything can become more complex. Understanding these drawbacks helps us plan better.

1. Hard to manage many services

A single application can have dozens of microservices. Each one needs to be monitored, updated, and deployed carefully. Keeping track of all services can take extra time and effort.

Example: In a food delivery app, if there are ten services, like login, restaurant menu, order, payment, delivery, and more, each one needs to be checked regularly. If one service fails, finding the problem can take longer.

2. More communication between services

Since microservices talk to each other to get work done, the number of calls between services increases. If one service is slow or fails, it can affect others.

Example: When an order is placed, the order service talks to both the restaurant service and the payment service. If the restaurant service takes too long to respond, the whole order process slows down.

3. Difficult testing

Testing one single system is simple, but testing many services together can be tricky. All services need to work properly with each other, not just alone.

Example: The payment service may work fine alone, and the order service may work fine alone. But when they are connected, new issues may appear that were not seen before.

4. More deployment steps

In a monolithic system, there is only one thing to deploy. In a microservices system, each service can have its own deployment process. Handling many deployments can be more time-consuming.

Example: If the food delivery app has 12 services, some may be updated weekly and some daily. All deployments must be tracked and managed carefully.

13. Microservice 9: What are the challenges you see while developing the microservices?

Building microservices gives us flexibility, but it also brings some real challenges in complexity, data consistency, and inter-service communication.

1. Clear service boundaries

The first challenge is deciding what each service should do. If the boundaries are not clear, services can start mixing responsibilities. This can create confusion and extra work later.

Example: In a food delivery app, if both the order service and the payment service handle discounts, it becomes unclear which one should control it. A better plan would be to keep discounts inside one clear service.

2. Communication between services

Each service needs to talk to others to get things done. If this communication is not handled well, the system can become slow or break in unexpected ways.

Example: When an order is placed, the order service talks to the payment service and the restaurant service. If these connections are not planned properly, the whole order flow can fail.

3. Handling failures

Since there are many services, there are more chances for something to fail. A single failed service should not stop the full system. Handling failures carefully is one of the biggest challenges.

Example: If the payment service is down, the order service should not crash. It should handle the error and give a clear message.

4. Data consistency

In a monolithic system, all data sits in one place. In microservices, each service may have its own data. Keeping data in sync across many services is not easy.

Example: If an order is placed and the payment succeeds but the delivery service does not update the status, the system can show wrong information to the user.

5. Testing the whole system

Testing each service alone is simple. But testing how all services work together is harder. Small changes in one service can affect other services.

Example: If a small change is made in the login service, it can affect how the order service receives user information. This needs careful testing.

6. Deployment and version control

Each service has its own version and its own deployment cycle. Making sure all versions work together is a challenge.

Example: If the payment service is updated to a new version, but the order service is still using an old version, they might not talk properly.

14. **Microservice 10:** When to choose monolithic and when to use microservice?

Deciding between a monolithic application and microservices depends on the size of the project, the team, and how the system is expected to grow.

When to Choose Monolithic

A monolithic system is simpler and easier to start with. It is best when the application is small, has fewer features, and the team is small. Since everything is in one place, building, testing, and deploying is easier.

Example: If we are building a small online bookstore with only a few features like browsing books, adding them to a cart, and making payments, a monolithic system is enough. All the code sits together, making it easier to manage at the start.

When to Choose Microservices

Microservices are better when the application is large, has many features, or is expected to grow. They are also useful when different teams work on different parts of the system. Microservices allow independent development, easy scaling, and better fault control.

Example: In a large food delivery app, there are many features like login, restaurant listing, order tracking, payments, ratings, and delivery tracking. Splitting these into microservices allows each feature to grow independently. Teams can work on payments while another team improves the restaurant search without interfering with each other.

Key Points to Decide

- If the project is small, simple, and unlikely to grow quickly, monolithic is faster and easier.
- If the project is large, complex, and expected to scale with multiple teams, microservices are better.

- Microservices are more complex to manage, so the team should be ready for monitoring, testing, and deployment challenges.

15. **Microservice 11: What is load balancer and how it plays important role in microservices?**

In a microservice system, we never rely on just one instance of a service. If only one instance is running and it receives too many requests, it may slow down or even crash. To avoid this, we run **multiple instances** of the same service.

Example:

```
order-service running at port 8081
order-service running at port 8082
order-service running at port 8083
```

Now the challenge is:

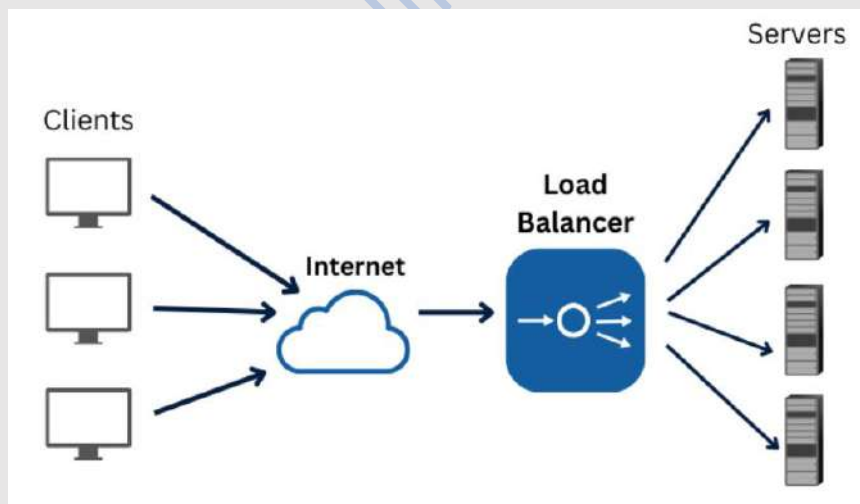
When a request comes, which instance should handle it?

If all requests go to only one instance, that instance gets overloaded while others sit idle. This is where a **Load Balancer** becomes important.

What Is a Load Balancer

A load balancer is a component that distributes incoming requests across multiple service instances. It makes sure no single instance gets overloaded and traffic is shared fairly.

Load balancer sits between the client and the microservices and plays the role of a traffic manager.



Real-Life Example

Think about a bank with three counters open. If all customers go to only one counter, it gets crowded and slow. So, a security guard at the entrance directs customers evenly to all counters.

Here,

Customers = Requests

Bank counters = Service instance

Security guard = Load Balancer

Load Balancer Tools Used in Spring Microservices

Tool	Type	Purpose
Spring Cloud LoadBalancer	Client-side	Built-in load balancer in Spring
Eureka + LoadBalancer	Client-side	Service discovery + load balancing
API Gateway (Spring Cloud Gateway)	Server-side	Routes and load balances
Nginx	Server-side	Works as reverse proxy and load balancer
Kubernetes LoadBalancer	Cluster level	Manages load balancing in containers
Ribbon (Deprecated)	Client-side	Old Netflix load balancer

Reference: [Load Balancing: System Design Fundamentals | by Abhirup Acharya | Medium](#)

Video link: [What is a LOAD BALANCER really about?](#)

16. **Microservice 12: What is are the types of load balancers?**

1. Client-Side Load Balancer

In this approach, the calling service decides which service instance to invoke. The client maintains a list of available instances, usually provided by a service registry, and distributes requests using a rule like Round Robin.

How it works

Calling Service > chooses instance ? calls target service

Correct example scenario

An Order Service calls Product Service. Product Service has multiple instances running:

- product-service:8081
- product-service:8082
- product-service:8083

The Order Service uses a client-side load balancer to select one instance for each request.

Tools used

- Spring Cloud LoadBalancer
- Feign Client
- RestTemplate with @LoadBalanced

2. Server-Side Load Balancer

Here, the client does not decide where to send the request. Instead, it sends the request to a load balancer or API Gateway, and that component forwards the request to one of the service instances.

How it works:

Client > API Gateway > service instance

Example scenario:

All requests go through: `http://api.company.com/products`

The gateway then sends them to one of the product-service instances.

Tools used: Spring Cloud Gateway, Nginx, AWS Elastic Load Balancer

17. Microservice 13: How load balancing works internally or Algorithms of Load Balancer?

Imagine a bank with three cash counters. If all customers go to counter 1 and ignore the other two, the work will be slow and messy. Instead, there must be a rule to guide each customer to a free counter. That rule is what we call a **load balancing algorithm** in software.

The same logic is applied in microservices to forward requests to different service instances.

Common Load Balancing Algorithms

Below are the most used load balancing methods in real systems.

1. Round Robin

This is the simplest and most common method.

The load balancer sends each incoming request to the service instances one by one in a circular order.

Example:

We have three instances:

```
payment-service:8081
payment-service:8082
payment-service:8083
```

Requests go like this:

```
1st request > 8081
2nd request > 8082
3rd request > 8083
4th request > 8081
5th request > 8082
```

2. Random Selection

As the name says, requests are sent to any instance randomly.

Example:

Requests may go like: `8082 > 8081 > 8083 > 8082 > 8081`

3. Least Connections

In this method, the request is assigned to the instance that has **the fewest active connections**. It is useful when each request takes a different amount of time to complete.

Example:

```
Current active requests: 8081 > 5 requests active
8082 > 2 requests active
8083 > 1 request active
Next request goes to -> 8083
```

4. Weighted Round Robin

This is a smarter version of Round Robin. If one service instance has **more power (CPU/RAM)**, we give it a **higher weight** so it can handle more traffic.

Example:

Weights:

```
8081 > weight 3
8082 > weight 1
8083 > weight 1
```

Order:

```
8081 > 8081 > 8081 > 8082 > 8083 > repeat: 8081 handles more traffic because it has higher capacity.
```

5. IP Hash

In this method, the request always goes to an instance based on **client IP address**. So, the same user always connects to the same backend instance.

Reference: [Load Balancing Algorithms](#)

[Load Balancing Algorithms Explained with Code \(and Visuals\)](#)

18. **Microservice 14:** Explain Microservice architecture?

Microservice architecture is the way we organize an application as a set of small, independent services. Each service has a clear job, manages its own data, and talks to other services when needed. This makes the system flexible, easier to maintain, and easier to scale.

Let's take a simple example to understand it better.

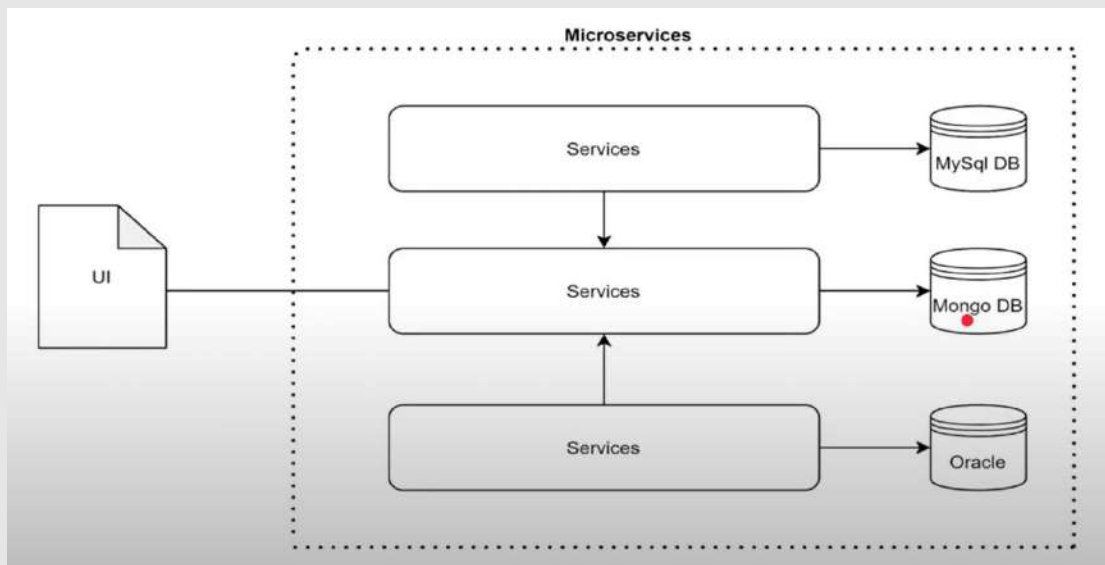
Example Scenario

Imagine an online shopping app. The **UI** is what the user sees. It could be a website or a mobile app. Behind the scenes; there are **three services**:

1. **User Service:** handles login, signup, and profile information
2. **Product Service:** manages product details, inventory, and categories
3. **Order Service:** handles placing orders, tracking them, and managing payments

Each service has its **own database**. For example:

- User Service uses **SQL** to store user details
- Product Service uses **Oracle** to store product information
- Order Service uses **MongoDB** to store orders and payment history



How the Services Work Together

When a user opens the app and places an order:

1. The **UI** sends a request to the **User Service** to verify the user.
2. Then the **Product Service** is asked to check if the product is available.
3. Once confirmed, the **Order Service** creates the order and processes the payment.

Even though the services talk to different databases and run independently, they communicate with each other through simple APIs.

Key Points

- Each service is **independent**. Changes in one service do not directly affect others.
- Each service **manages its own database**, which avoids conflicts and allows different types of storage.
- Services **talk only through defined APIs**, which keeps communication clear and simple.
- The UI can interact with multiple services at the same time to complete a single action.

19. Microservice 15: What are the design patterns of microservice.?

We can divide patterns to 7 types. Decomposition pattern, Database pattern, Communication pattern, Integration patterns, Deployment patterns, observability and cross cutting patterns.

Note: Each pattern contains sub patterns. We will see one by one from next question.

Microservices Patterns

1. Decomposition Patterns

- By Business Capability
- By Subdomain

2. Database Patterns

- Database per Service
- Shared Database

3. Communication Patterns

- Synchronous
- Asynchronous

4. Integration Patterns

- API Gateway
- Backend for Frontend (BFF)

5. Deployment Patterns

- Single Service Deployment
- Multiple Service Deployment

6. Observability Patterns

- Logging
- Monitoring
- Tracing

7. Cross-Cutting Patterns

- Security
- Configuration Management
- Error Handling

1. Decomposition Patterns

This pattern is about deciding how to split a large application into smaller services. There are two main ways:

- **By Business Capability:** Each service handles a specific business area.
Example: In a shopping app, we can have a User Service, Product Service, and Order Service. Each does one clear job.
- **By Subdomain:** Each service is built around a specific part of the system.
Example: In a movie booking system, there can be a Booking Service, Payment Service, and Review Service. Each focus on a subdomain.

2. Database Patterns

Every microservice usually has its own database. This avoids conflicts and allows each service to choose the best database type.

- **Database per Service:** Each service has its own database.
Example: User Service uses SQL, Product Service uses Oracle, and Order Service uses MongoDB.
- **Shared Database:** Sometimes services share a database, but this can create tight coupling and is less preferred.

3. Communication Patterns

Microservices need to talk to each other to get work done. There are two main ways:

- **Synchronous Communication:** Services call each other directly and wait for a response.
Example: Order Service calls Payment Service to confirm payment before completing an order.

- **Asynchronous Communication:** Services communicate using messages and do not wait for a response.
Example: When a user places an order, Order Service sends a message to Notification Service to send an email. The order does not wait for the email to be sent.

4. Integration Patterns

This pattern deals with how microservices work with other systems or external applications.

- **API Gateway:** A single-entry point for clients to access multiple services.
Example: The mobile app calls the API Gateway, and it forwards requests to User, Product, and Order Services.
- **Backend for Frontend (BFF):** A service designed specifically to serve a particular UI.
Example: The mobile app and website have separate BFF services to optimize responses for each.

5. Deployment Patterns

Deployment patterns explain how services are deployed and managed.

- **Single Service Deployment:** Each service is deployed independently.
Example: Payment Service can be updated without touching Order Service.
- **Multiple Service Deployment:** Several services are deployed together as a group.
Example: Minor updates to Product Service and Order Service are deployed at the same time.

6. Observability Patterns

Observability patterns help us monitor and understand how services are performing.

- **Logging:** Keep records of important events in each service.
- **Monitoring:** Track health and performance of services.
- **Tracing:** Track requests as they move between services.

Example: If an order fails, tracing shows whether the problem was in the Order Service or Payment Service.

7. Cross-Cutting Patterns

These patterns handle tasks that affect many services, like security, configuration, and error handling.

- **Security:** Services check authentication and authorization.
Example: User Service validates login before allowing access to Order Service.
- **Configuration Management:** Centralized settings for all services.
Example: Database URLs or API keys are managed in a single place for all services.
- **Error Handling:** Standard way to handle failures and retries.
Example: If Payment Service fails temporarily, Order Service retries the request a few times.

Reference: [Microservice Architecture and Design Patterns for Microservices | by madhuka udantha | Medium](#)

20. **Microservice 16: What is the Domain Driven Design Pattern?**

Domain Driven Design, or DDD, is a way of building systems where the focus stays on the business problem first and the code second. Instead of jumping into classes and tables, we start by understanding what the business actually does. Once the business area is clear, the code follows the same shape.

The goal is simple. Make the system easy to change and easy to understand.

Core Idea

Every business has multiple areas. For Example, In a billing system, we have invoicing, payment, pricing, tax and customer accounts. Each area has its own rules. When one area mixes with another, the code becomes confusing. DDD helps avoid this by giving each area a clean boundary.

This clean boundary is called a **bounded context**.

What is Bounded Context?

A bounded context is nothing but a clear boundary around a part of our application where one specific meaning of data and rules is followed. Inside this boundary, everyone uses the same words with the same meaning, and the logic is the same. Outside this boundary, the meaning can change.

In simple words, each bounded context is like a separate world with its own rules.

Think of a college

In a college, different departments exist:

- **Computer Science Department**
- **Mechanical Department**
- **Civil Department**

Each department uses some common words, but the meaning changes.

For example, the word "**Project**":

- In **Computer Science**, a project might be a software application.
- In **Mechanical**, a project might be a machine model.
- In **Civil**, a project might be a building design.

The same word has different meanings depending on which department we are talking about. Each department follows its own rules, subjects, teachers, and marks calculation method. That is a **bounded context**.

In Microservices

In microservices, each service usually follows one bounded context. That means each service handles only one part of the business. It has its own data, logic, and meaning for the things it deals with. It does not depend on the internal details of other services.

Example: E-commerce application

Think of an online shopping system. It has many areas. Each area can be one bounded context.

1. Order Management

This service handles orders placed by customers.

- It has its own idea of what an **Order** is.
- It knows the **Order Status**, **Payment details**, and **Shipping details**.
- These rules do not mix with other services.

2. Inventory Management

This service deals with product stock.

- It has its own meaning for **Inventory**, **Stock**, and **Warehouse**.
- It knows when items are available or when stock is low.
- It does not care how orders are created. Its focus is stock.

3. Customer Management

This service handles customer data.

- It defines what a **Customer** is.
- It stores customer profile, address, and loyalty points.
- It does not worry about stock or order rules.

Each of these services works inside its own boundary. They may talk to each other when needed, but they never mix their internal rules.

Reference: [Understanding the Bounded Context in Microservices | Bits and Pieces](#)

21. **Microservice 17:** What is Decomposition pattern or how you decompose a business into microservice?

Decomposition pattern is about **how to break a big application into small and clear microservices**. It is the first and one of the most important steps when we design a microservice architecture.

If the system is not broken in a smart way, it can create confusion, repeated work, and make the system hard to manage later. Decomposition helps us keep each service simple, focused, and easy to scale.

In Decomposition pattern, as part from business capability and subdomain, we have two another patterns

1. Strangler pattern.
2. Sidecar pattern.

22. **Microservice 18:** What is Strangler Pattern?

The **Strangler pattern** is like slowly replacing an old system with a new one. Instead of breaking the old system all at once, we move one part at a time into microservices.

Example:

Imagine we have a big monolithic food delivery app. It has login, restaurant listing, orders, payments, and delivery tracking in one single block of code.

Instead of rewriting everything, we start with one part, say Payments.

1. We build a new Payment microservice.
2. We redirect only the payment requests from the old system to the new Payment service.
3. The rest of the features keep using the old system.
4. Over time, we move other parts like Orders and Delivery the same way.

This pattern lets us modernize the system step by step without shutting it down. It reduces risk and makes the transition smooth.

It's called "Strangler" because the new microservices slowly **replace and surround** the old system until the old system is no longer needed.

Reference: [Strangler Design Pattern Tutorial with Examples for Programmer & Beginners](#)

23. Microservice 19: What are benefits of using Strangler Pattern?

The Strangler Pattern gives a safe way to **move from an old system to a new system step by step**.

Instead of replacing everything at once, we slowly build the new system around the old one and move one feature at a time. Over time, the old system gets smaller and eventually disappears.

Benefits of the Strangler Pattern

1. No big shutdown

The biggest advantage is that we do not need to stop the existing system. The business can continue normally while we build the new system piece by piece. This avoids long downtime and angry users.

2. Lower risk

Replacing a full system at once is risky. If something goes wrong, the whole system fails. With the Strangler Pattern, we move one feature at a time. If a new feature fails, only that part is affected, not the entire system. This lowers the risk of failure.

3. Works well with modernisation

It allows us to slowly modernise. For example, we can move one module from a monolithic Java application to microservices. Or we can move from REST to GraphQL, or from a SQL database to NoSQL, one step at a time.

4. Easy rollback

If something goes wrong in the new version of a feature, we can simply route traffic back to the old feature. Rollback becomes easy and safe.

5. Keeps business value running

The team can deliver improvements continuously instead of waiting for a big project release after 6–12 months. This keeps users happy and helps the business see value early.

6. Better code quality over time

Because we move feature by feature, we get time to clean code, improve design, write tests, and follow good engineering practices. This results in a healthier system overall.

Let us imagine we have an old online shopping website built as a big monolithic Java application. It has many features:

- User login
- Product search
- Cart
- Orders
- Payments

The team wants to move to microservices because the monolith is slow and hard to maintain. Instead of rewriting everything, they apply the Strangler Pattern like this:

1. First, they create a new Product Service as a separate microservice.
2. They add an API Gateway in front of the old system and route only /products calls to the new service.
3. Everything else still goes to the old monolith.
4. After product search is moved successfully, they migrate /cart next.
5. Then /orders, then /payments, and so on.
6. In the end, the monolith has no features left and can be removed.

24. Microservice 20: What are disadvantages or challenges of Strangler pattern?

- It takes time. Since we move feature by feature, full migration can take months or even years.
- Extra complexity. During migration, we maintain both old and new systems, which increases effort.
- Data syncing issues. Old and new services may use different databases, so keeping data in sync becomes difficult.

25. Microservice 21: What is Side Car Pattern?

The **Sidecar pattern** is about **attaching a helper service** next to the main service. The helper service adds extra features without changing the main service.

Example:

Let's say we have an Order Service. We want to add logging and monitoring but don't want to change its code.

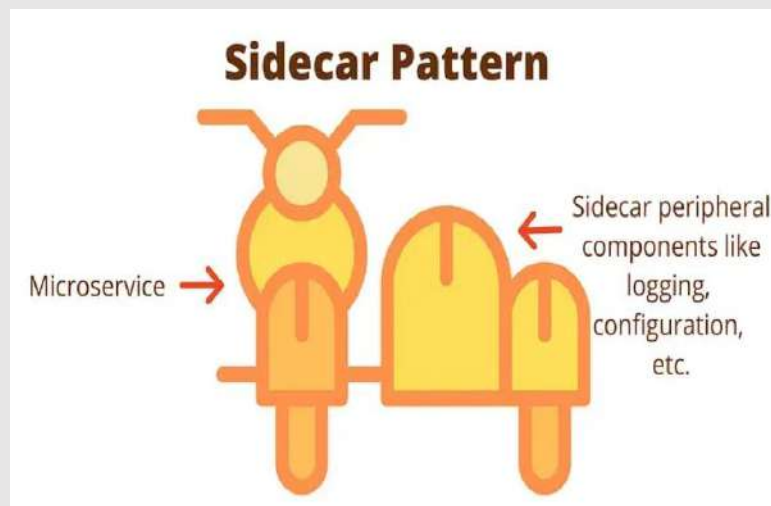
We can create a Sidecar service that sits next to the Order Service.

- The Order Service continues doing its job.
- The Sidecar service collects logs, metrics, and other useful information.
- Both run together in the same environment.

The sidecar is like a trusted assistant. It supports the main service but doesn't interfere with its logic. This makes it easy to add security, monitoring, or configuration without touching the main application.

A **sidecar service** can handle concerns such as:

- **Observability:** Centralized logging, distributed tracing (e.g., OpenTelemetry, Prometheus, Datadog).
- **Security:** Identity management, API security, authentication proxies (e.g., Istio, Envoy).
- **Resilience:** Circuit breaking, fault tolerance (e.g., Hystrix, Resilience4j).



Reference: [The Sidecar Pattern Explained: Decoupling Operational Features](#)

Let's see 2nd Design pattern, that is Database design pattern

26. **Microservice 22:** What are the Database design patterns in microservices?

In a microservice architecture, every service needs to manage its own data in a clear and reliable way. **Database pattern** is the method we follow to decide how services will use and store their data. It helps us keep the data organized, avoid unwanted mixing between services, and make the system easy to grow in the future.

Think of it like a group of shops in a mall. Each shop has its own cash counter, store room, and records. One shop doesn't depend on another shop's records to run its business. In the same way, each microservice should handle its data without depending too much on other services.

Here are the common **types of database patterns** in microservices:

1. Database per service pattern

2. Shared database pattern
3. Saga pattern
4. CQRS (Command Query Responsibility Segregation) pattern
5. Event sourcing pattern
6. Two-Phase commit patterns

Let's see each pattern from next question.

27. **Microservice 23: What is distributed transaction?**

A distributed transaction is a single business operation that touches more than one system or database at the same time. All parts of the operation must succeed together. If any part fails, everything must be rolled back to keep data correct and safe.

This type of transaction is called *distributed* because it is spread across different services or databases, may be running on different servers or networks.

Why do we need it?

In small systems with one database, a normal database transaction is enough. But in modern applications, data is stored in multiple services. When one task involves data from two or more services, we need a distributed transaction to keep everything consistent.

Working of Distributed Transactions

The working of Distributed Transactions is the same as that of simple transactions but the challenge is to implement them upon multiple databases. Due to the use of multiple nodes or database systems, there arises certain problems such as network failure, to maintain the availability of extra hardware servers and database servers.

Real Life Example of Distributed transaction

- **E-commerce Checkout:** Payment, inventory update, and order creation must all succeed or rollback together to ensure data consistency.
- **Banking Fund Transfer:** Transferring money between accounts in different banks involves debiting one account and crediting another atomically across multiple databases.
- **Flight Booking System:** Booking a flight may involve reserving a seat, processing payment, and updating frequent flyer miles across separate systems; all must succeed or fail together.

Example

Let us take an example of a bank money transfer:

A customer transfers 5,000 rupees from Account A to Account B.

Steps:

1. Debit 5,000 from Account A (Database 1)
2. Credit 5,000 to Account B (Database 2)

Both steps must succeed together. If money is deducted from Account A but not added to Account B, we lose money. That is a **data inconsistency** and must not happen.

Here, the transfer is a distributed transaction because it touches two databases.

To manage distributed transactions safely, systems use special methods like:

- **Two-Phase Commit (2PC):** ensures both databases commit or rollback together (Refer question - [Microservice 30](#))
- **Saga Pattern:** uses a series of steps with compensation logic to maintain data (Refer question - [Microservice 25](#))

Reference: [What is a Distributed Transaction? - GeeksforGeeks](#)

28. **Microservice 24:** What is data consistency and how we achieve in Microservices?

Data consistency means keeping data **correct and reliable** across the system. When one operation updates data, every part of the system must reflect the same the date. There should be no mismatch.

Example

Let us take a simple wallet system:

- Balance: 1,000 rupees
- A user sends 200 rupees to a friend

After the transfer:

- Wallet balance should become 800
- Transaction history should record the payment
- The friend should receive 200

If any one of these results is missing or wrong, the system is inconsistent. This can cause trust issues and serious business problems.

In a monolithic system, all data stays in one database, so transactions are simple. In microservices, every service has its own database. Services communicate through APIs or messages. Because of this, keeping data in sync becomes harder.

For example, in an e-commerce system:

- Order Service stores order data
- Payment Service stores payment data
- Inventory Service stores stock data

When a customer places an order, all three services must stay in sync. If payment fails, the order should be cancelled and stock must be restored. This must happen safely to keep data consistent.

How We Achieve Data Consistency in Microservices

We use patterns and strategies to maintain data correctness across services.

Method	How it helps
Saga Pattern	Keeps multiple services in sync using step-wise transactions and rollback logic
Event-Driven Communication	Services publish and listen to events like OrderCreated or PaymentFailed
Two-Phase Commit (2PC)	Ensures strong consistency across databases, often used in banking
Idempotency	Prevents duplicate updates when a request is repeated

Example Using Saga Pattern

When placing an order:

1. Order Service creates order
2. Payment Service takes payment
3. Inventory Service reduces stock

If payment fails:

- Inventory Service restores stock
- Order Service cancels order

This step-by-step handling keeps data consistent across services.

29. **Microservice 25:** What is Saga design pattern?

Saga pattern is a way to handle transactions that span across many services. In a microservice system, one business action may need changes in multiple services. Each service has its own database, so a single big transaction is not possible. Saga pattern helps us break this big action into many small local transactions.

Each local transaction updates its own data and then passes a message or event to the next service. If one of the steps fails, Saga pattern rolls back the changes by calling compensation steps. This keeps the system in a proper state even when something goes wrong.

Let's take a small example. Think about an online food delivery app. When we place an order, three services are involved:

- Order service
- Payment service
- Delivery service

First, the Order service creates the order in its database. Then it calls the Payment service to process the payment. After payment is successful, it calls the Delivery service to arrange delivery.

If payment fails, Saga pattern calls a compensation step in the Order service to cancel the order. This way, we do not keep half-done work in the system.

Saga pattern gives us a simple way to keep data correct across services without depending on one big transaction. It works step by step, and if any step fails, it knows how to undo what is already done.

Reference Links:

Video: [SAGA Design Pattern for Microservices Tutorial with Examples for Software Programmers](#)

30. Microservice 26: How do you implement saga pattern in Spring?

Saga pattern can be built in two main ways in Spring. One is using **events**, and the other is using a **central controller**. Both help us manage a business action that runs across many services.

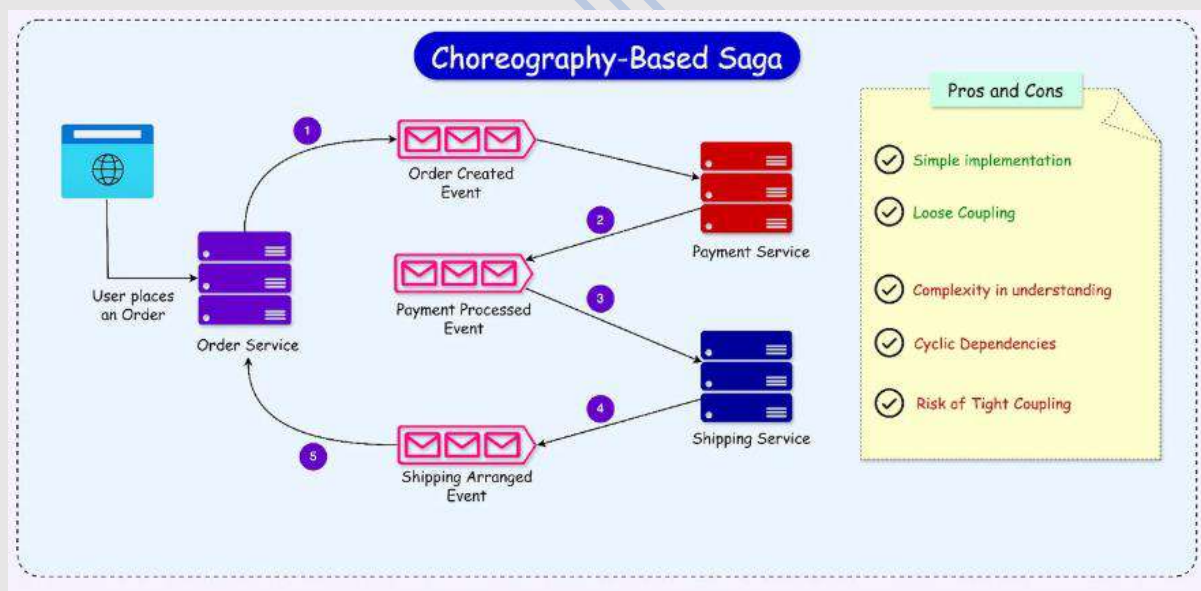
1. **Choreography Saga** - event based
2. **Orchestration Saga** - central coordinator

1. Choreography Saga

In this style, each service listens to events and reacts. There is no single service controlling everything. Let us take a small food delivery example.

1. Order service creates an order in its database and sends an OrderCreated event.
2. Payment service listens to this event. It processes the payment and sends a PaymentSuccess event.
3. Delivery service listens to this event and arranges the delivery.

If payment fails, Payment service sends a PaymentFailed event. The Order service listens to this and cancels the order.



Choreography-Based Saga Flow (Number-wise)

1. Order Created Event

The user places an order.

The Order Service saves the order in its database and publishes an **Order Created** event. It does not call any service directly. It only sends an event.

2. Payment Service reacts

The Payment Service is listening for the **Order Created** event. When it receives the event, it processes the payment and updates its own database.

3. Payment Processed Event

After payment is successful, the Payment Service publishes a **Payment Processed** event. Again, it does not call the next service directly. It only emits an event.

4. Shipping Service reacts

The Shipping Service listens for the **Payment Processed** event. Once it receives it, shipping is arranged and shipping details are saved

5. Shipping Arranged Event

The Shipping Service publishes a **Shipping Arranged** event. The Order Service listens to this event and updates the order status to shipped or completed.

Summary: In choreography-based saga, each service listens to events, performs its action, and publishes the next event, creating a chain without a central coordinator.

Reference: [Microservice Patterns: Saga Choreography | by Salitha Chathuranga | Medium](#)

Steps in Spring:

- Create events like OrderCreated, PaymentSuccess, and PaymentFailed.
- Use Spring Boot to build each service.
- Use a message broker like Kafka or RabbitMQ to send and receive events.
- Use @EventListener or message listener methods to react to events.
- Write simple compensation steps in case something fails.

This method works well when the process is small and easy to follow.

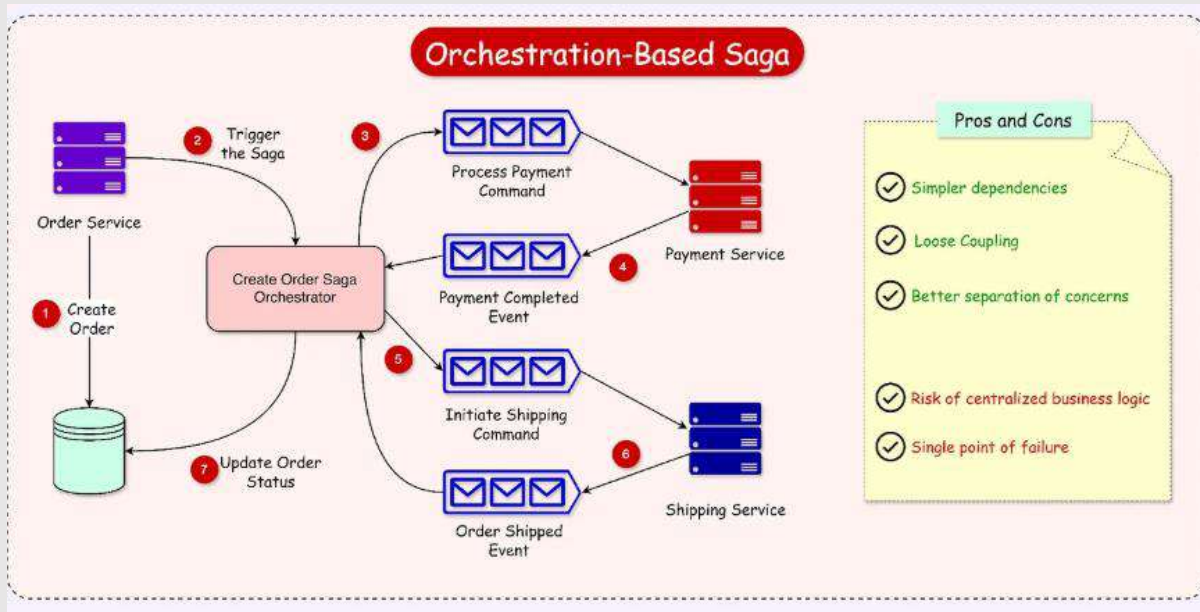
2. Orchestration Saga

In this style, one service acts like a controller. It tells each service what to do next.

Using the same food delivery example:

1. The Saga Orchestrator calls Order service to create the order.
2. Then it calls Payment service to complete the payment.
3. Then it calls Delivery service to arrange the delivery.

If payment fails, the Orchestrator calls Order service to cancel the order.



1. Create Order

The user places an order. The Order Service creates the order and saves it in the database.

2. Trigger the Saga

After creating the order, the Order Service triggers the **Order Saga Orchestrator**. From this point, the orchestrator controls the entire flow.

3. Process Payment Command

The orchestrator sends a **Process Payment** command to the Payment Service. This is a direct instruction, not just an event.

4. Payment Completed Event

The Payment Service processes the payment and sends back a **Payment Completed** event to the orchestrator.

5. Initiate Shipping Command

Once payment is confirmed, the orchestrator sends an **Initiate Shipping** command to the Shipping Service.

6. Order Shipped Event

The Shipping Service arranges delivery and sends an **Order Shipped** event back to the orchestrator.

7. Update Order Status

After receiving the shipping confirmation, the orchestrator updates the order status in the Order Service as shipped or completed.

Summary: In orchestration-based saga, a central orchestrator tells each service what to do and in what order, and updates the order status based on their responses.

Reference: [Implementing the Saga Pattern with Spring Boot and ActiveMQ in Microservice - DEV Community](#)

Steps in Spring:

- Create a separate Orchestrator service in Spring Boot.
- Call other services using REST or messaging.
- Keep track of the saga state in the Orchestrator.
- Define simple methods in each service to undo work if something fails.

This method gives more control and works better for bigger processes.

When to use each.

- Choreography fits well when the process is simple and the services are not tightly linked.
- Orchestration works better when the process has many steps and we need more control.

In real projects, we often use Spring Boot with Kafka or RabbitMQ to send messages between services.

Saga pattern helps us make sure that all steps in a big process are either fully completed or safely rolled back. This keeps the system stable and clean.

Sample for Orchestration: [Spring-Samples/SagaOrchestration at main · CodingLyf-Fullstack/Spring-Samples](#)

If we send orderId as "fail" in Request param, it will revoke back the order.

31. Microservice 27: What are advantages and disadvantages of choreography pattern?**Advantages of Choreography Pattern**

1. **Loose communication between services**
Each service talks through events, not direct calls. This removes tight connections and makes the system easier to grow and change.
2. **Good for scalability**
Since services do not wait for each other, they can handle their work in parallel. This allows the system to handle more load without slowing down.
3. **Faster development**
Teams can build and deploy services independently. As long as they follow the same event format, they can work without waiting for other teams.

Disadvantages of Choreography Pattern

1. **Difficult to track flow**
There is no central controller. When many services listen and react to events, it becomes hard to see where a business process starts and ends.
2. **Complex debugging**
When something fails in a workflow, finding the root cause is difficult because the steps are spread across many services and event messages.

32. Microservice 28: What are advantages and disadvantages of orchestration pattern?**Advantages of Orchestration Pattern**

1. Clear control and visibility

One central service control the workflow and calls other services in order. This makes it easy to understand the business flow and track what is happening step by step.

2. Simpler debugging and error handling

Since one service controls everything, it becomes easier to handle failures, retries, and logging in one place.

Disadvantages of Orchestration Pattern**1. Tighter coupling between services**

Services depend on the orchestrator to tell them what to do. If the orchestrator changes, other services may also need changes.

2. Single point of failure

If the orchestrator goes down, the whole workflow stops. We need to add high availability to reduce this risk.

33. Microservice 29: Questions on Saga pattern**1. How does the Saga pattern ensure data consistency across multiple services?**

Saga ensures consistency by breaking a long transaction into smaller steps, where each step updates its own service data. If any step fails, a compensating action is triggered to undo the previous step. This keeps data consistent without locking databases across services.

2. What happens if one step in a Saga fails?

If one step fails, the Saga pattern runs compensating transactions for all previously successful steps. For example, if payment succeeds but shipment fails, the system automatically triggers a refund. This rollback approach ensures all services return to a consistent state.

3. How do we choose between choreography-based and orchestration-based Saga?

We use choreography when services can react to events on their own, and there's no central coordinator. We use orchestration when one service (the orchestrator) controls the sequence of steps. For small, simple flows, choreography works well; for complex flows, orchestration gives better control.

4. Can Saga handle long-running transactions?

Yes, Saga is well-suited for long-running business processes like booking a hotel, flight, and car together. Each service completes its part independently. Even if one part is delayed, the overall process continues smoothly without blocking other services, making Saga ideal for such scenarios.

5. How does Saga differ from a distributed transaction?

A distributed transaction locks multiple databases until all services finish, which affects performance. Saga avoids locking by using local transactions and compensating actions. It focuses on eventual consistency rather than instant consistency, allowing services to stay available and scalable during transactions.

6. What kind of systems are not suitable for Saga?

Systems that need strict, immediate consistency are not ideal for Saga. For example, banking systems that require instant balance updates across multiple accounts may prefer a two-phase commit. Saga works best when slight delays in consistency are acceptable for better scalability.

34. Microservice 30: What is Two-Phase commit pattern?

Two-Phase Commit (2PC) is a distributed transaction protocol used to ensure that a transaction either commits (i.e., completes successfully) or rolls back (i.e., is undone) in a consistent manner across multiple systems or databases. It is commonly used in distributed systems to ensure **atomicity** and **consistency** in situations where multiple databases or services are involved in a transaction.

Phases of 2PC

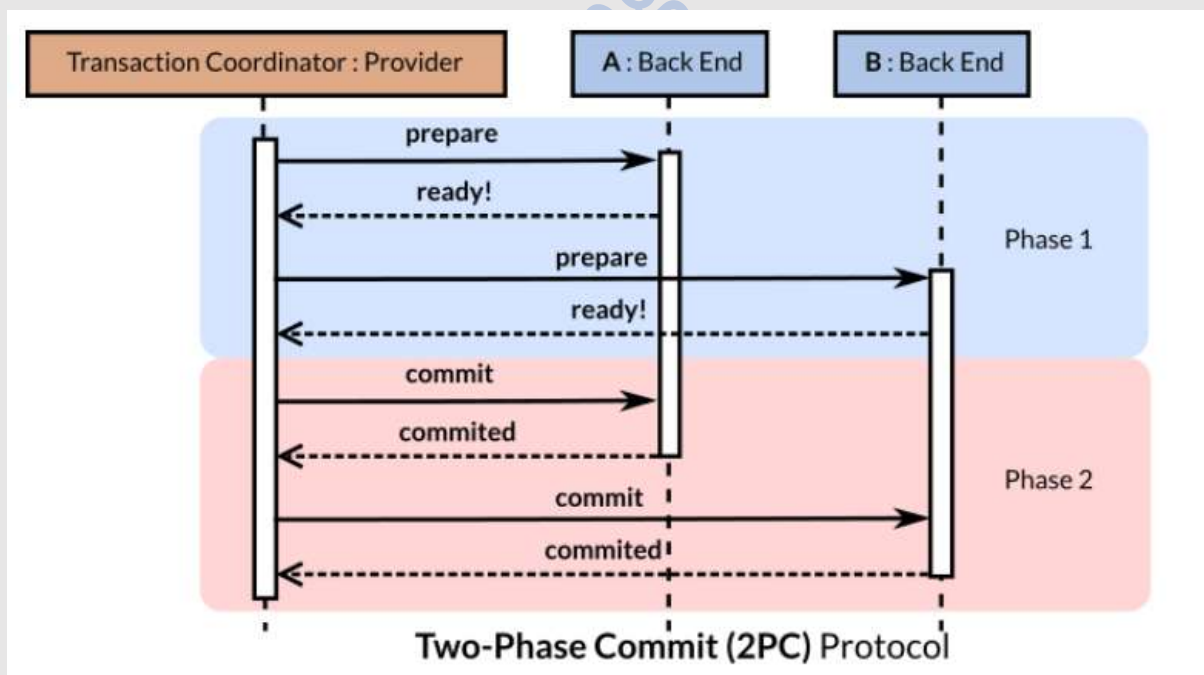
Phase 1: Prepare (Voting Phase)

The coordinator (the system initiating the transaction) sends a prepare message to all participants (the systems involved in the transaction).

Each participant checks if it can commit the transaction. If the participant is ready to commit, it responds with a YES vote. If there is any issue (e.g., resource constraints, validation failure), it responds with a NO vote.

Phase 2: Commit or Rollback (Decision Phase)

- If all participants vote YES, the coordinator sends a commit message to all participants, indicating that they should commit the transaction and finalize it.
- If any participant votes NO, the coordinator sends a rollback message to all participants, instructing them to undo any changes and revert to the previous state.



Phase 1: Prepare phase

1. A Transaction Coordinator starts the transaction.
2. It sends a prepare request to Service A.
3. Service A checks if it can commit and replies ready.

4. The coordinator sends a prepare request to Service B.
5. Service B also checks and replies ready.

At this point, no data is saved permanently. All services are only saying yes, I am ready.

Phase 2: Commit phase

6. Since all services replied ready, the coordinator sends commit to Service A.
7. Service A commits the data and replies committed.
8. The coordinator sends commit to Service B.
9. Service B commits the data and replies committed.

Now the transaction is complete.

Important points to remember

- If any service says no in Phase 1, the coordinator sends rollback to all.
- Services lock resources during the prepare phase.
- This guarantees strong consistency, but makes the system slow.
- If the coordinator fails, services can stay locked and wait.

Real-Time Use Case

Scenario: E-commerce Order Placement

In a real-world scenario like an e-commerce platform, when a user places an order, multiple systems or services need to work together:

- **Inventory System:** Decreases the stock of the ordered items.
- **Payment System:** Charges the customer's payment method.
- **Shipping System:** Creates a shipment for the order.

All these systems need to be involved in the transaction, and if one of them fails (e.g., payment fails), the entire transaction must be rolled back to maintain consistency.

Steps Using Two-Phase Commit (2PC):

Prepare Phase:

- The order service (the coordinator) sends a "prepare" message to the inventory, payment, and shipping systems.
- Each of these systems checks if they can commit:
 - **Inventory** checks if there is enough stock.
 - **Payment** checks if the transaction can be processed.
 - **Shipping** checks if the address and delivery options are valid.

Decision Phase:

- If all systems respond with a YES, the order service sends a commit message to all systems to finalize the order (inventory is reduced, payment is processed, shipping is created).
- If any system responds with a NO (e.g., if payment fails), the order service sends a rollback message to all systems to undo their operations (stock is returned, payment is not charged, no shipment is created).

Reference: [Understanding Two-Phase and Three-Phase Commit Protocols: Key Differences, Use Cases, and Practical Examples | by Damini Bansal | Medium](#)

35. Microservice 31: What is difference between Saga and Two-Phase commit pattern?

Both Two-Phase Commit (2PC) and Saga are used to manage transactions in systems where multiple services or databases are involved. The goal is to keep data consistent. However, they work in very different ways.

1. How They Work**2PC (Two-Phase Commit):**

- Works like a strict referee.
- All services must agree to commit before any changes are saved.
- If one service fails, everything is rolled back immediately.
- It is **synchronous**, services wait for each other.

Saga:

- Works like a step-by-step story.
- Each service completes its own work and then informs the next service.
- If a service fails, previous steps are undone using **compensation actions**.
- It is usually **asynchronous**; services don't block each other.

Example

Imagine an online shopping system with **Order Service** and **Payment Service**.

2PC:

1. Coordinator asks both services if they can commit.
2. Payment Service and Order Service say yes.
3. Coordinator commits both.
4. If Payment Service cannot commit, Order Service also rolls back.

Saga:

1. Order Service creates an order.
2. Payment Service processes the payment.
3. If payment fails, Order Service cancels the order using a compensation action.
4. Services do not wait for a central coordinator all the time.

36. **Microservice 32: When to use Saga and Two-Phase Commit?**

Choosing between Saga and Two-Phase Commit (2PC) depends on the size of the system, how services interact, and how fast we want the system to work. Both solve the problem of keeping data consistent across multiple services, but in different ways.

When to Use Two-Phase Commit (2PC)

2PC is best when strong consistency is very important and the system is small or medium-sized. It works well when the number of services involved in a transaction is limited.

Tiny example:

In a banking system, when transferring money from one account to another:

- Account A must decrease the balance.
 - Account B must increase the balance.
- Both steps must happen together. If one fails, the other must not happen.
2PC ensures that either both succeed or both fail, keeping money safe.

Key point: Use 2PC when we cannot allow partial changes and can tolerate a bit slower performance.

When to Use Saga

Saga is better when the system is large, distributed, and built with many microservices. It allows services to work independently, improving speed and scalability.

Example:

In an online shopping app:

- Order Service creates the order.
 - Payment Service processes the payment.
 - Delivery Service schedules delivery.
- If payment fails, Saga triggers a compensation step to cancel the order.

Key point: Use Saga when we need the system to be fast, resilient, and scalable, even if the steps are handled asynchronously.

37. **Microservice 33: What is CQRS Pattern?**

Command Query Responsibility Segregation (CQRS) is a design pattern that separates the operations that read data (queries) from those that write or update data (commands)

Command Model:

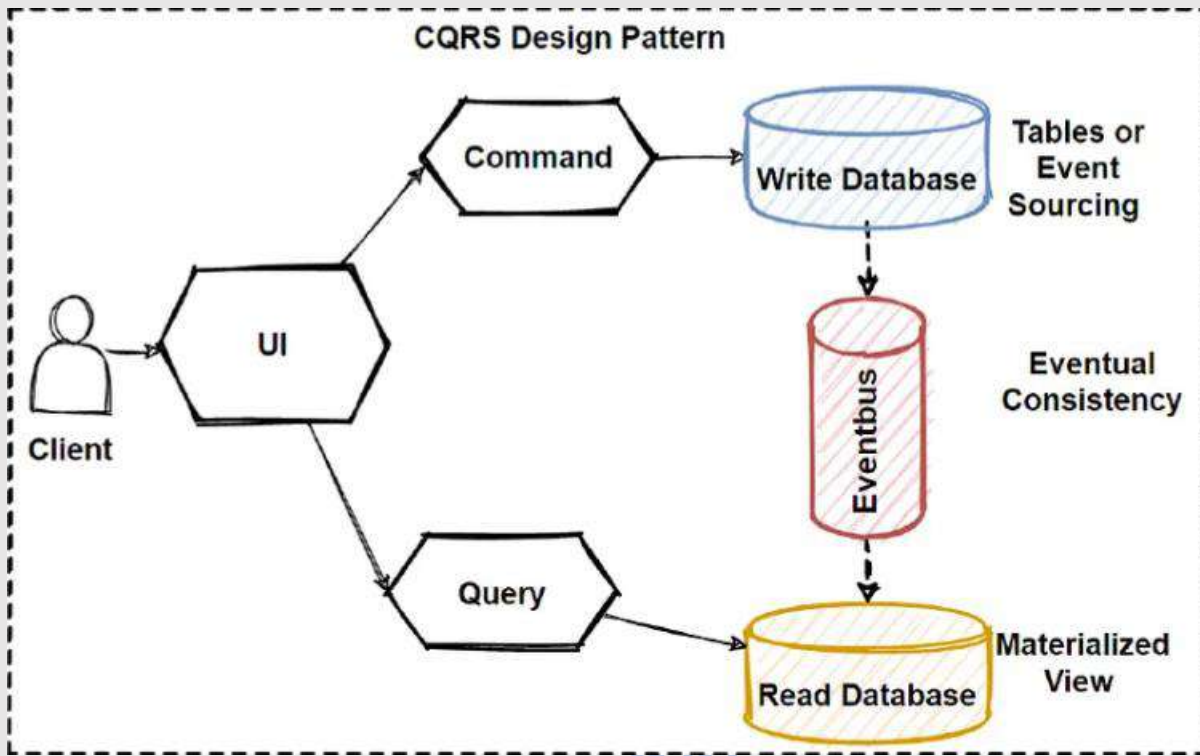
- Handles operations that write or update data (e.g., Create, Update, Delete).
- Uses normalized, transactional, relational databases like PostgreSQL or MySQL to ensure consistency.

Query Model:

- Handles operations that read data (e.g., Fetch, Retrieve).

- Uses denormalized, read-optimized stores like Elasticsearch, MongoDB, or even in-memory caches (e.g., Redis).

Typically, the Command Model and Query Model use different databases optimized for their respective tasks:



Let us understand the above diagram:

We start with the client. The client talks to the system through the UI. From the UI, two different paths are possible. One path is for changing data. The other path is for reading data. CQRS separates these two paths clearly.

When we want to change something in the system, we send a command. A command represents an intention to change the state of the application. For example, placing an order, updating a profile, or cancelling a booking are all commands. These commands go to the write side of the system.

On the write side, the command is processed with full business rules and validations. The data is then saved in the write database. This database is designed to keep data correct and consistent.

Once the data is saved, an event is published. This event is sent to an event bus. The event simply says that something has changed in the system. For example, order created or payment completed. The write side does not care who listens to this event.

The read side listens to these events. When an event arrives, the read side updates its own database. This database is built only for reading. The data is already prepared in a simple format so screens can load fast. This prepared data is often called a materialized view.

Now let us look at the reading flow. When we want to view data, such as order history or dashboard details, the UI sends a query. Queries never change data. They only ask for information. The query directly reads from the read database and returns the result quickly.

There is an important point here. The write database and read database are not updated at the same exact time. There can be a small delay between a command and when the data appears in the read side. This is called **eventual consistency**. After a short time, both sides show the same information.

To make this more real, let us take a simple example. In a food delivery app, placing an order is a command. It goes to the write database and creates the order. An order created event is published. The read side listens and updates the order history view. When we open the order history screen, the query reads from the read database and shows the order.

Example: Food Delivery Application

Command Model

Use case: A customer places a food order.

Steps:

- The customer taps "**Order Now**" after selecting food items from a restaurant.
- The application checks if the restaurant is open, items are available, and delivery partners are active in that area.
- Once validated, an **Order** is created with items, price, taxes and delivery location.
- The order is stored in the **Command Database** (often a relational DB with normalized tables like *Orders*, *OrderItems*, *Restaurants*).
- An **OrderCreatedEvent** is published to notify other services like payment, restaurant display, and delivery assignment.
- **Important point:**

A separate service (called the Query Updater or Projection Service) listens to **OrderCreatedEvent**.

It transforms this data into a read-friendly format. The transformed data is stored in MongoDB. Instead of complex normalized tables, MongoDB stores a single document:

Query Model

Use case: The customer wants to track the order status.

Steps:

- The customer opens the "**Track Order**" screen in the app.
- The system fetches details such as delivery partner name, restaurant, live order status (Preparing, Picked Up, Out for Delivery), ETA and map location.
- These details are pulled from the **Query Database** (often a document-based or search engine store like MongoDB or Elasticsearch), which holds **denormalized** order views optimized for fast reads.
- The response is returned within milliseconds since everything needed for tracking is stored in one read-friendly document.

Technologies:

- **Command Side:**
 - Relational DB: PostgreSQL or MySQL for ACID compliance.
 - Event Bus: Apache Kafka, RabbitMQ, or AWS SNS.
- **Query Side:**

- NoSQL DB: MongoDB for flexibility.
- Search Engine: Elasticsearch for full-text search capabilities.

Disadvantages of CQRS

1. Increased complexity

CQRS introduces separate write and read paths, multiple databases and event handling. This makes the system harder to design, understand and maintain, especially for small applications.

2. Eventual consistency issues

The read data is not updated immediately after a write. Users may see old data for a short time, which can be confusing if the system expects instant updates.

3. More operational effort

We need to manage events, message brokers and multiple data stores. This increases deployment, monitoring and troubleshooting effort compared to a simple CRUD system.

Reference: [CQRS Design Pattern in Microservices Architectures | by Mehmet Ozkaya | Design Microservices Architecture with Patterns & Principles | Medium](#)

38. **Microservice 34:** When to use CQRS Pattern?

CQRS stands for Command Query Responsibility Segregation. It is a design pattern where we separate the part of the system that changes data (commands) from the part that reads data (queries). This separation helps us scale, organize, and optimize the system for both reading and writing.

CQRS is useful in situations where:

Read and write workloads are different

Some applications have many reads but fewer writes, or vice versa.

Separating read and write operations helps optimize performance.

We want clear separation of responsibilities

Commands only focus on updating data.

Queries only focus on fetching data.

This makes the system easier to maintain and extend.

Example

Imagine a **social media app**:

- When a user posts a photo, the **command side** saves it in the database.
- When users browse the feed, the **query side** fetches posts quickly using a separate optimized structure.

Here, writes are handled carefully to ensure data integrity, while reads are fast and can be scaled separately.

39. **Microservice 35:** Questions on CQRS Pattern

1. Why do we separate read and write operations in CQRS?

In CQRS, we separate read and write operations because they have different needs. Writes focus on updating data safely, while reads focus on fetching it quickly. By handling them separately, we can design each side for its specific goal, improving performance and flexibility.

2. What are the main benefits of CQRS in a microservice system?

CQRS improves scalability, speed, and maintainability. It allows us to scale read and write sides independently. For example, if most traffic is for reading data, we can scale only the query side. It also simplifies complex business logic on the write side, making systems cleaner and faster.

3. How does CQRS improve performance in large-scale systems?

In large systems, most requests are read operations. CQRS improves performance by allowing the read side to use faster, optimized databases like Elasticsearch or Redis. Meanwhile, the write side can focus on accuracy and business rules. This separation reduces load and speeds up the overall system.

4. How does CQRS handle real-time updates between write and read models?

When a change happens in the write model, an event is published to notify the read model. Tools like Kafka or RabbitMQ carry these updates. The read model listens, updates its data, and stays current. This way, the system remains almost real-time and responsive.

7. What types of applications benefit most from CQRS?

CQRS fits best for systems with heavy reads, complex business rules, or large-scale data. Examples include e-commerce, banking dashboards, or social media feeds. These applications often have more reads than writes, so separating them improves speed, scalability, and overall performance.

8. How do we keep write data in sync with the read side?

We keep them in sync using event-driven updates. Every time a write happens, an event is created and sent to update the read model. Message brokers or event queues ensure delivery, so the read model eventually matches the latest data.

9. What's the difference between CQRS and a simple CRUD system?

In CRUD, the same model handles create, read, update, and delete operations. In CQRS, we separate reads and writes into different models. CRUD is simpler for small apps, while CQRS suits larger systems where performance and scalability are more important.

10. How can we test and monitor systems built using CQRS?

We test each side separately. The write side is tested for correctness and validation, while the read side is tested for accuracy and performance.

Reference Video: [CQRS Design Pattern in Microservices Interview Questions and Answers | Code Decode](#)

40. **Microservice 36:** What is Event-Driven database pattern?

The **Event-Driven Database Pattern** is a way to keep different microservices in sync using **events** instead of direct database calls. Instead of one service updating another service's database directly, a service publishes an event whenever its data changes. Other services listen to these events and update their own databases accordingly.

This helps each service stay independent, reduces tight coupling, and keeps data consistent across the system.

This approach is commonly used in **event-driven architectures** to decouple services, improve scalability, and achieve **eventual consistency**.

Example: Food Delivery System

Consider two services:

- **Restaurant Service:** manages restaurant menus and availability
- **Delivery Service:** assigns delivery partners to active orders

When a restaurant temporarily stops taking orders (maybe due to overload or closing time):

1. Restaurant Service updates its status in its own database.
2. It publishes a **RestaurantUnavailable** event.
3. Delivery Service listens to this event and removes the restaurant from its list of assignable options. It may cancel pending delivery assignments or stop assigning new ones to that restaurant.

Notice that the Delivery Service never queries the Restaurant Service database. It reacts only to events and updates its own data.

41. **Microservice 37:** What is Event Driven Architecture?

Event-Driven Architecture (EDA) is a way to design a system where services communicate by sending and reacting to events. Instead of calling each other directly, services produce events when something happens, and other services listen for those events and act accordingly.

This approach makes the system flexible, scalable, and easier to maintain because services are loosely connected.

Imagine a riding app like a taxi service:

- **Booking Service:** When a customer requests a ride, it creates a RideRequested event.
- **Driver Service:** Listens to RideRequested events and finds a nearby driver.
- **Notification Service:** Listens to the same event to send a message to the customer: "Driver is on the way."
- **Billing Service:** Listens to RideRequested and calculates the estimated fare.

Each service reacts to the event independently. The Booking Service doesn't need to know the details of drivers, notifications, or billing.

We use **Kafka**, **RabbitMQ** or **Amazon SQS** for publishing and consuming the events.

42. Microservice 38: What is Eventual Consistency?

Consistency means all databases in a distributed system see the same data at the same time. If one service changes a value, every other service should show that same new value. No one should see old or different data.

For example, if we have ₹1000 in a bank account and we spend ₹200, every place that checks the balance should now show ₹800. If one server shows ₹800 and another shows ₹1000, the data is not consistent. So, consistency makes sure the data is always correct and the same everywhere.

Data consistency is crucial for ensuring that all users and systems have access to the same, up-to-date information. It helps prevent errors, confusion, and conflicts that can arise from inconsistent data.

Strong Consistency means every service in the system always sees the latest data. When one service updates some data, that update is saved first and only after that other services are allowed to read it. So, if the Order service changes a bank balance from ₹1000 to ₹800, the Payment service, Wallet service, and any other service will all see ₹800. No service will ever see the old ₹1000. There is no delay or mismatch between services. Everyone reads the same latest value.

Eventual Consistency is a way to handle data in distributed systems where all copies of the data may not be updated at the same time, but they will become consistent eventually. Instead of forcing every service to update immediately, changes are allowed to propagate slowly. This helps systems scale and remain fast, even when many services or databases are involved.

Real-World Example

Imagine a social media app:

- A user updates their profile picture.
- The change is saved in the main database and an event is sent to other services, like the feed service and search service.
- These services might not update instantly. A few seconds later, the new picture appears in feeds and search results.

At first, some parts of the system may show the old picture, but eventually, all parts will show the same updated picture.

Eventual Consistency example

We send money from Account A to B

Transfer completes

But balance in mobile app is not updated instantly

After few seconds, app refreshes and shows correct balance

That is **eventual consistency** (system becomes correct after delay)

Key Points

- Eventual consistency is not immediate, but it guarantees that all systems will match over time.
- It is common in large distributed systems where strong consistency (immediate updates everywhere) is too slow or expensive.

- Systems that use event-driven architecture, microservices, or cloud databases often rely on eventual consistency.

Reference: [Strong Consistency vs Eventual Consistency | by Abhirup Acharya | Medium](#)

43. **Microservice 39: What is the recommended approach when two microservices using the same database?**

When two microservices need to update the same database, the safest approach is to **avoid direct shared writes**. Sharing a database sounds simple, but it causes problems like race conditions, data corruption, tight coupling, and difficulties in scaling. Instead, we follow a pattern that keeps each service independent and the data consistent.

1. Each microservice owns its own data

The cleanest design is when each service updates only the tables it owns.

If another service needs to change something, it should **call the owning service**, not touch the database directly.

Example:

- Order Service should call Inventory Service to reduce stock.
- Inventory Service should not write directly into Order tables.

This keeps the boundaries clear and avoids conflicts.

2. Use service-to-service communication

If two microservices need to change related data, one service talks to the other through an API or message queue.

Example:

When a payment is successful:

- Payment Service publishes an event: *"PaymentCompleted"*.
- Order Service listens to that event and updates its own tables.

Both services update only their own data, but the system stays in sync.

3. Use Events (best for modern systems)

Instead of sharing a database, services share **events**.

- Service A finishes work and publishes an event.
- Service B listens to the event and updates its own data.

This avoids direct database sharing and keeps everything loosely connected.

4. Implement optimistic locking:

Optimistic locking is a technique in which a version number is attached to a record in the database. When a service updates the record, it increments the version number. If another service

attempts to update the same record concurrently, it checks the version number and rejects the update if the version number has changed. This technique can prevent conflicts and ensure consistency.

In Spring, we achieve this using JPA's `@Version` annotation. We add a version field in the entity class, usually a number. Every time the record is updated, Spring increments this version automatically. When two updates happen at the same time, Spring compares the version in the database with the version in the request. If they don't match, it means another update already happened, so Spring throws an **OptimisticLockException**.

3.Communication Patterns

44. **Microservice 40: What is communication pattern and its types?**

Communication pattern is the way microservices **talk to each other** to share information or perform a task. Since each service is independent, they need clear ways to communicate. Choosing the right pattern helps the system be **fast, reliable, and easy to maintain**.

Types of Communication Patterns

1. Synchronous Communication

- Services talk directly and wait for a response.
- One service call another, waits for it to finish, and then continues.
- This is simple but can slow down the system if a service is busy or down.

Example:

In a hotel booking system, the Booking Service calls the Room Service to check availability. It waits for Room Service to respond before confirming the booking.

2. Asynchronous Communication

- Services send messages and do not wait for an immediate response.
- The receiving service processes the message when it can.
- This makes the system more resilient and scalable.

Example: In a banking system, when a customer transfers money:

- The Transfer Service publishes a MoneyTransferRequested event.
- Notification Service listens to this event and sends an SMS.
- Transfer Service does not wait for the SMS to be sent.

45. **Microservice 41: How do you achieve Synchronous Communication in Microservices?**

Synchronous communication happens when one service calls another service directly and waits for a response before continuing. It is simple to understand and works well when services need immediate answers.

How can we achieve It

HTTP/REST Calls

- Services use REST APIs over HTTP to talk to each other.
- One service sends a request, waits for the response, and continues.

GraphQL

- GraphQL allows a service to provide a single endpoint where other services or clients can ask for only the data they need.
- When a service makes a GraphQL request, it waits for the response before moving forward, just like a normal synchronous call.
- This is especially useful when we need to **get information from multiple services** at once, because GraphQL can combine the data in a single request instead of making many separate REST calls.

(Refer Questions – Microservices 42-44)

gRPC (Remote Procedure Call)

- A faster alternative to REST, especially for internal service-to-service communication.
- Services call methods on other services like normal functions.

SOAP or other RPC protocols

- Used in legacy systems for strict contracts between services.

Follow-up Question:

What is REST?

REST is not a protocol or standard but rather a set of guiding principles that leverage the existing HTTP protocol to enable communication between clients and servers.

At its core, REST is built around resources. Each resource (such as a user, order, or product) is uniquely identified by a URL (Uniform Resource Locator), and clients interact with these resources using a fixed set of HTTP methods.

- **GET** → Retrieve a resource (e.g., GET /api/users/123 to fetch user data).
- **POST** → Create a new resource (e.g., POST /api/users to add a new user).
- **PUT/PATCH** → Update an existing resource (e.g., PUT /api/users/123 to update user details).
- **DELETE** → Remove a resource (e.g., DELETE /api/users/123 to delete a user).

For example, let's say a client needs information about a specific user with **ID 123**.

- The client makes a request
- The server responds with a JSON representation of the user

REST APIs typically **return data in JSON** and use **HTTP status codes** to communicate the outcome of the request:

- **200 OK** → Success

- **201 Created** → Resource successfully created
- **400 Bad Request** → Client error (e.g., missing required fields)
- **404 Not Found** → Requested resource does not exist
- **500 Internal Server Error** → Unexpected server issue

Benefits of REST

- **Simplicity and Intuitive Design:** The resource-based model aligns well with most business domains, making REST intuitive for developers.
- **Statelessness:** Each request contains all the information needed to complete it, making REST scalable across distributed systems.
- **Cacheability:** HTTP's caching mechanisms can be leveraged to improve performance.
- **Scalability:** REST APIs can be easily scaled using load balancers and CDNs.
- **Mature Ecosystem:** With nearly two decades of widespread use, REST enjoys robust tooling, documentation, and developer familiarity.

Drawbacks of REST

- **Over-fetching:** REST endpoints often return **more data than needed**, leading to inefficient network usage. For example, if a mobile app only needs a user's name and email, but the API response includes additional fields like address, phone number, and metadata, it results in **wasted bandwidth**.
- **Under-fetching:** If an API doesn't return related data, the client may need to **make multiple requests** to retrieve all required information. For example, to get user details and their posts, a client might have to make:
 1. GET /api/users/123 (fetch user)
 2. GET /api/users/123/posts (fetch user's posts)
- **Versioning issues:** When APIs evolve, maintaining backward compatibility becomes difficult. REST APIs often require **versioned URLs** (/v1/users, /v2/users), adding maintenance overhead.

For years, **REST** was the standard for building APIs. However, as applications grew more complex, REST began to show limitations—especially in scenarios where clients needed fine-grained control over the data they fetched.

To address these challenges, **Facebook introduced GraphQL**, offering a more flexible and efficient approach to data retrieval.

46. **Microservice 42: What do you know about GraphQL?**

GraphQL is a query language for APIs, developed by Facebook, and released to the public in 2015. It helps developers interact with a server in a way that's more flexible and efficient compared to traditional REST APIs.

A **single GraphQL endpoint** (/graphql) replaces multiple REST endpoints, allowing clients to structure their own queries instead of relying on predefined responses.

It was introduced because REST APIs have some limitations:

A REST API is an architectural concept for application communication. On the other hand, GraphQL is a specification, an API query language, and a set of tools. GraphQL operates over a single endpoint using HTTP.

In addition, REST development has been more focused on making new APIs. Meanwhile, GraphQL's focus has been on API performance and flexibility.

Over-fetching: Sometimes, a REST API returns more data than you need.

Example: You fetch a list of users, but each user object includes unnecessary details like their address, age, etc.

Under-fetching: REST endpoints often don't provide all the data you need in one call.

Example: You need both a user's profile and their posts, so you must make two separate requests.

Rigid structure: REST endpoints are predefined and can be inflexible. If you need different data, the API often needs to be updated.

GraphQL solves these issues by allowing clients to specify exactly what data they need and get everything in a single request.

How GraphQL fixes this

- The client asks only for required fields. Nothing extra.
- One request can bring related data.
- The backend doesn't need to create multiple endpoints for different data needs.

Example query: One request, both user name and posts.

```
query {  
  user {  
    name  
    posts {  
      title  
    }  
  }  
}
```

Use Cases for GraphQL

GraphQL shines in scenarios where flexibility, efficiency, and dynamic data fetching are required. Here are some of the most common use cases:

1. Dynamic UI Applications

- **Example:** Dashboards, content-driven websites, and admin panels.
- **Why GraphQL?** UI components can fetch only the necessary data, improving performance and reducing unnecessary API calls.

2. Mobile Applications

- **Example:** Social media apps, e-commerce platforms.
- **Why GraphQL?** Mobile devices have limited bandwidth, so fetching only required data optimizes performance.

3. Integrating Data from Multiple Sources

- **Example:** Aggregators, business intelligence tools.
- **Why GraphQL?** Can unify data from multiple REST APIs, databases, or third-party services into a single query.

4. Microservices Architecture

- **Example:** Large-scale enterprise applications.
- **Why GraphQL?** Acts as an API gateway that consolidates multiple microservices into a single schema.

Reference: [GraphQL in 2025: Pros & Cons, Public APIs, and Use Cases — Part 1](#) | by Frontend Highlights | Medium

Reference links for sample:

[Getting Started](#) | [Building a GraphQL service](#)

[Tutorial with Spring for GraphQL](#) | [GraphQL Java](#)

Code Sample: [Spring-Samples/graphqldemo/graphqldemo at main · CodingLyf-Fullstack/Spring-Samples](#)

Steps to run:

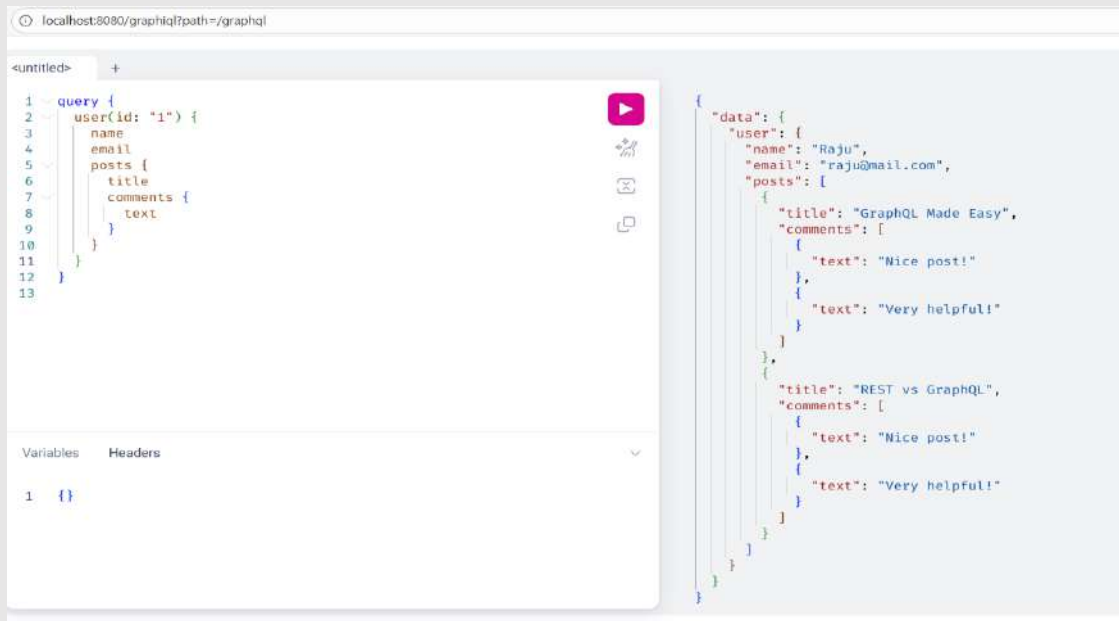
1. Run mvn clean compile
2. Run GraphqldemoApplication.java
3. Open <http://localhost:8080/graphiql?path=/graphql> in Browser
4. Copy below code and click play button
5. GraphQL returns only requested data like name, mail, title not others like ids

```
query {  
  user(id: "1") {  
    name  
    email
```

```

posts {
  title
  comments {
    text
  }
}

```

Output:**Follow-up Question:****Difference between REST and GraphQL?****REST**

We create many URLs like /users, /orders, /products to get different data.

The backend decides what data is sent in the response.

If we need user and order data, we call two APIs.

It may send extra data that we do not need.

It is simple and easy to build for most projects.

GraphQL

We use one single URL to get all types of data.

We decide what data we want to get in the request.

We can get user and order data in one single call.

It sends only the exact data we ask for.

It is better when serving multiple clients (mobile, web, IoT) with different data needs

Reference: [GraphQL vs REST API | Similarities & Differences | by David Mosyan | Medium](#)

[REST vs GraphQL - by Ashish Pratap Singh](#)**47. Microservice 43: Questions on GraphQL?****1. What is dependency used for GraphQL in Spring?**

We add the dependency spring-boot-starter-graphql in our project to enable GraphQL support. This brings everything needed to handle GraphQL requests, queries, schema files and mapping annotations. Once added, we can expose GraphQL endpoints and write queries without creating multiple REST controllers or endpoints.

Example:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-graphql</artifactId>
</dependency>
```

2. What is schema.graphqls?

The schema.graphqls file defines the structure of our GraphQL API. It tells what queries, inputs and data types exist. The client asks data only from what is defined in this schema. It works like a contract between the client and server. The file is usually placed under src/main/resources/graphql.

Example:

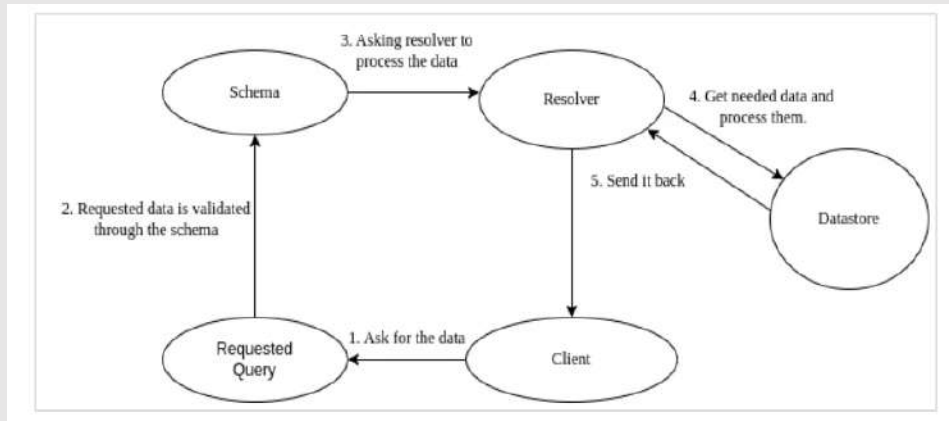
```
type Query {
  getBook(id: ID): Book
}
```

3. If a greater number of queries are there, will all be saved in same single file?

We can put all queries in one schema file, but it becomes hard to read when the application grows. Spring allows splitting schema into multiple .graphqls files. The framework automatically merges them. This helps us keep schemas based on modules like user, order and product for better organization.

4. How does GraphQL work?

- GraphQL provides a different set of types that query on that particular data by using schema and then, queries and mutations are come, and validated by using schema, and processed by using resolver.



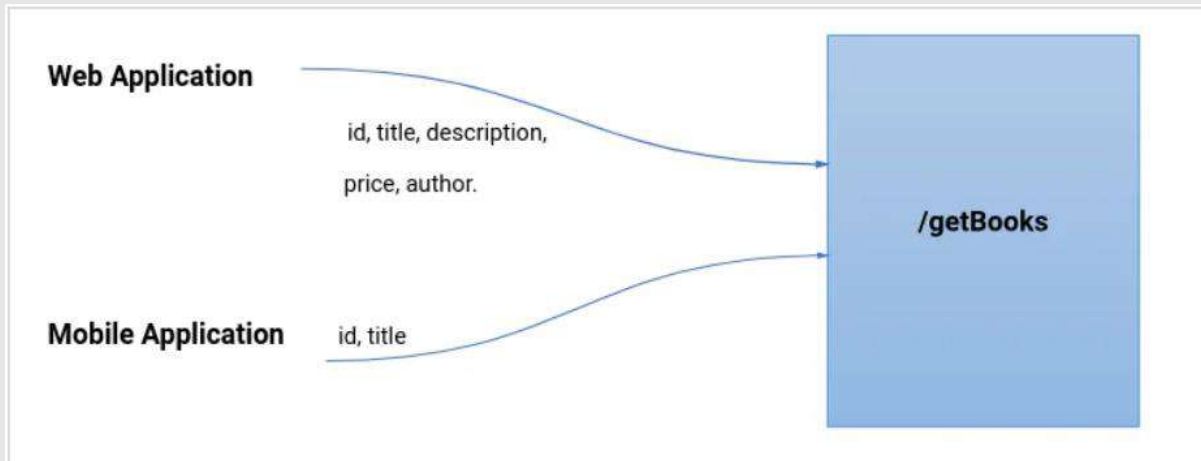
- **Schema**- Schema is how you will define your data or how you request your data and how the API will respond to that data. The schema is the concept of server and client.
- **Resolver**- Resolver is used for fetching the data, processing the data, then transferring the fetched data and processing the data by using GraphQL array. Resolver gives the result by using the callable function.

5. Why use GraphQL instead of REST?

- It gives us a single endpoint, we do not need to create the different APIs for the same data. GraphQL is strongly typed, you can also check or validate the GraphQL query before the execution.
- GraphQL helps to build powerful APIs and clients also know what they exactly want and what different types of operations they can perform.

Example-

- Suppose, we have two applications one is Web Application and the second is Mobile Application, and /getBooks is an API for fetching the data.
- Web Application can send the request for getting the book details by using the field like id, title, description, price, and author, also Mobile application wants the same data but uses only two fields id and title only.
- But we made the normal APIs, so Mobile Application wants only two fields of id and title, but by using normal APIs Mobile Application gets all the data like id, title, description, price, and author when the Mobile application hits the same endpoint /getBooks.
- Even though we use different data we made the different REST APIs for Web Application and also for Mobile Application, so this is a very costly and inefficient way. This problem is solved by GraphQL. By using GraphQL the data is passed into the form of the query by using a single endpoint.



6. Types of operations in GraphQL

1. Query

Used to **fetch data** from the server. It does not change anything. Think of it like a SELECT in SQL.

Example

```
query {  
  getBook(id: 1) {  
    title  
    author  
  }  
}
```

2. Mutation

Used to **create, update, or delete** data. It changes the state of the server. This matches INSERT, UPDATE, DELETE in SQL.

Example

```
mutation {  
  addBook(title: "Spring GraphQL") {  
    id  
    title  
  }  
}
```

3. Subscription

Used to **receive real-time updates** from the server. When data changes, the server pushes updates automatically. Works like a live feed or notification system.

Example

```
subscription {  
  bookAdded {  
    id  
    title  
  }  
}
```

48. **Microservice 44: When to use GraphQL?**

Some clients or services need only certain pieces of data. GraphQL lets them ask for exactly what they want.

Instead of calling many REST endpoints, GraphQL can fetch all needed data in a single request.

Real-World Example

Imagine a hospital management system with these services:

- Patient Service: stores patient details.
- Appointment Service: stores doctor appointments.
- Lab Service: stores test results.

A doctor's dashboard needs:

- Patient name and age
- Upcoming appointments
- Latest lab results

Without GraphQL, the dashboard might make three separate REST calls.

With GraphQL:

1. The dashboard sends a single query requesting patient info, appointments, and lab results.
2. GraphQL collects data from all three services and combines it into one response.

What this really means is GraphQL itself does not store data. It acts like a middle layer. It calls Service A, Service B, and Service C, gets data from each one, then puts everything together and sends one clean response back.

3. The response contains exactly what the dashboard needs in one call.

Use REST if:

- Your API is simple and doesn't require flexible queries.
- You need caching benefits from HTTP.
- You need a standardized, well-established API approach.
- You're integrating with third-party services.
- Your team is already familiar with REST and need faster implementation.

Use GraphQL if:

- You need flexible and efficient data fetching.
- Your API serves multiple clients (mobile, web, IoT) with different data needs.
- Real-time updates are required (GraphQL subscriptions).
- You want to avoid API versioning issues.
- Your application requires deeply nested data

Reference: [What Is GraphQL and How It Works - GraphQL Academy | Hygraph](#)

49. **Microservice 45: What is gRPC and how it is useful?**

gRPC stands for **g**oogle **R**emote **P**rocedure **C**all.

It is a communication framework developed by Google that allows services to talk to each other efficiently. Instead of sending text-based messages like REST, gRPC uses binary data with Protocol Buffers, which makes communication faster, lighter and suitable for microservices and real-time systems.

gRPC is a way for services to communicate with each other quickly and efficiently. It allows one service to call a method on another service as if it were a local function, even if the services are on different servers.

gRPC is faster than traditional REST because it uses a binary protocol instead of text, and it supports streaming data in both directions. It is especially useful for internal communication between microservices in large systems.

gRPC supports different ways for services to communicate. These patterns make it flexible for different types of tasks.

1. Unary RPC

- **What it is:** One service sends a single request and gets a single response.
- **Example:** In a smart city system, Emergency Service asks Traffic Service for the current traffic status at a specific location. Traffic Service replies with one response.

2. Server Streaming RPC

- **What it is:** One request is sent, but the server sends multiple responses over time.
- **Example:** Traffic Service sends live traffic updates to Public Display Service. The display gets updates continuously without sending new requests.

3. Client Streaming RPC

- **What it is:** The client sends multiple requests and waits for a single response from the server.
- **Example:** Sensors in a building send temperature readings every few seconds to Building Management Service. Once all readings are sent, the service calculates and responds with the average temperature.

4. Bidirectional Streaming RPC

- **What it is:** Both client and server send messages continuously.
- **Example:** In a hospital monitoring system, Patient Monitoring Service streams live vital signs to Doctor Service. Doctor Service can send alerts or instructions back in real time.

Real-World Example:

Imagine a smart city system with multiple services:

- Traffic Service: monitors traffic cameras and sensors.
- Emergency Service: responds to accidents or emergencies.
- Public Display Service: updates electronic signboards with traffic info.

Flow using gRPC:

1. Traffic Service detects a traffic jam and calls Emergency Service using gRPC to alert them.
2. Emergency Service responds immediately with the estimated response time.
3. Traffic Service streams live updates to Public Display Service to show current traffic on signboards.

Here, gRPC allows fast, real-time communication between services, even with large amounts of data like live sensor feeds.

Reference video link: [gRPC for Beginners 📺 | What, Why & How gRPC ? | Must-Know Basics! @Java Techie](#)

50. **Microservice 46: What is asynchronous communication and how it is useful?**

Asynchronous communication is a way for services to talk without waiting for a response immediately. One service sends a message or event, and the other processes it when it can. This makes systems more scalable, reliable, and efficient.

How It Works

- Service A sends a message to Service B.
- Service A continues its work without waiting for Service B to respond.
- Service B processes the message later and may trigger another event.

Real-World Example

Imagine a university notification system:

When a student finishes the admission process in the **Admission Service**, a message is sent out saying "New Student Admitted."

This message is not sent to one single service. Instead, it goes to a **message broker** (like Kafka or RabbitMQ, [Refer below questions Kafka and RabbitMQ](#)).

- The **Library Service** listens and creates a library account for the student.
- The **Hostel Service** listens and checks for room availability.
- The **ID Card Service** listens and starts preparing the ID card.

The Admission Service does not wait for any of them to finish. It just sends the message and continues admitting other students. Each service does its work when it receives the event.

This pattern makes the system more flexible. If tomorrow a new service is added, like a **Sports Club Service**, it can also listen to the same event without touching the Admission Service.

Commonly used tools for asynchronous communication in microservices:

1. Apache Kafka
2. RabbitMQ
3. Amazon SQS (Simple Queue Service)
4. Google Pub/Sub

51. **Microservice 47:** How can you implement Asynchronous communication in Microservice?

We can use a message broker to handle asynchronous messages. **Kafka** and **RabbitMQ** are two common choices.

Example using Spring with RabbitMQ.

1. Add message sender in Admission Service

```
@Service
public class AdmissionService {
    private final RabbitTemplate rabbitTemplate;

    public AdmissionService(RabbitTemplate rabbitTemplate) {
        this.rabbitTemplate = rabbitTemplate;
    }

    public void admitStudent(String studentId) {
        // business logic
        rabbitTemplate.convertAndSend("studentExchange", "student.admitted", studentId);
    }
}
```

2. Add message listener in Library Service

```
@Service
public class LibraryListener {

    @RabbitListener(queues = "studentQueue")
    public void handleNewAdmission(String studentId) {
        // create library account
        System.out.println("Library account created for: " + studentId);
    }
}
```

3. Add simple configuration

```
@Configuration
public class RabbitConfig {

    @Bean
    public TopicExchange exchange() {
        return new TopicExchange("studentExchange");
    }

    @Bean
    public Queue queue() {
        return new Queue("studentQueue");
    }

    @Bean
    public Binding binding(Queue queue, TopicExchange exchange) {
        return BindingBuilder.bind(queue).to(exchange).with("student.admitted");
    }
}
```

How this works:

Admission Service sends an event.

Library Service listens to the event and works on it when ready.

Both services are independent. If the Library Service is down, Admission Service still works.

When the Library Service comes back, it will receive the pending messages.

52. Microservice 48: When to use Asynchronous and when to use synchronous communication?**Synchronous Communication**

Synchronous means, one service sends a request to another and waits until it gets the response. Only after getting the response does it continue its work.

When to Use It

- When we need an instant response.
- When the request and response are part of one workflow.
- When real-time confirmation matters.

Example: In an online shopping portal, when we place an order, the Order Service needs an immediate confirmation from the Payment Service before marking the order as complete.

This is a synchronous call because the Order Service must wait for the Payment Service to confirm the payment before moving ahead.

Asynchronous Communication

Asynchronous means, a service sends a message and continues its work without waiting for the other service to respond immediately.

The other service processes the message later and sends the result when ready.

When to Use It

- When the task can happen later without blocking the main flow.
- When we want services to be independent and loosely connected.
- When performance and scalability are more important than real-time replies.

Example:

In the same shopping portal, after an order is confirmed, a Notification Service sends an email or SMS to the user.

The Order Service need not to wait for the Notification Service to finish sending messages. It just sends a message to a queue like Kafka or RabbitMQ and continues processing.

53. Microservice 49: How does Apache Kafka works and why it is fast?

Think of Kafka as a large, really fast post office that receives messages (data) from various sources and sends them to their respective destinations.

Let's imagine a system like Uber, but we'll focus only on how Kafka fits in, not the full product.

Real world example: Ride booking system

Think about how many events happen when a ride is booked.

- A rider books a cab.
- A driver gets the ride request.
- The price is calculated.
- A live map is updated.
- A bill is generated after the trip ends.
- A notification is sent to the rider.

If all these services talked directly to each other, it would get messy. If one service fails, the others might break too. This is where Kafka becomes powerful.

Here's how it works step by step.

Step 1: Producer sends message

When a rider books a cab, the Booking Service acts as a producer. It sends a message like *"Ride Requested"* to a Kafka topic called ride-events.

Step 2: Kafka stores the event

Kafka receives the event and keeps it in the ride-events topic. It doesn't care which service will use it. It just stores the message safely and in order.

Step 3: Multiple services read the same event

- Driver Matching Service reads the event and finds a nearby driver.
- Pricing Service reads the same event and calculates the fare.
- Notification Service reads it to send a push message to the rider.
- Tracking Service reads it to update the live location.
- Billing Service reads it after the ride ends to create the invoice.

All these services are consumers of the same Kafka topic.

Step 4: System runs smoothly even if one service is slow

Let's say the Billing Service is down for a few minutes. No problem. Kafka still keeps the messages. When Billing comes back, it reads the stored events and continues processing. The rest of the system keeps running.

Why Kafka works well here

- One event can trigger many actions.
- Services are not tightly connected.
- If one part fails, the rest of the system keeps working.
- The system can handle millions of ride events per day without crashing.

This is exactly why companies like Uber use Kafka at the center of their system. It lets the platform move fast, stay stable, and scale without losing messages.

Why is Apache Kafka so fast?

In the data streaming world, speed is often of paramount importance. Whether it's tracking user activities on a website, processing financial transactions, or monitoring IoT devices, organizations require a system that can handle a continuous flow of data with minimal latency.

Kafka excels in this regard, and its speed can be attributed to several key factors.

1. Distributed Architecture

At the heart of Kafka's speed lies its distributed architecture. Unlike traditional message queuing systems, Kafka doesn't rely on a single centralized server to handle all the data.

Instead, it distributes data across multiple nodes or brokers.

This **parallel processing capability allows Kafka to scale horizontally**, meaning it can handle an increasing volume of data by adding more machines to the cluster.

2. Partitioning

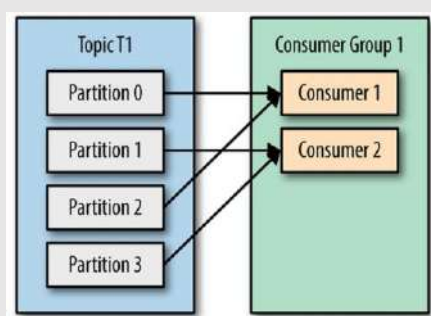
Partitioning is a core concept in Kafka that contributes significantly to its speed. **Data is divided into partitions, and each partition is assigned to a specific broker.**

This division of data allows Kafka to parallelize the processing of messages.

As messages are produced and consumed, Kafka ensures that each partition is handled by a single consumer at any given time.

The use of partitions allows Kafka to achieve both parallelism and order. Within a partition, messages are processed sequentially, ensuring that the order of events is maintained.

However, since different partitions can be processed independently by different brokers or consumers, Kafka can achieve high levels of parallelism.



Reference: [How does Apache Kafka work? Why is Kafka So fast? - DEV Community](#)

54. **Microservice 50:** Explain the Kafka architecture?

Kafka works like a smart post office that can handle thousands of messages every second. It has four main parts: Producer, Topic, Broker, and Consumer. Along with these, Zookeeper or Kafka Controller helps to keep everything in sync.

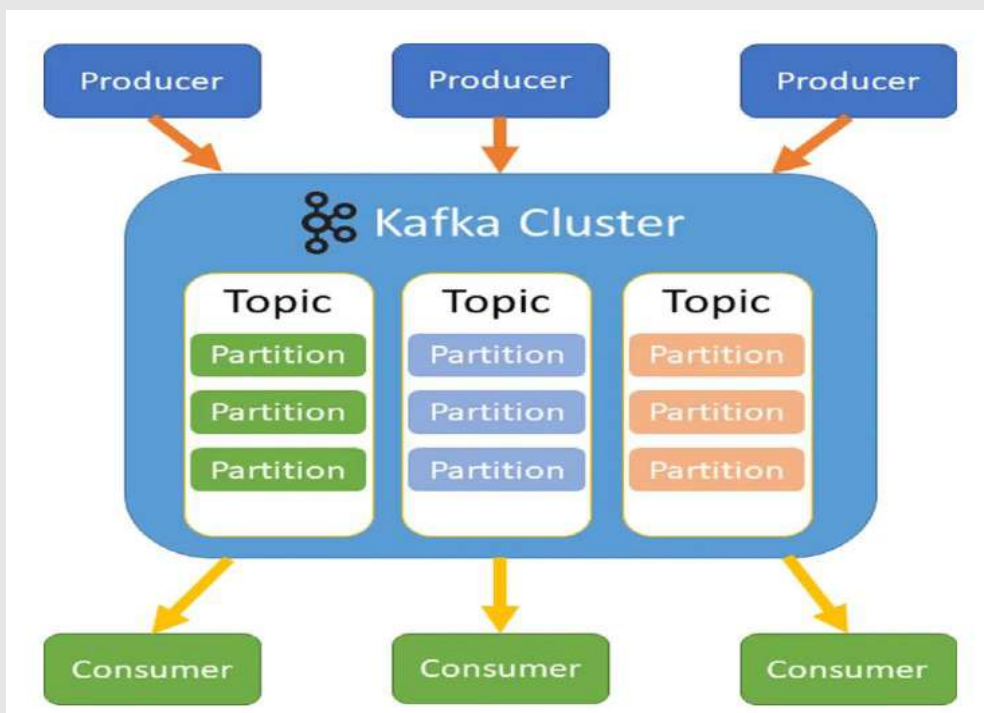
Real world example: Weather alert system

Imagine a big city with many weather sensors. Each sensor tracks temperature, humidity, air quality, and sends data every few seconds.

Many departments use this data:

- One team updates the city weather dashboard
- Another team sends alerts to mobile apps
- A third team stores the data for reports and analysis

We want to handle all this data smoothly. Kafka helps us here.



1. Producer: who sends the message

Each weather sensor acts like a **producer**. It sends messages to Kafka like "Temperature in Zone A is 33°C" or "Air Quality Alert in Zone B".

2. Topic: mailbox to store messages

Kafka has a **topic** for each type of data. For example:

- temperature-topic for temperature updates

- air-quality-topic for pollution alerts
- humidity-topic for humidity changes

Topics help to group messages so they are easy to read later.

3. Broker: the Kafka server

Kafka runs on servers called **brokers**. These brokers keep all messages safely and distribute them across multiple machines.

If one broker goes down, another broker still has the data. This keeps the system safe and reliable.

4. Partition: splitting the load

Each topic is divided into **partitions**.

Think of partitions as multiple boxes in the same mailbox. This helps Kafka process many messages at the same time without slowing down.

For example, the temperature-topic may have 3 partitions, so 3 different sensors can send data at the same time.

5. Consumer: who reads the message

Different services act as **consumers**.

- The Dashboard Service reads temperature updates to show them live on the screen.
- The Alert Service reads air quality alerts to warn citizens.
- The Analytics Service reads everything and stores it in a data warehouse.

Consumers can read from the same topic without affecting each other.

6. Zookeeper or Controller: keeps things in order

Kafka uses **Zookeeper** or its built-in controller to manage the cluster, track brokers, and balance the load.

It's like the manager who ensures everything runs smoothly.

Final flow

1. Sensors send weather data to Kafka topics (Producer).
2. Kafka brokers receive and store the data in partitions.
3. Different services read the data as per their need (Consumer).
4. If something goes wrong, Kafka still keeps the data safe.

55. **Microservice 51:** Can you tell some real-world use cases for Kafka?

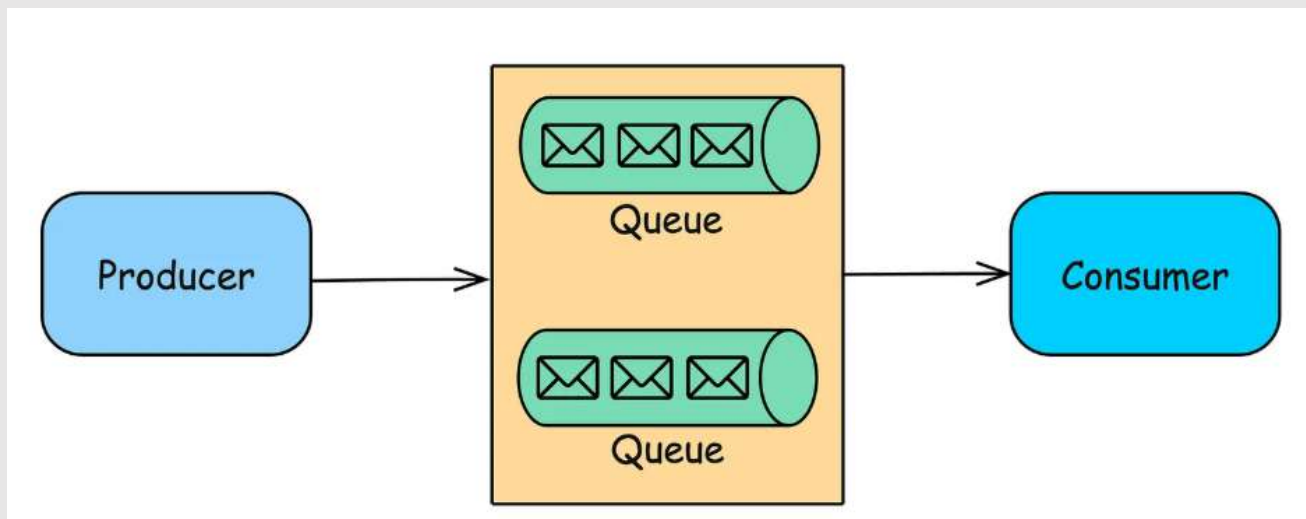
Real-World Usage of Apache Kafka

- **Netflix** uses Kafka to apply a recommendation in real-time while you're watching TV shows.
- **Uber** uses Kafka to gather taxi user and trip data in real time and they will use it to compute and forecast demand and then they can compute the infamous search pricing in real-time to know how much to charge you for a ride in case there is a lot of high demand.
- **LinkedIn** uses Kafka to prevent spam and collect user interactions to make better connection recommendations in real time.

For more questions on Kafka Refer Question No. 370

56. Microservice 52: What are message queues?

A message queue is a system that allows services to send messages to each other without waiting. Messages are stored in a queue until the receiving service is ready to process them. This makes systems more reliable and flexible, because services do not depend on each other to work at the same time.



Real world example: Call Center Support System

Imagine a call center with multiple support agents:

- Customers submit support tickets online or via phone.
- The **ticket system** receives all tickets and puts them into a queue.
- Agent 1 picks a ticket and starts resolving it.
- Agent 2 picks another ticket and works on it.
- If a new agent joins the team, they can also start picking tickets from the same queue without disrupting the system.

Each ticket waits in the queue until an agent is ready to process it. If an agent is busy or temporarily unavailable, the ticket stays safely in the queue. Nothing is lost, and all tickets are handled efficiently.

How Message Queues Work:

1. **Producer:** The service that sends the message.
 - Customers submitting tickets act as the **producer**.

2. **Queue:** The storage place where messages wait.
 - The **ticket system** is the queue. Tickets wait here until an agent is available.
3. **Consumer:** The service that reads and processes the message.
 - The **support agents** are consumers. Each agent picks a ticket from the queue and resolves it.

4. Broker/Queue Manager

The software or service that manages the message queue, handles the delivery of messages, and ensures that messages are routed correctly between producers and consumers.

4. **Acknowledgment:** The consumer tells the queue the message is processed.
 - Once an agent finishes resolving a ticket, the message is removed from the queue.

Examples

- **AWS SQS:** A managed service for message queuing.
- **RabbitMQ & Kafka:** Popular message brokers for various architectures.
- **IBM MQ:** Enterprise-grade message queuing middleware.

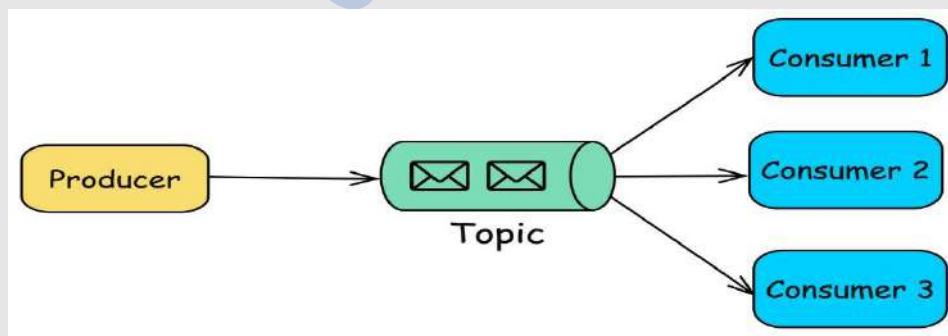
Types of Message Queues:

There are several types of message queues, each designed to solve specific problems:

1. Point-to-Point (P2P) Queue

In this model, messages are sent from one producer to one consumer. Used when a message needs to be processed by a single consumer, such as in task processing systems.

2. Publish/Subscribe (Pub/Sub) Queue



In this model, messages are published to a topic, and multiple consumers can subscribe to that topic to receive messages.

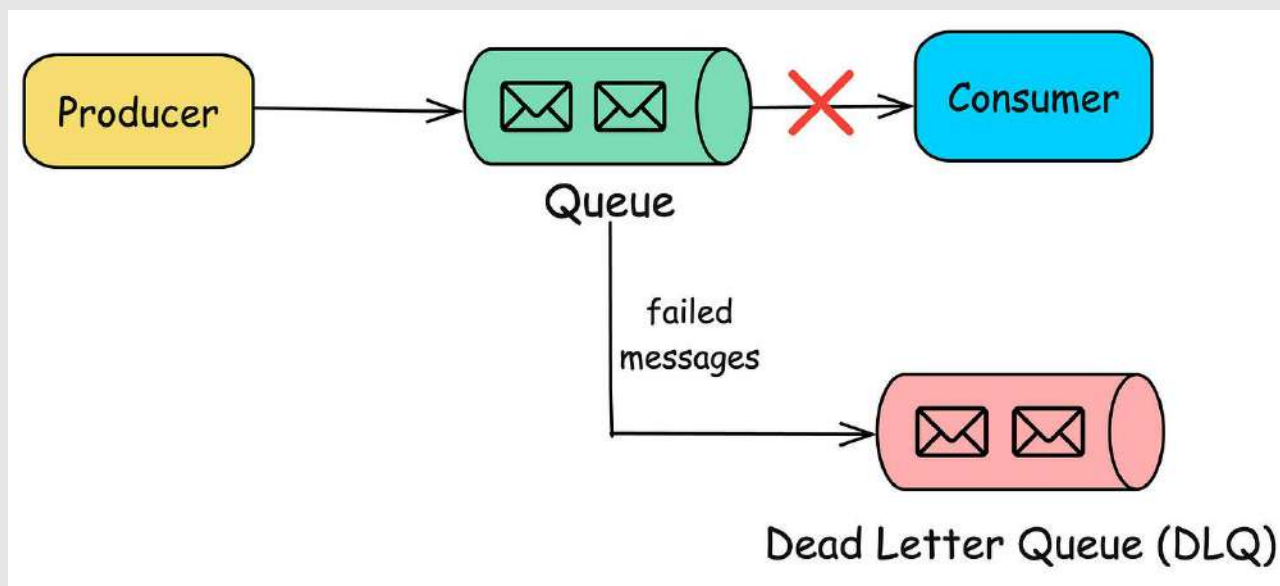
Used for broadcasting messages to multiple consumers, such as in notification systems.

3. Priority Queue

Messages in the queue are assigned priorities, and higher-priority messages are processed before lower-priority ones.

Used when certain tasks need to be handled more urgently than others.

4. Dead Letter Queue (DLQ)



A special type of queue where messages that cannot be processed (due to errors or retries) are sent. Useful for troubleshooting and handling failed messages.

57. **Microservice 53:** What is message broker?

A message broker is a software that enables applications, systems, and services to communicate with each other and exchange information.

Think of it like post office where it receives messages from one service and delivers them to other services. The broker ensures messages are delivered safely, in order, and without losing any, even if some services are busy or temporarily down.

Message brokers are very useful in microservices because they decouple services. Producers (services that send messages) don't need to know who the consumers are or when they will process the message.

Popular Message Brokers in the Market

1. **RabbitMQ:** Easy to use, reliable, works well for most queue-based tasks. *RabbitMQ is an open-source message broker that implements the Advanced Message Queuing Protocol (AMQP). It helps in decoupling services and ensures reliable message delivery.*
2. **Apache Kafka:** Handles very high volumes of messages, good for streaming and event-driven systems.
3. **ActiveMQ:** Another general-purpose messaging system, supports multiple protocols.
4. **Amazon SQS:** Cloud-based, fully managed queue service, no server setup required.
5. **Google Pub/Sub:** Cloud messaging service, scales automatically, good for event-driven systems.

Reference: [Messaging Made Easy: A List of Message Brokers and Event Streaming Platforms](#) | by Carlos Armando Marcano Vargas | Medium

58. **Microservice 54:** When to use message queues?

Message queues are useful whenever services need to communicate without waiting for each other. They help systems work smoothly even if some parts are slow or temporarily down, and make scaling easier.

1. Microservices Architecture

- **Problem:** Microservices need to communicate with each other, but direct communication can lead to tight coupling and cascading failures.
- **Solution:** Use message queues to enable asynchronous communication between microservices, allowing each service to operate independently and resiliently.

2. Task Scheduling and Background Processing

- **Problem:** Certain tasks, such as image processing or sending emails, are time-consuming and should not block the main application flow.
- **Solution:** Offload these tasks to a message queue and have background workers (consumers) process them asynchronously.

3. Event-Driven Architectures

- **Problem:** Events need to be propagated to multiple services or components, but direct communication would be inefficient.
- **Solution:** Use a Pub/Sub message queue to broadcast events to all interested consumers, ensuring that all parts of the system receive the necessary updates.

4. Load Leveling

- **Problem:** Sudden spikes in requests can overwhelm a system, leading to degraded performance or failures.
- **Solution:** Queue incoming requests using a message queue and process them at a steady rate, ensuring that the system remains stable under load.

5. Reliable Communication

- **Problem:** Communication between components needs to be reliable, even in the face of network or service failures.
- **Solution:** Use persistent message queues to ensure that messages are not lost and can be retried if delivery fails.

Real time Examples:

- **Instagram (Meta):** Used RabbitMQ in early systems to queue tasks like photo processing before scaling up to Kafka.
- **Reddit:** Uses RabbitMQ for background jobs like email delivery and notification processing.
- **Spotify:** Uses Kafka to handle streaming events, like logging song plays and updating recommendations.
- **Uber:** Uses Kafka to manage events from drivers, riders, and trip updates in real-time.

- **Airbnb:** Uses RabbitMQ to queue background tasks such as sending booking confirmations and notifications.
- **LinkedIn:** Uses Kafka to process activity feeds, notifications, and data pipelines efficiently.

59. **Microservice 55:** When to use Kafka, RabbitMQ and Pub/sub?

When to use Kafka?

Best for: Large-scale event streaming, High-throughput real-time data processing, Multiple consumers reading the same data

Real-Time Example: User Activity Tracking (E-commerce App) or Driver location updates in Uber

- Imagine Amazon wants to track user actions (searches, clicks, purchases).
- This data needs to be analysed in real-time for recommendations, fraud detection, and analytics.
- Kafka is perfect here because:
 - It acts like a log, where events are stored and processed later.
 - Multiple services (analytics, recommendation engine, etc.) can consume the same data.
 - It can handle millions of events per second efficiently.

When to use RabbitMQ?

Best for: Short-lived messages (tasks that should be processed once and deleted), Asynchronous communication (background jobs).

Real-Time Example: Food Delivery App (Swiggy/Zomato)

- A customer places an order.
- The system needs to:
 - 1 Send order details to the restaurant.
 - 2 Notify the delivery agent.
 - 3 Send an SMS confirmation to the user.
- RabbitMQ is ideal here because:
 - It queues messages and ensures they are processed once.
 - Retries failed messages (e.g., if restaurant API is down, it retries later).
 - It pushes messages to consumers instead of waiting for them to fetch data.

When to use Pub/Sub?

Best for: Cloud-based applications (Google Cloud, AWS, Azure), Real-time notifications across multiple services.

Real-Time Example: Live Sports Score Updates

- ESPN or Cricbuzz wants to send live cricket scores to millions of users.
- Each score update should be sent to multiple services like:
 - The mobile app
 - The website
 - A TV ticker service

- Pub/Sub is ideal here because:
 - It can push real-time updates to millions of subscribers.
 - It scales automatically based on load.
 - No need to manage infrastructure (Google/AWS handles it).

Reference: [Day 8: Messaging Queues: RabbitMQ vs Kafka — Understanding the Right Tool for the Job | by Dev Cookies | Medium](#)

60. **Microservice 56:** Implementations of Kafka and RabbitMQ in Spring boot?

if you are interested to do samples follow the below links.

Kafka: [Integrating Kafka in a Spring Boot Application: A Detailed Guide | Medium](#)

RabbitMQ: [Spring Boot Microservices with RabbitMQ Example | by Ramesh Fadatara | Medium](#)

[Spring Boot and RabbitMQ: Messaging with AMQP | by Igor Venturelli | CodeX | Medium](#)

Code Link: [codedecode25/Kafka](#)

61. **Microservice 57:** Difference between Kafka and RabbitMQ?

Good to know these differences to answer scenario-based questions on Kafka and RabbitMQ

Feature	Kafka	RabbitMQ
Type	Distributed event streaming platform	Traditional message broker / message queue
Use Cases	Real-time analytics, log aggregation, streaming pipelines, event-driven microservices	Task queues, background jobs, async workflows, reliable message delivery
Message Model	Publish–Subscribe with consumer groups	Queue-based (Pub/Sub + Work queues + Routing)
Message Ordering	Guaranteed per partition	Ordering not guaranteed across consumers
Delivery Guarantees	At-least-once (default), at-most-once, exactly-once (with Kafka Streams)	At-most-once, at-least-once via ack, manual retry
Replay Capability	Yes - replay old events anytime	No - messages are deleted once consumed unless manually stored
Retention	Time-based or size-based retention	Messages removed once acknowledged

Feature	Kafka	RabbitMQ
High Throughput	Designed for huge throughput (millions/sec)	Lower throughput compared to Kafka
Best For	Streaming large volumes of ordered events	Reliable task processing and message delivery
When to Use	For data pipelines, logs, streaming	For job queues, notifications, workflows

4. Integration Patterns

62. **Microservice 58:** What are the integration patterns used in Microservice?

When we build a system with multiple microservices, clients want to communicate with our system.

For example:

Imagine we are building a university management system with multiple microservices:

- Student Service: stores student details
- Course Service: manages course and subject information
- Library Service: tracks book records and issued books
- Hostel Service: manages hostel room allocations

The client here is the university web portal used by staff and students.

If the client talks to each service directly:

1. The portal calls Student Service to get student personal details.
2. The portal calls Course Service to get the list of subjects.
3. The portal calls Library Service to get which books are issued.
4. The portal calls Hostel Service to get room details.

Every time someone logs in, the portal has to make four different calls to collect all the data.

This setup works but becomes messy when the number of services grows.

The client needs to know every service's location, handle errors from each, and combine data on its own.

Note: Big companies like **Netflix and Uber** also started this way when their systems were small, then later moved to more centralized ways to reduce client complexity.

This is exactly why many teams introduce **API Gateway or Aggregator patterns** later.

63. Microservice 59: What is API gateway pattern and why we use it?

When we build a system with many microservices, it's hard for the client to talk to each one. The client needs to know all service locations, handle multiple calls, and combine the data. This is where the API Gateway pattern helps.

An **API Gateway** works like the single main entrance to a building. Instead of letting everyone enter through different doors, we create one main door. Inside the building, people can go to different rooms. Similarly, the gateway takes the client's request and sends it to the right microservice.

Real world example: Online Shopping Portal

Let's say we build an online shopping portal with these microservices:

- Product Service – gives product details.
- Order Service – handles orders.
- Payment Service – manages payments.
- Delivery Service – tracks deliveries.

Without an API Gateway, the shopping website would need to talk to each of these services directly.

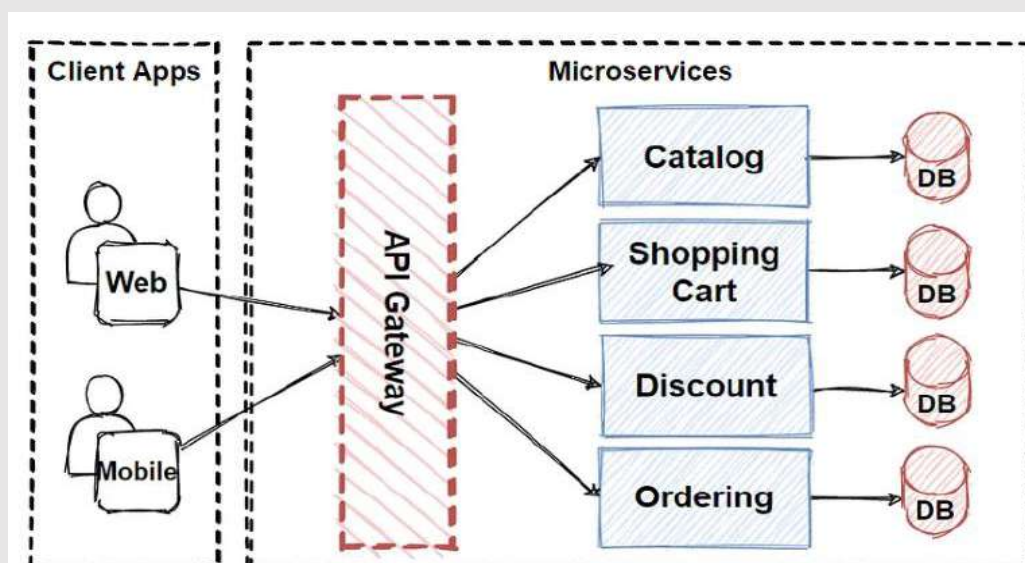
1. Product page calls Product Service.
2. Checkout page calls Order Service and Payment Service.
3. Delivery tracking page calls Delivery Service.

That means multiple calls, more complexity, and more chances of things breaking.

With an API Gateway, the website talks to just one gateway.

- The gateway receives the product page request and sends it to Product Service.
- It receives checkout requests and passes them to Order and Payment Services.
- It receives tracking requests and passes them to Delivery Service.

The client doesn't need to know where the services live or how they work. It only knows the gateway.



Reference: [API Gateway Pattern. In this article, we are going to talk... | by Mehmet Ozkaya | Design Microservices Architecture with Patterns & Principles | Medium](#)

64. **Microservice 60: What are the features of API gateway?**

An **API Gateway** is more than just a single-entry point for microservices. It provides several features that make a system easier to manage, faster, and more secure. Let's explain these features with an **online shopping portal** example.

1. Request Routing

The gateway decides which microservice should handle a client request.

Example:

- When a user opens the product page, the gateway sends the request to Product Service.
- When the user checks out, the request goes to Order Service and Payment Service.

The client doesn't need to know where the services are or how to call them.

2. Load Balancing

The gateway can distribute requests evenly among multiple instances of a microservice.

Example:

- If thousands of users open the product page at the same time, the gateway sends requests to different Product Service instances so that no single instance is overloaded.

3. Authentication and Security

The gateway can check if a request is valid and secure before letting it reach the services.

Example:

- Only logged-in users can place orders.
- The gateway verifies the user's token before forwarding the request to Order Service.

4. Response Aggregation

Sometimes the client needs data from multiple services at once. The gateway can combine responses into one.

Example:

- On the checkout page, the client wants product details, delivery options, and user balance.
- Instead of calling three services separately, the gateway collects all the data and sends one response.

5. Protocol Translation

The gateway can translate requests and responses between different formats.

Example:

- The client may call the gateway using HTTP, but internally, the gateway can talk to a service using gRPC.

6. Caching

The gateway can store frequently requested data to reduce repeated calls to services.

Example:

- Product catalog data doesn't change often. The gateway can cache it and serve many users quickly without hitting the Product Service each time.

7. Logging and Monitoring

The gateway can track all requests and help monitor system performance.

Example:

- If checkout requests fail often, the gateway logs it.
- Developers can use these logs to quickly find and fix issues.

65. Microservice 61: What are the ways to implement API gateway in Spring?

In a system with many microservices, an API Gateway acts as the single-entry point for the client. In Spring, we have simple ways to implement it. Let's see how, using an online shopping portal example.

1. Spring Cloud Gateway

This is the official Spring solution to create an API Gateway. It works with Spring Boot applications and is easy to configure.

Example:

- The online shopping portal has Product Service, Order Service, Payment Service, and Delivery Service.
- We set up Spring Cloud Gateway to route requests:
 - /products/** -> Product Service
 - /orders/** -> Order Service
 - /payments/** -> Payment Service
 - /delivery/** -> Delivery Service

Features we get:

- Routing requests to the correct service
- Load balancing using Spring Cloud LoadBalancer
- Security and authentication filters
- Logging and monitoring

This makes the client only talk to one URL instead of four separate services.

2. Zuul Gateway

Zuul is another popular API Gateway from Netflix, now integrated with Spring Cloud. It works similarly to Spring Cloud Gateway but was more widely used before Spring Cloud Gateway became standard.

Example:

- The shopping portal client calls /api/** on the Zuul Gateway.
- Zuul forwards requests to the correct microservice and can handle authentication, rate limiting, and logging.

3. Custom Spring Boot Gateway

We can also create a custom gateway using Spring Boot by writing a small service that acts as a router.

Example:

- The gateway service receives all requests at `/api/**`.
- It checks the path and forwards the request to the correct service using `RestTemplate` or `WebClient`.
- We can add security, caching, or response aggregation manually.

This approach gives full control, but it requires more effort to implement features like load balancing or monitoring.

Code Sample using spring-cloud-gateway

[Spring-Samples/APIGateway at main · CodingLyf-Fullstack/Spring-Samples](#)

Note: If you find any issues while running this sample, you can ping me or mail me.

This project contains 3 services.

- Product service
- Order service
- API gateway: we have used `spring-cloud-starter-gateway-server-webmvc` dependency for gateway

spring-cloud-starter-gateway-server-webmvc is a Spring Boot starter that enables the use of Spring Cloud Gateway with Spring WebMvc, providing a servlet-based implementation of the API Gateway. we can also use spring-cloud-starter-gateway-server-webflux for reactive programming model.

1. Start Product Service which runs on port 8081.
2. Start Order Service which runs on port 8082.
3. Start API Gateway which runs on port 8080.

Client calls through the gateway:

- GET `http://localhost:8080/products` -> returns the list of products from Product Service
- POST `http://localhost:8080/orders` with body "Laptop" -> returns "Order placed for: Laptop" from Order Service

How it works (You can test it from Postman):

Step 1: Client Sends Request

The client calls the gateway: `http://localhost:8080/products`

The client does not know about the internal services. It only talks to the Gateway.

Step 2: Gateway Receives Request

- Gateway is configured with routes in `application.yml`:

```
server:
  port: 8080 # Port where the API Gateway will run

spring:
  application:
    name: api-gateway # Name of the application
```

```

cloud:
  gateway:
    server:
      webmvc: # Using Spring MVC stack for the gateway
      routes: # List of routes the gateway will handle
        - id: product-service # Unique identifier for this route
          uri: http://localhost:8081 # The actual service URL where the request
will be forwarded
          predicates:
            - Path=/products/** # This route will handle any request that starts
with /products

        - id: order-service # Unique identifier for the second route
          uri: http://localhost:8082 # Forward requests to the Order Service
          predicates:
            - Path=/orders/** # This route handles requests starting with /orders

```

Gateway sees the path `/products/**` and it should forward the request to Product Service.

Step 3: Gateway Forwards Request to Service

- The Gateway makes an internal HTTP call to the Product Service at `http://localhost:8081/products`.
- If the path was `/orders/**`, it would forward to `http://localhost:8082/orders`.

Step 4: Service Processes Request

- **Product Service** receives the request.
- It executes the controller method:
- Service prepares the **response** (a list of products).

```

@RestController
public class ProductController {

    @GetMapping("/products")
    public List<String> getProducts() {
        return List.of("Laptop", "Phone", "Tablet");
    }
}

```

Step 5: Gateway Receives Response

- Gateway gets the response from Product Service.
- Any filters configured at the Gateway (like adding headers, rewriting paths, logging) are applied here.

Step 6: Gateway Sends Response to Client

- Gateway sends the final response back to the client.
- The client sees the response as if it came directly from the gateway, unaware of the internal microservices.

Example Response: `["Laptop", "Phone", "Tablet"]`

66. **Microservice 62: How API gateway works as load balancer?**

In a microservice system, we often run **multiple instances** of the same service to handle more traffic and avoid downtime. But then a question comes up:

When a request comes in, how does it get distributed across these service instances?

This is where the API Gateway acts like a load balancer. It does not just forward requests but also distributes them smartly to keep the system fast and stable.

Let us say we have three instances of **product-service** running in three instances:

Instance	Address
product-service-1	localhost:8081
product-service-2	localhost:8082
product-service-3	localhost:8083

When a client sends a request to `/products`, we do not want all traffic to go to just one instance. Instead, we want to share the load. That is exactly what the API Gateway does automatically.

How It Works Internally

1. API Gateway receives a request.
2. It checks which service to forward the request to.
3. It gets a list of service instances from Eureka (or static config).
4. It applies load balancing strategy.
5. It selects one instance and sends the request to it.

By default, it uses Round Robin, one request goes to instance 1, next to instance 2, and so on.

Tools for Load Balancing with API Gateway

Tool	Type	Purpose
Spring Cloud LoadBalancer	Client-side	Built-in load balancer in Spring
Eureka + LoadBalancer	Client-side	Service discovery + load balancing
API Gateway (Spring Cloud Gateway)	Server-side	Routes and load balances
Nginx	Server-side	Works as reverse proxy and load balancer
Kubernetes LoadBalancer	Cluster level	Manages load balancing in containers
Ribbon (Deprecated)	Client-side	Old Netflix load balancer

Let us see how load balancing works in a simple microservice setup using Spring Cloud Gateway and Eureka.

Step 1: pom.xml dependencies

```
<dependency>
```

```
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

Step 2: application.yml configuration

```
spring:
  application:
    name: api-gateway

  cloud:
    gateway:
      routes:
        - id: product-service
          uri: lb://product-service //
          predicates:
            - Path=/products/**
```

Key Point

lb://product-service -> here **lb** stands for **load balancer**.

This tells Spring Cloud Gateway: "Send /products requests to product-service using load balancing."

So, if three instances of product-service are running, the Gateway will forward traffic to all of them in rotation using **Spring Cloud LoadBalancer**.

67. **Microservice 63:** What is difference between API gateway and load-balancer?

In a microservice system, both API Gateway and Load Balancer are used to handle incoming traffic, but they solve different problems.

A load balancer only distributes traffic across multiple servers or instances to improve performance and availability. It does not understand business logic or API routes. It simply takes incoming requests and shares them across instances to avoid overload.

API Gateway is more powerful. It sits at the entry point of the system and controls how requests move to different microservices. It can route requests to the correct service based on the URL path. It can also handle security, logging, authentication, request filtering, response transformation, and API composition.

Load balancer only works at the network level, but API Gateway works at the application level.

Reference: [API Gateway vs Load Balancer](#) | [Microservices Interviews](#)

68. **Microservice 64:** Can API Gateway replace load balancer completely?

No, API Gateway cannot fully replace a load balancer. A load balancer manages traffic distribution, fault tolerance, and high availability at the network level, while an API Gateway handles routing, authentication, rate limiting, and cross-cutting concerns. They solve different problems and are often used together in production.

69. **Microservice 65: Real time uses of API Gateway?**

To understand the real power of an API Gateway, let's see a few real-world examples from platforms we use every day.

API Gateway acts like the main entry point for all client requests in a system that has many microservices. It handles routing, authentication, and response aggregation. Let's see how it plays out in real life.

1. **Instagram - Managing Different Services Behind One Door**

When we open Instagram, so many things happen at once.

Our profile loads, new posts appear, stories show up, and notifications appear instantly.

Each of these comes from a different microservice:

- Feed Service (for posts)
- Story Service (for stories)
- Notification Service (for likes and comments)
- Chat Service (for messages)
- User Service (for profile data)

Instead of the mobile app calling each of these services separately, all requests go through one single API Gateway.

The Gateway receives the request, checks security, and routes it to the right service. It then combines the responses and sends them back to the app.

This makes the app faster, lighter, and easier to maintain.

70. **Microservice 66: Some more Questions on API Gateway**

1. What if multiple instances of same service are running, then how API gateway routes?

When multiple instances of the same service are running, the API Gateway does not choose randomly. It asks the Service Registry like Eureka (**Refer Question – Microservice 71**) for all available instances. Then it uses load balancing, usually round robin, to send each request to a different instance. This helps share load and improves performance.

2. What if Eureka is not included?

If Eureka is not used, the API Gateway cannot discover service instances automatically. In that case, we must configure fixed service URLs manually in the gateway. Load balancing will not be dynamic. It can only work if we provide static addresses or use another service registry like Consul or Kubernetes.

3. How service registry works with Kubernetes?

In Kubernetes, we do not use a separate service registry like Eureka because Kubernetes itself becomes the registry.

Let us see how it works .

When a microservice is deployed, Kubernetes runs it as many pods. These pods keep changing. Some die, some get created, some move to new machines. Their IP addresses also keep changing.

Now we create a Kubernetes Service for that microservice. This Service gets a fixed name like *order-service*

Kubernetes stores a live list of all pods that belong to this service. It knows which pods are healthy and running.

Inside the cluster, every service name is added to DNS. So when another service or an API Gateway calls *http://order-service*

Kubernetes DNS finds the service. Kubernetes then picks one healthy pod behind it. The request is sent to that pod.

If a pod crashes, Kubernetes removes it from the list. If a new pod starts, Kubernetes adds it.

So, the service name always points to the right running pods.

4. What happens if the API Gateway goes down?

If the API Gateway goes down, all client requests fail because it's the main entry point.

To avoid this, we usually run multiple gateway instances with load balancing and health checks. If one fails, another takes over, keeping the system available without interruptions.

5. How does an API Gateway handle traffic during peak hours or sudden load spikes?

During high traffic, the gateway scales horizontally by adding more instances automatically. Load balancers distribute requests across these instances. It may also use caching and rate limiting to reduce backend load, ensuring all services continue running smoothly without delays or failures.

6. What strategies can we use to make an API Gateway highly available?

We can make an API Gateway highly available by running it on multiple servers or containers across different zones. Load balancers manage traffic, while health checks detect failures and reroute requests. Auto-scaling and backups ensure smooth operation even during heavy traffic or downtime.

7. Can API Gateway perform authentication and authorization? How?

Yes, the API Gateway can handle authentication and authorization before sending requests to services. It checks tokens, API keys, or credentials, ensuring only valid users access protected routes. This centralizes security, reduces duplication across services, and simplifies managing permissions for all incoming requests.

8. What role does API Gateway play in request and response aggregation?

The API Gateway collects data from multiple microservices, combines it, and sends a single response to the client. For example, on a shopping app, it can merge product, price, and review data. This reduces the number of client requests and improves overall performance.

9. Can API Gateway be used to connect both internal and external clients?

Yes, an API Gateway can handle requests from external users like mobile apps and internal systems like dashboards. It can separate routes or apply different rules for each, ensuring security and controlling how external or internal users interact with microservices.

10. How do microservices communicate internally if the Gateway is for external calls?

Microservices often communicate internally through direct service-to-service calls, message queues, or event-driven systems. The API Gateway is mainly for handling external client requests, while internal communication usually happens through a private network or service mesh designed for secure and efficient interactions.

11. Why do large companies like Netflix and Amazon rely heavily on API Gateways?

Large companies like Netflix and Amazon use API Gateways to manage millions of requests efficiently. The gateway handles routing, security, caching, and load balancing in one place. It simplifies

system management, reduces network complexity, and keeps user experiences fast and consistent across platforms.

If need more questions on Spring cloud gateway, check this link:

[Spring Cloud Gateway — Crash Course | by A cup of JAVA coffee with NeeSri | Medium](#)

71. **Microservice 67:** If API gateway is single entry point, how does it handle millions of requests?

When we say an API Gateway is a single-entry point, we don't mean there's only one machine sitting somewhere doing all the work.

It means, it's one **logical entry point** for clients means a single URL or domain that clients talk to. Behind that, there are many **instances** (or copies) of the gateway running in parallel.

Think of it like a big airport with one main gate for passengers.

From the outside, it looks like one entry point. But once we go in, there are dozens of counters and staff members handling passengers at the same time. That's how the system scales.

How API Gateway Handles Millions of Requests

1. Load Balancing

There are multiple gateway instances running behind the same URL. A load balancer sits in front of them and distributes incoming traffic evenly.

Example:

If one million requests come in, the load balancer splits them across hundreds of gateway instances. Each instance handles a small portion of the total traffic.

This prevents any single server from overloading.

2. Auto Scaling

Modern gateways are deployed in containerized environments like **Kubernetes** or **AWS ECS**.

These systems automatically create more gateway instances when traffic increases and remove extra ones when traffic drops.

Example:

If traffic doubles during a sale or an event, new gateway containers are automatically added within seconds.

3. Caching

The gateway can store frequently accessed data in memory for quick access.

For example, static responses like product categories or user profiles can be cached.

This reduces the number of calls to backend services.

Example:

If thousands of people request the same product details, the gateway returns the cached response instead of asking the Product Service every time.

4. Rate Limiting and Throttling (Refer Question – Microservice 106,107)

The gateway controls how many requests a client can make in a given time.

This prevents misuse and protects backend services from sudden traffic spikes.

Example:

If one user tries to send 500 requests per second, the gateway allows only a few and blocks the rest.

5. Fault Tolerance and Circuit Breakers (Refer Question – Microservice 102-104)

If a backend service fails or becomes slow, the gateway detects it and temporarily stops sending requests to that service.

It can return a fallback response or an error message to the client without affecting other parts of the system.

Example:

If the Review Service goes down, the gateway can still serve product and price details while skipping reviews.

6. Horizontal Scaling

The gateway doesn't need a single powerful machine. It's better to have many smaller machines or containers running in parallel.

Together, they can handle millions of requests per second efficiently.

Real World Example - Netflix

Netflix is one of the best examples of this.

It has a single API Gateway that manages requests from millions of users every minute.

Behind the scenes, hundreds of gateway instances run across different regions, automatically scaling up or down depending on traffic.

Load balancers distribute requests, caching reduces load, and monitoring tools watch performance in real time.

72. Microservice 68: What is the Aggregator pattern?

When a system has many microservices, there are times when we need to combine data from more than one service to build a single response. This is where the Aggregator pattern helps.

Instead of the client calling each service one by one and waiting for all responses, one single service takes that job. It collects data from different services, combines them, and returns one final result.

Let's imagine an online shopping portal. When we open a product page, we don't just see just the product name. We will also see the description, price, reviews, ratings, delivery details, and stock availability. All this information comes from different microservices:

- Product Service gives the name and details of the product.
- Price Service gives the current price.
- Review Service gives customer reviews and ratings.
- Inventory Service gives stock status.

If the client tries to call each service directly, the client must make multiple calls, handle different responses, and merge everything. That adds complexity and increases load time.

Instead, we use an Aggregator Service.

The Aggregator Service receives one request for product details. It then calls all required services in the background, collects their responses, merges them, and returns one clean and complete response.

So, the client gets everything in one shot.

This also makes the system easy to change later. If one of the services changes, only the aggregator logic needs to be updated. The client stays the same.

Simple flow for product details page:

1. Client sends request to Aggregator Service -> /product-details/123
2. Aggregator calls Product Service, Price Service, Review Service, and Inventory Service.
3. All the data is combined into one single structured response.
4. Client receives a single response with everything needed for the page.

This pattern improves performance and keeps the client simple. It also gives better control over how different service data is stitched together.

In Spring, this is often done by creating a dedicated aggregator controller or using the API Gateway with routing and fan-out logic. Tools like WebClient, RestTemplate, or Feign clients can be used to call other services and merge the results.

Sample Code using Aggregator Controller + Feigh Client.

```
//Aggregator Controller
@RestController
public class UserProfileAggregatorController {

    @Autowired
    private UserServiceClient userServiceClient;

    @Autowired
    private AddressServiceClient addressServiceClient;

    @Autowired
    private ContactServiceClient contactServiceClient;

    @GetMapping("/user-profile/{userId}")
    public UserProfile getUserProfile(@PathVariable String userId) {

        // Call multiple microservices
        User user = userServiceClient.getUser(userId);
        Address address = addressServiceClient.getAddress(userId);
        Contact contact = contactServiceClient.getContact(userId);

        // Build combined response
        UserProfile profile = new UserProfile();
        profile.setUserId(userId);
        profile.setName(user.getName());
        profile.setEmail(contact.getEmail());
        profile.setPhone(contact.getPhone());
        profile.setCity(address.getCity());
        profile.setCountry(address.getCountry());

        return profile;
    }
}
```

Feign Client:

```

@FeignClient(name = "user-service")
public interface UserServiceClient {
    @GetMapping("/users/{id}")
    User getUser(@PathVariable String id);
}

@FeignClient(name = "address-service")
public interface AddressServiceClient {
    @GetMapping("/address/{userId}")
    Address getAddress(@PathVariable String userId);
}

@FeignClient(name = "contact-service")
public interface ContactServiceClient {
    @GetMapping("/contact/{userId}")
    Contact getContact(@PathVariable String userId);
}

```

This code acts as an **API Aggregator** in Spring Boot. It receives one request from the client and internally calls multiple services like User Service, Address Service, and Contact Service. It then combines all responses and sends one final merged response back to the client.

73. Microservice 69: What is the API Composition and how does it differ from Aggregator?

In a microservice system, a single feature sometimes needs data from many services. API Composition is a pattern used to combine data from multiple microservices and build a meaningful response. It not only merges data but may also apply small business logic before sending the result back.

What is API Aggregation then?

A service (Aggregator) calls multiple micro-services, retrieves their data, and combines them into a single, unified response. An Aggregator collects data from multiple services and combines them into a single, unified response. It is Often implemented as a separate micro-service or gateway layer.

Examples:

1. Client wants to get user profile details. Aggregator calls:

- UserService → user data
 - OrderService → user orders
 - LoyaltyService → points balance
- Then combines the results into one DTO.

2. Fetching user details, order history, and loyalty points from three different micro-services and returning a single response to the frontend.

What is API Composition

API Composition is not a business service. It is a read layer for screens. It is used only to show data on UI. It calls many services and joins their data. But it does not apply rules.

For example, a Dashboard API.

It calls

- InventoryService for stock
- SalesService for sales
- NotificationService for alerts

It simply puts them together and sends them to the screen. It is just fetch and join.

API Composition is often used in query side of CQRS, Often used in GraphQL, BFF (Backend for Frontend), or API Gateway designs.

Reference:

74. **Microservice 70:** How API Composition is different from GraphQL?

API Composition and GraphQL are both used to collect data from multiple services, but they are not the same. They solve different problems and follow different approaches.

API Composition is a backend design pattern. It lives inside the server and combines data from different services to build a meaningful response. It can also apply business rules while merging data. The shape of the response is always fixed by the backend team. Clients cannot change it. API Composition is useful when the response needs some backend decisions before sending it to the client, like calculating a final status or merging business information.

GraphQL is a query language for APIs on Frontend. It allows the client to decide what data it needs. This gives full control to the frontend. Instead of receiving a fixed response, the client selects only the required fields. GraphQL works best when the main goal is to reduce multiple API calls and avoid sending unwanted data.

75. **Microservice 71:** What is service discovery and service registry?

In traditional systems, everything used to run on one server. We could simply say:

“Connect to localhost on port 8080.”

But in a microservice setup, things are different. Services can move between machines, restart automatically, or even scale up and down depending on demand.

One day, the Payment Service might run on 10.0.1.12, and the next day it might move to 10.0.1.15. If we hardcode these addresses, we'll break communication every time something changes.

We need a system that always knows where each service is running, no matter how many times it moves.

That's where **Service Registry** and **Service Discovery** come in.

What Is a Service Registry

A Service Registry is like a dynamic address book for the microservices. The Service Registry stores that information. If a service shuts down or crashes, it removes that entry automatically.

This means at any moment, the Service Registry always knows the current list of healthy, running services and where they are located.

What Is Service Discovery

Service Discovery is how other services find and connect to each other using the information inside the Service Registry.

When one service wants to talk to another, it doesn't guess the address. Instead, it asks the Service Registry: The registry responds with the current location. Then the calling service connects directly.

This discovery process happens automatically, and the services don't need to know each other's exact locations anymore.

How It Works in Microservices

Let's take an example of three services:

- Order Service
 - Payment Service
 - Notification Service
1. When the **Payment Service** starts, it registers itself with the Service Registry (for example, Eureka).
 2. The **Order Service**, before calling Payment Service, asks the registry where it is.
 3. The registry replies with the current address of Payment Service.
 4. The **Order Service** sends its request.

If Payment Service moves to a new server, the registry automatically updates its address. No manual change is needed anywhere in the system.

There are two ways to discover services:

1. **Client-Side Discovery**
2. **Server-Side Discovery**

Tool for Service Registry and Discovery is:

- **Netflix Eureka**: very popular in Spring Boot microservices.

Reference: [What is service discovery really all about? - Microservices Basics Tutorial](#)

76. Microservice 72: What is Server-side discovery and Client-side discovery.?

1. Client-Side Discovery

In client-side discovery, the client is smart.

When Service A wants to call Service B, it does not call it directly. Instead, it first asks the Service

Registry (like Eureka or Consul) where Service B is running. The registry gives the current address, and Service A connects to it.

The client decides which instance of Service B to call if there are multiple running. It even handles load balancing on its own.

How It Works

Client -> Service Registry -> Get address of Service B -> Call Service B

Example

Let's say Order Service wants to talk to Product Service. It asks Eureka for the address of Product Service and then calls it.

Tools Used

- Eureka (Netflix OSS)
- Ribbon (Client-side load balancer)

Used

- Used in Spring Boot with **Eureka + Feign Client**
- Common in Netflix-style microservice systems

2. Server-Side Discovery

In server-side discovery, the client is simple.

It sends the request to a load balancer or API gateway, which then talks to the Service Registry, finds the correct instance of the service, and forwards the request.

The client does not worry about where the services are. The server or gateway takes care of everything.

How It Works

Client -> Load Balancer / API Gateway -> Service Registry -> Service B

Example

A mobile app sends a request to the **API Gateway**. The gateway checks the service registry to find **Product Service** and forwards the request there.

Tools Used

- Spring Cloud Gateway
- AWS Elastic Load Balancer (ALB)
- Nginx
- Kubernetes Service

Used

- Common in cloud systems (AWS, Kubernetes)
- Modern microservice architecture

Simply:

Client-side discovery is like every person checking Google Maps themselves to find a location.

Server-side discovery is like asking a receptionist who guides us to the right place.

77. Microservice 73: When to choose Server-side discovery and Client-side discovery?

In client-side discovery, each service directly talks to the service registry to find the location of another service. The client is responsible for choosing which instance to call.

Example Scenario

We build a small system with 6–8 microservices using Spring Boot and Eureka. We want fast service-to-service communication inside the system and don't want to add a gateway yet. In this case, client-side discovery is easy and fits well.

In server-side discovery, the client does not discover services. Instead, it sends the request to a load balancer or API Gateway, and that component finds the service location using the registry.

Example Scenario

We have 20+ services and our frontend and mobile apps connect through a single URL using API Gateway. The API Gateway handles routing, load balancing, rate limiting, logging, and authentication. In this case, server-side discovery fits perfectly.

78. **Microservice 74:** How do you implement Service registry in Spring?

We can use **Eureka**, built by Netflix, is one of the most popular tools for this in Spring Boot.

How Eureka Works

1. Eureka Server (Service Registry)

This is the main directory where all microservices register themselves.

2. Eureka Clients (Microservices)

Each service (like User Service, Product Service) registers itself with the Eureka Server. It also keeps checking in ("heartbeat") so Eureka knows it's still alive.

3. Service Discovery

When one service wants to call another (for example, Order Service wants to call Product Service), it asks Eureka for the current location of that service.

Code link: [Spring-Samples/ServiceRegistryWithEureka at main · CodingLyf-Fullstack/Spring-Samples](#)

Project Setup Overview

We'll create **three small Spring Boot projects**:

1. **Eureka Server**: the service registry
2. **Product Service**: a client that registers itself
3. **Order Service**: another client that discovers and calls Product Service

Step 1: Create Eureka Server

Dependencies (pom.xml)

Add this dependency:

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>
```

Main Class

```
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

application.yml

```
spring:
  application:
    name: eureka-server
server:
  port: 8761

eureka:
  client:
    register-with-eureka: false # Server doesn't register itself
    fetch-registry: false # Server doesn't fetch registry
```

1. register-with-eureka: false

This setting tells the Eureka Server not to register itself as a service inside the registry. Only client services like product-service or order-service should register. The Eureka Server is just a directory, not a service that others should discover, so it does not need to register itself.

2. fetch-registry: false

This tells the Eureka Server not to fetch or download a list of services from another Eureka Server. Since this setup has only one Eureka Server, there is no other registry to communicate with. That means it should not try to get any service information from anywhere else.

Simply, fetch-registry: false means the Eureka server will **not try to get a list of services** from another Eureka server.

Now start the server and open: <http://localhost:8761>

We can see the Eureka dashboard; it's empty for now because no services are registered yet.

Step 2: Create Product Service**Dependencies (pom.xml)**

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

Main Class

```
@SpringBootApplication
@EnableDiscoveryClient
public class ProductServiceApplication {

    public static void main(String[] args) {
```

```

        SpringApplication.run(ProductserviceApplication.class, args);
    }
}

```

application.yml

```

server:
  port: 8081

spring:
  application:
    name: productservice

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/

```

This setting tells the microservice where the Eureka Server is running so it can register itself there. defaultZone is the address of the Eureka Server. Here, it is running on localhost at port 8761. So this product-service will connect to Eureka at <http://localhost:8761/eureka/> during startup.

Controller

```

@RestController
public class ProductController {

    @GetMapping("/products")
    public List<String> getProducts() {
        return List.of("Laptop", "Phone", "Tablet");
    }
}

```

Now, when we start this service, it automatically registers itself with the Eureka server. We can see it listed under Instances currently registered on the Eureka dashboard which is running here <http://localhost:8761>.

Step 3: Create Order Service

This service will discover the Product Service from Eureka and call it.

Dependencies (pom.xml)

```

<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>

```

Main Class

```

@SpringBootApplication
@EnableDiscoveryClient
@EnableFeignClients

public class OrderserviceApplication {

```

```

    public static void main(String[] args) {
        SpringApplication.run(OrderserviceApplication.class, args);
    }
}

```

application.yml

```

server:
  port: 8082

spring:
  application:
    name: orderservice

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/

```

Feign Client (Refer Question – Microservice 125)

```

@FeignClient(name = "productservice")
public interface ProductClient {
    @GetMapping("/products")
    List<String> getProducts();
}

```

Controller

```

@RestController
public class OrderController {

    private final ProductClient productClient;

    public OrderController(ProductClient productClient) {
        this.productClient = productClient;
    }

    @GetMapping("/orders")
    public List<String> getOrders() {
        List<String> products = productClient.getProducts();
        return List.of("Order1 - " + products.get(0), "Order2 - " + products.get(1));
    }
}

```

Eureka Dashboard View

Once all three services are running, open: <http://localhost:8761>

The screenshot shows the Spring Eureka web interface. The top navigation bar includes 'HOME' and 'LAST 1000 SINCE STARTUP'. The main content area is divided into several sections:

- System Status:** A table showing environment details.

Environment	test	Current time	2025-12-11T09:58:20 +0530
Data center	default	Uptime	00:03
		Lease expiration enabled	false
		Renews threshold	5
		Renews (last min)	0
- DS Replicas:** A section with a link to 'localhost'.
- Instances currently registered with Eureka:** A table listing registered services.

Application	AMIs	Availability Zones	Status
ORDERSERVICE	n/a (1)	(1)	UP (1) - DESKTOP-LOQ5583.orderservice.8082
PRODUCTSERVICE	n/a (1)	(1)	UP (1) - DESKTOP-LOQ5583.productservice.8081
- General Info:** A section for additional information.

We Can see:

- product-service
- order-service

Both marked as "UP."

How It Works

1. When **Product Service** starts, it registers itself with **Eureka Server**.
2. **Order Service** also registers itself.
3. When we hit <http://localhost:8082/orders>, the Order Service asks Eureka for "productservice."
4. Eureka returns its current location (host and port).
5. The call is made automatically through **Feign**, no hardcoded IPs needed.

We can see what are the services running in Eureka server from this address:

localhost:8761/eureka/apps

79. Microservice 75: How Eureka server works internally?

Eureka is a Service Registry used in microservices to keep track of all running service instances. It knows which services are active, which are down, and where they are running.

To understand it better, let us see how it works inside.

a) How It Tracks Service Instances?

When a microservice like order-service starts, it does not know who else is running. So, it sends a message to Eureka Server and says: "I am order-service and I am running at this address."

- This first message is called service registration.
- Eureka Server saves this information in memory as a service registry list.
- If we run two or three instances of the same service, like:

order-service running on [port 8082](#)

order-service running on [port 8083](#)

```
order-service running on port 8084
```

Then all three instances register separately with Eureka, but under the same service name. Eureka tracks them as separate entries and can send traffic to any one of them.

b) How Eureka Knows If a Service Is Alive?

Just registering once is not enough. A service may crash later or we may stop one instance. To solve this, Eureka uses a **heartbeat system**.

Every service that registers with Eureka **sends a heartbeat signal every 30 seconds**.

This is a simple message that says: "I am still alive. Keep me in the registry."

If Eureka does not receive a heartbeat from a service for around **90 seconds**, it assumes that the service is dead and removes it from the list.

c) Who Sends the Heartbeat?

The **Eureka Client** sends the heartbeat automatically.

There is no manual work here. Spring Cloud Eureka has a small background task that keeps sending heartbeat messages to Eureka Server every 30 seconds using HTTP.

So, if we start three instances of a service, each one sends its own heartbeat:

4) What if service stops sending heart beat due to network failures?

Eureka is careful during network issues. If many heartbeats suddenly stop (due to network failure, not real service failure), Eureka does not remove services quickly. It activates something called **self-preservation mode**.

This avoids false removals and keeps the system stable during temporary issues.

For detailed explanation about service discovery, check this video link: [Java Microservices Concept Walkthrough - Service Discovery and Registration | Spring Cloud Eureka #5](#)

80. **Microservice 76:** If two or more instances of same service are running, how Eureka client decides which service to call?

In a microservice system, we often run more than one instance of the same service. This is done to handle more users, improve performance, and avoid failure if one instance crashes. For example:

All three instances register themselves with Eureka Server. Now the question is:

When another service, like order-service, calls product-service, which instance does Eureka choose?

```
product-service running on port 8081
```

```
product-service running on port 8082
```

```
product-service running on port 8083
```

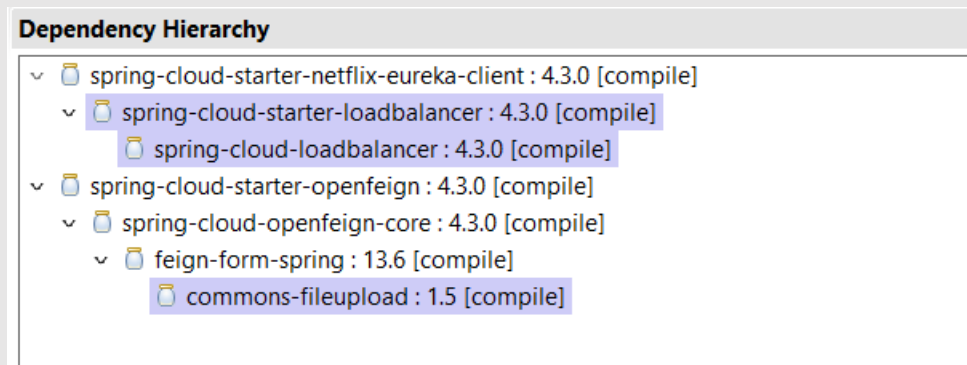
a) How It Works?

Eureka itself does not decide which instance to call. It only gives a list of available service instances. The Eureka Client in the calling service (for example, order-service) decides which one to call by using load balancing.

b) Who Does the Load Balancing?

In a Spring Boot Eureka setup, load balancing is handled on the client side. Earlier, this was done by Netflix Ribbon, and now it is done by Spring Cloud LoadBalancer, which replaced Ribbon.

spring-cloud-starter-netflix-eureka-client dependency internally contains *spring-cloud-loadbalancer*



c) How Instance Is Selected?

Spring Cloud LoadBalancer uses **Round Robin** by default. That means it sends requests one by one to each instance in order.

Point to remember: When we add Eureka-Client in a Spring Boot microservice, Spring automatically enables load balancing without any extra setup.

Earlier, Netflix Ribbon was used for client-side load balancing, but it is now deprecated. Spring replaced Ribbon with Spring Cloud LoadBalancer, which is lighter, faster, and maintained by the Spring team.

81. Microservice 77: What is the difference between API Gateway and Service Discovery?

At first, both API Gateway and Service Discovery may seem similar because they both deal with how services communicate.

But their purposes are different. API Gateway focuses on external access, and Service Discovery focuses on internal communication.

Let's break it down in simple, clear terms.

1. Purpose

API Gateway acts as the single-entry point for all external client requests. It decides which microservice should handle the request and forwards it there.

It's like the main reception desk of a company, everything from the outside world passes through it.

Service Discovery helps microservices find and communicate with each other *inside* the system.

It's like the internal directory of that company, where departments look up where others are located.

2. How They Work

- **API Gateway:**

A client (like a mobile app or website) never calls microservices directly. It always sends the

request to the API Gateway first.

The gateway then routes the request to the right microservice, for example, Order, Payment, or User.

- **Service Discovery:**

Inside the system, when one service (say, Order) needs to talk to another (say, Payment), it asks the **Service Registry** for the current address of that service.

It helps the services locate each other dynamically without hardcoding IPs or URLs.

3. Example in a Microservice Setup

Let's say we have:

- **Client (Frontend App)**
- **API Gateway**
- **Order Service**
- **Payment Service**
- **Notification Service**
- **Eureka (Service Registry)**

Here's what happens:

1. The **Order**, **Payment**, and **Notification** services register themselves with Eureka (Service Registry).
2. The **API Gateway** also connects to Eureka.
3. When a client sends a payment request, the Gateway looks up the **Payment Service** address in Eureka.
4. The request is then routed correctly to the live instance of Payment Service.

Without Service Discovery, the Gateway wouldn't know where to send the request.

82. **Microservice 78:** How API Gateway and Eureka server works together?

In a microservice system, both **Eureka** and **API Gateway** play important roles, and they often work side by side. They solve two different problems:

- Eureka helps services find each other
- API Gateway helps control how requests enter the system

Together, they make a clean, safe, and flexible architecture.

Why Do We Need Them Together?

Let us say we are building an **online movie platform (Book my show)**. We have these services:

- user-service
- movie-service
- review-service

- payment-service

Now imagine a mobile app wants to fetch a user's watchlist. Without a gateway, it would need to know the address of each service. This becomes messy when services run on different servers or different ports, and when we scale to more instances. Also, we want to secure all services at one place.

How Eureka Helps

Eureka acts like a **service directory**. Each microservice registers itself with Eureka when it starts. If a service has two or three instances, all of them register with Eureka. Eureka keeps track of which ones are UP and healthy.

Example registry inside Eureka:

Service Name	Instance Address
movie-service	localhost:8081
movie-service	localhost:8082
review-service	localhost:8083
user-service	localhost:8084

Now Eureka knows where each service is running and movie-service is running in two instances.

How API Gateway Helps

Instead of the mobile app talking to services directly, all requests go through the API Gateway. It is the single-entry point to our system.

The gateway checks the incoming request, decides which service to route to, and sends it there. It also adds extra features like logging, authentication, rate limiting, and load balancing.

How They Work Together, Step by Step

Let us say a request comes from the mobile app: `GET /movies/top10`

Here is what happens inside:

1. The request first reaches the **API Gateway**
2. The Gateway checks its route table and sees this belongs to movie-service
3. But the Gateway does not know where movie-service is running, so it asks **Eureka**
4. Eureka responds with a list of available movie-service instances:
 - localhost:8081
 - localhost:8082
5. Gateway picks one instance using load balancing (round robin)
6. Gateway forwards the request to that instance
7. The response goes back to the client through Gateway

Everything happens behind the scenes. The client only talks to the Gateway and does not know anything about service locations. Services can scale, restart, move, and the client never gets affected.

Code Sample link: [How to Build an API Gateway with Spring Cloud Gateway and Eureka — The Beginner's Guide | by Vivek Rajyaguru | Medium](#)

83. **Microservice 79: Is Service discovery mandatory to have in distributed systems?**

No, Service Discovery is not always mandatory, but in most true distributed or microservice systems, it becomes almost essential.

When It's NOT Mandatory

We can skip Service Discovery when:

1. **We have a small, static system**
 - For example, only 2 or 3 services running on fixed servers.
 - Each service has a known IP or DNS name that rarely changes.
2. **We use a load balancer with static backend configuration**
 - Tools like Nginx or AWS ALB can forward traffic to fixed backend servers.
 - This works fine when scaling isn't dynamic.
3. **We deploy everything in a single machine or simple VM setup**
 - For small internal tools or prototypes, a discovery service might be unnecessary.

In these simple setups, hardcoded URLs or environment-based configuration files are enough.

When It BECOMES Mandatory

Service Discovery becomes critical when:

1. **We scale dynamically**
 - In container environments like **Kubernetes** services can come and go frequently.
 - The system must automatically detect these changes.
 - In Kubernetes, the service registry is a built-in, automated system that maps **Service names** to their stable **ClusterIP** addresses, which in turn route traffic to dynamic Pod IPs
2. **We use many microservices**
 - When 20+ services talk to each other, managing addresses manually becomes impossible.
3. **We deploy in cloud environments**
 - In AWS, Azure, or GCP, services are distributed across zones and networks, Service Discovery keeps track of them automatically.
4. **We need fault tolerance**
 - If one service instance fails, the discovery mechanism removes it from the registry, so requests aren't sent to dead nodes.

5. Deployment patterns

84. **Microservice 80:** What is the deployments pattern we use Microservice?

When we work with microservices, one of the key parts of the system is how we deploy these services. Deployment patterns help us decide where and how each service should run in production.

Each service can have its own lifecycle. That means we can update one service without touching the others. To make this work smoothly, we use different deployment patterns.

Let's imagine an online shopping portal with services like Product, Price, Order, Review, and Inventory. Each of these can be deployed in different ways depending on the system's needs.

1. Multiple Service Instances per Host

In this pattern, we run many services on the same machine or server. For example, the Product Service and Price Service might run on one machine, while Order and Review run on another.

This is simple to manage when the system is small. But as the number of services grows, it becomes harder to isolate issues. If one service fails, it may affect others on the same machine.

Example:

Product and Price Service are deployed together on one server. If the server goes down, both services become unavailable.

2. Single Service Instance per Host

Here, each service runs on its own machine or virtual machine. This gives better isolation and control. It's easier to restart or scale one service without touching the others.

Example:

Product Service runs on one machine, Price Service runs on another, and Order Service runs on a separate machine. If Price Service needs more resources, we add more machines only for it.

3. Service Instance per Container

In modern systems, we use containers to deploy services. A container is a lightweight package that has everything the service needs to run. This gives flexibility and makes deployments fast and consistent.

Example:

Product Service runs inside one Docker container, Price Service in another, and so on. We can scale each service by adding more containers whenever needed.

4. Serverless Deployment

Here, we don't manage any servers. The service runs only when needed. It's event-driven. This is useful for small or less frequently used services.

Example:

A Notification Service sends order confirmation messages. It doesn't run all the time. It runs only when an order is placed, sends the message, and stops.

85. **Microservice 81:** What is End to End Deployment Steps we follow in Microservice?

Note: This is high level of information on how we can handle deployment in distributed systems. From next question we will go deep on each step like (containerization, Docket, Kubernetes)

Let's walk through how a real microservice project is deployed from start to finish. Think of this like a journey. We build the project, pack it, move it, deploy it, and finally keep it running smoothly.

Step 1: Build the Application

Everything starts with the application itself. Let's say we build a small online shopping portal with services like Product, Order, and Payment.

Each service is developed using Spring Boot. We write code, add dependencies, and build it using Maven or Gradle to create a JAR file. This JAR file is what will run inside our container.

Step 2: Create a Docker Image (Refer Question Microservice-83 for Docker)

Once the service is ready, we containerize (Refer Question Microservice-82 for Containerization) it. We create a **Dockerfile** that tells Docker how to build the container image for our service. This image contains:

- JDK
- Our Spring Boot JAR
- All required configurations

Then we run:

```
docker build -t product-service:1.0 .
```

This creates an image named product-service:1.0.

We repeat this for other services like order and payment. Sample docker file:

```
# Step 1: Use OpenJDK as the base image
FROM openjdk:17-jdk-slim

# Step 2: Create a directory inside the container
WORKDIR /app

# Step 3: Copy the Spring Boot JAR into the container
COPY target/product-service-1.0.jar app.jar

# Step 4: Expose port (optional, for documentation)
EXPOSE 8080

# Step 5: Run the application
ENTRYPOINT ["java", "-jar", "app.jar"]
```

To build the image: `docker build -t product-service:1.0 .`

To run the container: `docker run -p 8080:8080 product-service:1.0`

The image contains:

- JDK (from openjdk:17-jdk-slim)
- Your Spring Boot JAR (copied as app.jar)
- All configs packaged inside the jar

Step 3: Push the Image to a Container Registry

A container registry is like a storage place for these images.

Instead of keeping everything on our laptop, we push them to a place where our deployment platform can pull from.

We can use:

- Docker Hub
- Amazon ECR
- Google Container Registry
- Azure Container Registry

For example:

```
docker tag product-service:1.0 myrepo/product-service:1.0
```

```
docker push myrepo/product-service:1.0
```

Now the image is safely stored in the registry.

Step 4: Prepare Kubernetes Deployment Files - Container Orchestration

When we have one container, Docker is enough. But when we have many containers (for product, order, payment, notification services), we need something to manage them.

That's where Kubernetes comes in.

Kubernetes can:

- Deploy multiple containers
- Restart failed containers
- Scale up or down automatically
- Load balance traffic

Here, we write YAML deployment files. Sample YAML File:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: product-service
spec:
  replicas: 3
  selector:
    matchLabels:
      app: product-service
  template:
    metadata:
      labels:
        app: product-service
    spec:
      containers:
        - name: product-service
          image: product-service:1.0
          ports:
            - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
```

```
name: product-service
spec:
  type: ClusterIP
  selector:
    app: product-service
  ports:
    - port: 8080
      targetPort: 8080
```

Kubernetes learns from this YAML:

- **Which image to run:** product-service:1.0
- **How many containers:** replicas: 3
- **How to expose it:** Service on port 8080
- **Self-healing:** Kubernetes restarts failed pods automatically
- **Scaling:** Change replicas for scale up/down

Step 5: Deploy to Kubernetes Cluster

Once YML files ready, we apply them to the Kubernetes cluster.

```
kubectl apply -f product-deployment.yaml
```

```
kubectl apply -f product-service.yaml
```

Kubernetes pulls the image from the registry and runs it in containers.
We repeat the same for the order and payment services.

Step 6: API Gateway and External Access

Once the services are running, we usually don't want clients to call each one directly.
We set up an API Gateway that acts like the front door.

The gateway receives incoming requests and forwards them to the correct service.
For example:

- /products goes to Product Service
- /orders goes to Order Service

We can use Spring Cloud Gateway, Nginx, or any API gateway.

Step 7: Configuration and Secrets

Every environment (dev, test, prod) has its own configuration.
We store them safely using:

- ConfigMaps (for normal configs)
- Secrets (for passwords, tokens)

Kubernetes mounts them into containers at runtime. This keeps everything secure and clean.

Step 8: Scaling and Load Balancing

If traffic increases, we scale up the service.

```
kubectl scale deployment product-deployment --replicas=5
```

Now 5 containers handle incoming traffic. Kubernetes load balances them automatically.

Step 9: Observability and Monitoring

We set up tools to watch what's happening in the system. Metrics, logs, and tracing help us keep everything healthy.

Tools commonly used:

- Prometheus and Grafana for metrics

This step is key to catch problems early.

Step 10: Automate with CI/CD

Doing all these steps manually every time is slow. We set up a CI/CD pipeline.

Typical flow:

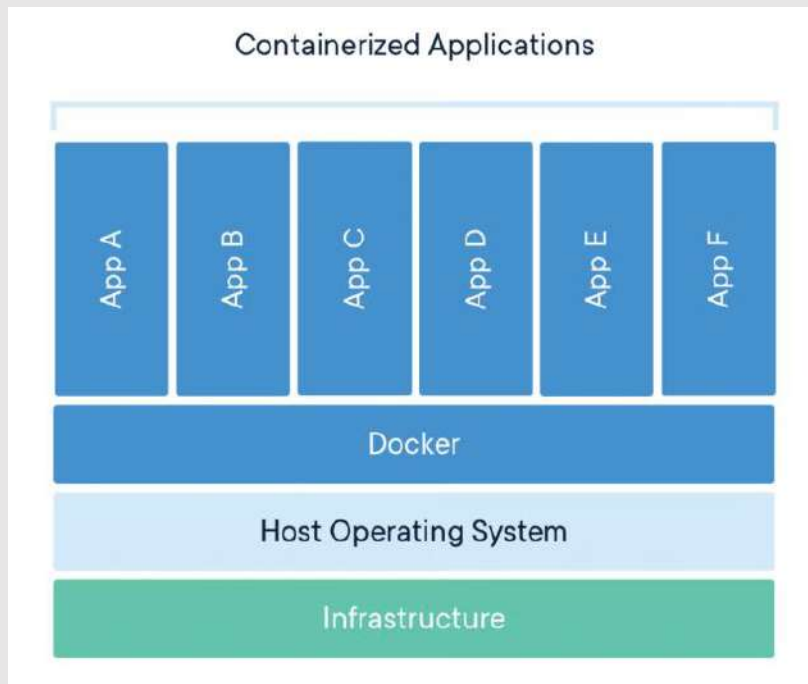
1. Developer pushes code to Git
2. Pipeline builds the Docker image
3. Pushes it to the registry
4. Deploys it to Kubernetes
5. Runs tests
6. Updates the running services

Tools like Jenkins, GitHub Action are often used.

86. **Microservice 82:** What is containerization and why it useful?

Let's imagine we built three services for a food delivery app: Items, Order, and Payment. Each one runs well on a local system. But when we move them to a testing server or production, things may break. One machine may have a different Java version. Another may not have the right library. This is where containerization comes in.

Containerization means packing everything needed to run an application into a single unit called a **container**. This includes the code, libraries, configuration, and runtime. Once we build the container, it will run the same way on any machine, whether it's a laptop, a server, or the cloud.



Simply put, containerization allows an application to be written once and run anywhere

Common Tools

The most used tool for containerization is **Docker**. It allows us to build, run, and manage containers with simple commands.

Other tools that work with containers are:

- Kubernetes (for managing many containers in production)

Example with Product Service

Let's say we have a simple Spring Boot Product Service.

Step 1: We first build the JAR file: `mvn clean package`

Step 2: Then we create a file called **Dockerfile** in the root folder:

```
# Base image with Java
FROM openjdk:17-jdk-slim

# Copy the jar file into the container
COPY target/product-service-0.0.1-SNAPSHOT.jar product-service.jar

# Run the application
ENTRYPOINT ["java", "-jar", "/product-service.jar"]
```

Step 3: Build the Docker image: `docker build -t product-service:1.0`

Step 4: Run the container: `docker run -p 8081:8081 product-service:1.0`

The Product Service is now running inside a container. If we move this image to any machine that has Docker installed, it will behave the same way.

Uses:**1. Runs the same everywhere**

A container has the application and everything it needs. So it runs the same on any machine.

Example

A Spring Boot app works the same on a laptop, a test server or a cloud server without setup issues.

2. Fast startup time

Containers start quickly because they are light.

Example

A product service container can start in a few seconds during deployment, which keeps downtime low.

3. Easy scaling

We can run more containers when load increases.

Example

During a big sale, an order service can scale from 2 containers to 10 containers to handle more traffic.

4. Easy to roll back

If a new version fails, we can switch back to the old image.

Example

Version 1.1 of payment service fails. We instantly move back to version 1.0 without trouble.

5. Better resource usage

Containers use memory and CPU more efficiently than full virtual machines.

Example

Instead of running five VMs, we run ten containers on the same server without performance issues.

6. Simple deployments

A Dockerfile builds a repeatable image, making deployments predictable.

Example

A team can deploy the same image to development, testing and production without changes.

7. Easy updates

We can update a service by replacing the old container with a new one.

Example

A bug fix in the catalog service is pushed as a new image, and Kubernetes replaces old containers automatically.

Reference Video : [Lets understand containers | 5 simple steps to create a docker container](#)

87. Microservice 83: How are Docker and Kubernetes related??

Docker and Kubernetes are often mentioned together, but they solve different parts of the same problem. Docker helps us create and run containers. Kubernetes helps us manage many containers once our system grows.

Docker is a container tool. It lets us pack the application, its libraries and its settings into one small unit. This unit runs the same on any machine. It is simple to build and great for small or medium services.

The challenge starts when we have many containers. A real system may have product, order, payment and notification services running at the same time. Each service may need five or ten containers during busy hours. Managing all of these by hand becomes difficult.

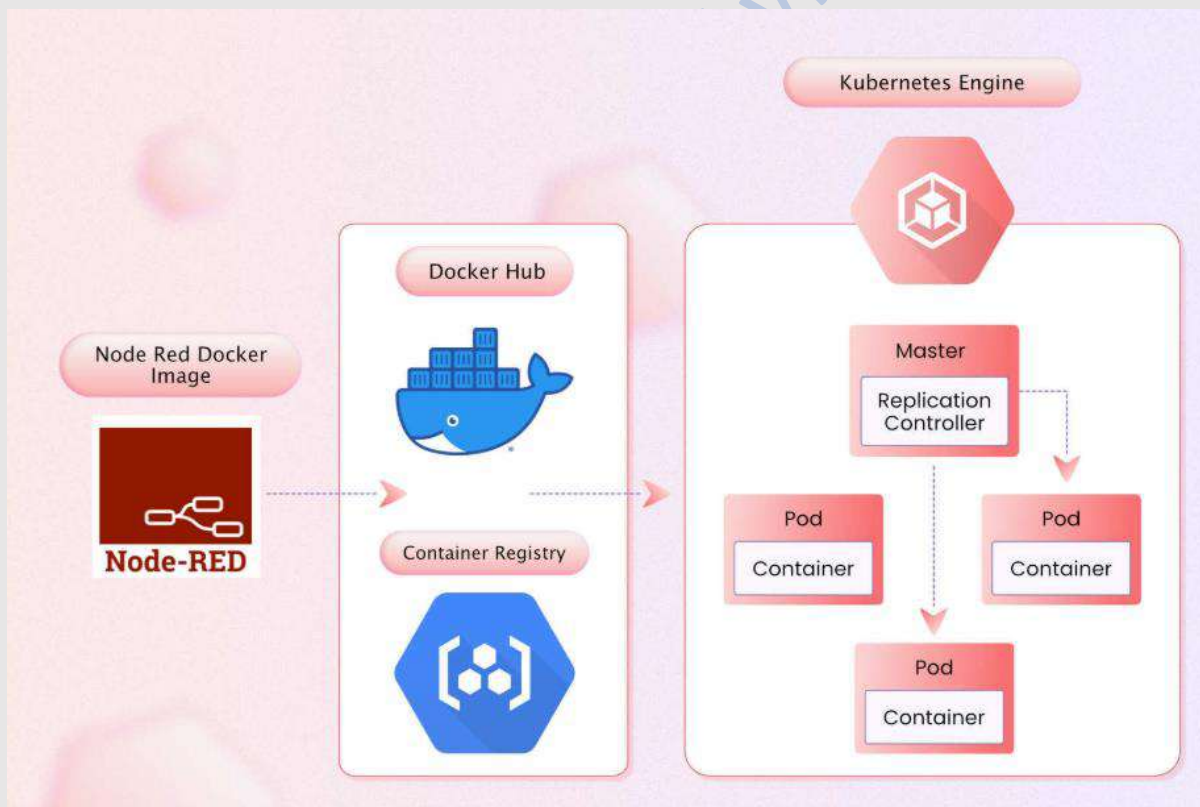
This is where Kubernetes helps. Kubernetes is a container manager. It decides where containers run, restarts them if they fail, adds more containers when traffic increases and removes them when traffic goes down. It can also balance requests between containers and handle smooth updates.

A simple example makes it clear.

Docker builds a container for the order service.

Kubernetes runs ten copies of that container during a festival sale, keeps them healthy and sends traffic to all of them. When the sale ends, Kubernetes reduces it back to two containers.

Docker creates containers. Kubernetes takes care of them. They work side by side and make the whole system stable and scalable.



Common Tools

The most used tool for containerization is **Docker**. It allows us to build, run, and manage containers with simple commands.

Other tools that work with containers are:

- Kubernetes (for managing many containers in production)

Example with Product Service

Let's say we have a simple Spring Boot Product Service.

Step 1: We first build the JAR file: `mvn clean package`

Step 2: Then we create a file called **Dockerfile** in the root folder:

```
# Base image with Java
FROM openjdk:17-jdk-slim

# Copy the jar file into the container
COPY target/product-service-0.0.1-SNAPSHOT.jar product-service.jar

# Run the application
ENTRYPOINT ["java", "-jar", "/product-service.jar"]
```

Step 3: Build the Docker image: `docker build -t product-service:1.0`

Step 4: Run the container: `docker run -p 8081:8081 product-service:1.0`

The Product Service is now running inside a container. If we move this image to any machine that has Docker installed, it will behave the same way.

88. **Microservice 84:** What is Docker, how it useful and how it works?

Before Docker came into the picture, running software on different machines was a trouble. Every machine had its own setup, versions, and dependency issues.

Let's imagine a Product Service built on Java 17. On the development machine, everything works fine. But when it moves to the testing server, Java might be of a different version or some library might be missing. The service breaks.

Developers spend hours fixing environment problems instead of building features.

To fix these problems, many teams started using **virtual machines**.

How Virtual Machines solved it

VMs gave each application its own full operating system, so the environment stayed the same across machines. This reduced dependency issues but used heavy resources.

But virtual machines are heavy. Each VM runs its own full operating system, takes time to start, and eats a lot of resources.

There was a clear need for something lighter, faster, and more flexible.

That's where containers came in.

How Containers solved it

Containers packed the application with only what it needed, including the exact Java 17 runtime and all required libraries. So the Product Service behaves the same on any machine, because the container carries its own fixed environment. No server needs Java installed, and version issues disappear completely.

Containers use the operating system of the host machine instead of creating a separate operating system for each application. This makes them much lighter, because they don't need to start a full OS every time. They also start faster and can be moved or copied easily between different machines without breaking.

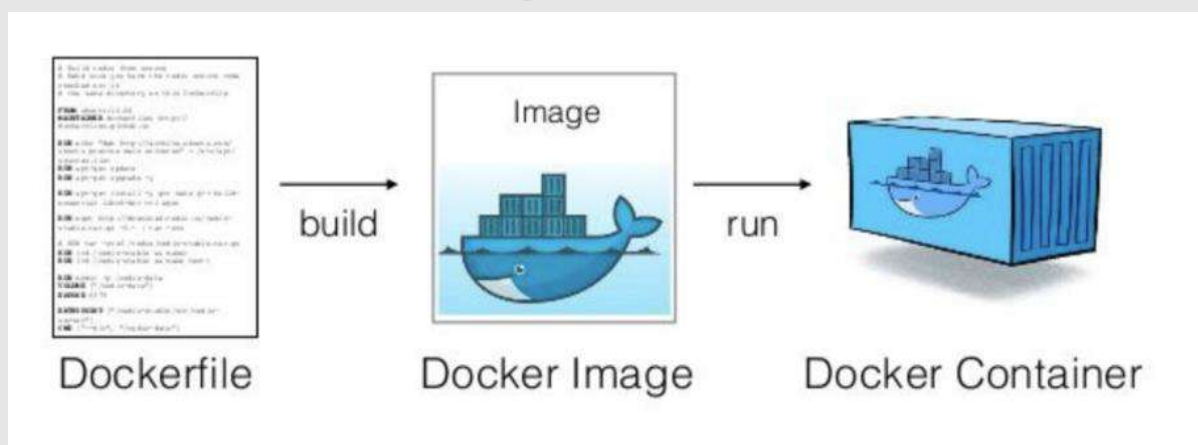
Docker made containers simple to build and run. It gave developers a way to package everything needed for an application into one single image that works everywhere.

Definition: Docker is a tool that makes containerization simple and practical. It helps us build, ship, and run applications inside containers without worrying about what kind of machine or environment we are using.

Think of Docker as a box that carries everything our application needs: code, libraries, and runtime. Once we build that box, we can run it anywhere and it will work exactly the same way.

How Docker Works

1. We create a **Dockerfile**. It is a simple text file with instructions on how to build the container image.
2. Docker reads the Dockerfile and builds an **image**.
3. An image is a blueprint. A **container** is a running copy of that image.
4. We run the container, and the application starts.



Example: Product Service

Let's say we built a Spring Boot Product Service that returns a list of products.

```
@RestController
public class ProductController {
```

```
@GetMapping("/products")
public List<String> getProducts() {
    return List.of("Laptop", "Phone", "Tablet");
}
```

Step-1: We build the jar file: `mvn clean package`. This creates a jar in the target folder. Now we create a Dockerfile to containerize it.

Step-2: Create a Dockerfile

```
# Use OpenJDK 17 as the base image
FROM openjdk:17-jdk-slim

# Copy the jar file from target folder to the container
COPY target/product-service-0.0.1-SNAPSHOT.jar product-service.jar

# Expose the port the application runs on
EXPOSE 8081

# Run the jar file when the container starts
ENTRYPOINT ["java", "-jar", "/product-service.jar"]
```

Step-3: Now we build the Docker image: `docker build -t product-service:1.0`

Step-4: Run the container from the image: `docker run -p 8081:8081 product-service:1.0`

What happens here:

- Docker uses the image as a blueprint
- Starts a new container
- Runs the Product Service on port 8081
- We can access it on `http://localhost:8081/products`

89. Microservice 85: Difference of Virtual machine, container and Docker?

A **Virtual Machine (VM)** is like running a full computer inside another computer.

A **Container** is like running just the application inside a shared computer.

Docker is the most popular tool that helps build and run containers.

Virtual Machine

A virtual machine comes with its own full operating system. It has its own kernel and runs on top of a hypervisor.

Think of it like having multiple separate computers inside one big computer.

Example: In an online shopping portal, suppose we want three services, Product, Order, and Payment. With VMs, each service would need its own operating system.

This means three OS installations, more memory use, and slower startup time.

Container

A container shares the same host operating system but runs its own isolated environment. It only carries what the application needs, not the whole OS. This makes it light, fast, and easy to move between machines.

Example: For the same Product, Order, and Payment services, we can run three containers on one OS. No extra OS needed. They start in seconds and use less memory.

Docker

Docker is a platform that helps build, package, and run containers easily. It acts as a manager for containers, making sure everything runs the same way on any machine.

Example: We create a Docker image for each service and run it anywhere, in our local system, testing server, or cloud, without changing the code.

90. Microservice 86: List out some docker commands?

Here is a list of important Docker commands, categorized for clarity:

Image Management:

- `docker pull <image_name>`: Downloads an image from a registry (e.g., Docker Hub).
- `docker build -t <image_name> <path_to_Dockerfile>`: Builds a Docker image from a Dockerfile.
- `docker images`: Lists all local Docker images.
- `docker rmi <image_name_or_id>`: Removes one or more images.
- `docker push <image_name>`: Pushes an image to a registry.

Container Management:

- `docker run <image_name>`: Creates and starts a new container from an image.
- `docker ps`: Lists running containers.
- `docker ps -a`: Lists all containers, including stopped ones.
- `docker start <container_name_or_id>`: Starts a stopped container.
- `docker stop <container_name_or_id>`: Stops a running container.
- `docker restart <container_name_or_id>`: Restarts a container.
- `docker rm <container_name_or_id>`: Removes one or more stopped containers.
- `docker exec -it <container_name_or_id> <command>`: Executes a command inside a running container.
- `docker logs <container_name_or_id>`: Displays the logs of a container.

91. **Microservice 87: What is Kubernetes and how it is useful in Microservices?**

Before Kubernetes came into the picture, managing applications across multiple servers was tough.

Let's imagine we built an online shopping portal with services like Product, Order, and Payment. We containerized each one using Docker. Everything worked great on one or two servers. But as traffic grew, we needed to run dozens, sometimes hundreds, of containers.

That's where the real trouble began.

Before Kubernetes

In the early days, when we ran multiple containers, we had to handle everything manually. We had to:

- Start and stop containers ourselves
- Restart them when they crashed
- Keep track of which server each container was running on
- Distribute traffic evenly among containers
- Update containers without taking the system down

Let's say our Product Service crashed at midnight. Someone had to log in, restart it, and make sure it connected back to the rest of the system. If a new version of the Order Service needed deployment, we had to stop the old one first and start the new one carefully, all manually.

Managing this manually was fine for a few containers but became impossible when dozens of services were involved.

How Kubernetes Changed Everything

Kubernetes was created to solve all these problems. It's an **open-source system** that helps us **deploy, manage, and scale containers automatically**.

Definition:

Kubernetes is an open-source container orchestration platform that automates many of the manual processes involved in deploying, managing, and scaling containerized applications.

Think of it as a smart manager for containers. We tell Kubernetes what our application should look like, how many containers, what images to use, what ports to open, and it takes care of everything.

If a container crashes, Kubernetes restarts it automatically.

If traffic increases, Kubernetes creates new containers to handle the load.

If a node (server) fails, Kubernetes moves containers to healthy machines without downtime.

We no longer worry about where each container runs, Kubernetes handles the placement, scaling, and recovery automatically.

Real World Example

Let's go back to our online shopping portal. We have three main services: Product, Order, and Payment.

- Each service runs inside Docker containers.
- We deploy them using Kubernetes.

When 10,000 users visit the site during a festival sale, Kubernetes notices the traffic spike. It automatically creates more containers of the Product and Order services to handle the load. When traffic drops later, it scales down to save resources.

If one container crashes, Kubernetes replaces it instantly.

If we release a new version of the Payment Service, Kubernetes updates it gradually without stopping the old one. Users never notice the switch.

92. **Microservice 88:** Explain the architecture of Kubernetes?

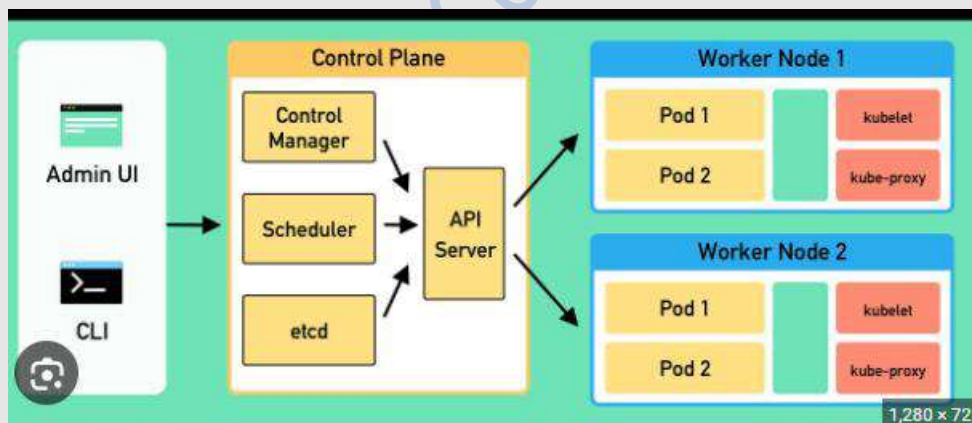
To understand how Kubernetes works, think of it like a team or crew in the airport.

Example: Every plane (a container) must take off, land, refuel, and follow clear instructions. There are people who plan the schedule, people who control flights, and workers who maintain the planes. Kubernetes works exactly like that, but for containers.

Video links: [Kubernetes crash course: In less than 15 minutes](#)
[Kubernetes Architecture in 7 minutes | K8s explained](#)

It has two main parts:

1. **Control Plane** – the brain that makes decisions.
2. **Worker Nodes** – the machines that actually run the containers.



Let's go step by step.

1. Control Plane - The Brain of Kubernetes

The Control Plane manages everything happening in the cluster. It decides where to run containers, monitors them, and reacts if something goes wrong.

It has a few key parts that work together:

a. API Server

The API Server is like the receptionist of the airport. It's the main communication point for everything. Whenever we give instructions (like "run five containers of Product Service"), the API Server receives them and passes them to the right people inside the control plane.

b. etcd

etcd is like the database that stores all important information about the cluster which containers are running, their current state, and the overall configuration.

If we think of Kubernetes as a control tower, etcd is where all flight plans and records are safely kept.

c. Scheduler

The Scheduler decides where to place new containers.

It looks at all worker nodes, checks which have enough resources, and assigns containers accordingly. For example, if Node A is busy but Node B is free, the Scheduler will tell Kubernetes to run new containers on Node B.

d. Controller Manager

The Controller Manager makes sure everything in the cluster stays as it should be.

If a container crashes, it creates a new one.

If we asked for five containers and only four are running, it starts another.

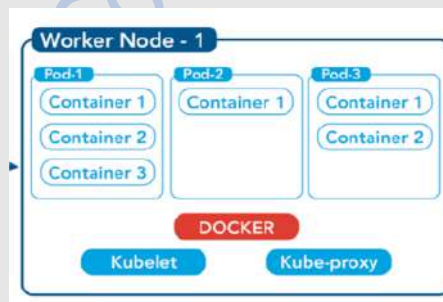
It constantly watches the system and fixes things automatically.

2. Worker Nodes - The Hands That Do the Work

Worker Nodes are the machines where our containers actually run.

Each node has its own small set of tools that help it communicate with the control plane.

Each node has three main components:



a. Kubelet

The Kubelet acts like a local supervisor.

It talks to the control plane and makes sure the containers assigned to that node are running properly.

If a container crashes, the Kubelet reports it and restarts it if needed.

b. Kube Proxy

The Kube Proxy manages communication between containers and the outside world.

It routes network traffic correctly so that services can talk to each other, even if they're running on different nodes.

c. Container Runtime

This is the software that actually runs the containers. Docker used to be the most common runtime. The runtime starts, stops, and manages the lifecycle of containers on the node.

3. Pods -The Smallest Building Block

In Kubernetes, containers don't run directly. They are grouped into small units called **Pods**. A Pod can have one or more containers that share the same network and storage. Pods are like small rooms inside a building; they hold containers that work closely together.

For example, in an online shopping app:

- The Product Service might have its own Pod.
- The Order Service runs in another Pod.
- The Payment Service in yet another.

4. Services - Connecting the Pieces

Services in Kubernetes act like stable doors to access Pods. Even if Pods are replaced or moved, the Service keeps the same name and IP. It ensures smooth communication between different parts of the application.

5. Ingress - The Entry Gate

Ingress is like the main gate for outside users to reach the application. It routes incoming traffic (like from a browser or app) to the correct Service inside the cluster.

93. **Microservice 89:** What are the types of deployments?

When we release a new version of our application, we want to do it safely, without breaking what's already running.

1. Rolling Update Deployment

This is the most common type used in real-world projects. Here, Kubernetes replaces old containers gradually, one by one, while keeping the application running.

Example:

Order Service is running five containers of version 1.0. Kubernetes creates one container of version 2.0, waits until it's ready, then removes one old container. It repeats this process until all are upgraded.

Pros:

- No downtime.
- Easy to rollback if something goes wrong.

Cons:

- Slightly slower than Recreate because of the gradual process.

When to use:

Perfect for production systems where uptime is critical, like online stores or payment system

2. Blue-Green Deployment

In this method, we run two environments side by side, one with the old version (Blue) and one with the new version (Green).

We send all user traffic to the Blue version while testing Green in the background.

Once the new version is tested and ready, Kubernetes switches traffic from Blue to Green instantly.

Example:

In an online shopping portal, version 1.0 (Blue) handles real users.

Version 2.0 (Green) runs quietly in the background.

When we are confident it's stable, Kubernetes routes all users to version 2.0 immediately.

Pros:

- Zero downtime.
- Instant rollback (just switch back to Blue).

Cons:

- Needs double the resources during deployment.

When to use:

Used in large-scale, high-traffic systems like e-commerce or banking apps.

3. Canary Deployment

Canary deployment means testing the new version with a small portion of users before giving it to everyone.

It's like releasing the update slowly to see if it works well.

Example:

Let's say our Payment Service has version 1.0 live.

We deploy version 2.0 but let only 10% of users use it.

If it works fine, we slowly increase it to 25%, then 50%, and finally 100%.

Pros:

- Safe and gradual rollout.
- Problems are detected early without affecting all users.

Cons:

- Takes more time and setup to manage traffic splits.

94. **Microservice 90:** What is the role of continuous integration (CI) and continuous deployment (CD) in Microservices??

Continuous Integration (CI) and Continuous Deployment (CD) play a very important role in microservices because each service is small, independent, and changes often.

To keep all services healthy, we need a smooth way to build, test, and release them without breaking anything.

The Role of CI in Microservices

CI makes sure every change we write is tested quickly and safely.

When a developer updates even one microservice, the CI pipeline automatically:

- pulls the new code
- runs unit tests

- runs integration tests
- builds a fresh service image

If anything fails, CI stops the process and tells us what broke.

Example:

If we change the Payment Service, CI immediately tests only the Payment Service code and its dependent tests.

This helps us catch bugs early before they reach production.

In microservices, CI is very important because each service changes often and must stay independent and stable.

The Role of CD in Microservices

CD takes the next step.

It deploys the successfully tested microservice automatically.

Once the CI tests pass, CD:

- creates a new version of the microservice
- deploys it to staging or production
- updates the service in Kubernetes or any target environment
- verifies that the service is running correctly

Since each microservice is small, deployments are quick and safe.

Example:

If we update only the Notification Service, CD deploys just that service without touching the others. This makes releases faster and reduces downtime.

CI/CD helps us:

- deploy small changes safely
- avoid breaking other services
- catch bugs early
- release updates faster
- keep the system stable

In simple words: **CI tests the change; CD delivers the change.**

95. Microservice 91: Can we use embedded server in microservice deployment for production?

Yes, we can use an embedded server in microservices for production deployment. In fact, most modern microservices use embedded servers by default. So that we need to setup separate sever to run the application.

What is an Embedded Server?

An embedded server means the server runs inside the application itself instead of being installed separately. In Spring Boot, when we build a microservice, it comes with **Tomcat** or **Jetty**

embedded inside the application. So, we do not need to deploy a WAR file on an external server. Instead, we package the application as a JAR and run it directly.
Example:

6.Observability pattern

96. **Microservice 92:** What observability design pattern?

When we build a system with many microservices, things can go wrong at any layer like network, database, or even inside a single container. Sometimes a request passes through five or six services before reaching its destination. If something fails midway, how do we know where it broke?

That's where the **Observability Design Pattern** helps.

Observability is not just about collecting logs or metrics. It's about understanding **what's happening inside a system** by looking from the outside.

It helps us answer three big questions:

1. Is the system healthy?
2. If something is slow, where is the problem?
3. Why did the problem happen?

Why We Need Observability

Let's imagine we built an online shopping portal.

When a customer places an order, the request moves through:

- API Gateway
- Order Service
- Payment Service
- Inventory Service
- Notification Service

If the order fails, how do we find the real issue?

Was it a database timeout? A slow API call? Or did the payment gateway fail?

Without observability, we would be guessing.

With observability, we can trace the entire path of that request and find exactly where it broke.

The Three Pillars of Observability

Observability mainly depends on three key elements: **Logs, Metrics, and Traces.**

1. Logs

Logs are like diary entries. They record what each service is doing and what went wrong. Each log contains details such as timestamp, service name, and message.

Example:

In the Payment Service, a log might say:

Payment failed for Order #12345 due to timeout from bank API.

By reading logs, we understand what happened inside each service.

2. Metrics

Metrics are numbers that tell us how the system is performing.

They show CPU usage, memory, request count, and error rates.

Metrics help us see trends, for example, if response times are slowly increasing over time.

Example:

The Order Service might show:

- Average response time: 250ms
- Error rate: 1.2%
- Active containers: 4

These numbers help us decide when to scale up or fix performance issues.

3. Traces

Traces follow a single request as it travels through multiple services. Each part of the request is called a **span**, and all spans together form a **trace**.

Example:

For an order request, a trace might look like this:

1. API Gateway -> 5ms
2. Order Service -> 20ms
3. Payment Service -> 150ms
4. Inventory Service -> 10ms
5. Notification Service -> 15ms

If the payment step took 150ms, we instantly know where the delay happened.

How It Works in Practice

In a real system, we use tools to collect and visualize this data:

- **Prometheus** and **Grafana** for metrics.
- **ELK Stack (Elasticsearch, Logstash, Kibana)** for logs.
- **Spring Cloud Sleuth** or **Zipkin** for distributed tracing.

Reference link: [Distributed Tracing with Spring Cloud Sleuth and Zipkin | by Bubu Tripathy | Medium](#)

Distributed Tracing System: Spring Cloud Sleuth

[\(23\) The Importance of Observability Patterns in Microservices Architecture | LinkedIn](#)

Each service sends its logs, metrics, and traces to these tools.

From a single dashboard, we can view all system activity and pinpoint problems in seconds.

Real World Example

Let's say a customer reports that payments are failing during checkout.

Without observability, we'd check logs manually in each service, hoping to find the error.

With observability in place:

- The trace shows the request was delayed in the Payment Service.
- The logs there reveal the "Bank API timeout."
- Metrics confirm that timeouts increased after 3 PM.

Now we know the issue is not in our code but in an external dependency.

This saves hours of debugging.

Note: You can mention what tools you have used in your project for logging and metrics.

97. **Microservice 93:** What is monitoring in the microservices?

In a monolithic application, everything runs in one place, so debugging is easier. But in microservices, the application is split into many small services. Each service runs separately, has its own database and its own network calls. If something breaks, it is hard to find where the problem started.

Monitoring in microservices means watching what is happening inside our system so that we know if everything is working correctly or if something is going wrong.

Example:

Let us take a simple food delivery system:

- Order Service
- Payment Service
- Delivery Service

When a customer places an order, the request passes through all three services. One day, customers start complaining that orders are failing. We do not know why. Payment might be slow or maybe Delivery Service is down. Without monitoring, we must check each service manually, read logs, and guess the issue. This takes hours.

With monitoring, we can see that:

- Order Service: healthy
- Delivery Service: healthy
- Payment Service: error rate 40 percent, response time high

At this point, we know the problem is inside Payment Service. We open logs and see "Timeout while connecting to bank API". Now we can fix the issue quickly.

Common tools used:

- **Prometheus**: collects metrics
- **Grafana**: shows metrics using graphs
- **Zipkin** or **Jaeger**: trace request flow between services
- **ELK Stack**: reads logs from all services

98. **Microservice 94: What is centralized logging in the microservices?**

Centralized logging means collecting logs from all microservices in one place so we can search and troubleshoot problems easily.

In microservices, each service runs on different servers, so logs are scattered. If a request fails, finding the root cause becomes difficult without a single log view.

Centralized logging solves this by storing all logs together and allowing us to filter them by service, time, or request ID.

Centralized logging tools do not dump logs randomly. They organize them with filters and search options.

We can search logs by:

- Service name
- Time range
- Log level (error, warn, info)
- User ID
- Request ID
- Keyword

Example: If a payment failed for user 1023, we can search like: `service:payment AND userId:1023 AND level:error`

Common tools: ELK Stack (Elasticsearch, Logstash, Kibana), Splunk etc.

99. **Microservice 95: What is distributed tracing in the microservices?**

Distributed tracing is a way to track a single request as it travels through multiple microservices.

In a microservice system, one user request may pass through many services like Order, Payment and Inventory. If the request fails or becomes slow, it is difficult to find where it got stuck. Distributed tracing solves this by giving each request a unique trace ID and following it across services. With this ID, we can see the complete flow, time taken by each service and where the problem started.

Tools used for distributed tracing are Spring Sleuth, Zipkin, Jaeger, OpenTelemetry.

Spring Cloud Sleuth is a library used in microservices to add distributed tracing easily. It automatically generates Trace ID and Span ID for every request and attaches them to logs. This helps us track the complete journey of a request across multiple services.

Sleuth is often used together with Zipkin or Jaeger to visualize the trace. It helps find slow services and failure points quickly.

100. **Microservice 96: What are the best practices we use for monitoring?**

Monitoring in microservices works well only when we follow some best practices.

- First, every service should expose health and metrics endpoints so we can track uptime, errors and response time.
- We should use dashboards to watch important metrics like CPU, memory and latency.
- Alerting rules must be added so teams know when a service is going down before users complain.
- Each request should include a trace ID for tracking.
- Logs, metrics and traces must be connected together to find root causes faster.
- Finally, monitoring should be tested in real conditions, not only after deployment, but also during development and testing.

101. **Microservice 97: What are tools and techniques we use for monitoring and logging in the microservices?**

ELK Stack

The ELK Stack is used for centralized logging. It collects logs from all services, stores them, and allows us to search and analyse them in one place.

It includes **Elasticsearch** for storage, **Logstash** for processing logs and **Kibana** for visualizing them. It helps in debugging and monitoring production systems.

Zipkin

Zipkin is a distributed tracing tool. It shows how a request moves from one microservice to another. It helps find slow services and performance issues.

By using trace IDs, Zipkin shows the complete call flow and time taken by each service, making troubleshooting simple.

Jaeger

Jaeger is another distributed tracing tool similar to Zipkin but built for large systems. It helps track request flow and detect latency issues between services.

It also supports performance tuning and root cause analysis in microservices.

Sleuth

Spring Cloud Sleuth automatically adds tracing information like Trace ID and Span ID to logs.

This helps us track a single request across multiple microservices. Sleuth is often used together with Zipkin or Jaeger.

Prometheus

Prometheus is a monitoring tool that collects metrics from microservices like CPU usage, memory, request count and error rate.

It stores time-series metrics and helps us understand the health and performance of services.

Grafana

Grafana is used to create dashboards for monitoring systems. It connects with Prometheus and shows metrics in graphs and charts.

It helps us watch real system behaviour in real time and take action before failures happen.

Log Correlation ID

A Log Correlation ID is a unique ID assigned to every request. It connects logs from different services related to the same request.

This makes debugging easier because we can track the journey of one request across many microservices by searching a single ID.

Reference: [Microservices Observability - What You Need to Know | Wissen](#)

7. Cross-cutting pattern

102. **Microservice 98:** What are the cross-cutting pattern in Microservices?

When we build a microservice system, each service usually has its own job, like processing payments, managing orders, or handling user details. But apart from these main responsibilities, there are some common tasks that every service needs.

For example, every service needs to handle errors, manage configurations, discover other services, and sometimes log or secure data.

These are not business features but are essential for the system to work correctly.

These shared needs are called **cross-cutting concerns**, and the way we handle them across all services is known as the **Cross-Cutting Concern Design Pattern**.

3 Important things we need to have as part of cross cutting concerns.

1. Error handling
2. Configuration Management
3. Service Discovery

1. Error Handling

In any system, errors are bound to happen like network failures, invalid inputs, or database timeouts. If we don't handle these properly, one small issue can cause a chain reaction of failures.

To make the system reliable, we apply error handling as a cross-cutting concern so all services handle failures in a consistent and graceful way.

Here are the main techniques used:

a. Try-Catch and Fallbacks

Every service wraps risky operations in a try-catch block and defines fallback responses. For example, if the Product Service is down, the Order Service can still accept the order and mark it as “pending” until the product data is available again.

b. Circuit Breaker (Refer Question: Microservice-103)

If one service repeatedly fails, a circuit breaker temporarily stops sending requests to it. Once the failing service recovers, the circuit “closes” automatically. This prevents one weak service from bringing down the whole system. Tools like **Resilience4j** or **Hystrix** are often used here.

c. Retry Mechanism (Refer Question No: Microservice-106)

Some errors are temporary, like a momentary network glitch. In such cases, the service can retry a few times before failing completely. We set retry limits and intervals to keep it balanced.

d. Centralized Exception Handling

In frameworks like **Spring Boot**, we can use `@ControllerAdvice` to handle all exceptions in one place. This ensures every service returns uniform, clear error messages.

Example:

In an online store, if the Payment Service fails during checkout, the Order Service uses a circuit breaker to pause communication. It then logs the failure, shows a friendly error message, and retries later - without crashing the system.

2. Configuration Management

Each service has its own configuration like database details, API URLs, tokens, and more. If these are stored separately inside every service, updating them becomes painful. Imagine changing a database password in 20 different services one by one.

Configuration Management as a cross-cutting concern solves this problem. We keep all configurations in one centralized place, and services read from there automatically at startup.

Example:

Using **Spring Cloud Config Server** (Refer Question No: Microservice-116), we can store configuration files in a Git repository. If we update one file (like changing a URL or key), every service refreshes its configuration automatically without redeployment.

Benefits:

- One place to manage all settings.
- Easy updates without downtime.
- Environment-based configuration (Dev, Test, Production).

3. Service Discovery (Refer Question No: Microservice-71)

In a microservice setup, services are dynamic. They start, stop, scale up, or move to other machines. If one service needs to talk to another, how does it know where it's running?

That's where **Service Discovery** comes in.

It allows services to find each other automatically without hardcoding addresses.

There are two main parts:

- **Service Registry:** Keeps track of all running services and their locations.
- **Service Discovery:** Lets services look up others when needed.

Example:

The Order Service doesn't need to know the Payment Service's exact location.

It simply asks the **Service Discovery Server** (like **Eureka**, **Consul**, or **Zookeeper**) for "Payment Service." The server returns the correct address, and communication happens smoothly.

This avoids manual configuration and helps the system adapt automatically when services scale or move.

103. Microservice 99: What is Resilience or Resilient System?

In the real world, things don't always go as planned. A network might fail for a few seconds, a database might restart, or a third-party service might slow down. In a large system with many microservices, these small problems happen all the time.

A **resilient system** is one that continues to work even when parts of it fail. It doesn't collapse when something breaks. Instead, it absorbs the problem, handles it gracefully, and recovers on its own.

In very simple terms, **Resilience** is the ability of something to withstand or recover quickly from difficulties or challenges. In the context of architecture and design, resilience refers to the ability of systems to adapt and recover from failures or disruptions. It's like having a strong and flexible foundation that can handle unexpected problems and keep going.

Short note: Resilience is not about avoiding failure. It's about **surviving failure**.

A Simple Way to Understand It

Let's imagine we run an **online movie streaming platform**.

The platform has several microservices:

- User Service
- Subscription Service
- Payment Service
- Recommendation Service
- Video Streaming Service

Now imagine the **Recommendation Service** goes down for a few minutes.

In a poorly designed system, this failure might cause the whole platform to stop working.

In a **resilient system**, everything else continues to function.

Users can still log in, play movies, and manage subscriptions. The only thing that fails temporarily is the “Recommended Movies” section.

104. **Microservice 100: How do you achieve resilience?**

We use a few design patterns and techniques to make our systems resilient.

1. Circuit Breaker

Stops calling a service that keeps failing and gives it time to recover.

For example, if the Payment Service fails multiple times, the Order Service stops calling it temporarily.

2. Retry

Tries the same request again if it fails once due to a temporary issue, like a short network drop.

3. Timeout

Limits how long a service waits for another service’s response.

This prevents threads from getting stuck waiting forever.

4. Fallback

Provides a backup response when the main service fails.

For instance, if the Recommendation Service is down, we can show popular movies instead.

5. Bulkhead (Refer Question: Microservice-111)

Separates services into isolated sections so one failure doesn’t overflow into others , just like compartments in a ship.

6. Load Balancing

Distributes requests evenly across service instances so no single one gets overloaded.

7. Health Checks and Monitoring

Continuously watch services to detect failures early and restart them automatically.

Reference: [Resilient Design Principles - GeeksforGeeks](#)

105. **Microservice 101: What is Fault Tolerance?**

Every system faces problems like a server might crash, a network might go down, or a piece of code might fail.

But a good system doesn’t stop working just because one-part breaks. It finds a way to keep going.

That ability to keep running even when something goes wrong is called **fault tolerance**.

Example:

Let’s imagine we are driving a car. If one headlight stops working at night, the car doesn’t shut down. We can still drive using the other headlight until we reach home or a repair shop. That’s fault tolerance means one part fails, but the system continues to function.

Now, think about an online system the same way. If one service or server fails, the others should still work, so users don’t face a complete outage.

Fault tolerance is about designing systems that can handle failure gracefully without total breakdown.

Why It Matters

In a microservice architecture, we usually have many small services talking to each other, like Order, Payment, User, and Notification.

If one service stops responding, and the others can't handle it, the entire system may go down.

Fault tolerance helps us make sure that even if one or two parts fail, the rest of the system keeps running. It keeps users happy and the business running smoothly.

106. **Microservice 102: What is the difference between fault tolerance and Resilience?**

Fault tolerance is about **continuing to work** even when some part of the system fails.

The system is built in such a way that it doesn't stop or crash when one-piece breaks.

Resilience is about **recovering and adapting** when failures happen.

It's not just about surviving a failure, but also bouncing back quickly and continuing to function smoothly afterward.

Example from Daily Life

Let's say we are watching a movie online.

- **Fault tolerance** is when the streaming platform automatically switches to another server if one fails, and we don't even notice any pause.
- **Resilience** is when the stream pauses for a second but then adjusts the quality and continues smoothly instead of stopping completely.

Example from Software Systems

Imagine we have three microservices: **Order**, **Payment**, and **Notification**.

- In a **fault-tolerant system**, if one instance of the Payment Service fails, another identical instance immediately takes over, users never notice the failure.
- In a **resilient system**, if the Payment Service fails, the Circuit Breaker opens, retries are triggered later, and the system automatically recovers without manual help.

Reference: [Making microservices Fault Tolerant and Resilient](#) | by Ritesh Panigrahi | Medium

107. **Microservice 103: What is circuit-breaker pattern?**

In a microservice system, services talk to each other constantly. for example, the **Order Service** calls the **Payment Service**, which might call an external **Bank API**.

Now imagine what happens if the Bank API goes down or becomes very slow.

Every time the Payment Service tries to reach it, it waits... and waits... until it times out.

This delay doesn't just affect one request, it piles up. Soon, all threads are busy waiting, and the whole system slows down.

That's exactly the kind of chain reaction we want to avoid. And that's why we use the **Circuit Breaker Pattern**.

The Idea Behind Circuit Breaker

The Circuit Breaker works like an **electrical fuse** in our home.

When something goes wrong, it “trips” and stops the current to prevent further damage.

Similarly, when a microservice detects too many failed calls to another service, the circuit “opens” meaning no new requests are sent until things recover.

This helps protect the rest of the system from repeated failures and unnecessary waiting.

Definition: The Circuit Breaker pattern is a resilience design pattern in distributed systems that prevents cascading failures by monitoring service calls; if a service fails repeatedly, the breaker “trips,” immediately stopping further calls to it and allowing it to recover, thereby protecting the entire system from overload and improving user experience by failing fast.

How It Works

The Circuit Breaker has three states:

1. Closed

Everything is working normally. All requests go through as usual. The breaker keeps an eye on failures and response times.

2. Open

If failures cross a certain limit, for example, 5 failed requests in a row, the circuit “opens.”

No more calls are made to the failing service.

Instead, the system immediately returns a fallback response or a friendly error.

This saves time and prevents overload.

3. Half-Open

After a short delay, the breaker allows a few test requests to see if the failing service has recovered.

If they succeed, the breaker closes again.

If they fail, it stays open for a while longer.

Example: Online Shopping Portal

Let’s say our **Order Service** calls the **Payment Service** for every new order.

If the Payment Service becomes slow or crashes, the Order Service will keep trying and waiting for responses.

After a few failed attempts, the circuit breaker opens and stops sending requests.

Instead, it immediately returns a message like *“Payment service is temporarily unavailable, please try again later.”*

Meanwhile, the system stays responsive, other features like viewing products or adding to cart still work fine.

After some time, the breaker tries again (half-open state).

If the Payment Service responds successfully, everything goes back to normal.

This way, one failing service doesn’t drag down the entire system.

Real Tools Used

In Java microservices, we often use:

- **Resilience4j**
- **Netflix Hystrix** (older but popular)

These libraries make it easy to define thresholds , for example:

- “If 50% of requests fail within 10 seconds, open the circuit.”
- “Try again after 30 seconds.”

All this happens automatically behind the scenes.

Reference: [Implementing the Circuit Breaker Pattern in Spring Boot: A Comprehensive Guide | Medium](#)

108. **Microservice 104: How do you implement Circuit breaker in Spring?**

We can use **Relience4J** or **Netflix** hystrix to implement. Resilience4j is the newer and lighter option, while Hystrix is older and no longer actively developed.

1. Implementing Circuit Breaker with Resilience4j

Resilience4j is designed to work perfectly with **Spring Boot**.

It gives us features like retries, rate limiting, and most importantly, circuit breaking, all with simple annotations.

Let’s imagine we are building an **Order Service** that calls a **Payment Service**.

If the Payment Service is slow or unavailable, we don’t want the Order Service to keep waiting.

We will use a circuit breaker so that after a few failed attempts, it stops calling the Payment Service for a short time and returns a fallback response instead.

Step 1: Add Dependency

In the Order Service, we add this to the pom.xml file:

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
</dependency>
```

Step 2: Enable Circuit Breaker

We enable Resilience4j by adding this annotation in the main application class:

```
@SpringBootApplication
@EnableCircuitBreaker
public class OrderServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(OrderServiceApplication.class, args);
    }
}
```

Step 3: Add Circuit Breaker to a Method

Now, in our service class, where we call the Payment Service:

```
@Service
public class PaymentClient {

    @CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
    public String makePayment() {
        RestTemplate restTemplate = new RestTemplate();
        return restTemplate.getForObject("http://localhost:8082/pay", String.class);
    }

    public String paymentFallback(Exception ex) {
        return "Payment service is currently unavailable. Please try again later.";
    }
}
```

Here's what happens:

- When the call to the Payment Service fails multiple times, the circuit breaker “opens”.
- New requests are blocked for a while.
- The fallback method runs immediately, giving a quick response.
- After some time, the circuit “half-opens” and tests the Payment Service again.
- If it responds successfully, the circuit closes.

Step 4: Configure Settings

We can adjust how the circuit behaves in application.yml:

```
resilience4j:
  circuitbreaker:
    instances:
      paymentService:
        failure-rate-threshold: 50
        sliding-window-size: 10
        wait-duration-in-open-state: 10s
        permitted-number-of-calls-in-half-open-state: 3
```

This means:

- If 50% of the last 10 calls fail, open the circuit.
- Keep it open for 10 seconds before trying again.
- Allow 3 trial calls in the half-open state.

Reference link: [Micro Service Patterns: Circuit Breaker with Spring Boot | by Salitha Chathuranga | Medium](#)

Code: [SalithaUCSC/spring-boot-circuit-breaker: Spring Boot Circuit Breaker](#)

The above link one of the best implementations I found. Download the code from GitHub, Run and follow the steps to understand how it works.

mvn clean install before running the project.

2. Implementing Circuit Breaker with Netflix Hystrix (Older Option)

Before Resilience4j, Hystrix was the most popular circuit breaker library. It works similarly but uses a different annotation:

```
@Service
public class PaymentClient {

    @HystrixCommand(fallbackMethod = "paymentFallback")
    public String makePayment() {
        RestTemplate restTemplate = new RestTemplate();
        return restTemplate.getForObject("http://localhost:8082/pay", String.class);
    }

    public String paymentFallback() {
        return "Payment service is down. Please try again later.";
    }
}
```

To use Hystrix, we add this dependency:

```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-hystrix</artifactId>
</dependency>
```

And enable it in the main class:

```
@EnableCircuitBreaker
@SpringBootApplication
public class OrderServiceApplication { }
```

Hystrix is still found in some legacy projects but has been replaced by Resilience4j because it's faster, lighter, and works well with Java 17 and Spring Boot 3.

109. **Microservice 105: When to use Circuit breaker in Spring?**

In a microservice world, every service depends on others. One small failure can spread quickly if we don't manage it well. That's where the **Circuit Breaker pattern** becomes useful. It helps us stop calling a service that is already failing and gives it time to recover.

Let's understand when and why we actually need to use it.

1. When One Service Depends on Another

Suppose we have an **Order Service** that calls a **Payment Service** to complete every order. If the Payment Service becomes slow or goes down, the Order Service will keep waiting for a response. Each request waits until it times out, using up memory and threads.

Over time, this delay can affect all users and may even crash the Order Service.

By using a Circuit Breaker, we can detect repeated failures and temporarily stop sending requests to the Payment Service.

Instead, we can return a quick fallback response like:

"Payment service is currently unavailable, please try again later."

This protects the Order Service from unnecessary waiting and keeps the rest of the system healthy.

2. When External APIs or Third-Party Services Are Involved

Sometimes our microservices depend on **external APIs**, for example, a bank API for payments, a shipping API for deliveries, or a currency conversion API for prices.

If these APIs fail or respond too slowly, our system shouldn't suffer. A Circuit Breaker helps us handle such unstable or unpredictable services gracefully.

Example:

In a travel booking app, the **Flight Service** calls an external airline API. If that API goes down, we open the circuit and stop calling it for a while.

During this time, we show users a message like *"Flight details are being updated, please check again shortly."*

The system stays responsive even though one external service failed.

3. When Performance Starts to Degrade

Even if a service is technically "up," it might be too slow to handle the load.

A Circuit Breaker can watch for performance issues. If calls start taking longer than expected or exceed a timeout limit, it opens the circuit.

That way, instead of waiting on a slow service, our system quickly returns a fallback response. It's better to fail fast than to slow down everything.

4. When We Have Limited Resources

In real systems, we have limited threads and memory.

If all threads are waiting for a response from a failing service, no new requests can be processed. The system becomes unresponsive even though it hasn't crashed.

Circuit Breakers prevent this by stopping calls to failing services early.

This ensures that healthy services and new users continue to work smoothly.

5. When We Need to Improve System Resilience

In distributed systems, failure is normal. Networks go down, APIs crash, databases restart. A resilient system accepts that things will fail and plans for recovery instead of collapse.

The Circuit Breaker pattern helps us build such resilience. It detects repeated failures, stops making risky calls, and retries automatically after a short break.

This simple behaviour can prevent one small issue from turning into a system-wide outage.

110. **Microservice 106: What is retry mechanism and how you achieve it?**

Sometimes, we call an external service or a database, and it fails, maybe because of a network glitch, a timeout. In such cases, instead of immediately throwing an error, we might want to try again a few times before giving up.

Retry Mechanism is a simple technique used when a request or operation fails due to a temporary issue, like a network glitch or timeout. Instead of giving up immediately, the system automatically tries the same request again after a short delay, improving reliability and avoiding unnecessary failures.

We achieve it using Spring-Retry. It gives us a neat way to retry failed operations automatically without writing boilerplate loops.

How it Works

When we put `@Retryable` on a method:

1. Spring wraps that method in a proxy.
2. If the method throws an exception that we have marked for retry, Spring will call the method again.
3. It repeats this until either:
 - The method succeeds
 - The maximum retry attempts are exhausted
4. If retries are exhausted, we can handle the failure using `@Recover`.

Example:

Step 1: Add Dependency

```
<dependency>
  <groupId>org.springframework.retry</groupId>
  <artifactId>spring-retry</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

Step 2: Enable Retry

```
@SpringBootApplication
@EnableRetry
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

Step 3: Use `@Retryable`

```
@Service
public class PaymentService {

    private int counter = 1;

    @Retryable(
        value = RuntimeException.class, // Which exception to retry
        maxAttempts = 3,                // How many times
        backoff = @Backoff(delay = 2000) // Wait 2s between retries
    )
```

```

    )
    public void makePayment() {
        System.out.println("Trying payment... attempt " + counter);
        counter++;
        throw new RuntimeException("Payment failed!");
    }

    @Recover
    public void recover(RuntimeException e) {
        System.out.println("All retries exhausted. Payment failed permanently: " +
e.getMessage());
    }
}

```

Key Points to Remember

- @Retryable works only on Spring-managed beans.
- It needs Spring AOP (that is why we added spring-boot-starter-aop).
- Use @Recover to define a fallback method after retries fail.
- We can control retries with:
 - maxAttempts means how many times
 - backoff means delay, multiplier (exponential backoff)
 - value to which exceptions trigger retry

Reference: [Implementing Method-Level Retry in Spring Boot | Medium](#)

111. **Microservice 107: What is Rate limiting and how do you implement in Microservices?**

In a microservice system, hundreds or even thousands of requests can reach a service every second. Some may come from genuine users, and some may come from automated scripts or even by mistake. If too many requests hit a service at once, it can become slow or even crash.

Rate Limiting helps us prevent that. It is a simple idea, we control **how many requests** a user, client, or system can make within a certain period of time.

Real-World Example

Let's say we run an online movie ticket platform. We have a Booking Service that handles seat reservations.

During a big movie release, thousands of users open the app and start booking tickets at the same time.

If every request goes to the Booking Service instantly, the database will get flooded, causes slow responses or even system crashes.

With Rate Limiting, we can control how many booking requests each user can make per second or minute. For example:

- Each user can send 5 booking requests per minute.
- If they exceed that, we show: *"You're trying too fast. Please wait a moment."*

This keeps the system safe and ensures fair access for all users.

How Rate Limiting Works

Rate limiting can be done at different levels:

1. **At API Gateway:** most common approach.
The gateway sits in front of all microservices and decides how many requests each client can make.
2. **At Service Level:** each microservice can implement its own limit for specific endpoints.
3. **At Network Level:** tools like Nginx or cloud load balancers can limit incoming requests globally.

112. **Microservice 108:** How can you implement Rate Limiter in Spring?

There are a few popular ways to add rate limiting in microservices:

1. Spring Cloud RateLimiter (Using API Gateway)

Most API Gateways like **Spring Cloud Gateway**, **Kong**, or **AWS API Gateway** support rate limiting out of the box.

Example in **Spring Cloud Gateway** (YAML configuration):

```
spring:
  cloud:
    gateway:
      routes:
        - id: booking-service
          uri: http://localhost:8081
          predicates:
            - Path=/bookings/**
          filters:
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 5 # requests per second
                redis-rate-limiter.burstCapacity: 10 # max burst capacity
                key-resolver: "#{@userKeyResolver}"
```

Here:

- Each user can make 5 requests per second.
- Burst capacity allows a short spike of up to 10.
- Redis keeps track of counts for each user across instances.

When to Use Spring Cloud RateLimiter

- When we already use Spring Cloud Gateway in front of microservices.
- Best for centralized rate limiting , all limits are enforced at the entry point.

2. Bucket4j

Bucket4j is a pure Java library for token-bucket-based rate limiting. It doesn't depend on Spring or any specific framework, so we can use it anywhere: inside microservices, filters, gateways, or REST controllers.

```
import io.github.bucket4j.*;
```

```

Bucket bucket = Bucket4j.builder()
    .addLimit(Bandwidth.classic(10, Refill.greedy(10, Duration.ofMinutes(1))))
    .build();

if (bucket.tryConsume(1)) {
    // process request
} else {
    // return "Too Many Requests"
}

```

When to Use Bucket4j

- When we want in-memory or distributed rate limiting logic inside our service.
- Works great for Spring Boot REST controllers or filters.

Reference: [Understanding Bucket4J use cases. Bucket4j is a Java rate-limiting... | by Arvind Kumar | Medium](#)

3. Resilience4j RateLimiter:

Resilience4j is a fault tolerance library, best known for circuit breakers and retries. It also provides a RateLimiter module, which works slightly differently from Bucket4j.

Instead of tracking “tokens,” Resilience4j RateLimiter uses permits per time window, meaning it controls how many calls can be made within a time frame.

```

import io.github.resilience4j.ratelimiter.*;
import java.time.Duration;

RateLimiterConfig config = RateLimiterConfig.custom()
    .limitForPeriod(5)
    .limitRefreshPeriod(Duration.ofSeconds(10))
    .timeoutDuration(Duration.ofSeconds(1))
    .build();

RateLimiter rateLimiter = RateLimiter.of("serviceA", config);

Runnable limitedCall = RateLimiter.decorateRunnable(rateLimiter, () -> {
    System.out.println("Processing request...");
});

limitedCall.run();

```

When to Use Resilience4j RateLimiter

- When we already use Resilience4j for Circuit Breaker, Retry, or Bulkhead, and want to keep consistency.
- When we want fine-grained control at method level using annotations.

Video link for Sample: [API Rate Limiter - Resilience4J + Spring Boot Implementation Example](#)

113. Microservice 109: What are the Rate limiting algorithms?**Reference video links:**

At a high level: [\(1\) Rate Limiter and its types #systemdesign #softwareengineer - YouTube](#)

More in depth with good graphics: [Five Rate Limiting Algorithms ~ Key Concepts in System Design](#)

Let's look at the main algorithms one by one and understand how each one works, along with the tools that use them in real projects.

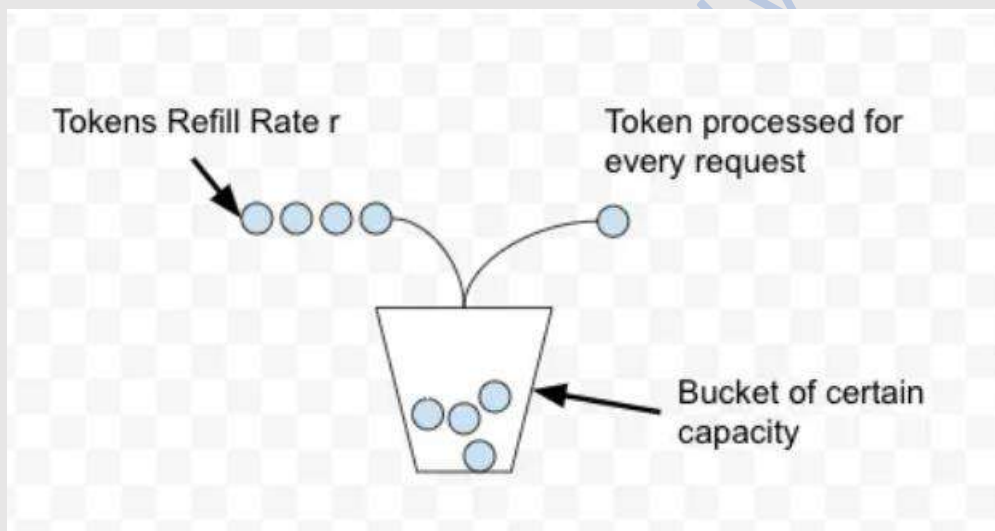
1. Token Bucket Algorithm

This is one of the most popular and practical algorithms used for rate limiting.

- Imagine a bucket that slowly fills up with tokens.
- Each request needs one token to pass through.
- If the bucket has tokens, the request is allowed.
- If it's empty, the request has to wait until more tokens are added.

Tokens are refilled at a steady rate, for example, 10 tokens every second.

This means we can allow short bursts of traffic without crashing the system, but still maintain an overall limit.

**Example:**

If we allow 10 requests per second, and a user sends 15 requests, the first 10 will pass, and the rest will be blocked or delayed until more tokens are available.

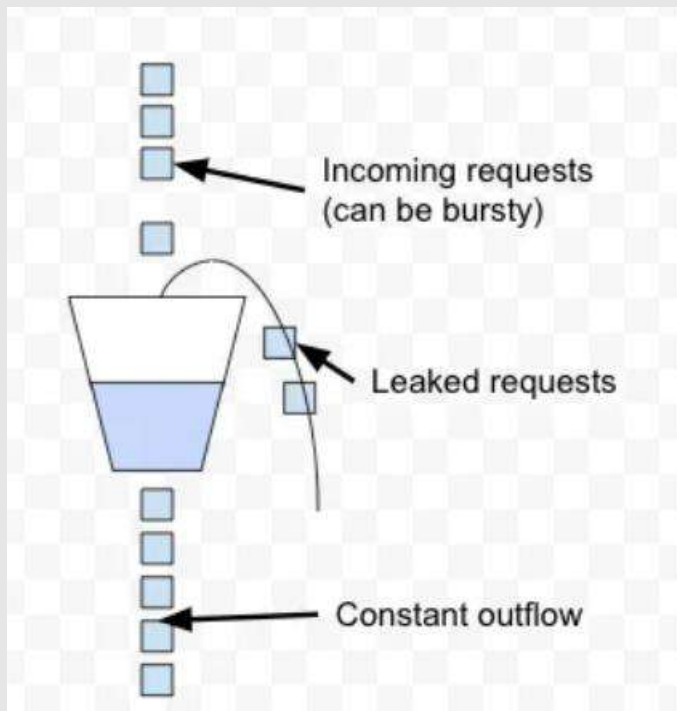
2. Leaky Bucket Algorithm

This is one of the oldest and simplest methods.

Imagine a bucket with a small hole at the bottom.

Requests enter the bucket like drops of water, but they leave at a fixed rate through the hole.

If too many requests come at once, the bucket overflows, and extra requests are dropped.



3. Fixed Window Counter

Time is divided into fixed windows, like every 1 minute or every 10 seconds.

We count how many requests arrive in that time.

If the count crosses the limit, new requests are blocked until the next window starts.

Good for: simple use cases, internal systems, or low traffic.

4. Sliding Window

This one is smarter than the fixed window. Instead of counting requests in strict one-minute boxes, it looks at the last few seconds continuously.

It checks how many requests arrived in the last “window” (say, last 10 seconds) and makes sure the total stays under the limit.

114. **Microservice 110:** Explain about Token bucket algorithm

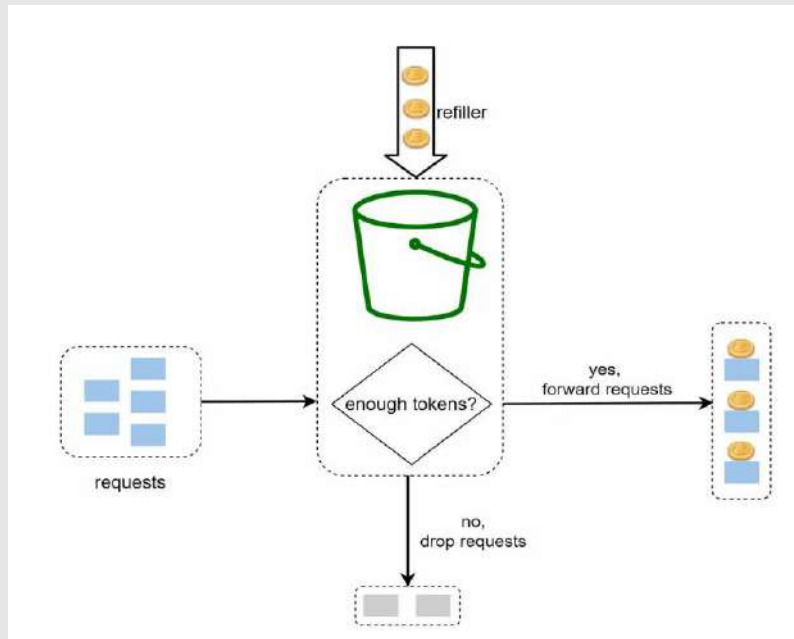
The bucket has tokens with a pre-defined capacity. When a request comes in it takes a token from the bucket and is processed further. If there is no token to be picked, the request is dropped and the user will have to retry.

The payment gateway, Stripe ([read here](#)) uses a token bucket algorithm to throttle their API requests.

From Stripe documentation:

We use the **token bucket algorithm to do rate limiting**. This algorithm has a centralized bucket host where you take tokens on each request, and slowly drip more tokens into the bucket. If the bucket

is empty, reject the request. In our case, **every Stripe user has a bucket, and every time they make a request, we remove a token from that bucket.** We implement our rate limiters using Redis.



Example use-case:

- We have our rate-limiting rules set to 3 requests per user per minute.
- User1 makes the request at 0th time interval, currently the bucket is full with 3 tokens so request will be processed. Tokens will be updated in the bucket to 2 now.
- User1 makes a 2nd request at 10th sec, bucket has 2 tokens so again the request will be processed further. Tokens will be updated in the bucket to 1 now.
- User1 makes a 3rd request at say 30th sec, bucket has 1 token so again the request will be processed further. Now the bucket is empty till the whole 1 minute completes.
- User1 makes a 4th request at say 55th sec, the bucket has 0 tokens so the request is throttled and the user is returned with a 429 status code - too many requests and asked to retry it later in some time. The HTTP 429 Too Many Requests response status code **indicates the user has sent too many requests in a given amount of time.**
- At the completion of that 1 minute, the token count is refreshed at a fixed rate and the bucket is again full with 3 tokens and requests can be processed again for that particular user.

In simple words:

In the Token Bucket algorithm, we process a token from the bucket for every request. New tokens are added to the bucket with rate " r ". The bucket can hold a maximum of b tokens. If a request comes and the bucket is full it is discarded.

The token bucket algorithm takes two parameters:

- Bucket size: the maximum number of tokens allowed in the bucket

- Refill rate: number of tokens put into the bucket every second

Sample Code:

```
public class TokenBucket {

    private static final double MAX_BUCKET_SIZE = 3.0; // Max tokens the bucket can hold
    private static final double REFILL_RATE = 1.0;      // Tokens added per second

    private double currentBucketSize;
    private long lastRefillTimestamp; // in nanoseconds

    public TokenBucket() {
        this.currentBucketSize = MAX_BUCKET_SIZE;
        this.lastRefillTimestamp = System.nanoTime();
    }

    // Method to check if a request can be allowed
    public synchronized boolean allowRequest(double tokens) {
        refill(); // refill tokens before checking

        if (currentBucketSize >= tokens) {
            currentBucketSize -= tokens;
            return true; // request allowed
        }

        return false; // not enough tokens, request dropped
    }

    // Refill tokens based on elapsed time
    private void refill() {
        long now = System.currentTimeMillis();
        long timePassed = now - lastRefillTimestamp;

        long tokensToAdd = (timePassed * refillRate) / 1000; // refillRate is tokens per
second
        currentBucketSize = Math.min(MAX_BUCKET_SIZE, currentBucketSize + tokensToAdd);
        lastRefillTimestamp = now;
    }

    public static void main(String[] args) {
        TokenBucket bucket = new TokenBucket();

        System.out.println("Request processed: " + bucket.allowRequest(1)); // true
        System.out.println("Request processed: " + bucket.allowRequest(1)); // true
        System.out.println("Request processed: " + bucket.allowRequest(1)); // true
        System.out.println("Request processed: " + bucket.allowRequest(1)); // false
(bucket empty)
    }
}
```

Explanation:

1. The bucket starts with **3 tokens**, and the current time is stored.
2. Every request calls `allowRequest()`, which first triggers a `refill()`.

3. `refill()` checks how much time passed since the last refill.
4. Based on that time, new tokens are added at **1 token per second**, but never above 3.
5. After refilling, the method checks whether the bucket has enough tokens for the request.
6. If enough tokens exist, they are deducted and the request is allowed.
7. If not enough tokens exist, the request is rejected and must wait for tokens to refill.

115. Microservice 111: What is the Bulkhead pattern, and how does it prevent system-wide failures?

The **Bulkhead pattern** is a simple but powerful idea used in software systems to prevent one failure from spreading and breaking everything. The concept comes from ships. In a ship, the lower part is divided into separate sections called bulkheads. If one section gets damaged and fills with water, the other sections are still safe, and the ship does not sink. The damage won't spread.

In the same way, we apply the bulkhead idea in microservices and backend systems. We divide the system into separate parts so that if one part becomes slow, overloaded or fails, the rest of the system continues to work.

Why do we need the Bulkhead pattern?

In real systems, we often depend on many external services like payment, authentication, notifications, search and so on. If all of them share the same resources such as thread pools, memory or connection pools, then one slow service can block everything. The entire system slows down and becomes unhealthy even though only one part had a problem. This is called a cascading failure.

The Bulkhead pattern stops this from happening.

How it works

We separate resources by feature or by dependency. Each part of the system gets its own safe space. For example, we can give each service its own thread pool or limit the number of calls a service can consume. So, if one part fails, its failure stays in that part only. The rest of the system remains healthy.

Real world example

Imagine we are building an online shopping application. When a customer visits the home page, three services are called

- Product Service to show product listings
- Recommendation Service to show suggested items
- Review Service to show product ratings

Now let us say the Review Service becomes very slow due to heavy traffic. If all three services share the same thread pool, the slow Review Service will occupy many threads and stop Product and Recommendation services from responding. The whole website will feel slow or even timeout.

If we use the Bulkhead pattern

- Product Service gets ten threads
- Recommendation Service gets five threads
- Review Service gets three threads

Now even if Review Service is slow, it only affects its three threads. Product pages still load fast and recommendations still work. The customer experience stays smooth.

Reference: [Bulkhead Pattern with Spring Boot | Vinsguru](#)

116. **Microservice 112: How to implement bulkhead pattern using Spring?**

Take example of E-commerce application. The Order Service needs to call three other services

- Payment Service
- Inventory Service
- Notification Service

Sometimes, the Payment Service becomes slow because of a bank issue. If we are not careful, this delay will block the Order Service and it will stop accepting new orders. This is bad for business.

With the **Bulkhead pattern**, we can protect our Order Service by giving each dependency its own separate space. If the Payment Service is slow, it will only affect its own space and not the whole system.

Tools we use

- **Spring Boot** for building the service
- **Resilience4j** for applying the bulkhead pattern
- **Thread pool isolation** to protect resources

Step 1: Add dependency

We need Resilience4j in our application. In Maven, we add:

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
</dependency>
```

OR

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot2</artifactId>
</dependency>
```

Step 2: Protect Payment Service calls

We add a **bulkhead** around the Payment Service call. This will limit how many threads are used for that call so it cannot block total system resources.

```
@Service
public class OrderService {

    @Bulkhead(name = "paymentServiceBulkhead", type = Bulkhead.Type.THREADPOOL)
    public String processPayment(String orderId) {
        return paymentClient.makePayment(orderId);
    }
}
```

The method now has a safety boundary. If the Payment Service becomes slow, it will not block all threads.

Step 3: Configure bulkhead limits

In application.yml, we configure limits for each service. Here we control how many simultaneous calls we allow.

```
resilience4j.bulkhead:
  instances:
    paymentServiceBulkhead:
      maxConcurrentCalls: 5
      maxWaitDuration: 2s
```

This means:

- Only 5 payment calls are allowed at the same time
- Others must wait a maximum of 2 seconds

If more calls come in, they will be rejected quickly instead of blocking everything. The Order Service stays responsive.

Step 4: Optional fallback for better user experience

If the Payment Service is overloaded, we can add a fallback.

```
@Bulkhead(name = "paymentServiceBulkhead", fallbackMethod = "paymentFallback")
public String processPayment(String orderId) {
    return paymentClient.makePayment(orderId);
}

public String paymentFallback(String orderId, Throwable t) {
    return "Payment service is busy, please try again in a moment";
}
```

This allows us to return a friendly response instead of an error.

Reference: [Bulkhead pattern in Microservices: Spring Boot example | by S Sivaraman | Medium](#)

117. **Microservice 113: What is difference between Rate-limiting and bulkhead pattern?**

What is Rate Limiting?

Rate limiting controls how many requests are allowed to enter a system in a given period of time. It is like a security guard at the entrance of a building who allows only a fixed number of people inside every minute. If too many people try to enter at once, new people are stopped at the gate.

Why do we use it?

To protect a service from too many incoming requests. It prevents abuse, sudden traffic spikes and denial of service situations.

Real world example

Imagine a ticket booking website during a cricket match sale.

One user keeps clicking the **Book Now** button very fast or refreshes the page again and again. This sends many requests in a short time. If this continues, it can overload the system.

With rate limiting for a single user, only a fixed number of requests from that user are allowed per second. When the user crosses that limit, further requests are blocked.

The user then sees a polite message like “Too many requests, please try again in a few seconds.”

Rate limiting protects the system by **controlling the entrance**.

What is the Bulkhead Pattern?

The Bulkhead pattern controls **how many resources one part of the system can consume**. It isolates parts of the system so a problem in one part does not break the others.

Why do we use it?

To stop a failure from spreading across services. It prevents one slow or failing dependency from blocking the whole system.

Real world example

In an online store, the Order Service calls three other services

- Payment Service
- Inventory Service
- Recommendation Service

If the Payment Service becomes slow, and there is no bulkhead protection, requests waiting for Payment can consume all available threads in the Order Service. This can block calls to Inventory and Recommendation as well, even though they are healthy.

With bulkhead protection, we give the Payment Service its own fixed number of threads or connections. If Payment becomes slow, only its allocated threads are affected. Calls to Inventory and Recommendation continue to work normally.

118. Microservice 114: How do you handle failures in Microservices?

When one service depends on another, things can go wrong. The network may fail, the service may be overloaded, or a bug may cause bad responses. To keep the system healthy, we use a few simple techniques.

1. Retry with Limits

Sometimes a service fails only for a moment. We try the request again after a small delay. But we retry only a few times so we don't overload the service.

Example: If the Payment Service times out once, the Order Service retries twice before giving up.

2. Timeout

We never wait forever for another service. If a service takes too long, we stop the request and handle it gracefully.

Example: If the Inventory Service doesn't reply in 2 seconds, we show a message like "Unable to check stock now."

3. Circuit Breaker

If a service keeps failing, we temporarily stop calling it.
This protects the system from making the situation worse.
Once the service recovers, the circuit closes and calls resume.

Example: If User Service fails 10 times in a row, Order Service pauses all calls for a short time.

4. Fallback Response

When the main service is down, we return a safe backup response.

Example: If Review Service is not responding, the Product Page can still load without reviews instead of showing an error.

5. Bulkhead Pattern

We separate services so one failing service doesn't pull down others.
It's like having separate rooms in a ship so a leak in one room doesn't sink the ship.

Example: High load on Chat Service should not affect Payment Service.

119. **Microservice 115:** Explain about security in microservices?

Note: In-depth security related questions along with scenarios and implementations (**30+ questions on Spring security**) are covered in Java & Spring knowledge boost

Link: <https://topmate.io/codinglyf/1746709>

Microservices communicate over networks, which increases security risk. Each service exposes APIs that can be attacked. Managing authentication across services, securing internal communication, handling service-to-service communication and avoiding token misuse become difficult as the number of services grows.

OAuth2 in Microservices

OAuth2 allows secure authorization between services. Instead of sharing passwords, a user gives permission using an access token. Microservices use this token to verify requests. For example, a user can log in through Google and access different services without sharing their login details.

JWT Authentication

JWT (JSON Web Token) is used for authentication in microservices. After login, a token is generated and sent to the client. The client includes this token in every request. Each service can validate the token and identify the user without checking a central database every time.

Role-Based Access Control

Role-Based Access Control (RBAC) ensures that users can only access what they are allowed to. Users are assigned roles like admin, user or manager. Each role has limits on what actions can be done. This avoids unauthorized access to sensitive parts of the system.

API Gateway Security

The API Gateway is the entry point for all clients, so it must validate tokens, limit requests, block invalid users and prevent attacks. It acts like a security guard that checks identity and permissions before allowing access to internal microservices.

Token Validation

In microservices, every request must be trusted. Token validation makes sure that the JWT or OAuth token is real, not expired and not changed. Each service verifies the token before serving the request. This protects the system from fake or stolen tokens.

SSL/TLS Usage

SSL/TLS secures communication between microservices by encrypting data. Without encryption, data can be read or modified during transfer. Using HTTPS ensures that passwords, tokens and sensitive data remain safe while moving between users and services.

120. **Microservice 116:** How do you manage the configurations in distributed systems?

Normally, Spring Boot reads its configuration from `application.properties` or `application.yml` files at startup. So, if we change them, we need to restart the app

To solve this, Spring provides ways to apply globally and refresh configurations without restarting.

To do this we need to follow these things:

1. Store configurations centrally in a separate Git repo.
2. Set up a Spring Cloud Config Server
3. Connect your main applications to Config Server
4. Enable actuator > refresh API

1. Store configurations centrally in a separate Git repo.

Create a Git repository (say *config-repo*). Inside it, place environment-specific property files:

```
application-dev.properties
application-test.properties
application-prod.properties
```

2. Set up a Spring Cloud Config Server

Create a new Spring Boot app with `@EnableConfigServer` and add

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-config-server</artifactId>
</dependency>
```

Point it to the Git repo (config-repo) in application.properties:

```
spring.cloud.config.server.git.uri=https://github.com/org/config-repo
```

```
import org.springframework.cloud.config.server.EnableConfigServer;

@SpringBootApplication
@EnableConfigServer
public class ConfigServer {
    public static void main(String[] args) {
        SpringApplication.run(ConfigServer.class, args);
    }
}
```

@EnableConfigServer. This annotation tells Spring Boot to enable the Config Server functionality for the application.

3. Connect your main applications (all the services) to Config Server

- Add *spring-cloud-starter-config* dependency
- To connect the application to config server, we need to give below property pointing to the config server URL.

```
spring.cloud.config.uri=http://config-server:8888
spring.profiles.active=prod
```

4. Enable actuator > refresh API:

- Add `spring-boot-starter-actuator` dependency.
- Annotate beans that should reload on config change with `@RefreshScope`. Example:

```
@RestController
@RefreshScope
public class ConfigController {
    @Value("${featureX.enabled}")
    private boolean featureEnabled;

    @GetMapping("/feature-status")
    public String featureStatus() {
        return "Feature X is " + (featureEnabled ? "ENABLED" : "DISABLED");
    }
}
```

```
}  
}
```

The **@RefreshScope** annotation in Spring Cloud is used to enable dynamic refreshing of configuration properties within a Spring Boot application without requiring a full application restart.

How it works?

- Suppose you want to disable the feature in prod.
- Edit application-prod.properties in config-repo branch: `featureX.enabled=false`
- Commit and push changes to Git.
- Go to main running app.
- Call the actuator refresh endpoint on your service (main app):
- POST `http://main-app:8080/actuator/refresh`
- Spring reloads properties from Config Server into your beans without reboot.

Reference: [Complete Guide to Spring Cloud Config | Medium](#)

8. Cache

121. **Microservice 117: What is Local-Cache and Distributed-Cache?**

Caching is one of the simplest ways to speed up a system. Instead of fetching the same data again and again from a slow source like a database, we keep a copy of it in a faster place.

Now the question is, where should this fast copy live? Inside the service itself or outside as a shared system?

That is where local cache and distributed cache come in.

Local Cache

A local cache lives inside the application instance. Each service keeps its own copy of the cached data in its memory.

For example, we run three instances of the same microservice. Each instance has its own small storage inside its memory.

Benefits

- Very fast, because the data is in the service memory
- Simple to set up with libraries like **Caffeine** or **Ehcache**

Limitations

- Each instance holds its own copy, so there is a chance that data may not match across instances
- Memory usage increases as instances grow
- Not suitable for shared or large data

Distributed Cache

A distributed cache lives outside the service. All service instances connect to the same cache system.

Redis and **Hazelcast** are popular choices.

Here the cache works more like a shared storage. Every instance reads and writes to the same place.

Benefits

- Data stays consistent across all service instances
- Can store large data without affecting service memory
- Easy to scale when traffic grows

Limitations

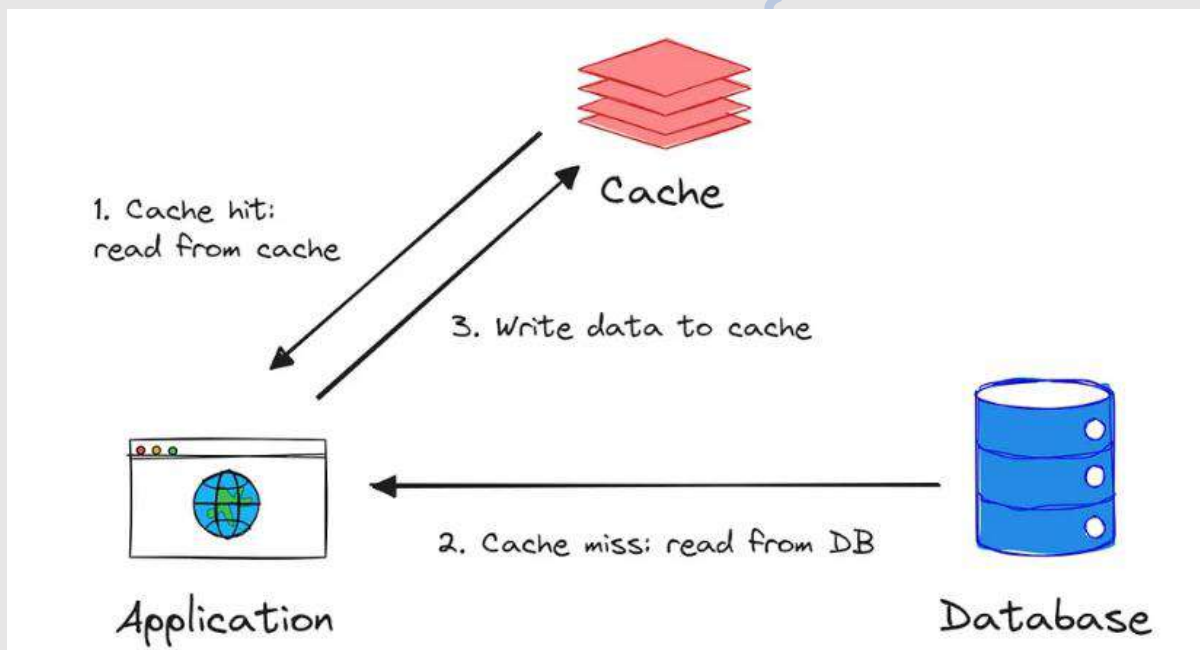
- Slightly slower than local cache because it needs a network call
- If the cache server goes down, the system may slow down

Example

When we store customer session data or user login tokens, every service instance must see the same value. A distributed cache like Redis fits this case well.

122. Microservice 118: What is In-memory caching and give any example?

In-memory caching is a way of storing data in the system's RAM instead of fetching it from a database or external source every time it's needed.



Since RAM is incredibly fast compared to disk storage, you can access cached data almost instantly. This approach is perfect for information that doesn't change very often, like API responses, user profiles, or rendered HTML pages.

Example: Redis

Redis is an open-source, in-memory data store that developers use to cache and manage data in real time.

Unlike traditional databases, Redis stores everything in memory, which makes data retrieval incredibly fast. But Redis isn't just a simple key-value store. It offers a wide range

Follow up:**So, what is *not* in-memory?**

Any cache or storage that keeps data on disk instead of RAM.

1. Traditional databases: MySQL, Postgres, Oracle These store data on disk. Reads and writes are slower.
2. File-based storage: Data stored directly as files
3. Disk-based caches: Some systems keep cache on disk when memory is not enough.

Example:

Ehcache has a mode where it stores extra data on disk. This avoids memory overflow but is slower than RAM.

In-memory cache:

Redis, Memcached, Caffeine, Hazelcast (in-memory mode)

Not in-memory (Disk cache):

MySQL, Postgres, disk-based Ehcache, file storage, object storage

123. Microservice 119: How do you achieve caching in distributed systems?

In a distributed system, we usually deal with a lot of requests, large databases, and many services talking to each other. Every service might need to fetch the same kind of data repeatedly, for example, product details, user profiles, or recent transactions.

Fetching the same data again and again from the database slows everything down. That's where caching helps.

Caching means storing frequently used data in memory so that the next time it's needed, we can get it quickly without going all the way to the database.

Why Caching Is Important

Without caching, every request goes straight to the database. As requests increase, the database becomes overloaded, responses slow down, and eventually, the system struggles to keep up.

With caching, we store recently or frequently used data in a faster storage (like memory). When the same request comes again, we serve it directly from cache

How We Implement It**1. Choose a caching tool**

The most popular tools are:

- **Redis and Hazelcast:** These are distributed caches. They run outside the application and can be shared by all service instances. If one instance writes something to Redis or Hazelcast, every other instance can read it
- **Ehcache and Caffeine:** These are in-memory caches inside the application JVM. They store data locally in the same process where the code is running. This makes them extremely fast, but the data stays only inside that instance

2. Store and Retrieve Data

When data is fetched from the database:

- First check if it's available in the cache.
- If found, return it immediately.
- If not, fetch it from the database, store it in the cache, and then return it.

3. Set Expiration (TTL)

Cached data should expire after some time to stay updated.

For example, we might cache course details for 10 minutes.

After that, the next request fetches fresh data from the database and updates the cache.

124. **Microservice 120: What are pros and cons of caching?**

Pros:

1. Speed: Caching makes your system faster by reducing data fetching time.
2. Reduced Server Load: Caching reduces the load on your database or primary servers.
3. User Experience: Quick response times lead to happier users!

Cons:

1. Stale Data: Cache data can become outdated, leading to data inconsistency.
2. System Complexity: Implementing caching adds an extra layer of complexity to system design.
3. Cache Invalidation: Determining when to refresh or clear cache can be challenging.

125. **Microservice 121: What is Cache Invalidation?**

Caching improves speed, but it comes with one tricky issue. When data changes in the main database, the cache must also reflect that change.

If the cache shows old data and the database shows new data, the system becomes unreliable.

This challenge is called the **cache invalidation problem**.

Problem 1: Stale Data

Stale data happens when the cache still holds old values even after the database has been updated.

Example

In an internet billing system, a customer changes their monthly plan from 50 Mbps to 100 Mbps.

The database gets updated, but the cache still holds the old plan value.

Now billing calculations become wrong because the service reads from cache instead of database.

Stale data is the most common cache issue.

Problem 2: Multiple Updates at the Same Time

When many requests update the same data, the cache can get confused about which value is the latest.

Example

A customer tries to change their plan at the same time customer care also updates the plan. If one update hits the database and another hits the cache, the values may not match.

Race conditions like this can break the flow in high-traffic systems.

Problem 3: Cache and Database Sync Timing

There is always a small delay between writing to the database and updating the cache. If the service reads the cache in this exact moment, it may read old data.

This is known as the “cache sync gap”.

Example

A payment is marked as completed in the database. Before the cache is updated, another service reads the old value from cache and thinks payment is still pending.

Problem 4: Cache Deletion Failure

Sometimes the cache fails to delete or update because of network issues or service crashes. The database is correct, but the cache still shows old data.

This is rare but dangerous because the system will silently behave wrong.

Problem 5: Distributed Cache Issues

When many service instances share a distributed cache like Redis, all instances must see the updated value at the same time.

If one instance updates and another instance read old data, problems start. This becomes harder when the system runs many replicas.

Follow-Up**How We Solve Cache Invalidation**

There are three common ways:

1. Cache Aside Pattern

Read from cache. If not present, fetch from database and fill cache. On update, first update the database, then remove the cache entry.

This avoids stale data in most cases.

2. Write Through

Every write goes to cache first and then to database. Good for strong consistency.

3. Time-to-Live (TTL)

Cache entries expire automatically after a fixed time. Even if something goes wrong, the cache becomes fresh again soon.

126. Microservice 122: Explain Cashe Aside, Write Through and Write-Behind?

Caching looks simple from the outside, but the real challenge begins when the data in the database changes.

The cache must stay updated or the whole system starts showing old values. To handle this safely, we rely on three patterns that shape how the cache and database work together.

Cache Aside

Cache Aside is the most common pattern. The service decides when to read from cache and when to go to the database.

When a request comes in, the service looks for the data in the cache. If the data is found, the service returns it immediately. If the data is not found, the service loads it from the database and then stores a copy in the cache. This way, the next request finds the data in memory and becomes faster.

For updates, the service first writes to the database. After that, it removes the old data from the cache. This forces the next read to pull fresh data from the database.

Example is a product catalog in an e-commerce site.

- Product names and prices do not change very often.
- When a user opens a page, the service checks the cache.
- If the product is missing, the service loads it from the database, places it into cache and continues.
- If the price is updated by an admin, the database gets the new price and the old cache entry is cleared.

Write Through

Write Through works differently. Every write goes into the cache first and the cache writes the same data into the database. Reads continue as usual, often coming directly from the cache.

Imagine updating a customer's profile.

- When the change comes in, the service writes the new value into the cache.
- The cache then writes the updated data into the database.
- Both stay in sync without gaps.

The only drawback is that writes can become slower because the data must pass through two layers.

Write Behind

Write Behind looks similar to Write Through, but with one important difference. The cache does not write the data to the database immediately. It collects changes for a short time and writes them in batches. This reduces database load and improves performance during heavy traffic.

Think of updating customer usage records every few seconds.

- If we update the database on every small change, it becomes slow.
- Write Behind allows the cache to buffer these updates and write them in a controlled way.

Write Behind is used only in systems where small delays are acceptable

127. Microservice 123: Where would you place caching and which is best (DB level, Service Level or API Gateway)?

Caching is one of the easiest ways to make a system fast. Instead of doing the same work again and again, we store the result once and reuse it.

The question is where to place this cache so it helps the most.

1. API Gateway
2. Service layer
3. Database layer

Each one has a purpose. Let's walk through them with simple examples.

1. Caching at the API Gateway

The gateway sits in front of all services. This is the first door every request hits.

When this helps

This works well when the data is the same for everyone and does not change often.

Example

- A list of top products rarely changes.
- If thousands of users request it every minute, the gateway can store the response once and give it back instantly.

2. Caching at the Service Layer

This sits inside the backend code. We control exactly which methods or operations should be cached.

When this helps

This is the best place when the service performs heavy work. Maybe it calls another service, maybe it runs a complex calculation, or maybe it hits the database many times.

Example

- A service loads product details and combines them with ratings.
- This work does not need to be repeated for every request.
- We store the result in Redis or Caffeine inside the service.

3. Caching at the Database Layer

This sits inside the database engine. The database keeps the hot queries in memory. Databases like MySQL, Postgres, MongoDB all have their own query caching or buffer caching. This helps return results quickly without touching disk.

When this helps

This is useful when read traffic is heavy and the same queries keep repeating.

Example

A query like “select product by id” runs thousands of times. The database keeps this result in its buffer. Later calls return much faster because the data is already in memory.

9. Spring Cloud

128. **Microservice 124:** What is Spring cloud and how it is useful in Microservices?

When we move from building a simple application to building a full microservice system, everything becomes harder. A microservice application is not just code. It needs **service discovery, load balancing, API gateway, centralized configuration, fault tolerance, logging, monitoring, and security**. Doing all this manually is painful and slows down development.

This is where **Spring Cloud** helps us.

Spring Cloud is a set of tools built on top of Spring Boot that helps us solve common problems in microservices. It does not force any rules. Instead, it gives solutions for real challenges like service communication, routing, failure handling, configuration, and deployments.

Without Spring Cloud, we would have to write many things by hand. With it, we focus more on business logic and less on infrastructure problems.

Why We Need Spring Cloud: Instagram Example

Let us imagine **Instagram with microservices**. We would have services like:

Service	Responsibility
user-service	manage profiles and followers
post-service	store posts
comment-service	store comments
like-service	handle likes
notification-service	send notifications
feed-service	show home timeline

These services must talk to each other and stay reliable even when traffic grows. Now imagine Instagram has millions of users online at the same time. That means the system needs to handle:

- service discovery (find service locations)
- load balancing (share traffic across instances)
- communication (REST/gRPC)
- API security and routing
- failure handling
- monitoring and logging
- centralized configuration

- scaling easily

Doing all this from scratch is nearly impossible.

How Spring Cloud Helps

Problem in Microservices	Spring Cloud Solution
Services must find each other	Eureka (Service Discovery)
Single entry point needed	Spring Cloud Gateway
Load balancing required	Spring Cloud LoadBalancer
Configuration changes should be centralized	Config Server
Service failures must be handled	Resilience4j
Communication between services	Feign Client
Tracing and logging	Sleuth + Zipkin
Distributed messaging	Spring Cloud Stream + Kafka/RabbitMQ

Example Flow: Feed Page in Instagram

When we open Instagram and refresh the feed:

1. The request goes to **API Gateway**.
2. API Gateway sends it to **feed-service**.
3. feed-service wants details of users and posts, so it calls:
 - **user-service** to get user info
 - **post-service** to get posts
4. These calls are routed through **Eureka service discovery**.
5. If post-service is slow, **Resilience4j** applies retry or fallback.
6. If load is high, **Spring Cloud LoadBalancer** balances traffic.
7. Configuration for all services is taken from **Config Server**.
8. Logs and tracing go to **Zipkin** for debugging.

Reference: [What Is Spring Cloud? A Beginner's Guide to Microservices Made Easy | by harsh_singh | Medium](#)

129. **Microservice 125: What do you know about feign-client?**

In a microservice system, services often need to talk to each other. One service may need data from another service to complete a task. Normally, we write HTTP calls manually using RestTemplate or WebClient, which takes more code and becomes hard to maintain as the project grows.

Example: In an ecommerce app. Order-service need to call the payment-service.

Feign Client solves this problem by giving a simple and clean way to call other services. Instead of writing HTTP code, headers, and request handling again and again, we only define an interface and Feign creates the HTTP call in the background. It makes service-to-service calls look like normal method calls in Java. This keeps the code readable and easy to follow.

Feign Client is a way to make communication easy between microservices in Spring Boot.

@FeignClient belongs to Spring Cloud, which provides tools for building applications that work well in cloud and microservice setups. Feign helps us call other services using simple Java interfaces instead of writing low-level HTTP code.

Feign also works well with service discovery tools like Eureka and load balancing tools like Spring Cloud LoadBalancer. It is widely used in Spring Boot microservices and is considered a standard way for internal service communication.

```
//Step 1: Enable Feign in main application
@EnableFeignClients
@SpringBootApplication
public class App { }

//Step 2: Create Feign Client to call another service
@FeignClient(name = "payment-service")
public interface PaymentClient {
    @GetMapping("/payments/{id}")
    Payment getPayment(@PathVariable String id);
}

//Step 3: Use Feign Client in service class
@Service
public class OrderService {
    @Autowired
    private PaymentClient paymentClient;

    public Payment fetchPayment(String id) {
        return paymentClient.getPayment(id);
    }
}
```

Step 1: Enable Feign

Feign is activated using @EnableFeignClients in the main Spring Boot class. This tells Spring to scan and load Feign clients in the project.

Step 2: Define a Feign Client

The @FeignClient interface declares a connection to another microservice. Instead of writing HTTP code, we define a method that matches the API of another service.

Step 3: Call another service using Feign

The Feign client is injected into a service class and used like a normal Java method. When the method is called, Feign sends an actual HTTP request in the background to another microservice.

Reference: [Mastering @FeignClient in Spring | Medium](#)

Reference with Eureka server and load balancer: [How To Implement Feign Client In Spring Boot Microservices? - JavaTechOnline](#)

Sample code from the above link: [Spring-Samples/FeignClient at main · CodingLyf-Fullstack/Spring-Samples](#)

In the sample:

- Producer is BOOK-SERVICE
- Consumer is STUDENT-SERVICE
- Consumer contains BookRestConsumer.java which calls producer using Feign.
@FeignClient(name="BOOK-SERVICE").
- Means we are calling BOOK-SERVICE from STUDENT-SERVICE using FeignClient

130. **Microservice 126:** How Feign-client work internally?

Feign Client is a way for microservices to talk to each other in Spring Boot without writing long HTTP code. It gives a clean method-call style for calling another service. Inside, it still uses HTTP, but it hides the complexity and gives a simple interface.

How Feign Client Works

When a method in Feign Client is called, it does not run directly. Instead, Spring creates a dynamic proxy behind that interface. The proxy builds the HTTP request and sends it to the target service. After getting the response, it converts it into a Java object and returns it to the caller.

Steps inside Feign:

1. We define an interface with @FeignClient.
2. Spring creates a proxy for that interface during startup.
3. When we call the method, the proxy sends an HTTP request to another service and returns the response.

131. **Microservice 127:** How does Feign Client know which service URL to call in a microservice system?

Feign Client does not store or hardcode service URLs inside it. Instead, it depends on the service discovery mechanism in a microservice system. When a Feign Client is created with a service name, that name is registered in a service registry like Eureka or Consul. During the call, Feign asks the service registry to find the current network location of the target service.

If multiple instances of the same service are running, Feign works with load balancers to choose one healthy instance. After the right instance is found, Feign builds the full URL and sends the HTTP request. This process happens automatically in the background.

132. Microservice 128: How Feign client works if Service registry is not used in the project?

Feign client can still be used even when there is no service registry like Eureka in a project. In this case, Feign does not discover services automatically. Instead, the target service URL must be given manually in the configuration. Feign will always call that fixed URL for every request.

This approach works well in small systems or local setups where services run on known ports. But it does not support automatic scaling or load balancing. If the service location changes, we must update the configuration manually. So, this method is simple but not flexible compared to using a service registry.

```
@FeignClient(  
    name = "payment-service",  
    url = "http://localhost:8082"    // Fixed URL since no Eureka  
)  
public interface PaymentClient {  
  
    @GetMapping("/payments/{id}")  
    Payment getPayment(@PathVariable String id);  
}
```

```
@Service  
public class OrderService {  
  
    @Autowired  
    private PaymentClient paymentClient;  
  
    public Payment fetchPayment(String id) {  
        return paymentClient.getPayment(id); // Feign calls fixed URL  
    }  
}
```

133. Microservice 129: How feign-client is different from Rest -Template and Web Client?

In a microservice system, one service often needs to call another service. In Spring Boot, this can be done using RestTemplate, WebClient, or Feign Client. All three solve the same problem, but they work in different ways.

RestTemplate is a simple way to make HTTP calls, but it needs more code for every request. We must build the URL, set headers, and handle responses manually. It is synchronous and blocks the thread until a response comes back.

```
@Service  
public class UserService {  
    @Autowired  
    private RestTemplate restTemplate;  
  
    public User getUser(String id) {  
        return restTemplate.getForObject("http://user-service/users/" + id, User.class);  
    }  
}
```

WebClient is a newer and non-blocking way to make HTTP calls. It supports asynchronous programming and can handle high traffic better. But the code becomes complex when handling multiple service calls or when combining responses.

```
@Service
public class UserService {
    private final WebClient webClient = WebClient.create();

    public Mono<User> getUser(String id) {
        return webClient.get()
            .uri("http://user-service/users/" + id)
            .retrieve()
            .bodyToMono(User.class);
    }
}
```

Feign Client is the simplest among the three. It hides the HTTP call and allows us to call another service just by writing an interface. It keeps the code clean and readable and fits well in microservices.

```
@FeignClient(name = "user-service")
public interface UserClient {
    @GetMapping("/users/{id}")
    User getUser(@PathVariable String id);
}

@Service
public class UserService {
    @Autowired
    private UserClient userClient;

    public User getUser(String id) {
        return userClient.getUser(id);
    }
}
```

Reference: [Feign Client vs Rest Client. A Comprehensive Guide | by Lahiru Rajapakshe | Medium](#)

134. **Microservice 130: When to use feign-client, Rest -Template and Web Client?**

RestTemplate: RestTemplate is a simple way to call REST APIs. It works in a blocking manner, which means it waits for the response before moving to the next step. It is easy to understand and good for small projects, but it needs more code to handle headers, errors, and response mapping.

Use RestTemplate when:

- The project is small
- The system has fewer API calls
- Performance is not a priority
- Simplicity is more important than flexibility

WebClient: WebClient is the modern way to call APIs in Spring Boot. It is asynchronous and non-blocking, which makes it faster and more efficient. It can handle multiple parallel requests easily.

Use WebClient when:

- The system needs high performance
- There are many APIS calls
- The service must handle more traffic

Feign Client: Feign Client is built for microservices. It keeps code clean by hiding the HTTP call logic. Instead of writing full request code, we define an interface and Feign takes care of the rest. It works well with load balancing and service discovery. It is the best choice for service-to-service communication.

Use Feign Client when:

- The system follows microservice architecture
- Services call each other frequently

135. **Microservice 131:** What are the advantages of Feign Client?

1. Declarative approach

Feign client follows a declarative style. We only write a Java interface and mark it with `@FeignClient`. We do not write HTTP code, connection logic or response handling. Spring takes care of calling the API and converting the response. This keeps the code clean and easy to understand.

Example idea

An order service calls product service using a simple method call instead of HTTP code.

2. Built-in load balancing

In microservices, many instances of the same service can run at the same time. Feign works with service discovery tools like Eureka. When a request is made, Feign automatically sends it to one of the available instances. This spreads the load and avoids sending all traffic to one server.

3. Integrated security

Feign works smoothly with Spring Security. Security tokens and headers can be passed automatically between services. This ensures that only allowed services can talk to each other. Communication stays protected without adding extra code in every service call.

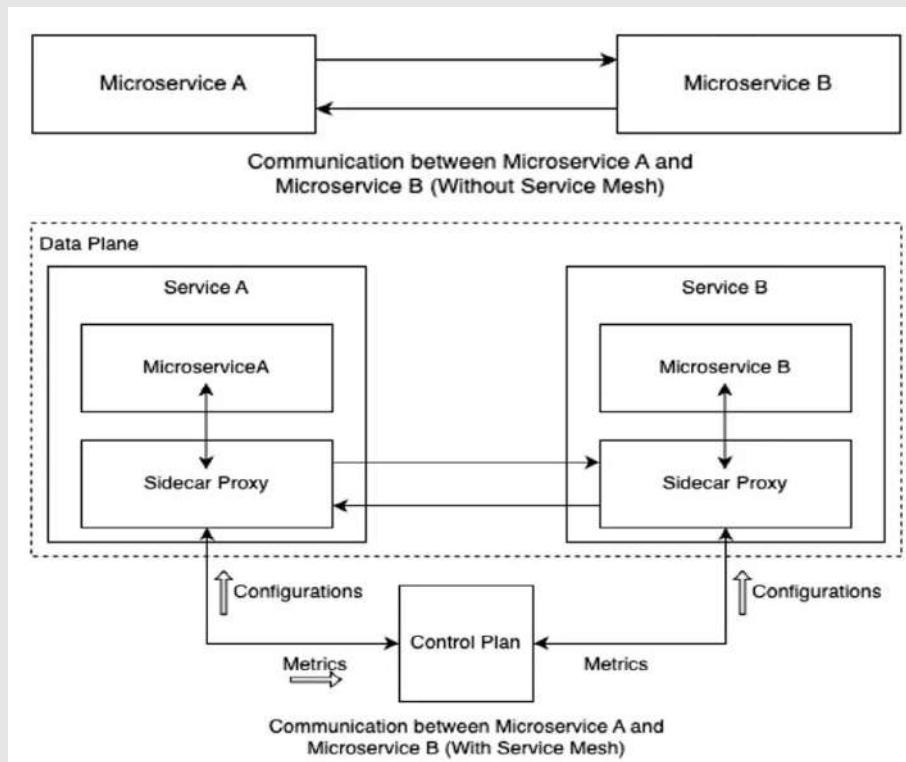
4. Fallback mechanism

Service failures can happen at any time. Feign allows us to define fallback logic. If a service is down or slow, a fallback method runs instead. This returns a safe response and keeps the system working instead of failing completely.

136. **Microservice 132:** What is service mesh and how it is useful in microservices??

A **service mesh** is a dedicated layer in a microservices system that manages how services communicate with each other. In microservices, many small services talk to each other over the network. Each service must handle security, retries, monitoring, timeouts and traffic control. Managing all this inside every service becomes difficult and leads to repeated code.

A service mesh solves this problem by moving these responsibilities out of the services and into a separate network layer. Each service keeps only business logic, while the mesh takes care of communication.



How it works

A service mesh uses **sidecar proxies**. These small proxy programs run next to each service. When one service talks to another, the traffic goes through these proxies. The proxies manage security, control network failures, collect logs and send metrics. This keeps the services clean and focused.

Example

Imagine a food delivery system with services like order service, payment service, delivery service and notification service. Each service communicates with others. If every service handles retries, timeouts and security by itself, the system becomes complex. Code is repeated everywhere.

With a service mesh like **Istio**, we add proxies beside each service. Now features like retry, logging and monitoring are handled by the mesh. The services only focus on what they are built to do.

Where it is used

Service mesh is used in large microservice architectures where there are many moving parts and communication must be managed centrally. It improves visibility, security and control without changing service code.

Reference: [Service Mesh for Microservices. Microservices architecture has been... | by Kasun Indrasiri | Microservices in Practice | Medium](#)

137. Microservice 133: How service mesh is different from API Gateway?

Both API Gateway and Service Mesh handle communication, security and monitoring in microservices, so they look similar. But they solve different problems and work in different parts of the system.

Let us understand the real difference in a simple and clear way.

Where they are used

Feature	API Gateway	Service Mesh
Works at	System entry point	Inside the system between services
Controls	Client to service traffic	Service to service traffic
Used for	External communication	Internal communication

Simply

- An API Gateway is like the main gate of a building. It handles everything coming from outside before entering the system.
- A Service Mesh is like the internal security system in the building. It manages internal traffic between rooms (services).

Example

Let us say we run an e-commerce app with these services:

- User Service
- Cart Service
- Order Service
- Payment Service

API Gateway

All users from the outside world enter through the gateway.

- It validates tokens
- Controls access
- Routes /orders to Order Service
- Protects system from high traffic

Service Mesh

Once inside, services talk to each other.

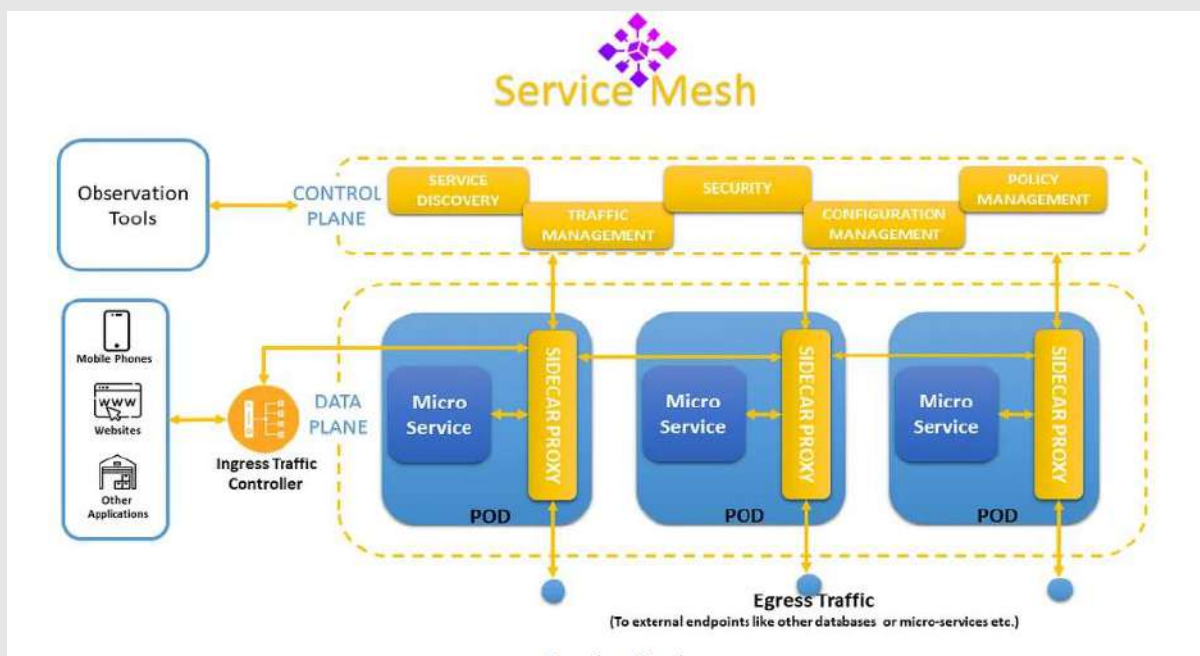
- Order Service calls Payment Service
- Payment Service calls Notification Service
- Mesh handles retries, timeouts and security between services

138. Microservice 134: Explain service mesh architecture?

In a service mesh, each microservice does not talk directly to another service. Instead, every service has a small helper program running next to it. This helper is called a **sidecar proxy**. All network communication goes through these proxies.

- When **Service A** wants to talk to **Service B**, it sends the request to its own proxy.
- The proxy forwards the request to the proxy of **Service B**.
- The proxy of **Service B** delivers the request to **Service B**.
- The same path is followed for the response.

This way, services do not need to worry about networking and security.

**Control Plane and Data Plane**

A service mesh has two main parts:

1. Data Plane

This is made up of all sidecar proxies. They handle the actual traffic between services. They perform encryption, retries, rate limits and collect metrics.

2. Control Plane

This manages and configures all the sidecar proxies. It tells them how to route traffic, when to retry and what security rules to follow.

Example

Imagine a food delivery app with services like Order Service, Payment Service and Delivery Service. Normally, Order Service talks directly to Payment Service. But if Payment Service is slow, Order Service must retry, log errors and handle failures.

With a service mesh, Order Service does not handle any of that. It sends the request to its sidecar proxy. The proxy handles retry if Payment Service is slow, logs the request and secures communication. Payment Service also has a proxy to receive the request safely.

Reference: [Understanding the Role of Service Mesh in Microservices Architecture | by ShivajiKant | Medium](#)

Microservices Scenarios

139. Scenario 1: Monolith to Microservice?

Question: You have given a big single application and your job is to convert into microservices, how would approach?

Solution:

We never break the system in one shot. That is risky and can create problems in production. Instead, we follow a step-by-step plan.

Step 1: Understand the Business Domains

First, we study the business and group related features together.

For example, let us say we have an e-commerce application. The monolith may have these modules:

- Product Management
- User Management
- Order Management
- Payment
- Inventory
- Delivery Tracking

All of these are mixed together in one codebase. This is a problem when the code grows bigger.

Step 2: Identify Services

We divide the application based on business areas. Each area becomes a separate microservice.

For our example, we can split like this:

Microservice	Responsibility
Product Service	Manage products, categories
User Service	Handle users, login, profile
Order Service	Create orders, order history

Microservice	Responsibility
Payment Service	Process payments
Inventory Service	Track stock
Delivery Service	Shipping and tracking

Each microservice will have its **own database** so that it stays independent and safe from other services.

Step 3: Communication

Now we must decide how these services talk to each other. They cannot share code directly anymore. They must speak through APIs.

- For simple calls, we can use **REST APIs** between services.
- For asynchronous events like order placed or payment success, we can use **message queues** like Kafka or RabbitMQ.

Example:

When the Order Service creates a new order, it sends a message to the Inventory Service to reduce stock.

Step 4: Create a Strangler Plan

We do not delete the monolith suddenly. We slowly replace parts of it. This safe method is called the **Strangler Pattern**. We take one module at a time, move it out as a microservice, and connect it back to the old system.

For example:

- First, move Product module out of the big app.
- Then connect it using API.
- Test it.
- Then move User module.
- Continue like this step by step.

Step 5: Build Deployment and Monitoring

Microservices bring complexity, so we must plan deployment and monitoring before going live.

- Container tools like Docker help run each service separately.
- Kubernetes can manage deployment and scaling.
- Monitoring tools like Prometheus or Grafana help check if services are healthy.
- Logs must be centralised to track errors.

Step 6: Testing and Migration

Before we go live, every service must be tested.

We write **unit tests**, **integration tests**, and **API tests**.

Also, we must migrate old data carefully into new service databases without losing anything.

140. Scenario 2: Monolith to Microservice: Billing system

Question: You have single Internet billing system application and you need to divide into invoice, Payment & Pricing Microservices

Solution:

The best way to bring back order is to divide the system based on the natural flow of internet billing. This flow always follows three steps:

1. Decide the price
2. Generate the bill
3. Collect the payment

These steps map perfectly to three microservices: **Invoice**, **Pricing**, and **Payment**.

1. Invoice Service

Internet billing is mostly monthly.

The invoice service is responsible for creating and tracking each month's bill.

What it handles

- Monthly invoice generation
- Billing cycles
- Due dates
- PDF or email bill
- Invoice status such as generated, pending, late, or paid

Example

It is the first of the month.

The invoice service picks all active internet connections.

For each connection, it calls the pricing service to get the current charge and generates the monthly invoice.

If the user upgraded speed mid-month, it calculates a partial charge as well.

2. Pricing Service

Internet plans usually come with different speeds, data limits, add-ons, and promos.

The pricing service handles all of this. It becomes the single place where billing asks for the cost of a plan.

What it handles

- Base rate of internet plans
- Speed-based pricing
- Promotions and discounts
- Taxes
- Final billable amount

Example

A user upgrades from a 50 Mbps plan to a 200 Mbps plan.
The invoice service calls pricing with the user's plan details.
Pricing calculates the new monthly rate, applies tax, and returns the amount.
This keeps the pricing logic in one clean place.

3. Payment Service

The payment service deals with the actual money movement. Users may pay through online banking, UPI, card, or auto-debit. This service manages all of it.

What it handles

- Payment requests
- Payment success and failure
- Refunds
- Transaction logs
- Informing the invoice service that payment is complete

Example

A user pays the bill through UPI.
The payment gateway sends a success callback.
The payment service updates the transaction record and informs the invoice service that the bill is paid.

4. Connecting the Services

Even though the system is now split, the services still need to work together.

How they talk

- The invoice service calls pricing to calculate the amount
- The payment service notifies invoice after a successful payment using message queues like kafka or RabbitMQ

141. Scenario 3: Authentication Token Refresh - Streaming App

Scenario: In a video streaming app like Netflix, every service call needs a valid authentication token. Tokens expire often.

Question: Will you add refresh token logic in every service; how would you implement it?

Solution:

Instead of writing token refresh logic in every service, a **sidecar service** handles:

- Checking token expiry
- Refreshing tokens from Auth Server
- Attaching tokens to outgoing API calls

Main service stays clean and focuses only on streaming logic.

Or We can use API Gateway to handle refresh token logic.

142. Scenario 4: Shared data between multiple services of Database

Scenario: How would you handle Shared Data in Microservices?

Solution:

In a microservices architecture, handling shared data between multiple services is tricky because each service should be loosely coupled and own its data. Here's how you can handle it:

1. Database per Microservice (Best Practice)

Each microservice should have its own database/schema. Prevents tight coupling and improves scalability.

Example: UserService manages users table, OrderService manages orders table.

2. Data Sharing via APIs (Service-to-Service Communication)

Services communicate using REST/gRPC/GraphQL. Instead of directly accessing another service's database, they call APIs.

Example: OrderService calls UserService API to fetch customer details.

3. Event-Driven Architecture (Asynchronous Sharing)

Use a message broker (Kafka, RabbitMQ, AWS SQS, etc.) to publish and consume events.

Example: When PaymentService processes a transaction, it publishes an event, then OrderService and NotificationService consume it.

4. Shared Data through Caching (Read-Heavy Use Cases)

For frequently accessed data, use distributed caches (Redis, Hazelcast). Reduces dependency on synchronous calls.

Example: User profile info cached for quick access by multiple services.

5. API Gateway for Aggregation

Instead of services directly calling each other, use an API Gateway to aggregate responses.

Example: Customer Dashboard request -> API Gateway -> fetches from UserService, OrderService, PaymentService -> merges response.

143. Scenario 5: Selection of Database

Scenario: You are working on a microservices application and you have to use either database per service or shared database for all the services.

Question. Which one you prefer and tell the reason?

Solution:

In a microservice system, it is better to follow the **database per service** pattern. Each service should have its own database so that it stays independent and safe from failures in other services. This also allows each service to choose the right database for its own need.

For example, a Product Service may use MongoDB for fast reads, while an Order Service may use MySQL for transactions. If we use a shared database, services become tightly connected, and a change made by one team can break others

The only challenge with separate databases is data consistency, but this can be solved using patterns like Saga and event-driven communication.

144. **Scenario 6: Book a Flight - Travel App**

In a travel booking app, we book a flight through multiple services:

1. **Flight Booking Service** reserves the flight ticket
2. **Payment Service** charges the customer
3. **Notification Service** sends confirmation

Now imagine this situation:

- Money is successfully deducted by Payment Service
- But Flight Booking Service fails while reserving the seat

Question: How would you solve this issue in real time?

Solution: This problem happens because each service runs separately. The payment succeeds, but the flight booking fails. So, money is lost and no ticket is confirmed.

We solve this using the Saga pattern. It is a simple step-by-step process that also supports undo steps when something fails.

When the user tries to book a flight, we start a booking workflow.

First, the Flight Booking Service tries to reserve a seat. If this is successful, then the Payment Service collects money.

After that, the Notification Service sends the booking message.

If any step fails, we run a compensation step.

For example, if the payment is done but later flight booking fails due to no seats, the Saga will undo the payment by calling the Payment Service to refund the amount. So, each step has a reverse step. This way, the system stays consistent even when one service fails. No data will be lost and the experience stays safe.

145. **Scenario 7: Bank Account Transfer Between Two Different Databases**

Imagine a banking system where we transfer money from Account A to Account B. But the catch is:

- Account A is in Database 1 (maybe in one branch system)

- Account B is in Database 2 (maybe another branch system)

Both databases are separate. Still, the money transfer must be 100% consistent. Money should not disappear or duplicate.

Problem

We perform a transfer of ₹10,000 from A to B.

Steps:

1. Debit 10,000 from Account A (DB1)
2. Credit 10,000 to Account B (DB2)

What if DB1 is updated but DB2 fails to update? Money will disappear. This is a **serious financial risk**.

Solution: Use 2 Phase Commit (2PC)

We use a Transaction Coordinator to manage this safely in two phases. 2PC is a protocol that ensures all participating systems in a distributed transaction either commit or roll back together, guaranteeing strong data consistency.

Phase	What Happens
Prepare Phase	Coordinator asks both databases if they are ready to commit
Commit Phase	If both say yes, then coordinator commits in both DBs, else rolls back

Additional Question: Why can't we use Saga pattern here?

Saga works well when **eventual consistency is acceptable**.

For example in e-commerce:

- If payment succeeds but inventory update fails, we can cancel order later
- A short delay is okay
- Business can survive temporary inconsistency

But bank money transfers are different they expect Strong consistency

Saga is good for long business workflows that can tolerate delay and compensation. But money transfer needs strong consistency, not eventual consistency. So 2PC is preferred over Saga for bank transfers.

146. Scenario 8: Multiple instances of a Service

Scenario: Multiple instances of a service is running and one instance got down while processing the DB transaction. How would you ensure it won't break the functionality?

Solution:

When many instances of a service are running, one of them can fail in the middle of a database operation. A good design should handle this without breaking the flow. The safest way is to rely on two things: database transactions and idempotent logic.

Use Distributed Transaction Patterns: A transaction makes sure the database commits everything together or cancels everything. So, if an instance goes down halfway, the database rolls back the incomplete work automatically. Nothing stays in a half-done state. Use Distributed Transaction Patterns like Two-Phase commit or Saga

The second part is **idempotency**. This means the same task can be retried safely without creating duplicates. For example, creating an order with the same request id should always produce one final order, even if the system retries it after a crash.

Another example, every transaction can carry a unique transaction ID; if retried, the service checks if it already processed that ID before executing again.

Retry using message queue: To make retries possible, we can store each task in a queue or a job table. When one instance fails, another instance sees the pending task and completes it cleanly.

This way, even if one instance stops during a transaction, the system recovers smoothly and continues without data corruption or repeated work.

147. Scenario 9: Communication between two services - 1

Imagine an e-commerce app, where it has two services order-service and payment-service. After user added items to add, from order-service we need to call payment-service for payment

Question: How would you achieve this communication between those two services?

Solution: We need to use Synchronous communication in this case or we can use webhooks for this..

When we buy something online and click Pay Now, the Order Service calls the Payment Service and waits for a response before proceeding. If payment is successful, the order gets confirmed. If payment fails, the order is cancelled. The system waits for the reply.

148. Scenario 10: Communication between two services - 2

Imagine an e-commerce app, now after the order is placed you need to send confirmation mail or message to user. For that we have a new service called notification-service.

Question: How would you achieve this communication between those order-service and notification-service?

Solution: We need to use Asynchronous communication in this case.

After placing the order, the system does not need to wait for email or SMS to be sent. The Order Service just sends a message to a message queue like Kafka or RabbitMQ. The Notification Service picks it up later and sends the email.

149. Scenario 11: Invoke API from Frontend

We have a **blog application** with three REST APIs inside the same backend project:

API	Purpose
/api/user/{id}	Get user profile
/api/posts/{userId}	Get all posts by user
/api/comments/{postId}	Get comments for a post

Now the frontend wants to load a User Profile Page that shows:

- User details
- User posts
- Comments for each post

With REST, the frontend must make multiple calls like this:

```
GET /api/user/1
GET /api/posts/1
GET /api/comments/11
GET /api/comments/12
```

Question: How would you implement this in better way?

Solution: Use GraphQL

Instead of calling multiple REST APIs, we add GraphQL on top of the existing REST APIs. Frontend now makes one GraphQL call:

```
query {
  user(id: "1") {
    name
    email
    posts {
      title
      comments {
        text
      }
    }
  }
}
```

GraphQL internally calls the existing controllers and combines the data.

150. Scenario 12: Real-Time Data Requirements

Scenario: Your application requires real-time updates, such as displaying new messages in a chat application. How would you implement this using REST and GraphQL?

Solution:

REST typically uses polling or long-polling to check for updates, which can be inefficient and resource-intensive.

GraphQL supports real-time data through **subscriptions**, allowing clients to receive updates over a WebSocket connection when data changes on the server.

151. Scenario 13: API Gateway

Scenario: If the frontend needs data from 5 different microservices, how will you reduce the number of API addresses from the frontend?

Solution: We can solve this using API Aggregator at the API Gateway. Instead of five API addresses from the frontend, the frontend will make only one request to the gateway. The gateway will call the required services internally, merge the data, and return a single combined response.

This keeps the frontend clean and improves performance.

Frontend -> API Gateway -> Calls 5 Services -> Returns one combined response

```
@RestController
public class DashboardController {

    @Autowired UserClient userClient;
    @Autowired OrderClient orderClient;
    @Autowired WalletClient walletClient;
    @Autowired CartClient cartClient;
    @Autowired OfferClient offerClient;

    @GetMapping("/dashboard/{userId}")
    public Dashboard getDashboard(@PathVariable String userId) {

        Dashboard dashboard = new Dashboard();
        dashboard.setUser(userClient.getUser(userId));
        dashboard.setOrders(orderClient.getOrders(userId));
        dashboard.setWallet(walletClient.getWallet(userId));
        dashboard.setCart(cartClient.getCart(userId));
        dashboard.setOffers(offerClient.getOffers(userId));

        return dashboard;
    }
}
```

Follow-up Question: Would you use API Gateway aggregation or GraphQL?

Use GraphQL when the frontend wants to choose what data to receive and reduce extra fields.

Use API Gateway aggregation when the backend must combine data and apply business logic before sending the response.

152. Scenario 14: Rate Limiting

Scenario: How will you implement a Rate limiting, different rate limits for free users and premium users?

Solution: Use API Gateway Filter + Bucket4j

Rate limiting is used to control how many requests a user can make in a given time in a real product, we do not want all users to have the same request limit. Free users can have limited access, but premium users should get better speed and more allowed requests. This is done at the API Gateway level before the request reaches any microservice.

To build this, we check the user type based on a token or user role. Then we apply different request limits. For example, free users may get 10 requests per minute, but premium users can get 100 requests per minute. If the limit is crossed, the gateway blocks extra requests for that period.

Sample code:

```
@Component
public class RateLimitFilter extends
AbstractGatewayFilterFactory<RateLimitFilter.Config> {

    private final Map<String, Bucket> buckets = new ConcurrentHashMap<>();

    @Override
    public GatewayFilter apply(Config config) {
        return (exchange, chain) -> {
            String userType = exchange.getRequest().getHeaders().getFirst("X-User-
Type");

            Bucket bucket = buckets.computeIfAbsent(userType, k ->
createBucket(userType));

            if (bucket.tryConsume(1)) {
                return chain.filter(exchange);
            }
            exchange.getResponse().setStatusCode(HttpStatus.TOO_MANY_REQUESTS);
            return exchange.getResponse().setComplete();
        };
    }

    private Bucket createBucket(String userType) {
        int limit = userType.equals("PREMIUM") ? 100 : 10;
        return Bucket4j.builder()
            .addLimit(Bandwidth.simple(limit, Duration.ofMinutes(1)))
            .build();
    }

    public static class Config {}
}
```

- The class extends AbstractGatewayFilterFactory, so it runs as a filter inside API Gateway for every request.
- It reads the X-User-Type header to check whether the user is FREE or PREMIUM.
- It creates a separate Bucket4j rate limit bucket for each user type using createBucket().
- If the bucket allows a request (tryConsume(1)), the request is forwarded.
- If the request limit is crossed, it returns HTTP 429 (Too Many Requests) and blocks the call.

Reference links: [Getting Started with Spring Cloud Gateway — Part Two | by Jayaramanan Kumar | Medium](#)

153. Scenario 15: API Aggregation or Composition

A user API waits on different services. Response time is slow. What patterns reduce client-side latency?

Scenario: A digital banking application wants to show a personalized dashboard when a customer logs in. The dashboard must show:

- Account balance from Account Service
- Recent transactions from Transaction Service
- Reward points from Reward Service

The key point: All this data must be returned together in a single API call for performance reasons, and some data depends on combining results from multiple services.

Solution: API Composition using API Gateway. The gateway aggregates data from multiple backend services parallelly and returns a single response, reducing client calls and improving performance.

Below is a simple and clean example of API Composition using Spring Boot API Gateway. The API Gateway calls 3 backend services and builds one combined dashboard response.

Step1: pom.xml dependencies (Spring Cloud Gateway + Feign)

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```

Step 2: Enable Feign Client

```
@SpringBootApplication
@EnableFeignClients
public class ApiGatewayApplication {
    public static void main(String[] args) {
        SpringApplication.run(ApiGatewayApplication.class, args);
    }
}
```

Step 3: Feign clients to Microservices

```
@FeignClient(name = "account-service")
public interface AccountClient {
    @GetMapping("/accounts/{customerId}")
    Account getAccount(@PathVariable String customerId);
}

@FeignClient(name = "transaction-service")
public interface TransactionClient {
    @GetMapping("/transactions/{customerId}")
    List<Transaction> getTransactions(@PathVariable String customerId);
}
```



```

}

@FeignClient(name = "reward-service")
public interface RewardClient {
    @GetMapping("/rewards/{customerId}")
    Reward getRewards(@PathVariable String customerId);
}

```

Step 4: Response DTO

```

@Data
@AllArgsConstructor
public class DashboardSummary {
    private String customerId;
    private double balance;
    private List<Transaction> recentTransactions;
    private int rewardPoints;
}

```

Step 5: API composition

```

@RestController
@RequiredArgsConstructor
public class DashboardController {

    private final AccountClient accountClient;
    private final TransactionClient transactionClient;
    private final RewardClient rewardClient;

    @GetMapping("/dashboard/{customerId}")
    public DashboardSummary getDashboard(@PathVariable String customerId) {

        Account account = accountClient.getAccount(customerId);
        List<Transaction> transactions = transactionClient.getTransactions(customerId);
        Reward reward = rewardClient.getRewards(customerId);

        return new DashboardSummary(
            customerId,
            account.getBalance(),
            transactions,
            reward.getPoints()
        );
    }
}

```

Request: GET <http://localhost:8080/dashboard/101>

Response

```

{
  "customerId": "101",
  "balance": 54000.0,
  "recentTransactions": [
    {"amount": -500, "type": "DEBIT"},
    {"amount": 2000, "type": "CREDIT"}
  ],
  "rewardPoints": 120
}

```

}

Why other options fail:

Approach	Why it fails
Frontend multiple REST calls	Too many calls, slow, bad for mobile
One backend REST API	Becomes a fat monolith
GraphQL	Frontend-driven, but cannot handle backend business logic combinations needed here

154. Scenario 16: Traffic routing

Scenario: How do we route traffic to multiple instances of the same microservice using API Gateway?

Solution: Use API Gateway Filter + Load balancing

In a microservice system, we often run more than one instance of the same service. This is done to handle more traffic and keep the application available even if one instance fails. But once we have multiple instances, we need a way to route incoming requests across them. This is done through load balancing at the API Gateway level.

```
spring:
  cloud:
    gateway:
      routes:
        - id: user-service
          uri: lb://USER-SERVICE # Load balancing
          predicates:
            - Path=/users/**
```

In this setup, if three instances of USER-SERVICE are running, the API Gateway will automatically balance the traffic between them using round-robin load balancing. No extra code is needed.

Follow-up: Will you use service discovery here?

Yes, service discovery is required here. It helps the API Gateway find all running instances of a microservice. Without service discovery, the gateway cannot distribute traffic dynamically.

155. Scenario 17: Secure the microservices

Scenario: How do we route traffic to multiple instances of the same microservice using API Gateway?

Solution: Use API Gateway + Spring Security

In a microservice system, services should not be exposed directly to the outside world. Only the API Gateway should be visible to the public. All other internal services must stay protected and allow calls only from trusted sources. The gateway works like a security guard at the main gate, checking every request before sending it inside.

To secure internal communication, we use authentication and authorization at the API Gateway level. Every request must include a valid token, such as a JWT token. The gateway verifies this token before forwarding the call to any microservice. After that, services trust only the gateway and reject any direct request from outside. This makes the system safe from bypass attacks and keeps internal APIs protected from illegal access.

Follow-up: Will you expose microservices directly to the public?

No, microservices should not be exposed directly to the public. They must stay behind the API Gateway for security and control. Only the gateway should be publicly accessible, and it should handle authentication, authorization, and request filtering before forwarding requests to internal services. This prevents direct attacks on microservices.

156. Scenario 18: Routing

Scenario: Based on user role in JWT, requests should go to different service clusters. Example: Free users to basic cluster, paid users to faster cluster. Can we do this at API Gateway?

Solution:

Yes, we can do this using API Gateway.

Every request that reaches the gateway carries a JWT token for authentication. Inside this token, we have user details like id, email, and role. The API Gateway can read this token before sending the request to any service.

Based on the role in the token:

- If role is FREE, send the request to the basic service cluster
- If role is PAID, send the request to the premium cluster

This way, the gateway controls how traffic flows without changing anything in the backend code.

157. Scenario 19: Handling errors

Scenario: Imagine an e-commerce system where three microservices are involved when a customer places an order:

- **Order Service (A)**
- **Payment Service (B)**
- **Inventory Service (C)**

The flow is simple:

Order > Payment > Inventory

Now let's see what happens if Inventory Service (C) throws an error.

1. Order Service (A) calls Payment Service (B)

A customer tries to buy a product. Order Service starts the flow by asking Payment Service to process the payment.

2. Payment Service (B) calls Inventory Service (C)

Once the payment is done, Payment Service asks Inventory Service to reduce the stock for that product.

Real-world situation:

Let's say Inventory Service finds that the product is already out of stock. It throws an error like: **"Stock not available"**

3. Inventory Service (C) throws exception

This error should not be hidden. It should travel back up the chain so the system behaves correctly.

4. How Payment Service (B) handles the exception

Payment Service receives error from Inventory Service.

It should:

- catch the error
- add a meaningful message
- send back a clear response to Order Service

Example:

Instead of sending a raw 404 error, Payment Service returns: **"Stock not available in Inventory Service"**. This makes debugging and logging easier.

5. Payment Service passes the error to Order Service (A)

Order Service receives the final clear error message.

It can now:

- cancel the order
- refund the payment if necessary
- show a simple message to the customer like **"Item is out of stock"**

The user sees a clean message even though multiple services were involved.

158. Scenario 20: Event Driven System

Scenario: A microservice needs to perform a long running task like generating the report, How would you implement it?

Solution

1. Asynchronous Processing with Message Queues

Use a message broker like Kafka, RabbitMQ, or AWS SQS. The main service sends the task request to a queue, and a background worker service processes it asynchronously. The client immediately receives an acknowledgment instead of waiting for completion.

2. Event-Driven Architecture

Publish an event (e.g., "GenerateReportRequested") and let another service subscribe to that event. This

ensures loose coupling and allows independent scaling.

4. Polling or Webhooks for Result Delivery

The client can poll a status endpoint to check progress or use webhooks to notify clients when the task completes. Webhooks are preferred.

159. **Scenario 21: Synchronous API Processing**

Scenario: Imagine a system like Netflix, every time a user does something, like play a video, pause, search, browse a genre, or give a thumbs up. How it can be handled over millions of users?

Problem: Millions of users generate events every second. Netflix cannot use a traditional request-response database model for this.

Solution

- In a real-time event streaming system like Netflix, Kafka is used to process millions of user interaction events per second, such as play, pause, and search.
- These events are published to Kafka topics and consumed by multiple microservices for real-time recommendations, analytics, and playback monitoring.
- Kafka is preferred over RabbitMQ because it supports high throughput, event retention, replay, and scalable consumer groups suitable for event-driven architecture

Kafka Brokers

- These are the servers that store and manage events happening on Netflix.
- Example: You press Play on a movie. That action is stored safely by the brokers.

Kafka Topics

- Topics are simple buckets that group similar types of events.
- Example:
Play events are stored in a topic called play_events
Pause events are stored in pause_events
Like events are stored in like_events

Producers

- A producer sends events into Kafka. In Netflix, the app behaves like a producer.
- Example: When you tap Pause, the Netflix app sends that event to Kafka.

Consumers

- These are services that read events from Kafka and take action.
- Example:
The watch history service adds the show to Continue Watching
The recommendation service updates Top Picks for You

Partitions

- Each topic is divided into smaller parts called partitions which helps Kafka process many events at the same time.
- Example: Even if millions of people press Play at the same moment, Kafka handles everything without delay.

Durability and Replay

- Kafka keeps events stored and safe. If any service goes down, it can replay the missed events later.
- Example: Your watch history is never lost even if a service crashes. It can replay your actions and recover.

Summary

Kafka helps Netflix track and stream user actions in real time. It keeps data safe, scales easily and allows different services to react instantly.

160. Scenario 22: Real-time Event Streaming

Scenario: Imagine a system like Netflix, every time a user does something, like play a video, pause, search, browse a genre, or give a thumbs up. How it can be handled over millions of users?

Problem: Millions of users generate events every second. Netflix cannot use a traditional request-response database model for this.

Solution

- In a real-time event streaming system like Netflix, Kafka is used to process millions of user interaction events per second, such as play, pause, and search.
- These events are published to Kafka topics and consumed by multiple microservices for real-time recommendations, analytics, and playback monitoring.
- Kafka is preferred over RabbitMQ because it supports high throughput, event retention, replay, and scalable consumer groups suitable for event-driven architecture

Kafka Brokers

- These are the servers that store and manage events happening on Netflix.
- Example: You press Play on a movie. That action is stored safely by the brokers.

Kafka Topics

- Topics are simple buckets that group similar types of events.
- Example:
Play events are stored in a topic called `play_events`
Pause events are stored in `pause_events`
Like events are stored in `like_events`

Producers

- A producer sends events into Kafka. In Netflix, the app behaves like a producer.
- Example: When you tap Pause, the Netflix app sends that event to Kafka.

Consumers

- These are services that read events from Kafka and take action.
- Example:
The watch history service adds the show to Continue Watching
The recommendation service updates Top Picks for You

Partitions

- Each topic is divided into smaller parts called partitions which helps Kafka process many events at the same time.
- Example: Even if millions of people press Play at the same moment, Kafka handles everything without delay.

Durability and Replay

- Kafka keeps events stored and safe. If any service goes down, it can replay the missed events later.
- Example: Your watch history is never lost even if a service crashes. It can replay your actions and recover.

Summary

Kafka helps Netflix track and stream user actions in real time. It keeps data safe, scales easily and allows different services to react instantly.

161. Scenario 23: Real-time Analytics vs Task Queue

Scenario: You need to track user clicks, page views, and login activity from a web app and run real-time analytics on it. Which one would you choose, Kafka or RabbitMQ, and why?

Solution

- Kafka is better for event streaming, high throughput, replay ability, long-term retention, publish-subscribe model.
- RabbitMQ is not ideal because logs must be retained for analytics and reprocessing later.
- Kafka supports consumer groups well and scales better for analytics workloads.

162. Scenario 24: Order Processing in E-Commerce

Scenario: You are designing order processing in an e-commerce platform. When a user places an order, the system should trigger email notification. Which one would you choose, Kafka or RabbitMQ, and why?

Solution

RabbitMQ fits better for transactional flows that need acknowledgement, priority queues, and retries.

Kafka is event streaming oriented; not ideal when message ordering and acknowledgment per message is critical.

So RabbitMQ is better here for order pipelines if real-time analytics is not needed.

163. Scenario 25: Microservices Communication

Scenario: You have 25+ microservices that need to communicate using domain events like USER_CREATED, ORDER_PLACED, PAYMENT_SUCCESS. Would you recommend Kafka or RabbitMQ??

Solution

- Kafka is better for event-driven microservices.
- Services subscribe to domain events decoupled.
- Kafka handles replay, scaling, audit logs.
- RabbitMQ is more point-to-point

164. Scenario 26: Payment Retry System

Scenario: In a payment processing system, failed payments must be retried 5 times, then moved to a Dead Letter Queue. Messages must be processed exactly once. Kafka or RabbitMQ?

Solution

- RabbitMQ is better due to Dead Letter Queue and retry policies out of the box.
- Kafka can do it, but needs custom implementation.
- Better choice: RabbitMQ for message orchestration + Kafka only if event history is required.

What is Dead Letter Queue?

A dead-letter queue (DLQ) is a special type of message queue that stores messages that could not be processed or delivered to their intended recipient. It acts as a "catch-all" for messages that fail due to errors, like incorrect formatting, network issues, or system failures, and is used to isolate these problematic messages for later analysis and handling without stopping the main message flow

165. Scenario 27: Real-Time Fraud Detection

Scenario: A banking app wants to process transaction streams in real-time to detect fraud patterns using ML models. System must support millions of messages per second. Kafka or RabbitMQ? and why?

Solution

- Kafka is the right choice.
- Supports high throughput + partitioning + streaming.
- RabbitMQ can't handle massive streaming workloads.

166. Scenario 28: Deploying 50 Microservices - Docker vs Kubernetes

Scenario: Your team has 50 microservices, each deployed separately using Docker on virtual machines. Managing deployments, scaling and service discovery is becoming hard. What will you use and why?

Solution

- In this situation, Kubernetes becomes the natural choice. Docker alone only solves the problem of packaging and running containers. But we also need automation for deployment, scaling, service discovery, and high availability.
- Kubernetes gives us exactly that. It can restart crashed containers automatically, distribute traffic across services, and scale up during heavy load.
- With Kubernetes Deployments, we can push new versions without downtime.
- With Kubernetes Services, we get built-in service discovery. So, for large microservice deployments, Kubernetes is preferred over plain Docker.

167. Scenario 29: Zero Downtime Deployment

Scenario: You need to deploy a new version of an order service without downtime. Which deployment strategy will you choose and why? Blue-Green or Rolling Updates?

Ex: Deploy a new version of a payments API

Solution

- Blue-Green runs two environments side by side, old and new.
- Traffic instantly switches only after testing, so rollback is easy.
- Rolling update is safer for simple deployments but slower to recover from production bugs.
- For high-risk services like payments, **Blue-Green deployment is safer**.

If fast rollback is important, **Blue-Green deployment** is a safe choice. In this strategy, two versions of the application run at the same time. The current version (Blue) serves traffic, while the new version (Green) waits. After testing, traffic is switched to Green instantly. If something goes wrong, you simply switch back to Blue. The downside is that it uses extra infrastructure temporarily.

168. Scenario 30: Managing Dev, QA, Stage, and Production Environments

Scenario: You need to manage deployments for Dev, QA, Stage, and Prod for 25 microservices. How would you design the deployment strategy?

Solution

The best approach is to use CI/CD pipelines and Kubernetes namespaces.

Each environment gets its own namespace inside the Kubernetes cluster.

Configuration values like database URLs or API keys are stored separately using ConfigMaps and Secrets.

Tools like Helm make it easy to manage environment-specific change

169. Scenario 31: Handling Traffic Spikes

Scenario: Your microservice system gets 10X traffic during festive sales. How would you handle deployment to scale quickly??

Solution

In Kubernetes, scaling is handled automatically using **Horizontal Pod Autoscaling (HPA)**. When CPU or memory usage increases, HPA adds more pods. If load reduces, it scales back down to save cost.

For very high traffic, we can also enable **Cluster Autoscaling**, so that even new nodes are added automatically. This makes sure the system stays healthy under stress.

Combined with **rolling deployments**, new versions can be released even during high traffic without any service interruption.

Reference links for more info:

[Kubernetes: The Magic of Automated Horizontal Scaling | by Akshay Kumar | Medium](#)

[Kubernetes Horizontal vs Vertical scaling - GeeksforGeeks](#)

170. Scenario 32: Ensuring Backward Compatibility when a breaking changes in a Microservice:

Scenario: Your microservice new deployment introduced some issues, how do you ensure backward compatibility

Solution

API Versioning: Simple way to handle this is to introduce changes slowly and keep both versions running for a while. For example, if we add a new field or change how an API responds, we keep the old response format active until every other service is ready to move to the new one. This is often done by versioning the API. One endpoint serves the older format, and another endpoint serves the new format.

Graceful fallbacks are another good technique. If a client still calls an endpoint, we return a safe default or convert the request to the new format inside the service. This avoids breaking older clients even when they haven't upgraded yet.

Clear communication also matters. When a change is coming, we share release notes, examples, and migration steps with all consumers. This gives teams enough time to adjust.

171. **Scenario 33: Managing Dev, QA, Stage, and Production Environments**

Scenario: You need to manage deployments for Dev, QA, Stage, and Prod for 25 microservices. How would you design the deployment strategy?

Solution

The best approach is to use CI/CD pipelines and Kubernetes namespaces.

Each environment gets its own namespace inside the Kubernetes cluster.

Configuration values like database URLs or API keys are stored separately using ConfigMaps and Secrets.

Tools like Helm make it easy to manage environment-specific change

172. **Scenario 34: Choosing deployment strategy**

Scenario: Your team is releasing a new version of the Recommendation Service in production. The code change is risky because it includes a new ML model. The team wants zero downtime but quick rollback if something goes wrong. Which deployment strategy will you choose: Blue-Green, Rolling, or Canary?

Explanation

- **Rolling Deployment** replaces old servers/pods gradually with new ones but treats all users the same. It is safe for normal releases but not ideal for risky changes.
- **Blue-Green Deployment** allows quick rollback but still shifts 100 percent of traffic to the new version at once. Risk is high if a hidden bug exists.
- **Canary Deployment** sends the new version to a small percentage of users first. If metrics look good, traffic is increased step by step. If something fails, rollback is instant with minimal impact.
- For risky releases, **Canary Deployment is the best choice** because it allows controlled rollout and protects user experience.

173. **Scenario 35: Service Failure**

Scenario: A microservice called Order Service depends on Payment Service. During high traffic, Payment Service becomes slow and starts timing out. As a result, Order Service also hangs and users can't place new orders. How will you prevent one failed service from affecting others?

Explanation

1. **This is a classic cascading failure scenario.** When a downstream service becomes slow, it blocks upstream services. To prevent this, we use a **Circuit Breaker** pattern. If Payment Service is failing repeatedly, the circuit breaker stops making new calls to it for a short time. During this time, Order Service can respond faster with a fallback like "Payment temporarily unavailable, try again."

Libraries like **Resilience4j** or **Hystrix** implement this easily.

2. Retry Mechanism with Exponential Backoff

Retries failed requests after a delay. The delay increases with each failure (exponential backoff). Useful for handling temporary issues like network glitches.

3. Fallback Mechanism

Provides a default response or alternative flow when a service fails. Keeps the user experience smooth by returning cached data or a friendly message.

4. Bulkhead pattern

The bulkhead pattern is a resilience design pattern in microservices that isolates components and resources to prevent a failure in one from cascading and taking down the entire system

For ex: let's assume that there are 2 Order Service and Payment Service. Order Service depends on Payment Service. For some reason, Payment Service is very slow. So, when we get multiple concurrent requests to Order Service, Order Service performance will also get affected. It could block Order Service's threads. Due to that Order Service might not be able to serve other requests which do NOT depend on Payment Service. So, the idea here is to isolate resources / allocate some threads in Order Service for Payment. So that we do not consume all the threads of Order Service and prevent Order service from hanging for all the requests!

Reference for bulkhead pattern: [Resilient Microservice Design – Bulkhead Pattern](#)

174. Scenario 36: Retry without causing overload

Scenario: Your Notification Service sends emails and SMS using a third-party provider. Sometimes the provider fails temporarily due to network issues. You try retries, but retrying too quickly causes overload. How will you retry safely?

Explanation

Blind retries can make things worse during an outage. Instead, use **Retry with Exponential Backoff**. That means first retry after 1 second, then 2 seconds, then 4 seconds, and so on

175. Scenario 37: Fallback on failure

Scenario: A travel Booking app collects hotel reviews from an external Review Service. When that service goes down or becomes slow, the entire booking page fails. The business team says even if reviews are not available, the page must load. How will you handle this failure??

Solution

Option 1: Fallback with Cached Response

If the external service fails:

- Try reading cached reviews from Redis or in-memory cache.
- If even cache is empty, show a friendly message:
"Reviews are temporarily unavailable. Please try again later."

This avoids a broken page and keeps the user experience smooth.

Option 2: Use Resilience4j Fallback (Java Example)

```
public String getHotelReviews(String hotelId) {  
    return Try.of(() -> reviewClient.getReviews(hotelId))  
        .recover(ex -> getCachedReviews(hotelId))  
        .get();  
}  
  
private String getCachedReviews(String hotelId) {  
    return redisClient.get("reviews:" + hotelId); // fallback cache  
}
```

This code:

- First tries the real Review Service.
- If it fails due to timeout or exception, it automatically calls fallback logic.

176. Scenario 38: Limit the resources

Scenario: Your app has three features: search, Wishlist, and user profile. During a sale event, the search feature got overloaded, and because everything shared the same thread pool, the whole system slowed down. How will you isolate failures?

Explanation:

Use the **Bulkhead Pattern**. Just like compartments in a ship prevent it from sinking, bulkheads isolate failures. Assign separate resource limits to each feature. So, if search goes down, Wishlist and profile still work.

177. Scenario 39: Dynamic IPs in microservices

Scenario: In your microservices system, services are running in Docker containers and Kubernetes pods. Their IP addresses keep changing every time they restart or scale. Hardcoding IPs in configuration files has become a maintenance nightmare. How do we make services discover each other dynamically?

Explanation:

This is exactly why a **Service Registry** is used. Services must not depend on fixed IP addresses. Instead, each service registers itself with a central directory called a service registry. Other services ask this registry to discover where to send requests.

For example, if we use **Eureka**, the Order Service registers itself as ORDER-SERVICE. The Payment Service does not need its IP. It simply asks Eureka for ORDER-SERVICE and Eureka returns the active instances. This solves the problem of dynamic service locations and avoids manual configuration changes.

178. Scenario 40: Load Balancing without an external Load Balancer

Question:

We have multiple instances of the Product Service running. The challenge is, how do other services know which Product Service instance to call? Do we always need an external load balancer?

Explanation:

No, we do not need an external load balancer if we use a service registry. When services register themselves with Eureka or Consul, the registry keeps track of all instances. A client can then use client-side load balancing.

For example, Spring Cloud LoadBalancer with Eureka can pick a Product Service instance automatically before sending a request. This improves performance and removes the dependency on traditional load balancers like NGINX for internal service communication.

179. Scenario 41: Handling service failures using service registry**Question:**

What happens if one instance of a service crash? How do other services avoid calling a dead instance?

Explanation:

A service registry performs health checks and maintains a list of only healthy service instances.

If one instance fails, it is automatically removed from the registry. Other services will not call it because they do not see it anymore.

For example, in Eureka, services send heartbeat signals to the registry. If no heartbeat is received for a fixed time, Eureka marks the instance as unavailable. This makes the system fault tolerant.

180. Scenario 42: Handling usage spikes

Scenario: Your e-commerce website receives normal traffic most of the year, but during festivals and flash sales, traffic increases by 10 times. Single server deployment is failing even after increasing CPU and RAM. How do we handle this situation?

Explanation:

- Horizontal scaling is a better solution than vertical scaling in this case. Instead of using one powerful server, we add more servers and distribute traffic across them using a load balancer like Nginx or AWS Elastic Load Balancer (ELB).
- The load balancer acts like a traffic manager and sends user requests to available servers. This improves performance and keeps the system stable even when traffic spikes.
- This approach is cheaper, more fault-tolerant and easier to scale.

181. Scenario 43: Load balancing inside microservices

Scenario: A blog platform runs on 5 servers behind a load balancer. But your monitoring dashboard shows something strange. One server is overloaded, and others are free. Why is this happening and how will you fix it?

Explanation:

- This usually happens when the wrong load balancing algorithm is used. If **Round Robin** is used blindly, traffic is spread equally without checking server load.
- Instead, using **Least Connections** or **Least Response Time** algorithm will distribute requests based on current load.

- Example: If Server A has 2 active users and Server B has 10 active users, the load balancer should send new users to Server A first. Choosing the right load balancing strategy avoids uneven load and improves reliability.

182. **Scenario 44: Some servers are overloaded while others are idle**

Scenario: In a microservices system, the Order Service calls the Payment Service which has 3 running instances. Should we use an external load balancer here?

Explanation:

- For internal service-to-service communication, we use client-side load balancing instead of a full network load balancer.
- Tools like Spring Cloud LoadBalancer, Ribbon (legacy), gRPC Load Balancing, or Consul pick a Payment Service instance automatically from a service registry.
- This reduces network overhead and improves performance. Use API Gateway + Service Registry + Client-side Load Balancer for clean microservice traffic management.

183. **Scenario 45: Rate-limiting**

Scenario: A banking application exposes a Login API. Hackers are trying to brute-force passwords by making thousands of login attempts per minute, causing system slowdown. How will you protect this service?

Explanation: This problem is not about performance; it is about abuse.

- We need to limit requests per user or per IP using rate limiting.
- For example, allow only 5 login requests per minute per user/IP. If someone exceeds this, block them temporarily. This protects the system from malicious attacks.
- In real systems, this can be done with API Gateway rate limiting (NGINX, Kong, AWS API Gateway) or with Resilience4j RateLimiter inside microservices.

184. **Scenario 46: Debugging Failures in a Microservice Chain**

Scenario: Your system has 12 microservices. A user reports that their order is not getting confirmed. The request flows through API Gateway > Order Service > Payment Service > Inventory Service. Logs show nothing helpful. How will you trace the exact point of failure??

Explanation:

In distributed systems, logs from one service are not enough. We need distributed tracing. Using tools like Zipkin with OpenTelemetry, we assign a trace ID to each request at the gateway. This trace ID is passed across services so we can follow the complete journey of the request. When a failure happens, we see which service took how long and where the request stopped.

185. Scenario 47: Handling High Volume Logs

Scenario: Your application generates millions of logs every day. Developers need logs for debugging but storing them on each server is filling disk space and making the system slow. How do you handle centralized logging?

Explanation:

Storing logs locally is risky and hard to search. The real solution is **centralized logging**. We send all logs to a common storage using the ELK stack (Elasticsearch, Logstash, Kibana)

We can search logs by service, trace ID or user ID. For example, we can search: `orderId = 5678 AND level = ERROR`. This makes debugging faster and keeps the system clean.

186. Scenario 48: Detecting Problems Before Users Report Them

Scenario: In production, issues are found only after users complain. The team wants a way to detect performance issues or service failures early. How would you do monitoring?

Explanation:

This is not just logging. We need metrics and alerting.

Tools like **Prometheus + Grafana** can collect metrics like CPU usage, memory, latency and request failure counts.

Then we define alerts. For example: "Trigger alert if error rate for Payment Service is more than 5 percent for 2 minutes." We can use alerting systems like **PagerDuty** to notify the Engineering teams or support teams.

187. Scenario 49: Debugging Slow or failing Requests

Scenario: Users report that some API calls are slow or failing but only at certain times. Logs are normal. Servers are healthy. How do you find the root cause of latency

Explanation:

For debugging slow API requests in microservices, we use **Distributed Tracing**. In Spring-based systems, **Spring Cloud Sleuth** adds tracing information like `traceId` and `spanId` to every request and log. Then we send this tracing data to a tool like **Zipkin** to see a timeline view of where time is being spent.

188. Scenario 50: Partial Failure

Scenario: Imagine we are building an online food ordering system. When a customer places an order, three services work together:

1. Order Service creates the order
2. Payment Service collects the money
3. Restaurant Service confirms the food availability

Now imagine this flow

- Order Service successfully creates the order

- Payment Service successfully takes the payment
- Restaurant Service fails due to timeout

Money is taken but food is not confirmed. The system is stuck in the middle. That is a partial failure.

Solution:

To handle failures safely, we use four key ideas: **Retry**, **Circuit Breaker**, **Compensation**, and **Eventual Consistency**.

1. Retry - Try again safely

Sometimes services fail for temporary reasons like network delays or slow responses. In such cases, retrying solves the problem.

But retry must be safe

- Retry only for temporary errors, not for bad requests
- Use retry with backoff to avoid overloading the service
- Set retry limits

Example

If the Restaurant Service fails once due to timeout, Order Service retries 2 more times before giving up.

2. Circuit Breaker - Stop failure from spreading

If a service keeps failing, calling it again and again is useless and dangerous. It causes delays and blocks threads. The Circuit Breaker pattern stops calling the failed service for a short time. This keeps the system fast and stable.

Example

If Restaurant Service is down, Circuit Breaker opens and Order Service immediately responds with "Restaurant is currently busy. Please try another restaurant."

This prevents long waiting time.

3. Saga pattern

When a failure happens after some steps succeed, we undo the completed steps using a compensation action.

Example

- Order is created
- Payment is taken
- Restaurant fails

To fix this, we run a compensation step: refund the payment and cancel the order.

This is called a Saga pattern in microservices.

4. Eventual Consistency

In the real world, everything is consistent immediately. In microservices, that is not always possible. Instead, we allow temporary inconsistency but fix it in the background.

Example

Payment Service sends a message: "Payment successful"

If Restaurant fails later, a compensating event is sent

Restaurant Service reads it and cancels the pending action
System becomes consistent after a short delay
This is called eventual consistency.

189. Scenario 51: Data consistency -1

Scenario: How do you ensure data consistency across services with separate databases??

Solution:

In a microservice system, each service has its own database. This gives flexibility and independence, but it also creates a challenge. How do we keep data consistent when one business operation touches many services? This becomes a serious problem in systems like banking, ecommerce, and booking platforms.

Let us take a simple example.

When a user places an online order, three services work together.

- The Order Service creates the order.
- The Payment Service collects money.
- The Inventory Service reduces stock.

Each service has its own database. What happens if payment succeeds but stock update fails? The system becomes inconsistent.

To solve this, we cannot use traditional database transactions because distributed systems do not support a single transaction across many services. Instead, we follow patterns built for microservices.

The Saga pattern breaks a big transaction into small local transactions. Each service completes its own step and publishes an event. If any step fails later, we run a compensating action. For example, refund payment and cancel the order.

We can also use Kafka to send events between services and keep them in sync. With these patterns, we reach eventual consistency, which is the practical form of consistency in microservices.

190. Scenario 52: Data consistency -2

Scenario: How do you ensure data consistency across services with separate databases??

Solution:

In a microservice system, each service has its own database. This gives flexibility and independence, but it also creates a challenge. How do we keep data consistent when one business operation touches many services? This becomes a serious problem in systems like banking, ecommerce, and booking platforms.

Let us take a simple example.

When a user places an online order, three services work together.

- The Order Service creates the order.

- The Payment Service collects money.
- The Inventory Service reduces stock.

Each service has its own database. What happens if payment succeeds but stock update fails? The system becomes inconsistent.

To solve this, we cannot use traditional database transactions because distributed systems do not support a single transaction across many services. Instead, we follow patterns built for microservices.

The Saga pattern breaks a big transaction into small local transactions. Each service completes its own step and publishes an event. If any step fails later, we run a compensating action. For example, refund payment and cancel the order.

We can also use Kafka to send events between services and keep them in sync. With these patterns, we reach eventual consistency, which is the practical form of consistency in microservices.

191. **Scenario 53: Sending notifications / Broadcast an event**

Scenario: Your registration service needs to send email or sms after successful creation of the user. How do you design it?

Solution:

When a new user registers, the system often needs to send a welcome email or an SMS. The main idea is simple: the registration flow should stay fast, and the notification work should happen in the background.

Here is a clean way to design it.

1. User registers

The registration service creates the user in the database.

Once the record is saved, the service adds a small event into a queue. This event only contains basic details like user id, email, or phone number.

2. Using a message queue

A queue helps us handle extra work without slowing down the main request.

The registration service pushes a message like "UserCreated" into the queue and finishes its job.

This keeps the registration API fast.

3. Notification service picks it up

A separate service listens to the queue.

Whenever a new event arrives, this service reads it and sends the email or SMS using a provider like AWS SES, SendGrid, or Twilio.

4. Handling failures

If sending fails, the event goes into a retry queue so the system can try again later.

This avoids losing notifications during temporary issues.

192. Scenario 54: Scaling the microservice

Scenario: Your microservice should scale according the traffic, how would you do it?

Solution:

Scaling a single microservice independently is one of the biggest advantages of microservices architecture. Here's how I approach it:

1. Containerization (Docker/Kubernetes)

Package the microservice into a container. Use Kubernetes (K8s) or Docker Swarm to run multiple instances (pods) of that service. This makes horizontal scaling easy by just increasing the replica count.

2. Load Balancing

Place a load balancer (NGINX, AWS ALB) in front of the microservice. Incoming requests are evenly distributed across multiple service instances. Ensures high availability and avoids overload on a single instance.

3. Auto-Scaling Policies

Define Horizontal Pod Autoscaling (HPA) in Kubernetes or AWS Auto Scaling Groups. Scale based on CPU, memory, or request latency thresholds.

Example: If CPU > 70% for 5 minutes then add 2 more instances.

4. Database & Cache Optimization

Use read replicas or sharding for databases to handle load. Implement caching (Redis, CDN) to reduce database hits. This ensures the microservice scales effectively without bottlenecks

193. Scenario 55: Polling

Scenario: In a ride booking system, different microservices work together. Let's take two of them:

- **Driver Service:** holds driver status, location, and availability
- **Ride Service:** assigns rides to drivers

The Ride Service needs driver information to match riders quickly.

Now the question is: **Should Ride Service keep polling Driver Service again and again?**

No. That would overload both services and slow everything down.

There are better approaches.

1. Request Only What Is Needed (Direct API Call)

Ride Service can call a small API in Driver Service asking only the required data.

Example:

When a customer requests a ride, Ride Service sends a simple request like:

Driver Service returns only what is needed: `GET /driver/status?driverId=312`

```
{
  "driverId": 312,
  "status": "available",
  "location": "Madhapur"
```

```
}
```

No need to poll every few seconds. Ride Service asks only when required.

2. Event-Based Updates using Kafka (No Polling Needed, better approach)

A better approach is to let Driver Service **push** updates whenever something changes.

Whenever a driver changes status:

- comes online
- goes offline
- finishes a ride
- moves to a new location

Driver Service publishes an event like: DriverStatusUpdated

Ride Service subscribes to these events and updates its local cache.

This means Ride Service always has the latest driver information without asking again and again.

194. **Scenario 56: Scheduler + Multiple Instances**

Scenario: If I have a scheduler or cron job in my application, and it is deployed on multiple instances, will the scheduler job run simultaneously on all instances, how to prevent it?

Solution:

When we run our application on many instances, every instance loads the same code.

This means the scheduler or cron job inside the application will run **on every instance at the same time**.

If the job is supposed to run only once, this becomes a problem.

For example, consider a system that generates report logs every night.

If we have five instances, all five will run the cleanup at midnight, which is not what we want.

To avoid this, we make sure **only one instance runs the job**, even though the application is deployed everywhere.

1. Use a Distributed Lock (Most Common)

Before running the job, the instance tries to acquire a lock in a shared place like Redis, Mongo, MySQL, or Zookeeper.

Only the instance that gets the lock runs the job.

Others skip it.

Real example:

- A Report generating job runs every night.
- All instances try to acquire a Redis lock.
- Only one gets it and runs the job.
- The lock expires after the job finishes.
- This is the cleanest solution and used in many companies.

195. Scenario 57: API is failing

Scenario: Your external endpoint you are trying to call using Feign client is failing, how you handle this scenario?

Solution:

When an external endpoint called through a Feign client fails, we should handle it in a controlled way so that the failure does not break the whole application. The goal is to keep the system stable and give a safe response.

First, we avoid calling the failing service again and again. We do this by using a circuit breaker. If the service keeps failing, the circuit opens and stops further calls for some time. This protects our threads and resources.

Second, we provide a fallback response. A fallback is a simple backup logic that runs when the external call fails. Instead of throwing an error, we return default data or a safe message. This keeps the user flow working.

Third, we set timeouts. We do not wait forever for a response. If the service does not reply within a fixed time, we stop waiting and move to fallback logic.

```
@FeignClient(name = "product-service", fallback = ProductClientFallback.class)
interface ProductClient {
    Product getProductById(String id);
}

class ProductClientFallback implements ProductClient {
    public Product getProductById(String id) {
        return new Product(id, "Product unavailable");
    }
}
```

In this example, if the product service is down or slow, the fallback class runs and returns a safe response.

196. Scenario 58: Security

Scenario: In your application, multiple microservices need security like token validation, how you design it efficiently?

Solution: In a system with many microservices, doing token validation inside every service creates repetition and slows things down. A better design is to place token validation at a single-entry point. We use an **API gateway** that checks the token once for every request. If the token is valid, the request is allowed to go inside the system.

Inside the system, services trust the request because it already passed security. Services only focus on business logic, not security rules. For example, when a request comes from a mobile app, the gateway validates the token and forwards user details to services. This reduces duplicate work, improves performance, and keeps security logic in one clear place

197. Scenario 59: Read heavy Service

Scenario: One service (e.g., Catalog) gets most of the reads. How would you redesign to improve performance

Solution:

Approach 1: When one service like Catalog gets most of the reads, the first problem is repeated database hits for the same data. The redesign starts by separating read and write paths. We keep writes simple and safe, but reads are optimized.

For example, when a product is added or updated, we save it in the main database and also update a cache. For read requests, we first read from the cache instead of the database. If data is not found, we load it once and store it. We can also create a read-only copy of data shaped exactly for screens. This reduces load, speeds responses, and keeps the system calm under traffic.

Approach 2: We can use CQRS pattern, CQRS means we handle writing data and reading data in two different ways.

In a service like Catalog, most requests only ask for data. Very few requests change data. So we design the write side only to save correct data and follow business rules. We design the read side only to return data quickly.

For example, when a product is added or updated, we save it in the main database. At the same time, we copy that data into a fast store made only for reading. When screens need product details, they read from this fast store, not from the main database. This keeps reads fast, lowers database load, and helps the system handle many users smoothly.

SYSTEM DESIGN

198. System design 1: What is system design?

System design is nothing but a clear plan to build a software system that works well for many users and handles real world problems.

It is similar to building a house. Before building, we do not start with bricks. First, we sit and plan. We decide how many rooms are needed, where the kitchen will be, how much space is required and how people will move inside the house. Only after this plan is ready, construction starts. In software also, system design is that planning stage.

In system design

- we think about how data will flow,
- how users will interact
- how fast the system must respond,
- how the system will deal with errors and
- how to handle growth when more users come.

It is not about writing code. It is about making the system strong and easy to maintain.

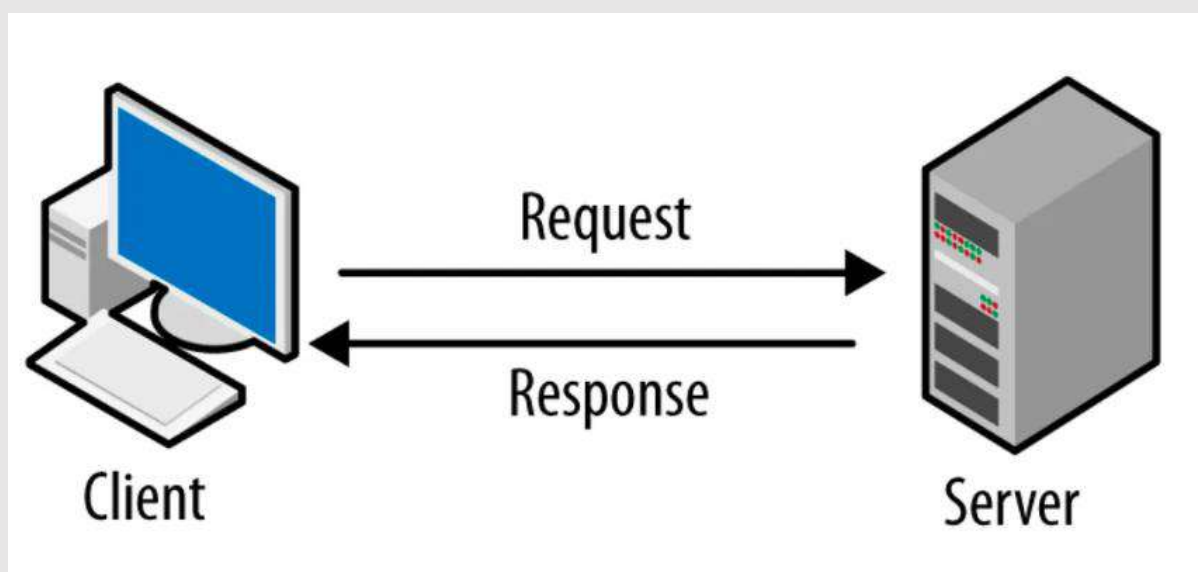
Let us take a simple example. Think about a chat application like WhatsApp. If we want to build a basic chat system, we need to plan a few things

- How will one user send a message to another
- Where will we store messages
- How will we deliver messages when someone is offline
- How do we make sure messages are safe and not lost
- How do we support millions of users together

199. System design 2: What is Client-Server architecture?

Client Server architecture is a simple way of building software systems where two main parts work together to complete a task. One part is called the client and the other part is called the server. Both have different jobs.

- The client is like a person who makes a request. It sends a message asking for some information or service.
- The server is like a helper who listens to requests, processes them and sends back the required result.



Now let us take a real software example. Suppose we open a website like an online shopping site. Our browser acts as the client. When we search for a product, the browser sends a request to the

server of that website. The server then searches the product details in the database and sends the result back to the browser. The browser displays it on the screen.

For some more information: [Client-Server Architecture Explained](#)

200. **System design 3: Types of Hypertext protocols (HTTP vs HTTPS vs HTTP/2)?**

HTTP

HTTP means Hyper Text Transfer Protocol. It is the basic rule used by the web. When we type a website address in a browser and press enter, the browser sends an HTTP request to the server. The server understands this request and sends back a response. This response may contain a web page, an image or any other data.

For example, when we open `http://example.com`, the browser sends a request like

```
GET / HTTP/1.1 Host: example.com
```

The server replies with a page. This is normal HTTP.

The problem with HTTP is that data travels in plain text. Anyone in the middle can see the data. If we log in to a site using HTTP, our password is not safe.

HTTPS

HTTPS is safer HTTP. The letter S stands for Secure. HTTPS uses encryption so that nobody can read the data while it travels between client and server. It uses a certificate to prove that the server is real and trusted.

For example, when we open `https://bank.com`, our login password, account details and other data are protected. Even if someone tries to intercept the network, they will see only random unreadable data.

Today almost all websites follow HTTPS because security is important.

HTTP2

HTTP2 is a faster and smarter version of HTTP. It was created to solve speed problems in HTTP1. In HTTP1, the browser opens one connection and sends one request at a time. If we load a web page with many images and files, it becomes slow.

HTTP2 allows multiple requests together in a single connection. It also compresses headers and manages data better. The result is faster loading time.

A simple example. When opening a news website, many small files like images, ads and scripts are loaded. With HTTP1, each file may wait. With HTTP2, they are sent together, so the page opens faster. Most modern websites use HTTPS with HTTP2.

Reference: [What are HTTP, HTTPS, and HTTP/2/3? • Internet Essentials - FastComet](#)

201. System design 4: What is TLS/SSL connection?

TLS and SSL are security layers that make HTTPS possible. Without them, HTTPS cannot exist. We already know that HTTPS is the secure version of HTTP. But how does it become secure. The answer is TLS or SSL.

What are SSL and TLS

SSL means Secure Sockets Layer. TLS means Transport Layer Security. Both are security protocols used to protect data that travels between a client and a server. TLS is the newer and safer version. SSL is old and not used much now, but people still say SSL out of habit even when they actually mean TLS.

Why TLS or SSL is needed

When we send data using normal HTTP, it travels in plain text. Anyone can read it. For example, if we log in to a website using HTTP and someone captures the network, they can see our password. This is dangerous.

TLS solves this problem. It locks the data using encryption. Only the client and server can unlock and read it.

How HTTPs works with TLS?

1. We enter a website link with https in the browser. Example `https://google.com`
2. The browser tries to connect to the server of `google.com`
3. The server sends its TLS certificate to the browser. This certificate contains the server public key and information about the website
4. The browser checks the certificate in these steps
 - It checks if the certificate is signed by a trusted Certificate Authority
 - It checks if the certificate is not expired
 - It checks if the certificate domain name matches the website domain
 - It checks if the certificate is not cancelled or blocked
 - If the certificate is trusted, the browser creates a random secret key
5. The browser encrypts this secret key using the server public key and sends it to the server. Only the server can unlock it because it has the private key
6. Now both browser and server have the same secret key. This key will be used to encrypt and decrypt data
7. A secure connection is now ready This full process is called the TLS handshake
8. After this handshake, normal HTTP requests and responses start. But now they travel inside the encrypted channel

Reference: [SSL, TLS, HTTPS Explained](#)

202. System design 5: What is IP Address and DNS?

IP Address

Every device connected to the internet has a unique number called an IP address. It is like a home address but for computers and servers. This address helps one system find another system on the network.

Example of an IP address

192.168.1.10 or 142.250.182.14

When we open a website like google.com, our system actually connects to the IP address of the Google server. The browser uses this IP address to send a request and get back a response.

DNS

DNS means Domain Name System. We cannot remember long IP addresses for every website. So, DNS helps us by converting website names into IP addresses.

Example

When we type: `https://amazon.com`

DNS converts it internally to something like 54.239.28.85

Then our browser connects to that IP to open Amazon.

DNS works in the background automatically. We do not see it, but every time we open any website, DNS does this name to number conversion.

203. System design 6: What will happen when you enter google.com OR How the Web works?

1. Browser Cache Check

When we type google.com in the browser and press enter, a series of small but important steps take place behind the screen. Everything happens so fast that we hardly notice it, yet each step is necessary to load the page.

- The journey begins inside the browser.
- First it checks if it already knows the IP address of google.com from a previous visit.
- If it is stored in the local cache, it uses it directly.
- If not, it asks the operating system.
- If the system also does not know, then the real search begins.

2. DNS Lookup

A DNS request is sent to a DNS server to find the IP address of google.com.

DNS converts names into numbers.

The browser waits until it gets a reply such as **142.250.182.14**, which is one of Google's server addresses.

3. TCP Connection

Now that the browser knows where Google lives on the internet, it tries to connect.

A TCP connection is created.

This step makes sure that both sides can talk safely.

4. TLS Handshake (HTTPS Security)

Since Google uses HTTPS, the browser also starts a TLS handshake.

In this step the browser checks Google's security certificate and creates a secure channel so that no one can read or change the data during transfer.

5. Sending the Request

Once the secure connection is ready, the browser finally sends a request to Google.

The request is simple:

GET / HTTP/1.1

Host: google.com

6. Server Response

Google receives this request on its server.

The server understands that the browser needs the homepage and prepares a response.

It sends back an HTML file along with some instructions for loading style files, images and scripts.

7. Rendering the Page

The browser receives the response, reads the HTML and starts rendering the page step by step.

It builds the layout, loads the logo, search bar and other elements.

After downloading everything needed, the page becomes fully visible to us.

8. Final View

That is how the Google homepage appears.

It looks simple, but behind that single page load, networking, security and browser systems quietly work together.

Reference: [How The Web Works - The Big Picture](#)

204. **System design 7: What is Database and its types?**

A **database** is simply a place where data is stored, organized, and managed so that it can be easily accessed, updated, and analysed.

SQL Databases (Relational Databases)

SQL stands for Structured Query Language, which is the standard way to manage and query data in relational databases. These databases store data in tables that are connected to each other by relationships. They are best used when data is structured and needs to be organized in rows and columns, like in a spreadsheet.

Example:

- MySQL
- PostgreSQL
- SQL Server
- Oracle Database

NoSQL Databases (Non-Relational Databases)

NoSQL stands for **Not Only SQL**. These databases are designed to handle unstructured or semi-structured data. They are more flexible than SQL databases because they do not rely on a fixed schema. This makes them suitable for storing large volumes of varied data types, like Catalog data in ecommerce app, logs etc.

Example:

- **MongoDB** (Document-based)
- **Cassandra** (Columnar-based)
- **Redis** (Key-value store)

Reference: [NoSQL vs Relational Databases \(RDBMS\): Key Differences and When to Use Which | by Yasemin Büşra Karakaş | Medium](#)

Video link: [\(2\) Database Design Tips | Choosing the Best Database in a System Design Interview - YouTube](#)

205. System design 8: What is RDMS and when to use it?

A Relational Database Management System (RDBMS) is a type of database that stores data in tables made of rows and columns. These tables can be linked to each other using relationships. This makes data easy to organize, search and manage. SQL databases like MySQL, PostgreSQL, Oracle and SQL Server are all RDBMS.

The main goal of RDBMS is to ensure data is accurate, reliable and consistent even when many users access it at the same time. It follows ACID properties, which makes it strong for critical applications.

What are ACID Properties

ACID is a set of rules that help maintain data consistency and integrity.

Property	Meaning
----------	---------

Atomicity	A transaction is all or nothing. If one step fails, everything is cancelled.
-----------	--

Consistency	Data always remains valid according to rules.
-------------	---

Isolation	Many users can work at the same time without affecting each other.
-----------	--

Durability	Once data is saved, it will not be lost even if the system crashes.
------------	---

Example: In a banking system, if money is transferred from one account to another, both debit and credit must succeed. If debit happens and credit fails, the whole operation should roll back. RDBMS handles this perfectly using ACID.

When to Use RDBMS

- When data is structured and fits into tables
- When strong consistency and correctness is important
- When data follows clear rules and relationships

206. System design 9: What is Document Based DB and when to use it?

A **Document Based Database** is a type of NoSQL database that stores data in the form of documents, usually in JSON or similar formats. Each document can have different fields and structure, which makes it flexible for real world data that does not always fit into tables.

In relational databases data must follow a fixed schema with rows and columns, but document databases allow us to store data in a natural way as objects. This is closer to how we write data in applications.

Example of a document: A product in a document database may look like this:

```
{
  "id": 101,
  "name": "Laptop",
  "brand": "Dell",
  "price": 55000,
  "specs": {
    "ram": "8GB",
    "storage": "512GB SSD"
  },
  "available": true
}
```

For some other project let's say for milk, brand and specs will not be there, it contains type and expiryDate. So, the data structure is keeps changing.

Use a document database when:

- Data structure is flexible or keeps changing
- Fast reads and writes are required
- Data fits naturally as JSON or objects

Use Case

Why Document DB fits

Product catalogs (e-commerce) Each product can have different fields

User profiles Different users can store different data

Popular Document Databases

- MongoDB
- CouchDB
- Firebase Firestore
- Amazon DocumentDB

Document Databases vs. Relational Databases

Relational databases store data in tables with rows and columns. Each row represents a complete record, and tables are linked using relationships. This structure works well when data is fixed and strongly related. We usually use relational databases when accuracy and consistency are very important.

For example, in a banking system, we store customers, accounts and transactions in separate tables. These tables are connected using keys. When we update a transaction, we want the change to be correct everywhere. Relational databases handle this very well.

Document databases store data as documents, usually in a format similar to JSON. Each document contains all related information in one place. The structure is flexible, and documents do not need to follow a strict schema. This makes document databases easy to work with when data changes often.

For example, in an e-commerce or content application, one product document can store name, price, images, reviews and tags together. We do not need to join multiple tables to read this data. We can fetch the whole document in one go.

Relational databases are strong in handling transactions and complex relationships. They are best when data structure is stable and rules are strict. Document databases are better when data structure changes frequently and fast reads are more important than complex joins.

Reference: [Document Database: Definition, Features, Use Cases | FerretDB Blog](#)

207. **System design 10: What is Columnar DB and when to use it?**

A Columnar Database, also called a Wide Column Database, is a type of NoSQL database that stores data in columns instead of rows. This is different from relational databases like MySQL where each row is stored together. Column databases group data by columns which makes it very fast to read and write large amounts of data.

Simple way to understand

Imagine a table of user orders like this:

Example of Columnar Database

Here is an example of a simple database table with four columns and three rows.

ID Number	Last Name	First Name	Bonus
534782	Miller	Ginny	6000
585523	Parker	Peter	8000
479148	Stacy	Gwen	2000

In a Columnar DBMS, the data stored is in this format:

```
534782, 585523, 479148; Miller, Parker, Stacy; Ginny, Peter, Gwen; 6000, 8000, 2000.
```

In a Row-oriented DBMS, the data stored is in this format:

```
534782, Miller, Ginny, 6000; 585523, Parker, Peter, 8000; 479148, Stacy, Gwen, 2000.
```

This makes it very fast to find information from a single column, even when there are millions of records.

Use a columnar database when

- We need to store and analyse huge volumes of data
- Fast reads are required more than fast writes
- We do not need complex joins

Use Case	Why Column DB fits
Event logs from applications	Handles high write speed and large data
Real time dashboards	Fast column reads
Recommendation systems	Good for large scale queries

Popular Columnar Databases

- Apache Cassandra - is great at **storing billions of rows quickly**, but not great at **analyzing** those billions of rows later.
- HBase
- Google Bigtable
- Google BigQuery: BigQuery is a fully-managed, serverless data warehouse that enables scalable analysis over petabytes of data. It's a platform as a service (PaaS) that uses an SQL-like syntax and is deeply integrated with other google cloud services.

Columnar database vs row-oriented database: What is the difference?

Columnar databases store data vertically, optimizing for analytics and large-scale aggregations, while **row-oriented databases** store data horizontally, prioritizing transactional processing and quick access to entire records.

Columnar databases excel in data warehouse scenarios where complex queries involving aggregations and analytics are common, while row-oriented databases are more suited for transactional applications where quick access to entire records is essential.

Reference: [What is a Columnar Database? - GeeksforGeeks](#)

[20 examples of columnar databases - RisingWave: Real-Time Event Streaming Platform](#)

208. System design 11: What is key value pair database and when to use it?

A Key Value database is the simplest type of NoSQL database. It stores data as a pair of two values: a key and a value. The key is like a unique name and the value is the data linked to that key. This makes lookups very fast because we can get data directly using the key without searching.

```
"user:101" -> "John"
"cart:202" -> ["Laptop", "Mouse"]
```

We use key value databases when speed matters and data is simple like caching, user sessions, shopping cart data or storing temporary values.

Popular tools are Redis, DynamoDB and Memcached.

209. System design 12: What is data warehouse and when to use it?

A **data warehouse** is a place where we store large amounts of data collected from many different systems. This data is mainly used for reporting, analysis and decision making. A data warehouse is not used for daily transactions like placing orders or making payments. It is used to understand what already happened.

In normal applications, data lives in operational databases. These databases are busy handling day to day work. For example, an order database stores new orders, updates order status and processes payments. Querying this database for heavy reports can slow down the system. A data warehouse solves this problem by keeping a separate copy of data only for analysis.

Data from different sources is regularly moved into the data warehouse. These sources can be order systems, payment systems, user activity logs or even external data. Before storing, the data is cleaned and organized so it becomes easy to analyze.

Let us take a simple example. In an online shopping company, orders are placed every second. The operational database handles this traffic. At the end of the day, business teams want answers to questions like total sales today, best selling products, sales by city and monthly growth. Running these queries on the order database is risky and slow. Instead, all order data is copied to a data warehouse, and reports are generated from there.

Common data warehouses are Snowflake, Amazon Redshift, BigQuery and Azure Synapse.

BigQuery is a cloud-native, serverless, petabyte-scale data warehouse provided by Google Cloud, designed for high-performance analytics on massive datasets using standard SQL

210. System design 13: What is CAP Theorem?

The CAP Theorem tells us an important rule about distributed systems. When data is stored on many servers, we cannot achieve everything perfectly at the same time. We must choose what to prioritise because there is always a compromise.

CAP stands for **Consistency, Availability and Partition Tolerance**.

- **Consistency** means all users get the same and latest data, no matter which server they connect to.
- **Availability** means the system always gives a response, even if some parts are slow or not working.
- **Partition Tolerance** means the system keeps running even if communication between servers breaks due to network issues.

The theorem says that in a distributed system, we can choose only **two** of these three, not all three together.

Example: Imagine a banking system that runs on many servers. If one server loses network connection, we must choose what to keep.

- **When we choose Consistency and Partition Tolerance (CP)**

The system cares more about showing the correct data than giving a quick reply. If two servers

lose connection, some requests are paused until they agree on the same data.

For example, during a money transfer, the system will wait before showing the balance so we never see a wrong amount. **Examples:** HBase, MongoDB with majority write and read

- **When we choose Availability and Partition Tolerance (AP)**

The system always gives a response, even if servers cannot talk to each other. But the data may be slightly old on some servers.

For example, one server may still show an old account balance for a short time after a transfer.

Examples: Cassandra, DynamoDB

- **When we choose Consistency and Availability (CA)**

The system gives correct data and fast replies, but only when there are no network problems.

In real systems, networks fail sometimes, so this choice is rarely possible in practice.

Examples: MySQL in a single server, PostgreSQL on one machine

So, CAP theorem teaches a simple lesson. In distributed systems, failures happen and network breaks. We must choose what matters most for our system: accuracy of data or system availability. Different applications choose differently based on their needs.

In practice, partitions are inevitable, so systems must choose between consistency and availability when a partition occurs.

- For a **banking app**, consistency is more important than availability.
- For a **social media app**, availability is often prioritized over strict consistency.

Reference: [CAP Theorem in Real-World Scenarios - DEV Community](#)

211. **System design 14: What is Database sharding?**

Database sharding is a way of splitting a large database into smaller parts called **shards**. Each shard holds a portion of the data. Instead of keeping all data in one single database server, we spread it across many servers. This helps the system handle more users and more data without becoming slow.

Need for Sharding

When data grows, a single database starts to slow down. Queries take longer. Writes become heavy. The system may even crash. Upgrading the server hardware only works for some time. After that, the only option is to scale across multiple servers. That is where sharding helps.

Simple example

Imagine a user database with 10 million users. If we store all users in one database, it becomes slow. Instead, we can shard it like this:

- Shard 1: Users with IDs 1 to 2 million
- Shard 2: Users with IDs 2 million to 4 million
- Shard 3: Users with IDs 4 million to 6 million

- Shard 4: Users with IDs 6 million to 8 million
- Shard 5: Users with IDs 8 million to 10 million

Now each shard has less data, so each query becomes faster and easier to manage.

Large platforms like Amazon, Twitter and Instagram use sharding. For example, Twitter may store tweets of users based on regions like Asia shard, Europe shard and America shard. This makes tweet access faster worldwide.

Reference: [What is Sharding in DBMS? - GeeksforGeeks](#)

212. System design 15: What is partitioning in Database?

Database partitioning is a way of dividing a large table into smaller parts called **partitions** so that data becomes easier to manage and faster to access. Even though the data is divided, it still appears as one single table to the application.

As data grows, a table can become very large and slow. Searching, updating or deleting data takes more time. Partitioning helps by breaking this large table into smaller logical pieces. Each partition holds only a part of the data which makes operations faster.

Example

Imagine we have a table that stores order history with hundreds of millions of records from many years. Instead of keeping one giant table, we can partition it by year like this:

- Partition 1: Orders from 2019
- Partition 2: Orders from 2020
- Partition 3: Orders from 2021
- Partition 4: Orders from 2022
- Partition 5: Orders from 2023

Now if we want to get all orders from 2023, the database reads only that partition instead of scanning the whole table.

Point to remember: Database is sharded and data is partitioned

For more information watch this: [Database Sharding and Partitioning](#)

213. System design 16: What are the types of partitioning methods?

To create a partitioned table in **MySQL** we need to define the partitioning scheme. The MySQL supports several types of the partitioning methods:

1. **Range Partitioning:** Data is divided based on a range of values.
2. **List Partitioning:** Data is divided based on a list of the values.
3. **Hash Partitioning:** Data is divided based on the hash function.
4. **Key Partitioning:** Data is divided based on the key.

Range partitioning divides data into partitions based on the specified range of values. This is useful for partitioning data based on the dates or numeric ranges.

```
CREATE TABLE orders (  
    order_id INT,  
    order_date DATE,  
    customer_id INT,  
    amount DECIMAL(10, 2)  
)  
PARTITION BY RANGE (YEAR(order_date)) (  
    PARTITION p0 VALUES LESS THAN (2019),  
    PARTITION p1 VALUES LESS THAN (2020),  
    PARTITION p2 VALUES LESS THAN (2021),  
    PARTITION p3 VALUES LESS THAN (2022)  
);
```

In this example, the **orders table** is partitioned by the year with each partition containing orders from the specific year.

The **List partitioning** divides data based on a predefined list of the values. This is useful when partitioning data by categories or specific values.

```
CREATE TABLE employees (  
    emp_id INT,  
    emp_name VARCHAR(50),  
    department VARCHAR(50)  
)  
PARTITION BY LIST COLUMNS(department) (  
    PARTITION p0 VALUES IN ('HR', 'Finance'),  
    PARTITION p1 VALUES IN ('Engineering', 'Sales'),  
    PARTITION p2 VALUES IN ('Marketing', 'Support')  
);
```

In this example, the employees table is partitioned by the department with each partition containing employees from the specific departments.

The **Hash partitioning** divides data based on the hash function which distributes data evenly across partitions. This is useful for ensuring an even distribution of the data.

```
CREATE TABLE users (  
    user_id INT,  
    user_name VARCHAR(50),  
    email VARCHAR(50)  
)  
PARTITION BY HASH(user_id) PARTITIONS 4;
```

In this example, the user's table is partitioned using the hash function on the **user_id** column distributing data across the four partitions.

The **Key partitioning** is similar to the hash partitioning but uses MySQL's internal function for the partitioning.

```
CREATE TABLE products (  
    product_id INT,  
    product_name VARCHAR(100),  
    price DECIMAL(10, 2)  
)  
PARTITION BY KEY(product_id) PARTITIONS 4;
```

In this example, the products table is partitioned by the **product_id** column using MySQL's internal key function.

When product_id goes from 1 to 100, MySQL does **not** put 1–25 in partition 1, 26–50 in partition 2, and so on. That would be range partitioning. Key partitioning works very differently.

Because we created **4 partitions**.

So for example, internally it may behave like this.

- product_id = 1 -> hash(1) -> 9 -> $9 \% 4 = 1$ -> goes to partition 1
- product_id = 2 -> hash(2) -> 14 -> $14 \% 4 = 2$ -> goes to partition 2
- product_id = 3 -> hash(3) -> 5 -> $5 \% 4 = 1$ -> goes to partition 1
- product_id = 4 -> hash(4) -> 8 -> $8 \% 4 = 0$ -> goes to partition 0

And so on up to 100.

Reference: [Partitioning in SQL Databases —. Data partitioning in SQL databases is a... | by Mayank Jain | Javarevisited | Medium](#)

214. **System design 17: What is Database indexing?**

Database indexing is a technique used to speed up data search in a database. Without an index, the database has to scan every row in a table to find the required data, which takes time when the table becomes large. An index works like the index page of a book. Instead of searching every page, we directly jump to the right page and find what we want quickly.

Example

Imagine a table with one million users. If we search by “name” without an index, the database checks every row one by one until it finds the right name. This is slow. But if we create an index on the “name” column, the database can find the user quickly using the index, just like looking up a word in a dictionary.

When to use indexing: Indexing is useful when we often search, filter or sort based on a column like email, username, phone number or order date. These columns are good candidates for indexing.

Important point: Indexes improve read speed but can slow down write operations like insert and update, because the index also needs to be updated. So, we must add indexes wisely only where they are needed.

Reference: [How Does Indexing Work | Atlassian](#)

215. **System design 18: What is Master-Slave architecture in Database?**

Master Slave architecture in databases is a way of copying data from one main database to one or more backup databases. The main database is called the **Master** and the backup databases are called **Slaves**.

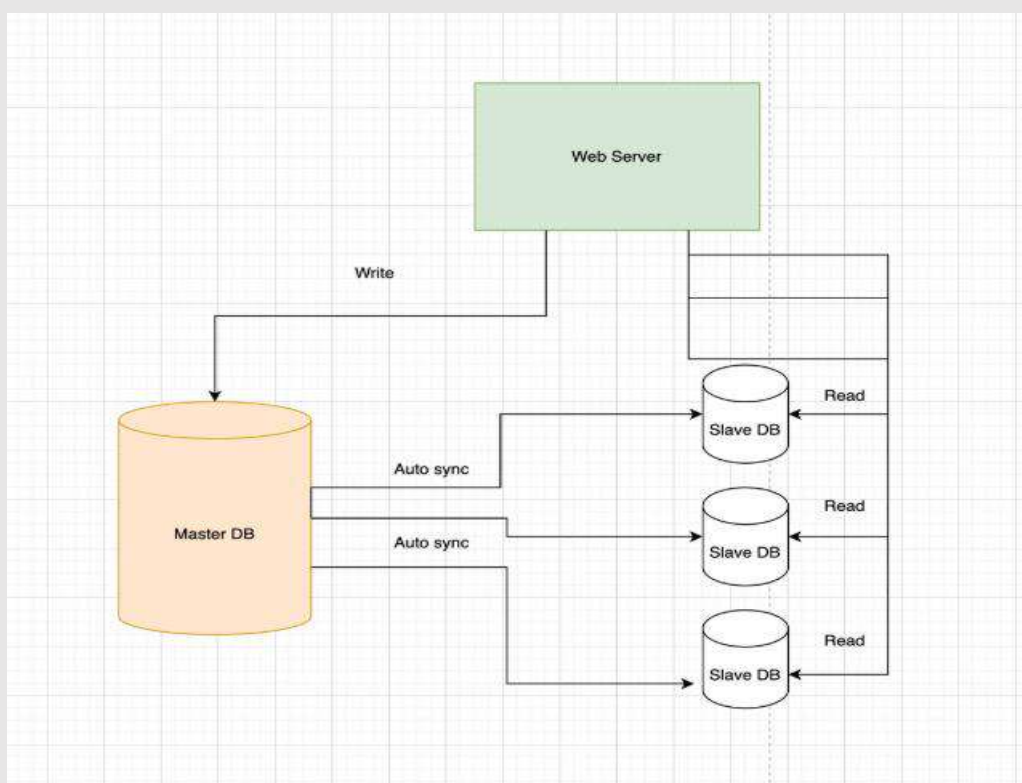
The Master handles all write operations like insert, update and delete. Each time data changes in the Master, it automatically sends a copy of that change to the Slave databases. The Slaves usually handle only read operations. This helps in reducing load on the Master and improves performance.

Why it is used

As applications grow, a single database cannot handle all read and write requests. Master Slave setup solves this by sharing the workload. The Master focuses on writing data safely, while Slaves answer read queries. This makes the system faster and also provides backup in case the Master fails.

Example

Imagine a social media app. When a user updates their profile, the change goes to the Master database. The Master stores it and then copies it to all Slave databases. But when users view each other's profiles, that data is served from the Slave databases. This way the system handles heavy traffic without slowing down.



Pros of Master-Slave Database Architecture

1. Reliable backups

Slave databases keep a copy of the master data. We can take backups from a slave without stopping the master. Even if the master fails, recent data is still available.

2. Easy application scaling

As users increase, read traffic also increases. We can send read requests to slave databases instead of the master. This keeps the application fast and improves user experience.

3. Load distribution for analysis

Different slave databases can serve different teams. For example, sales reports can run on one slave and finance reports on another, reducing pressure on the master database.

4. Better read performance

Multiple slaves handle read queries, so the master focuses mainly on write operations. This separation improves overall performance, especially in systems with many read operations.

Cons of Master–Slave Database Architecture

1. Write operations are hard to scale

All write requests must go to the master database. To handle more writes, we must increase master server capacity, which is costly and has practical limits.

2. Risk of data mismatch

Replication is usually asynchronous. If the master crashes before data reaches slaves, some recent updates may be lost, causing data inconsistency between master and slave databases.

3. No automatic failover

If the master database fails, a slave does not automatically become the new master. Manual intervention or extra tools are required to restore write operations.

Reference: [Master-slave replication database concept for beginners | by Denny Lesmana | Medium](#)

216. **System design 19: What is Event Driven System?**

An **Event Driven System** is a way of building software where actions happen as a response to events. An event is anything that happens in the system, like a new user registration, a payment success or a file upload. Instead of services calling each other directly, they communicate by sending and receiving events.

How it works

In an event driven system, when something happens, a service publishes an event. Other services that are interested in that event react to it. There is no direct connection between them, so each service works independently. This makes the system more flexible and easier to scale.

Example:

Imagine an e-commerce system. When a customer places an order, an **order_placed** event is published. Many other services may need this information:

- The payment service listens to the event and starts payment processing
- The inventory service reduces stock
- The email service sends an order confirmation email
- The analytics service records the event

All these services react to the same event in their own way. None of them need to know about each other. They only know about the event.

1. Flexibility and agility

Event Driven System allows us to add or change features by listening to events, without touching existing services or breaking the system.

Example: Adding email alerts without changing order or payment services.

2. Better scalability

Each service works independently. When load increases, we scale only the services that process heavy events.

Example: More order events trigger more order processors during a sale.

3. Real-time response

Events are handled as soon as they happen, so the system reacts immediately without waiting for other steps.

Example: Payment success instantly triggers invoice and notification events.

4. Loose coupling

Services do not call each other directly. They only react to events, which reduces dependency and failure impact.

Example: Inventory service does not depend on order service availability.

5. Modular system design

Event Driven System breaks a big system into small event-based services, making development, testing and maintenance much easier.

Tools we use: Apache Kafka, RabbitMQ, ActiveMQ, Google Pub-Sub etc

Reference: [Event-Driven Architecture - System Design - GeeksforGeeks](#)

217. System design 20: What is Message Queue?

A **Message Queue** is a way for different services in a system to communicate with each other by sending messages through a queue. Instead of calling each other directly, one service puts a message into the queue and another service reads it later and processes it. This makes the system more stable and flexible.

In real systems, not everything happens at the same time. Some tasks take longer, some services may be slow or even go down for a while. A message queue helps by allowing services to work independently without waiting for each other.

It also protects the system from heavy load. If a sudden spike of work comes in, the queue holds the extra messages until the system is ready to process them. This prevents failures and keeps the system smooth.

Example

Imagine we are building an e-commerce application. When a user places an order, three things must happen:

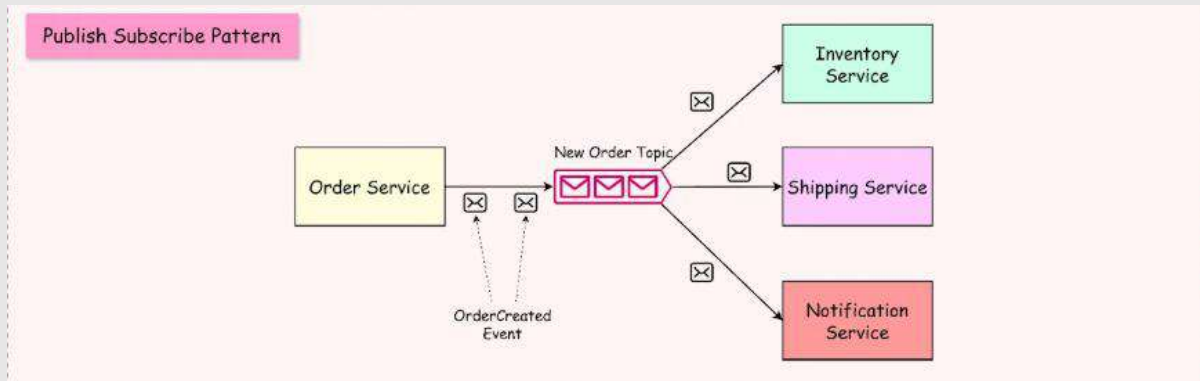
1. Save the order in the database
2. Reduce stock from inventory
3. Send confirmation email

If all three tasks are done at the same time inside one service, the system becomes slow and may fail if many users place orders together.

Instead, we can use a message queue:

- The order service only saves the order and sends a message to the queue like "order_created".
- The inventory service reads this message later and reduces stock.
- The email service also reads the message and sends a confirmation email.

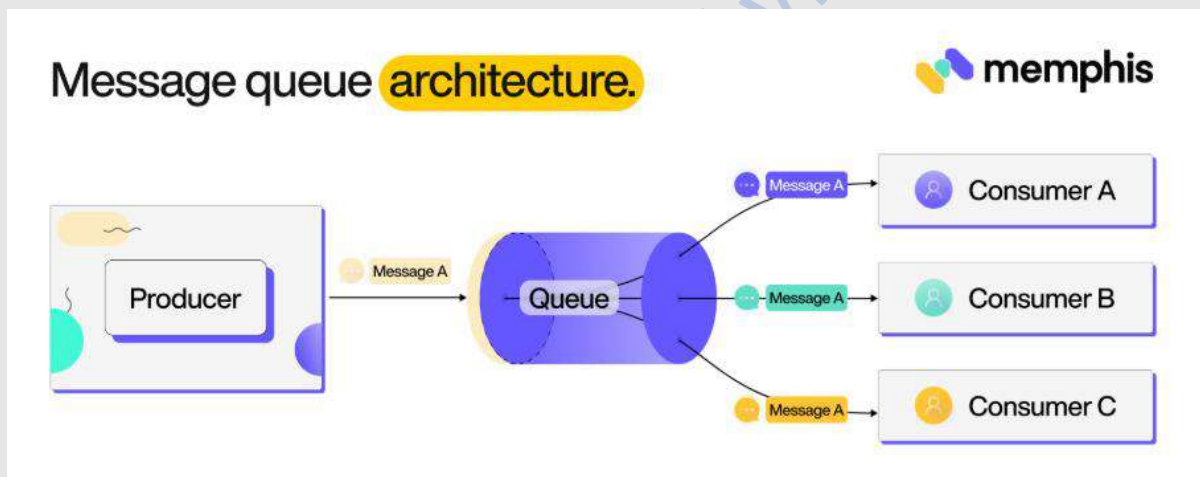
Each task runs in its own time without blocking the main flow. If one service is slow, others still continue working.



Message Queue Architecture

A message queue uses a straightforward architecture:

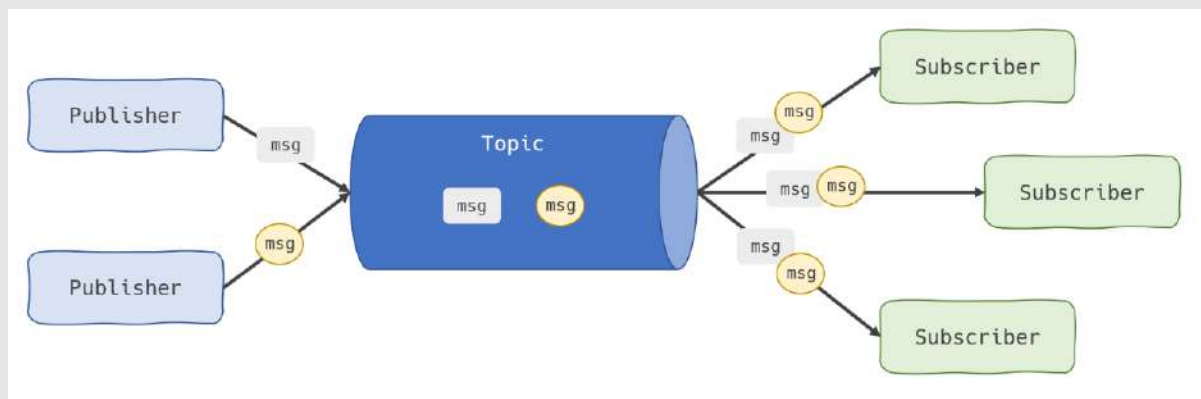
- Producers – It consists of client components that generate and send messages to the message queue.
- Consumers – It comprises server destination services that process messages from the queue.
- The queue – The central component of the architecture that stores messages. Messages placed in the queue are stored until the consumer retrieves them.



Reference: [What Is a Message Queue?](#)

218. System design 21: What is publisher-subscriber model and how does it differ from message queues?

The **Publisher Subscriber model**, also called **Pub Sub**, is a way for services to communicate without depending on each other directly. In this model, a **publisher** sends messages to a **topic** and **subscribers** receive those messages if they are interested in that topic. The publisher does not know who the subscribers are. The communication is loosely connected and flexible.



Example

Imagine a news system. A sports news service publishes updates to the "sports" topic. Users who subscribed to sports updates will receive those messages automatically. If someone subscribed to "technology", they will receive only tech news, not sports.

How it is different from Message Queues

Feature	Message Queue	Publisher Subscriber
Message delivery	One message goes to one consumer	One message goes to many subscribers
Flow style	Point to point	Broadcast style
Use case	Background tasks, job processing	Real time updates
Dependency	Consumer reads from a queue	Subscribers listen to topics
Order guarantee	Usually yes	Yes, Pub/Sub can provide message ordering, but it requires specific configuration like using ordering keys, publishing within the same region, and enabling the feature on the subscription,

Difference between pub-sub and message queue

Message queues and publish subscribe are both used to send messages between parts of a system, but they are used for different needs. Understanding this difference helps us design systems that behave correctly.

In a message queue, a message is meant to be processed by **one consumer** only. A producer sends a message to the queue; one message is processed by only one consumer in a consumer group and one consumer takes it and processes it. After the message is processed, it is removed from the queue. This model is useful when work must be done only once.

For example, in a food delivery app, when an order is placed, one worker service should prepare the order. The message goes into a queue, and one worker picks it up. No other worker processes the same order. This avoids duplicate work.

Publish subscribe works in a different way. A **producer publishes an event, and many consumers** can receive the same event. The publisher does not know who will listen. Each subscriber gets its own copy of the event and processes it independently.

For example, when a payment is completed, an event is published. The notification service sends a message to the user. The invoice service generates a bill. The analytics service updates report. All of them receive the same event.

Real-world example

- **Message Queue**
An e-commerce app places an order. The order service sends a message to a queue. Only one worker reads that message and processes the order.
- **Pub Sub**
A user uploads a video. The video service publishes a message "video_uploaded". Many services may need this information
 - Notification service sends a message to followers
 - Analytics service records upload data
 - Recommendation service updates video suggestionsAll of them receive the same message through Pub Sub.

Tools: Google Pub/Sub, Kafka topics, AWS SNS

219. System design 22: What is caching and how it works in distributed systems

Caching is a method used to store frequently used data in a fast storage layer so that it can be accessed quickly. The main goal of caching is to reduce the time taken to fetch data and to reduce the load on the main database or service.

Example: If many users keep checking the same product price on an online store, it is slow to fetch the price from the main database every time. Instead, we store the product price in a cache like Redis or Memcached. The next time someone requests that product price, it is returned from the cache in milliseconds.

In distributed systems, many servers work together. If each server goes to the main database for data, it creates heavy load and slows down the system. To avoid this, we add a **distributed cache** between the application and the database.

Here are the most popular caching tools used in distributed systems: Redis, Memcached, Hazelcast, Cloudflare CDN

220. System design 23: What is CDN, how they are useful?

A **Content Delivery Network (CDN)** is a network of servers placed in different parts of the world to deliver content to users faster. Instead of loading a file like an image, video or website script from one

main server, a CDN keeps copies of that content in many nearby servers. When a user opens a website, the CDN gives the content from the closest server. This reduces delay and improves speed.

Why CDN is useful

Without a CDN, every request goes to the main server. If the server is far from the user, loading becomes slow. If many users visit at the same time, the server may become overloaded. CDN solves both problems by sharing the load across many servers and serving data from a nearby location.

Example: Imagine our main server is in Mumbai. A user from the United States opens our website. Without CDN, the request travels all the way to Mumbai, waits for the response and then returns to the user. This takes time.

With a CDN, our website files are stored in a CDN server in New York. So when the same user opens the website, the content comes directly from New York instead of Mumbai. The experience becomes fast and smooth.

CDN Can store Images, Videos, CSS and JavaScript files. Website pages

Popular CDN providers Cloudflare, Amazon CloudFront, Google CDN

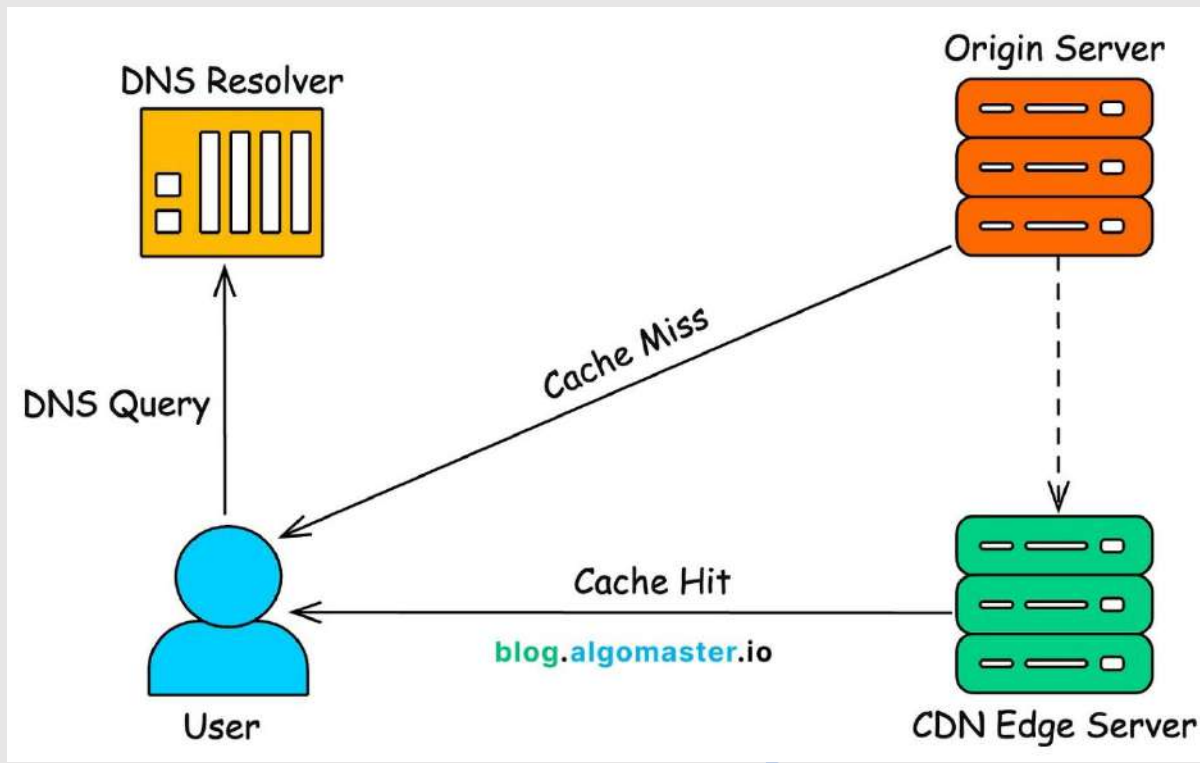
221. System design 24: How does CDN works?

A CDN operates using three key components:

- Edge Servers - Located at Points of Presence (PoP) locations, these servers cache and deliver content closer to users.
- Origin Servers - The primary servers where the original content is stored.
- DNS (Domain Name System) - Directs user requests to the nearest edge server instead of the origin server.

By leveraging edge servers distributed across multiple geographical regions, CDNs minimize latency and accelerate content delivery.

Here's a step-by-step breakdown of how a CDN works:



1. **User Request:** A user visits a website and requests content, such as an image, a webpage, or a video. This request must be resolved to a server that can serve the content.
2. **DNS Resolution:** The browser sends a DNS query to resolve the web content address (e.g., `https://cdn.example.com/images/logo.png`) into an IP address. The DNS return the nearest CDN edge server's IP address rather than the origin server.
3. **Cache Check**
 - If the content is already cached at the edge server, it is served immediately to the user.
 - If not, the edge server forwards the request to the origin server. The origin server processes the request and sends the content back to the edge server. The edge server caches the content it retrieved from the origin server.
4. **Subsequent Requests:** Once cached, future requests for the same content are served directly from the edge server, reducing load on the origin and improving speed.

CDNs use a Time-to-Live (TTL) mechanism to determine how long content remains cached before expiring. To ensure users always receive the latest version, CDNs periodically refresh and update cached content from the origin server.

Reference: [What is a Content Delivery Network?](#)

222. System design 25: How do you design an API, explain with example?

Designing an API means planning how two systems will talk to each other using requests and responses. A good API is simple to understand, easy to use and consistent.

Let us design a simple **User API** for an app. The app needs to:

- Create a user
- Get user details
- Update user information
- Delete a user

Step 1: We will use REST API, which uses normal HTTP methods like GET, POST, PUT and DELETE.

Step 2: Decide the base URL `https://api.myapp.com`

Step 3: Design endpoints clearly

Action	HTTP Method	Endpoint	Request Body	Response
Create user	POST	/users	JSON	New user created
Get user by id	GET	/users/{id}	None	User details
Update user	PUT	/users/{id}	JSON	Updated user
Delete user	DELETE	/users/{id}	None	Success message

Step 4: Define request and response format

```
POST /users
{
  "name": "Ravi",
  "email": "ravi@example.com",
  "age": 28
}
```

Response:

```
{
  "id": 101,
  "name": "Ravi",
  "email": "ravi@example.com",
  "age": 28,
  "createdAt": "2025-01-10"
}
```

Step 5: Handle errors A good API must handle errors clearly.

Step 6: Add validation Like Name must not be empty, Email must be valid, Age must be a number

Step 7: Make it secure Use HTTPS Add authentication (like API key or JWT token)

Step 8: Consider idempotency, pagination and rate limiting.

223. System design 26: What is the difference between Synchronous API and Asynchronous API?

A **Synchronous API** and an **Asynchronous API** both allow systems to communicate, but they work in different ways. The main difference is in how they handle waiting for a response.

In a **Synchronous (Sync) API**, the client sends a request and waits until it gets a response. It cannot continue its work until the response comes back. Sync APIs are simple to use, but if the response takes time, the client must wait.

Example

A user logs in to a website. The browser sends a request to the login API and waits for the response. Only after receiving "login successful" does it move to the next page. Login, payment and search APIs are usually synchronous because the result is required immediately.

An **Asynchronous (Async) API** works differently. The client sends a request and does not wait for the response. It can continue other tasks while the server finishes the work in the background. When the result is ready, it is sent back through a callback, event or message.

Example

A video upload in a social media app is an async process. When the user uploads a video, the app sends it to the server and does not wait for the video to finish processing. Instead, the server sends a message later saying "video is ready". Email sending, report generation and background processing are common async use cases.

224. System design 27: What are best practices while designing the APIs?

The first important practice is to keep the **API simple and meaningful**. Endpoints should clearly describe what they do. For example, if the API returns user details, the path should look like `/users/101`. It should not be something confusing like `/getUserInfoData`.

Another best practice is to use **HTTP methods** correctly. We must follow standard behaviour: use GET for fetching data, POST for creating, PUT for updating and DELETE for removing. When a new team member joins, they should be able to understand the API by just looking at the method.

Good APIs **return clear messages**. If something goes wrong, the API must explain the issue in a human readable way. A simple error like 400 Bad Request is not enough.

```
{
  "error": "Email is required",
  "status": 400
}
```

Validation must always be done at the API level. Data should not enter the system without checking.

Security is not optional. APIs should use HTTPS and must protect sensitive actions using authentication tokens or API keys.

Versioning is another best practice that helps when the API grows. By using versions like /v1/users and /v2/users we can upgrade the API while keeping old versions safe for existing users

Finally, every good API must have proper **documentation**. It explains how to use each endpoint, what data is required and what responses can be expected.

225. System design 28: What is polling, its types and when to use each?

Polling is a way for a client to check if new data is available on the server by sending requests repeatedly. Instead of waiting for the server to notify, the client keeps asking at regular time intervals. Polling is simple and works with normal HTTP, but it may cause unnecessary network traffic if not used carefully.

Types of Polling

1. Regular/Short Polling: In regular polling, the client sends a request to the server at fixed time intervals, like every 5 seconds, even if no new data is available.

Example: A dashboard checks server health every 5 seconds to show live CPU usage.

When to use: Use regular polling when updates are not frequent but real time is not critical. Simple systems and small traffic benefit from it.

2. Long Polling: In long polling, the client sends a request and waits until the server has new data. If there is no new data, the server keeps the request open for some time instead of responding immediately. When new data is available, the server sends the response.

Example: A chat application using long polling waits for new messages and gets them as soon as they are available.

When to use: Use long polling when real time updates are needed but WebSockets are not available. It reduces unnecessary requests.

3. Webhook-based Polling (Hybrid)

In this method, polling is used with callbacks. The client polls only to confirm that an event has been processed and uses a webhook to receive main updates. This reduces both delay and traffic.

Example: Payment systems use webhook-based polling. After a payment, the system receives a webhook, and polling is used only to check final payment confirmation.

When to use: Use this when accuracy is important and duplicate requests must be avoided.

226. System design 29: What are Webhooks and how they are useful?

A **webhook** is a way for one system to send information to another system automatically when something happens. The data is sent immediately, without waiting for the other system to ask for it.

Normally, APIs work in a request and response way. We call an API again and again to check if something has changed. This is slow and wastes resources.

Webhooks work differently. When an event happens, the system sends the data directly to a given URL. There is no need to keep checking.

For example, when a payment is completed, the payment system sends a webhook to our application. Our system receives the data instantly and updates the order status.

In short, APIs pull data. Webhooks push data when an event occurs.

Razorpay example

Let us say we are building an online course website. A student pays using Razorpay. The payment happens on Razorpay servers, not inside our application. So how do we know when a payment is successful?

We ask Razorpay to inform us when payment is done. That notification is sent using a webhook.

How webhook works in Razorpay

1. We create a backend endpoint in our app to receive webhook events
Example endpoint: `https://myapp.com/webhook/payment`
2. We go to Razorpay Dashboard -> Webhooks -> Add new webhook. We give the above URL there. Now Razorpay knows where to send payment updates.
3. A student makes a payment on our site, Razorpay processes the payment.
4. Razorpay sends a POST request to our webhook URL. Razorpay sends payment status like this:

```
{
  "event": "payment.captured",
  "payload": {
    "payment": {
      "entity": {
        "id": "pay_12345",
        "order_id": "order_9876",
        "status": "captured",
        "amount": 154
      }
    }
  }
}
```

5. Our backend reads this data. If status is "captured", we mark the order as Paid.

In short, **webhooks are a powerful and efficient way for web applications to communicate in real-time**, eliminating the need for constant polling.

Reference: [What Is a Webhook \(and How Does It Work\)? | Twilio](#)

227. System design 30: What is WebSocket and how they are useful?

A **WebSocket** is a way for a client and server to communicate with each other in real time over a single, long-living connection.

WebSockets are a powerful communication protocol that enables bidirectional, real-time communication between a client and a server over a single TCP connection.

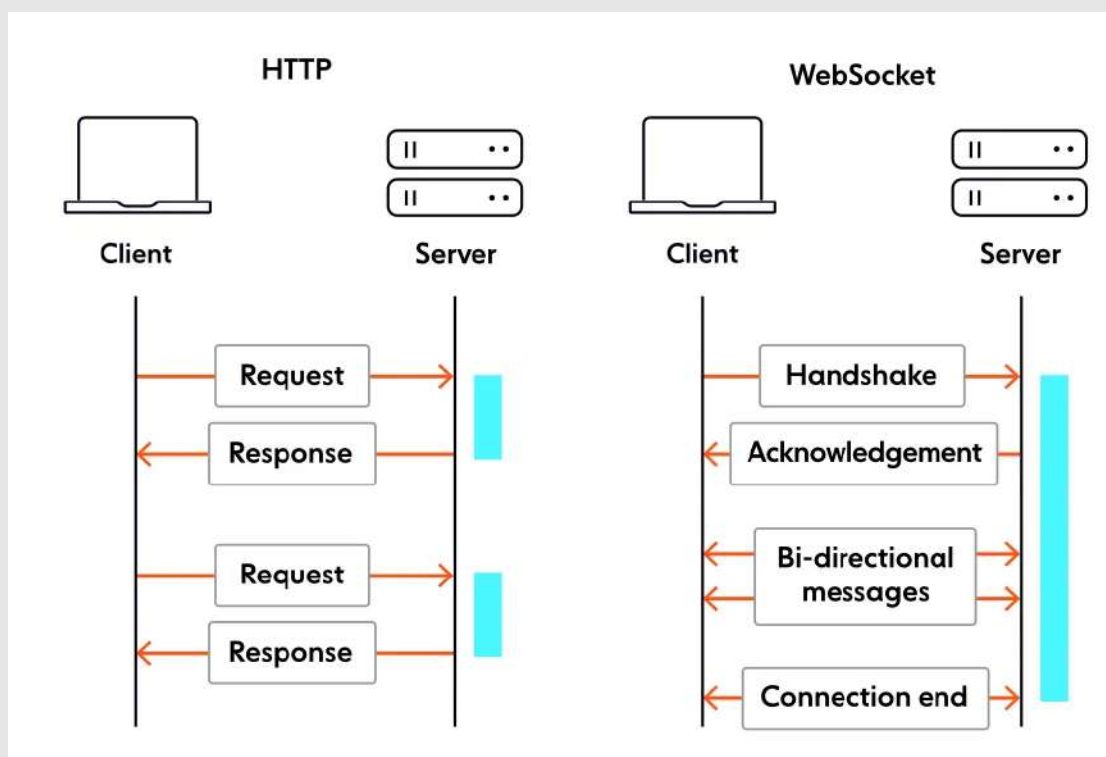
In normal HTTP, only the client can send a request first. The server cannot send data on its own. If the client needs live updates, it must keep sending new requests again and again, which is slow.

Example:

WhatsApp Web uses WebSocket to keep the chat connected in real time. When the browser connects to WhatsApp servers, a WebSocket connection is created and kept open. This connection allows **two-way communication**, which means:

- We can send a message to a friend
- We can receive messages instantly without reloading
- Message status updates like **sent, delivered, blue tick** come in real time
- Typing status like “Ravi is typing...” works live
- Online/offline status is also updated instantly

For In depth: [Understanding Websockets In depth | by vishal rana | Medium](#)



228. System design 31: What is the Server-Sent Event and how is it useful?

Server-Sent Events (SSE) is a technology that allows a server to send data to a web client in real-time over a single, persistent HTTP connection. It is a unidirectional protocol, meaning data only flows from the server to the client.

In some applications, we need data updates continuously from the server but the client does not need to send anything back. SSE is perfect for this one-way communication. It uses less resources compared to other methods and is easy to implement.

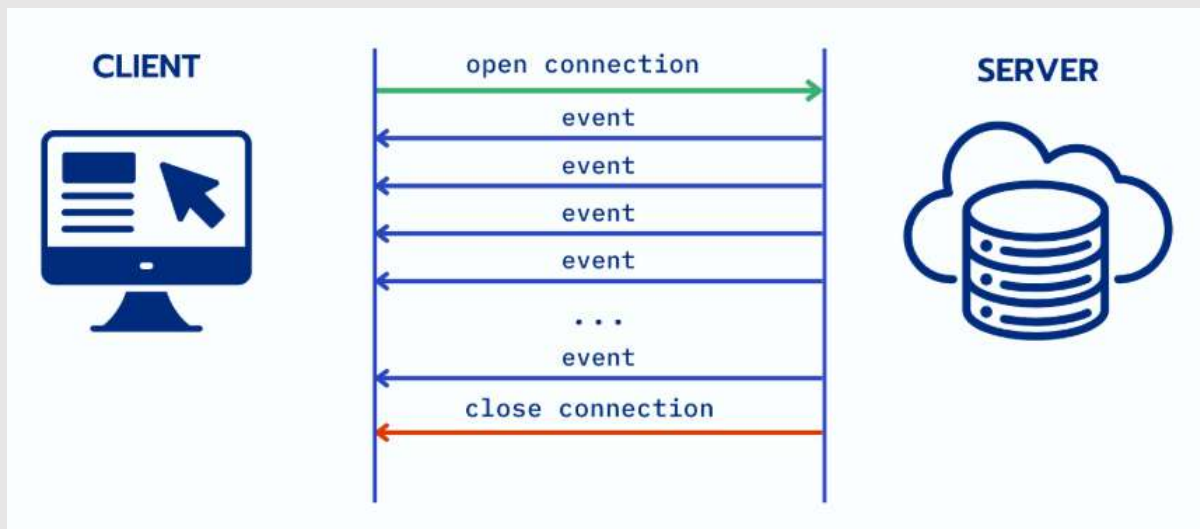
Example

Imagine a live cricket score website. Every time the score changes, the server should send the update to all users automatically. If we use normal HTTP, we must keep calling the server again and again to check for new scores. This is slow and wasteful.

With SSE, once the user opens the website, a single connection is created to the server. Every time the score changes, the server pushes the new score to the browser immediately like this:

```
India 142/3 in 16.2 overs  
India 148/3 in 17 overs  
India 160/4 in 18.5 overs
```

The score updates automatically in real time without any refresh. This is exactly what SSE is built for.



Examples: Stock price updates, Live weather updates

Reference: [What is Server-Sent Events \(SSE\) and how to implement it? | by Gökhan Ayrancıoğlu | Delivery Hero Tech Hub | Medium](#)

229. System design 32: When to use Webhook, WebSocket and Server-sent events?

When to use Webhook

Use a webhook when one system needs to notify another system automatically when an event happens. It is best for backend-to-backend communication and event-driven workflows. It works even when systems are not always connected. Common use cases include payment status updates, order confirmations and external system callbacks.

When to use WebSocket

Use WebSocket when real time two-way communication is required. Both client and server can send messages to each other instantly. It is best for live chat apps, multiplayer games.

When to use Server-Sent Events (SSE)

Use SSE when the server needs to send continuous data to the client in real time and the client only listens. It is simple to implement and works over HTTP. Best for live dashboards, score updates, notifications and streaming data where only server-to-client flow is needed.

230. System design 33: WebSocket implementation in Spring

Code link: [Spring-Samples/websocketdemo/websocketdemo at main · CodingLyf-Fullstack/Spring-Samples](https://github.com/CodingLyf-Fullstack/Spring-Samples/tree/main/websocketdemo/websocketdemo)

Explanation:

Step1: Add Maven Dependency

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-websocket</artifactId>
</dependency>
```

Step2: Configure WebSocket in Spring Boot

- Create a configuration class to configure WebSocket in your Spring Boot application and use `@EnableWebSocket` annotation.

```
@Configuration
@EnableWebSocket
public class WebSocketConfig implements WebSocketConfigurer {

    @Override
    public void registerWebSocketHandlers(WebSocketHandlerRegistry registry) {
        registry.addHandler(new WebSocketHandler(),
            "/websocket").setAllowedOrigins("*");
    }
}
```

Step 3: Handle WebSocket Messages

- Define methods in your WebSocket handler class to handle various types of WebSocket messages (e.g., text, binary).
- Use these methods to process incoming messages and send responses back to clients as needed.

```
public class WebSocketHandler extends TextWebSocketHandler {

    private static Set<WebSocketSession> sessions = new HashSet<>();

    @Override
    public void afterConnectionEstablished(WebSocketSession session) throws Exception {
        sessions.add(session);
    }

    @Override
    protected void handleTextMessage(WebSocketSession session, TextMessage message)
    throws Exception {
        for (WebSocketSession webSocketSession : sessions) {
```

```

        if (websocketSession.isOpen()) {
            try {
                websocketSession.sendMessage(message);
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    }
}
}
}
}

```

We can use different classes for handling different types of WebSocket messages. *TextWebSocketHandler* class as it is specifically designed for handling text based WebSocket messages. However, Spring provides other handler classes for handling different types of messages:

- *BinaryWebSocketHandler*: used for handling binary messages
- *AbstractWebSocketHandler*: a generic class used for handling both binary and text messages

On terminal,

- Navigate to your Project directory
- Build the project using mvn clean install
- Run, mvn spring-boot: run

We can use postman to test, check the video in this link.

Reference: [Real-Time Communication with WebSocket in Spring Boot](#) | by Anjali Chaudhari | Medium

For chat application sample: [WebSocket + Spring Boot: Build a Real-time, Bidirectional Applications](#) | Simform Engineering

231. **System design 34: Server-sent events Implementation** (Optional)

If you want to do sample follow below links

[Implementing SSE in Spring Boot](#) | Medium

[Server-Sent Events in Spring Boot](#) | by Mangesh More | Globant | Medium

232. **System design 35: What is proxy server, proxy types and when to use each?**

A proxy server refers to a server that acts as an intermediary between client requests and a particular server. The basic purpose of **Proxy** servers is to protect the direct connection of Internet clients and Internet resources

let me explain it with an example: Imagine you're in a foreign country and don't speak the local language. Your tour guide (the proxy) takes your requests in English, translates them to the local language for shopkeepers or restaurant staff, and then translates their responses back to English for you.

A **proxy** can work in two ways: **forward proxy** or **reverse proxy**. Both sound similar but solve different problems.

Forward Proxy?

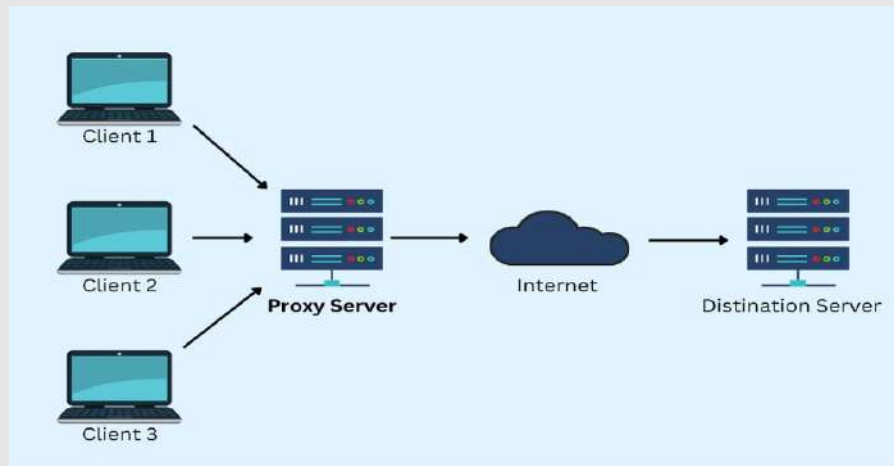
A forward proxy sits between a client and the internet. It hides the identity of the client and makes requests on its behalf. It is used mostly by users inside a private network.

Example:

A company may use a forward proxy so that all employees access the internet only through it. The proxy can control what websites can be opened and also hide internal IP addresses for privacy.

When to use:

- To control or filter outgoing traffic
- To add security and hide user identity

**Reverse Proxy?**

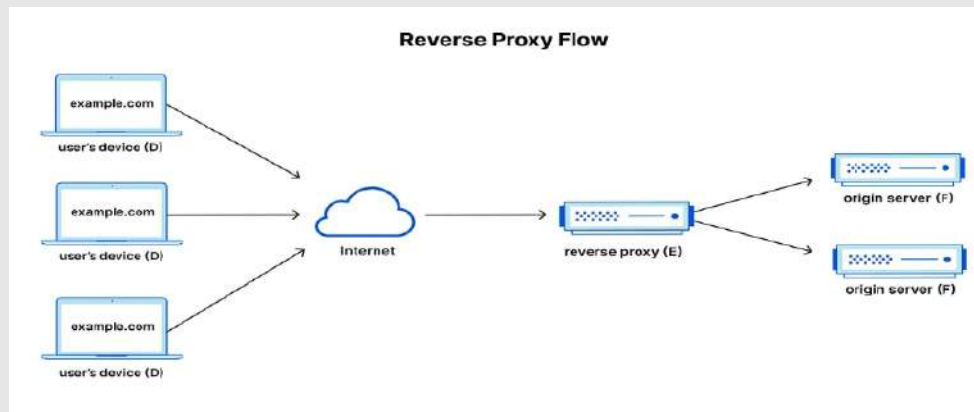
A reverse proxy sits in front of servers and receives requests from clients. It decides which server should handle the request. The client does not know which server is serving the request. It protects backend servers and helps distribute load.

Example:

If an application is hosted on three servers, a reverse proxy like **Nginx** or **HAProxy** receives traffic and forwards it to different servers. It improves performance and prevents direct access to backend systems.

When to use:

- For load balancing
- To improve security of backend servers
- For caching and faster performance
- To route traffic to different services



Reference: [Forward Proxy vs Reverse Proxy. Proxies explained in 5 minutes! | by Ishaan Gupta | Level Up Coding](#)

233. System design 36: What are authentication mechanisms we need to consider while designing the systems.

Note: In-depth security related questions along with scenarios and implementations (30+ question on Spring security) are covered in Java & Spring knowledge boost

Link: <https://topmate.io/codinglyf/1746709>

Authentication is the first line of security in any system. Its purpose is simple: make sure the person or application trying to access something is real and trusted. Without authentication, any user could log in as anyone, and any unknown system could talk to our servers. Different situations need different authentication methods. Let us understand them one by one

1. Username and Password

This is the most basic form of authentication. A user creates an account using a username and a password. When they try to log in, the system checks if the password is correct.

But to keep it safe, the password must not be stored as plain text. It should be stored as a hashed value so no one can read it even if the database is leaked.

Example: Login into Gmail, Instagram or Netflix using email and password.

2. Session Based Authentication

This method is used in websites that maintain a login session. After the user logs in, the server stores session details in memory and gives a session ID to the browser as a cookie. The browser sends this session ID with every request so the server knows who is active.

Example: While shopping on Amazon, we log in once and stay logged in across all pages until we log out.

3. Token Based Authentication (JWT)

Token authentication is used mostly in modern web and mobile apps. After **login**, the server gives a token to the user. This token is then used for future requests instead of sending the password again. One common token format is **JWT (JSON Web Token)**. It is small, secure and easy to use.

Example: After Login into a mobile banking app, we stay logged in using a token.

4. API Key Authentication

API keys are used mainly when one system talks to another system. An API key is a secret value that is sent in the request to prove identity.

Example: When a backend service calls Razorpay or Google Maps API, it includes an API key to prove it is allowed to use that service.

5. OAuth 2.0

OAuth allows users to log in without creating a new account. Instead, they can log in using another trusted service like Google or Facebook. OAuth never shares passwords. It only shares permission in a safe way.

Example: Login into Spotify or Zomato using Google account.

6. Multi-Factor Authentication (MFA)

MFA adds an extra step to make login safer. Along with the password, another verification is required like OTP, fingerprint or email code. Even if someone knows the password, they cannot get in without the second step.

Example: Login into a bank account requires password + OTP.

7. Single Sign-On (SSO)

SSO allows users to log in once and access many applications without logging in again. It is used mostly in companies for employee logins.

Example: An employee signs in once in the company portal and then uses HR, payroll and email tools without Login again.

234. **System design 37: What is scalability and why it is important?**

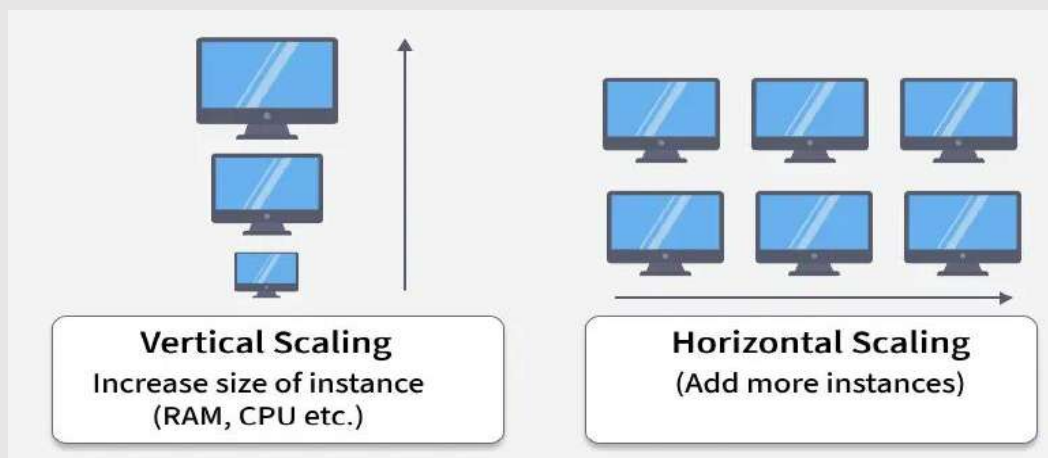
Scalability means the ability of a system to handle growth without breaking or slowing down. Growth can come in many forms: more users, more traffic, more data or more features. A scalable system is built in a way that it can expand smoothly when demand increases.

When a system is small, everything looks easy. But as more users join and traffic grows, the system begins to struggle. Pages load slowly, features fail and users get frustrated. If the system cannot scale, it loses reliability and trust. Businesses that cannot scale lose customers to competitors.

Example: Let us say we launch a new social media app. On day one, only 100 people use it. One server is enough. But what if the app becomes popular and 10 lakh users join in a month? If the system is not scalable, it will crash. To support growth, we must add more servers, balance traffic and optimize storage. This is scalability in software systems.

235. System design 38: What is Horizontal Scaling and Vertical Scaling

Horizontal scaling and vertical scaling are two ways to handle growth when a system gets more users, more traffic or more data. Both methods help improve performance, but they work in different ways.

**Vertical Scaling**

Vertical scaling means increasing the power of a single machine. We upgrade the same server by adding more CPU, more RAM or faster storage. I

Example:

Let us say our backend server is slow because it has only 4GB RAM and 2 CPU cores. If we upgrade it to 16GB RAM and 8 CPU cores, it can handle more users and run faster. This is vertical scaling

Horizontal Scaling

Horizontal scaling means adding more machines instead of upgrading one machine. Traffic is shared among these machines.

Example:

If one server can handle 1000 users and traffic increases to 5000 users, instead of upgrading the machine, we add 4 more servers. Now each server handles 1000 users and the load is balanced.

Conclusion: Vertical scaling is good in the beginning because it is easy. But as a system grows, horizontal scaling becomes the better long-term solution. Most large companies like Amazon, Netflix and Google use horizontal scaling to handle millions of users smoothly.

Reference: [Scalability — Vertical or Horizontal Scaling when Designing Architectures | by Mehmet Ozkaya | Design Microservices Architecture with Patterns & Principles | Medium](#)

236. System design 39: What are the advantages and disadvantages of Vertical Scaling**Advantages of Vertical Scaling****1. Easy to implement**

Vertical scaling is straightforward. We do not need to change the system architecture. We simply upgrade the existing server. For example, increasing from 8GB RAM to 32GB RAM is quick and simple.

2. No code changes needed

Since everything still runs on the same machine, the application behaves the same way. There is no need to change application logic or handle distributed systems.

3. Good for small systems

For early-stage projects or low traffic applications, vertical scaling is enough. It gives a fast and cheap performance boost in the beginning.

Disadvantages of Vertical Scaling**1. Limited growth**

Every machine has a maximum limit. At some point, we cannot add more CPU or RAM. When that limit is reached, we must switch to horizontal scaling.

2. Single point of failure

If this one powerful machine goes down, the whole system stops. There is no backup server to share the load or replace it.

3. Becomes expensive

High performance servers cost a lot of money. It is often cheaper to use multiple small servers instead of one very powerful machine.

237. System design 40: What are the advantages and disadvantages of Horizontal Scaling?**Advantages of Horizontal Scaling****1. High scalability**

We can keep adding more servers as traffic grows. There is no practical limit. This makes it suitable for large platforms like Amazon, Netflix and YouTube.

2. Better reliability

If one server fails, other servers continue to run. The system does not go down completely. This improves uptime and makes the system more dependable.

Disadvantages of Horizontal Scaling**1. More complex setup**

It requires a load balancer to distribute traffic and data management becomes harder. We must handle caching, replication and consistency carefully.

2. Requires code and architecture changes

The application must support distributed processing. Sessions, logging and data sharing must be designed for multiple servers.

3. Challenging data consistency

When data is spread across many nodes, keeping it correct and in sync becomes harder. We must deal with problems like replication lag and eventual consistency.

238. System design 41: Latency vs Throughput

Latency:

Latency is the **time taken to process a single request**. It tells how fast a system responds. Lower latency means faster response time.

Example: When we click on a product in Amazon, the time it takes to show the product details is latency. If it loads in 200 milliseconds, that is good latency. If it takes 5 seconds, that is poor latency.

Throughput:

Throughput means **how many requests a system can handle in a given amount of time**. It measures the overall capacity of the system. Higher throughput means the system can serve more users at the same time.

Example: If Flipkart can handle 10,000 orders per minute during a sale, that is the throughput of the system. A system with low throughput will slow down or fail when too many users try to access it.

Why it matters in system design? When building systems, we must balance both latency and throughput. Users want fast responses and businesses want to handle many users at once. To improve latency, we use caching and faster databases. To improve throughput, we use load balancing and horizontal scaling.

Reference: [Latency vs. Throughput: Striking the Right Balance in System Design | by Prajwal Ahluwalia | Medium](#)

239. System design 42: What is Extensibility and why it is important?

Extensibility means designing a system in such a way that it can grow and support new features in the future without needing major changes. A system that is extensible is easy to improve when new requirements come. We add new parts to it instead of breaking or rewriting the old parts.

In real projects, requirements always change. A system that is not extensible becomes hard to maintain. Every new feature creates problems. But an extensible system allows us to add new features without disturbing the existing ones. This keeps development fast and safe.

Example:

Imagine we build a simple payment system that supports only credit card payments. Later, the business wants to add UPI payments, net banking and wallets. If the system is not extensible, we must modify the main code again and again. This increases risk and can cause bugs.

But if we follow extensible design, we create a clean structure where each payment method is a separate module. When a new method like UPI is added, we plug it in without touching old code.

240. System design 43: What is maintainability and why it is important?

Maintainability means how easy it is to understand, fix and improve a system over time. A system is not successful just because it works today. It must also be easy to manage tomorrow when

new features are added, bugs appear or performance needs to improve. Maintainability keeps a system healthy for long-term use.

In real projects, code lives for many years. Teams change; business needs change and features keep growing. If the system is hard to maintain, every small change will take more time, cost more money and increase chances of breaking something. A system with poor maintainability becomes slow, risky and painful to work with.

Example: If we build an online shopping system and write all business logic in one file, it may work in the beginning. But later, when we add features like coupons, EMI payments or refunds, it becomes difficult to change anything. Small changes will break old features. This is poor maintainability.

Instead, if we divide the system into modules like order service, payment service and user service, future updates become easier. That is maintainable design.

241. **System design 44: What is consistent hashing?**

In a distributed system, many servers work together. We often need to decide which server should handle a request, a user session or a cache entry. A simple method like $\text{hash}(\text{key}) \% \text{numberOfServers}$ seems easy, but it has a big problem. Whenever a server is added or removed, almost everything gets reassigned to new servers, which causes instability and overload.

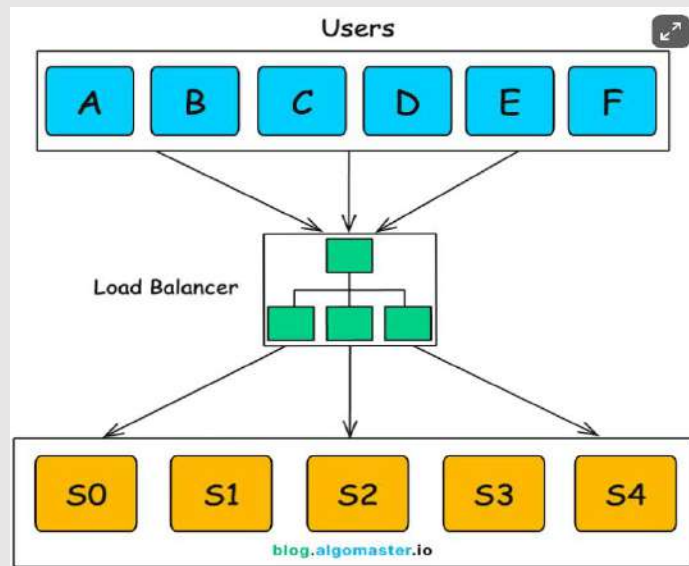
Consistent hashing fixes this problem by **reducing the amount of reassignment**. When servers change, only a small portion of the work is moved. Most of the system continues to run smoothly.

Consistent hashing is a way to distribute work across multiple servers so that the system remains stable even when servers are added or removed. It is not only used for storing data but also for sharing load, routing traffic and managing distributed systems without causing big changes.

Video link: [\(2\) 6 \) What is Consistent Hashing | Virtual Node | System Design Basics - YouTube](#) (Before going further, watch this video, nicely explained)

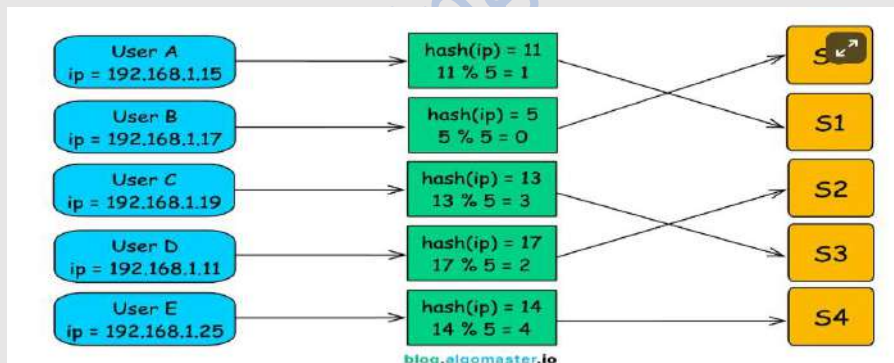
It might be little confusing, Let's understand clearly with example:

The system consists of **5 backend servers** (S0, S1, S2, S3, S4), and requests are assigned using a hash function that maps each user's **IP address to a specific server**.



The process works like this:

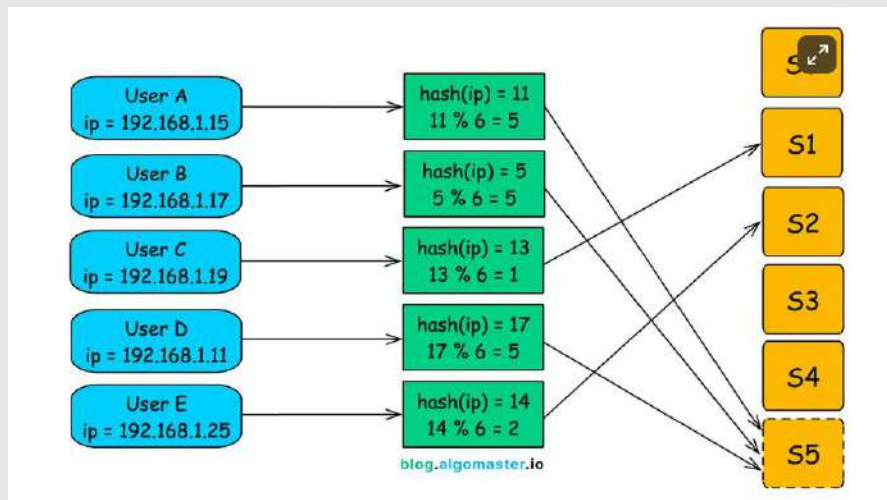
1. The load balancer takes a user's IP address (or session ID).
2. A **hash function** maps the IP to one of the backend servers by taking the **sum of bytes in the IP address** and mod 5 [(Sum of bytes in the IP Address) % 5] (since we have 5 servers).
3. The request is **routed to the assigned server**, ensuring that the same user is always directed to the same server for session consistency.



This approach works **as long as the number of servers remains constant**. But what happens when you **add or remove a server**?

Scenario 1: Adding a New Server (S5)

As traffic increases, you decide to **scale up** by adding a new backend server (S5). Now, the hash function must be modified to use mod 6 instead of mod 5 since we have 6 servers now.

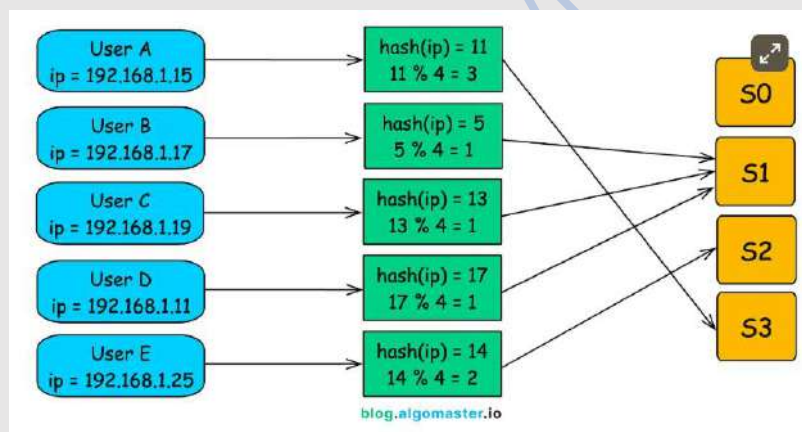


This seemingly simple change completely disrupts the existing mapping, causing most users to be reassigned to different servers.

This results into massive rehashing, leading to high overhead, and potential downtime.

Scenario 2: Removing a Server (S4)

Now, let's say one of the servers (S4) fails or is removed. The number of servers drops to 4, forcing the hash function to switch from mod 5 to mod 4.



Even though only one server was removed, most users are reassigned to different servers. This can cause:

- **Session Loss:** Active users may be logged out or disconnected.
- **Cache invalidation:** Cached data becomes irrelevant, increasing database load.
- **Severe performance degradation:** The system may struggle to run efficiently.

The Solution: Consistent Hashing

Consistent hashing offers a more scalable and efficient solution by ensuring that only a small fraction of users is reassigned when scaling up or down.

It performs really well when operated in dynamic environments, where the system scales up and down frequently.

Consistent hashing is a **distributed hashing technique** used to efficiently distribute data across multiple nodes (servers, caches, etc.).

It uses a **circular hash space** (hash ring) with a large and constant hash space.

Both nodes (servers, caches, or databases) and keys (data items) are mapped to positions on this hash ring using a **hash function**.

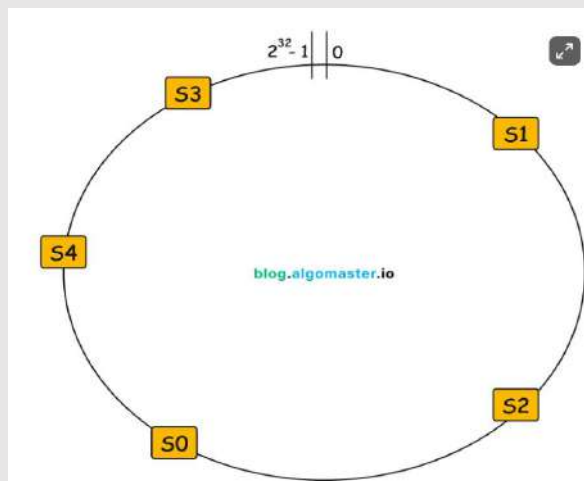
Instead of distributing keys based on $\text{Hash}(\text{key}) \bmod N$, consistent hashing places both servers and keys on a circular hash ring.

Defining the Hash Space

- We use a large, fixed hash space ranging from 0 to $2^{32} - 1$ (assuming a 32-bit hash function).
- This creates a circular structure, where values wrap around after reaching the maximum limit.

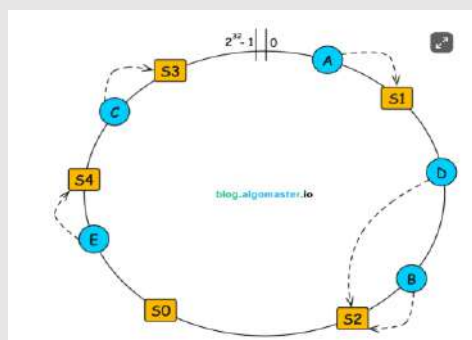
Placing Servers on the Ring

- Each **server (node)** is assigned a position on the hash ring by computing $\text{Hash}(\text{server_id})$.
- Using the above example with **5 servers (S0, S1, S2, S3, S4)**, the hash function distributes them at different positions around the ring.



Mapping Keys to Servers

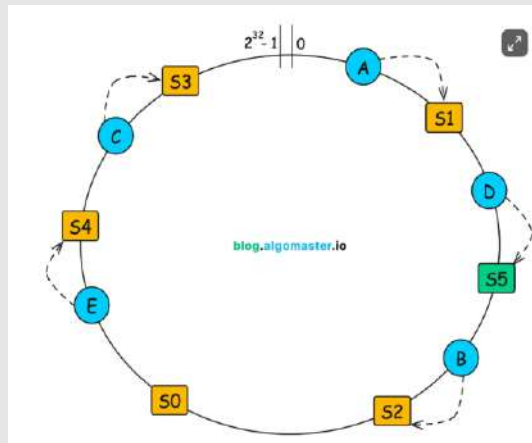
- When a key (or request) is added, its position is determined by computing $\text{Hash}(\text{key})$.
 - Example: a user's request is assigned a position on the ring based on the hash of their IP address: $\text{Hash}(\text{IP Address})$
- We then move clockwise around the ring until we find the next available server.
- The key (or request) is assigned to this server.



1. Adding a New Server

Suppose we add a **new server (S5)** to the system.

- The position of S5 falls between S1 and S2 in the hash ring.
- S5 takes over all keys (requests) that fall between S1 and S2, which were previously handled by S2.
 - **Example:** User D's requests which were originally assigned to S2, will now be redirected to S5.



This demonstrates how consistent hashing efficiently redistributes keys with minimal disruption, ensuring that only a small subset of keys are reassigned when new servers are added.

2. Removing a Node

When a server, such as S4, fails or is removed from the system:

- All keys previously assigned to S4 are reassigned to the next available server in the ring (S3).
- Only the keys (requests) that were mapped to S4 need to move, while all other keys remain unaffected.

This results in **minimal data movement**, unlike traditional hashing where removing a node would require reassigning most keys.

Reference: [Consistent Hashing Explained - by Ashish Pratap Singh](#)

242. System design 45: Where Consistent hashing is used?

Consistent hashing is used in many places:

- Load Balancers
To distribute user requests among multiple servers in a stable way.
- Session Management
To ensure the same user always connects to the same backend server.
- Distributed Caching
In Redis Cluster or Memcached to spread cached keys across nodes.
- Distributed Databases
In systems like Apache Cassandra and DynamoDB to divide data into partitions.

- Message Queues
To route messages to specific partitions or workers.

243. **System design 46: What is the problem of consistent hashing and how it solved??**

Consistent hashing is powerful, but it also has a problem. It does not distribute load evenly across servers when the number of servers is small. Some servers may get too much traffic while others get very little. This creates **hot spots**, where one server becomes overloaded while others stay idle.

Why this happens

Consistent hashing places servers and requests on a hash ring using a hash function. But hash functions do not always spread things evenly. If the ring has only a few servers, many requests may fall near the same server. That server ends up doing most of the work while others are underused.

Example: Imagine a system with 3 servers handling users. We hash user IDs to decide which server handles which user. If by chance 70% of users map to Server 2, then it will carry most of the load. It may slow down or fail, even though Server 1 and Server 3 are free. This imbalance is the main problem of consistent hashing.

How it is solved

The solution is Virtual Nodes (also called VNodes).

Instead of placing each physical server only once on the hash ring, we place it multiple times using virtual positions. This spreads the keys more evenly across the ring.

244. **System design 47: What is Serverless architecture?**

Serverless architecture is a way of building applications without managing servers. The word serverless does not mean there are no servers. Servers are still used, but the cloud-provider manages them for us. We focus only on writing code, and the cloud takes care of everything else like server setup, scaling, and maintenance.

Why is it called Serverless?

In traditional systems, we set up servers, install software, and maintain them. In serverless, we do not worry about any of that. We write small pieces of code called functions, and these functions run only when needed. So from our view, it feels like there are no servers to manage.

How it Works

In serverless, our code runs in response to events. An event can be a user request, a file upload, a message, or a scheduled job. When an event happens, the cloud provider runs our function automatically.

Popular serverless platforms:

- AWS Lambda
- Google Cloud Functions
- Azure Functions

When to Use Serverless

Serverless works well when we want to build fast, handle changes in load, and pay only for actual usage.

Event Driven Applications

We use serverless when actions happen based on events like uploads or clicks, and logic runs only when needed.

Example: When a user uploads an image, a function runs, resizes it, saves it, and then stops automatically.

Microservices Architecture

We use serverless to split one big application into small services that work independently and talk through APIs.

Example: Login, payment, and notification services run as separate functions and scale independently based on traffic.

When to Avoid Serverless

Serverless is not always the right choice. In some cases, other approaches work better and cost less.

High and Predictable Traffic

If traffic is constant and heavy every day, fixed servers are cheaper and easier to manage than serverless.

Long Running Processes

When a task runs for a long time like data migration, serverless limits can stop it midway.

High Throughput Applications

Apps needing continuous connections like chat systems work better with servers that stay alive all the time.

Reference: [Serverless Architecture: Pros, Cons, and Use Cases | by Dave Patten | Medium](#)

245. System design 48: What is stateless and stateful architecture?

What is Stateless Architecture?

In a stateless architecture, the server does not store any information about the client between requests. Each request is treated as a new request, without any memory of previous actions. If any data is needed, it must be sent again with the request.

Example: Let us take a login session. In a stateless design, once a user logs in, the server does not remember it. So, to identify the user in later requests, we send a token with every request. This token confirms who the user is. The server remains simple and does not store any session data.

Example: REST APIs are stateless. Each request must carry all required data.

- A weather service that takes a location and returns the temperature.
 - The service does not store what the user searched earlier.
 - It gets the location, fetches the temperature and sends it back.
 - Nothing is saved in the service memory.
 - Stateless services keep the system light and fast.
1. **Microservices Architecture:** Each service handles requests independently, relying on external databases or caches instead of maintaining session data.
 2. **Public APIs (REST, GraphQL):** Clients send tokens with each request, eliminating the need for server-side sessions.

What is Stateful Architecture?

In a stateful architecture, the server remembers the user or process. It stores session data at the server side. Each request depends on previous requests.

A **stateful service** remembers some information between requests. It keeps the state either in its own memory or in a session tied to a specific instance.

Once the service starts storing state, the next request from the same user often needs to come back to the same instance. If it goes to another instance, the new instance will not know what happened earlier unless the state is stored somewhere else.

Example: When a user logs in to an online banking system, the server creates a session and keeps user details in memory. The next time the user sends a request, the server checks the session and continues from where the user left.

Example: Messaging Apps (WhatsApp, Slack) – Maintains active user sessions and message history for real-time communication.

Video Streaming Services (Netflix, YouTube) – Remembers user watch progress, recommendations, and session data for a seamless experience.

246. System design 49: Common patterns in stateless architecture?

1. Token-Based Authentication (JWT)

A very popular way to implement statelessness is through tokens, particularly **JWTs** (JSON Web Tokens):

1. **Client Authenticates Once:** The user logs in using credentials (username/password) for the first time, and the server issues a signed **JWT**.
2. **Subsequent Requests:** The client includes JWT token in each request (e.g., Authorization: Bearer <token> header).
3. **Validation:** The server validates the token's signature and any embedded claims (e.g., user ID, expiry time).
4. **No Server-Side Storage:** The server does **not** need to store session data; it just verifies the token on each request.

Many APIs, including OAuth-based authentication systems, use JWTs to enable stateless, scalable authentication.

2. Idempotent APIs

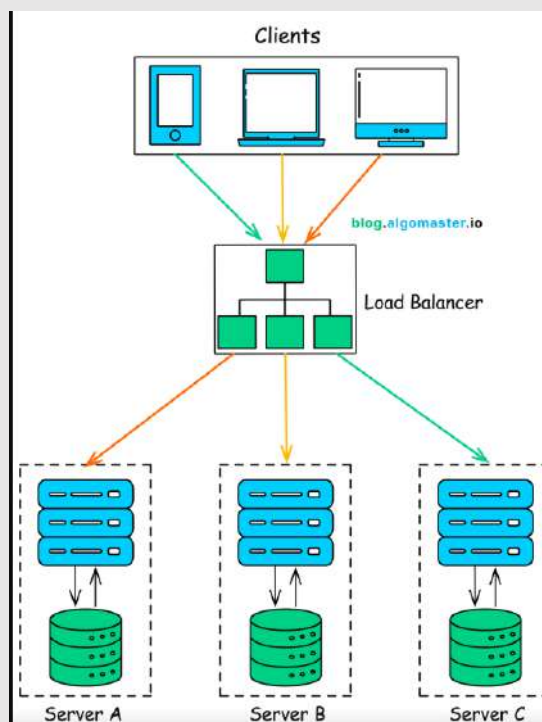
Stateless architectures benefit from **idempotent operations**, ensuring that repeated requests produce the same result. This prevents inconsistencies due to network retries or client errors.

Example: A PUT /users/123 request with the same payload **always** updates the user's data but doesn't create duplicates.

247. System design 50: Common patterns in stateful architecture?

1. Sticky Sessions

If you use **in-memory** session storage (i.e., each app server keeps its own sessions locally), you can configure your load balancer for “sticky sessions.”

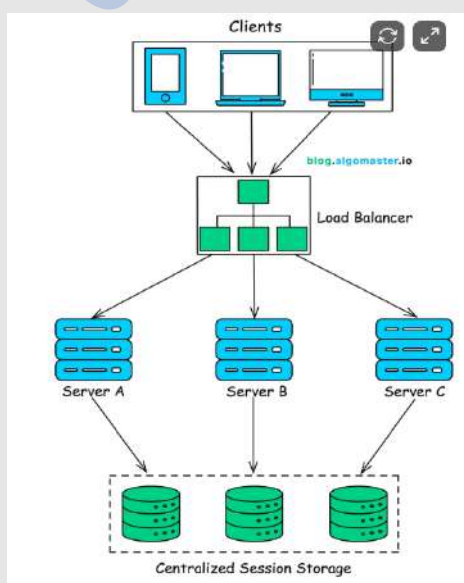


This means: Once a client is assigned to **Server A**, all subsequent requests from that client are routed to **Server A**.

Trade-Off: If Server A fails, the user’s session data is lost or the user is forced to re-log in. Sticky sessions are also less flexible when scaling because you can’t seamlessly redistribute user traffic to other servers.

2. Centralized Session Store

A more robust approach is to store session data in a **centralized** or **distributed** store (e.g., Redis).



This allows:

- **Shared access:** All servers can access and update session data for any user. Any server can handle any request, because the session data is not tied to a specific server’s memory.

Trade-Off: You introduce network overhead and rely on an external storage. If the centralized storage fails, you lose session data unless you have a fallback strategy.

248. System design 51: When to use stateless and stateful architecture?

We use a **stateless architecture** when each request from a client can be handled independently without needing to remember past actions. The server does not store any session information. This makes the system simple, fast and easy to scale across many servers because no server depends on stored memory. It is ideal for services where every request carries all the information needed to process it.

For example, a weather API or a product search API does not need to remember who made the last request. Each request is processed on its own, so stateless design works best.

A **stateful architecture** is used when a system must remember information between requests. The server keeps track of user activity and stores session data so that future actions can continue from where they left off. This design is necessary for tasks that happen step by step and depend on earlier progress.

For example, in online banking, a money transfer cannot happen in a single request. First the user logs in, then selects an account, enters an amount, confirms with OTP and only then completes the transfer. The server must remember the user's session during this flow, so a stateful design is required.

In short, we choose stateless architecture for high performance systems where each request is independent, and stateful architecture when the system must maintain context and continuity across multiple requests.

Reference: [Stateful vs. Stateless Architecture](#)

249. System design 52: What is Full text search?

Full text search is a method used to search words or sentences inside large amounts of text quickly and intelligently. If we use simple LIKE %keyword% queries on such a huge dataset, performance will degrade badly because SQL has to look at every single row, one by one.

Why it is needed

In real applications, users do not search with exact database values. They search with words the way they speak. A normal SQL query can only match exact keywords, but full text search can understand similar words, rank results by relevance and highlight matching parts.

Example

If we search for "best laptops for gaming" in an online store, full text search does not just look for products with that exact phrase. It also finds products with related words like "gaming laptop", "high performance laptop" or "laptop for gamers". It also shows the most relevant products first.

Where it is used

- Search bars in e-commerce websites
- Document search in Google Drive

Tools that support full text search

- Elasticsearch
- OpenSearch
- PostgreSQL Full Text Search

Reference:

[MySQL FULLTEXT Indexes: Usage & Examples](#)

[MySQL and PostgreSQL for Advanced Full-Text Search.](#)

250. System design 53: What is Elastic Search and how does it differ from Full text search?

Elastic search is a fast search and analytics engine used to search large amounts of data in milliseconds. It is built to handle full text search, filtering and ranking results with high speed. It stores data in a special way that makes searching extremely fast, even when data is huge.

We commonly use Elasticsearch in applications that need powerful search features like auto suggestions, typo tolerance, relevance ranking and smart filtering. Many companies use it for product search, log search, monitoring dashboards and analytics.

Example: If an online store has 50 lakh products and we search for "wireless headphones", a normal SQL database will take time to find results. But Elasticsearch returns results almost instantly and also sorts them by relevance. It can even suggest similar results and handle spelling mistakes.

Difference

Full text search is a **search technique**, but Elasticsearch is a **tool** that implements this technique along with many advanced search capabilities.

Implementation in Spring but optional:

[Step-by-Step Guide to Using Elasticsearch in a Spring Boot Application | by Abhishek Ranjan | Medium](#)
[Spring Boot Elasticsearch Integration | Medium](#)

251. System design 54: How elastic search stores data to make searching fast?

When we save something in Elasticsearch, we are not saving it like a table in a database. We are saving it as a document, something like a small JSON object. Each document has fields like title, description, or price.

Example: inserting a document

Let's say we have this new product:

```
{
  "id": "42",
  "title": "Laptop Stand",
  "description": "Lightweight aluminum laptop stand for desks",
  "popularity": 1250
}
```

To insert into we call the API http://localhost:9200/<index_name>/_doc/<id> Ex: [/products/_doc/42](http://localhost:9200/products/_doc/42)

This tells Elasticsearch:

- store this document under index 'products'
- assign ID 42
- and analyze text fields like title and description.

What happens internally when data is stored?

Step 1: Breaking text into pieces (tokenization)

When Elasticsearch receives the text, it doesn't store it as one long sentence. It breaks the text into smaller words called tokens.

For example, title Laptop Stand breaks into ["laptop", "stand"].

It allows Elasticsearch to search by each word, not just by exact sentences. So if we type only *stand*, it can still find "Laptop Stand".

Step 2: Normalization

Before saving these tokens, Elasticsearch cleans them up.

It changes all letters to lowercase, removes symbols, and applies simple filters so "LAPTOP" and "laptop" are treated the same.

It converts them to lowercase, removes punctuation, and might even apply stemming (like "running" to "run").

These small clean-ups are done by something called an **analyzer**.

Step 3: Edge n-grams (for autocomplete)

Now, since we want autocomplete search, the index uses an **edge n-gram analyzer** to generate prefix tokens.

For the word laptop, it creates tokens like below:

```
1a
lap
lapt
lapto
laptop
```

Step 4: Build an inverted index

After Elasticsearch breaks the text into tokens, it does not simply store them as plain text.

It builds a special data structure called an **inverted index**.

Think of it like a dictionary. Instead of words explaining meanings, the word points to which documents contain it.

So, for our example text

Token	Document IDs
1a	[42]

```
lap      [42]
lapt     [42]
lapto    [42]
laptop   [42]
st       [42]
sta      [42]
stan     [42]
stand    [42]
```

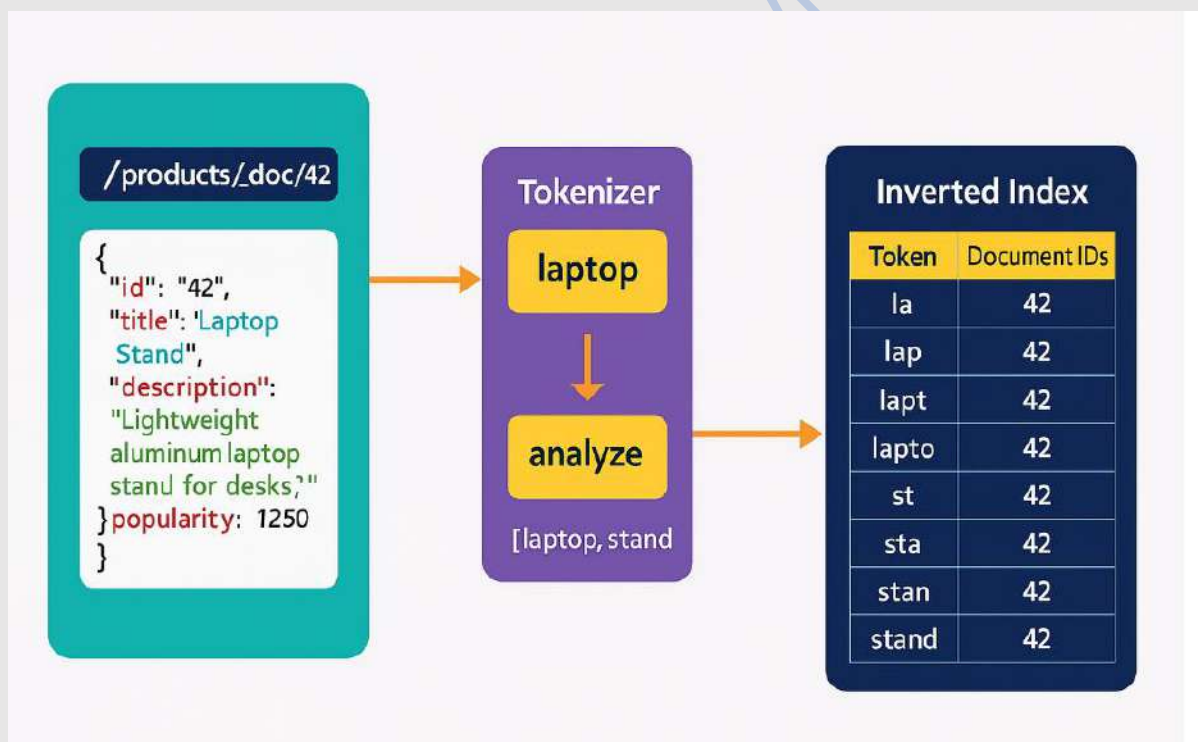
This is the **inverted index** - the core data structure that makes search lightning-fast.

Here, **42 is the document ID**, meaning these tokens came from document number 42.

So now Elasticsearch knows:

- If someone types **lap**, it already knows this belongs to document 42
- If someone types **stand**, it already knows this belongs to document 42
- No need to scan the full text again

The beauty here is speed. Elasticsearch does not search text from the beginning every time. It directly jumps to results using this index.



Why this makes search fast

When a user types: *lap*

Elasticsearch:

1. Looks up the token *lap*
2. Instantly finds matching document IDs

3. Returns the results immediately

252. System design 55: Will Full Text Search and Elasticsearch handle typos and how??

Yes, full text search and Elasticsearch can handle typos using a technique called **fuzzy search**. In real applications, users often make small spelling mistakes while typing. A normal search would fail in such cases, but full text search systems are designed to understand and fix these mistakes automatically.

How fuzzy search works

Fuzzy search does not look for an exact match. Instead, it looks for words that are **close in spelling** to what was typed. It calculates how many characters are different between the typed word and the real word. If the difference is small, it still considers it a valid match.

Fuzzy search uses a method called **Levenshtein distance**. It checks how many character changes are needed to reach the correct word. If the number of changes is small, it treats it as a match.

Example

Let us search for mobile phones. A user wants to search for "Samsung" but types "Samsang" by mistake.

- Normal SQL search: Returns nothing
- Elasticsearch fuzzy search: Suggests "Samsung".

Reference: [What is a Fuzzy Search?](#)

253. System design 56: What are functional and non-functional requirements in HLD?

When we start designing any system, the first step is to understand what the system should do and how it should behave. In High-Level Design, we classify this into two parts: **functional requirements** and **non-functional requirements**.

What are Functional Requirements?

Functional requirements describe **what the system should do**. These are the core features of the system. They tell us the actions the system must perform for users.

These include:

- What operations are supported
- What data should be processed
- How the system responds to user inputs

Example

Let us take an online food delivery system. Functional requirements will be:

- Users can register and log in
- Users can search for restaurants
- Users can add food items to a cart
- Users can place an order
- Users can make online payments

- Delivery partner can update order status

These requirements define **what features must be built**.

What are Non-Functional Requirements?

Non-functional requirements describe **how the system should work**. They do not focus on features but focus on system quality. These requirements make sure the system is reliable, fast, secure, and easy to use.

Common non-functional requirements include:

- Performance
- Scalability
- Security
- Availability
- Reliability
- Maintainability
- Usability
- Fault tolerance
- Data consistency

Example using the same food delivery system

- System should handle 10,000 orders per minute
- Response time should be less than 2 seconds
- System should be available 99.9 percent of the time
- All passwords should be stored securely
- System should recover automatically if one server fails

254. **System design 57:** How do you ensure high availability while designing the system?

High availability means keeping the system up and running even when some part of it fails. In simple words, the system should not stop working just because one server is down or one service has a problem. Users should still be able to access the system without any large interruption.

When we write a High-Level Design, we must plan how to achieve high availability. Below are the main ways we do that.

1. Remove Single Point of Failure

A single point of failure is a part of the system that, if it fails, the whole system stops working. We must avoid that by always having backup components.

Example: Instead of using only one database server, we use a primary database and a secondary replica. If the primary fails, the secondary takes over.

2. Load Balancing

We never send all user requests to a single server. We use a load balancer to distribute requests across many servers so that no server gets overloaded.

Example: If many users are placing orders at the same time, the load balancer sends each request to different application servers. This way one server does not crash due to heavy load.

3. Replication

Replication means keeping copies of data so that even if one copy fails, the system still works.

Example: In a shopping website, product information is stored in multiple read-only database replicas. When the main database is busy, read requests can use replicas. This improves performance and availability.

4. Failover Mechanism

Failover means automatic switching to a backup system when the main system fails. This helps reduce downtime.

Example: If the payment service fails, traffic switches to backup payment service in another region.

5. Health Checks and Monitoring

We monitor each service. If any service becomes slow or stops responding, it is removed from the system until it becomes healthy again.

Example: A health check runs every few seconds. If one server does not respond, the load balancer stops sending traffic to it until it recovers.

6. Using Multiple Regions or Zones

In cloud systems, we deploy services in more than one region. Even if one region has issues, the system will still run from the other region.

Example: We deploy the application in two regions like Mumbai and Hyderabad. If the Mumbai region goes down, traffic automatically switches to Hyderabad.

7. Caching

Caching reduces load on the main system by serving responses quickly from memory. With less load, fewer chances of failure.

Example: Frequently asked data like product details or user profiles are stored in Redis cache. Even if the database is slow for a moment, the system can still respond fast using cache.

8. Graceful Degradation

If part of the system fails, the rest of the system should still work with limited features.

Example: If the recommendation service fails in an e-commerce website, users should still be able to view products and place orders. Only recommendations will not show.

255. System design 58: What are the key considerations while designing a scalable system in HLD?

A scalable system is a system that can handle growth without breaking. Growth may come in the form of more users, more data, or more requests. When we design a High-Level Design, we must plan how the system will grow in the future. If we do not think about scalability early, the system will struggle when traffic increases.

Below are the key considerations for designing a scalable system.

1. Horizontal Scaling

Horizontal scaling means adding more machines instead of increasing the power of one machine. This is easier and more cost effective for large systems.

Example: If one application server is not enough to handle traffic, we add more servers behind a load balancer instead of buying a bigger server.

2. Stateless Services

Services should not store user session data inside them. They should be stateless so that any service instance can handle any request.

Example: When a user logs in, we store user session data in Redis instead of storing it inside the server memory. Then any server can handle the next request from that user.

3. Database Scalability

Databases often become a bottleneck as the system grows. We should plan for database scaling in advance.

Ways to scale a database

- Read replicas for handling read-heavy traffic
- Sharding to split large tables by user or region
- Caching to reduce direct database calls

Example: In a social media application, posts are stored in a sharded database. Users from India go to one shard and users from Europe go to another shard.

4. Caching Strategy

Caching speeds up responses and reduces load on backend services. We must plan what to cache, where to cache, and when to clear it.

Example: In an e-commerce website, product details change rarely. So we store product data in cache like Redis. This reduces load on the database.

5. Asynchronous Processing

Not everything must be done immediately. Heavy tasks can be handled in background to improve response time.

Example: When a user uploads a video, we store the request first and return a response. Then video compression and thumbnail generation happen in the background using a message queue like Kafka or RabbitMQ.

6. Load Balancing

Traffic must be spread across multiple instances to avoid overload. Load balancing also supports both high availability and scalability.

Example: A load balancer distributes traffic across five backend servers. If traffic increases, we add more servers, and the load balancer automatically distributes load to new servers.

7. Microservices Approach

Breaking the system into smaller services makes it easier to scale parts of the system that get more load.

Example: In a food delivery system, the search service gets more traffic during peak hours. So we scale only that service without affecting other services like payment or notifications.

8. Auto Scaling

The system should grow and shrink automatically based on traffic. This helps save cost and manage load better.

Example: In cloud platforms like AWS, auto-scaling adds more servers when traffic increases and removes them when traffic reduces.

256. System design 59: How do you design for disaster recovery?

Disaster recovery means preparing the system to come back to life after a major failure. A disaster can be anything that stops the system completely. It may be a data centre outage, a fire, a natural disaster, or a major system crash. In High level design, we plan how the system will recover from such events with minimum data loss and minimum downtime.

Below are the key ideas to design disaster recovery in a High-Level Design.

1. Set Recovery Goals

Before designing disaster recovery, we must define two goals

- **RPO** Recovery Point Objective: How much data loss is acceptable during a disaster
- **RTO** Recovery Time Objective: How much time we can take to bring the system back after failure

Example: In a payment system, RPO can be 0 seconds because no data loss is acceptable. RTO can be 5 minutes. This means the system should recover in 5 minutes.

2. Data Backup Strategy

Backups are the foundation of disaster recovery. We must plan how often data is backed up and where it is stored.

Example: Database backups happen every 30 minutes or every day and stored in another region. Old backups are deleted after 7 days.

3. Data Replication

Backups help restore data but they take time. Replication helps keep a real-time copy of data in another location so recovery is faster.

Example: User data is written to the main database and a replica in another city at the same time. If the main database fails, we switch to the replica.

4. Multi Region Deployment

If all services run in a single region and that region fails, the entire system will go down. So we deploy services in multiple regions.

Example: Application runs in two regions Mumbai and Chennai. If Mumbai region faces a disaster, traffic automatically shifts to Chennai.

5. Failover Plan

Failover means shifting to backup systems without manual delay. A clear failover plan must be part of HLD.

Types

- Active Passive: Backup system waits and becomes active only during failure
- Active Active: Both main and backup systems run together and share traffic

Example

Payment service uses active active mode across two regions to avoid downtime.

Reference: [🔗 The Ultimate Guide to Disaster Recovery: RTO, RPO, & Failover!](#)

257. System design 60: How do you design for data privacy and protection in HLD?

Data privacy and protection mean keeping user data safe from misuse, theft, and unauthorized access. In simple words, we must protect the data that users trust our system with. When we prepare a High-Level Design, we must include clear steps on how the system will secure sensitive data.

Below are the key ideas for designing data privacy and protection in HLD.

1. Identify Sensitive Data

Not all data needs the same level of protection. We first classify data based on how sensitive it is.

Example

In a banking system

- Highly sensitive data: account number, PIN, Aadhaar number
- Medium sensitive data: name, phone number, email
- Low sensitive data: profile picture

Once data is classified, we apply the right security measures.

2. Data Encryption

Encryption converts data into unreadable form so even if someone steals it, they cannot understand it.

Types of Encryptions

- Encryption in transit protects data while moving across the network
- Encryption at rest protects data stored in database or files

Example: All login passwords and card numbers are stored in encrypted form using AES or RSA encryption.

3. Data Masking

In some places we show only part of the data to reduce risk.

Example: Instead of showing full card number 1234567812345678, we show **** * 5678.

4. Access Control or Role-Based Access Control

Only the right people and systems should access data. Give access only when required.

Example: Customer support team can see only customer name and order details but cannot see payment information.

258. System design 61: How do you design Caching in your application?

Caching is a simple idea. We keep a copy of something so we don't have to fetch or calculate it again. When we design any system, caching helps us load faster, reduce work on the servers, and handle more users without breaking.

Here are the main types of caching that most real systems use.

1. Browser cache

This is the cache stored directly in the user's own browser. Items like images, CSS, JavaScript, or even small pieces of data can stay there.

So when someone opens the app again, the browser loads these files instantly instead of asking the server.

For example, our logo or theme colours appear right away because they are already saved in the browser.

2. CDN cache

A CDN keeps static files like images, videos, or scripts on servers spread across the world.

So when someone from India opens a page, the content comes from a nearby CDN location, not from a faraway server.

This makes pages load much faster, especially for large files.

3. Application cache

This cache sits inside our application layer.

We store frequently requested data so the server does not compute the same thing again and again.

For example, if the homepage API is called millions of times, we can store its response in memory using Redis and serve it quickly.

4. Database cache

Some queries take more time, especially when they involve large tables.

We can store the result of those heavy queries for a short period.
For example, the count of active users can be cached for a minute instead of calculating it every time.

5. Reverse proxy or edge cache

A reverse proxy like Nginx or Varnish can store full pages or API responses before they reach our backend.

So if many people request the same page, the proxy returns the stored copy without calling the application at all.

This reduces load and improves speed.

259. System design 62: What are the key components in High Level Design?

A High-Level Design, often called HLD, is like a blueprint of a software system. It explains how the system will be built from a broader view without going deep into code-level details.

1. Introduction and Purpose

This section explains what problem the system solves and why we are building it. It gives a short background about what we are designing.

Example:

We are building an Online Food Delivery System. The purpose of this system is to allow customers to order food from nearby restaurants using a mobile app or website.

2. Architecture Diagram

This is a simple block diagram that shows the overall structure of the system and how the main parts connect with each other. It gives a big picture of the system.

Example:

The system may have blocks like User Interface, Order Service, Payment Service, Restaurant Service, Delivery Service, and Database. Lines between blocks show communication between them.

3. Modules and Components

This part explains each module in the system and what it does. Each module has a clear responsibility.

Example:

In the food delivery system:

- Order Module handles new orders and order tracking
- Payment Module manages payments and refunds
- Restaurant Module shows menus and availability
- Delivery Module assigns delivery partners and tracks delivery
- Notification Module sends SMS and email updates

4. Data Flow

This part describes how data moves in the system. It shows how information flows from one module to another.

Example:

Customer places an order -> Order Module saves order details -> Payment Module processes payment -> Restaurant Module receives order -> Delivery Module assigns driver -> Notifications are sent.

5. Technology and Tools

This section lists the technologies used in building the system.

Example:

- Frontend: React
- Backend: Java with Spring Boot
- Database: PostgreSQL
- Message Queue: Kafka
- Cloud: AWS

6. Database Design Overview

At HLD level, we do not show full table designs. We show only main entities and their relationships.

Example:

Main entities: Customer, Restaurant, Menu, Order, Payment, Delivery.

7. Integration Details

This explains how the system connects with external services.

Example:

- Payment gateway for online payments
- Google Maps API for location and delivery tracking
- SMS gateway for sending notifications

8. Security Overview

This section covers basic security ideas used in our system.

Example:

- All requests must be authenticated using JWT tokens
- Passwords stored using encryption
- Sensitive data protected using HTTPS

9. Scalability and Performance Plan

This explains how the system will handle growth when users increase.

Example:

- Load balancer used to distribute traffic
- Caching to reduce database load
- Services run as microservices so they can scale independent

10. Non-Functional Requirements

These are qualities of the system that are not related to features.

Example:

- Availability: 99.9 percent uptime
- Response time: Less than 2 seconds
- Data backup daily

260. **System design 63: How to approach a system design interview?**

A system design interview is not about giving the perfect answer. It is about showing how we think, how we solve problems and how we make decisions for large systems.

We need to identify the **functional requirements** (example: upload photos, follow users, view feed) and **non-functional requirements** (scalability, availability, low latency, consistency), then we build the system step by step. We start with a simple version and then grow the architecture. The high-level system can be divided into the following components:

1. Client Layer (User Interaction), 2. API Gateway, 3. Microservices Layer (Business Logic), 4. Database Layer (Data Storage), 5. Communication Layer, 6. Caching Layer, 7. Monitoring and Logging, 8. Load Balancer, 9. Scalability & High Availability

1. Client Layer (User Interaction)

This is where users interact with the system using mobile apps or web browsers. The client sends requests like uploading a photo, following a user or viewing a feed. It communicates with backend services using APIs.

2. API Gateway

The API Gateway is the single-entry point for all client requests. It routes requests to the correct service, applies rate limiting, handles authentication and adds security before sending traffic inside the system.

3. Microservices Layer (Business Logic)

Each major feature is handled by a separate microservice. For Instagram-like design, services may include:

- User Service
- Post Service
- Feed Service
- Follow Service
- Notification Service

Each service handles its own logic and data.

4. Database Layer (Data Storage)

Stores user accounts, posts, comments, follower relationships and likes. Different types of databases may be used:

- Relational databases for structured data
- NoSQL databases for scalable social graphs
- Object storage for media files

5. Communication Layer

Handles messaging between services. Uses message brokers like Kafka or RabbitMQ for asynchronous communication. This helps in decoupling services and improving reliability.

6. Caching Layer

Improves performance by storing frequently accessed data in temporary storage like Redis or Memcached. Reduces database load and speeds up responses, especially for feed and profile views.

7. Monitoring and Logging

Tracks system health using tools like Prometheus, Grafana and ELK (Elasticsearch, Logstash, Kibana). Helps detect failures and understand system behaviour in real time.

8. Load Balancer

Distributes incoming traffic across multiple servers to prevent overloading. Ensures better performance and high availability even during peak user traffic.

9. Scalability and High Availability

The system is designed to scale horizontally by adding more servers when load increases. Replication and failover strategies keep the system available even if individual services fail.

High Level Designs

261. HLD 0: Steps to Design a System

1. Gather Requirements

- List functional requirements (what the system should do)
- List non-functional requirements (performance, security, scalability)

2. Identify Components

- Decide what parts are needed
- Example: Load Balancer, API Gateway, Microservices, Message Queue, Cache

3. Choose Database and Design It

- Select SQL or NoSQL based on data needs
- Design tables or collections
- Plan indexes and data storage

4. Design APIs

- Define API endpoints
- Write request and response formats
- Plan authentication and error handling

5. Draw Architecture Diagram

- Show components and connections
- Show data flow
- Keep it clear and easy to explain

Note: We will follow these same steps in all upcoming high level system design questions.

262. HLD 1: Tiny URL Design

A TinyURL service takes a long web link and turns it into a short link. Later, when someone clicks that short link, the service redirects them to the original long link.

1. Functional Requirements:

- Generate a unique short URL for a given long URL
- Redirect the user to the original URL when the short URL is accessed
- Prevent duplicate short URLs for the same long URL.

2. Non-Functional Requirements:

- High availability (the service should be up 99.9% of the time)
- Low latency (url shortening and redirects should happen in milliseconds)
- Scalability (the system should handle millions of requests per day)

3. Components

Component	Purpose
Load Balancer	Shares incoming traffic across many servers
Short URL Service	Handles API requests like create and redirect
Cache	Stores popular short URLs for fast access
Database	Stores mapping of short URL and long URL
ID Generator	Creates unique short codes

4. Database

SQL vs No SQL

1. We need to store billions of records.
2. Each object we store is small (less than 1K).
3. There are no relationships between records

Since we need to store billions of rows, and we don't need to use relationships between objects, a NoSQL store like DynamoDB, Cassandra is best choice.

We use two tables:

1. user - stores user account information
2. url_mapping- stores short and long URLs created by users

User Table

user_id	name	email	password
101	Ravi Kumar	ravi.kumar@mail.com	encrypted_pwd_123

url_mapping:

short_url	long_url	creation_date	expiration_date	user_id	click_count
aB12cd	https://www.wikipedia.org/wiki/Systems_design	2025-01-20 10:30:22	2025-07-20 10:30:22	101	54

5. API Design

We need Two APIs

1. URL Shortening API: Endpoint- POST /shorten

This endpoint creates a new short URL for a given long URL.

Request:

```
{
  "long_url": "https://codinglyf.com/long/url/ ",
  "user_id": "user123"
}
```

Response:

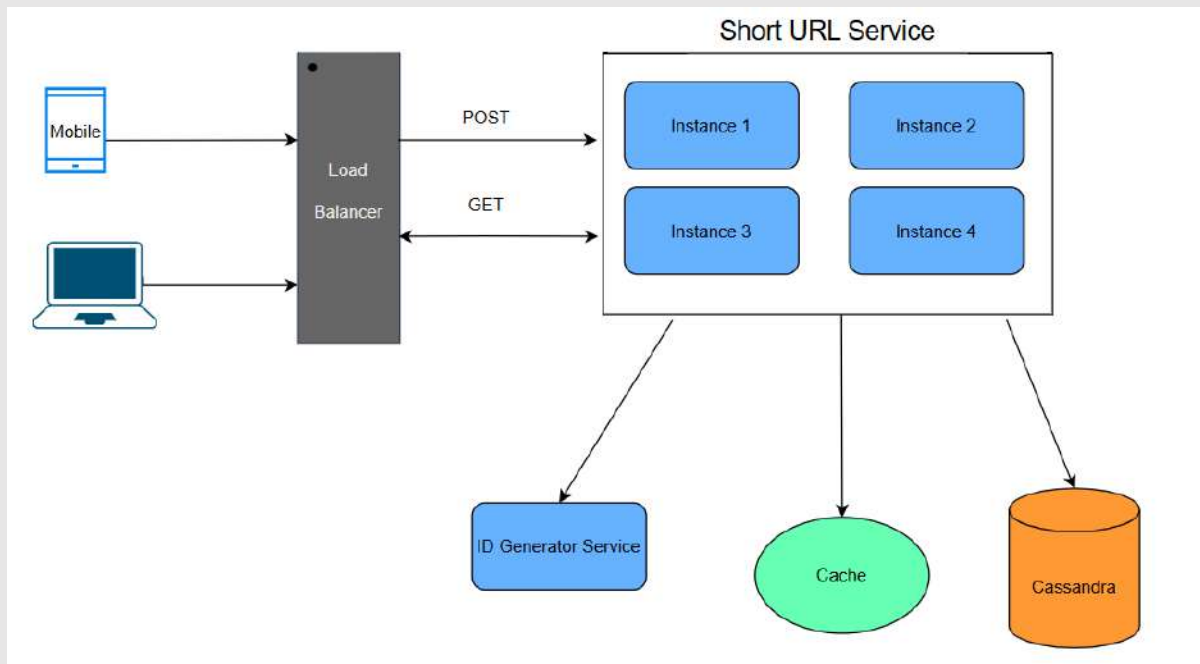
```
{
  "short_url": "http://short.url/abC123",
  "long_url": "https://codinglyf.com/long/url/ ",
  "expiry_date": "2022-03-31T23:59:59Z",
  "created_at": "2024-12-15T10:30:00Z"
}
```

2. URL Redirection API: Endpoint - GET /{short_url_key}

This endpoint redirects the user to the original long URL.

Sample Response: <https://codinglyf.com/long/url/>

6. Architecture



a. Clients Mobile or Web

These are the places where users interact with the short URL system.

- When a user wants to shorten a long URL, they send a POST request.
- When a user clicks a short link, they call a GET request to redirect them to the original long URL.

These requests go through the Load Balancer.

b. Load Balancer

The load balancer is the front door of our system. It receives all incoming requests and spreads them across many servers so that no single server gets overloaded.

- If millions of users send requests at the same time, the load balancer keeps the system stable by sharing the load.
- It also improves availability. If one server goes down, traffic is sent to the rest.

c. Short URL Service (Instances)

On the right of the load balancer, we see four instances of the Short URL Service. These are simple application servers running the business logic.

- They handle POST requests to create short URLs.
- They handle GET requests to redirect users.
- All instances do the same work, so we can add or remove instances easily based on load.

These instances do not store data themselves. Instead, they depend on the ID generator, cache, and database.

d. ID Generator Service

When someone creates a short URL, we need a unique ID for it, like aB12cd. This is where the ID Generator Service comes in.

- It creates short, unique IDs.

- It ensures no two links get the same short code.
- It can follow patterns like base-62 encoding to make short codes look neat.

This service makes sure we never create duplicate links.

e. Cache

The cache is used to speed up the system. We place frequently accessed data here so we do not need to go to the database every time.

- If many people access the same short link, the URL will be served faster from cache.
- Cache reduces load on the database.
- Redis or Memcached can be used here.

Example: If the short URL aB12cd is requested again and again, the cache stores the original long URL temporarily so it can be fetched in microseconds.

f. Cassandra (Database)

Cassandra stores the permanent mapping between short URLs and long URLs. It is a NoSQL database, and we use it because:

- We need to store billions of records.
- Each record is small.
- There are no relationships between entries.
- The service must scale as traffic grows.

7. Recap / Flow

1. The client sends a request.
2. Load balancer routes it to one of the service instances.
3. The instance asks the ID Generator to create a unique code.
4. It stores the result in the Cassandra database.
5. It also saves it in the cache for fast access.
6. When anyone clicks the short link, the service checks the cache first.
7. If not found, it fetches from database and then stores it in cache.

Reference: [ByteByteGo | Technical Interview Prep](#)

[Design a URL Shortener - System Design Interview](#)

Video Reference: [TinyURL System Design | URL Shortner System Design Interview Question | Bitly System Design](#)

263. **HLD 2: Design Load balancer**

A Load Balancer is a system that distributes incoming requests evenly across multiple servers. It helps make our application faster, more reliable, and always available, even if one of the servers fails.

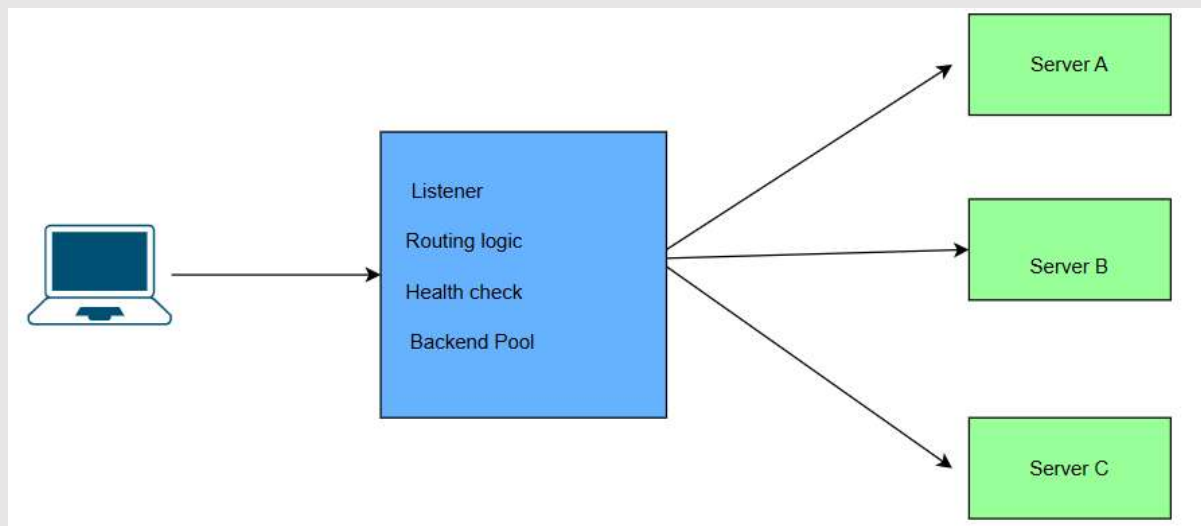
Why We Use a Load Balancer

1. To share traffic between servers instead of overloading one

2. To improve application speed and response time
3. To avoid downtime when one server crashes
4. To scale easily when more users come in

Components:

Component	Description
Listener	Listens to incoming client requests (like on port 80 or 443).
Routing Logic	Decides which backend server should handle the request.
Health Checker	Regularly checks if each server is working fine.
Backend Pool	List of all servers where traffic can be sent.
Metrics and Logs	Collects data about traffic, latency, and errors.



Let's take NGINX as a Load balancer example and see each section

Reference: [HTTP Load Balancing | NGINX Documentation](#)

1. Listener

The listener is the entry point.

It listens on specific ports like **80 (HTTP)** or **443 (HTTPS)** and forwards requests to the backend servers.

Example:

```
# Listener section
server {
    listen 80;                # Listens on port 80 for HTTP
    server_name example.com;  # Domain name

    location / {
        proxy_pass http://backend_pool; # Forwards all requests to the backend pool
    }
}
```


2. Backend Pool

The backend pool is a **group of all available servers** where the Load Balancer can send traffic. All backend servers usually run the same version of the application.

Backend Pool Definition

```
upstream backend_pool {  
    server backend1.example.com;  
    server backend2.example.com;  
    server backend3.example.com;  
}
```

We can add or remove servers here anytime without restarting NGINX, which helps with scaling up or down.

3. Routing Logic

Routing logic decides which backend server will handle each request. We can choose different load balancing algorithms depending on our needs.

Let's look at common ones.

a) Round Robin (default)

Each request goes to the next server in order.

Routing Logic - Round Robin

```
upstream backend_pool {  
    server backend1.example.com;  
    server backend2.example.com;  
    server backend3.example.com;  
}
```

b) Least Connections

Sends new requests to the server with the fewest active connections.

Routing Logic - Least Connections

```
upstream backend_pool {  
    least_conn;                # Enables least connection algorithm  
    server backend1.example.com;  
    server backend2.example.com;  
    server backend3.example.com;  
}
```

c) IP Hash

Keeps the same client connected to the same server (useful for sticky sessions).

Routing Logic - IP Hash

```
upstream backend_pool {  
    ip_hash;                   # Enables session stickiness  
    server backend1.example.com;  
    server backend2.example.com;  
    server backend3.example.com;  
}
```

Each method serves a different purpose:

- **Round Robin:** Simple and fair for small traffic.
- **Least Connections:** Good when servers handle requests of different durations.
- **IP Hash:** Best for session-based apps (like login systems).

4. Health Checker

The health checker makes sure only healthy servers receive traffic. If a server fails, it's skipped until it becomes healthy again.

Health Checker setup

```
upstream backend_pool {
    server backend1.example.com max_fails=3 fail_timeout=30s;
    server backend2.example.com max_fails=3 fail_timeout=30s;
    server backend3.example.com max_fails=3 fail_timeout=30s;
}
```

Explanation:

- max_fails=3 , if a server fails 3 times
- fail_timeout=30s, remove it from rotation for 30 seconds
During this time, NGINX automatically sends requests to healthy servers only.

5. Metrics and Logs

Metrics help us monitor traffic, response time, and failures. NGINX provides built-in access logs and status modules.

Metrics and Logging

```
http {
    access_log /var/log/nginx/access.log;    # Stores request logs
    error_log /var/log/nginx/error.log;      # Stores error logs

    server {
        listen 80;
        location /nginx_status {
            stub_status;                    # Shows live metrics
            allow 127.0.0.1;                # Allow access from localhost
            deny all;                        # Block others
        }
    }
}
```

264. HLD 3: Design API Gateway

An API Gateway is the single-entry point for all client requests in a microservices system. It receives requests, forwards them to the right service, combines responses if needed, and sends the final response back to the client.

Why We Need an API Gateway

1. To hide internal microservice details from external users
2. To route requests to the correct backend service

3. To apply authentication and authorization in one place
4. To reduce the number of direct client-service calls
5. To manage common features like logging, caching, and rate limiting

Basic Example

Let's say we have three microservices:

- User Service: manages user data
- Order Service: handles customer orders
- Product Service: manages product details

Instead of clients calling each service directly, they send all requests to the API Gateway, and it forwards them internally.

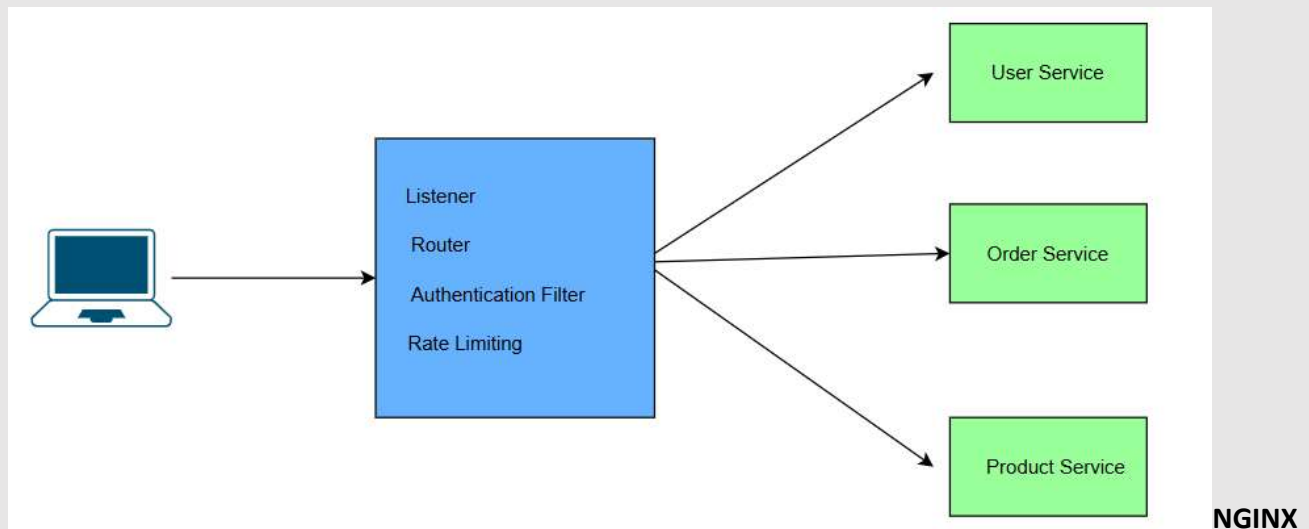
Components:

Component	Description
Listener	Accepts client requests through HTTP or HTTPS.
Router	Maps incoming URLs to backend microservices.

API Gateway can also be used for authentication, rate limiting and to cache

Authentication Filter Validates tokens and checks access rights.

Rate Limiter	Controls the number of requests per second per user or IP.
Cache	Stores frequent responses to improve performance.
Logger	Captures request and response data for monitoring.
Backend Services	Actual microservices that process the request.

**Sample:**

```
server {  
    listen 80;  
    server_name api.example.com;  
  
    # Route to User Service  
    location /api/users/ {  
        proxy_pass http://user_service;  
    }  
  
    # Route to Order Service  
    location /api/orders/ {  
        proxy_pass http://order_service;  
    }  
  
    # Route to Product Service  
    location /api/products/ {  
        proxy_pass http://product_service;  
    }  
}  
  
# Define upstream services  
upstream user_service {  
    server user1.example.com;  
    server user2.example.com;  
}  
  
upstream order_service {  
    server order1.example.com;  
    server order2.example.com;  
}  
  
upstream product_service {  
    server product1.example.com;  
}
```

Design

1. **Client** sends a request to the API Gateway.

2. **Listener** accepts the request on port 80 or 443.
3. **Authentication Filter** checks the user token (for example, JWT).
4. **Router** decides which backend service to forward the request to.
 - /api/users: User Service
 - /api/orders: Order Service
 - /api/products: Product Service
5. **Rate Limiter** ensures no one sends too many requests.
6. **Cache** returns data instantly if it was already fetched before.
7. **Backend Service** processes the request and sends the response.
8. **Logger** records the request, response time, and status code.
9. **API Gateway** returns the final response to the client.

265. HLD 4: Design the Rate limiter

When we build APIs, one big challenge is handling too many requests from the same client. A Rate Limiter helps us control that. It makes sure that a client can only make a fixed number of requests in a given time, like 10 requests per minute.

1. Functional Requirements

- Limit how many requests a client can make within a time window (for example, 100 requests per minute).
- Identify clients by API key, IP address, or user ID.
- Block or delay requests that exceed the limit.
- Return proper error responses when the limit is reached (like 429 Too Many Requests).

2. Non-Functional Requirements

- The limiter should be fast, because it runs for every request.
- It should scale horizontally when traffic increases.
- It should work correctly in a distributed environment (multiple servers).

3. Identify Components

To make a Rate Limiter work smoothly, we need these parts:

- **Client** - The one making requests (could be an app or a browser).
- **Rate Limiter Service** - The logic that tracks how many requests each client has made and decides whether to allow or block.
- **Data Store (Cache)** - Used to keep counters of requests per client (like Redis).
- **Application Server** - The actual backend service that handles valid requests.

4. Database

For a Rate Limiter, we need fast reads and writes. That's why Redis is a perfect choice

Example: Redis Key Design We can store data like this:

Key (Client ID + Time Window)	Value (Request Count)	Expiry (seconds)
client_123_2025103012	12	60

5. Design APIs

We can build a small internal API for the Rate Limiter service.

Endpoint: POST /check-limit

Request:

```
{
  "clientId": "client_123",
  "apiPath": "/getOrders"
}
```

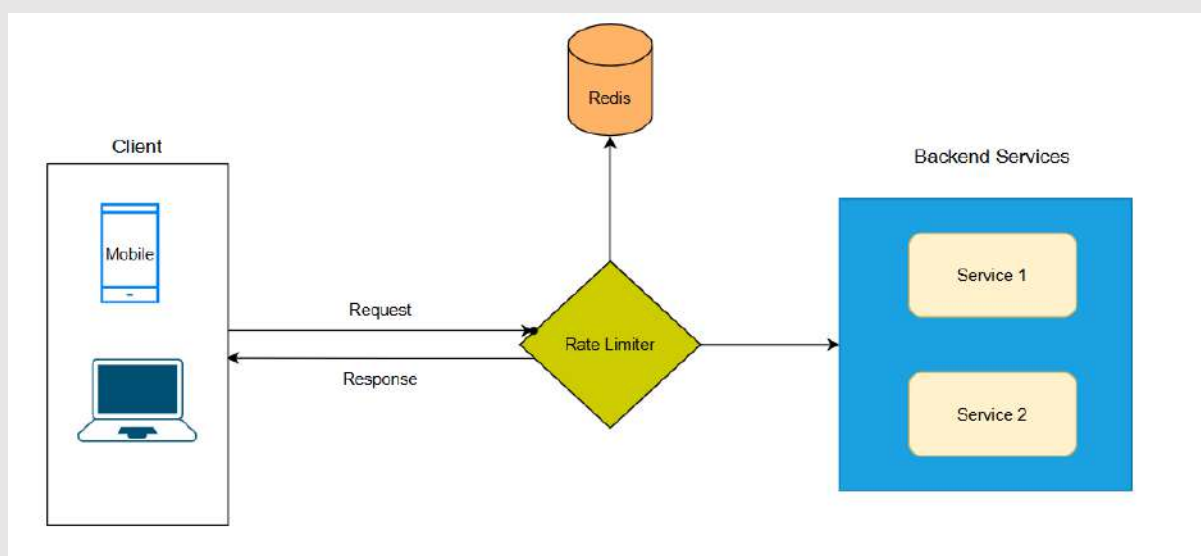
Response (if allowed):

```
{
  "allowed": true,
  "remaining": 5
}
```

Response (if blocked):

```
{
  "allowed": false,
  "retryAfter": 30,
  "message": "Rate limit exceeded. Try again after 30 seconds."
}
```

6. Architecture:



a. Client

On the left side, we see the Client box.

This can be a mobile app or a web application.

The client sends requests to our system, for example, fetching user data or submitting a form.

Every time the client makes a request, it first passes through the Rate Limiter before reaching the actual backend services.

b. Rate Limiter

At the center, we have the Rate Limiter (the diamond shape).

This is the brain of the system.

Its job is simple:

- Check how many requests the client has made recently.
- Decide if this new request should be allowed or blocked.

If the client has not crossed the limit, the Rate Limiter forwards the request to the backend service.

If the client has already sent too many requests in a short time, it blocks the request and sends a “Too Many Requests” response back to the client.

This helps prevent misuse, server overload, and ensures fair usage for everyone.

c. Redis

Above the Rate Limiter, we see Redis. Redis acts like a temporary memory store that keeps track of how many requests each client has made.

For example, it can store data like this:

Client ID	Request Count	Last Request Time
client_101	3	10:05:20

Redis is very fast and perfect for this job because it can quickly read and update data for thousands of clients at once.

Whenever a request comes in, the Rate Limiter checks Redis to see the current count and updates it after processing.

d. Backend Services

On the right side, we see the Backend Services box. This contains our actual business logic

Examples:

- Service 1 might handle user information.
- Service 2 might manage orders or payments.

The Rate Limiter ensures that only valid and limited requests reach these backend services, keeping them healthy and fast.

7. Recap / Flow

1. The client sends a request (for example, to view account details).
2. The request goes to the Rate Limiter.
3. The Rate Limiter checks Redis to see if the client still has tokens (remaining request quota).
4. If allowed, the request moves forward to Service 1 or Service 2.

5. The service processes it and sends the response back to the client through the Rate Limiter.
6. If the client has exceeded the limit, the Rate Limiter directly sends a message like *“Too many requests, try again later.”*

7. Additional Questions:

1. Can we add API Gateway as single point of contact?

Yes, an **API Gateway** can and frequently is used as the single point of contact (or single entry point) for client applications in a microservices architecture. It acts as a reverse proxy, sitting in front of your backend services to manage all incoming traffic from external users.

Reference: Below two links explained along with Rate limiting algorithms.

[System Design — Design A Rate Limiter | by Rahul Kapoor | Geek Culture | Medium](#)

[API Rate Limiter System Design](#)

Video link: [Rate Limiter System Design: Token Bucket, Leaky Bucket, Scaling](#)

266. HLD 5: Design a search service with autocomplete / “type-ahead” feature

Refer Question System Design 53 and 54 for elastic search

1. Functional Requirements

- Show suggestions as we type in the search box.
- Upload new data (like product names or movie titles) into the system.
- Keep suggestions updated when new data is added.

2. Non-Functional Requirements

- Fast response (should return suggestions within milliseconds).
- Handle large amounts of search data.
- System should scale easily when traffic increases.

3. Components

To build this system, we need these main parts:

Client (UI): The part where the user types a few letters and expects suggestions.

API Gateway: Receives requests from the client and routes them to the right microservice.

Suggestion Service: Fetches data from Elasticsearch and returns the best matching suggestions.

Upload Data Service: Used when new data needs to be added (for example, new product names). It sends this data to **Kafka** (a message queue). Kafka then forwards it to Elasticsearch for indexing.

Kafka: Works like a middleman that ensures data is safely passed from the upload service to Elasticsearch, even if Elasticsearch is busy.

Elasticsearch: Stores the searchable data and helps in returning fast suggestions.

4. Database

We use Elasticsearch to store data because it is designed for search-based use cases. Each record in Elasticsearch is stored as a document (like a JSON object).

Example document:

```
{
  "id": "101",
  "title": "Wireless Mouse"
}
```

This way, when we type “wire”, it can quickly match and show “Wireless Mouse”.

5. Design APIs: Two main APIs.

1. Suggestion API

Endpoint: /api/suggestions?query=lap

Method: GET

Request:

```
{
  "query": "lap"
}
```

Response:

```
{
  "suggestions": ["Laptop Stand", "Laptop Bag", "Laptop Table"]
}
```

2. Upload Data API

Endpoint: /api/upload

Method: POST

Request:

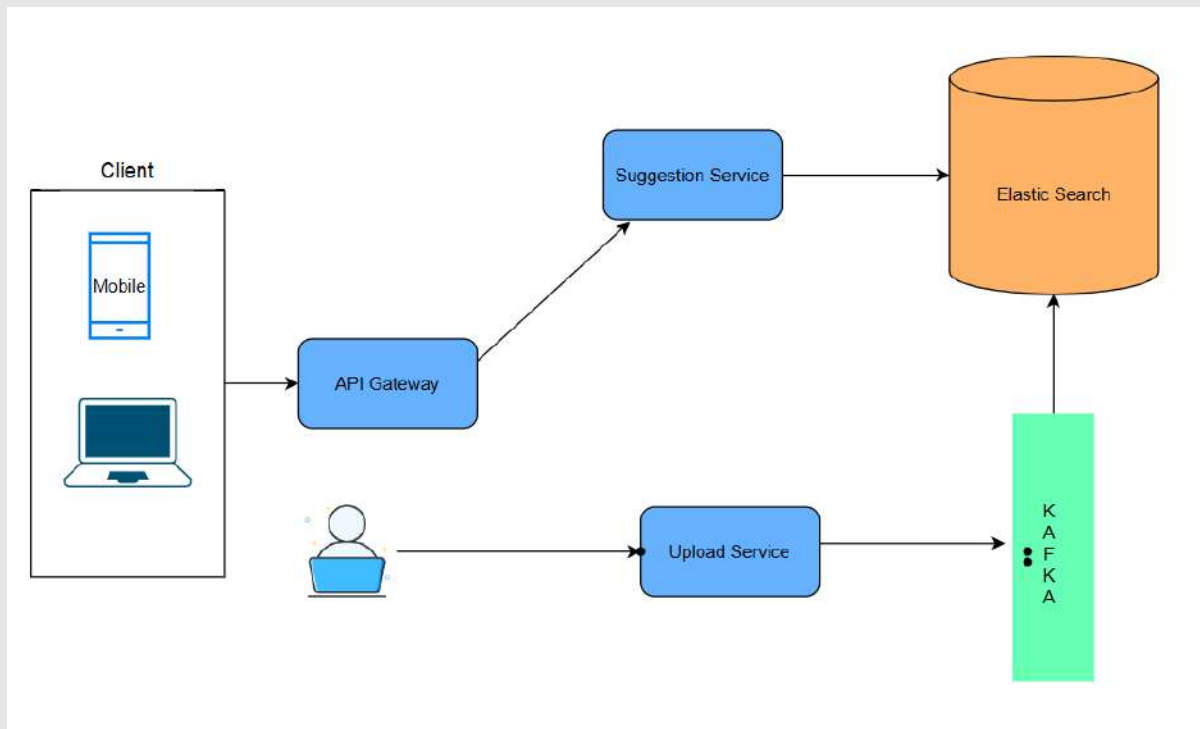
```
{
  "id": "42",
  "title": "Laptop Stand"
}
```

Response:

```
{
  "message": "Data uploaded successfully"
}
```

Once the upload API is called, the data goes to Kafka, which then passes it to Elasticsearch for indexing.

6. Architecture:

**a. Client**

The client can be a mobile app or a web application. It sends requests to upload data or get search suggestions. All requests first go through the API Gateway for validation and routing to the right service.

b. API Gateway

Acts as the entry point for all client requests. It handles authentication, routing, and rate limiting. It forwards suggestion requests to the Suggestion Service based on the API endpoint.

c. Upload Service

This service takes care of receiving uploaded data, like product details or files. Once received, it publishes the event to Kafka,

d. Kafka

Kafka works as a message queue. It stores and streams uploaded events. It helps in decoupling services, meaning the Upload Service and Elastic Search do not directly depend on each other's availability

e. Suggestion Service

This service provides search suggestions to users. It fetches related results quickly from Elastic Search based on keywords or partial inputs, making the search feature faster.

f. Elastic Search

Stores indexed data to make searches fast and accurate. It gets updated data from Kafka, ensuring the suggestions and results stay fresh whenever new uploads or updates happen in the system.

Note: **Elasticsearch Sink Connector**, which is part of **Kafka Connect**, is used to push data from Kafka topics into Elasticsearch indices

7. Recap / Flow

Let's take a real example.

Say we type "lap" in the search box.

- The UI sends a request to `/api/suggestions?query=lap`.
- The Suggestion Service looks up Elasticsearch.
- Elasticsearch finds matching titles like "Laptop Stand" and "Laptop Bag".
- The Suggestion Service returns these as a list of suggestions to the UI.

If a new product called "Laptop Cover" is added:

- Upload Data Service sends it to Kafka.
- Kafka forwards it to Elasticsearch.
- Now "Laptop Cover" will also start showing up in the suggestions.

267. HLD 6: Design ATM System

Before designing the system, let's revisit how an ATM works. Suppose a customer visits an ATM. To start the transaction, the user has to enter their ATM card. Next, the machine asks the user for their Personal Identification Number (PIN). The ATM machine sends the user's information on the card and the PIN entered to the bank for authentication. Once the PIN is authenticated, the screen on the ATM machine will display a set of options.

Let's suppose the user selects the 'withdraw' option. Next, the machine will ask the user for the amount to be withdrawn. Once the user enters the amount in digits, the ATM system will process the request and dispense cash through the cash dispenser. Once the cash is out, the ATM asks the user if another transaction is to be made. If no other transaction is to be made, an exit message is displayed and the card is returned to the user.

1. Functional Requirements

- **Card Authentication:** Validate the ATM card.
- **PIN Verification:** Securely verify the user's PIN.
- **Account Selection:** Allow users to select the type of account (e.g., savings, checking).
- **Cash Withdrawal:** Allow users to withdraw money.
- **Balance Inquiry:** Check the account balance.
- **Deposit Money:** Accept cash or checks.
- **Transaction History:** Provide a summary of recent transactions.
- **Fund Transfer:** Transfer money between accounts.
- **Printing Receipts:** Provide receipts for transactions.
- **Security Alerts:** Provide alerts in case of failed transactions or multiple failed PIN attempts.

2. Non-Functional Requirements

- **Availability:** The ATM should have high availability (99.99%), as it will be accessed at any time.
- **Security:** Ensure that the ATM is highly secure, encrypting all sensitive data (PIN, account numbers).
- **Performance:** The ATM should process transactions quickly, within a few seconds.
- **Reliability:** The machine should not go offline unexpectedly and should handle failures gracefully.
- **Scalability:** Support for a large number of users and transactions across multiple ATMs.
- **Fault Tolerance:** Should have backup mechanisms in case of hardware or software failures.
- **Auditing and Logging:** Keep track of every transaction and store logs for auditing purposes.

3. Components

To build this system, we need these main parts:

Component	Description
ATM Machine	User interface to insert card, enter PIN, and perform actions
Authentication Service	Validates card and PIN, generates session tokens
User Interaction Service	It displays the ATM menu, Check Balance, Withdraw, Deposit, Exit.
Account Service	Handles balance checks, withdrawals, deposits, and transfers
Transaction Service	Logs each transaction for audit and record
Database (SQL)	Stores customer, account, and transaction details
Notification Service	Sends SMS or email alerts after transactions
Cash Dispenser	Physically releases cash when withdrawal is approved

4. Database

We use **SQL Database** because banking transactions need strong consistency and ACID properties.

Customer Table

customer_id	name	phone
101	Rahul Sharma	9876543210

Account Table

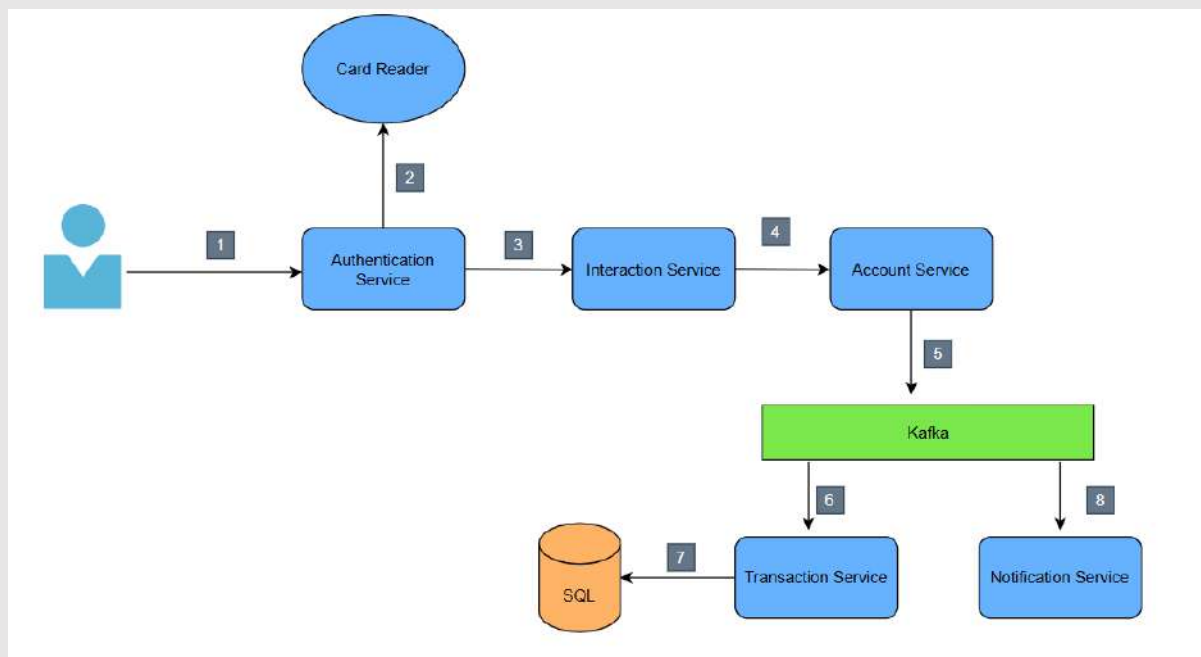
account_id	customer_id	balance
5001123456	101	12500.75

Transaction Table

txn_id	account_id	type	amount	txn_time	status
90045	5001123456	Withdraw	2000.00	2025-11-10 10:42:16	Success

5. Design APIs: Two main APIs.

API	Method	Description	Request Example	Response Example
/authenticate	POST	Validate card and PIN	{ "card_no": "1234-5678", "pin": "1234" }	{ "status": "success", "token": "abc123" }
/balance	GET	Check current balance	{ "token": "abc123" }	{ "balance": 8500.75 }
/withdraw	POST	Withdraw money from account	{ "token": "abc123", "amount": 2000 }	{ "status": "success", "remaining_balance": 6500.75 }
/deposit	POST	Deposit cash into account	{ "token": "abc123", "amount": 5000 }	{ "status": "success", "new_balance": 11500.75 }
/transactions	GET	Fetch last 5 transactions	{ "token": "abc123" }	{ "transactions": [{ "type": "withdraw", "amount": 2000 }] }

6. Architecture:

Follow the number in the diagram

1. Customer to Card Reader

The process begins when a customer inserts the ATM card. The card reader reads the card chip or magnetic stripe to identify the account number and bank network.

2. Card Reader to Authentication Service

The Authentication Service encrypts the entered PIN and validates it against the stored PIN offset in the bank database using a **Hardware security Module**. Only after successful validation does the system move forward.

In banking, a **Hardware Security Module (HSM)** is a physical device that protects and manages critical cryptographic keys for financial transactions, ensuring data security, compliance, and tamper resistance

3. Authentication Service to Interaction Service

Once authenticated, the Interaction Service displays the menu options like Withdraw, Deposit, Balance Inquiry, and Transfer. It captures user input and sends it for processing.

4. Interaction Service to Account Service

The selected action is passed to the Account Service. For example, if a user selects “Withdraw ₹2000,” the service checks account balance and validates the transaction request.

5. Account Service to Kafka

After validating, the Account Service publishes a message to Kafka, ensuring all downstream systems like Transaction and Notification Services are updated asynchronously without delay.

6. Kafka to Transaction Service

The Transaction Service listens to Kafka events. It records the transaction details in the SQL database, including account number, amount, type, time, and status for audit and history.

7. Transaction Service to SQL Database

The SQL database stores structured data like account info, transaction history, and balances. For example, account_id 5001123456 withdraws ₹2000 and balance updates from ₹12500 to ₹10500.

8. Kafka to Notification Service

The Notification Service also listens to Kafka. Once the transaction is successful, it sends an SMS or email confirmation like “₹2000 withdrawn from your account ending with 3456.”

7. Recap

- A customer inserts their card, and the Card Reader identifies the account.
- The Authentication Service verifies the PIN securely using HSM.
- After successful login, the Interaction Service shows options like withdraw or balance check.
- The Account Service validates the request and publishes the event to Kafka.
- Transaction Service records the details in SQL, while Notification Service alerts the user.
- This design keeps processes modular, secure, and fast with async updates.

Reference: [System Design Interview: ATM Machine Design | Tech Wrench](#)

[System design: Design an ATM Machine - DEV Community](#)

268. HLD 7: Design Notification System

You are building a notification system for a company like **Zomato**. Every day, Zomato wants to send lakhs of notifications to its users about offers, festival discounts, cashback rewards, free delivery campaigns, or personalized restaurant suggestions.

Notifications must go through different channels:

- **Push Notifications** (to the mobile app)
- **SMS** (for users without internet or app inactive)
- **Email** (for marketing and detailed offers)

Explanation:

First Let's start with Functional and Non-Functional Requirements.

Functional Requirements:

1. **Multi-channel Support:** The system must support sending notifications through various channels including email, SMS, push notifications, and in-app messages.
2. **User Preferences & Opt-Out:** Respect user notification preferences. The system should check if the user has unsubscribed or opted out of certain notification types or channels. For example, if a user opted out of promotional emails. The system should not send notifications
3. **Scheduled Delivery:** Support scheduling of notifications for future delivery.
4. **Rate Limiting:** Ensure that users receive only a limited number of promotional messages in a given day to prevent spam.
5. **Retry Mechanism:** Handle notification delivery failures, retrying when necessary (e.g., failed SMS or email).

Non-Functional Requirements:

- **Scalability:** The system should handle millions of notifications per minute, supporting millions of concurrent users.
- **High availability:** Ensure minimal downtime so that notifications are delivered even in case of failures.
- **Reliability:** Guarantee at-least-once delivery of notifications.
- **Low latency:** Notifications should be sent as quickly as possible to ensure timely delivery.

2. Identity the components:

- We need a notification provider to generate events like offers, alerts triggered by business systems.
- We need a service to accept requests, validate data, and forward messages for processing.
- We need a service to check if user is opted for email, SMS, push notifications, and preferred language.
- We need a database to store user contact details, user device details to send notification, and notification history.
- We need a service to time-based notifications like reminders or campaign launches.

- We need a message queue like Kafka to handle high traffic and decouple request handling from processing.
- We need a message processor to read from the queue, enrich data, and select the correct notification channel.
- We need channel adapters to integrate with third-party providers for SMS, email, and push notifications

3. DB Design:

User Table:

user_id	email	phone	device_token	pref_email	pref_sms	pref_push
101	rahul@gmail.com	9876543210	abc123xyz	true	false	true

Notification Table: Once after notification delivered we can save the details in this table

notif_id	user_id	message	channel	status	created_at
5001	101	Flat 40% OFF on pizza today 🍕	push	SENT	2025-01-15 10:23:00

Schedule Table

notif_id	user_id	message	send_time	status
7001	101	Reminder: Weekend offer starts!	2025-01-20 09:00:00	SCHEDULED

4. API Design:

Notification Service contains these two APIs. Notification Provider will use them to post the message

Create Notification

POST /notifications/send

```
{
  "userId": 101,
  "message": "Flat 40% OFF today",
  "channels": ["push", "email"]
}
```

Schedule Notification

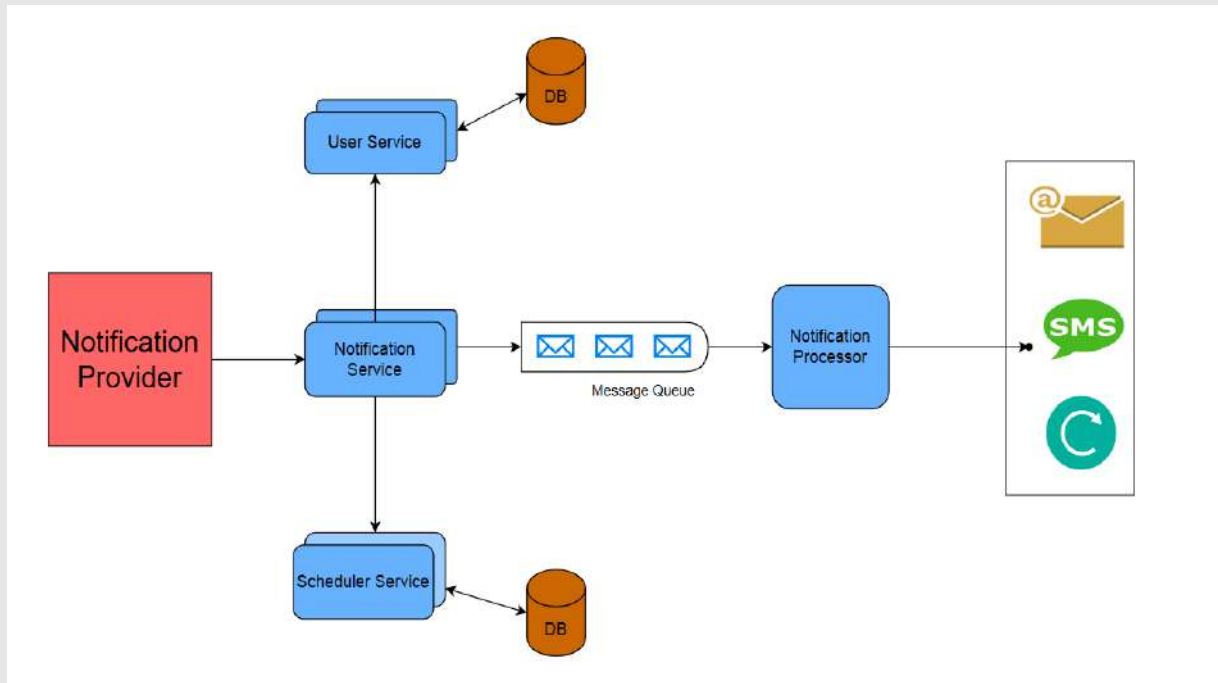
POST /notifications/schedule

```
{
  "userId": 102,
```



```
"message": "Big Offer Coming Tonight!",  
"sendTime": "2025-12-26 18:00:00"  
}
```

5. Architecture diagram



1. Notification Provider

This is the entry point. It can be any system that wants to send notifications.

Example: Zomato's marketing system triggers an offer like **"Weekend Special: Flat 40% off on your next order!"**

This service does not send messages directly. Instead, it sends a request to the **Notification Service**.

2. Notification Service

This is the heart of the system. It receives notification requests, validates them, and saves them. It does not send messages directly, because sending can take time and may overload the system.

So instead, it pushes each notification into a **Message Queue**. This keeps the system fast and stable even when traffic is high.

The Notification Service also talks to the **User Service** to fetch user details like email, phone number, device tokens, and notification preferences. It may also communicate with the **Scheduler Service** when notifications must be sent later (e.g., tomorrow at 10 AM).

3. User Service

This service reads information about user like whether he opted for notifications or not and user device details

4. Scheduler Service

Some messages should not be sent immediately. For example:

- "Happy Birthday Rahul!" - must be sent at 9:00 AM on his birthday
- "Friday Offer - 50% Off Tonight!" - send at 6:00 PM

These scheduled messages are stored in a database and picked later by the Scheduler Service, which pushes them to the **Message Queue** when the time comes.

5. Message Queue

This is a buffer between receiving notifications and sending them. By decoupling the notification request submission from the notification delivery, the system safe is during heavy traffic.

Once the Notification Service has created and formatted the messages for the required channels, it places each message into the respective topic in the Notification Queue System (e.g., Kafka, RabbitMQ, AWS SQS).

Kafka is ideal for large-scale notification systems like Zomato or Swiggy because it handles millions of events, guarantees ordering per user, supports retries and dead-letter queues

6. Notification Processor

This service takes messages from Kafka and sends them through the right channel:

- Email to Email provider (SendGrid)
- SMS to SMS provider (Twilio)
- Push to Firebase Cloud Messaging

Additional Questions:

1. How can we implement Rate Limiting?

Rate limiting protects users from spam and controls system load. We can store send counts per user and per service in Redis with expiry time. Before sending, we check the limit. If the limit is crossed, the request is delayed or rejected with a safe error message.

2. How to handle failures?

Failures are common in notification systems because vendors may timeout or reject requests. We handle this with retry logic and a dead letter queue. Temporary failures are retried automatically with backoff. Permanent failures can be sent to a separate queue for manual investigation without blocking the main flow

3. What is a Dead Letter Queue (DLQ)?

A DLQ is a special queue used to capture messages that can't be processed successfully after multiple attempts. Instead of letting failed messages be lost or dropped, they are stored in the DLQ for further inspection and action.

References:

[Designing a Scalable Notification System: From HLD to LLD | by Tanushree | Medium](#)

[Design a Scalable Notification Service - System Design Interview](#)

Video link: [System Design - Part 14 | Design a Notification System | SMS , Email and Push](#)

269. HLD 8: Design Payment System

Idempotency is main part while payment system.

Refer Question: Real-World System Design Scenarios-1 If user clicks two times on pay now, how would you prevent double payments?

1. Functional Requirements

- Users can make payments using credit card, debit card, UPI, or wallet.
- The system should verify payment details and process the transaction securely.
- Notify merchant and user once payment is completed or failed.
- Support integration with external payment gateway

2. Non-Functional Requirements

- High security for sensitive payment data (encryption, tokenization).
- High availability and fault tolerance.
- Scalability during festival sales or high-traffic events.
- Low latency for real-time transaction updates.
- Reliability to avoid double payments.

3. Components

Component	Description
Payment Service	Core logic for processing payments, communicating with external payment gateways.
Payment Gateway Adapter	Connects with third-party gateways like Razorpay, Stripe, or PayPal.
Notification Service	Sends SMS, email, or push notifications for payment updates.
Message Queue (Kafka/RabbitMQ)	For async communication between services, useful for retries or notification triggers.
Cache (Redis)	Stores short-lived transaction states or tokens for quick access.
Database (SQL)	Stores transaction details, refund logs, and user payment methods.

4. Database

We can use **SQL** since transaction consistency is critical.

Table	Columns	Description
Users	user_id, name, email, phone	Stores basic user details.
Transactions	txn_id, user_id, amount, status, created_at	Main table tracking payment status.
Payment_Methods	method_id, user_id, type, token, last4	Saves user's saved cards or UPI details.
Refunds	refund_id, txn_id, amount, status, created_at	Tracks refund requests.

5. Design APIs

API	Method	Description
/initiate-payment	POST	Starts a payment request. Takes user_id, merchant_id, amount, and payment method.
/confirm-payment	POST	Confirms the transaction after external gateway approval.
/get-transaction-status/{txn_id}	GET	Returns the current status of a transaction.
/refund-payment	POST	Initiates refund for a successful transaction.
/webhook/payment-update	POST	Endpoint for gateways to send async updates (success/failure).
/transactions	GET	Returns list of user transactions.

Request Example POST /initiate-payment

```
{
  "user_id": "U123",
  "amount": 2500,
  "payment_method": "UPI",
  "upi_id": "codinglyf@upi",
}
```

Request Example POST /confirm-payment

```
{
  "user_id": "U123",
  "amount": 2500,
  "payment_method": "UPI",
  "upi_id": "codinglyf@upi",
  "callback_url": "https://ourapp.com/webhook/payment-update"
}
```

Webhook Example

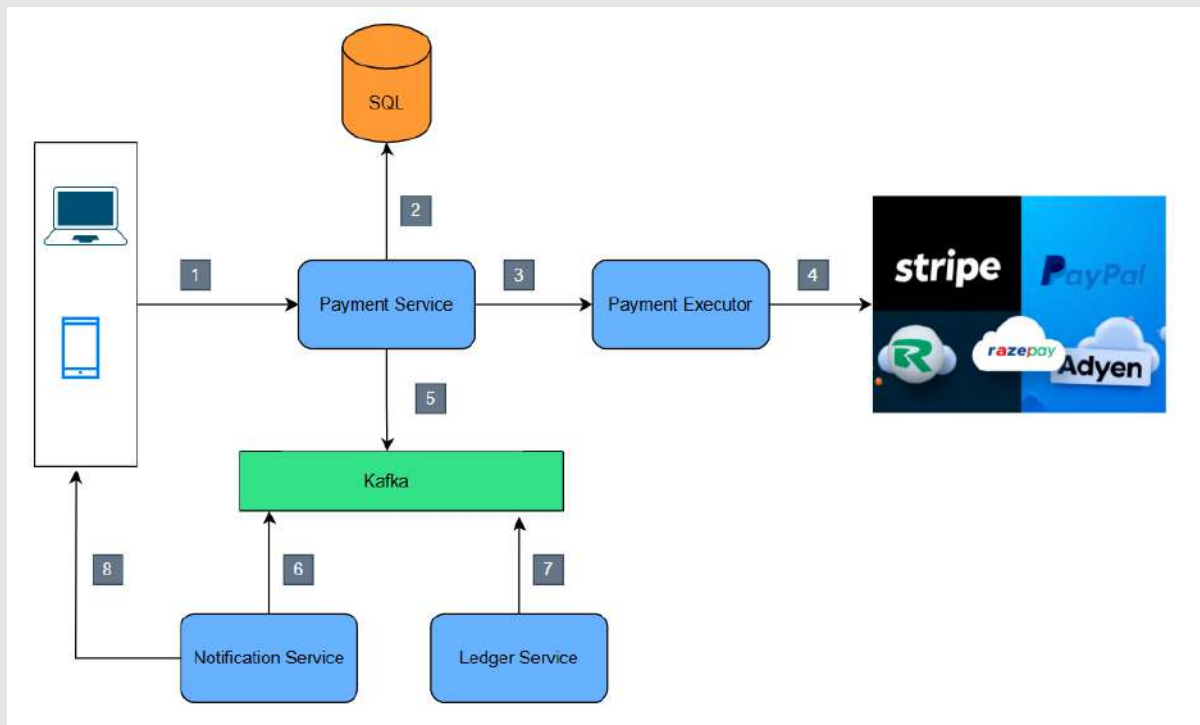
When payment is done, the external gateway sends a webhook to the system:

POST /webhook/payment-update

```
{
  "txn_id": "TXN89012",
  "status": "SUCCESS",
  "amount": 2500,
  "reference_id": "PG12345"
}
```

The Webhook Service validates the signature, updates the Transaction table, and sends notification to the merchant.

6.Architecture:



(Follow the numbers in the diagram)

1. Client to Payment Service

The payment starts from the web or mobile app.

The client sends details like amount, user_id, and payment method to the Payment Service through an API call `/initiate-payment`.

This starts the transaction process.

2. Payment Service to SQL Database

The Payment Service creates a new transaction record with status PENDING in the SQL database. This ensures every payment attempt is tracked with a unique transaction ID like TXN89012.

3. Payment Service to Payment Executor

After saving the transaction, the Payment Service calls the Payment Executor (`/confirm-payment`). The executor prepares a request to the right external payment gateway, depending on the method chosen, like UPI, card, or wallet.

4. Payment Executor to External Payment Gateway

The Payment Executor connects with gateways such as Stripe, Razorpay or PayPal. Once the gateway processes the payment, it sends a **webhook** to our system with the result. That webhook tells whether the payment was successful, failed, or pending. This callback ensures our system stays updated even if the user closes the app midway.

Example:

Razorpay sends

```
{
  "txn_id": "TXN89012",
  "status": "SUCCESS",
  "reference_id": "PG99876"
}
```

to our `/webhook/payment-update` endpoint.

Payment service updates the same transaction in SQL with the new status.

5. Payment Service to Kafka

Once the transaction status is confirmed, the Payment Service publishes an event into Kafka. This event is consumed by other services like Notification and Ledger, so they act on the completed payment without delays.

6. Kafka to Notification Service

The Notification Service reads the payment event from Kafka. It sends messages or emails to inform the user about payment success or failure.

7. Kafka to Ledger Service

The Ledger Service listens to Kafka events and records financial details for accounting and auditing. This ensures all money movements are traceable and consistent. The ledger system is very important in post-payment analysis, such as calculating the total revenue of the e-commerce website or forecasting future revenue.

8. Notification Service to Client

Finally, the Notification Service sends confirmation to the client app. The user sees a “Payment Successful” message or a retry option if it failed.

References: [Designing a Payment System - by Gergely Orosz](#)

[Designing Payment Processing System for E-commerce](#)

To know how webhook works in Stripe and Razorpay, Refer these:

[Webhook vs API: When to Use Each & Stripe \(Payment Gateway\) Webhook Integration in Java | by Priya Srivastava | Medium](#)

[About Webhooks | Razorpay Docs](#)

270. HLD 9: Design Ticket booking system or Hotel Booking system

In a booking system, users search, reserve, and pay for available options like seats or rooms. It handles real-time availability, locks selections to prevent double booking, processes secure payments, and confirms bookings instantly. Core parts include search, booking, payment, webhook, and notification services for a smooth end-to-end experience.

Let's design a simple ticket booking system that can be used for **airlines**, **bus** reservations, **hotels**, or apps like **BookMyShow**.

For In-depth search – Refer Question No. HLD 5

For In-depth payment design – Refer Question No. HLD 8

1. Functional Requirements

- Users can search for available tickets based on location, date, and category.
- Users can book tickets, cancel bookings, and view booking history.
- The system should hold seats temporarily during payment to avoid double bookings.
- Payment should be integrated with external payment gateways.
- Notifications should be sent after successful booking or cancellation.

2. Non-Functional Requirements

- High availability and scalability during peak booking times (like festivals or weekends).
- **Consistency**. no two users should get the same seat.
- Implement locking system, so that other users won't see while another is booking
- Secure handling of payment and personal data.
- Low latency when searching for available seats or shows.

3. Components

Component	Description
API Gateway	Entry point for all user requests. Routes to respective services.
Load Balancer	Distributes incoming traffic evenly across multiple service instances.
User Service	Manages user registration, login, and profiles.
Search Service	Finds available shows, flights, or buses based on filters.
Booking Service	Handles ticket reservation, seat locking, and booking confirmation.
Payment Service	Integrates with payment gateways and updates payment status.
Notification Service	Sends booking confirmation and cancellation alerts via email or SMS.

Component	Description
Cache (Redis)	Stores frequently searched data like routes, movies, and seat availability.
Database (SQL)	Maintains user data, bookings, payments, and seat details.
Message Queue (Kafka/RabbitMQ)	For async updates like sending notifications or ledger entries.

4. Database

We use **SQL** because consistency and transactions are important.

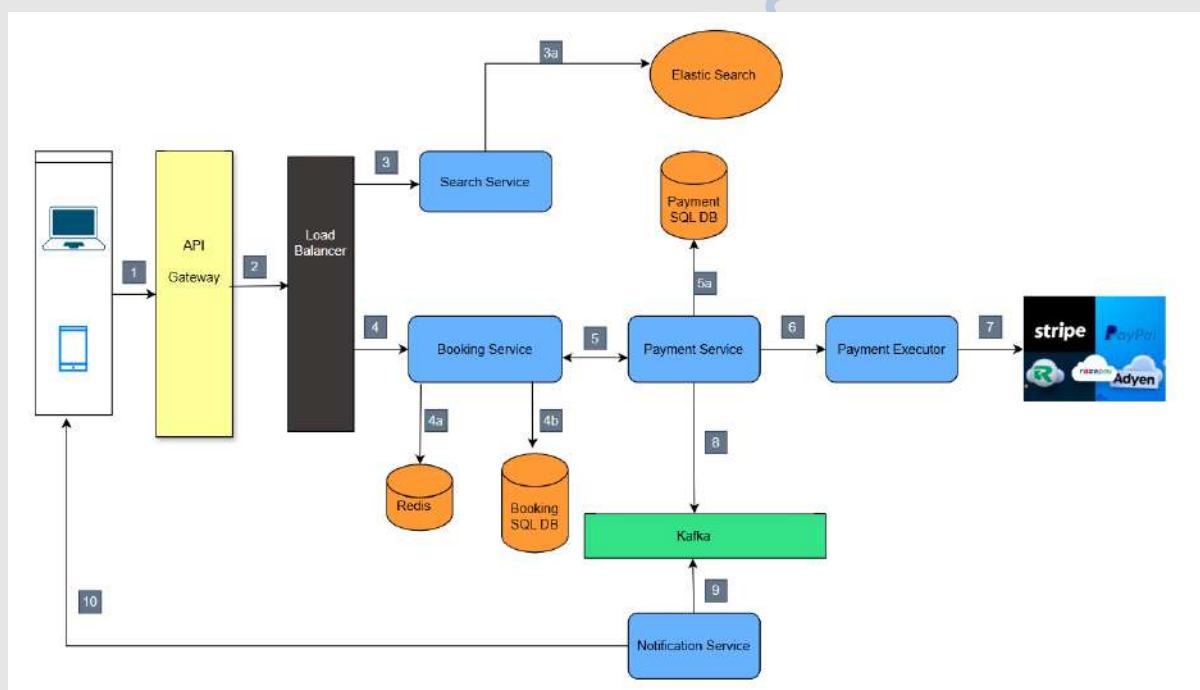
Table	Columns	Description
Users	user_id, name, email, phone, password_hash	Stores user details.
Shows / Routes / Flights	id, name, start_time, end_time, origin, destination	Information about movies, buses, or flights.
Seats	seat_id, show_id/flight_id, status (available/locked/booked)	Stores seat information.
Bookings	booking_id, user_id, show_id/ flight_id, seat_id, amount, status	Records ticket bookings.
Payments	payment_id, booking_id, amount, status, gateway_ref	Stores payment transaction details.

5. Design APIs

API	Method	Description	Request
/search	GET	Returns available flights, buses, or movies for given filters.	
/lock-seat	POST	Locks selected seats for a few minutes to prevent double booking.	
/book	POST	Confirms booking after successful payment.	{"user_id": "U123", "show_id": "S456",

API	Method	Description	Request
			"seat_ids": ["A1", "A2"], "amount": 600 }
/cancel/{booking_id}	POST	Cancels an existing booking and releases the seat.	
/payment/initiate	POST	Starts the payment process for a booking.	
/payment/webhook	POST	Receives async payment updates from the gateway.	
/bookings/{user_id}	GET	Lists all bookings made by the user.	

6. Architecture:



(Follow the numbers in diagram)

1. User to API Gateway

The user (web or mobile) sends a request through the app. All requests first go to the API Gateway, which handles authentication, rate limiting, and routing. It ensures that only valid and secure requests move forward to backend services.

2. API Gateway to Load Balancer

The API Gateway forwards the request to the Load Balancer, which distributes the traffic evenly across multiple instances of backend services like Search Service and Booking Service. This improves performance and reliability by preventing any single service instance from getting overloaded.

3. Load balancer to Search Service

The user searches for available buses, flights, or movie shows. The **Search Service** fetches data from **ElasticSearch**, which makes search fast and smooth.

3a. Search Service to ElasticSearch

ElasticSearch stores indexed show and route details for quick lookups, much faster than querying SQL.

4. Load balancer to Booking Service

The user selects seats and sends them to the **Booking Service**. Here comes the important part that is seat locking.

4a. Booking Service uses Redis

When seats are selected, the Booking Service first checks **Redis**. If seats are available, it temporarily locks them by storing something like S456:A1 -> locked by U123 (expires in 5 minutes). Redis automatically removes the lock after expiry. This ensures no one else books the same seat at that time.

4b. Booking Service to Booking SQL DB

The service creates a booking record with status **PENDING** and links the locked seats.

5. Booking Service to Payment Service

Once the booking is ready, Booking Service calls **Payment Service** with booking details and amount.

5a. Booking Service to Payment SQL DB

Payment service saves Payment record in SQL DB with status like Payment_Initiated

6. Payment Service to Payment Executor

Payment Service validates:

- whether the booking exists and is still active,
- whether the amount matches,
- whether the user is authenticated.

It then sends payment details securely to **Payment Executor**, which connects with the right gateway.

7. Payment Executor to Payment Gateway

Gateways like Stripe or Razorpay handle all card, wallet, and UPI validations. After processing, they send a **webhook** to our system with the payment status. That webhook updates the **Payment SQL DB** and marks the payment as SUCCESS or FAILED.

8. Payment Service to Kafka

After the webhook update, Payment Service pushes an event to **Kafka**. Kafka carries messages like payment_success or payment_failed to other services.

9. Kafka to Notification Service

The **Notification Service** reads from Kafka and sends messages to users confirming their booking or refund.

10. Notification Service to Client

The user receives the final confirmation through app notification, email, or SMS.

7. Recap

1. User searches for buses, flights, or movies through the Search Service, which fetches results from Elasticsearch for fast search.
2. Search results show available seats, times, and prices fetched from indexed data.
3. When the user selects seats, the Booking Service checks Redis to see if they're available.
4. Booking Service locks seats temporarily in Redis (for 5 minutes) to prevent double booking, then stores booking details in the Booking SQL DB as PENDING.
5. The Booking Service calls the Payment Service to start payment, sending booking_id, user_id, and amount.
6. Payment Service validates booking status, seat lock, and amount before calling Payment Executor.
7. Payment Executor interacts with gateways like Stripe, Razorpay or PayPal, which verify card or UPI details and process the transaction.
8. Gateway sends a webhook to update payment status in the Payment SQL DB once payment is done.
9. Payment Service pushes an event to Kafka, which notifies other services about the payment result.
10. Notification Service reads from Kafka and informs the user through app, email, or SMS that the ticket is booked or payment failed.

Reference:

For Airlines: (26)  07.2 Case Studies: Airline Booking System – A Scalable System Design Case Study | [LinkedIn](#)

For TicketMaster/ BookMyShow: [Design a Ticket Booking Site Like Ticketmaster | Hello Interview System Design in a Hurry](#)

271. HLD 10: Library Management system

A Library Management System helps manage books, members, and borrowing activities efficiently.

It handles book inventory, user registration, issuing and returning books, and tracking due dates or fines

1. Functional Requirements

- Members can search for books by title, author, or category.
- Members can borrow and return books.
- Admins can add, remove, or update books.
- System should track due dates and calculate fines for late returns.
- Notifications are sent for due dates and overdue fines.

2. Non-Functional Requirements

- Data consistency between books and member records.
- Fast search using indexes or caching.
- Scalable enough to handle large libraries or institutions.
- Secure access control for members and admins.

3. Components

Component	Description
API Gateway	Handles incoming requests and routes them to correct services.
User Service	Manages members and admin registration, login, and profiles.
Book Service	Maintains book catalog and availability.
Borrow Service	Handles issuing, returning, and renewing books.
Notification Service	Sends reminders for due dates and overdue books.
Payment Service	Handles fine payments for late returns.
Cache (Redis)	Stores frequently accessed book data for faster search.
SQL Database	Stores user, book, and transaction records.
Kafka (optional)	Sends async events like due-date reminders or fine updates.

4. Database

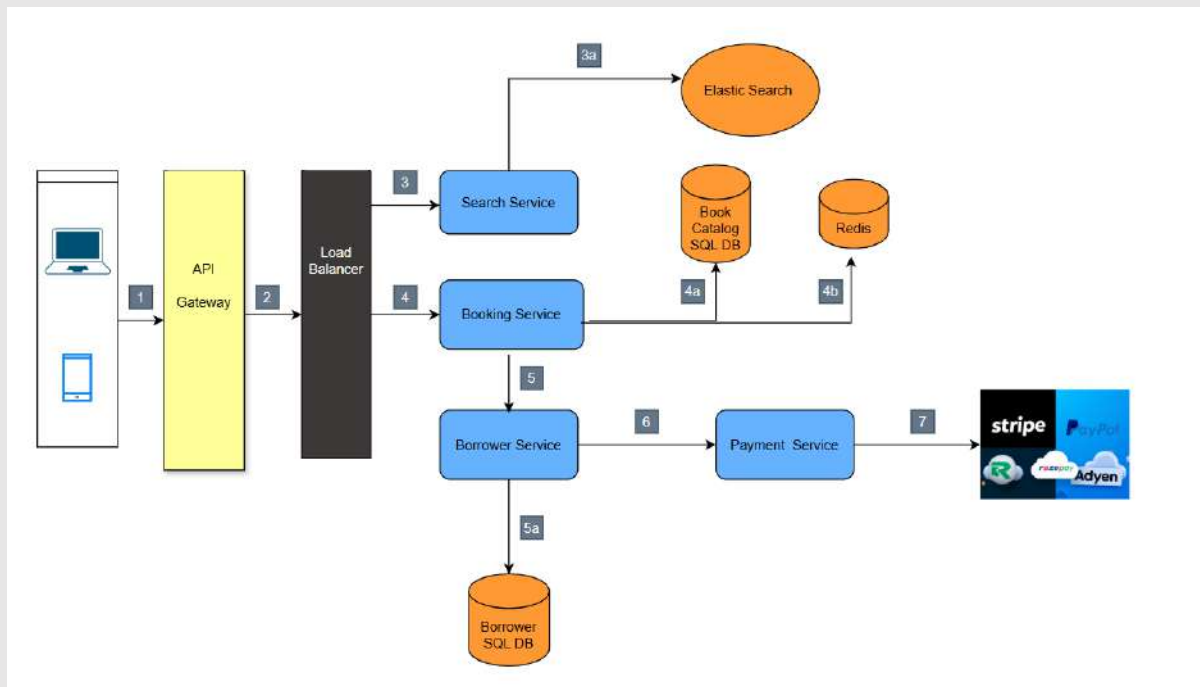
We use **SQL** because consistency and transactions are important.

Table	Columns	Description
Users	user_id, name, email, phone, password_hash	Stores user details.
Books	id, name, genre	Information about books
Borrow	booking_id, user_id, book_id, amount, status	Information about borrowing
Payments	payment_id, booking_id, amount, status, gateway_ref	Stores payment transaction details.

5. Design APIs

API	Method	Request Body	Description
/books/search	GET	None	Fetches books that match given filters.
/books/issue/{book_id}	POST	json { "user_id": "U101" }	Issues the selected book to the member and reduces available copies.
/books/return/{book_id}	POST	json { "user_id": "U101" }	Returns a borrowed book, updates stock, and checks for fines.
/books/renew/{book_id}	POST	json { "user_id": "U101" }	Extends the due date for the borrowed book if allowed.
/users/register	POST	json { "name": "John Doe", "email": "john@gmail.com", "role": "member", "password": "pass123" }	Registers a new member or admin in the system.
/fines/pay/{user_id}	POST	json { "amount": 50, "reason": "Late return", "payment_method": "card" }	Pays pending fine amount and updates payment record.

6. Architecture:



(Follow the numbers in diagram)

1. Client to API Gateway

The user opens the library app or website to search, issue, or return books. The request first goes to the API Gateway, which routes it to the correct service.

2. API Gateway to Load Balancer

The API Gateway forwards the request to the Load Balancer. The Load Balancer distributes traffic across multiple instances of the Search, Booking, or Borrower services for better performance.

3. Load Balancer to Search Service

The Search Service handles all search queries. When a user searches for a book by title or author, it checks data stored in ElasticSearch for fast and relevant results.

3a. Search Service to ElasticSearch

ElasticSearch stores indexed book data like title, author, and category. This helps in returning results quickly even when the library has thousands of books.

4. Load Balancer to Booking Service

When a book is selected, the Booking Service takes over. It checks if copies are available in the Book Catalog Database and ensures no one else is issuing the same copy at the same time.

4a. Booking Service to Book Catalog SQL DB

It decreases the available_copies count. This ensures data consistency between available stock and issued books.

4b. Booking Service to Redis

To prevent double issuing, the Booking Service temporarily locks the book entry in Redis for a short time. If payment or confirmation is delayed, Redis releases the lock automatically.

5. Booking Service to Borrower Service

After issuing, the Booking Service passes borrower details and the book record to the Borrower Service. This service maintains who borrowed which book and tracks the return status.

5a. Borrower Service to Borrower SQL DB

The Borrower Service inserts a new record into its database with user ID, book ID, issue date, due date, and current status. It updates this record once the book is returned.

6. Borrower Service to Payment Service

If the user has pending fines for late returns or needs to pay a membership renewal fee, the Borrower Service sends a payment request to the Payment Service.

7. Payment Service to Payment Gateway

The Payment Service validates the request and then connects to third-party gateways like Stripe, Razorpay, or PayPal. Once payment is done, the gateway sends back a success or failure response through a webhook.

Return Flow

When the user returns a book, the Borrower Service receives the request and updates the Borrower SQL DB with the return_date and calculates any fine if it's overdue.

It then calls the Booking Service, which increases the available_copies count in the Book Catalog SQL DB. Finally, the record in Redis is cleared, making the book available again for the next borrower.

7. Recap

1. The user searches for books through the API Gateway, which routes requests to the Search Service using Elasticsearch.
2. The Booking Service checks availability, decreases available copies, and locks the book in Redis.
3. The Borrower Service records issue details and due dates in its database.
4. The Payment Service handles any fine or membership payment through external gateways.
5. On return, the system updates databases and restores available copies for the next borrower.

272. HLD 11: Recommendation system

A recommendation system is nothing but suggestions on platforms like Netflix, Amazon, or Spotify. It helps users discover items they may like based on their interests and past behaviour.

1. Functional Requirements

- Show personalized recommendations for each user.
- Support different types of recommendations like "Similar items," "Trending now," and "For you."
- Continuously update suggestions as new users, items, and interactions come in.
- Collect user interactions such as likes, clicks, purchases, or ratings.

2. Non-Functional Requirements

- Low latency for real-time recommendations.
- Scalability to handle millions of users and products.
- Data consistency between stored items and generated recommendations.

3. Components

Component	Description
API Gateway	Handles all incoming requests from apps or websites and routes them to the right service.
Load Balancer	Distributes traffic evenly across multiple Recommendation Service instances.
Recommendation Service	Core logic that fetches, scores, and ranks recommended items for each user.
User Service	Stores user profiles, preferences, and activity history.
Model Service	Hosts and serves machine learning models for generating recommendations.
Feature Store	Stores user and item embeddings (numerical representations used by models).
Cache (Redis)	Keeps precomputed recommendations for fast responses.
SQL / NoSQL Database	Stores items, users, and interaction logs for long-term reference.
Data Pipeline (Kafka + Spark)	Processes live and historical data for model updates.

4. Database

We can use NoSQL Database like Cassandra or Mongo DB to store User Interactions and Recommendations as they can handle high volume data

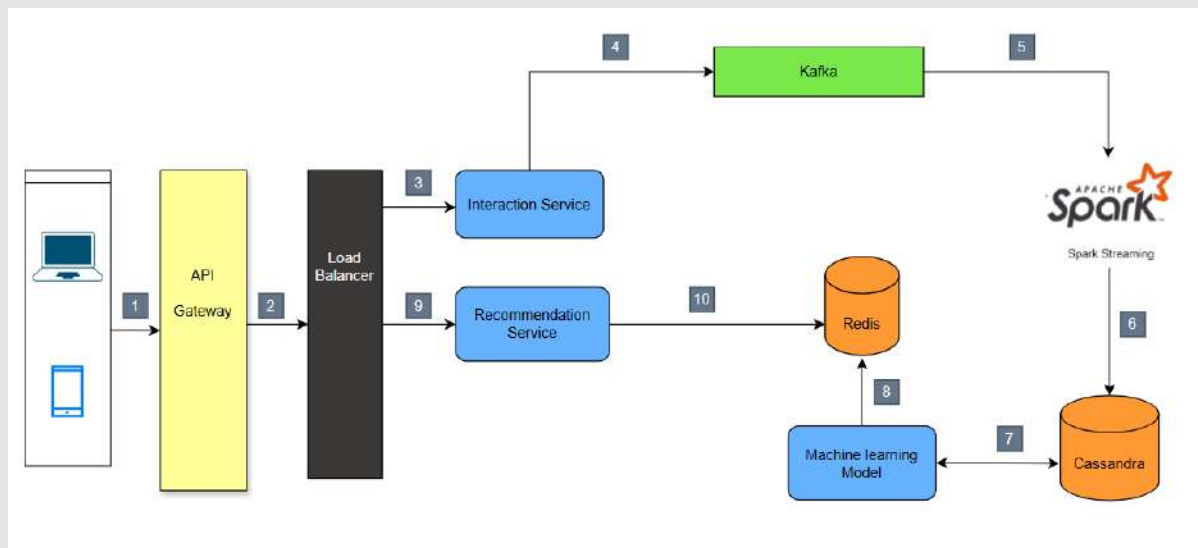
Table	Columns	Description
User_Interactions	user_id, item_id, action_type, rating, timestamp	Stores clicks, likes, or purchases.
Recommendations	user_id, recommended_items[], model_type	Saves precomputed recommendations.

5. Design APIs

API	Method	Description
/recommendations/{user_id}	GET	Returns top recommendations for a user.
/items/trending	GET	Returns currently trending or popular items.

API**Method Description**

/user/{user_id}interactions POST Records user actions like clicks, likes, or purchases.

6.Architecture:

(Follow the numbers in diagram)

1. Client to API Gateway

The user opens the app or website to get personalized recommendations. The request first reaches the API Gateway, which acts as the single-entry point for all incoming traffic. It routes the request safely to the backend services that handle interactions or recommendations.

2. API Gateway to Load Balancer

The API Gateway sends the request to the Load Balancer. The Load Balancer checks which backend instance has the least load and forwards the request there. This helps manage heavy traffic, especially when thousands of users are active at once.

3. Load Balancer to Interaction Service

The Interaction Service captures every user activity like views, clicks, likes, or purchases. These actions help us understand what each user is interested in. For example, if a user often clicks on fitness shoes, this data helps in recommending similar items later.

4. Interaction Service to Kafka

Once the Interaction Service collects these actions, it sends them to Kafka. Kafka acts like a message queue that keeps all user events in order and makes sure no data is lost, even when the system is busy.

5. Kafka to Spark Streaming

Spark reads these events from Kafka and processes them in real time. It converts raw data into

useful information such as “top categories viewed,” “most active users,” or “trending items.” This processed data is prepared for model training and stored efficiently.

Spark: Apache Spark is an open-source, distributed computing system for fast and general-purpose big data processing, capable of handling tasks like batch processing, real-time analytics, and machine learning

6. **Spark Streaming to Cassandra**

After processing, Spark writes the cleaned and structured interaction data into Cassandra. This database holds user-item interaction history, making it easy to fetch patterns when the model retrains. Cassandra works well here because it handles high-speed writes and stores large amounts of time-based data.

7. **Cassandra to Machine Learning Model**

The Machine Learning Model reads data from Cassandra to learn how users behave. It applies algorithms like collaborative filtering, content-based filtering, or hybrid methods to find relationships between users and items. The model is retrained at regular intervals so it always reflects the latest user behaviour.

8. **Machine Learning Model to Redis**

Once the model finishes training, it generates new recommendations for each user and stores them in Redis. Redis is used as a cache because it is fast. When a user opens the app, recommendations can be fetched in milliseconds without running the model again.

9. **Load Balancer to Recommendation Service**

When the user requests recommendations, the Load Balancer forwards the request to the Recommendation Service. This service fetches personalized results for that user.

10. **Recommendation Service to Redis**

The Recommendation Service pulls the ready recommendations from Redis and sends them back to the user instantly. If Redis doesn't have data for that user, it can request the model to generate new ones and update the cache.

273. **HLD 12: Banking System**

A banking system manages customer accounts, transactions, and balances securely and efficiently. It handles operations like deposits, withdrawals, and fund transfers while ensuring data consistency and fraud prevention.

1. **Functional Requirements**

- Create and manage customer accounts.
- Customer can update their profiles like address, email
- Handle deposits, withdrawals, and fund transfers.
- Maintain real-time account balances and transaction history.
- Send transaction alerts and monthly statements.

- Support authentication and authorization for customers and staff

2. Non-Functional Requirements

- High security for sensitive financial data.
- Consistent and reliable transaction handling.
- Scalable to handle millions of customers.
- Fault tolerance and quick recovery on failures.
- Fast response for balance and transaction queries.

3. Components

Component	Description
Authentication Service	Verifies user credentials and manages session tokens.
Account Service	Manages account creation, balance, and details.
Transaction Service	Handles deposits, withdrawals, and fund transfers.
Loan Service	Manages loan applications, interest calculations, and repayment plans.
Payment Service	Online payments, bill payments, and card processing.
Notification Service	Sends SMS or email alerts for every transaction.
Kafka (Message Queue)	Used for async updates between services like transaction and notification.
SQL Database	Stores structured data like accounts, users, and transactions.
No SQL	To Store Logs and notifications

4. Database

We need to both SQL and NoSQL.

We use **SQL Database** (like PostgreSQL or MySQL) because banking needs strong consistency and ACID transactions.

Tables:

Customer Table

customer_id	name	phone	email
101	Rahul Sharma	9876543210	rahul@gmail.com

Account Table

account_id	customer_id	balance	type
5001123456	101	12500.75	Savings

Transaction Table

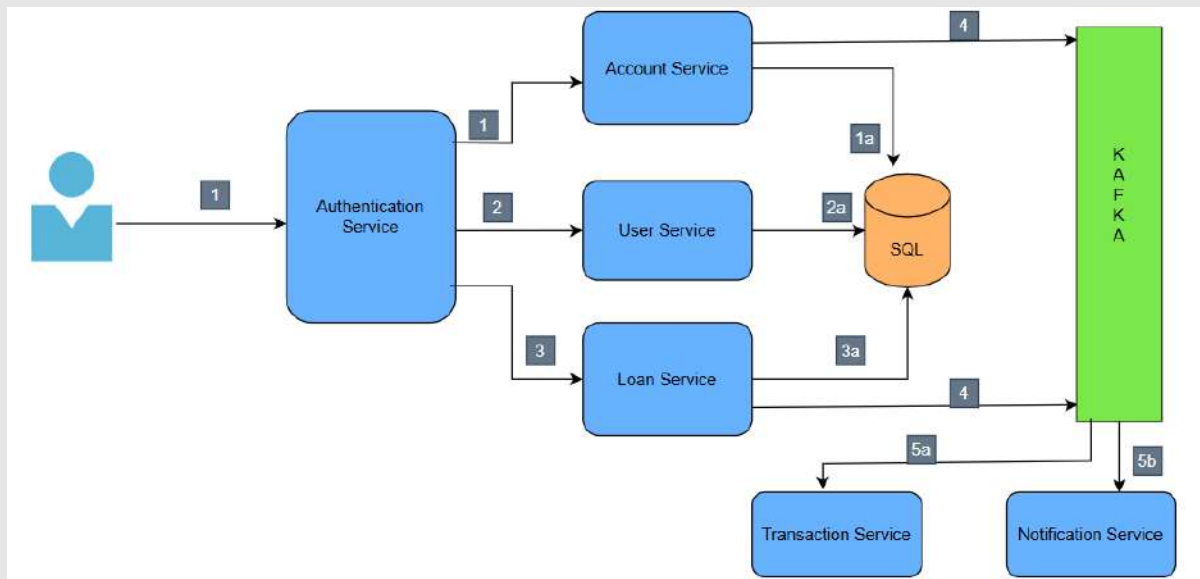
txn_id	account_id	type	amount	time	status
90045	5001123456	Withdraw	2000.00	2025-11-10 10:42:16	Success

NoSQL Databases: For services handling unstructured data like notifications or logs, NoSQL databases like MongoDB or Cassandra may be used.

5. Design APIs

API	Method	Description	Request Example
/users/login	POST	Authenticate user credentials	{ "username": "rahul", "password": "1234" }
/account/create	POST	Create new account	{ "customer_id": 101, "type": "Savings" }
/account/balance/{id}	GET	Check account balance	account_id=5001123456
/transaction/deposit	POST	Deposit money to an account	{ "account_id": 5001123456, "amount": 1000 }
/transaction/transfer	POST	Transfer money between accounts	{ "from": 5001123456, "to": 5001134444, "amount": 1500 }
/notifications/send	POST	Send transaction alert	{ "account_id": 5001123456, "message": "₹1500 transferred" }

6.Architecture:



(Follow the numbers in diagram)

1. Customer to Authentication Service

The customer logs in using mobile or web banking. The Authentication Service checks the username and password. Once verified, it issues a token that allows access to all banking features.

2. Authentication Service to Account Service

After successful login, the Account Service manages customer accounts, handles balance checks, deposits, and transfers. For example, Rahul checks his savings account balance before transferring ₹2000 to a friend.

3. Authentication Service to User Service

The User Service stores customer details like name, phone number, and address. It updates any profile change, for instance, when Rahul updates his contact number from 98765 to 91234.

4. Authentication Service to Loan Service

The Loan Service helps customers view their ongoing loans, EMI schedules, and apply for new loans. For example, Rahul checks his remaining car loan balance and EMI due date.

5. Account, User, and Loan Services to SQL

All these services store and retrieve data from SQL. For instance, when Rahul transfers ₹2000, SQL updates his balance and creates a transaction record with a timestamp.

6. Account, User and Loan Services to Kafka

Whenever a transaction or loan update happens, events are sent to Kafka. This helps other services like Transaction and Notification stay updated without slowing down the main process.

7. Kafka to Transaction Service

The Transaction Service listens to Kafka events and records each transaction for history, dispute tracking, and monthly statements.

8. Kafka to Notification Service

The Notification Service also listens to Kafka. It instantly sends Rahul an SMS like “₹2000 transferred to account ending 5432 successfully.”

Reference: [\(25\) Banking application system design | LinkedIn](#)

274. HLD 13: News feed service design (Facebook/Instagram/Twitter)

Newsfeed is an important feature in Facebook, Instagram, Twitter where a user will be able to see his friend's photos, posts, video and other activities in the scrollable format

1. Functional Requirements

- Newsfeed is based on the posts from people, pages and groups that a user follows. For simplicity let's assume only a user can post.
- User may follow friends, groups or pages.
- Feed may contain photos, videos or just text.
- The feed should automatically update the active user's feed as and when new posts arrive.

2. Non-Functional Requirements

- Maximum latency is 2s for any user's newsfeed.
- When a user posts anything, it should not take no more than 5s to appear on his follower's feed.

3. Components

Component	Purpose
Load Balancer	Distributes incoming requests evenly to prevent overload.
API Gateway	Handles authentication and routes requests to the correct service.
Post Service	Handles post creation, captions, and media linking.
Fanout service	Fanout is the process of delivering a post to all friend
News Feed Service	Fetches the feed and show it to the followers
Cache (Redis)	Stores feed lists and counts for faster access.
Message Queue (Kafka)	Sends background events like "new post created."
Database	Stores user details, posts, and relationships.

4. Database

To support core functionality of our news feed system, we maintain several key entities (tables) in the database. These form the foundation for features like post creation, following, liking, commenting, and feed generation.

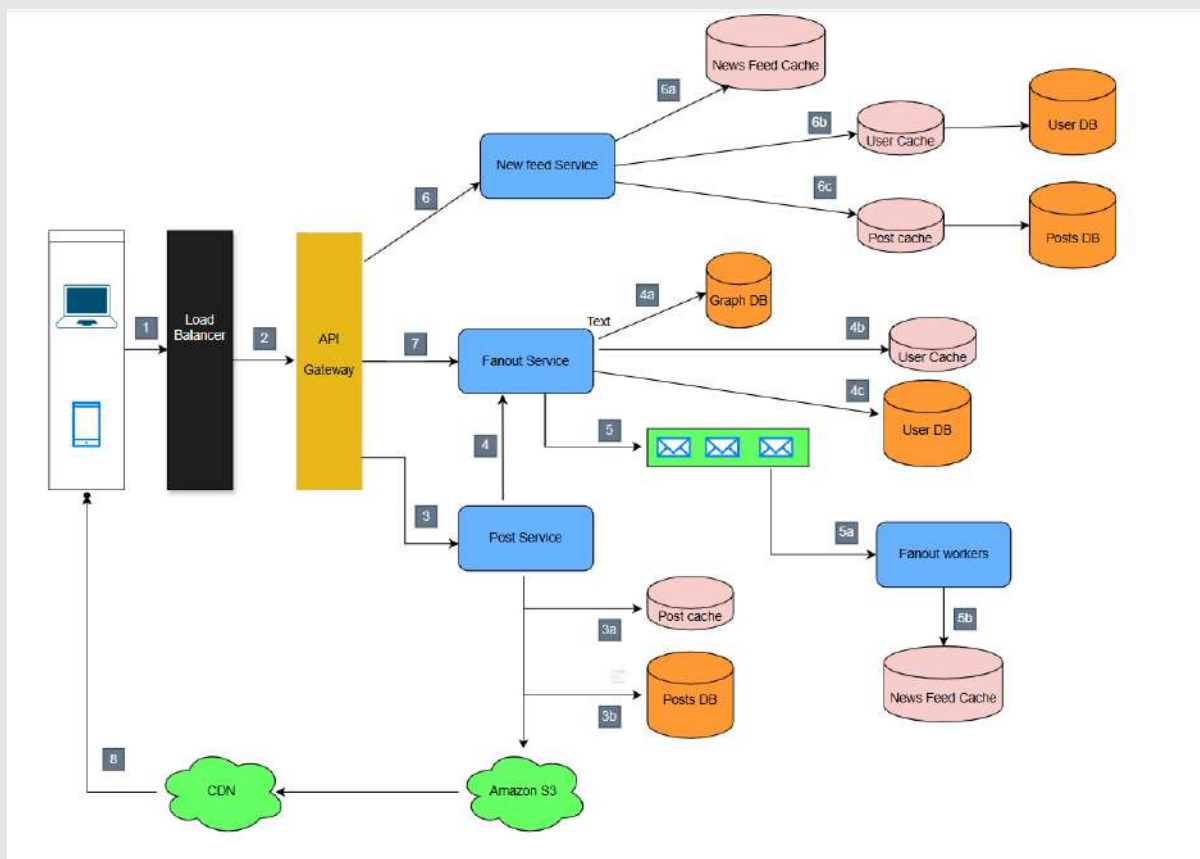
- **Users:** Stores user profile information.

- **Schema:** user_id, name, bio, profile_pic_url
- **Posts:** Stores individual posts created by users.
 - **Schema:** post_id, user_id, content, timestamp, like_count, comment_count
- **Follows:** Represents the social graph, who follows whom.
 - **Schema:** follower_id, followee_id
- **Likes:** Tracks which user liked which post, along with the timestamp of the action.
 - **Schema:** post_id, user_id, timestamp
- **Comments:** Stores comments added to posts. Each comment is associated with a specific post and user.
 - **Schema:** comment_id, post_id, user_id, content, timestamp

All of this is stored in a relational database such as PostgreSQL or MySQL.

We add indexes on commonly queried fields like user_id, post_id, and timestamp to support efficient lookups, sorting, and joins.

5. Architecture: Go through the explanation along with architecture diagram. If you only see the diagram, it can be confusing.



We will divide this into 3 main flows

Post service: Which handles the posts uploaded by the user, referred as Feed publishing.

Fanout service: Fanout service pushes the content to friend's news feed, NewsFeed cache will have this data for faster retrieval

Newsfeed service: Fetches news feed from the cache.

1. Post Service (Feed publishing):

1. A user posts some content "I am feeling great"
2. Load balancer distributes the traffic to API server
3. API server checks for user authenticity and rate limiting service
4. Post service posts the data in the cache and DB
5. It saves Media (Images and Video) in cloud storage like Amazon S3.
6. It calls the Fanout service, to push the content to User friends [4].

2. Fanout Service: The fanout service works as follows

1. Suppose user-A has posted a video. As soon as user-A posted, fanout service fetches his friend IDs from the graph database. [4a]

Graph databases are suited for managing friend relationship and friend recommendations

2. Fanout service get friends info from the user cache and user db. The system then filters out friends based on user settings. For example, if you mute someone, posts will not show up on your news feed even though you are still friends. [4b, 4c]
3. Now fanout service[7] Send friends list and new post ID to the message queue [5].
4. Fanout workers fetch data from the message queue and store news feed data in the news feed cache. You can think of the news feed cache as a $\langle post_id, user_id \rangle$ mapping table. Whenever a new post is made, it will be appended to the news feed table as shown in below .
5. Store $\langle post_id, user_id \rangle$ in news feed cache. Below image shows an example of what the news feed looks like in cache [5b].

post_id	user_id
post_id	user_id
post_id	user_id
post_id	user_id
post_id	user_id
post_id	user_id
post_id	user_id
post_id	user_id

6. Newsfeed service fetches the data from News Feed cache and display it in the feed of the user

But what if user-A million of followers or friends Ex: Virat Kohli ?

To solve this, there are two primary approaches that will be used by Fanout service for feed generation: pull (on-demand / fan-out-on-read) and push (fan-out-on-write).

Push-based (Fan-Out-on-Write) Model (which we discussed above)

In a push model, the system does the heavy work **upfront, when a new post is created**.

Whenever a user posts something, it find all their followers and immediately *insert that new post into each follower's feed store or cache*.

In practice, this often means maintaining a **precomputed feed** for each user (think of it like each user having an inbox of posts).

So, when we open the app, feed is basically already prepared and just needs to be fetched from a fast store (like an in-memory cache or a feed database). This results in super-fast feed read times (low latency) and much less database work at read time.

Pull-Based / fan-out-on-read: In this model, the news feed is generated on-the-fly every time a user opens the app not precomputed.

Here's how it works:

1. **Lookup followees:** The system first retrieves the list of users that the requesting user follows.
2. **Fetch recent posts:** Then it fetches recent posts from those users, sorted by timestamp. This is done using a SQL query like:
3. **Return feed to client:** The server returns the top 50/100 most recent posts as a JSON response. The client displays them in the feed UI.

This approach is called **fan-out-on-read**, because the feed is computed at read time by querying posts from all followed users.

But which approach is used when?

Doing **Push-based (Fan-Out-on-Write)** can be expensive when someone with **millions of follower's** posts, that single post might need to be copied to millions of feeds, which is a heavy write burst. In reality, **most large social platforms use a hybrid approach** to get the best of both worlds.

For the majority of users (who don't have enormous followings), the push model is used and their followers' feeds are updated in near real-time.

For celebrity or very popular accounts with millions of followers, a pure push is not suggestable, so those might use a pull based (the system might mark that "User X posted" and handle fetching it when those followers actually open their feed).

3. Newsfeed service

1. A user sends a request to retrieve her news feed. The request looks like this: `/v1/me/feed [6]`

2. News feed service gets a list post IDs from the news feed cache. **[6a]**
3. A user's news feed is more than just a list of feed IDs. It contains username, profile picture, post content, post image, etc. Thus, the news feed service fetches the complete user and post objects from caches (user cache and post cache) to construct the fully hydrated news feed **[6b, 6b]**.
4. The fully hydrated news feed is returned in JSON format back to the client for rendering.
5. Media content (images, videos, etc.) are served from CDN **[8]**

How Media Upload Works

Let's walk through what happens when a user creates a post with an image or video.

1. **Client requests upload URL:** The client requests the backend (Post Service) to provide a secure upload URL. The server responds with a **pre-signed URL** from object storage provider and derives a CDN-accessible URL from the storage path.
2. **Client uploads media directly:** Using the pre-signed upload_url, the client uploads the file **directly to object storage**. This reduces load on our servers since media never passes through them.
3. **Client creates the post:** After the upload completes, the client sends a POST /posts request with the post text, timestamp, and media URL. The **Post Service** saves the new post in the database with the media_url included.

How Media is Delivered?

When a user fetches their feed (via GET /feed), the server includes the media_url for each post in the response. The client then fetches the image or video directly from that location.

Since images and videos are large and bandwidth-intensive, we deliver them through a **Content Delivery Network (CDN)**.

A CDN is a network of globally distributed edge servers that cache content close to end users. When a user views their feed:

- Text and metadata come from our backend servers
- Images and videos are fetched in parallel from the **nearest CDN edge node**

This improves page load time, reduces latency, and takes the pressure off our origin servers.

7. Recap

1. When a user creates a post, the **Post Service** stores it in the **Posts DB** and cache.
2. When a user creates a post, the system decides between **push (fanout-on-write)** or **pull (fanout-on-read)** based on how many followers they have:
 - **Fanout-on-write (Push):**
Used for users with **normal or fewer followers**.

Their post is immediately pushed to each follower's **News Feed Cache** through **Fanout Workers**, **News Feed Service** Fetches **News Feed Cache** and displays to users. so it appears instantly in their feeds.

- **Fanout-on-read (Pull):**

Used for users with **millions of followers** (like celebrities or big accounts).

Instead of pushing the post to everyone's feed (which would be huge), the post is stored in the **Posts DB** and fetched **on demand** when a follower opens their feed by reading from the **Posts Cache**, **User Cache**, and **Graph DB**

3. Images and videos are served from **S3 via CDN**, ensuring quick content loading.

Reference for fan-out: [Twitter Timeline Architecture | Fanout | System Design](#)

Reference for Newsfeed: (These are the links I followed, if you want to learn more you can refer)

1. [System Design: News Feed System. Today, I would like to share how... | by Ishwarya Hidkimath | Medium](#)
2. [Designing a Scalable News Feed System \(Step-by-Step\)](#)
3. [How to Design Social Media News Feed](#)
4. [ByteByteGo | Technical Interview Prep](#)

If you want to watch the video: [System Design - Part 16 | Design a Newsfeed | Facebook, Instagram, Reddit timeline - YouTube](#)

275. **HLD 14.1: Design eCommerce application like Amazon (Merchandise Flow)**

We will divide this into 5 parts.

1. Search Design (**Refer Question No. 266 HLD-5**)
2. Merchandise Flow Design
3. Purchase Flow Design
4. Recommendations Design
5. Reviews Flow

We have already seen Search Flow: HLD 5.

In this section we will see Merchandise flow.

1. Functional requirements

- Create and update product details (name, brand, category, price, etc.)
- Upload product images and videos
- Track stock levels in different warehouses
- Sync catalog and inventory updates through Kafka
- Fetch published product details for frontend display

2. Non-functional requirements

- Quick product reads (under 100 ms)
- Horizontal scalability for high traffic

- Event-driven updates (using Kafka)
- Secure uploads (signed URLs)

3. Components

Component	Purpose
API Gateway	Entry point for all product and inventory APIs. Handles auth and routing.
Catalog Service	Manages product details, categories, and metadata in MongoDB. Publishes product events to Kafka.
Inventory Service	Tracks stock levels. Consumes events from Kafka when new products are created.
Media Service	Handles image and video uploads to Object Storage (like S3).
Kafka	Used to communicate product and inventory changes asynchronously.
MongoDB	Stores product and inventory collections.
Frontend / Admin Portal	Used by merchants to add, edit, and publish products.

3. Database

Choose MongoDB (flexible document structure, easy to store product variants and media info)

Collection	Key Fields
products	_id, title, category, price, stock_list, images, videos, status
inventory	_id, stock_id, stock, warehouse_id, updated_at
media_files	_id, product_id, type (image/video), url, size, uploaded_at

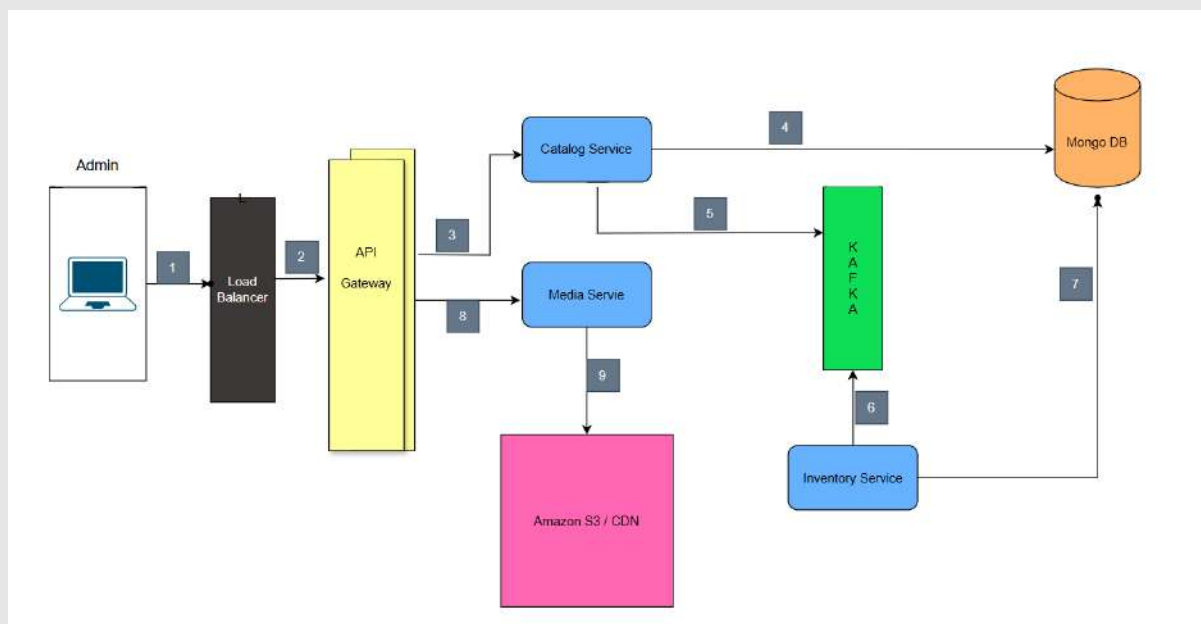
Example Product

```
{
  "_id": "p123",
  "title": "Wireless Mouse",
  "category": "Electronics",
  "price": 999,
  "stock_list": ["sku-001", "sku-002"],
  "images": ["https://cdn.site/mouse1.jpg"],
  "videos": ["https://cdn.site/mouse1.mp4"],
  "status": "published"
}
```

5. Design APIs

API	Method
Create Product	POST /api/catalog/products
Upload Image/Video	POST /api/media/upload
Update Inventory	POST /api/inventory/update
Get Product Details	GET /api/catalog/products/{id}

6. Architecture



Merchandise Flow (Follow the Numbers in the Diagram)

1. The admin sends a product request to the Load Balancer. It helps distribute requests evenly so that no single server gets overloaded, keeping the system stable and responsive.
2. The Load Balancer forwards the request to the API Gateway. The Gateway checks authentication, rate limits, and routes the request to the correct service like Catalog or Media.
3. The API Gateway sends the product creation request to the Catalog Service. This service handles all product details like name, category, price, and attributes before saving them.
4. The Catalog Service stores product data in MongoDB. MongoDB keeps the product document in a flexible JSON format, making it easy to handle variations in product information.
5. Once the product is created, the Catalog Service publishes a message to Kafka. This message contains product details so that other services can react without direct dependency.
6. The Inventory Service listens to the Kafka topic. When a new product message arrives, it automatically creates an inventory record and sets the initial stock count.

7. The Inventory Service updates MongoDB with the latest stock data. This ensures the product's availability and quantity are always in sync with actual inventory.
8. When the admin uploads images or videos, the request again goes through the API Gateway to the Media Service. This service handles file validation and upload setup.
9. The Media Service uploads the files to Amazon S3. These files are stored securely and served quickly to customers when they view product pages.

7. Recap / Flow

- Admin creates a new product , Catalog Service saves to MongoDB.
- Catalog Service sends product-created event to Kafka.
- Inventory Service consumes that event, creates stock entry in MongoDB.
- Admin uploads images, Media Service generates signed URLs and uploads to S3.
- Product details (with image URLs and inventory count) are fetched by frontend for display.

276. HLD 14.2: Design eCommerce application like Amazon (Purchase Flow)

We will divide this into 5 parts.

1. Search Design
2. Purchase Flow Design
3. Merchandise Flow Design
4. Recommendations Design
5. Reviews Flow

We have already seen Search Flow: Refer Question No HLD-5.

In this section we will see Purchase flow.

1. Functional Requirements

These are the main actions our system should perform:

- Users can add items to their cart and place an order from the cart.
- System should confirm product availability before order.
- Users can choose a delivery address and payment method.
- Payment should be processed securely through a payment gateway.
- Order details should be saved and visible to users.
- System should notify the user and seller after successful payment.
- Inventory should update automatically after a purchase.

2. Non-Functional Requirements

These define how the system behaves:

- **Performance:** Order creation should be quick, ideally under 2 seconds.
- **Scalability:** The system must handle thousands of orders per second during sales.
- **Security:** All payment and user details must be encrypted.

- **Reliability:** The system should handle partial failures without losing order data.
- **Availability:** It should be available 24x7.

3. Components

To build the purchase flow, we need these core parts:

Component	Purpose
Load Balancer	To distribute the load
API Gateway	Entry point for all requests. It forwards requests to the correct service.
Cart Service	Manages items added to cart before purchase.
Order Service	Creates and stores order details.
Payment Service	Handles communication with external payment gateways.
Inventory Service	Updates product stock after order confirmation.
Notification Service	Sends emails or messages about order status.
User Service	Fetches address and user info during checkout.
Database	Stores order, payment, and inventory data.
Message Queue (like Kafka)	Helps services communicate asynchronously, so no data is lost if one service fails.
Cache (like Redis)	Used to store frequently accessed product or cart details for faster checkout.

4. Database

We can use both SQL and NoSQL depending on data type.

- Orders need structured data so use SQL (PostgreSQL / MySQL).
- Users and Address need structured data so use SQL (PostgreSQL / MySQL).
- Cart and Inventory need quick reads and Inventory needs document type of data, so use NoSQL (MongoDB / Redis).

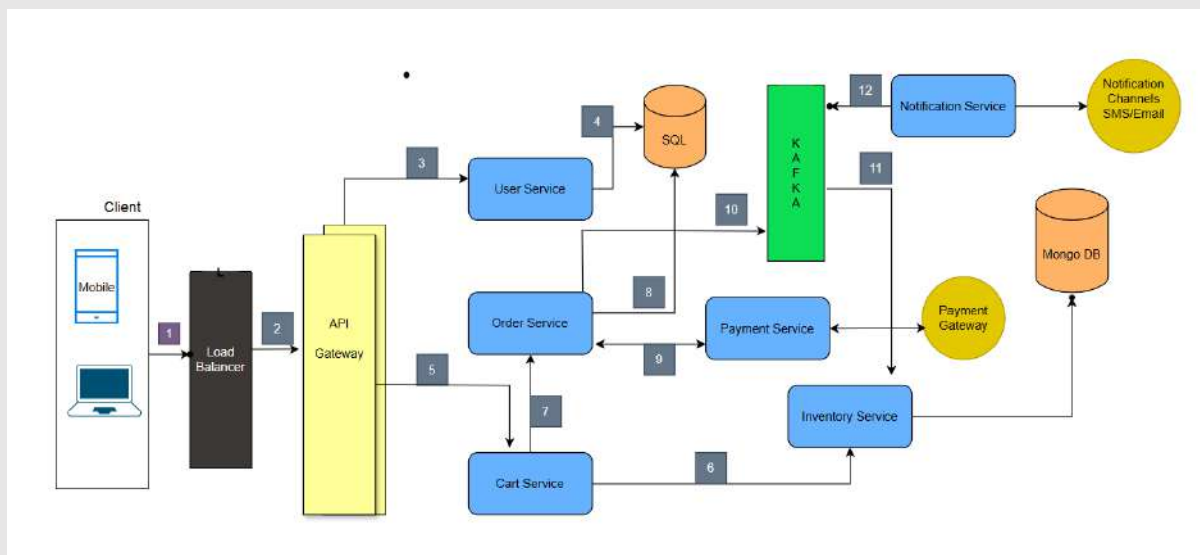
5. Design APIs

Here are the essential APIs in the purchase flow:

API	Method	Description
/cart/add	POST	Add an item to the cart

API	Method	Description
/cart/view	GET	View all items in cart
/checkout/create	POST	Create a new order from cart
/payment/initiate	POST	Start payment with gateway
/payment/verify	POST	Verify payment status
/order/status	GET	Get current status of an order

6. Architecture



Purchase Flow (Follow the Numbers in the Diagram)

1. Client to Load Balancer

Client (mobile or web) sends the “Place Order” request. Load Balancer spreads the requests across multiple servers to handle high traffic smoothly.

2. Load Balancer to API Gateway

API Gateway is the single-entry point. It checks authentication and routes the request to the right microservice.

3. API Gateway to User Service

During registration, User Service save user details like name, address, and payment preferences in the database.

4. User Service to SQL Database

SQL stores structured data such as user details, order history, and addresses.

5. API Gateway to Cart Service

On the purchase flow, when user clicks on pay now, Cart Service retrieves the items selected for purchase and calculates the total price.

6. Cart Service to Inventory Service

Cart service calls Inventory Service to check product availability. if product is available it will move to the next step to place the order.

7. Cart Service to Order Service

Cart service invokes Order service to go ahead with order.

8. Order Service to SQL

Order Service fetch user details like address, payment methods from SQL and trigger payment. After the payment Order service will save order details in SQL DB.

9. Order Service to Payment

Cart service invokes the payment gateway like Razorpay and Stripe, once payment is success it creates an event say "order_created".

10. Order Service to Kafka

Once payment is success Order Service creates an event say "order_created".

11. Kafka to Inventory Service.

Inventory Service can listen to kafka topic and reduce the stock

12. Kafka to Notification Service.

Notification Service can listen to kafka topic and send order confirmation mail or sms to the user

7. Recap / Flow

1. **User places an order** from the mobile app or website.
The request first goes to the **Load Balancer**, which distributes traffic across multiple servers.
2. **Load Balancer** forwards the request to the **API Gateway**, which checks authentication and routes it to the right service.
3. **API Gateway** sends a request to the **User Service** to save user details like address and contact information during registration.
4. **User Service** save this data in the **SQL database**.
5. During purchase flow, **API Gateway** now contacts the **Cart Service** to get all items the user added for checkout.
6. **Cart Service** communicates with the **Inventory Service** to verify if all selected products are still in stock.

7. Once items are confirmed, **Cart Service** sends the final cart data to the **Order Service**, which creates a new order entry.
8. **Order Service** calls the **Payment Service** to initiate the payment process.
9. **Payment Service** communicates with the **Payment Gateway** (like Razorpay or Stripe) to complete the transaction, then sends the payment status back to the **Order Service**.
10. After the order is created, the **Order Service** sends an event to **Kafka**, which acts as the message broker.
11. **Inventory Service** listens to **Kafka** and updates stock levels in real-time once payment is successful.
The updated inventory data is also stored in **MongoDB**.
12. Finally, **Notification Service** picks up the success message from **Kafka** and sends a confirmation message to the user through **Email or SMS**.

277. HLD 14.3: Design eCommerce application like Amazon (Recommendations Flow)

In this section we will see Recommendations flow, you can check HLD 11 for more detailed Recommendation system design..

1. Functional requirements:

- We need to show personalized product suggestions on the home page, product detail page, and cart page.
- The system should track user activity like searches, views, and purchases to make better recommendations.

2. Non-functional requirements:

- The system must respond quickly (within milliseconds), handle millions of users, and keep data safe.

3. Components

To make this work, these parts are needed:

- **Order Service:** When user ordered something, we will get the user and orderd information .
- **Search Service:** user searches the product.
- **Recommendation Service:** generates and stores recommended items.
- **Cache (like Redis):** stores frequently shown recommendations for fast access.
- **Message Queue (like Kafka):** moves user activity data to the recommendation engine without delay.

Example: when a user views a phone, when he ordered (search service and order service) sends that event to Kafka, which later helps the Recommendation Service learn what that user likes.

4. Database

We can use **NoSQL (Cassandra)** to store user interactions because they are large and come in different formats.

Cassandra is best for handling massive amounts of real-time data with high-speed writes and quick lookups. It's ideal for applications that need continuous availability, like event logging, IoT data collection, real-time counters, recommendation systems.

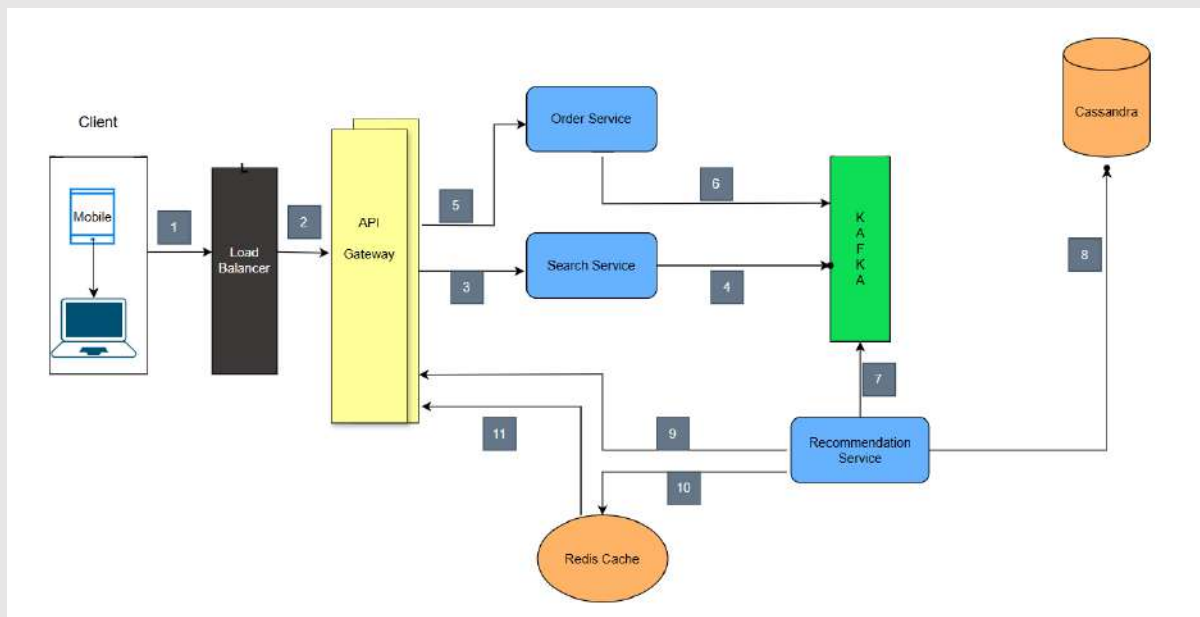
5. Design APIs

We design APIs that connect the front-end with the recommendation logic.

Example Endpoints:

- GET /recommendations/{userId} – returns list of products for that user.

6. Architecture



Recommendations Flow (Follow the Numbers in the Diagram)

1. Client to Load Balancer

The user opens the app on mobile or web. The request first goes to the Load Balancer. It distributes traffic equally to avoid overloading any single server.

2. Load Balancer to API Gateway

The request then reaches the API Gateway. It acts like a central entry point that decides which service should handle the request, search, order, or recommendation.

3. API Gateway to Search Service

When a user searches for a product like “running shoes”, the API Gateway routes that request to the Search Service. This service checks available items and sends results back.

4. Search Service to Kafka

Every search event is sent to Kafka. Kafka collects all user activity such as user information, searches, clicks, and purchases. This data is later used to train and improve recommendations.

5. API Gateway to Order Service

If the user buys something, the API Gateway passes the order details to the Order Service. This service records orders, updates payment status, and confirms purchases.

6. Order Service to Kafka

Order events are also sent to Kafka. They tell which items users actually bought. These events are important because they show strong buying intent.

7. Kafka to Recommendation Service

The Recommendation Service consumes events from Kafka. It studies patterns like “people who bought this also bought that” and updates its ML model to generate better suggestions.

8. Recommendation Service to Cassandra

All generated recommendations are saved in Cassandra. It stores large-scale data and allows quick reads when a user opens the app again.

9. Recommendation Service to API Gateway

When the user logs in or visits a product page, the Recommendation Service sends personalized product lists to the API Gateway, which delivers them to the client instantly.

10. Recommendation Service to Cache

Recommendation service stores the result in cache before sending to client

11. Client to Cache

If results are found, data is returned directly from Redis (fast retrieval).

If not found, API Gateway fetches from Recommendation Service and updates Redis.

7. Recap / Flow

- User searches or buys a product.
- Request goes to Load Balancer -> API Gateway.
- API Gateway routes to Search Service or Order Service.
- Search and Order events are sent to Kafka.
- Recommendation Service consumes events from Kafka and prepares personalized suggestions.
- Recommendations are stored in Cassandra for quick access.
- When the user opens the app again, API Gateway fetches recommendations from the Recommendation Service.
- Results are returned to the client, using cached data if already available
- If results are found, data is returned directly from Redis (fast retrieval).
- If not found, API Gateway fetches from Recommendation Service and updates Redis.

278. HLD 14.4: Design eCommerce application like Amazon (Reviews Flow)

In this section we will see Reviews flow.

1. Functional requirements:

- A customer can add, edit, and delete reviews.
- Reviews are linked to specific products.
- Each review includes a rating, comment, and timestamp.
- User can upload image/video
- Users can see all reviews for a product.

2. Non-functional requirements:

- Reviews should load fast even for popular products.
- The system should handle fake reviews safely and ensure data integrity.

3. Identify Components

- **API Gateway** to receive review-related requests.
- **Review Service** to manage review data.
- **User Service** to verify who is posting.
- **Product Service** to connect reviews to the right products.
- **Cache (Redis)** to store frequently viewed reviews.
- **Database** to keep all reviews safely stored.
- **Notification Service** if we want to notify sellers when a new review is added.

4. Database

Reviews are text-heavy and can grow fast. A NoSQL database like **MongoDB** fits better as it can handle unstructured data and scale easily.

```
{
  "reviewId": "R001",
  "productId": "P123",
  "userId": "U567",
  "rating": 4,
  "comment": "Good quality, fast delivery",
  "createdAt": "2025-10-25T10:00:00Z"
}
```

5. Design APIs

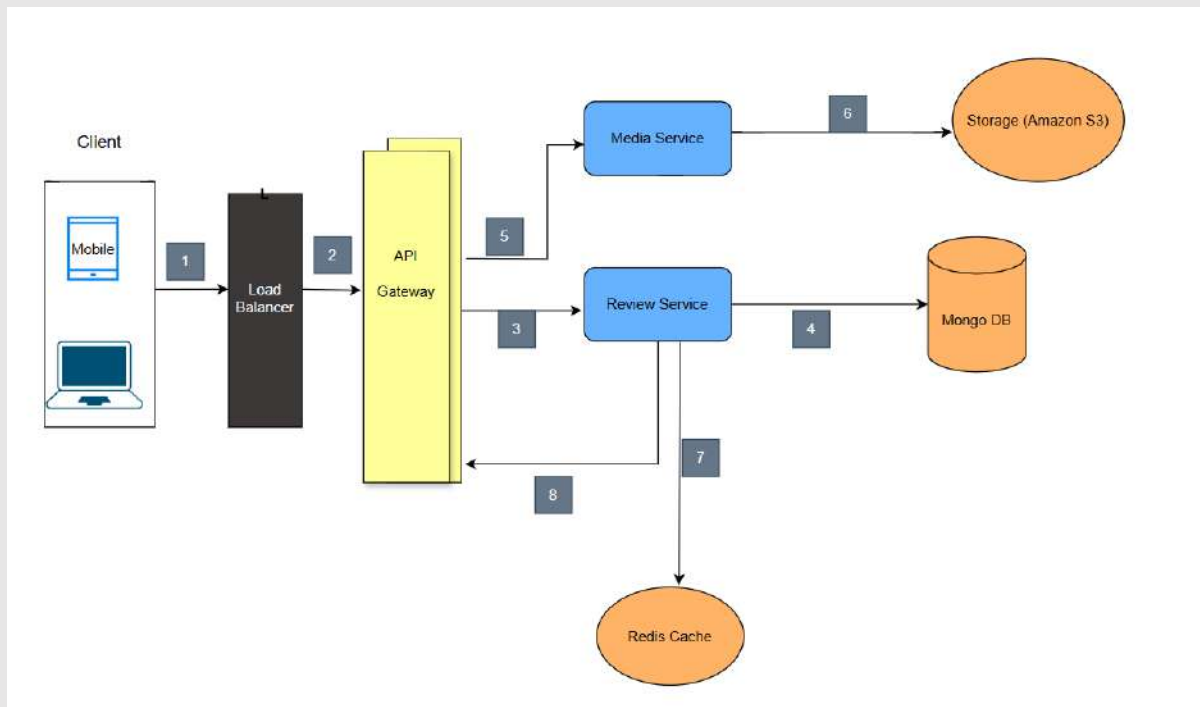
We define how services communicate.

Example endpoints:

- **POST /reviews**: Add a new review.
Request body: userId, productId, rating, Comments
- **GET /products/{id}/reviews**: Get all reviews for a product
- **POST /media**: To upload videos and images
- **PUT /reviews/{id}**: Update a review
- **DELETE /reviews/{id}**: Delete a review

Each API should handle authentication, validation, and errors properly.

6. Architecture



Reviews Flow (Follow the Numbers in the Diagram)

1. Client to Load Balancer

The user sends a request from mobile or web to view or post a review. The load balancer receives it and distributes traffic evenly among available servers, preventing overload and improving performance.

2. Load Balancer to API Gateway

Once the request is balanced, it moves to the API Gateway. The gateway routes it to the correct service based on the API endpoint, such as Review Service for text or Media Service for uploads.

3. API Gateway to Review Service

The Review Service receives the review details. It validates product ID, user ID, and content. For example, when adding a review with a 5-star rating, it checks if the product exists before saving.

4. Review Service to MongoDB

After validation, the Review Service stores the review data in MongoDB. The database keeps fields like rating, comment, productId, userId, and timestamps. It also stores links to any uploaded media files.

5. API Gateway to Media Service

If a review includes images or videos, the API Gateway forwards those to the Media Service. The service manages uploads, creates thumbnails, scans for inappropriate content, and stores metadata for each file.

6. Media Service to Storage

Processed media files are pushed to a S3. Then the CDN picks from Storage and ensures quick access for users worldwide. For example, when a user views a product review image, it loads from the nearest CDN server.

7. Review Service to Redis Cache

Frequently accessed reviews are saved in Redis Cache. When users view popular products repeatedly, data comes from Redis instead of MongoDB, reducing database load and speeding up responses.

8. Redis Cache to API Gateway to Client

When cached reviews exist, the Review Service retrieves them from Redis and returns them through the API Gateway. The user instantly sees reviews and ratings without waiting for a database query.

7. Recap / Flow

- User adds or views a product review through mobile or web.
- Request first goes to Load Balancer, then to API Gateway.
- API Gateway routes the request to Review Service or Media Service.
- Review Service validates product, user, and review data, then saves it to MongoDB.
- If media is attached, Media Service handles upload, scanning, and sends files to cloud storage like S3.
- Review Service updates Redis Cache for faster future access.
- When another user views the same product, Review Service fetches reviews from Redis if available.
- API Gateway returns final reviews and media URLs to the client for quick display.

Reference: [Amazon System Design](#) | [Flipkart System Design](#) | [System Design Interview Question](#)

279. HLD 15.1: Design OTT platform like Netflix / Prime / Hotstar (Video Upload)

We will divide this into 5 parts.

1. Video upload flow
2. Video Streaming Flow
3. Searching Flow (Refer Question No. 266 HLD 5)
4. Recommendations Flow (Refer Question No. 272 HLD 11)
5. Billing & Subscription Flow
6. Analytics Flow

In this section we will see Video upload flow.

Note: We assume the user is already logged in. So, every API request will include the user_id, and all database operations will store this user_id along with other details.

1. Functional Requirements

- Allow users to upload video files from web or mobile.
- Store the uploaded file securely in the backend.
- Convert videos into multiple formats and resolutions (for example: 240p, 480p, 720p).
- Save video details like title, description, uploader ID, and tags.

2. Non-Functional Requirements

- Support multiple users uploading at the same time.
- Make the system fault-tolerant, so even if one service fails, others continue.

- Ensure fast uploads and secure file transfer.

3. Components

Component	Purpose	Example
Load Balancer	Distributes incoming requests among servers	AWS ELB
API Gateway	Entry point that routes requests to upload or metadata services	Nginx / Spring Gateway
Upload Service	Accepts video file and metadata, validates file type and size	Custom microservice
Object Storage	Stores raw and processed video files	Amazon S3, Google Cloud Storage
Queue System	Sends message to start transcoding once upload is complete	Kafka / SQS
Video processor Service	Filter the content, converts raw video into multiple formats, multiple resolution,	FFmpeg / Elastic Transcoder
Metadata Database	Stores video info like title, status, duration, uploader ID	MySQL
Notification Service	Updates uploader when video is ready to watch	Email or push service

4. Database

For metadata, a **SQL database** like MySQL works well because we need structured relations.

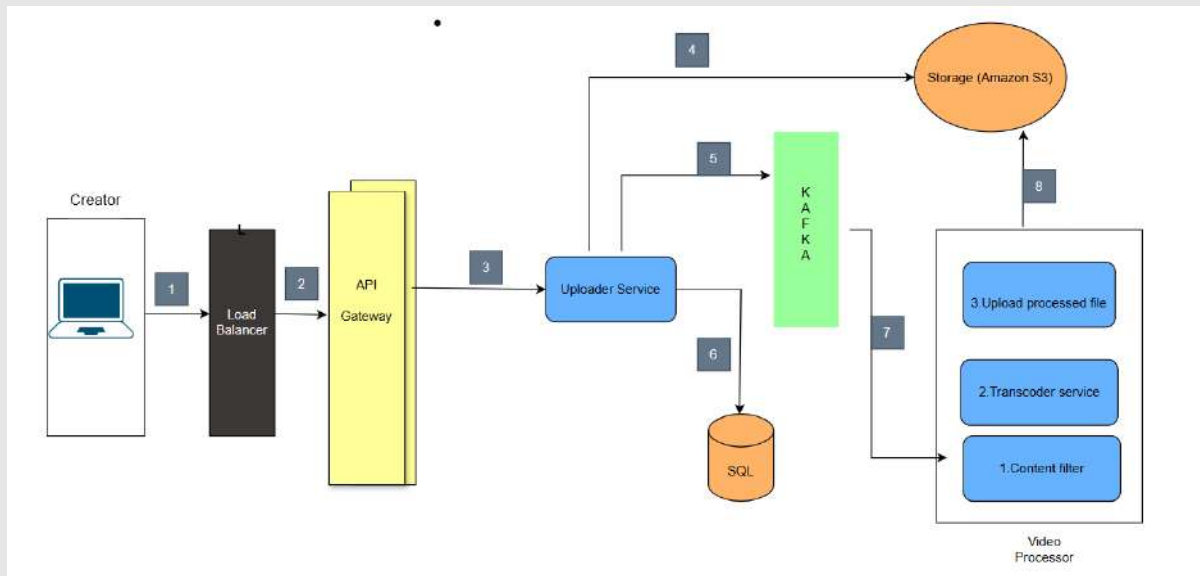
To store users and their uploaded videos, an **SQL database** is a good choice since we need to maintain clear relationships between them.

5. Design APIs

API	Method	Description	Request	Response
/upload	POST	Upload video file with metadata	Form-data: video file, title, description	Upload ID, status
/upload/status/{id}	GET	Check status of uploaded video	Path param: id	Status: Pending / Processing / Ready
/videos	GET	Fetch videos uploaded by a user	Query: uploader_id	List of video metadata

Example: When we send a POST request to /upload, the service saves metadata, uploads the file to storage, and pushes a message to the queue for transcoding.

6. Architecture



Upload Flow (Follow the Numbers in the Diagram)

1. Creator to Load Balancer

The creator uploads a video from a laptop or mobile app. The request first reaches the Load Balancer, which distributes incoming traffic across multiple servers to handle large numbers of uploads smoothly without overloading one server.

2. Load Balancer to API Gateway

The Load Balancer passes the request to the API Gateway. The gateway acts as the single entry point for all uploads. It checks authentication, file type, and routes the request to the correct backend service.

3. API Gateway to Uploader Service

The Gateway sends the validated request to the Uploader Service. This service handles the actual upload process, verifies metadata like title and file size, and starts storing the raw video in storage.

4. Uploader Service to Storage (Amazon S3 - Raw Storage)

The Uploader Service saves the unprocessed original video to S3. This is the raw version. We keep it safe so that if transcoding fails or new formats are needed later, we can reuse it.

Note: This service gets the files in chunks, usually large files are processed in chunks

Example: A creator uploads a 4GB .mov file. It's stored in S3 as raw/Workout.mov.

5. Uploader Service to Kafka

Once the upload finishes, the Uploader Service sends a message to Kafka. Kafka acts as a message queue to trigger further background jobs like filtering and transcoding. This keeps the system fast and non-blocking.

6. Uploader Service to SQL Database

The service records video details (metadata) in the SQL database. This includes video ID, uploader name, file path, upload status, and upload time. This helps in tracking every video's progress from upload to ready state.

7. Kafka to Video Processor

The Video Processor listens to Kafka for new upload events. When it gets one, it picks the video from S3, runs content filtering to check for violations, and then sends it for transcoding.

Inside Video Processor

- The Content Filter checks for copyrighted content or banned content.
- The Transcoder Service converts the raw video into multiple formats and resolutions such as 480p, 720p, and 1080p.
- Finally, the Upload Processed File module stores these processed outputs back into S3 for streaming.

8. Video Processor to Storage (Amazon S3 - Processed Storage)

The transcoded videos are stored again in S3 but this time in a different folder or bucket, marked as processed. These are the ready-to-stream versions. Each resolution is saved separately for adaptive playback.

Example:

processed/Workout_480p.m3u8

processed/Workout_720p.m3u8

processed/Workout_1080p.m3u8

7. Recap / Flow

- Creator uploads a video through web or mobile.
- Request first goes to Load Balancer, then to API Gateway.
- API Gateway routes the request to the Uploader Service.
- Uploader Service validates metadata and uploads the raw video to Amazon S3 (Raw Storage).
- Uploader Service sends a Kafka message to trigger background processing and saves video details in SQL.
- Video Processor consumes the Kafka event, runs content filtering, and transcodes the video into multiple resolutions.
- Processed videos are saved again in Amazon S3 (Processed Storage), separate from raw uploads.

Note: Here can add notification service, which listens to the Kafka topic and send notifications to users

280. HLD 15.2: Design OTT platform like Netflix / Prime / Hotstar (Video streaming)

We will divide this into 5 parts.

1. Video upload flow
2. Video Streaming Flow
3. Searching Flow

4. Recommendation Flow
5. Billing & Subscription Flow
6. Analytics Flow

In this section we will see Video streaming flow.

Note: We assume the user is already logged in. So, every API request will include the user_id, and all database operations will store this user_id along with other details.

After a video is uploaded and processed, the next step is streaming it to millions of users smoothly.

1. Functional Requirements

- Allow users to play videos from web or mobile devices.
- Fetch video details and stream links quickly.
- Adjust video quality based on internet speed.
- Support play, pause, and resume options.

2. Non-Functional Requirements

- Handle thousands of concurrent viewers efficiently.
- Low latency and smooth playback.
- Use caching and CDN for fast delivery.

3. Components

Component	Purpose	Example
Load Balancer	Distributes playback requests evenly to avoid overload	AWS ELB
API Gateway	Entry point for all streaming and metadata requests	Kong / Spring Cloud Gateway
Streaming Service	Fetches video info and generates streaming URLs	Custom microservice
Authentication Service	Verifies user login or subscription before allowing playback	JWT / OAuth
CDN (Content Delivery Network)	Delivers video files from nearest location for faster playback	CloudFront / Akamai
Metadata Database	Stores video details like title, duration, description	MySQL
Redis Cache	Keeps popular video metadata and playback info for quick access	Redis

4. Database

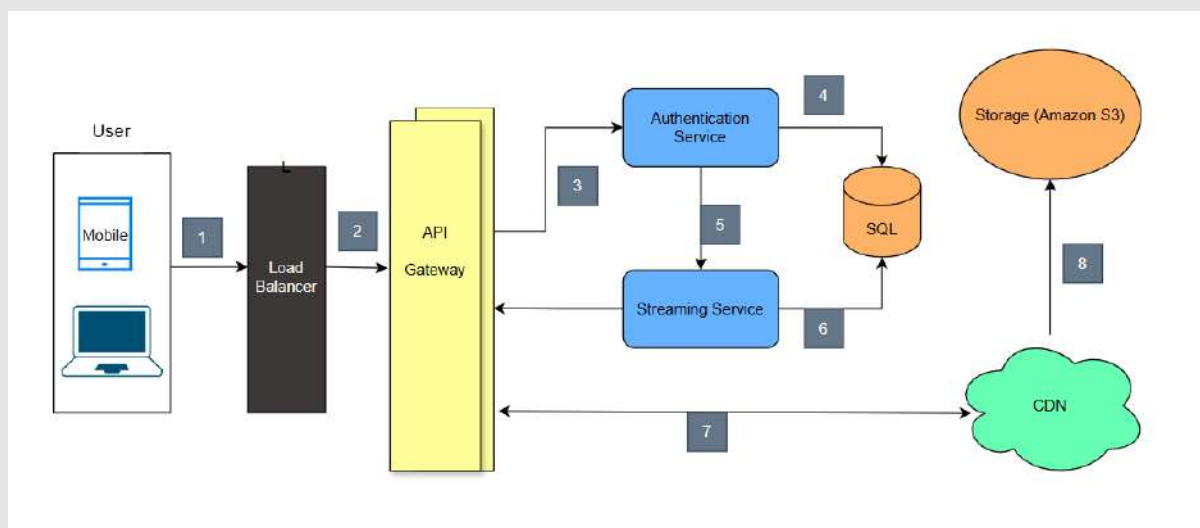
For metadata, a **SQL database** like MySQL works well because we need structured relations.

5. Design APIs

API	Method	Description
/videos/{id}	GET	Fetch video metadata
/videos/stream/{id}	GET	Generate playback manifest file (.m3u8 or .mpd)
/videos/view/{id}	POST	Increment view count when a video is watched

Example: When we call /videos/stream/{id}, the API Gateway checks user authentication, fetches metadata, and returns a manifest file with all available resolutions from the CDN.

6. Architecture



Streaming Flow (Follow the Numbers in the Diagram)

1. User to Load Balancer

A viewer opens the app or website and clicks on a video to play. The request first goes to the Load Balancer.

2. Load Balancer to API Gateway

The request is passed to the API Gateway. It is the main entry point to the backend. The Gateway decides where the request should go next, for authentication or for streaming.

3. API Gateway to Authentication Service

Before the video can play, the API Gateway sends the request to the Authentication Service. It checks if whether the logged user has subscription or not to watch this video.

4. Authentication Service to SQL Database

The Authentication Service checks user details like email, password, subscription, and expiry date from the SQL database. Once verified, it sends an approval back to the Gateway.

5. Authentication Service to Streaming Service

After successful verification, the request is passed to the Streaming Service. The Streaming Service gets video details and prepares playback information to Client.

6. Streaming Service to SQL Database

The Streaming Service looks up the video metadata stored in SQL. It finds the video title, description, duration, and the CDN path where the processed video files are stored.

7. Streaming Service Response to Client

The Streaming Service creates a response and sends it back to the client through the API Gateway. The response looks something like this:

```
{
  "video_id": "v12345",
  "title": "Morning Yoga Routine",
  "cdn_manifest_url": "https://cdn.streamly.com/videos/v12345/master.m3u8"
}
```

The **cdn_manifest_url** points to a manifest file, which contains the URLs for every resolution like 240p, 480p, 720p, and 1080p. The client's video player uses this manifest to start streaming the video.

When the player starts playback, it calls the CDN URL. The CDN checks if the requested video chunks are already cached on its edge servers.

If they exist, it serves them immediately. If not, it fetches them from the origin storage (S3) and keeps them in cache for later viewers.

8. Storage (Amazon S3) to CDN

S3 acts as the origin for the CDN. When a cache miss happens, the CDN automatically requests the missing video chunk from S3. For example:

S3 origin: <https://s3.amazonaws.com/streamly/videos/v12345/720p/chunk1.ts>

CDN path: <https://cdn.streamly.com/videos/v12345/720p/chunk1.ts>

Once the CDN gets the file, it stores it at the nearest edge server. The next viewer watching the same video in that region gets it directly from the CDN, without touching S3 again.

7. Recap / Flow

- Viewer clicks on a video from web or mobile.
- Request first goes to Load Balancer, then to API Gateway.
- API Gateway routes the request to the Authentication Service to verify the user's token or subscription.
- Authentication Service checks user details in SQL Database and sends back approval.
- API Gateway forwards the request to the Streaming Service once the user is verified.
- Streaming Service fetches video metadata and the CDN manifest URL from the SQL Database. The response contains a manifest link like:
<https://cdn.streamly.com/videos/v12345/master.m3u8>.
- The client's video player uses this link to request video chunks directly from the CDN.

- If the CDN already has the video cached, it streams immediately; if not, it fetches the chunks from Amazon S3 (Origin Storage) and caches them for future playback.

281. **HLD 15.3: Design OTT platform like Netflix / Prime / Hotstar (Billing and Subscription Flow)**

In this section we will see Billing and Subscription flow.

Note: We assume the user is already logged in. So, every API request will include the user_id, and all database operations will store this user_id along with other details.

1. Functional Requirements

- Allow users to choose and subscribe to different plans (Basic, Premium, Family).
- Process online payments using payment gateways like Stripe or Razorpay.
- Track active, expired, and failed subscriptions.
- Renew subscriptions automatically before expiry.
- Restrict content access based on active plan.

2. Non-Functional Requirements

- Must be secure and PCI compliant for handling payments.
- Should support high traffic during promotions or new releases.
- Should handle failures gracefully if payment or renewal fails.

3. Components

Component	Purpose	Example
Load Balancer	Distributes incoming requests across backend services.	AWS ELB
API Gateway	Routes user requests to Subscription or Billing Services.	Nginx / Spring Cloud Gateway
User Service	Checks whether user has subscription or not	Custom microservice
Subscription Service	Manages user plans, renewal dates, and access control.	Custom microservice
Billing Service	Handles payment initiation, transaction status, and receipts.	Custom microservice
Payment Gateway	External service for processing payments.	Stripe / Razorpay / PayPal
SQL Database	Stores plan details, billing records, and payment history.	MySQL / PostgreSQL
Notification Service	Sends confirmation emails and renewal reminders.	SES / Firebase Messaging

4. Database

For payments and subscriptions, a **SQL database** fits best because transactions need consistency and clear relationships.

Tables:

User - user_id, name, email.

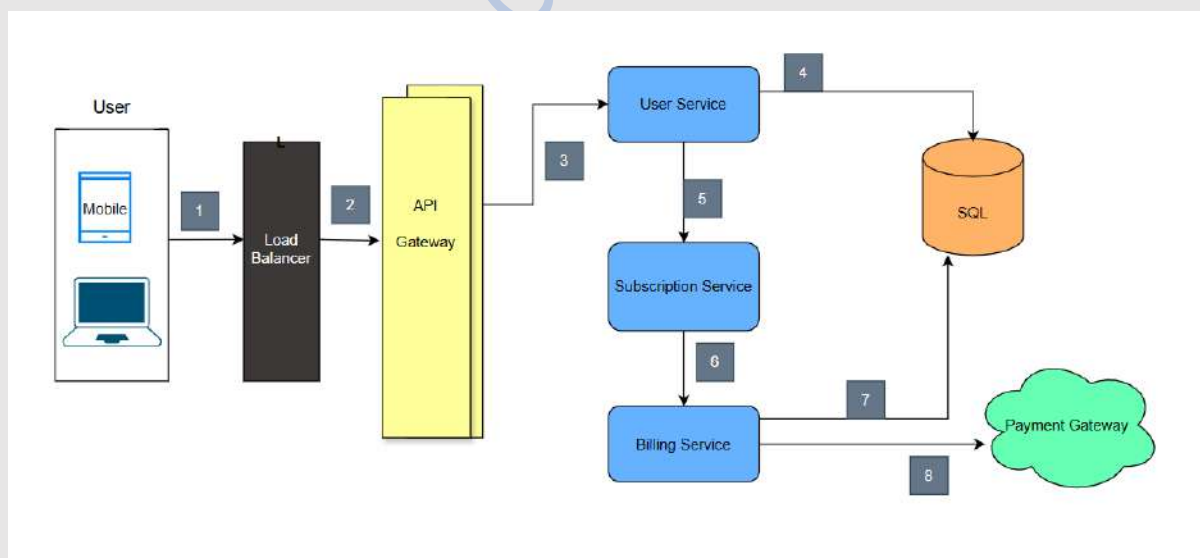
user_subscription - user_id, plan_id, start_date, end_date, status

user_transactions - transaction_id, user_id, payment_method, payment_amount, payment_status.

5. Design APIs

API	Method	Description
/plans	GET	Fetch available subscription plans
/subscribe	POST	Start a new subscription and payment
/payment/callback	POST	Receive payment confirmation from gateway
/subscription/status/{user_id}	GET	Get user's subscription status
/subscription/renew	POST	Renew user's plan automatically

6. Architecture



Billing and Subscription Flow (Follow the Numbers in the Diagram)

1. User to Load Balancer

The user opens the app or website and selects a plan. The request first goes to the Load Balancer. It

manages all incoming requests and distributes them to available servers so the system stays fast and stable even during heavy traffic.

2. Load Balancer to API Gateway

After the request is balanced, it reaches the API Gateway. The Gateway acts as the entry point to the backend. It decides where to send the request, It can also handle authentication.

3. API Gateway to User Service

The API Gateway passes the request to the User Service. The User Service checks if the user has subscription or not

4. User Service to SQL Database

The User Service connects to the SQL Database to fetch all the user related information.

5. User Service to Subscription Service

Once the user is verified, the request is sent to the Subscription Service. This service handles the subscription logic such as plan selection, start date, and renewal duration. It prepares a new subscription record for that user.

6. Subscription Service to Billing Service

Once user selects the Subscription, Service calls the Billing Service. The Billing Service is responsible for creating a new billing record and initiating the payment with the external gateway.

7. Billing Service to Payment Gateway

The Billing Service connects with the Payment Gateway such as Stripe or Razorpay. It passes the amount, plan name, and user details. The user completes payment on the secure gateway page.

Once the payment is done, the Payment Gateway sends back a confirmation response to the Billing Service.

8. If payment is successful, the Billing Service updates the subscription status in the SQL Database to "Active". If it fails, it updates it as "Failed" and asks the user to retry.

7. Recap / Flow

- User selects a subscription plan on web or mobile.
- Request first goes to Load Balancer, then to API Gateway.
- API Gateway routes the request to the User Service to validate the user account.
- User Service checks user details in the SQL Database and confirms validity.
- Request then moves to the Subscription Service, which creates a new subscription record with status "Pending Payment."
- Subscription Service calls the Billing Service to start the payment process.
- Billing Service connects with the Payment Gateway (like Stripe or Razorpay) to complete the payment.
- Once payment is successful, the Payment Gateway notifies the Billing Service, which updates SQL with status "Active."
- The user's plan becomes active, granting access to premium content until the next renewal.

282. HLD 15.4: Design OTT platform like Netflix / Prime / Hotstar (Analytics Flow)

In this section we will see Analytics flow.

After a video is uploaded and processed, the next step is streaming it to millions of users smoothly.

1. Functional Requirements

- Capture user activity like video views, likes, duration watched, and device type.
- Collect system-level data such as request counts, errors, and latency.
- Store all analytics data in a scalable storage system.
- Generate reports and dashboards for insights like top videos and peak hours.

2. Non-Functional Requirements

- Handle large volumes of data in real time.
- Ensure data accuracy and consistency.
- Must be scalable to support millions of events daily.

3. Components

Component	Purpose	Example
Client SDK	Captures playback or user activity data and sends it to backend	Custom in-app analytics SDK
Load Balancer	Distributes incoming analytics events evenly	AWS ELB
API Gateway	Entry point for event data requests	Nginx / Kong
Video Streaming Service	Receives user click on play button	Custom microservice
Message Queue (Kafka)	Buffers and streams events for processing	Apache Kafka
Analytics Processor	Aggregates and transforms events for reporting	Apache Spark
Data Warehouse / Storage	Stores processed data for queries	Amazon Redshift / BigQuery / Snowflake
Dashboard / Visualization Tool	Displays trends and reports	Grafana / Looker / Tableau

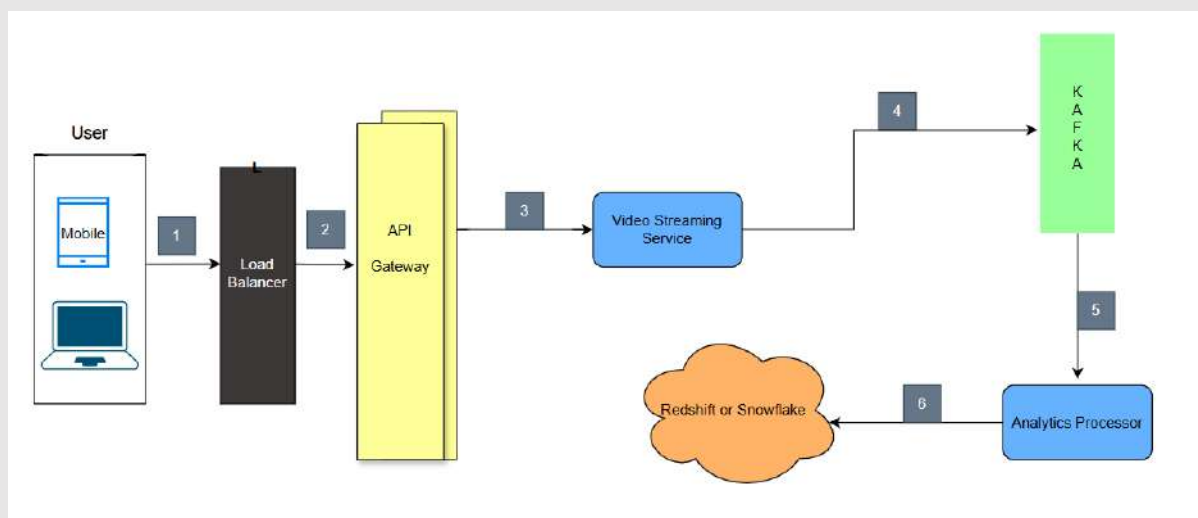
4. Database

For analytics, we usually deal with large, unstructured event data. Any Data warehouse like Big query, Amazon redshift or snowflake suits here

5. Design APIs

API	Method	Description
/analytics/event	POST	Collects a single playback or user action event
/analytics/summary	GET	Fetches summary stats like total views or watch time
/analytics/topvideos	GET	Returns most watched videos

6.Architecture



(Follow the numbers in the diagram)

1. User to Load Balancer

A user plays or interacts with a video on the web or mobile app. The request first goes to the Load Balancer, which distributes all incoming requests evenly across backend servers to maintain stability and handle large traffic smoothly.

2. Load Balancer to API Gateway

After balancing, the request moves to the API Gateway. The Gateway acts as a single-entry point to all backend services. It routes requests to the right service and ensures only authorized calls go through.

3. API Gateway to Video Streaming Service

The API Gateway forwards the user's request to the Video Streaming Service. This service handles video playback logic like fetching the manifest file and returning the video stream URL from the CDN. While serving the video, it also tracks user actions like play, pause, and complete.

4. Video Streaming Service to Kafka

The Video Streaming Service sends analytics data (for example, watch duration, user ID, and video ID) as small event messages to a Kafka topic. Kafka acts as a message broker and temporarily stores all events. This keeps the Streaming Service lightweight and prevents it from slowing down due to analytics processing.

Example message:

```
{
  "user_id": "u101",
  "video_id": "v202",
  "event_type": "complete",
  "duration": 1200
}
```

5. Kafka to Analytics Processor

Kafka delivers these events in real time to the Analytics Processor. This processor consumes the messages.

6. Analytics Processor to Redshift or Snowflake

After processing, the analytics data is saved into a Data Warehouse like Redshift or Snowflake. These platforms are built for analytical queries, making it easy to run reports, build dashboards, and analyze trends like “Top 10 videos by watch time” or “Peak streaming hours.”

Dashboards like Grafana or Looker connect to the data warehouse to show live reports — for example, “Top 10 videos today” or “Peak view times by region.”

7. Recap / Flow

- Viewer plays or interacts with a video from web or mobile.
- Request first goes to Load Balancer, then to API Gateway.
- API Gateway forwards the request to the Video Streaming Service to handle playback and serve the video manifest.
- While streaming, the Video Streaming Service sends analytics events (like play, pause, or complete) to Kafka.
- Kafka stores these events temporarily and delivers them to the Analytics Processor.
- Analytics Processor aggregates the data, such as total watch time or number of views per video.
- Processed results are stored in a data warehouse like Redshift or Snowflake for analysis.
- Dashboards and reports use this data to show insights like trending videos or user engagement patterns.

References:

[Design Netflix: A Comprehensive Guide](#)

[System Design: Netflix | by Karan Pratap Singh | Medium](#)

Video links:

Mock Interview (Good to watch): [System Design Interview: Design Amazon Prime Video - YouTube](#)

More detailed info: [Netflix System Design | YouTube System Design | System Design Interview Question](#)

To know Netflix real architecture, check below two links:

[Netflix System Design- How Netflix Onboards New Content - DEV Community](#)

[Netflix System Design- Backend Architecture - DEV Community](#)

283. HLD 16: Design YouTube (Comments)

For YouTube, we can include all the design flows discussed for Netflix from 15.1 to 15.4. In addition, we need to add one more flow for **comments, likes, and dislikes**

Note: We assume the user is already logged in. So, every API request will include the `user_id`, and all database operations will store this `user_id` along with other details.

1. Functional Requirements

- Allow users to add, edit, and delete comments on a video.
- Let users like or dislike a video or comment.
- Display the total number of likes, dislikes, and comments for each video.
- Show replies and maintain comment threads.

2. Non-Functional Requirements

- Handle high traffic as popular videos get millions of comments.
- Ensure low latency while fetching comments.

3. Components

Component	Purpose	Example
Load Balancer	Distributes all user interaction requests evenly across servers	AWS ELB
API Gateway	Routes requests to the correct microservice	Nginx / Spring Cloud Gateway
Comment Service	Manages adding, editing, deleting, and fetching comments	Custom microservice
Reaction Service	Handles likes and dislikes on videos and comments	Custom microservice
Notification Service	Sends alerts when someone replies to a comment	Firebase / SNS
SQL or NoSQL Database	Stores comments, likes, and replies efficiently	MongoDB / Cassandra / MySQL
Cache (Redis)	Keeps recent comments and counts for faster retrieval	Redis

4.Database

Comments and reactions are high in volume, so a NoSQL database like MongoDB or Cassandra fits best because it scales horizontally and handles nested structures like replies easily.

Comments Collection / Table

Field	Type	Description
comment_id	VARCHAR	Unique ID for each comment
video_id	VARCHAR	ID of the video
user_id	VARCHAR	ID of the user who posted
text	TEXT	Comment text
parent_id	VARCHAR	For replies, points to parent comment
likes	INT	Count of likes
dislikes	INT	Count of dislikes
created_at	DATETIME	Time comment was created

Reactions collection / table:

Field	Type	Description
reaction_id	VARCHAR	Unique ID for each reaction
user_id	VARCHAR	ID of the user who reacted
video_id	VARCHAR	Video on which reaction is made
reaction_type	ENUM	Like or Dislike
created_at	DATETIME	Timestamp of reaction

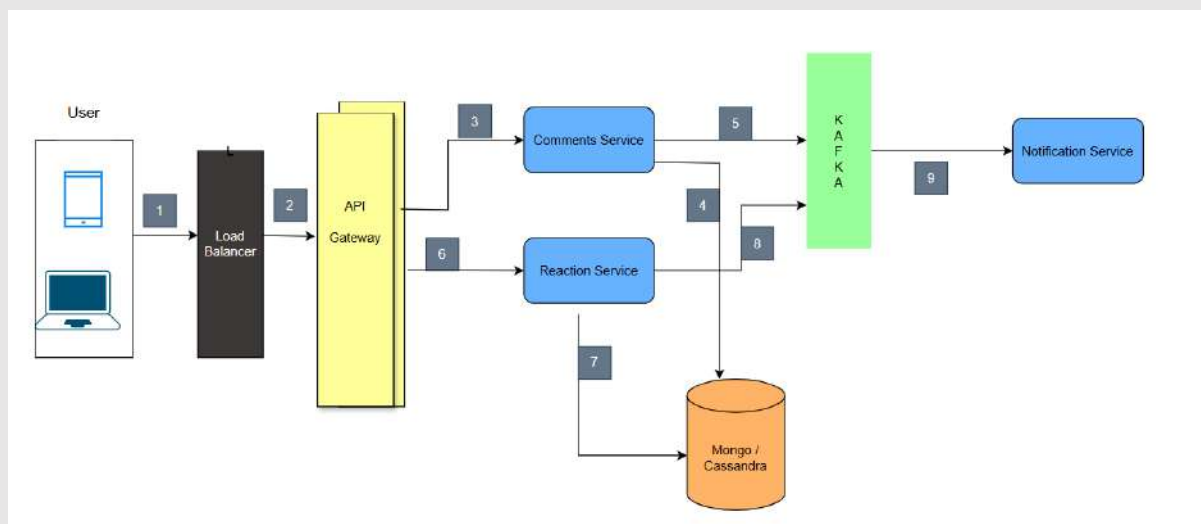
5. Design APIs (Note: Every API must include user_id)

API	Method	Description	Request	Response
/comments/add	POST	Add a comment to a video	JSON: video_id, user_id, text	Comment ID, timestamp
/comments/{video_id}	GET	Get all comments for a video	Path param: video_id	List of comments and replies
/comments/{id}/like	POST	Like a comment	Path param: comment_id	Updated like count

API	Method Description		Request	Response
/comments/{id}/dislike	POST	Dislike a comment	Path param: comment_id	Updated dislike count
/videos/{id}/like	POST	Like a video	Path param: video_id	Updated like count
/videos/{id}/dislike	POST	Dislike a video	Path param: video_id	Updated dislike count

Example:

When a user comments on a video “Morning Motivation,” the Comment Service saves the comment in MongoDB, adds an entry in Redis cache, and returns the updated comment list with the new one added on top.

6. Architecture**Follow the Numbers in the diagram****1. Creator to Load Balancer**

A creator or viewer performs an action like posting a comment or liking a video. The request first reaches the Load Balancer, which spreads traffic evenly across multiple backend servers to handle large user volumes smoothly.

2. Load Balancer to API Gateway

The API Gateway receives the request and decides which service should handle it. It also validates basic tokens or API keys to make sure the request is authorized.

3. API Gateway to Comments Service

If the action is a new comment or reply, the request is routed to the Comments Service. This service takes care of creating, editing, or deleting comments and manages reply threads.

4. Comments Service to Mongo DB

Comments Service save the data to MongoDB or Cassandra. These databases are chosen because they handle large-scale read and write operations efficiently, which is important since millions of comments and likes can be generated daily.

5. Comments Service to Kafka

After a comment is added successfully, an event is sent to Kafka. Kafka acts as an event stream system that stores messages like “new comment posted” or “reply added” for other services to consume later.

6. API Gateway to Reaction Service

If the user performs a like or dislike action, the API Gateway routes that request directly to the Reaction Service. The Reaction Service processes the action and updates counts accordingly.

7. Reaction Service to Mongo or Cassandra

Both Comments and Reaction Services store or update their data in MongoDB or Cassandra.

8. Reaction Service to Kafka

After a comment is added successfully, an event is sent to Kafka. Kafka acts as an event stream system that stores messages like “Someone has liked your post”

9. Kafka to Notification Service

Kafka sends the stored “new comment” or “reply” event to the Notification Service. The Notification Service uses this data to send real-time alerts like “Someone replied to your comment” or “Your video got a new comment.”

7. Recap / Flow

- User posts a comment, reply, or reacts to a video.
- Request first goes to Load Balancer, then to API Gateway.
- API Gateway routes the request to either the Comments Service or Reaction Service based on the action type.
- Comments Service saves new comments or replies in MongoDB or Cassandra and sends an event to Kafka.
- Reaction Service updates likes or dislikes in the database for videos or comments.
- Kafka stores these comment and reaction events and forwards them to the Notification Service.
- Notification Service sends alerts like “Someone replied to your comment” or “Your video got a new like.”
- All actions are handled asynchronously, keeping the user experience smooth and the system scalable.

284. HLD 17.1: Design Instagram (User Signup and Connection Flow)

User Signup and Connection Flow explain how a new user joins the platform, creates their profile, and connects with others. It covers account creation, verification, and building relationships through follows.

During User onboarding, user details are stored in a relational database, passwords are hashed, and verification happens through email or phone. Once registered, a unique user ID is created, and the follow system links users using follower_id and followee_id in a graph-like structure stored in a database. Let's see the design

1. Functional Requirements

- Allow new users to register with basic details like name, email, and password.
- Verify user details through email or phone.

- Let users follow or unfollow others.
- Show how many followers and followees each user has.

2. Non-Functional Requirements

- The system should handle millions of users.
- API responses must be fast (under 200ms for basic actions).
- Passwords must be stored securely (hashed).
- Data should stay consistent across users and relationships.

3. Components

Component	Purpose
Load Balancer	Distributes incoming traffic to multiple servers.
API Gateway	Handles all incoming requests and routes them to the correct service.
User Service	Manages signup, login, and profile details.
Follow Service	Handles follow and unfollow operations.
Database	Stores user info and relationships.
Cache (Redis)	Keeps quick counts of followers and followees.
Message Queue (Kafka)	Sends notifications like "X started following you."

4. Database

We use SQL Database (like PostgreSQL) for structured data and clear relationships. It's easier to manage joins and maintain data consistency.

User:

user_id	username	email	password_hash	created_at	Followers_Count
1	arjun_dev	arjun@gmail.com	9sd89ds89ds	2024-01-03	1
2	rhea_fit	rhea@gmail.com	8ab77cd91aa	2023-12-01	1
3	Virat	vk@gmail.com	75b77sfd91aa	2025-11-03	
4	Mahi	msd@gmail.com	7aaf845acs	2024-11-01	1

Follower:

follower_id	followee_id	created_at
1	2	2025-11-03
2	1	2025-11-03
3	4	2025-11-03

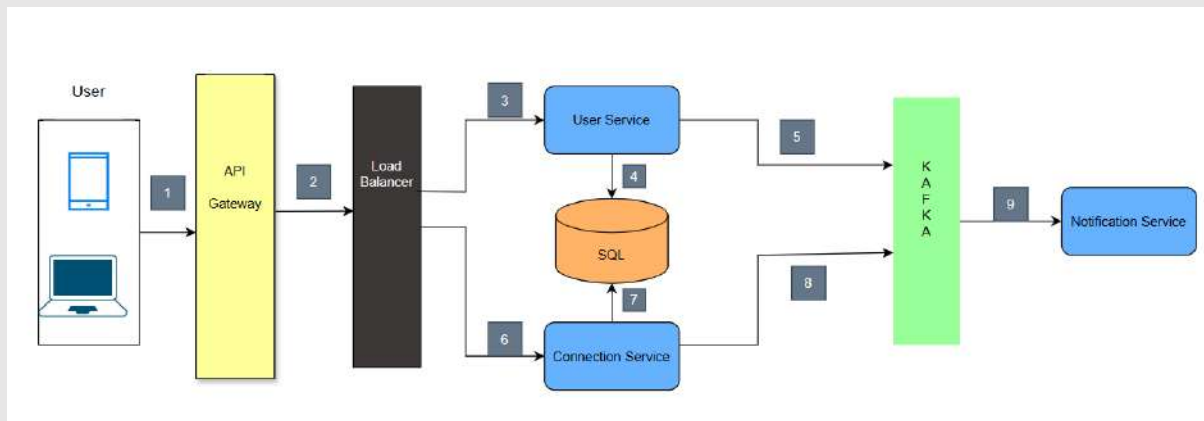
Tables explanation:

- Arjun follows Rhea.
- Rhea follows Arjun back.
- Virat follows Mahi.
- In the Follower Table, Mahi's user_id = 4 never appears in the follower_id column.
- That means Mahi is not following anyone.
- But Mahi's ID appears in the followee_id column, which means someone (Virat) is following Mahi.
- So, Mahi's Following Count = 0 and Follower Count = 1 (Virat).
- Arjun and Rhea follow each other, so both have Following Count = 1 and Follower Count = 1.
- Virat follows only Mahi, so Virat's Following Count = 1 and Follower Count = 0.
- This shows how follow and followee relationships are stored and connected in the Follower Table.

5. Design APIs

API	Method	Description	Request Example	Response Example
/signup	POST	Create a new user	{ "username": "arjun_dev", "email": "arjun@gmail.com", "password": "abc123" }	{ "message": "User created successfully", "user_id": 1 }
/login	POST	Authenticate a user	{ "email": "arjun@gmail.com", "password": "abc123" }	{ "token": "xyz123" }
/follow	POST	Follow another user	{ "follower_id": 1, "followee_id": 2 }	{ "message": "Now following user 2" }
/unfollow	DELETE	Unfollow a user	{ "follower_id": 1, "followee_id": 2 }	{ "message": "Unfollowed user 2" }
/followers/{id}	GET	Get list of followers for a user	/followers/2	[{ "follower_id": 1, "username": "arjun_dev" }]

6. Architecture



Follow the numbers in the diagram

1. Client to API Gateway

The user interacts from a mobile app or web browser. The request first reaches the API Gateway. It checks authentication, routes the request, and controls traffic between client and backend services.

2. API Gateway to Load Balancer

The API Gateway sends the request to the Load Balancer. The Load Balancer's job is to distribute requests across multiple instances of backend services so the system stays stable during heavy traffic.

3. Load Balancer to User Service

The Load Balancer forwards signup or login requests to the User Service. The User Service handles account creation, password hashing, and profile setup. For example, when Arjun signs up, details are processed here.

4. User Service to SQL Database

User details are stored in the SQL database. The database saves data like username, email, and hashed password. This makes sure user information is structured and easy to retrieve later.

5. User Service to Kafka

After signup, the User Service pushes a message to Kafka. This message tells other parts of the system that a new user has joined, so actions like welcome notifications can happen.

6. Load Balancer to Connection Service

When a user follows another person, the request is sent to the Connection Service. This service handles relationships between users, like who follows whom or who unfollowed.

7. Connection Service to SQL Database

The Connection Service stores follow details in the same or related SQL database tables. For example, if Arjun follows Rhea, a record is saved with follower_id = 1 and followee_id = 2.

8. Connection Service to Kafka

Once a follow action is recorded, the Connection Service sends a message to Kafka. This message triggers notifications or updates follower counts for both users.

9. Kafka to Notification Service

Kafka passes the message to the Notification Service. The Notification Service sends updates like "Arjun started following you" in real time to the target user's device.

7. Recap

- User signs up or logs in from the mobile app or web.
- Request goes to the API Gateway, then to the Load Balancer.

- Load Balancer routes signup requests to the User Service.
- User Service validates input, hashes the password, and stores details in the SQL database.
- User Service sends a signup event to Kafka for background actions like welcome notifications.
- When a user follows another, the request again passes through the API Gateway and Load Balancer.
- Connection Service receives the follow request and stores follower–followee details in SQL.
- Connection Service sends a follow event to Kafka for further processing.
- Kafka forwards this event to the Notification Service.
- Notification Service delivers updates like “Arjun started following you.”
- All operations are handled asynchronously through Kafka, keeping the system responsive and scalable.

285. **HLD 17.2: Design Instagram (Post and Feed System Design)**

The Post and Feed system is the heart of Instagram. It decides how content is created, stored, and delivered to millions of users in real-time.

In the Post and Feed system, when a person uploads a photo or video, it is saved in storage and the post details go into the database. And then this post has to be added all followers’ feeds.

For example, if Arjun posts a photo or video, his followers instantly see it in their feed.

Let’s see how this happen (It is very interesting to know how it works in real time as we use this every day).

1. Functional Requirements

- Allow users to upload photos or videos with captions.
- Save post details like media, caption, and timestamp.
- Show posts from people we follow in the feed.
- Keep likes and comments updated in real time.

2. Non-Functional Requirements

- Feed should load quickly, even with millions of users.
- The system must handle heavy traffic during peak hours.
- Data must stay secure and consistent.
- Posts should reach followers instantly after upload.

3. Components

Component	Purpose
Load Balancer	Distributes incoming requests evenly to prevent overload.
API Gateway	Handles authentication and routes requests to the correct service.

Component	Purpose
User Service	Manages user profiles and validation.
Post Service	Handles post creation, captions, and media linking.
Media Service	Compresses, stores, and retrieves images or videos.
Feed Service	Builds and ranks feeds for users.
Cache (Redis)	Stores feed lists and counts for faster access.
Message Queue (Kafka)	Sends background events like “new post created.”
Database	Stores user details, posts, and relationships.

4. Database

Instagram uses Cassandra, Cassandra is designed for both high write and high read workloads, but it's especially good at high write throughput, meaning it can handle millions of writes per second without slowing down reads.

But for simplicity, we have used SQL to understand it better.

User Table:

user_id	username	email	created_at	follower_count	following_count
1	arjun_dev	arjun@gmail.com	2025-11-03	1000	50
2	manoj_fit	manoj@gmail.com	2025-11-03	10000	50

Post Table

post_id	user_id	caption	media_url	created_at
101	1	Java notes	s3://posts/fit101.jpg	2025-11-03
102	2	Healthy meal	s3://posts/meal102.jpg	2025-11-03

Follower Table

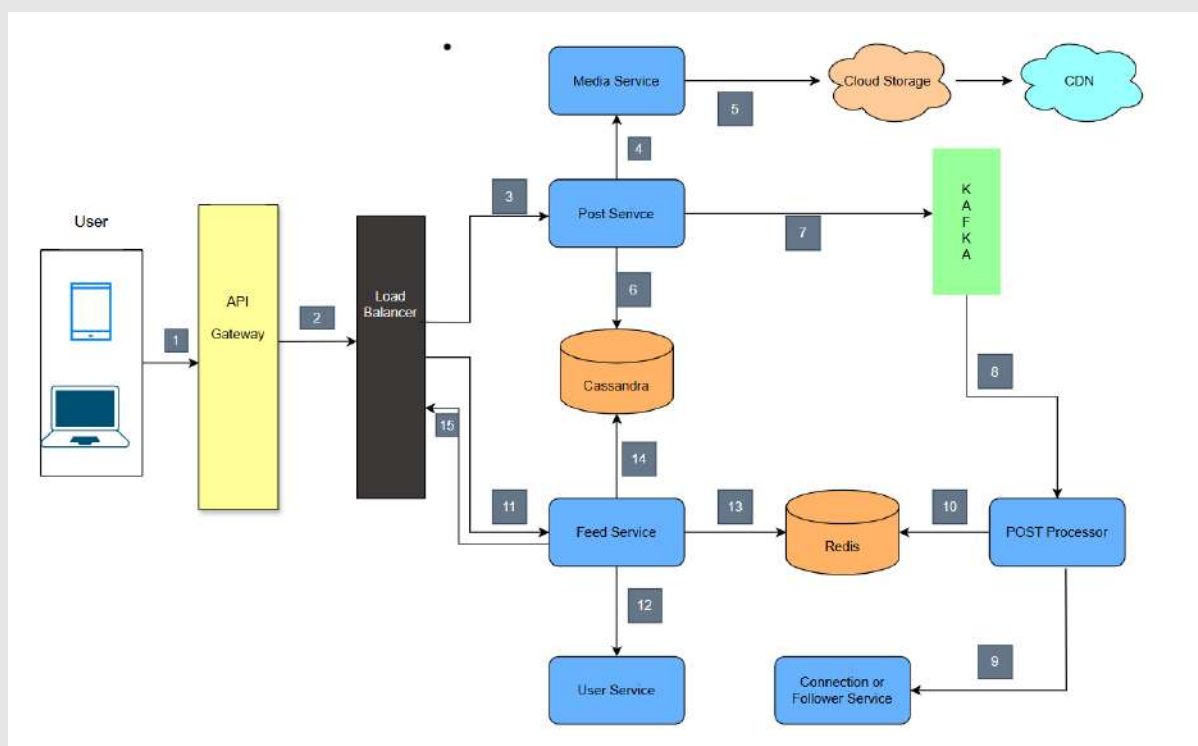
follower_id	followee_id
2	1
3	1
4	2

User-2 (Manoj) and User-3 follow user-1(Arjun), and user-4 follows user-2 (Manoj).

5. Design APIs

API	Method	Description	Request Example	Response Example
/create-post	POST	Upload new post	{ "user_id": 1, "caption": "Java Notes", "media_url": "s3://posts/fit101.jpg" }	{ "message": "Post created", "post_id": 101 }
/get-feed	GET	Get feed for user	/get-feed?user_id=2	[{ "post_id": 101, "user": "arjun_dev", "caption": "Morning workout" }]
/follow	POST	Follow another user	{ "follower_id": 2, "followee_id": 1 }	{ "message": "Now following user 1" }

6. Architecture



How Posts and Feeds Work

let's understand how posts and feeds actually work.

We will focus mainly on three key services: Post-Service, Post-Processor, and Feed-Service.

1. Post-Service: Handles User Uploads

Let's say there are 3 users: Arjun, Virat Kohli, and Manoj. Manoj follows both Virat and Arjun.

User Table

user_id user_name followers_count following_count

1	Arjun	1000	0
2	Virat	4M	0
3	Manoj	0	2

Followers Table

follower_id	following_id	Explanation
3	2	Manoj follows Virat
3	1	Manoj follows Arjun

Now, let's say Arjun posts a reel.

When a person uploads a photo or reel, the process starts from the mobile app. The Post-Service takes care of everything related to that post.

Here's what happens:

1. Post-Service receives the caption, user ID, tags, and media file.
2. The media file is sent to the Media Service, which compresses and resizes it, then stores it in cloud storage like S3.
3. The Media Service returns a storage link to the Post-Service.
4. The Post-Service saves this link, along with other details, in the Post Database.

Posts Table

post_id	user_id	caption	media_url	created_at
201	1 (Arjun)	Early morning gym	s3://posts/gym1.jpg	2025-11-03

Now that Arjun's post is saved, Manoj should be able to see it in his feed. This is where the Post-Processor comes into the picture.

After saving the post, the Post-Service sends an event to Kafka. This event says, "A new post has been created by user-1 (Arjun)."

The Post-Processor consumes this event and starts building feeds for followers. it will **pre-compute** the feed for followers before they open the app.

2. Post-Processor: Prepares the Feed

When the Post-Processor receives the Kafka event saying "A new post has been created by Arjun (user-1)," it asks the Follower Service for Arjun's list of followers.

Follower Information

follower_id	followee_id	Explanation
3	1	Manoj follows Arjun

Since Manoj follows Arjun, the Post-Processor now precomputes Manoj's feed. It stores Arjun's post ID in Redis so that Manoj's feed is ready even before he opens the app.

Redis (Precomputed Feed)

Key	Value
feed:user:3	[201, 99, 95]

Here, 201 is Arjun's new post ID.

This means when Manoj opens the app, Arjun's new post appears instantly at the top of his feed.

This process is called fan-out, because one post "fans out" to many followers' feeds.

This technique is called **fan-out-on-write**, when a user posts something, the Post-Processor immediately "fans out" (pushes) that post to all their followers' feeds. So, by the time follower open the app feed is already ready to fetch.

Here Arjun has only 1000 follower, so Post-Processor can easily update the feed in the Redis.

The Problem with Large Fan-Out

Now imagine **Virat** posts a reel. He has **4 million followers**.

Updating 4 million feed entries every time he posts is not efficient.

Posts Table

post_id	user_id	caption	media_url	created_at
501	2 (Virat)	Cricket Session	s3://posts/cricket.jpg	2025-11-03

To handle this kind of scale, Instagram uses another method called **fan-out-on-Read**.

Fan-out-on-Read: This Smarter Approach for Celebrities

In Fan-out-on-Read, instead of pushing posts to all followers immediately, the Feed-Service builds the feed only when the follower opens the app.

From our example:

When Manoj opens the app, his feed already has Arjun's post (because it was precomputed), and Virat's new post will be fetched directly from the database on demand.

So, by mixing both methods, fan-out-on-write for normal users and fan-out-on-read for celebrity accounts, Instagram keeps the system balanced and efficient.

Why We Need Post-Processor to Precompute Feeds

Let's say Manoj follows 2,000 people.

If there was no Post-Processor, here's what the Feed Service would have to do each time Manoj opens the app:

1. Get the list of all 2,000 people he follows.
2. Fetch their latest posts from Cassandra.
3. Sort all those posts by time or popularity.
4. Send the top 20 back to the app.

That means thousands of database queries for just one user.

Now imagine millions of users doing this at the same time , the database would slow down heavily.

This is why precomputation done by the Post-Processor is so important.

It makes sure the feed is ready ahead of time and the app feels fast when we open it.

3. How Feed-Service works?

It Shows the Final Feed

We know both Arjun and Virat have posted their reels. When Manoj opens the app, the Feed-Service gets the request charge.

User Table

user_id	user_name	followers_count	following_count
---------	-----------	-----------------	-----------------

1	Arjun	1000	0
2	Virat	4M	0
3	Manoj	0	2

- User 1 (Arjun): Normal user - uses fan-out-on-write (feed prebuilt in Redis).
- User 2 (Virat): Celebrity - uses fan-out-on-read (posts fetched from DB on demand).

How Feed-Service Actually Works

When Manoj opens the app, the Feed-Service gets a request like: `GET /feed?user=3`

Feed-Service then asks the User Service: “Does user-3 (Manjo) follow anyone with more than 50 lakh followers?”

User Service checks the database and replies, “Yes, user-3 follows Virat (user-2), who has 4M followers.”

Now Feed-Service knows:

- Arjun’s data is already in cache (Redis).
- Virat’s data needs to be fetched from the database.

So, Feed-Service does both:

- Fetches Arjun’s precomputed posts from Redis.
- Runs a database query like `SELECT post_id FROM posts WHERE user_id = 2 ORDER BY created_at DESC LIMIT 3;` to get Virat’s latest posts.

It also calls User-Service to fetch each user’s name and profile image, and finally sends the combined feeds back to Manoj’s app.

Reference videos:

✓ [System Design 13: Design Social Media like Facebook | Instagram | Feeds Generation | HLD | LLD](#)
[Facebook System Design | Instagram System Design | System Design Interview Question](#)

Reference articles:

[Instagram architecture & database – How does it store & search billions of images - Scaleyourapp](#)

286. [HLD 17.2\3: Design Instagram \(Comment\)](#)

Refer the Question . HLD-16.

The comments flow follows the same approach as we discussed in the YouTube comments section.

287. [HLD 18: Design Twitter](#)

We will divide this into 4 Sections

1. User onboarding and Follows ([Refer Question No. 284 HLD 17.1](#))
2. Post the tweet
3. Comments ([Refer Question No. HLD 16](#))
4. Search ([Refer Question No. HLD 5](#))

We will Post and Timelines Flow. It is almost like Instagram “Post and Feed” Design

The differences are

- Insta is Media overload, Twitter and Text overload.
- Insta, we call it feed, In Twitter we call it as Timeline.

Twitter is built around two main features: **Posting tweets** and **showing the timeline**.

Let us keep it simple and walk through how both parts work step by step.

When we post a tweet, it goes through a service that handles writing, storing, and sending updates to followers.

When we open the app, the timeline shows tweets from people we follow.

To make this fast, Twitter uses caching and background updates to build the feed for each user.

1. Gather Requirements

Functional Requirements

- Post a new tweet with text, image, or video
- View a timeline showing tweets from followed users
- Follow or unfollow users
- Like and retweet tweets

2. Non-Functional Requirements

- Handle millions of tweets per second
- Low latency for reading timelines
- Scalable for high traffic
- Secure and reliable data storage

3. Components

Component	Purpose
Load Balancer	Distributes incoming requests to backend servers.
API Gateway	Routes and verifies requests.
Tweet Service	Handles creation and storage of tweets.
Media Service	Processes and uploads photos or videos.
User Service	Manages user data and authentication.
Timeline Service	Builds and returns timelines for each user.
Follower Service	Maintains follower and followee relationships.
Cache (Redis)	Stores hot timelines and tweet data for faster reads.
Database (Cassandra/MySQL)	Saves tweet details and relationships.
Kafka	Passes tweet events to update timelines.
CDN	Delivers media files like profile images or videos quickly.

4. Database

We can use **Cassandra** for tweets because it handles high write traffic well. Relational DB (like MySQL) can be used for user and follower data.

User Table

user_id username followers_count following_count

1	arjun_dev	150	90
2	richa_fit	210	130

Tweet Table

tweet_id	user_id	content	media_url	created_at
101	1	"First tweet!"	NULL	2025-11-03
102	2	"Morning vibes!"	s3://tweets/t2.jpg	2025-11-03

Follower Table

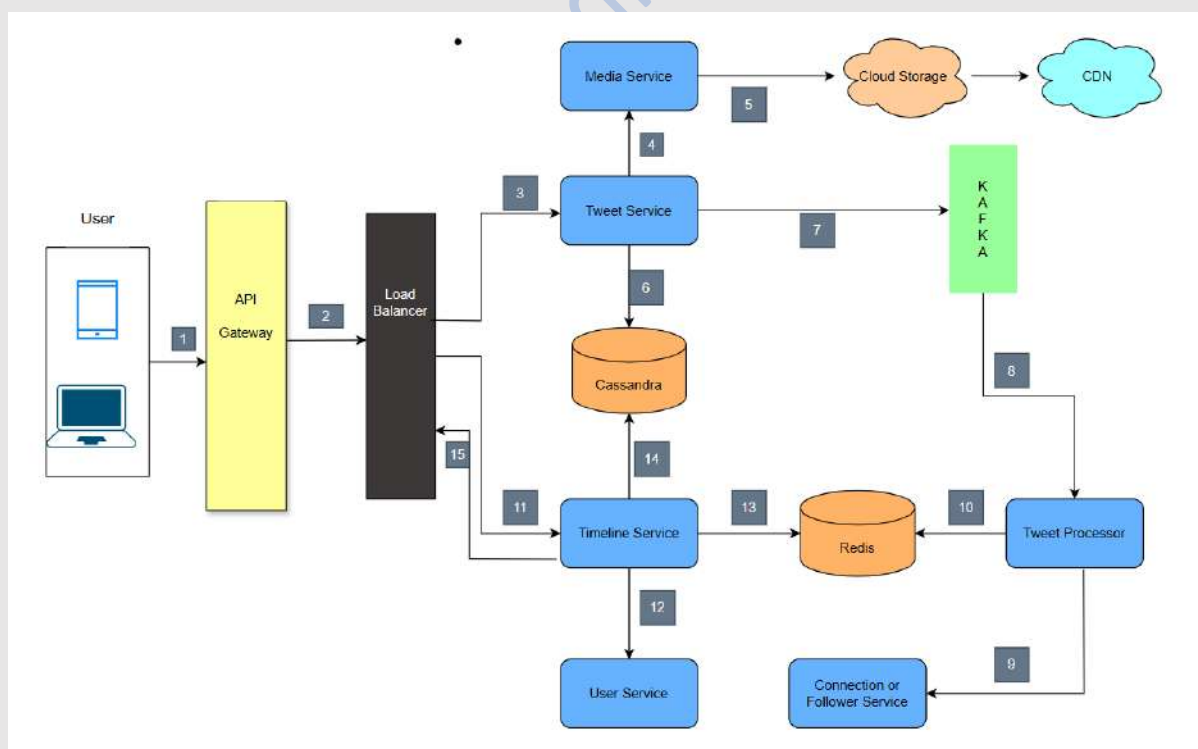
follower_id followee_id

1 2

2 1

5. Design APIs

API	Method	Description	Request Example	Response Example
/tweet	POST	Create a new tweet	{ "user_id": 1, "content": { "message": "Tweet posted", "Good morning!" } }	{ "tweet_id": 101 }
/timeline/{user_id}	GET	Get timeline for a user	/timeline/1	[{ "tweet_id": 102, "author": "rhea_fit", "content": "Morning vibes!" }]
/follow	POST	Follow another user	{ "follower_id": 1, "followee_id": 2 }	{ "message": "Now following user 2" }
/unfollow	DELETE	Unfollow a user	{ "follower_id": 1, "followee_id": 2 }	{ "message": "Unfollowed user 2" }



Explanation (Follow the numbers in the diagram)

1. The user clicks Post Tweet from mobile or web. The request first goes to the API Gateway.

2. The API Gateway sends the request to the Load Balancer. The load balancer decides which backend service should handle this request.
3. The request reaches the Tweet Service. This service is responsible for creating and saving a new tweet.
4. If the tweet has an image or video, Tweet Service calls the Media Service. Media Service prepares the media for storage.
5. Media Service stores the image or video in Cloud Storage. From there it is pushed to CDN so it can be served fast when users view it.
6. Tweet Service saves the tweet text and media link in Cassandra. This is the main database for storing tweets.
7. Tweet Service publishes a Tweet Created event to Kafka. Kafka is used to tell other parts of the system that a new tweet was posted.
8. Tweet Processor reads the new tweet event from Kafka. This service now starts fanout work for timelines.
9. Tweet Processor calls the Connection or Follower Service. It asks who all are following this user.
10. Tweet Processor writes the tweet into Redis. This is done for fast access to timelines.
11. When a user opens the app and asks for their timeline, the request goes through the Load Balancer to the Timeline Service.
12. Timeline Service calls the User Service. It gets user details needed to build the timeline.
13. Timeline Service reads tweets from Redis. Redis gives very fast timeline data.
14. If some tweets are not in Redis, Timeline Service fetches them from Cassandra.
15. Timeline Service sends the final timeline back through the Load Balancer to the API Gateway, and then to the user.

a. Flow of Posting a Tweet

1. Request starts at **API Gateway** and passes through the **Load Balancer**.
2. **Tweet Service** receives the request and validates the user through **User Service**.
3. If the tweet has media, it goes to **Media Service**, which uploads it to **Cloud Storage** and serves it via **CDN**.
4. The Tweet Service stores the tweet in **Cassandra** and sends a “new tweet” event to **Kafka**.
5. **Tweet processor service** consumes this event and adds the tweet ID to the feeds of all followers (fan-out on write).
6. The tweet ID lists are stored in **Redis** for quick reads later.

Example:

If Arjun has 3 followers and posts a tweet with tweet_id=101, Redis stores:

Key	Value
timeline:user:2	[101, 99, 90]

Key	Value
timeline:user:3	[101, 95, 93]
timeline:user:4	[101]

b. Flow of Viewing Timeline

1. When Richa opens her app, **Timeline Service** checks Redis for her timeline: timeline:user:2.
2. Redis returns tweet IDs in sorted order.
3. **Timeline Service** fetches tweet details (text, media, author) from cache or Cassandra.
4. User profiles are fetched from **User Service**.
5. Media URLs are served from **CDN**, ensuring fast image and video loading.
6. The final timeline is returned to the user instantly.

c. Fan-out Model

- **Normal users:** Fan-out on write (tweets are pushed to all followers' timelines in Redis).
- **Celebrities:** Fan-out on read (followers get celebrity tweets only when they open their timeline).

This balance keeps both storage and performance under control.

d. CDN in Twitter

CDN helps deliver images, videos, and profile photos quickly. Once Media Service uploads a file to Cloud Storage, CDN copies it to edge servers worldwide.

So when we scroll our timeline, media loads from the nearest CDN node, not the main storage bucket.

Reference: [System Design: Twitter. Learn how to design twitter for... | by Karan Pratap Singh | Medium](#)

Very detailed but only HLD: [Twitter System Design | System Design Interview Question](#)

288. HLD 19.1: Design Riding APP like Uber or Ola (Driver Perspective)

For simplicity, have divided the into 4 parts

1. Driver Side Design
2. Rider Side Design
3. Payment design
4. Rating design

Let's see how the system works from the driver's side.

This part of the design mainly focuses on how drivers update their availability, share live location, and how those updates reach the rider side.

1. Functional Requirements

- Driver can set availability (online/offline)
- Driver's current location is tracked in real time
- Driver can accept or reject ride requests

- Driver's app shows pickup location on map
- Location updates are sent to backend for rider matching

2. Non-Functional Requirements

- Location updates should be real-time (low latency)
- System should handle millions of concurrent connections
- Location data should be secure
- Driver should not be assigned two rides
- Should be scalable across regions

3. Components

Component	Purpose
Driver App (Mobile)	Lets the driver go online/offline, share location, and accept rides
Driver Service	Manages driver profile, status (available, busy), and updates
Location Tracking Service	Stores and processes live driver locations
Geocoding/Geocaching Service	Converts coordinates into readable addresses
Map Service	Shows routes, pickup/drop points, and estimated distance
Surcharge Service	Calculates surge pricing based on demand in a specific area
Message Queue	Handles ride request notifications asynchronously
WebSocket Server	Maintains live connection with driver apps for instant updates
Cache (Redis)	Stores nearby driver locations temporarily for quick lookup
Database	Stores driver info, rides, and location history

4. Database

For driver-related data, both SQL and NoSQL can be used.

- **SQL (e.g., PostgreSQL)** for structured data like driver profile, trip history.
- **NoSQL (e.g., Redis, Cassandra)** for frequently updated data like current location.

Example Tables

Driver Table (SQL)

Field	Type	Description
driver_id	int	Unique driver ID
name	text	Driver name
status	enum	online / offline / busy
rating	float	Average driver rating

We can also maintain Vehicle details here in Driver table

Driver_Location (Cassandra and Redis)

Key: driver_id

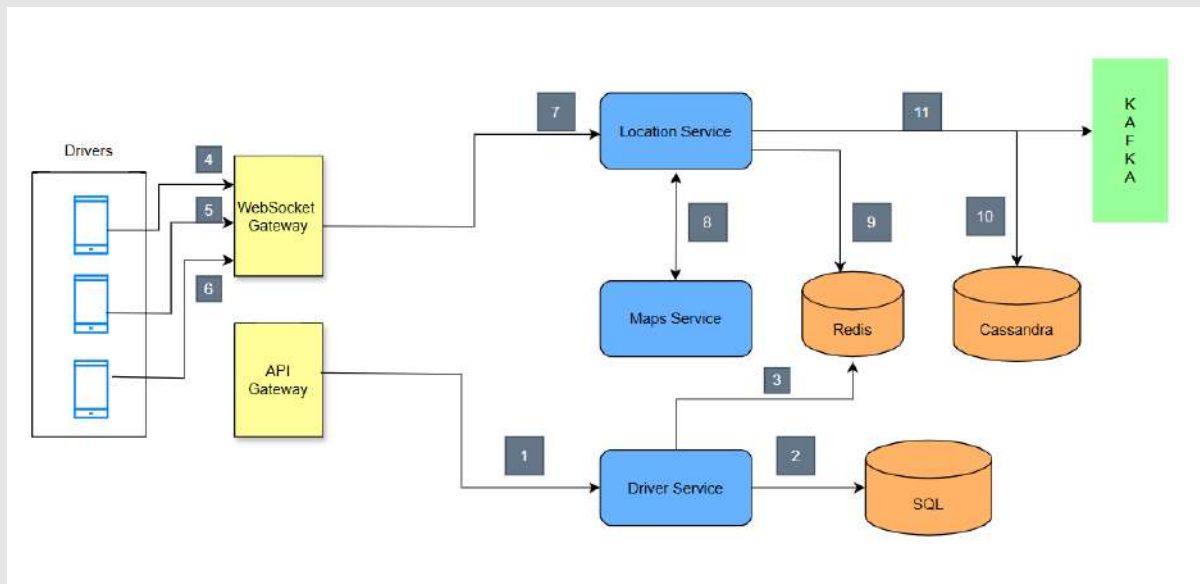
Value: { latitude, longitude, timestamp, city, isAvailable }

This helps the system quickly find available drivers near a location.

4. Design APIs

API	Method	Description
/driver/login	POST	Authenticate driver
/driver/{driverId}/status	PUT	Update availability (online/offline)
/driver/{driverId}/location	POST	Send current location
/driver/{driverId}/ride/{rideId}/accept	POST	Accept ride request
/driver/{driverId}/ride/{rideId}/accept	POST	Mark ride as completed

5. Architecture



Follow the numbers in the diagram.

1. API Gateway to Driver Service

When a driver logs in, updates their profile, or changes their availability, the request passes through the API Gateway and reaches the Driver Service.

The API Gateway checks authentication (like token or session) and forwards valid requests to the Driver Service safely.

2. Driver Service to SQL

The Driver Service stores all static and semi-static information in SQL.

This includes the driver's name, license details, phone number, rating, and current availability. For example, when a driver goes online, their status in SQL is updated as "available".

3. Driver Service to Redis

Once the status changes, the Driver Service updates Redis with the driver's latest state.

Redis keeps only real-time data such as "available", "busy", or "offline".

This makes it quick for the system to find nearby available drivers without checking SQL every time.

4, 5, 6. Driver App to WebSocket Gateway

- When a driver goes online, the app creates a **WebSocket connection** with the backend through the WebSocket Gateway.
- This connection stays open as long as the driver is active.
- It helps the system receive live updates instantly instead of waiting for new HTTP requests.
- The driver app continuously sends GPS coordinates (latitude and longitude) through the open WebSocket connection.
- Each driver has a separate connection, which keeps communication private and real-time.
- This is how the backend knows exactly where every driver is every few seconds.
- Regular actions that are not time-sensitive, like "accept ride" or "end trip", still use the normal API Gateway through REST calls.
- The WebSocket is only for live streaming data like location.

7. WebSocket Gateway to Location Service

The WebSocket Gateway receives live location data and forwards it to the Location Service. For example, if the driver moves from Gachibowli to Madhapur, new coordinates are sent every few seconds.

8. Location Service to Maps Service

The Location Service sends these coordinates to the Maps Service to convert them into readable addresses from Google Map API. Maps Service might return something like “Hitech City Road, Hyderabad”.

9. Location Service to Redis

After processing, the Location Service updates Redis with the driver’s latest location and timestamp. Redis keeps only fresh data, maybe for 60 seconds, so that other services like Rider Matching can use it instantly.

10. Location Service to Cassandra

The same location data is also written to Cassandra for long-term storage.

This helps with analytics, driver performance tracking, or building heatmaps showing where most drivers are active.

Old data in Cassandra can be cleared after a few days using TTL.

11. Location Service to Kafka

Every location update is also sent to Kafka, which acts as a live event stream.

Services like **Surge Pricing, Analytics, and Demand Prediction** read from Kafka to make decisions such as raising fares in high-demand areas.

7. Recap

- Driver logs in or updates profile through the API Gateway, which routes the request to the Driver Service.
- Driver Service stores profile and availability details in the SQL database.
- Driver Service also updates Redis with the driver’s online or offline status for quick access.
- When the driver goes online, the mobile app opens a WebSocket connection with the WebSocket Gateway.
- The app starts sending live GPS coordinates every few seconds through this WebSocket.
- WebSocket Gateway forwards those location updates to the Location Service.
- Location Service processes the coordinates and sends them to the Maps Service to get actual address from Map API
- The latest driver location is stored in Redis for real-time lookup by rider services.
- The same location data is written into Cassandra for history and analytics.
- Every update is also sent to Kafka for services like surge pricing, demand prediction, and monitoring.
- The WebSocket connection stays open for continuous updates and two-way communication, allowing the backend to send ride requests instantly to the driver.

289. HLD 19.2: Design Riding APP like Uber or Ola (Rider Perspective)

Let's see how the system works from the rider's side.

1.Functional Requirements

- Rider can register and log in.
- Rider can enter pickup and drop locations.
- System shows fare estimate before confirming.
- Rider can confirm a ride and get matched to a nearby driver.
- Rider can track driver's location live.
- Rider receives notifications like "Driver Accepted" or "Ride Started".
- Rider can make payment after the trip.
- Rider can rate the driver after completion.

2.Non-Functional Requirements

- Location updates should be real-time.
- The system must scale easily as riders increase.
- Data and payments should be secure.
- APIs must respond fast, especially during high load.
- All services must stay reliable even if one fails.

3. Components

Component	Description
Rider App	Used by rider to request rides, track drivers, and make payments.
API Gateway	Handles all incoming rider requests.
Rider Service	Manages rider details, ride history, and booking requests.
Pricing Service	Calculates estimated and final fares using distance and time.
Map Service	Finds routes, distance, and ETA between pickup and drop.
Matching Service	Finds nearby available drivers using cached driver data.
Notification Service	Sends updates like "Driver Accepted" or "Ride Arriving".
WebSocket Gateway	Keeps live two-way connection between rider and driver.
Location Service	Collects driver GPS updates every few seconds.
Real-Time Location Service	Pushes updated driver location to rider app live.

Component	Description
Driver Location Cache (Redis)	Stores current driver coordinates for faster searches.
Kafka (Message Queue)	Handles background updates like location streams and ride requests.

4. Database

For driver-related data, both SQL and NoSQL can be used.

- **SQL (e.g., PostgreSQL)** for structured data like driver profile, trip history.
- **NoSQL (e.g., Redis, Cassandra)** for frequently updated data like current location.

Example Tables

Table	Fields	Sample Data
Rider	rider_id, name, phone, rating	101, Ravi, 9876543210, 4.8
Ride	ride_id, rider_id, driver_id, pickup_lat, pickup_long, drop_lat, drop_long, fare, status	5001, 101, 303, 12.973787, 77.607879, 12.9987871, 77.647874, 280, completed
Payment	payment_id, ride_id, amount, mode, status	P900, 5001, 280, "UPI", success
Rating	rating_id, ride_id, rider_id, driver_id, rating, feedback	R1, 5001, 101, 303, 5, "Smooth and safe ride"

Driver_Location (Cassandra and Redis)

Key: driver_id

Value: { latitude, longitude, timestamp, city, isAvailable }

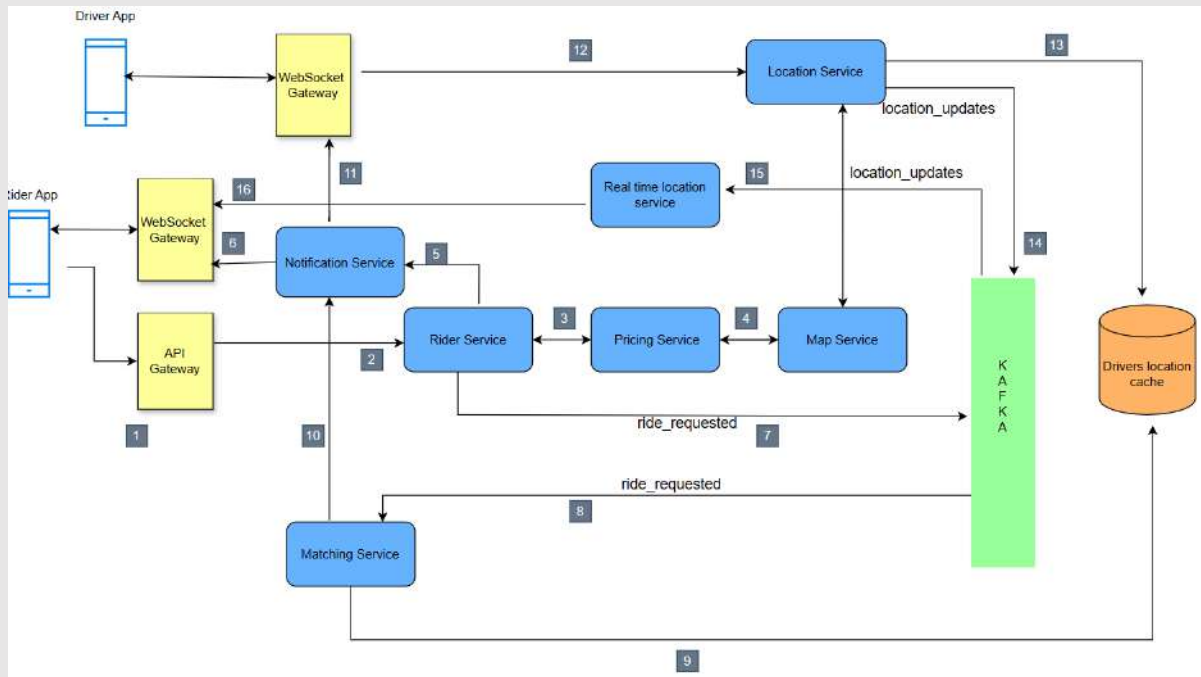
This helps the system quickly find available drivers near a location.

4. Design APIs

API	Method	Description
/rider/register	POST	Register new rider
/rider/{riderId}book	POST	Create a ride request
/rider/{riderId}/cancel	POST	Cancel the ride request
/rider/{riderId}/track	GET	Get driver's current location

API	Method	Description
/payment/pay	POST	Complete ride payment
/rating/add	POST	Submit driver rating

5. Architecture



Follow the numbers in the diagram.

1. Rider App to API Gateway

When a rider enters pickup and drops locations and taps confirm, the request first reaches the API Gateway. It's the main entry point that routes requests safely to backend services.

2. API Gateway to Rider Service

The request goes to the Rider Service. It checks rider details, ride history, and booking validity. If everything looks fine, it prepares data for fare estimation.

3. Rider Service to Pricing Service

Rider Service sends pickup and drops coordinates to Pricing Service. Pricing Service calculates an estimated fare using distance, time, and base rates before booking confirmation.

4. Pricing Service to Map Service

Pricing Service calls the Map Service to get route, distance, and ETA. Here, **geohashing** is used. It converts GPS coordinates into small rectangular grid codes, making it easier to search nearby drivers quickly. Example: two locations inside the same grid mean they are close to each other.

5. Rider Service to Notification Service

After the fare is confirmed, Rider Service tells the Notification Service to update the rider that a nearby driver search has started. This ensures instant feedback in the app.

5a. Rider Service to SQL

Ride service updates Ride table in SQL database, about ride details and status like “Driver assigned”, “Trip Started”, “Trip Completed”.

6. Notification Service to WebSocket Gateway (Rider App)

Notification Service sends a live message through WebSocket Gateway. The rider sees “Finding nearby drivers...” on the app without needing to refresh.

7. Rider Service to Kafka (ride_requested)

- Rider Service publishes a ride_requested event to Kafka.
- This message will be read by other backend services like Matching Service to find available drivers asynchronously.

8. Kafka to Matching Service

Matching Service consumes the event from Kafka. It fetches all available drivers from the location cache and finds the ones closest to the rider using the geohash grid.

9. Matching Service to Cache

Matching Service reads the locations from Cache, and find nearest driver .

10. Matching Service to Notification Service

- Matching service asks Notification service to send notification to driver about the ride.

11. Notification Service to WebSocket Gateway to Driver App

- Notification Service sends ride acceptance confirmation to the driver app.
- Once a driver accepts the request, Matching Service notifies the Notification Service to send confirmation details (driver info and ETA) back to the rider.
- Both rider and driver now stay connected through WebSocket for live updates.

12. Driver App to Location Service

Driver App starts sending GPS coordinates every few seconds to Location Service. This ensures that the system always knows the driver’s latest position.

13. Location Service to Driver Location Cache

- Location Service updates the in-memory driver location cache.
- This makes it faster for Matching Service to find available drivers without heavy database lookups.

14. Location Service to Kafka (location_updates)

Every location change is sent to Kafka as a location_update event. Other services like Real-Time Tracking or Surge Pricing can consume this data.

15. Kafka to Real-Time Location Service

Real-Time Location Service reads updates from Kafka. It pushes the driver’s current coordinates to the WebSocket Gateway for rider-side tracking.

16. WebSocket Gateway to Rider App

Rider receives live driver location updates through WebSocket. The driver's car moves smoothly on the map in real time until the ride starts and ends.

7. Recap

1. Rider logs in or updates their profile, request goes through the **API Gateway** to the **Rider Service**.
2. **Rider Service** stores profile, saved locations, payment info and ride details in the **SQL database**.
3. When the rider opens the app and wants a ride, the app sends a **ride request** to the backend through the **API Gateway**.
4. The **Ride Matching Service** takes this request and looks up nearby drivers using location data stored in **Redis**.
5. **Maps Service** helps calculate distance, ETA, and estimated fare between rider and available drivers.
6. The **Pricing/Surge Service** adjusts fare based on demand and sends the final price back.
7. Once a suitable driver is found, the **Matching Service** sends a **ride offer** to that driver through the **WebSocket Gateway**.
8. When the driver accepts, the system confirms the booking and updates both rider and driver apps in real time.
9. During the trip, both rider and driver apps send **location updates** through their WebSocket connections, these are processed by the **Location Service** for live tracking.

Reference links:

[Uber System Design | Websockets and Event Driven Architecture - YouTube](#)

[Uber - System Design Interview Question \(Ride Sharing Service\)](#)

290. HLD 19.3: Design Riding APP like Uber or Ola (Payment flow)

Let's see how the system works from the Payment flow.

After the ride ends, the app automatically shows the total fare. The rider can pay using UPI, card, wallet, or cash. Once the payment is done, both rider and driver get a confirmation message and a digital receipt for the trip.

1. Functional Requirements

- Calculate the final fare after the ride ends.
- Allow rider to pay using UPI, card, wallet, or cash.
- Verify payment success with payment gateway.
- Update ride and driver earnings after payment.
- Send a digital receipt and payment confirmation to the rider.

2. Non-Functional Requirements

- Payment must be secure and fast.
- The system should handle multiple payments at once.
- Must support refunds or failed transactions.
- Should store records safely for future reference.
- Payment confirmation should reflect instantly in the app.

2. Identify Components

Component	Description
API Gateway	Handles incoming payment-related requests securely.
Rider Service	Stores ride info and links ride with payment details.
Pricing Service	Sends final fare amount after ride completion.
Payment Service	Handles the payment process and connects with external gateways.
Payment Gateway	External third-party service (like Razorpay, Stripe, or Paytm) that processes the actual payment.
Notification Service	Sends success or failure messages and digital receipts.
Database (SQL)	Stores payment details, transaction ID, and status.
Kafka (Message Queue)	Used to send events like payment_completed or payment_failed for record updates and analytics.

4. Database

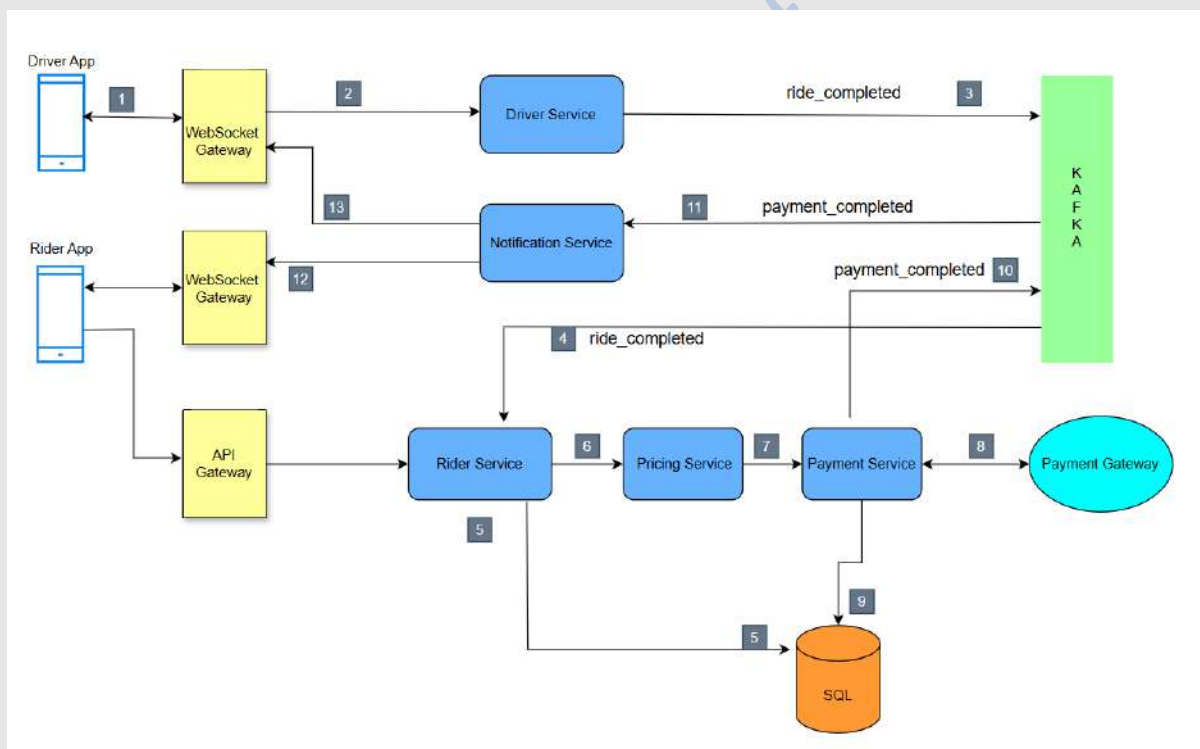
We can use SQL / Postgres SQL to store the data

Table	Fields	Example
Payment	payment_id, ride_id, rider_id, driver_id, amount, payment_mode, status, transaction_time	P1021, 5012, 101, 303, 280, "UPI", "success", 2025-11-06 18:20
Ride	ride_id, fare, payment_status	5012, 280, "paid"
Driver_Earnings	driver_id, ride_id, amount, status	303, 5012, 240, "credited"

5. APIs Design

API	Method	Description	Request Example	Response Example
/payment/initiate	POST	Start the payment process	{ "ride_id": 5012, "mode": "UPI", "amount": 280 }	{ "payment_id": "P1021", "status": "pending" }
/payment/confirm	POST	Confirm payment from gateway	{ "payment_id": "P1021", "status": "success" }	{ "status": "success", "message": "Payment received" }
/payment/{ride_id}/status	GET	Check payment result	{ "ride_id": 5012 }	{ "status": "success", "amount": 280 }
/payment/refund	POST	Process refund if payment failed or canceled	{ "payment_id": "P1021" }	{ "status": "refunded" }

6. Architecture



Follow the numbers in the diagram.

1. Driver App to WebSocket Gateway

Once the driver ends the trip, the app sends a “ride completed” signal to the backend through the WebSocket connection. This helps the system process the next payment step instantly.

2. WebSocket Gateway to Driver Service

The WebSocket Gateway passes the ride completion message to Driver Service. Driver Service updates the ride status to completed and prepares the data needed for payment and fare calculation.

3. Driver Service to Kafka

Driver Service sends a “ride_completed” event to Kafka. Kafka acts as a message broker, allowing other services like Rider and Payment Services to consume this event asynchronously.

4. Kafka to Rider Service

Rider Service listens to the “ride_completed” event from Kafka. Once received, it updates the rider’s ride record in the DB and triggers the Pricing Service to calculate the final trip fare.

5. Rider Service to SQL

Rider Service saves the updated ride status and timestamp into SQL for tracking and reporting purposes. This ensures data consistency before starting the payment process.

6. Rider Service to Pricing Service

Rider Service sends ride details like distance and duration to Pricing Service. Pricing Service calculates the total fare using the base rate, per-kilometer rate, and any surge multiplier.

7. Pricing Service to Payment Service

After calculating the fare, Pricing Service sends the total amount to Payment Service. Payment Service starts the payment flow by preparing details like amount, ride ID, and payment mode.

8. Payment Service to Payment Gateway

Payment Service connects with an external gateway such as Razorpay or Stripe. The gateway securely processes the transaction using UPI, card, or wallet and sends back the payment status.

9. Payment Service to SQL

Payment Service stores the transaction details in SQL, including payment ID, amount, mode, and status. This record helps in audits, refunds, and payment history tracking.

10. Payment Service to Kafka

Once the payment is completed successfully, Payment Service publishes a “payment_completed” event to Kafka. This helps other services like Notification or Driver Earnings to react automatically.

11. Kafka to Notification Service

Notification Service listens to the “payment_completed” event and prepares confirmation messages for both rider and driver about successful payment and trip closure.

12. Notification Service to WebSocket Gateway (Rider App)

The message is sent to the rider through WebSocket. The app instantly shows “Payment Successful” with the trip receipt and amount paid.

13. Notification Service to WebSocket Gateway (Driver App)

At the same time, the driver receives a confirmation message saying the payment has been received. This marks the end of the ride officially on both sides.

7.Recap

1. When the driver ends the trip, the Driver App sends a ride completion event through the WebSocket Gateway to the Driver Service.
2. Driver Service updates the ride status to completed and publishes a ride_completed event to Kafka.
3. Rider Service consumes this event from Kafka, updates ride records in SQL, and calls the Pricing Service to calculate the final fare.
4. Pricing Service computes total fare using base rate, time, distance, and surge.
5. Pricing Service sends the final fare and payment details to the Payment Service to begin the payment process.
6. Payment Service connects with an external Payment Gateway to process UPI, card, or wallet payments securely.
7. Once the payment is completed, Payment Service stores the transaction details like amount and status in the SQL database.
8. Payment Service publishes a payment_completed event to Kafka for asynchronous updates.
9. Notification Service consumes the payment event and prepares confirmation messages for both rider and driver.
10. Both Rider and Driver Apps receive instant updates through the WebSocket Gateway showing "Payment Successful" and trip closed.

291. **HLD 19.4: Design Riding APP like Uber or Ola (Rating flow)**

Let's see how the system works from the Rating flow.

After the payment is completed, both rider and driver can rate each other. The Rating Service receives these ratings, stores them in the database, and updates average scores. This helps improve trust and match quality in future rides, ensuring better driver behaviour and smoother experiences for every rider.

1. Functional Requirements

- After payment, both rider and driver can rate each other.
- Ratings and feedback are stored securely in the system.
- Average rating is calculated for both users.
- Notifications are sent confirming that feedback was received.
- The system shows overall driver and rider ratings in their profiles.

2. Non-Functional Requirements

- Ratings should be stored and retrieved quickly.
- The system must handle high traffic efficiently.
- Data should be safe, accurate, and consistent.
- Ratings should not be editable once submitted.
- The service must scale easily as user numbers grow.

3. Identify Components

Component	Description
API Gateway	Routes rating requests from apps to backend services.
Rating Service	Manages rating submission, validation, and average score calculation.
Rider Service	Updates rider profile with new rating details.
Driver Service	Updates driver profile and stores new average rating.
Notification Service	Sends confirmation messages to both rider and driver.
Kafka (Message Queue)	Handles asynchronous updates like analytics and behavior tracking.
SQL Database	Stores all ratings and feedback history.

4. Choose Database and Design It

SQL (Structured Storage)

Table	Fields	Example
Ratings	rating_id, ride_id, rider_id, driver_id, rating, feedback, created_at	R1002, 5012, 101, 303, 5, "Clean car and polite driver", 2025-11-06
Driver_Profile	driver_id, total_rides, average_rating	303, 258, 4.7
Rider_Profile	rider_id, total_rides, average_rating	101, 310, 4.8

Indexes:

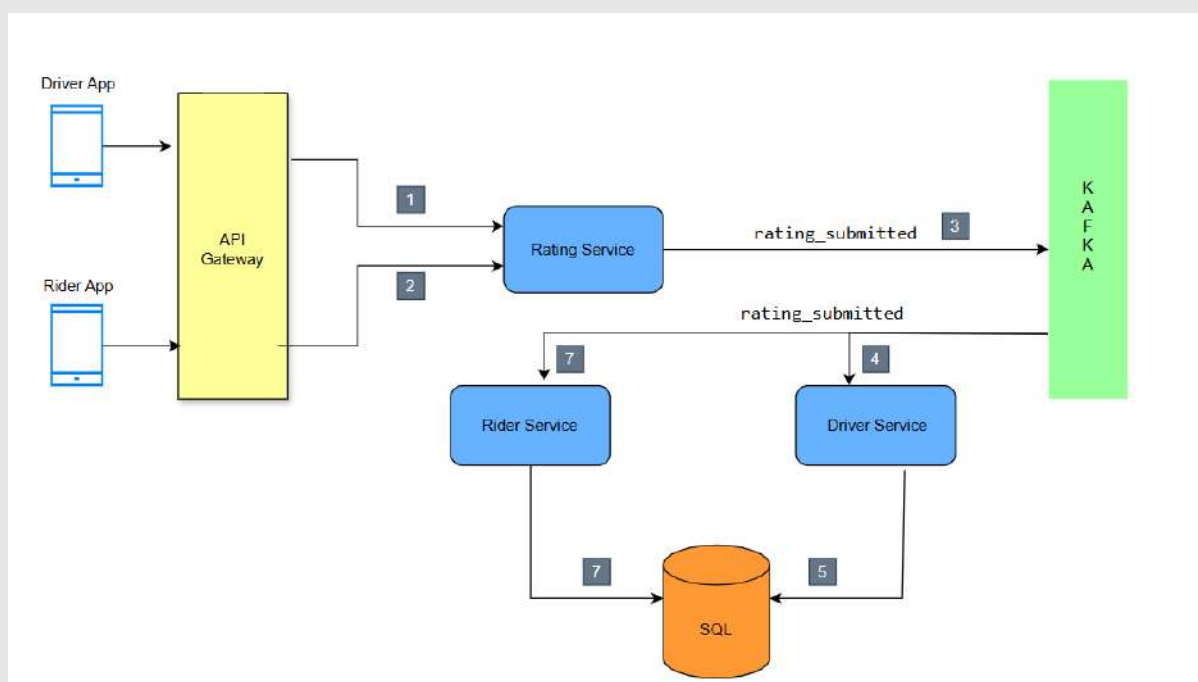
- Index on driver_id and rider_id for fast retrieval.
- Index on ride_id to link back to trip details.

5. Design APIs

API	Method	Description	Request Example	Response Example
/rating/submit	POST	Add rating and feedback after ride	{ "ride_id": 5012, "rider_id": 101, "driver_id": 303, "rating": 5, "feedback": "Very smooth ride" }	{ "status": "success", "message": "Rating submitted" }

API	Method	Description	Request Example	Response Example
/rating/{ride_id}/get	GET	Get past ratings for a ride	{ "ride_id": 5012 }	{ "rating": 5, "feedback": "Very smooth ride" }
/driver/{driver_id}/average	GET	Get driver's average rating	{ "driver_id": 303 }	{ "average_rating": 4.8 }

6. Architecture



Follow the numbers in the diagram.

1. Driver App to API Gateway

After completing the trip, the driver gives a rating and feedback to the rider. The request goes through the API Gateway before reaching the backend for processing.

2. Rider App to API Gateway

At the same time, the rider submits a rating for the driver. The API Gateway handles this request and sends it to the Rating Service for validation and storage.

3. Rating Service to Kafka

The Rating Service publishes a `rating_submitted` event to Kafka. This allows other services like Rider and Driver Services to pick up the event asynchronously and update profiles.

4. Kafka to Driver Service

Driver Service consumes the event from Kafka, updates the driver's average rating, and recalculates it based on previous rides. This ensures the driver's performance stays accurate and current.

5. Driver Service to SQL

Once the new rating is processed, Driver Service stores the updated score and feedback in the SQL database for reporting, analytics, and displaying on the driver's profile.

6. Kafka to Rider Service

Rider Service also consumes the same rating_submitted event. It updates the rider's average rating and keeps a record of the feedback given by the driver.

7. Rider Service to SQL

Finally, Rider Service saves the updated rating and feedback into SQL. This ensures both rider and driver histories are complete and available for future reference or trust calculations.

7.Recap

1. After the trip ends, both rider and driver open their apps to give ratings and feedback.
2. The rating request goes through the API Gateway to the Rating Service for validation.
3. Rating Service publishes a rating_submitted event to Kafka for further processing.
4. Driver Service consumes the event from Kafka and updates the driver's average rating.
5. The updated driver rating and feedback are stored in the SQL database.
6. Rider Service also consumes the same event and updates the rider's average rating.
7. The updated rider rating and feedback are stored in SQL for future reference.
8. Both ratings become visible instantly in their respective profiles on the app.

292. HLD 20: Design Google search Engine

A **Search Engine** like Google is built to find and show the most relevant web pages for what we search.

When we type a few words, and results appear instantly. It looks easy to us but behind the scenes, it's a massive system powered by **web crawling, indexing, and ranking**.

- **Web Crawlers:** Programs (often called spiders) that browse the internet and collect web pages.
- **Indexing System:** Processes crawled data, extracts meaningful information, and builds structures like inverted indexes for fast lookups.
- **Query Processor:** Interprets user queries, cleans them up, and decides what to search for in the index.
- **Ranking Engine:** Scores documents based on relevance, authority, freshness, and other signals.

Let's go step by step through its design in a simple

1. Functional Requirements

- Crawl and store web pages from the internet.
- Index all the pages based on their content and keywords.
- Accept search queries and return the most relevant pages instantly.

2. Non-Functional Requirements

- Handle billions of pages and queries efficiently.
- Show results within milliseconds.

3. Components

Component	Description
API Gateway / Search Gateway	Entry point for user search requests. Routes them to the Search Service.
Load Balancer	Distributes incoming search queries evenly across multiple Search Service instances.
Crawler Service (Googlebot)	Continuously visits web pages, reads their content, and follows links to discover new ones.
Scheduler and URL Frontier	Manages list of pages to crawl and decides crawl frequency for each domain.
Parser Service	Extracts useful information from HTML pages like text, title, and links.
Deduplication Service	Removes duplicate or near-identical pages before storing.
Indexer Service	Builds and updates the search index, which maps keywords to page URLs.
Ranking Service	Calculates relevance and orders pages using algorithms like PageRank and AI models.
Cache	Stores top results for repeated searches, improving response time.
Data Storage (Index Database)	Stores indexed data, page metadata, and keyword relationships.
Feedback & Analytics Service	Tracks user behavior (clicks, time on page) to improve ranking models.

4. Database

Google handles huge amounts of unstructured and semi-structured data, so it uses **NoSQL** and **distributed storage** systems.

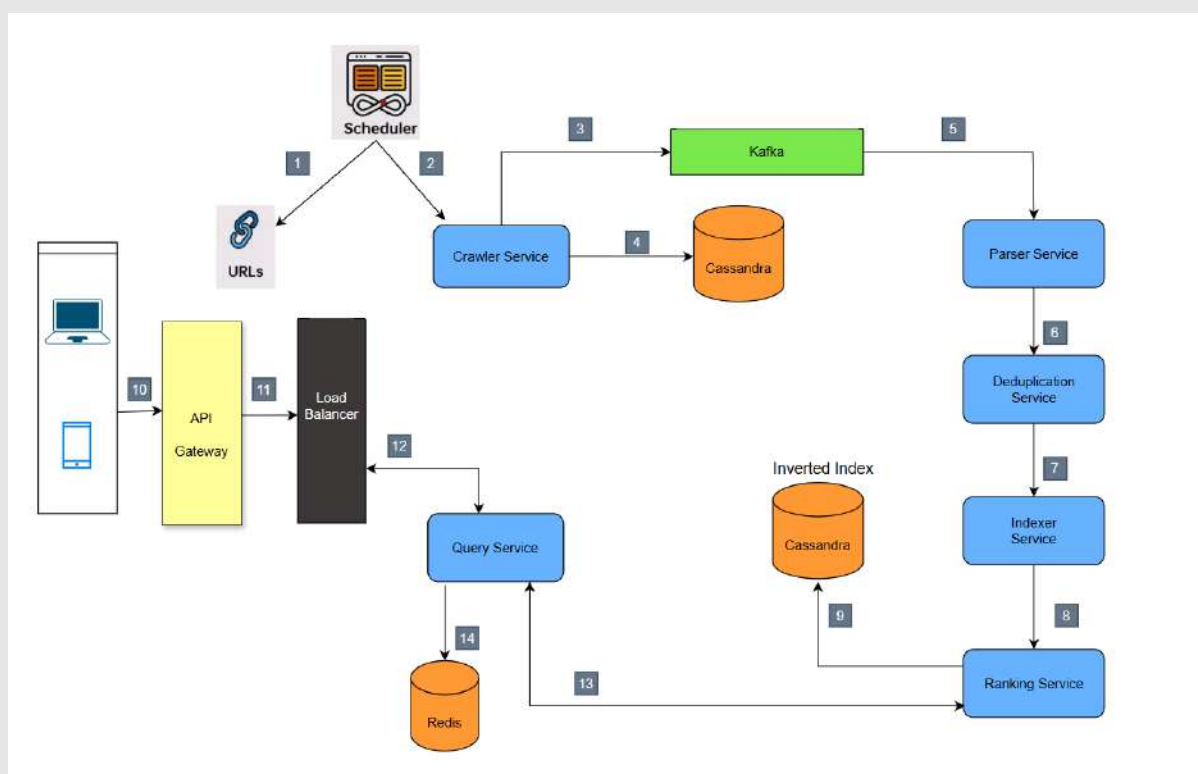
Table / Collection	Fields	Description
pages	url, title, content, last_crawled, rank_score	Stores crawled page content and metadata.

Table / Collection	Fields	Description
index	keyword, list_of_urls	Maps each keyword to related pages (used during search).

5. Design APIs

API	Method	Description
/search?q={query}	GET	Fetches top-ranked results for the given query.
/crawl/start	POST	Starts a crawl job with given seed URLs.

6. Architecture



Follow the numbers in the diagram.

1. Scheduler to URLs

The **Scheduler** decides which websites to crawl and when. It can receive URLs from from three places:

- Website owners** who submit their sites using *Google Search Console*.
Example: *cricbuzz.com*, *espncric.com* their sitemaps.
- Previously discovered links** from other pages.

3. **Public sources**, such as RSS feeds and shared links.

Example:

A cricket news site like <https://cricbuzz.com> submits its sitemap through Search Console. The Scheduler picks URLs such as <https://cricbuzz.com/match/india-vs-england> for crawling.

2. Scheduler to Crawler Service

The **Crawler Service** runs by the Scheduler and fetches the HTML pages. It downloads page content, metadata, and links within the page.

Example:

When it crawls the page <https://cricbuzz.com/match/india-vs-england>, it extracts player stats, match summaries, and links to scorecards or related matches.

Points to note:

- Google Search is one of the biggest real-world examples of a **web crawler system**. The crawler here is called **Googlebot**, and its job is to explore billions of web pages, collect information, and store it so that when we search for something, results appear in seconds.
- In the background, Googlebot works continuously, just like multiple crawler instances. It starts with a list of known pages (seed URLs), such as websites submitted via Google Search Console or already discovered links. Each page is fetched using HTTP, and the HTML is downloaded for analysis.

3. Crawler Service to Kafka

The Crawler Service sends the crawled pages to Kafka, which acts as a queue. Kafka ensures that even if thousands of pages are being processed, none are lost or skipped.

Example:

If 100 crawlers fetch match pages from ESPN, Cricbuzz, and ICC, Kafka lines up all this data for further processing in order.

4. Crawler Service to Cassandra

At the same time, raw HTML pages are stored in **Cassandra** for backup. This allows reprocessing later if parsing fails or data needs updating.

Example:

The raw HTML for <https://cricbuzz.com/match/india-vs-england> is saved, so if new information like final scores appear, it can be re-parsed later.

5. Kafka to Parser Service

The Parser Service reads data from Kafka, cleans it, and extracts only what's useful like titles, keywords, text, and links.

Example:

From the HTML, it extracts

- Title: *India vs England 2nd ODI Highlights*
- Keywords: *Rohit Sharma, Ben Stokes, ODI, Cricket*

- Content: *India beat England by 5 wickets...*

6. Parser to Deduplication Service

The Deduplication Service removes repeated or near-duplicate pages. It creates hashes (fingerprints) for each document and compares them.

Example:

If ESPN and Cricbuzz post the same match summary, only one version is kept for indexing to save space and avoid duplicate results.

7. Deduplication to Indexer Service

The Indexer Service breaks text into words (tokenization) and creates an inverted index. This helps find which pages contain a particular word.

Example Table:

Keyword Pages

India [cricbuzz.com/match1, espn.com/match1]

England [espn.com/match1, icc.com/match2]

ODI [cricbuzz.com/match1, espn.com/match1, icc.com/match2]

So, when someone searches “India ODI,” we instantly find relevant pages.

8. Indexer to Ranking Service

The Ranking Service assigns scores to each page to decide which should appear first.

It uses three types of ranking:

1. **Static Ranking:** How many high-quality pages link to it. (like PageRank).
2. **Content Ranking:** Based on relevance to the search query.
3. **User Ranking:** Based on engagement means how long users stay, clicks, or bounce rate.

Example Ranking Table:

URL	Score	Reason
cricbuzz.com/match1	98	Fast site, recent, high traffic
espn.com/match1	95	Trusted site, slightly older
icc.com/match2	92	Fewer backlinks

So, when someone searches “India vs England highlights”, Cricbuzz appears first.

9. Ranking Service to Cassandra (Inverted Index)

After ranking, the data is stored again in Cassandra, but now as a structured inverted index with ranking scores.

This makes future queries faster.

10. Client to API Gateway

When users search (like “India vs England score”), the request hits the API Gateway. It manages authentication, throttling, and logging before passing the query inside.

11. API Gateway to Load Balancer

The Load Balancer distributes search queries among multiple instances of the Query Service so that no single server gets overloaded.

It also handles retries if a service goes

12. Load Balancer to Query Service

The Query Service first checks Redis to see if the result is already cached.

If not, it fetches matching pages from the inverted index in Cassandra and sends them to the Ranking Service again for real-time scoring.

13. Query Service to Ranking Service

The Ranking Service applies fresh ranking based on the user’s location, device, and timing.

Example: A user in India sees Cricbuzz and NDTV Sports first, while one in the UK sees ESPN and BBC Sport higher.

14. Query Service to Redis Cache

The final ranked results are cached in Redis.

If another user searches the same query soon after, results are fetched instantly from cache.

Example:

If 10,000 users search “India vs England live score,” Redis serves them instantly without re-querying the database.

Reference:

[Google Search System Design: A Complete Guide](#)

[System Design of Google Search Engine | Deep Notes](#)

Web Crawler Design: [Web Crawler - System Design Interview Question - YouTube](#)

293. HLD 21: File sharing service like Google drive

Google Drive is a file hosting system powered by Google. It offers cloud file storage and synchronization service, allowing users to store their data on remote servers. Besides storing the file on these servers, Google Drive will also synchronize their files across multiple devices that they use and share it with other users as requested. Dropbox, OneDrive and Google Photos are similar applications that handle file storage and sharing for massive amounts of files for millions of users.

1. Functional Requirements

1. Users to upload and download their files from any device.

2. Users should be able to share files or folders with other users.
3. It should also support automatic synchronization between devices, i.e., after updating a file on one device, it should get synchronized on all devices.

2. Non-Functional Requirements

1. The system should support storage of large files of up to 1GB each.
2. The system should be able to scale to an enormous number of users (Google Drive has over 1 billion active users as of July 2018).
3. The system should be able to handle a high number of reads and writes (around 100 million requests per day). Read to write ratio is comparable in this case.
4. Minimum possible network bandwidth should be utilized for file synchronization.
5. There should be minimum latency in file transfer.

Before going further, let's see how a large file will be handled to reduce bandwidth and latency.

When we upload a large file let's say 10MB file, to the cloud storage, and if it like a single file upload, then it would involve high latencies and bandwidth utilization.

To improve efficiency, it handles the upload in chunks, Means Each file is divided into small pieces of equal, predetermined size, say 2 MB. Metadata database will hold information, including name and order, for each of these file chunks so the original file can be recreated using the chunks when downloading or syncing

3. Components

Component	Description
API Gateway	Handles requests like upload, download, share, or delete.
Client	The client is the Google Drive app installed on your device (computer or phone)
Metadata Service	Stores information about files (name, type, owner, permissions, version).
Upload Service	Handles actual file chunks and upload.
Download Service	To download the chunk and update the local file system.
Sync Service	Keeps files updated across multiple devices.
Search Service	Helps find files by name, tags, or content.
Notification Service	Sends updates when files are shared or changed.
Load Balancer	Distributes user requests to different services.

Component	Description
Message Queue (Kafka)	Used for asynchronous tasks like syncing or notifications.
Cache (Redis)	Caches metadata and frequent file lookups for quick access.

4. Database

Google Drive deals with both structured and unstructured data.

We use **SQL** for metadata and **Object Storage (like Google Cloud Storage or Amazon S3)** for files.

Tables Example

User Table

user_id	name	email
101	Rahul Sharma	rahul@gmail.com

File Metadata Table

file_id	user_id	file_name	size	version	shared_with	created_at
1001	101	project.pdf	2MB	v3	[102,103]	2025-11-12

Folder Table

folder_id	user_id	folder_name	parent_id
200	101	Work Docs	root

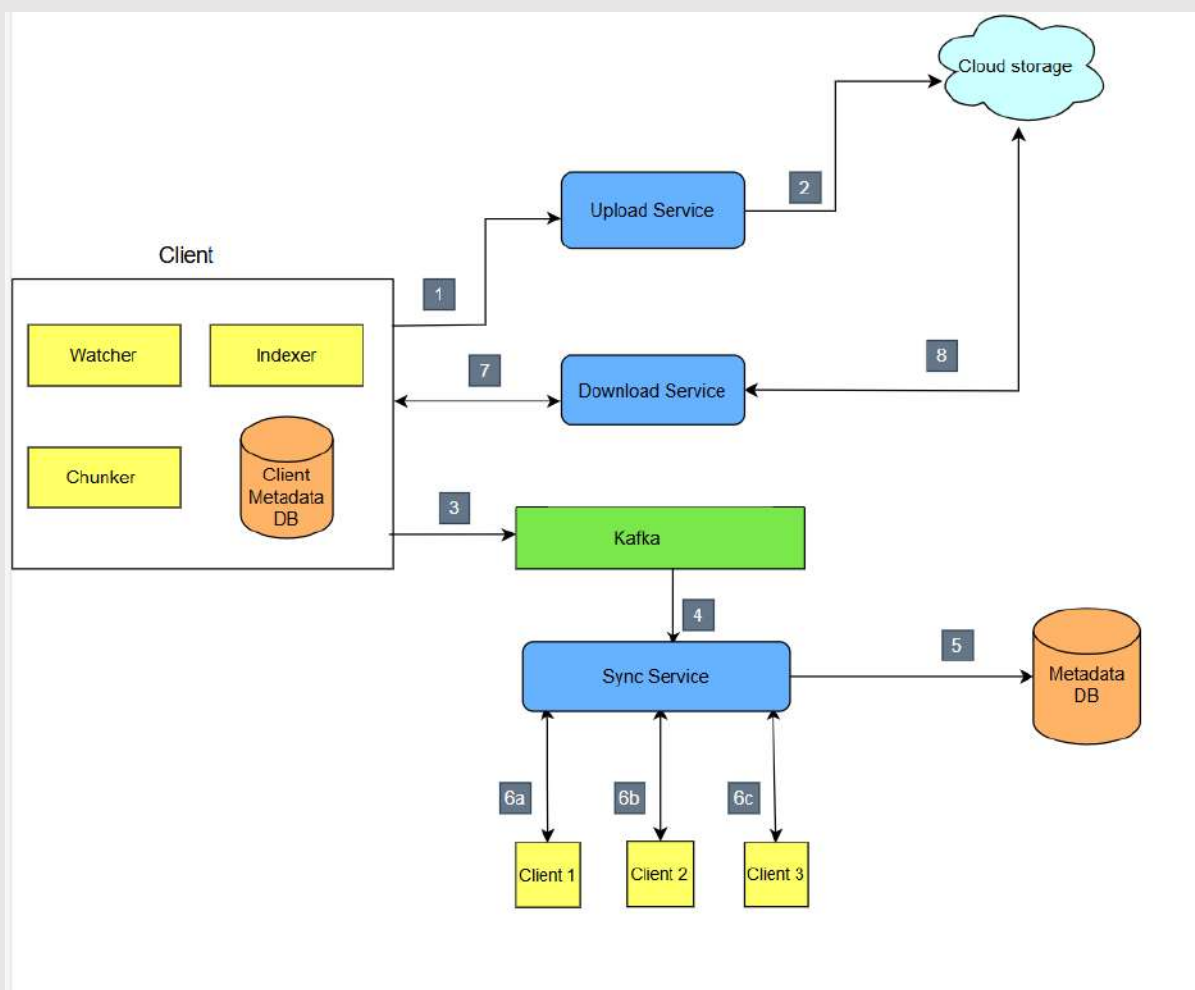
The actual files are split into **chunks** (like 4MB each) and stored in distributed file storage.

5. Design APIs

API	Method	Description	Request Example
/upload	POST	Uploads a file in chunks	{ "user_id":101, "file":"project.pdf" }
/download/{file_id}	GET	Downloads a file	file_id=1001
/share	POST	Share file with others	{ "file_id":1001, "shared_with":[102,103] }
/search	GET	Search by name or tag	?query=project

API	Method	Description	Request Example
<code>/delete/{file_id}</code>	DELETE	Deletes file metadata and chunks	<code>file_id=1001</code>
<code>/sync</code>	GET	Sync file changes	<code>{ "user_id":101 }</code>

6. Architecture



Let's see components from diagram

Client

The client is the Google Drive app installed on our device (computer or phone). The client actively monitors the workspace folders. Client will upload and download the files from cloud storage using Upload-Service and Download-Service.

Apart from that, there are multiple responsibilities that the client handles. To make it easier to understand and streamline performance, let's divide the desktop client into 4 subcomponents: Chunker, Watcher, Indexer and Client's Metadata DB.

Chunker:

- The chunker breaks a big file into smaller pieces called chunks. This helps upload files faster and saves data.
- If a file is edited, only the changed chunks are uploaded instead of the whole file.
- When we download, the chunker puts all the chunks back together in the right order to form the full file again.

Watcher

- The watcher keeps an eye on the folder where our files are stored.
- If we add, edit, or delete a file, it notices the change and informs the indexer.
- It also watches for any changes made to the files by other clients that it receives through the synchronization service.

Indexer

- The indexer receives changes made to the workspace folder via the watcher.
- It updates the local database with this information.
- After the changed chunks are uploaded to the cloud, the indexer sends a message to the synchronization service (through Kafka) to tell all connected devices about the new updates.

Client Metadata Database

- Other than the main remote metadata database, each device keeps a small local database that stores details about files, chunks, and versions.
- This helps us make changes even when offline.
- Once the device connects to the internet, these changes are sent to the cloud to stay updated.

Message Queue (Kafka)

- Kafka helps the client and synchronization service talk to each other smoothly.
- It manages all update messages, like which file changed or which chunk was uploaded.
- Kafka handles many messages at once and makes sure no update is lost, even if many users are working at the same time.

Synchronization Service: This is heart of the System.

The **synchronization service** keeps files and folders the same on all devices and in the cloud.

It works in two ways:

1. When a user makes a change on their computer (like editing or adding a file), that change is sent to the synchronization service using the message queue. The synchronization service updates the main cloud database with the new information.
2. When a change happens by other user (like someone shares or updates a file), the synchronization service sends that update to all connected devices through the message queue.

If a user's device was offline, it checks for missed updates once it's back online and downloads them.

In short, the synchronization service makes sure every device always has the latest version of the files.

Metadata and cloud storage

For file storage, we can use a cloud storage service, such as Amazon S3. The cloud storage service will simply store the files for us. From this cloud storage, our system can download or upload files as needed

And also, it will need to keep track of all the file chunks (including which file they belong to and their order), file versions, users and workspaces. All this metadata information can be stored in a metadata database. We can use a relational database such as MySQL,

Since multiple clients can make changes to the file at the same time, we need to ensure data consistency. MySQL might be a better choice in this case since it already supports ACID properties.

(Follow the numbers from diagram)

1. Client to Upload Service

When a file is added or changed, the Watcher detects it. The Chunker splits the file into smaller parts, and the Upload Service sends these chunks to the cloud.

2.Upload Service to Cloud Storage

The Upload Service uploads the chunks to cloud storage like Amazon S3. For example, if a user saves a 100MB file, it uploads it as small chunks for faster transfer.

3. Client to Kafka

Once the upload starts, the Indexer updates the Client Metadata DB and sends an update message through Kafka. This ensures other clients connected to the same account will know about the change.

4. Kafka to Sync Service

Kafka passes these update messages to the Sync Service. This helps synchronize files across all devices, making sure everyone sees the latest version of the file.

5. Sync Service to Metadata DB

The Sync Service stores or updates metadata in the main Metadata DB. For instance, it saves which chunks belong to which file and their version information.

6. 6a, 6b, 6c. Sync Service to Clients

The Sync Service then sends updates to all connected clients. If one user edits a shared file, others instantly receive the new version through this synchronization step.

7. Download Service to Client

When a client receives an update or needs a file, it uses the Download Service to fetch only the required chunks instead of downloading the whole file again.

8. Cloud Storage to Download Service

The Download Service fetches chunks from the Cloud Storage, reassembles them in the right order, and gives the updated file to the user.

Reference: [Architecting Cloud Storage Solutions: A Deep Dive into the System Design of Dropbox or Google Drive](#) | by Narendra Lakshmana Gowda | Medium

(11) [Google Drive Design](#) | LinkedIn

Video Reference: [System Design - Part 15 | Design Dropbox | Cloud Storage, Google Drive, OneDrive - YouTube](#)

294. **HLD 22:** Chat service like Facebook Messenger or WhatsApp

1. Functional Requirements

- Should support one-on-one chats
- Should support group chats
- Should have image, video and file-sharing capabilities
- The system should support message delivery acknowledgment, such as sent, delivered, and read
- Should show last seen time of users
- The system should be able to notify offline users of new messages once their status becomes online.

2. Non-Functional Requirements

- Low latency: Users should be able to receive messages with low latency.
- Consistency: Messages should be delivered in the order they were sent. Also, users must see the same chat history on all of their devices.
- Availability: The system should be highly available. However, consistency is more important than availability.
- Security: The system must be secure via end-to-end encryption. We need to ensure that only the communicating parties can see the content of messages. Nobody in between, not even we as service owners, should have access.
- Scalability: The system should be highly scalable to support an increasing number of users and messages per day.

3. Components

Component	Description
Load Balancer	Handles all incoming traffic and shares it across multiple servers. When millions of users connect at the same time, the load balancer keeps everything stable by spreading the work evenly.
Message Service	This service manages all chat messages. It sends, receives, and stores messages. For example, when we send "Hi" to a friend, the message service takes care of delivering it to that friend's device.
Group Service	Used when we chat in a group. It handles group creation, adding or removing people, and making sure one message reaches every group member without delay.

Component	Description
Asset Service	Manages all media like images, videos, and documents. When we send a photo, the asset service uploads it to cloud storage and sends a link instead of the full file.
Notification Service	Sends alerts when new messages arrive, especially if the user is offline. For example, when we get a message, a small “New Message” notification appears on the phone.
Message Queue (Kafka)	Acts as a middle layer to deliver messages reliably. It ensures messages are not lost even if one service is temporarily down. Every sent message passes through Kafka before reaching the receiver.
WebSocket	Keeps a live two-way connection open between the server and the app. This allows real-time chats without refreshing or repeatedly asking the server for updates.
WebSocket Manager	Keeps track of who is online and manages all active WebSocket connections. If a connection drops, it reconnects the user and continues message delivery.
Cloud Storage (Blob Storage)	Stores large files like images, videos, and voice notes. Instead of saving media directly on our phone, it's uploaded to cloud storage and accessed using secure links.
CDN (Content Delivery Network)	Speeds up downloads and uploads of media by keeping copies of files on servers close to where users live. This makes sending and viewing photos or videos much faster.

4. Database

To store the user, group and conversations data, we can use a SQL database like PostgreSQL.

For message data, it's preferred to use a NoSQL database like Cassandra due to high-write throughput.

For media files, an object store like AWS S3 provides scalable and secure storage.

For example, we can have tables like User, Messages, Groups and Group_Members.

users

user_id	name	phone_number	about	profile_picture_url
101	CodingLyf	+91-9876543210	Code. Eat. Sleep. Repeat.	https://cdn.app/profile101.jpg

messages

message_id	conversation_id	sender_id	message_type	message_content	media_url	sent_at	status
9007	5001	101	text	Bro, are we meeting today?	NULL	2025-11-13 20:15:42	delivered

groups

group_id	group_name	created_by	created_at	profile_picture_url	is_public
7001	Code Mates	101	2025-11-01 18:10:12	https://cdn.app/group7001.png	false

group_members

group_id	user_id	role	joined_at
7001	101	admin	2025-11-01 18:10:12

Before going to architecture let's discuss some key components

Why to choose WebSockets over HTTP? (Refer Question No. System design 30)

To understand why WebSockets are ideal for real-time messaging, let's examine other potential solutions and their limitations:

Polling: In polling, the client periodically sends HTTP requests to the server to check for new messages.

Drawbacks: Polling can be resource-intensive, especially with high polling frequency. Since the server responds with "no new messages" most times, this approach can add substantial overhead and waste server resources.

Long Polling: In long polling, the client holds an open connection with the server until a new message is available or a timeout occurs. When the server has new data, it responds, and the client immediately re-establishes the connection, restarting the process.

While this reduces the need for repeated requests as in standard polling, long polling has several limitations:

- **Connection Overhead:** Each message exchange requires re-establishing the connection, creating significant overhead and putting a heavy load on servers and network resources.
- **Resource Consumption:** The server must maintain many open connections, consuming memory and capacity even without active data exchange.
- **Latency Issues:** If the timeout is long, messages may be delayed. If short, frequent resets add the same overhead as standard polling.

Overall, long polling's connection and resource demands make it less suited for real-time chat applications.

WebSockets:

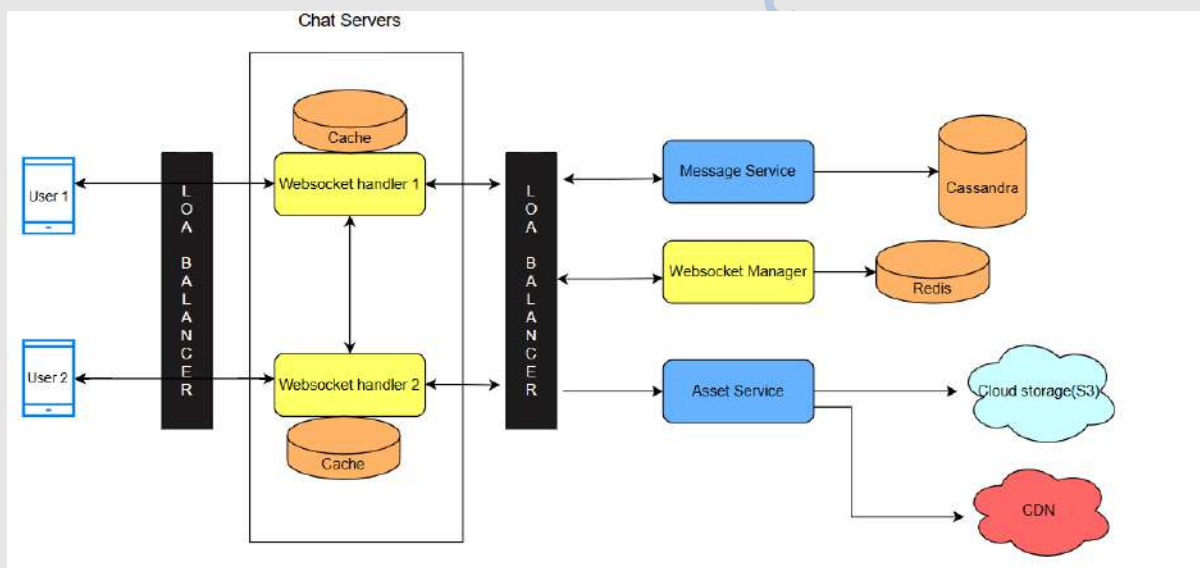
WebSockets on the other hand eliminate the need for repeated HTTP handshakes, headers, and responses, reducing overhead and enhancing performance.

The client and server establish a connection once, and this connection stays open for the entire chat session, enabling seamless data transfer.

This persistent connection makes WebSocket ideal for real-time communication, where both the client and server need to exchange data frequently and in a timely manner.

WebSocket Manager: One WebSocket server definitely won't be enough to handle billions of devices, so there should be enough servers to handle this. These services are responsible for providing a port to every online user. So, we need a **WebSocket manager**

The **Chat Servers** manage a large number of concurrent connections, facilitate real-time communication, and ensure that messages are delivered efficiently between users with minimal latency. They spread across the world.

6. Architecture: We will divide multiple sections.**1. Let's start with User to User (One to One) chat flow.**

Now User1 wants to send messages to User2.

Case1: Consider a happy path first, means **User1 and User2 are connected to internet**

So how does this work (Follow the Diagram):

1. User1 is connected to WebSocket-Handler-1 and User2 is connected to WebSocket-Handler-2
2. When User1 sends a message to User2, User1 communicates with the corresponding WebSocket-handler-1, saying send "Hello" message to User2.
3. WebSocket-Handler-1 now talks to WebSocket-Manager to know, to which WebSocket-Handler, User2 is connected.
4. **WebSocket Manager** which is a repository of information about which web socket handlers are connected to which users. It sits on top of a Redis which stores two types of information:

- Which user is connected to which web socket handler
- When a connection between a user and web socket handler breaks and they connect to another handler, through the web socket manager this information is updated into Redis.

5. At the same time, the WebSocket-Handler-1 call the message service and message stored in the database (Cassandra)

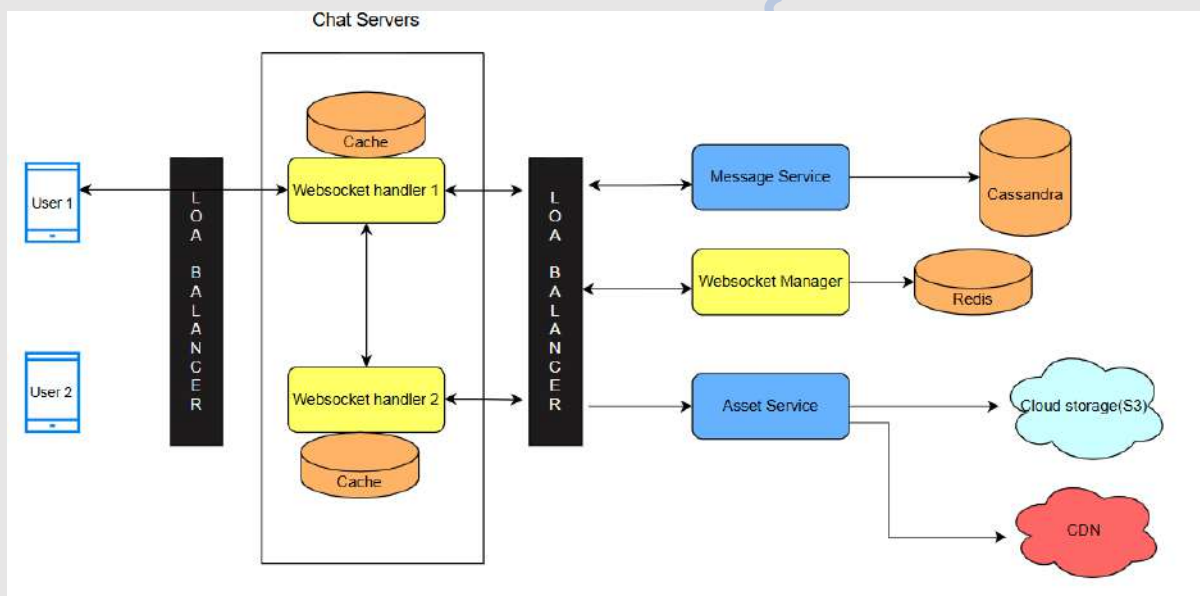
6. WebSocket-Handler-1 receives the information from WebSocket Manager, saying that User2 is connected to WebSocket-Handler-2

7. WebSocket-Handler-1 informs WebSocket-Handler-2 to deliver the message “hello” to User2 from User1

8. If User2 Received (double tick), or read (Blue tick),

- It will be informed to WebSocket-Handler-2
- WebSocket-Handler-2 inform to WebSocket-Handler-1 through WebSocket Manager.
- WebSocket-Handler-1 informs User1. So User1 can see either double tick or blue tick on his mobile

Case2: Now consider other case: **User2 is not connected to internet but User1 wants to send message**



From the above diagram, you can see that User2 is not connected to any WebSocket-Handler.

1. User1 is connected to WebSocket-Handler-1 and User2 is connected Websocket-Handler-2
2. When User1 send message to User2, User1 communicates with the corresponding Websocket-handler-1, saying send “Hello” message to User2.
3. Websocket- Handler-1 now talks to Websocket-Manger to know, to which Websocket-Handler, User2 is connected But User2 is not connected to any WebSocket-Handler.
4. In this case Websocket-Handler-1 gets information that, User2 is not connected. Websocket-Handler-1 informs User1 that message is not delivered. So User1 see single tick mark.
5. Websocket-Handler-1 call the message service and message stored in the database.

6. Now After sometime, User2 is to come to online, as soon as User2 is online, it connected to any WebSocket-Handler, lets say connected WebSocket-Handler-2.
7. Now WebSocket-Handler-2, calls MessageService to fetch all the pendings messages from DB and delivers to User2
8. User2 replies to any message, again the same process will kick start.

Case3: Now consider other case: User1 is not connected to internet but wants to send message

In this case, Local cache comes into place.

Let's say when User1 sent the message to User2 but User1 is not connected to internet. In this case, the message will be stored in a local database on the device. As soon as the device gets connected to the network, User1 will be connected WebSocket-Handler, it will pull all the messages from the local database and send them.

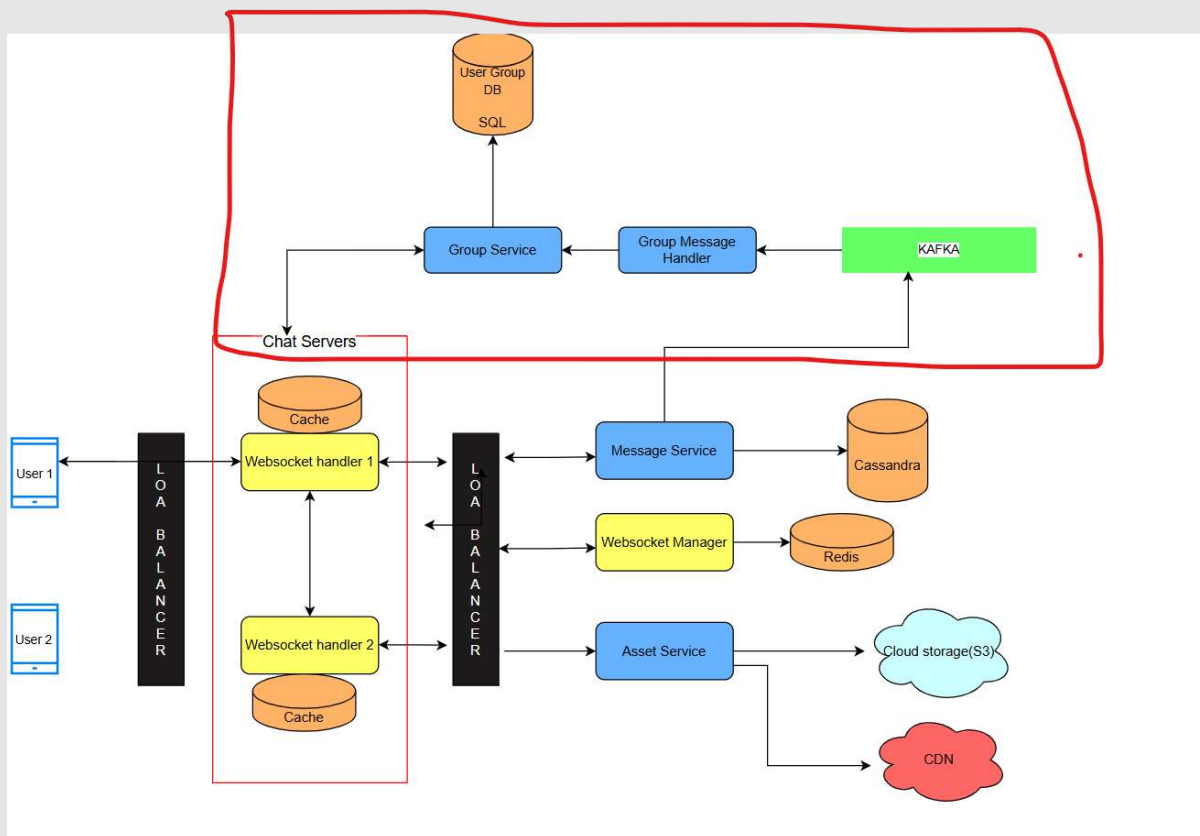
Let's see the Group Chat flow

What we discussed until now was how a personal chat works on a system like WhatsApp. But what about group chats?

Let's say User1 wants to send a message to Group1. Now, groups will behave a little differently from users

WebSocket servers don't keep track of groups because they only track active users. But in the group some users could be online and others could be offline. We need to think about other components for group messages, responsible for delivering messages to each user in a group.

- Group message handler
- Group message service
- The message queue ***Kafka**

**Flow:**

1. When User1 wants to send a message (Good morning, all) to Group1, WebSocket-Handler-1 invokes Message service, that User1 wants to send a message to Group1, and it gets stored in Cassandra as "Good morning, all".
2. Message service will now communicate with Kafka. Message gets saved in **Kafka** with an instruction that it has to be sent to Group1.
3. Now Kafka will interact with something known as **Group Message Handler**. The group message handler listens to Kafka and sends out all group messages.
4. There is something called **Group Service** which keeps track of all the members in all groups.
5. Group Service talks to SQL to get the all the list of users of that particular Group1.
6. Lastly, the Group Service follows the same process and handover the message to WebSocket-Handler the message to each user.

How Media will be handled?

Usually, the WebSocket servers are lightweight and don't support heavy logic such as handling the sending and receiving of media files. So, we need to add another service called **Asset Service**, which will be responsible for sending and receiving media files. The compressed and encrypted file will be sent to the asset service to store the file on blob storage.

Ideally, when content like this is sent out it will be compressed and encrypted at the device end and the encrypted content will be sent to the receiver. Even while receiving the content will be received in an encrypted format and decrypted on the device end. In this article let us just consider the compression step.

Suppose User1 is sending an image to User2. This will happen in two steps.

- User1 will upload the image to a server (Amazon s3) and get the image id
- Then it will send the image ID to User2 and User2 can download the image from the CDN.

We can add Analytics service to this System

Reference: [CodeKarle: WhatsApp System Design | Slack/FB Messenger System Design](#)

[System Design: Chat Application. Understanding the architecture and... | by Mariia Romaniuk | Medium](#)

295. **Real-World System Design Scenarios 1: If user clicks two times on pay now, how would you prevent double payments?**

Solution:

Let's say we are building a payment feature in an app. We add a button called "Pay Now." When we click it once, it should start the payment process. But sometimes, we might click it twice by mistake. If both clicks go through, the system may create two payment requests and charge the customer twice. That's a serious issue.

1. Disable the button after the first click

The first and simplest way is to disable the "Pay Now" button right after the first click. Once we click it, the button becomes inactive until the response comes back from the server.

2. Use an idempotency key on the backend

Disabling the button helps, but what if there's a network delay or the user refreshes and clicks again? That's why the backend must also protect the system.

Each time a payment starts, assign a unique ID to each API request so the backend recognizes duplicates. If another request with the same ID comes again, the backend should reject it, saying, "Payment already processed."

Example:

- When we click "Pay Now," the frontend creates a unique ID like txn_12345.
- This ID is sent to the backend.
- The backend checks if txn_12345 already exists in the database.
- If it's new, it continues the payment.
- If it already exists, it stops the second payment immediately.

This ensures that even if two payment requests come together, only one will succeed.

3. Add a lock or flag in the database

Prevent duplicate entries at the database level

For example, imagine our table has a column called order_id, and we mark it as unique. When the first order with order_id = 101 is saved, it gets added successfully. But if another request tries to save again with the same order_id = 101, the database immediately rejects it. This way, even if the same request is sent twice, only one record is stored.

296. Real-World System Design Scenarios 2: How Flipkart survives Big Billion Days without crashing?

Solution:

1. Auto-scaling Servers

When traffic increases, more servers are added automatically. When traffic drops, extra servers are removed. This way, the system adjusts itself based on the load and keeps running smoothly during peak times like Big Billion Days.

2. Load Balancers

Load balancers act like guards who direct traffic to the least busy server. If one server gets too crowded, new users are sent to another. This helps every server share the work equally and prevents slowdowns or crashes.

3. Caching

Caching keeps frequently accessed data ready in memory, so it loads instantly. For example, when thousands check the same product, it's served directly from cache instead of the database. This makes pages load faster and reduces pressure on servers.

4. Database Sharding

Instead of storing everything in one large database, data is split into smaller parts called shards. Each shard handles a portion of the data, like storing different categories in separate rooms. This improves speed and avoids database overload.

5. Message Queues

When too many tasks come at once, queues help manage them in order. Each request, like placing an order or making a payment, waits for its turn. Systems like Kafka and RabbitMQ make sure everything is processed smoothly and safely.

5. Rate Limiting

Rate limiting controls how many requests are accepted at a time. For example, if one phone model has limited stock, only a fixed number of orders are allowed per second. This prevents overselling and keeps the platform stable.

297. Real-World System Design Scenarios 3: How a Food Delivery System like Zomato handles millions of orders?

Solution:

Zomato looks simple from outside, but behind it runs a system that connects restaurants, delivery partners, and customers in real time. Here are the key parts that keep it running smoothly even during peak hours.

1. Microservices Architecture

Each feature works like a small, independent service. For example, one handles restaurant details, another handles delivery, and another manages payments. If one service fails, others keep running. This makes the system reliable and easier to scale.

2. Load Balancers

Millions open the app at once, but the load balancer divides that traffic across many servers. It sends each request to the least busy server so no single one gets overloaded. This helps Zomato stay fast and responsive during rush hours.

3. Caching

Popular data like restaurant menus or delivery zones is stored in memory using tools like Redis or CDN. This avoids fetching the same data again and again from the database, making the app load pages instantly.

4. Real-Time Tracking with WebSockets

When we track our food on the map, WebSockets keep the connection open between the app and the server. It continuously sends updates about the delivery partner's location without refreshing the app every time.

5. Message Queues

When too many orders come at once, message queues like Kafka manage them in a proper sequence. Orders are added to a queue and processed step by step, so nothing gets missed or duplicated.

6. Auto Scaling

During lunch or dinner hours, order volume jumps suddenly. Auto scaling detects this and adds more servers to handle the extra load. Once traffic reduces, extra servers are removed to save cost.

7. Database Sharding

The database is divided into smaller parts. For example, one shard stores restaurant data, another stores user data, and another keeps order details. This helps reduce read and write time and keeps performance stable.

298. Real-World System Design Scenarios 4: How a Food Delivery System like Zomato make sure order is accepted by only one rider?

Solution:

Zomato ensures an order is accepted by only one rider by using a real-time, notification-based system where multiple available riders are notified, but only the first one to accept the order is assigned the delivery. The system works by sending a notification to a batch of nearby, available riders simultaneously. The order is then automatically assigned to the first rider who accepts it, and all other notifications for that order are cancelled. This "first come, first served" approach prevents multiple riders from accepting the same delivery.

1. Order status management

Every order has a clear status like "Pending," "Accepted," or "Delivered." When the order is first created, its status is marked as "Pending." Once a rider accepts it, the status instantly changes to "Accepted," so no other rider can take it.

2. Optimistic locking or transactions

Some systems use a version number with each order. When a rider tries to accept an order, the system checks the version. If it has already changed, it means another rider accepted it first, and the new request is ignored.

3.Real-time updates with Pub/Sub or WebSocket

Once an order is accepted, the system instantly notifies all nearby riders that the order is no longer available. This prevents them from seeing or trying to accept it again.

299. Real-World System Design Scenarios 5: If two users are booking at the same time, how BookMyShow handle this??

Solution:

BookMyShow prevents double-booking by using a "first-come, first-served" system with temporary seat locks. When a user selects seats, the system locks them for a short period, typically 5-10 minutes, to give the user time to complete the payment. If the payment fails or the time expires, the lock is released and the seats become available again for others. This process is also called **Pessimistic Locking**, where a resource is locked before use to prevent others from accessing it simultaneously.

How it works

- **Seat selection:** When a user selects seats, the system communicates with the theatre's database to temporarily block those seats.
- **Temporary lock:** The seats are locked for a fixed time (e.g., 10 minutes) to allow the user to complete the payment.
- **Payment confirmation:** If the payment is successful, the booking is confirmed, and the ticket is generated with a unique ID.

Reference: (14) [Concurrency booking handling in BookMyShow application.](#) | [LinkedIn](#)

300. Real-World System Design Scenarios 6: When two messages sent at same time in WhatsApp, how it maintains the order??

Solution:

In chat systems like WhatsApp, timing matters a lot. When two messages are sent at the same moment, they still appear in the correct order. This is handled carefully using message sequencing and server timestamps.

1. Message ID generation

Each message gets a unique ID when it's created. This ID often includes details like the sender's device time or sequence number. Even if two messages are sent almost together, their IDs will still be different, which helps maintain order.

2. Server timestamp

When messages reach the WhatsApp server, each one gets a timestamp. The server uses this timestamp to arrange them in the correct order before delivering them to the receiver.

3. Sequential delivery

The server sends messages in the same order they were received. If two come at the same millisecond, the message with the smaller ID or earlier timestamp is sent first.

4. Client-side ordering

On the user's phone, the chat app also checks message timestamps. If a message arrives slightly late because of a slow network, the app reorders it based on the timestamp so that the chat looks consistent for both sides.

Reference: [Codemia](#) | [How does the messenger maintain the sequencing of the messages during chat and when users log in again?](#)

301. Real-World System Design Scenarios 7: When you delete the message in group, how does it delete in every member in group?

Solution:

When a message is deleted in a group, it disappears for everyone, not just for the sender. This happens because of how WhatsApp syncs message states across all connected devices.

1. Delete request creation

When we choose to delete a message for everyone, our app sends a delete request to the WhatsApp server along with the message ID that needs to be removed.

2. Server validation

The server first checks if the delete action is allowed. It verifies that it's within the allowed time limit for deletion.

3. Broadcast delete event

Once verified, the server sends a delete command to every member in that group. This command includes the same message ID that must be deleted from their devices.

4. Client-side deletion

Each user's app receives the delete command, finds the message by its ID, and replaces it with a note like "This message was deleted." The actual message content is then removed from the local chat storage.

5. Sync for offline users

If some members are offline, the delete event is stored on the server. When they come online, their app receives the pending delete command and removes the message immediately.

302. Real-World System Design Scenarios 8: Why WhatsApp took so much time to introduce the edit feature?

Solution:

WhatsApp took a long time to introduce the edit feature because it had to solve a few tough problems to keep privacy, security, and message consistency intact across millions of devices. Here's what made it tricky.

1. End-to-end encryption

Every WhatsApp message is encrypted, which means even WhatsApp can't read it. Editing a message meant finding a way to securely replace the old encrypted content with the new one without breaking that encryption or exposing the message to the server.

2. Message integrity

Once a message is sent, it's delivered to multiple devices in different locations. Allowing edits meant WhatsApp had to make sure the change reflects exactly the same way for everyone, even if some users were offline. Maintaining this consistency took careful design.

3. Security

If edits were allowed without limits, users could misuse the feature by changing the meaning of old messages after they were read. WhatsApp needed to build rules, like allowing edits only within a short time window, to prevent such misuse.

4. Real-time sync across devices

With multi-device support, one user can be logged in on several phones or computers. An edit made on one device has to appear instantly and correctly on all linked devices, which required major architectural updates.

303. Real-World System Design Scenarios 9: How WhatsApp show typing... when user is actually typing the message??

Solution:

When we see “typing...” below a contact’s name, it’s not magic. It’s a simple exchange of small signals handled in real time through a **WebSocket connection**.

1. Detecting the typing event

When a user starts typing in the chat box, the app detects this action and marks it as a “typing event.” It doesn’t send any actual message, just a signal that typing has started.

2. Sending signal through WebSocket

This “typing” signal is sent instantly to the WhatsApp server using a WebSocket connection, which stays open between the app and the server. Because the connection is always active, the signal reaches almost immediately.

3. Server forwards to the receiver

The server receives the typing event and forwards it to the recipient’s device through the same WebSocket connection.

4. Receiver displays typing status

Once the receiver’s app gets the signal, it shows “typing...” under the sender’s name. This status stays visible until a stop signal is received.

304. Real-World System Design Scenarios 10: How WhatsApp Delivers Messages Instantly to 2 Billion+ Users?

Solution:

WhatsApp feels instant because it’s built on a smart, real-time communication system that can handle billions of messages every single day. Here’s how it works behind the scenes.

1. WebSocket or TCP Connection

WhatsApp keeps a **WebSocket connection** open between our app and its server. This live connection acts like a walkie-talkie line, always ready to send or receive messages instantly. The moment we type and hit send, the message travels through this active link without waiting for a new request.

2. Scalability

Billions of users send messages at the same time. To handle that load, WhatsApp uses horizontal scaling, which means adding more servers whenever the traffic increases. It’s like opening more gates when a crowd suddenly grows, so no one has to wait in line.

3. Message Brokers (Kafka or RabbitMQ)

Inside WhatsApp’s backend, there are multiple services handling billions of messages every second. To manage that load, **message brokers** like Kafka are used *between internal services*.

They help keep messages ordered, retry failed deliveries, and balance the flow across many servers.

4. CDN for Media Files

When we send photos or videos, WhatsApp uses Content Delivery Networks (CDNs). These are servers placed in different regions to store media closer to users. It's like getting coffee from the nearest café instead of traveling across the city, which makes downloads faster.

5. Retry and Offline Handling

If the receiver is offline, the message doesn't disappear. WhatsApp stores it securely on the server and keeps trying until the receiver comes online. It's like leaving a sticky note on someone's desk that stays there until they read it.

Reference: [How WhatsApp Delivers Messages Instantly to 2 Billion+ Users: A Deep Dive into System Design ⚡ | by Bhavesh Vaswani | Oct, 2025 | Medium](#)

305. **Real-World System Design Scenarios 11: How messages stay private even WhatsApp can't read them?**

Solution:

WhatsApp uses end-to-end encryption to make sure that messages stay private between two people. It means no one else, not even WhatsApp, can read what is sent. The message is protected from the moment it leaves one phone until it reaches the other.

The system is built using the **Signal Protocol**, a method created by Open Whisper Systems. It uses a mix of two types of encryptions: one to lock and unlock messages, and another to exchange keys safely between devices.

Let's understand how it works in a simple way.

1. Key creation

When we install WhatsApp, our device creates two keys - one public and one private.

The public key is stored safely on WhatsApp's server, so others can use it to start a chat with us.

The private key never leaves our phone. It stays hidden and is used to unlock messages that we receive.

2. Starting a chat

When a new chat begins, our phone creates a small secret key called a *session key*. This key will be used only for that chat.

3. Sending the session key

Before sending, our phone locks this session key using the receiver's public key. Now only the receiver's private key can unlock it.

The encrypted session key is sent through WhatsApp's server to the receiver's phone.

4. Unlocking on the receiver's side

The receiver's phone uses its private key to open the session key.

Now both phones have the same secret session key, and the actual chatting begins.

5. Encrypting messages

Every message we send in that chat is locked using this session key. When the other person receives it, their phone uses the same key to open and read it.

Even if the message travels through WhatsApp's servers, it stays locked. WhatsApp cannot open it because it does not have the key.

6. Forward secrecy

WhatsApp adds another layer of safety by creating a new session key for every new message. So, even if one key is stolen somehow, it cannot be used to read older or future messages.

7. End-to-end protection

The full process ensures that messages are encrypted on the sender's phone and only decrypted on the receiver's phone. No one in between can see the real content.

For example, if we send "Good Morning" to a friend, our phone first turns it into an unreadable code before it leaves the device. WhatsApp's server just passes it forward without knowing what it contains. When the message reaches the friend's phone, it turns back into "Good Morning."

That is how WhatsApp keeps every message private using end-to-end encryption.

Reference: [Engineering Connectivity: Navigating the System Design of WhatsApp - DEV Community](#)

306. **Real-World System Design Scenarios 12:** How does Hotstar scale without crashing when a big match is happening like Ind vs Pak?

Solution:

When millions start watching the same match at the same moment, the load on servers shoots up within seconds. Handling that without crashing needs smart scaling, distributed systems, and fast delivery networks. Here's how Hotstar manages it.

1. **Content Delivery Network (CDN)**

Hotstar doesn't stream the match from one central server. Instead, it uses CDNs, which are groups of servers placed in different regions. Each viewer connects to the nearest server. This reduces delay and avoids overloading a single location.

2. **Adaptive Bitrate Streaming**

Not every user has the same internet speed. Hotstar adjusts the video quality automatically based on network strength. If a user's connection drops, it switches to a lower quality instead of buffering. This keeps playback smooth for everyone.

3. **Auto Scaling on Cloud**

When a big event starts, millions of users join suddenly. Hotstar's cloud setup automatically adds more servers as the traffic grows. When the traffic goes down after the match, the extra servers are removed. This helps handle sudden spikes efficiently.

4. **Load Balancers**

Load balancers sit in front of servers and divide user requests evenly. If one server gets busy, new users are sent to another. This keeps the entire system stable and prevents one server from crashing under pressure.

5. **Edge Caching**

Hotstar keeps popular parts of the stream like the live video chunks cached at the edge servers, which are closest to users. This reduces the time taken to fetch data from the main servers and speeds up video delivery.

By using CDNs, auto scaling, and load balancers together, Hotstar ensures that even during record-breaking matches, millions can stream at once without interruptions.

Reference: [How JioHotstar handled 60 Cr views during Ind vs Pak match\(Feat. my wifey's reactions!\) | by Krishnendu Bhowmick | Medium](#)

307. Real-World System Design Scenarios 13: How video streaming apps like Hotstar maintains logged in devices?

Solution:

When we log in to a video streaming app like Hotstar, it remembers our account across all our devices , phone, tablet, and TV. To make this possible, the app uses something called a **session management system**.

Each time we log in, the authentication service creates a **session token** for that specific device. This token acts like a temporary pass that proves the user is verified. The token is then saved in the **Session Database**, along with details such as device type, login time, and IP address.

For example, if we log in on a mobile and a smart TV, the system creates two different session records. Each one has its own token. When we open the app again, the token is verified by the authentication service before showing our watchlist or continuing playback.

If the same user tries to log in on too many devices, Hotstar checks the number of active sessions from the session database. If the limit is crossed, the system may log out the oldest session. This helps control account sharing and improves security.

When we manually log out or change the password, all session tokens are deleted from the database so that no device can access the account again without logging in.

308. Real-World System Design Scenarios 14: How apps like Hotstar / Amazon keep us logged in without asking again?

Solution:

When we log in to Hotstar for the first time, the app creates something called a **session token**. This is a small piece of data that proves “this device is allowed to access the account.”

The important part is that Hotstar stores this token safely both:

- on the device
- in the backend **Session Database**

When we open the app the next day, we do **not** log in again.

Instead, the app sends the same token to the server and asks, “Is this token still valid?”

If the backend finds a matching session in the database, it allows access immediately.

No password is needed again.

To make this possible, these apps use two tokens: an **access token** and a **refresh token**.

The **access token** works like a short-term entry pass. It lets the app load our profile, our watchlist, and the shows we are watching. But this pass expires very quickly, usually in a few minutes. This is done for safety. Even if someone steals it, they cannot misuse it for long.

The **refresh token**, on the other hand, lasts much longer. It stays valid for weeks or even months. This token is kept safely on the device and is not used for every request. It is only used for one job: getting a new access token when the old one expires.

Here's what happens:

- When the access token expires, the app does not show a login screen.
- It simply tries to use the access token, gets a response saying "expired," and quietly sends the refresh token to the server.
- The server checks the refresh token and, if it is still valid, gives the app a brand-new access token.
- This happens in the background within a second. The user never even notices.

This cycle repeats again and again.

The access token expires.

The app uses the refresh token.

The server gives a new access token.

As long as the refresh token is valid, we stay logged in without doing anything. Only when the refresh token finally expires, or when we change our password or log out, the app asks us to sign in again.

309. **Real-World System Design Scenarios 15: How YouTube handles millions of Uploads?**

Solution:

Every minute, thousands of hours of video are uploaded to YouTube. Managing that massive amount of data while keeping uploads smooth for everyone takes careful system design and distributed infrastructure.

1. Upload Servers and Load Balancers

When we upload a video, the request doesn't go to one single server. Load balancers distribute uploads across many servers based on current load and location. This helps handle millions of uploads at the same time without overloading any single server.

2. Chunked Uploading

Videos are not sent as one big file. They are broken into small chunks and uploaded piece by piece. If the internet connection breaks, uploading can continue from the last completed chunk instead of starting over. This makes the process more reliable.

3. Temporary Storage and Validation

Once a video is uploaded, it first lands in temporary storage. The system checks for file integrity, supported format, and any upload errors before sending it further for processing. This ensures only valid files move forward.

4. Video Processing and Transcoding

After validation, the video is converted into different formats and resolutions like 144p, 480p, 720p, and 1080p. This step is called transcoding. It allows users to watch videos smoothly on any device, regardless of their internet speed.

5. Distributed Storage Systems

Processed videos are stored across many data centers worldwide. Each copy is kept in multiple locations for backup and faster access. When a viewer plays a video, YouTube serves it from the closest data center.

6. Content Delivery Network (CDN)

Once stored, videos are pushed to edge servers in different regions using CDNs. This helps deliver videos quickly to users without depending on a single server.

7. Background Queues and Scheduling

Uploads, validations, and processing tasks are handled through background queues. This means each video goes through a sequence of steps in order, keeping the system stable even when millions of uploads happen together.

By combining load balancing, chunked uploads, distributed storage, and efficient video processing, YouTube handles millions of uploads every day while keeping the experience fast and reliable for everyone.

310. Real-World System Design Scenarios 16: How Google search bar shows instant suggestions when you are typing?

Solution:

When we start typing in the Google search bar, suggestions appear instantly even before we finish the word. This feature feels fast because Google combines real-time processing, caching, and smart prediction models behind the scenes.

1. WebSocket or AJAX Requests

As soon as we type a few letters, the browser sends a small request to Google's servers through a WebSocket connection or quick AJAX call. This request includes the partial word we've typed so far, like "cric" when we mean "cricket."

2. Autosuggest Service

The request goes to a special backend service that handles search suggestions. This service looks at popular searches, recent trends, and our previous search history to predict what we might be typing.

3. Cache and Preloaded Results

Google stores common suggestions in memory using caching. So when millions of people type something like "India vs Pakistan," the results are fetched directly from cache instead of the database, which makes suggestions appear instantly.

4. Prefix Matching and Ranking

The system compares our input with stored search terms that begin with the same letters. For example, typing "cric" matches "cricket," "cricket score," and "cricket world cup." These results are then ranked based on popularity and relevance.

5. Real-Time Model Updates

Google's machine learning models constantly learn from new searches happening around the world. If a new trend starts, it quickly becomes part of the suggestion list within minutes.

311. Real-World System Design Scenarios 17: How Uber updates the location without refreshing the app?

Solution:

When we book a ride on Uber, we can see the driver's car moving smoothly on the map in real time. The app doesn't reload or ask for updates again and again. This happens because Uber uses its own real-time communication system called **RAMEN** (Realtime Asynchronous Messaging Network).

1. Persistent Connection with RAMEN

Uber keeps a live connection between the app and its backend using **RAMEN**. This system is built on top of **gRPC**. It allows the server to send updates instantly without waiting for the app to make a new request. This connection stays open as long as the app is active.

2. Driver Location Updates

The driver's app continuously collects GPS data and sends the current latitude and longitude to the backend every few seconds. These updates flow through the same **RAMEN** connection, which forwards them immediately to the rider's app.

3. Publish-Subscribe Model

RAMEN works using a publish-subscribe pattern. The driver's app publishes location updates, and the rider's app subscribes to them. Whenever a new location is received, the rider's app gets the update instantly and redraws the car's position on the map.

4. Real-Time Data Streaming

Behind the scenes, Uber uses tools like **Apache Kafka** to handle millions of these location updates every second. Kafka manages the flow of data between the driver's and rider's apps, ensuring that messages arrive in the correct order and none are lost.

312. Real-World System Design Scenarios 18: How YouTube handles millions of User interactions?

Solution:

Every second, millions of people watch, like, comment, and upload videos on YouTube. Handling all these actions smoothly without slowing down or crashing requires a combination of powerful infrastructure and smart design.

1. Content Delivery Network (CDN)

Videos are stored across servers around the world. When we click play, YouTube picks the nearest server to deliver the video. This reduces buffering and keeps playback fast, even when millions are watching.

2. Load Balancers

When a huge number of users open YouTube at once, load balancers divide the traffic among many servers. This prevents any single server from getting overloaded and ensures everyone gets quick responses.

3. Microservices Architecture

YouTube's system is broken into smaller services like video upload, playback, comments, recommendations, and analytics. Each service works independently and can scale up or down based on how much it's being used.

4. Database Sharding

All user data and videos are split across multiple databases. One database might handle comments,

another might store likes, and another manages views. This keeps performance stable and avoids slowdowns when billions of records are being updated.

5. **Caching**

YouTube keeps frequently accessed data, like trending videos or thumbnails, in memory for quick access. This reduces the load on the main databases and makes browsing faster for everyone.

6. **Event Driven Architecture**

When we like or comment on a video, it doesn't directly hit the main database. Instead, it first goes into a queue like Kafka. The system processes these updates in order, keeping everything reliable even under heavy load.

313. **Real-World System Design Scenarios 19: How Uber finds driver near to your location?**

Solution:

When we request a ride, Uber instantly shows nearby drivers and assigns one within seconds. This happens through a mix of real-time location tracking, smart data indexing, and fast backend communication.

1. **Real-time Location Sharing**

Both riders and drivers continuously share their GPS coordinates with Uber's backend. The driver's app sends updated location data every few seconds. These updates travel through RAMEN, Uber's real-time messaging system, which keeps the backend aware of every driver's position at all times.

2. **H3 Geospatial Indexing System**

Uber uses a system called **H3**, built by Uber itself. It divides the entire world into small hexagonal cells. Each cell represents a specific area on the map. When a ride request comes in, the system checks which cell the rider is in and looks for available drivers in nearby cells.

3. **Efficient Nearest Driver Search**

Once the rider's location is known, the backend quickly runs a search within that hexagonal grid to find available drivers. The search expands outward from the rider's cell until a match is found. This makes finding nearby drivers fast and efficient, even in crowded cities.

4. **Matching Service**

After finding a few possible drivers, Uber's matching service selects the best one based on distance, estimated time of arrival, and driver status. The chosen driver receives the request first through RAMEN, which delivers it instantly without delay.

5. **Real-time Updates and Confirmation**

Once a driver accepts the ride, both apps (driver and rider) get updates in real time. The route, estimated time, and location are shared immediately through the same RAMEN connection.

If you want to learn how H3 works, refer this: [How Does Uber Find Nearby Drivers](#)

314. **Real-World System Design Scenarios 20: How apps like Amazon or Instagram manages Billion of records in Database?**

Solution:

Apps like Amazon and Instagram deal with an unbelievable amount of data every day. Millions of people browse products, upload photos, leave comments, or make purchases. Storing all this in a single

database won't work. It would be slow, crowded, and difficult to scale. So these apps break the problem into smaller, smarter pieces.

The first step is **sharding**. Instead of keeping all data in one giant place, the system splits it into many smaller databases. Each shard holds only a portion of the data. For example, Instagram might store users whose names start with A to M in one shard, and N to Z in another. This way, no single database carries the entire load, and everything stays fast.

The second idea is **replication**. Every important database has multiple copies. If one copy goes down, another takes over instantly. This keeps the app running even when hardware fails. Amazon does this to make sure product listings, orders, and customer data are always available.

These apps also use **NoSQL databases** like Cassandra, DynamoDB, or MongoDB for data that grows very fast. NoSQL databases handle huge volumes of reads and writes without slowing down. For example, Instagram uses Cassandra to store likes and comments since millions of people tap "like" every second.

Search features also rely on special storage. Amazon uses tools like Elasticsearch so that searching for "blue shoes" or "wireless headphones" feels instant, even though the catalogue has millions of items.

315. **Real-World System Design Scenarios 21: How does Elastics search work efficiently in just milli seconds?**

Elasticsearch feels fast because it doesn't search through text like we do. It prepares the data in a way that makes searching almost instant. When we store something in Elasticsearch, it breaks the text into small pieces called *tokens*. For example, "Best budget phone under 15000" becomes words like *best*, *budget*, *phone*, *15000*. These words are then stored in a special structure called an **inverted index**.

The inverted index works like a dictionary. Instead of listing all words in the document, it lists the words first and then the documents that contain them. So, when we search for "budget phone," Elasticsearch doesn't read every record. It simply looks up the words *budget* and *phone* in its index and finds the exact documents instantly. This is the main reason behind its speed.

Example Documents

Document ID Content

D1	Best budget phone for students
D2	New phone with great battery life
D3	Budget laptop and budget phone deals

Inverted Index (word and list of documents)

Word Documents Containing the Word**best** D1**budget** D1, D3**phone** D1, D2, D3**students** D1**new** D2**battery** D2**life** D2**laptop** D3**deals** D3**How Search Works Using This Table**

If someone searches for: **“budget phone”**

Elasticsearch looks up:

- **budget in D1, D3**
- **phone in D1, D2, D3**

The intersecting documents are: **D1 and D3**

So Elasticsearch instantly returns the matching documents without scanning anything else.

Elasticsearch also spreads data across many machines. Each machine stores a slice of the data called a shard. When we search, all shards work at the same time, and each one returns its results. Elasticsearch then merges everything and gives the final answer. This parallel processing saves a lot of time.

It also keeps frequently searched data in memory. RAM access is much faster than disk access, so repeated queries come back even quicker. For example, if many people search for “iPhone 15,” Elasticsearch can respond in milliseconds because it already has related index data loaded in memory.

Solid Principles & Design Patterns

316. Solid Principles 1: What are SOLID Principles?

In the real-world project, first the project may start with single feature. We can write the code in 1 or 2 classes. It works fine. But later new changes keep coming.

Think of a simple **real time situation**.

In an ecommerce project, we start with only card payments. Later the client asks for UPI, then wallet, then COD. If the payment code is tightly mixed inside one big class, adding a new method becomes a messy task and it became hard to maintain the code.

This is where SOLID principles comes into picture.

SOLID Principles gives a simple way to keep code clean. When we follow these principles, the project becomes easier to change, easier to test and easier to understand. Each part of the code has a clear role, so the project feels organised

Here is another example.

Think of a system that sends receipts. At first it only sends email. Later the team wants to add SMS. If everything sits inside one single class, adding SMS becomes tricky. One small change can affect other parts. But if the code follows SOLID, the email logic and SMS logic stay in separate classes. We just add a new class for SMS and connect it. Nothing else needs to be changed inside the existing code.

This is the real benefit of SOLID. It keeps the project clean and easy to grow, even when new features keep coming in.

SOLID is an acronym that stands for:

- Single Responsibility Principle
- Open/Closed Principle
- Liskov Substitution Principle
- Interface Segregation Principle
- Dependency Inversion Principle

Let's see each one of them with real time example from next question.

317. Solid Principles 2: What is Single Responsibility Principle?

This principle says that a class should focus on one task. When a class handles many different things, it becomes hard to read, hard to change and easy to break.

Let's look at a simple real-world problem.

We build a small feature that creates a user account. At first it only needs to save the user. Later the team asks for email sending. Then logging. Then validation. All of this is written in the same class.

After some time, even a small change becomes risky because the class now does too many things.

Problematic code (On High level)

```
class UserService {
    public void createUser(User user) {
        // validate
        if (user.getName().isEmpty()) {
            throw new RuntimeException("Name required");
        }

        // save to database
        System.out.println("Saving user to DB");

        // send welcome email
        System.out.println("Sending email");

        // log activity
        System.out.println("User created");
    }
}
```

This class is doing four different jobs: validation, saving, email sending and logging

If any part changes, we touch this file again and again. Over time it becomes messy and hard to trust.

Now let's see, how Single Responsibility Principle solves the problem

SRP guides us to split the responsibilities. Each class does one clear job. This keeps the main feature simple and safe.

```
class UserValidator {
    public void validate(User user) {
        if (user.getName().isEmpty()) {
            throw new RuntimeException("Name required");
        }
    }
}

class UserRepository {
    public void save(User user) {
        System.out.println("Saving user to DB");
    }
}

class EmailService {
    public void sendWelcome(User user) {
        System.out.println("Sending email");
    }
}
```

```

    }
}

class ActivityLogger {
    public void log(String message) {
        System.out.println(message);
    }
}

class UserService {
    private UserValidator validator = new UserValidator();
    private UserRepository repo = new UserRepository();
    private EmailService emailService = new EmailService();
    private ActivityLogger logger = new ActivityLogger();

    public void createUser(User user) {
        validator.validate(user);
        repo.save(user);
        emailService.sendWelcome(user);
        logger.log("User created");
    }
}

```

- Now the code is clearer.
- Each class does one job.
- If the email format changes, only the EmailService changes.
- If database storage changes, only the repository changes.
- The main flow stays clean.

318. Solid Principles 3: What is Open/Closed Principle?

It says, OPEN for **Extension** and CLOSE for **Modification**.

“Open to extension” means that we should design our classes so that new functionality can be added as new requirements. “Closed for modification” means that once we have written a class, we shouldn’t have to change its code to add new functionality.

Once we created a class, the Open-Closed Principle encourages expanding or introducing new features through inheritance or interfaces.

This way, there’s no need to change with the existing code. Think of it like *enhancing a rangoli design* by adding more elements (circles or expanding area of rangoli), *without having to repaint or change the design* of what’s already there.

Real-world problem

Think of a billing system in a small shop.

At first, it supports only **Cash** payments. Later the owner wants to add **Card** payments. After some time, they want to add **UPI**, then **Wallet**, then **Gift Card**, and so on.

Every time a new payment method arrives, the developer goes back to the same class and adds another if block.

The class keeps growing and testing becomes harder. One small mistake breaks the whole billing system.

Problematic Code:

```
class PaymentService {  
  
    void pay(String method) {  
  
        if(method.equals("CASH")) {  
            System.out.println("Paid using Cash");  
        }  
        else if(method.equals("CARD")) {  
            System.out.println("Paid using Card");  
        }  
        else if(method.equals("UPI")) {  
            System.out.println("Paid using UPI");  
        }  
        else if(method.equals("WALLET")) {  
            System.out.println("Paid using Wallet");  
        }  
    }  
}
```

What's wrong here?

1. Every time we add a new payment method, we must **open this class and modify it**.
2. Too many if/else blocks it is error-prone, and hard to test.
3. One change affects the entire payment flow.
4. Violates the Open/Closed principle.

How Open/Closed Principle solves this?

Our class should be **closed for modification** but **open for extension**.

This means: Instead of touching the existing code, we should just **add** new classes.

The main idea is:

- Create an **interface** for payment
- Every payment method will have its **own class**
- No more if/else inside the core service

Now adding a new method doesn't require touching the main logic.

Code Example:

1. Create a Payment Interface

```
interface Payment {  
    void pay();  
}
```

2. Implement Each Payment Method in Its Own Class

```
class CashPayment implements Payment {  
    public void pay() {  
        System.out.println("Paid using Cash");  
    }  
}  
  
class CardPayment implements Payment {  
    public void pay() {  
        System.out.println("Paid using Card");  
    }  
}  
  
class UpiPayment implements Payment {  
    public void pay() {  
        System.out.println("Paid using UPI");  
    }  
}
```

Payment Service

```
class PaymentService {  
  
    private Payment payment;  
  
    public PaymentService(Payment payment) {  
        this.payment = payment;  
    }  
  
    void process() {  
        payment.pay();  
    }  
}
```

Main class

```
PaymentService service = new PaymentService(new UpiPayment());  
service.process();
```

Why This Solves the Problem

1. We never touch PaymentService again.
2. To add a new method like **WalletPayment**, just create a new class.
3. System becomes safer because old code stays untouched.
4. Each payment method can be tested independently.

5. Programmer mistakes reduce because logic is split cleanly.

The application can grow without fear.

319. Solid Principles 4: What is Liskov substitution principle ?

The idea of this principle is:

If our program uses an object of a parent class, we should be able to replace it with any object of a child class, and everything should still work correctly.

In other words, we have to be able to substitute objects of a parent class with objects of child classes without causing the program to break. That's why the principle has the keyword 'substitution' in its name.

Imagine **P** is a parent class, and **C1**, **C2**, and **C3** are its child classes. Then we should be able to do.

```
P obj = new C1();
P obj = new C2();
P obj = new C3();
```

Real-time example

Imagine we write a program that works with a parent class called **Notification**.

Now we create child classes like **EmailNotification** and **SMSNotification**.

Both can be used wherever a Notification is expected. Everything works smoothly.

```
public class EmailNotification extends Notification {
    @Override
    public void send(String message) {
        System.out.println("Email sent: " + message);
    }
}

public class SMSNotification extends Notification {
    @Override
    public void send(String message) {
        System.out.println("SMS sent: " + message);
    }
}
```

Now we added new class for PushNotification.

```
class PushNotification extends Notification {
    @Override
    void send(String message) {
        throw new RuntimeException("Push notifications disabled!");
    }
}
```

If we call PushNotification like this. It crashes, saying push notification is disabled

```
Notification n = new PushNotification();
n.send("Welcome!");
```

It breaks the Liskov substitution Principle. Because the child class changed the basic promise of the parent class. We could not substitute parent class (Notification) with Child class.

Fixing the Problem

To fix the issue, we separate the idea of “a notification” from “a notification that can actually send messages.” This way, only the right child classes inherit the right behaviour.

1. Base Class: Notification (common features)

This class represents a general notification. It may store logs, metadata, or anything shared.

```
class Notification {  
    public void log(String message) {  
        System.out.println("Log: " + message);  
    }  
}
```

2. Subclass: SendableNotification

This subclass adds the ability to actually send messages. only child classes that truly support sending should inherit from this.

```
class SendableNotification extends Notification {  
    public void send(String message) {  
        System.out.println("Sending: " + message);  
    }  
}
```

3. Email Notification (can send)

Email can send messages, so it extends SendableNotification.

```
class EmailNotification extends SendableNotification {  
    @Override  
    public void send(String message) {  
        System.out.println("Email sent: " + message);  
    }  
}
```

4. SMS Notification (can send)

SMS can also send messages.

```
class SMSNotification extends SendableNotification {  
    @Override  
    public void send(String message) {  
        System.out.println("SMS sent: " + message);  
    }  
}
```

5. Push Notification (cannot send)

Push notifications may be disabled or limited. It should not be forced into the "send" behavior. So, it extends Notification, not SendableNotification.

```
class PushNotification extends Notification {
    @Override
    public void log(String message) {
        System.out.println("Push logged only: " + message);
    }
}
```

6. NotificationService

Now the service can handle both types:

```
class NotificationService {

    public void sendAll(List<Notification> list) {

        for (Notification n : list) {

            if (n instanceof SendableNotification) {
                ((SendableNotification) n).send("Hello!");
            } else {
                // Safe fallback for non-sendable notifications
                System.out.println("This type cannot send, logging instead");
                ((Notification) n).log("Hello!");
            }
        }
    }
}
```

7. Main class

```
public class LiskovSubstitution {
    public static void main(String[] args) {

        List<Notification> list = new ArrayList<>();
        list.add(new EmailNotification());
        list.add(new SMSNotification());
        list.add(new PushNotification()); // Works now

        NotificationService service = new NotificationService();
        service.sendAll(list);
    }
}
```

Code link: [DesignPatterns/DesignPatterns/src/com/designpatterns/LiskovSubstitution.java](https://github.com/codinglyf/DesignPatterns/src/com/designpatterns/LiskovSubstitution.java) at main · CodingLyf-Fullstack/DesignPatterns

Reference link: [SOLID Principles: Liskov Substitution Principle | by Amr Saeed | Medium](#)

Video link: [Liskov Substitution Principle Tutorial with Java Coding Example for Beginners](#)

320. **Solid Principles 5: What is Interface Segregation principle?**

It says, *A class should never be forced to depend on methods it does not need.*

What Does This Mean?

If an interface has too many methods, classes that implement it may be forced to define methods they do not need. This creates unnecessary dependencies, making the system harder to maintain and modify.

Why is ISP Important?

Without ISP, large interfaces create several problems:

Unnecessary dependencies - A class depends on methods it never uses.

Difficult maintenance - A change in one method affects all classes that implement the interface.

Violates the Single Responsibility Principle (SRP) - An interface tries to do too many things.

Harder testing - Mocking dependencies becomes more complex.

The solution? Instead of having **one large interface, we should break it down into smaller, more specific interfaces.**

1. The Problem

Imagine we are building a simple machine system.

We start with a single interface called Machine.

It has three actions:

- start
- stop
- print

Everything looks fine at first.

A PrinterMachine can use all three.

But then we add a new machine: CoffeeMachine. CoffeeMachine can start and stop, but it does not print anything.

However, because of the big interface, CoffeeMachine is forced to implement the print() method anyway.

It keeps methods it will never use. This makes the code ugly and confusing.

This is exactly what the Interface Segregation Principle warns us about.

2. Problematic Code

```
interface Machine {
    void start();
    void stop();
    void print();    // Not needed by CoffeeMachine
}

class PrinterMachine implements Machine {
    public void start() { System.out.println("Printer started"); }
    public void stop() { System.out.println("Printer stopped"); }
    public void print() { System.out.println("Printing document"); }
}

class CoffeeMachine implements Machine {
    public void start() { System.out.println("Coffee machine started"); }
    public void stop() { System.out.println("Coffee machine stopped"); }
    public void print() {
        throw new UnsupportedOperationException("Coffee machine cannot print");
    }
}
```

3.What's wrong here?

- CoffeeMachine is forced to implement print() method it will never use.
- The program may crash because of unnecessary exceptions.
- The interface is too big for some classes.

This violates ISP.

4.How ISP Fixes the Problem

The principle encourages us to break large interfaces into smaller, focused ones.

So instead of one big interface, we create separate interfaces for each behavior:

- Start/Stop behavior
- Print behavior

Now machines implement only what they actually need.

5. Corrected Code

1. Create two Interfaces:

```
interface Startable {
    void start();
    void stop();
}

interface Printable {
    void print();
}
```

2. Create two classes by extending the interfaces

```
class PrinterMachine implements Startable, Printable {
    public void start() {
        System.out.println("Printer started");
    }

    public void stop() {
        System.out.println("Printer stopped");
    }

    public void print() {
        System.out.println("Printing document");
    }
}

class CoffeeMachine implements Startable {
    public void start() {
        System.out.println("Coffee machine started");
    }

    public void stop() {
        System.out.println("Coffee machine stopped");
    }
}
```

3. Main class

```
public class InterfaceSegregationExample {
    public static void main(String[] args) {

        Startable printer = new PrinterMachine();
        Startable coffee = new CoffeeMachine();

        printer.start();
        printer.stop();

        coffee.start();
        coffee.stop();

        Printable p = new PrinterMachine();
        p.print();

        // CoffeeMachine is never forced to print – clean and safe
    }
}
```

Why This Solves the Problem

- By separating the interface into smaller, focused parts, the CoffeeMachine ends up with only the methods that actually make sense for it.
- It no longer carries extra methods like print() that it would never use.
- Because those unnecessary methods are gone, we avoid the risk of someone accidentally calling them and causing unexpected errors.
- Each class now exposes only the abilities it genuinely supports, which makes the code easier to read and understand.
- When new machine types are added later, we don't have to change the existing ones or force them to adopt features they don't need.
- They simply pick the small interfaces that match their behaviour.
- This keeps the system simple, flexible, and safe, and the overall design becomes much easier to grow and maintain.

Reference links:

[\(11\) Interface Segregation Principle \(ISP\) – SOLID Principles in Java | LinkedIn](#)

[Interface Segregation Principle in Java with Example](#)

[The Fourth SOLID Principle: Interface Segregation in Java | by Daniel Angel | Medium](#)

321. **Solid Principles 6: What is Dependency Inversion pattern?**

It says, *High-level modules should not depend on low-level modules. Both should depend on abstractions.*

What Does This Mean?

If an interface has too many methods, classes that implement it may be forced to define methods they do not need. This creates unnecessary dependencies, making the system harder to maintain and modify.

What Does This Mean?

Instead of tightly coupling high-level business logic with low-level implementation details, we should introduce **abstractions (interfaces)** that allow flexible and maintainable code. This makes our system extensible, testable, and independent of specific implementations.

Bad Example – Without DIP

Let's take an example of a database service that is tightly coupled to MySQL.

Problem:

- The DatabaseService class is hardcoded to use MySQL.
- If we need to switch to PostgreSQL or MongoDB, we must modify `***DatabaseService`, which violates DIP.

```
class MySQLDatabase {
```

```

    public void connect() {
        System.out.println("Connecting to MySQL database...");
    }
}

```

```

class DatabaseService {
    private MySQLDatabase database;

    public DatabaseService() {
        this.database = new MySQLDatabase();
    }

    public void fetchData() {
        database.connect();
        System.out.println("Fetching data from MySQL...");
    }
}

```

Why is this bad?

Tightly Coupled: DatabaseService is hardcoded to use MySQL.

Difficult to Extend: If we want to use PostgreSQL, we must modify DatabaseService.

Violates DIP: The high-level module (DatabaseService) depends directly on the low-level module (MySQLDatabase).

Good Example - Applying DIP

Instead of **hardcoding the dependency**, we introduce an **interface** (DatabaseConnection) that defines an abstraction.

Solution:

- The DatabaseService now depends on DatabaseConnection** (an abstraction), not a specific implementation**.
- We can switch databases **without modifying** ****DatabaseService.

```

interface DatabaseConnection {
    void connect();
}

```

```

class MySQLDatabase implements DatabaseConnection {
    @Override
    public void connect() {
        System.out.println("Connecting to MySQL database...");
    }
}

class PostgreSQLDatabase implements DatabaseConnection {
    @Override
    public void connect() {
        System.out.println("Connecting to PostgreSQL database...");
    }
}

```

```
}
}
```

```
class DatabaseService {
    private DatabaseConnection database;

    // Dependency Injection via Constructor
    public DatabaseService(DatabaseConnection database) {
        this.database = database;
    }

    public void fetchData() {
        database.connect();
        System.out.println("Fetching data...");
    }
}
```

Main.java

```
public class DependencyInversion {
    public static void main(String[] args) {
        DatabaseService mysqlService = new DatabaseService(new MySQLDatabase());
        mysqlService.fetchData();

        DatabaseService postgresService = new DatabaseService(new
        PostgreSQLDatabase());
        postgresService.fetchData();
    }
}
```

Explanation:

- First, an interface called DatabaseConnection is created. It defines a common rule for all databases by declaring a connect method. This interface represents the abstraction.
- Then MySQLDatabase and PostgreSQLDatabase classes implement this interface. These are low-level classes that contain the actual database connection details, but they follow the same abstraction.
- Next, the DatabaseService class is written as the main business logic. It does not depend on any concrete database class. Instead, it depends only on the DatabaseConnection interface. The actual database object is passed from outside through the constructor.
- In the main method, we choose which database to use and pass it to DatabaseService. The service works the same way for any database that follows the interface.
- Dependency Inversion is achieved here because the high-level class DatabaseService does not depend on low-level classes like MySQLDatabase or PostgreSQLDatabase. Both the service and the database classes depend on the abstraction DatabaseConnection, which makes the code flexible and easy to change.

Reference: [\(13\) Dependency Inversion Principle \(DIP\) – SOLID Principles in Java | LinkedIn](#)

322. Design Patterns 1: What are design patterns?

Design patterns are simply proven solutions to common problems in software development. They are not code but they are ideas or templates that guide us when writing the code.

For example, when a house has many rooms that need the same electric wiring, electricians follow a common plan instead of inventing a new one each time. Design patterns are the same idea but for code.

They give us a structured way to solve repeating problems so we don't redesign everything from scratch. They help us write cleaner code, avoid mistakes, and make our programs easier to change later.

These are called **The Gang of Four (GoF)** design patterns are 23 proven, reusable solutions for common problems in object-oriented software design

The Gang of Four Design Patterns is a set of solutions to common problems we encounter in software design and development. They were first introduced in the book Design Patterns: Elements of Reusable Object-Oriented Software, published in 1994. The book was written by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides, who are collectively known as **Gang of Four**.

A simple example is the **Singleton pattern**.

Sometimes a system needs exactly one object of a class - like one printer manager, one database connection manager, or one log writer.

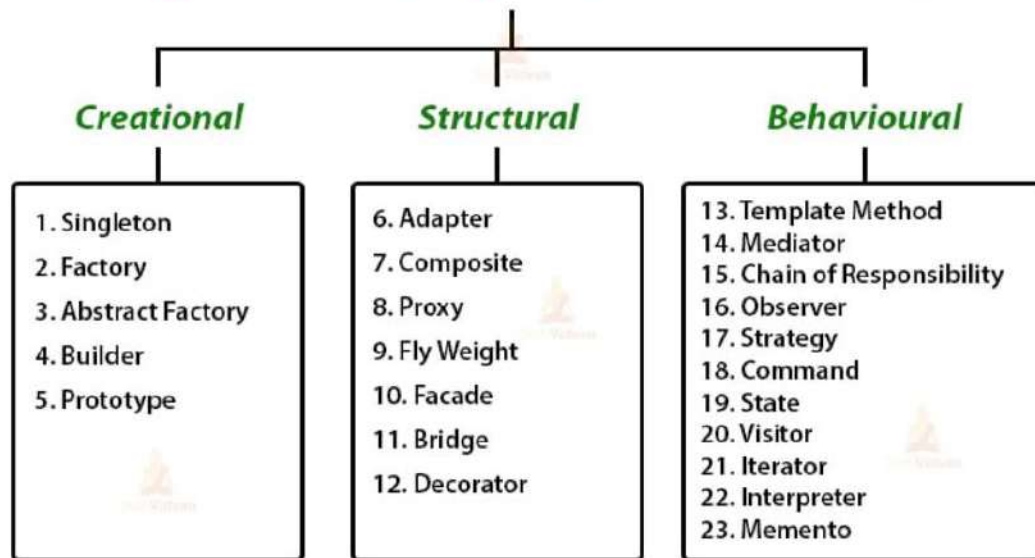
Instead of letting the application create multiple copies and run into problems, we make a class that guarantees only one instance will ever exist.

This pattern is reused across many applications because the problem repeats everywhere.

323. Design Patterns 2: What are the types of design patterns?

Design patterns are grouped into three simple categories. Each category focuses on a different kind of problem we face while building software.

Types of Design Pattern in Java



1. Creational Patterns

These patterns help us **create objects in a better way**. Sometimes, directly using **new** everywhere makes code messy or hard to control. Creational patterns give us controlled, flexible ways to create objects.

Example - Singleton

Some parts of a system should have only one object, like a single database connection manager. The Singleton pattern makes sure the class creates only one instance and shares it everywhere.

Other examples Factory Method, Builder, Prototype

2. Structural Patterns

Structural design patterns define how classes and objects are combined to form larger, flexible structures. They simplify object relationships, improve reusability and make systems easier to understand and maintain.

Structural design patterns provide ways to create a class structure. For example, creating a large object from smaller ones using inheritance and composition.

Example - Adapter

Suppose we have a new printer that doesn't match the old system's format.

We add a small "adapter" class in the middle that converts the old request into the format the new printer understands.

Other examples Decorator, Facade, Composite

3. Behavioural Design Patterns

Behavioural design patterns are a group of design patterns that focus on how objects and classes interact and communicate in software development.

Example - Observer

Imagine a mobile app where many screens should update when a user changes a setting. Instead of updating each screen one by one, we will ask screens to “subscribe” to the setting. When the setting changes, all subscribed screens get updated automatically.

Other examples Strategy, Command, State

Summary:

- Creational patterns help us create objects cleanly.
- Structural patterns help us connect objects the right way.
- Behavioral patterns help us manage communication between objects.

324. Design Patterns 3: What are the benefits of Design patterns?

Using design patterns in Java offers several benefits:

1. **Reusability:** Design patterns encapsulate reusable solutions to common design problems, promoting code reuse and minimizing redundancy.
2. **Maintainability:** By adhering to established design patterns, code becomes more organized, modular, and easier to maintain.
3. **Scalability:** Design patterns facilitate scalability by providing flexible and adaptable solutions that can evolve with changing requirements.
4. **Readability:** Design patterns improve code readability and comprehension, making it easier for developers to understand and collaborate on projects.

Creational Design Patterns

325. Design Patterns 4: What is singleton pattern and when to use it?

The **Singleton Pattern** ensures that a class has **only one instance** throughout the entire application and provides a **global point of access** to that instance.

1. Real-World Analogy

- A Singleton is like a **school principal**.
- The school has many teachers and many classes, but it has only one principal.
- Everyone in the school refers to the same person for major decisions.
- If each class had its own principal, the rules would conflict and the school would be impossible to manage.

2. The Problem

Let's imagine a system that writes logs to a file. Every part of the application creates its own Logger object:

- the payment module
- the order module
- the user module

Each one writes to the same log file, but because multiple objects are writing at the same time, the file becomes messy. Messages come in random order, some overwrite others, and debugging becomes painful.

```
class Logger {
    void log(String msg) {
        System.out.println("LOG: " + msg);
    }
}

public class SingletonPattern {
    public static void main(String[] args) {
        Logger l1 = new Logger();
        Logger l2 = new Logger();
        l1.log("Start");
        l2.log("Processing");
    }
}
```

Here, every part of the system keeps creating new Logger objects.

3. How Singleton Solves It

The Singleton pattern solves this by making sure the class creates **only one object** and returns that same object every time.

Instead of many copies of the Logger, we keep one instance in memory.

Every part of the system gets access to the same shared object.

It does not matter how many times we ask for the object;

we always get the one true instance.

This keeps values consistent, predictable, and easy to manage.

```
class Logger {

    private static Logger instance;

    private Logger() {
        // private constructor stops outside object creation
    }

    public static synchronized Logger getInstance() {
```

```

        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }

    public void log(String msg) {
        System.out.println("LOG: " + msg);
    }
}

public class SingletonPattern {
    public static void main(String[] args) {
        Logger logger1 = Logger.getInstance();
        Logger logger2 = Logger.getInstance();

        logger1.log("App started");
        logger2.log("Request received");
    }
}

```

Now both logger1 and logger2 objects refer to the **same object**

4. JDK Examples of Singleton

The Java standard library uses Singleton in many places:

1. Runtime class `Runtime r = Runtime.getRuntime();`
2. `Calendar.getInstance()` gives a managed instance
3. Logger classes in `java.util.logging`

Code link: [DesignPatterns/DesignPatterns/src/com/designpatterns/SingletonPattern.java at main · CodingLyf-Fullstack/DesignPatterns](https://github.com/CodingLyf/DesignPatterns/blob/main/DesignPatterns/src/com/designpatterns/SingletonPattern.java)

326. Design Patterns 5: How do you implement the singleton pattern?

The idea behind a Singleton is very straightforward: We want a class to have only one object in the entire application, and we want the same object to be returned every time someone asks for it.

To achieve this, we follow three simple steps:

1. **Stop others from creating objects directly.**
We do this by making the constructor private. Once the constructor is private, no one can call `new` from outside.
2. **Store one shared object inside the class itself.**
This object is kept in a static variable, so it stays in memory and can be accessed globally.
3. **Provide a public method that returns the shared object.**
 - Instead of using `new`, we ask the class for the instance through this method.

- The method checks if the object exists.
- If it doesn't, it creates one.
- If it already exists, it returns the same one.

With these three steps, we make sure only one object is created, and the entire application uses it.

Singleton class

```
class AppConfig {  
  
    private static AppConfig instance;  
  
    // Step 1: private constructor  
    private AppConfig() { }  
  
    // Step 2 and 3: public method to get the only instance  
    public static synchronized AppConfig getInstance() {  
        if (instance == null) {  
            instance = new AppConfig();  
        }  
        return instance;  
    }  
  
    // example field  
    String theme = "light";  
}
```

Using Singleton

```
public class Main {  
    public static void main(String[] args) {  
  
        AppConfig c1 = AppConfig.getInstance();  
        AppConfig c2 = AppConfig.getInstance();  
  
        c1.theme = "dark";  
  
        System.out.println(c2.theme); // prints "dark"  
    }  
}
```

Here, both c1 and c2 refer to the same object.

There is only one AppConfig in memory, and that is exactly what a Singleton does.

327. Design Patterns 6: What are the different implementations of Singleton pattern?

The Singleton pattern can be created in a few different ways.

1. Lazy Initialization (Object created when needed)
2. Thread-Safe Singleton (Synchronized)
3. Double-Checked Locking
4. Eager Initialization

5. Static Inner Class

1. Lazy Initialization (Object created when needed)

This is the most basic form. The object is created only when the application asks for it the first time.

```
class AppConfig {  
  
    private static AppConfig instance;  
  
    private AppConfig() { }  
  
    public static AppConfig getInstance() {  
        if (instance == null) {  
            instance = new AppConfig();  
        }  
        return instance;  
    }  
}
```

Here we have declared **getInstance()** static so that we can call it without instantiating the class. The first time **getInstance()** is called it creates a new singleton object and after that, it just returns the same object.

Note: Singleton obj is not created until we need it and call the `getInstance()` method. This is called lazy instantiation. The main problem with the above method is that it is not thread-safe.

2. Thread-Safe Singleton (Synchronized)

To make the lazy version safe for multiple threads, we add the synchronized keyword.

```
class Singleton {  
    private static Singleton obj;  
    private Singleton() {}  
  
    // Only one thread can execute this at a time  
    public static synchronized Singleton getInstance()  
    {  
        if (obj == null)  
            obj = new Singleton();  
        return obj;  
    }  
}
```

- Here using synchronized makes sure that only one thread at a time can execute **getInstance()**.
- The main disadvantage of this method is that using synchronized every time while creating the singleton object is expensive and may decrease the performance of your program.
- However, if the performance of **getInstance()** is not critical for your application this method provides a clean and simple solution.

3. Eager Initialization

Here the object is created as soon as the class is loaded, even if the application never uses it.

```
class AppConfig {  
  
    private static final AppConfig instance = new AppConfig();  
  
    private AppConfig() { }  
  
    public static AppConfig getInstance() {  
        return instance;  
    }  
}
```

Here we have created an instance of a singleton in a static initializer.

JVM executes a static initializer when the class is loaded and hence this is guaranteed to be thread-safe but we lose the flexibility of creating the object only when needed.

4. Double-Checked Locking

This version avoids the performance cost of synchronizing the entire method. It checks twice: once outside the lock, once inside.

```
class AppConfig {  
  
    private static volatile AppConfig instance;  
  
    private AppConfig() { }  
  
    public static AppConfig getInstance() {  
        if (instance == null) {  
            synchronized (AppConfig.class) {  
                if (instance == null) {  
                    instance = new AppConfig();  
                }  
            }  
        }  
        return instance;  
    }  
}
```

We have declared the instance volatile which ensures that multiple threads offer the instance variable correctly when it is being initialized to the Singleton instance

This method drastically reduces the overhead of calling the synchronized method every time.

5. Static Inner Class (Best Java-Specific Way)

This is often considered the cleanest approach in Java. The inner class holds the Singleton instance, and it gets created only when we call getInstance().

- A class is loaded into memory only once by the JVM.

- An inner class is loaded only when it is referenced.
- Therefore, the Singleton instance is created lazily, only when the getInstance() method accesses the inner class.

```
class AppConfig {  
  
    private AppConfig() { }  
  
    private static class Holder {  
        private static final AppConfig instance = new AppConfig();  
    }  
  
    public static AppConfig getInstance() {  
        return Holder.instance;  
    }  
}
```

In the above code, we are having a private static inner class **Holder** and having private field.

Through, getInstance() method of singleton class, we will access the field of inner class and due to being inner class, it will be loaded only one time at the time of accessing the **instance** field first time.

And the **instance** is a static member due to which it will be initialized only once.

When to use each?

1. Lazy Initialization

Use when the app is simple, single-threaded, and we want to create the object only when first requested.

2. Thread-Safe (Synchronized)

Use when the app is multithreaded, safety matters, and slight performance slowdown is acceptable for easy implementation.

3. Double-Checked Locking

Use when we want thread safety with better performance and avoid full method synchronization on every call.

4. Eager Initialization

Choose this when the object is light and we are sure it will be used, so creating it early is fine.

5. Static Inner Class

Use when we want lazy loading, thread safety, and no synchronization overhead; cleanest and most common modern choice.

Reference: [Singleton Method Design Pattern - GeeksforGeeks](#)

328. Design Patterns 7: How to break the Singleton pattern?

A Singleton is meant to create only one object.

Let's walk through the two main ways a Singleton breaks: **reflection** and **serialization**, and then see how to fix them.

1. Breaking Singleton Using Reflection

Reflection allows us to access private parts of a class, including its private constructor. Even if the constructor is private, reflection can still open it and create new objects.

Let's take this code:

```
class AppConfig {  
  
    private static AppConfig instance = new AppConfig();  
  
    private AppConfig() { }  
  
    public static AppConfig getInstance() {  
        return instance;  
    }  
}
```

It looks safe, but this code can create multiple objects:

```
Constructor<AppConfig> c = AppConfig.class.getDeclaredConstructor();  
c.setAccessible(true); // breaks the private rule  
AppConfig obj1 = c.newInstance();  
AppConfig obj2 = c.newInstance();
```

Now obj1 and obj2 are two different objects.

The whole idea of Singleton collapses because reflection bypassed the private constructor.

Fixing Reflection

We simply check inside the private constructor. if someone tries to create another object when one already exists, we stop it.

```
private AppConfig() {  
    if (instance != null) {  
        throw new RuntimeException("Object already created");  
    }  
}
```

Code Link:

2. Breaking Singleton Using Serialization

When a Singleton is serialized and then deserialized, Java creates a fresh object during deserialization. This means we can easily end up with two different objects:

```
AppConfig c1 = AppConfig.getInstance();
```

```
ObjectOutputStream out = new ObjectOutputStream(new
FileOutputStream("a.obj"));
out.writeObject(c1);

ObjectInputStream in = new ObjectInputStream(new FileInputStream("a.obj"));
AppConfig c2 = (AppConfig) in.readObject();
```

Even though both came from the Singleton class, c1 and c2 are not the same object anymore.

Fix for Serialization

We add a method called readResolve() to return the same instance during deserialization.

```
private Object readResolve() {
    return instance;
}
```

This tells Java: “Whenever this class is deserialized, give the existing Singleton instance, not a new one.”

Code Link: [DesignPatterns/DesignPatterns/src/com/designpatterns/BreakSingleton_Serialization.java at main · CodingLyf-Fullstack/DesignPatterns](#)

[DesignPatterns/DesignPatterns/src/com/designpatterns/BreakSingleton_Reflection.java at main · CodingLyf-Fullstack/DesignPatterns](#)

Reference link: [How to Break Singleton Design pattern in Java || Java Interview Questions and Answers | Code Decode - YouTube](#)

329. Design Patterns 8: What is factory pattern and when to use it?

When a program needs to create objects, the common approach is to use the new keyword. This works well at the start, but as the project grows, we need to create the object where ever needed. If tomorrow the class name changes, we end up modifying many places. This makes the system hard to change and easy to break.

The Factory Pattern gives us a simple idea: *Place all the object creation at one place.*

Instead of writing the new keyword everywhere, we move the creation logic into a separate class. So, the rest of the system only asks the factory for the object it needs.

This keeps the code clean and easier to manage.

1.Real-World Analogy:

Imagine a big car company (Maruthi).

Customers don't walk directly into the factory and start telling workers how to build cars. They don't choose the engine, fix the wheels, or point at the machines. That would be confusing and slow.

Instead, customers go to the showroom. They simply say, “I want this model.” The factory takes care of everything else behind the scenes. It builds the right car with the right parts and hands it over ready to use.

The Factory Pattern works the same way.

We don’t let every part of the program create objects on its own. We keep all creation in one place, just like a real factory. The rest of the system only asks for what it needs, and the factory gives the right object without exposing the complicated steps.

2. The Problem

Let’s say we have two services NotificationService and AdminAlertService and both NotificationService and AdminAlertService create objects like this:

```
new EmailNotification();
new SmsNotification();
```

NotificationService.java

```
public class NotificationService {

    public void send(String type, String message) {

        EmailNotification e = new EmailNotification();
        e.send(message);

        SmsNotification s = new SmsNotification();
        s.send(message);

    }
}
```

AdminAlertService.java

```
public class AdminAlertService {

    public void send(String type, String message) {

        EmailNotification e = new EmailNotification();
        e.send(message);

        SmsNotification s = new SmsNotification();
        s.send(message);

    }
}
```

Here, the problems are

1. The biggest issue is that the services becomes tightly coupled to every notification type.
2. Object creation logic is both the services, if we change class name or if we need to new notification type like WhatsappNotification, then we need to change both the services

3. Another hidden issue is testing. If the class directly creates objects using `new`, we cannot replace those objects during testing. This makes it difficult to test the logic without actually sending emails or SMS.

3. The Factory Pattern Solution

The Factory Pattern takes all creation logic out of the main class and moves it into a dedicated “factory” class.

This factory decides which object should be created and returns it. Now our main class only cares about *using* the object, not *creating* it.

Let’s fix the above example using the Factory Pattern.

Step 1: Define the Product Interface

```
interface Notification {  
    void send(String message);  
}
```

Step 2: Create Concrete Products

```
class EmailNotification implements Notification {  
    public void send(String message) {  
        System.out.println("Email sent: " + message);  
    }  
}  
  
class SmsNotification implements Notification {  
    public void send(String message) {  
        System.out.println("SMS sent: " + message);  
    }  
}  
  
class WhatsAppNotification implements Notification {  
    public void send(String message) {  
        System.out.println("WhatsApp sent: " + message);  
    }  
}
```

Step 3: Factory Class

```
class NotificationFactory {  
  
    public static Notification getInstance(String type) {  
        return switch (type.toLowerCase()) {  
            case "email" -> new EmailNotification();  
            case "sms" -> new SmsNotification();  
            case "whatsapp" -> new WhatsAppNotification();  
            default -> throw new RuntimeException("Unable to create object for an  
invalid notification type");  
        };  
    }  
}
```



```

    };
}
}

```

Step 5: Main class

```

public class NotificationService {

    public static void main(String[] args) {
        Notification n;
        try {
            n = NoficationFactory.getInstance("email");
            n.send("Hello World: This is Email");

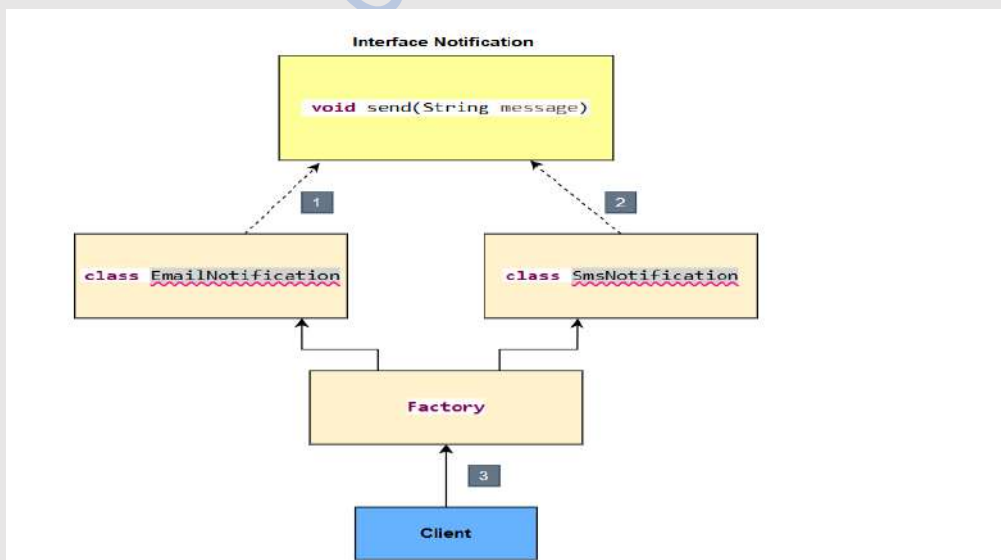
            n = NoficationFactory.getInstance("sms");
            n.send("Hello World: This is SMS");

            n = NoficationFactory.getInstance("whatsapp");
            n.send("Hello World: This is Whatsapp message");

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

- Now the service does not deal with object creation.
- If we add Telegram tomorrow, we change only the Factory.
- Everything else stays the same.



1. EmailNotification implements Notification

The dotted line means **implementation**. EmailNotification implements the Notification interface and provides its own version of the send() method for sending emails.

2. SmsNotification implements Notification

This dotted line also shows **implementation**. SmsNotification provides its own logic to send messages through SMS, but follows the same send() contract.

3. Client asks the Factory to create the required Notification object

The solid line shows **usage or invocation**.

The client does not create EmailNotification or SmsNotification directly.

Instead, it talks to the **Factory**, and the factory decides which concrete class to instantiate based on some input (like "email" or "sms").

4. JDK Examples

Java has many built-in examples of the Factory Pattern:

- Calendar.getInstance()
- NumberFormat.getInstance()
- DocumentBuilderFactory.newInstance()
- Connection connection = DriverManager.getConnection()
- Logger.getLogger()

All these methods create objects behind the scenes. We never call new on these classes directly.

5. When To Use Factory Pattern

We should use the Factory Pattern when:

- the program needs many related objects
- the object creation logic should be in one place
- object creation changes based on conditions
- we want the code to stay flexible and easy to extend

This pattern becomes helpful in payment systems, notification systems, storage systems, and anywhere we create multiple types of objects based on input.

6. Pros and Cons

Pros

1. One place to create objects: All object creation stays in one spot. So when we add a new type or change an old one, we touch only the factory, not the whole project.

2. Code becomes easier to test: Since services don't use new directly, we can replace real objects with fake ones during tests.

This makes testing simple and safe.

Cons

1. Factory can grow big: If we keep adding many types, the factory can become long and crowded. Then we must clean or reorganize it.

2. Extra layer to understand: It adds one more step, which can feel confusing at first.

7.Code & Reference link:

[DesignPatterns/DesignPatterns/src/com/designpatterns/FactoryPattern Notification.java at main · CodingLyf-Fullstack/DesignPatterns](#)

Reference: [Factory Pattern in Java | by Neelesh Janga | Medium](#)

330. **Design Patterns 9: What is abstract factory pattern and when to use it?**

2. The Problem

Let's consider the above example (Notification).

Suppose our application needs to support not just three or four notifications but 10 or even more. If we try to handle all these inside one normal Factory class, we end up writing long if-else chains. Every time we add a new notification type, we must open that class again, add more conditions. Over time this becomes hard to read, hard to test, and hard to maintain.

This is where the Abstract Factory pattern helps us. We can think of it as a **factory of factories**. Instead of putting all creation logic in one place, we create a small factory for each type of notification. Each factory only knows how to create its own object. The main code does not worry about how these objects are built. It only asks the correct factory and gets the right object back.

For example, In the below code,

- We have *EmailNotificationFactory* that creates email notifications.
- We have *SmsNotificationFactory* that creates SMS notifications.
- If tomorrow we need Push, Telegram, Slack or any other type, we simply add a new factory for each one.
- The rest of the system stays untouched.
- This keeps the project clean and easy to grow, even when the number of notification types increases.

3. The Abstract Factory Pattern Solution.

Before we dive into implementation details, let's see the primary components of the Abstract Factory Pattern:

1. **Product:** This is an interface or abstract class that defines the type of objects to be created. (Notification)

2. **Concrete Product:** These are the classes that implement the Product interface or extend the Product abstract class. They represent the actual objects created by the factory (EmailNotification, SMSNotification etc).
3. **Factory:** This is an interface or abstract class responsible for declaring the factory method, which creates objects of the Product type (Ex: NotificationFactory).
4. **Concrete Factory:** These are the classes that implement the Factory interface or extend the Factory abstract class. They override the factory method to produce specific instances of Product (EmailNotificationFactory, SMSNotificationFactory etc).

a. Product Interface

```
interface Notification {  
    void send(String message);  
}
```

b. Concrete Product

```
class EmailNotification implements Notification {  
    public void send(String message) {  
        System.out.println("Email sent: " + message);  
    }  
}  
  
class SmsNotification implements Notification {  
    public void send(String message) {  
        System.out.println("SMS sent: " + message);  
    }  
}  
  
class WhatsAppNotification implements Notification {  
    public void send(String message) {  
        System.out.println("WhatsApp sent: " + message);  
    }  
}
```

c. Factory Interface

```
interface NotificationFactory {  
    Notification createNotificationObject();  
}
```

d. Concrete Factory

```
class EmailNotificationFactory implements NotificationFactory {  
  
    @Override  
    public Notification createNotificationObject() {  
        return new EmailNotification();  
    }  
}  
  
class SmsNotificationFactory implements NotificationFactory {
```

```

@Override
public Notification createNotificationObject() {
    return new SmsNotification();
}
}

```

e.Main

```

public class AbstractFactoryPattern_Notification {

    public static void main(String[] args) {
        NotificationFactory emailFactory = new EmailNotificationFactory();
        Notification emailNotification = emailFactory.createNotificationObject();

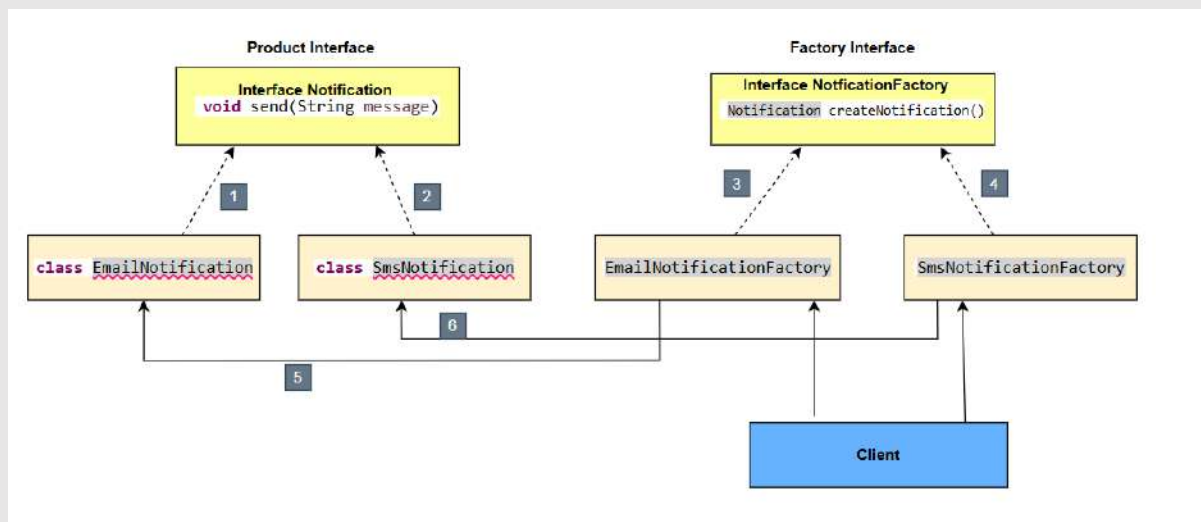
        emailNotification.send("Hello this is for email");

        NotificationFactory smsFactory = new SmsNotificationFactory();
        Notification smsNotification = smsFactory.createNotificationObject();

        smsNotification.send("Hello this is for sms");
    }
}

```

- This example shows how we create different types of notifications without using new in the main code.
- We first define a common interface called *Notification*, and every notification type follows this rule. Email, SMS, and WhatsApp are the actual classes that send messages.
- Instead of creating these objects directly, we use factories like *EmailNotificationFactory* or *SmsNotificationFactory*.
- Each factory knows how to create its own notification object, so the main program just asks the factory and uses whatever it returns.
- This keeps the code clean, easy to change, and safe from mistakes.



1. EmailNotification implements Notification

The dotted line shows an **implements** relationship.

EmailNotification provides its own version of the send() method defined in the Notification interface.

2. SmsNotification implements Notification

This dotted line also represents **implementation**.

SmsNotification also follows the Notification interface and implements send() for SMS messages.

3. EmailNotificationFactory implements NotificationFactory

The dotted line here shows that EmailNotificationFactory implements the NotificationFactory interface and provides its version of createNotification().

4. SmsNotificationFactory implements NotificationFactory

Again, dotted line means **implements**.

SmsNotificationFactory also implements NotificationFactory and returns an SMS-specific notification object.

5. EmailNotificationFactory creates EmailNotification

The solid line shows **object creation or dependency**.

When the client uses EmailNotificationFactory, it receives an EmailNotification object.

6. SmsNotificationFactory creates SmsNotification

This solid line represents **creation** as well.

Using SmsNotificationFactory, the client gets an SmsNotification object.

7. The Client calls the factory to get the required notification object (Solid Line to Factory)

The client does **not** create objects directly.

Instead, it calls a factory (either email or SMS factory) and receives a Notification object.

After that, the client simply invokes: notification.send("Hello User");

Why Abstract Factory is better than simple Factory:

With Abstract Factory, we can add new notification types by creating a new factory, without touching existing logic. This makes the system easier to extend and less likely to break.

Code Link:

[DesignPatterns/DesignPatterns/src/com/designpatterns/AbstractFactoryPattern Notification.java at main · CodingLyf-Fullstack/DesignPatterns](#)

Reference video: [Simplifying Software Development with Factory Design Pattern:Beginner's Guide | Design Patterns Java](#) (Here both factory and abstract factory explain with the example ice cream)

331. Design Patterns 10: Abstract Factory Pattern with another example?

Abstract Factory pattern is almost similar to Factory Pattern and is considered as another layer of abstraction over factory pattern. Abstract Factory patterns work around a super-factory which creates other factories

1.Real-World Analogy:

Think of a phone store. We walk in and ask for a model.

The seller already has a setup with different factories like Apple, Samsung, and OnePlus.

The seller talks to these companies behind the scenes and gets the right phone for us.

We never see how the seller checks stock or deal with each company.

We only get the phone we asked for.

So, the phone store works like a super-factory.

Each company acts as a small factory, and the store hides the details of all of them.

This is the core idea of the Abstract Factory pattern.

2. The Problem

Let's say, we need to support multiple phone brands in our application.

Each brand has its own models. if we handle everything with direct object creation, the code quickly becomes messy.

We end up with huge if-else or switch blocks scattered across the app. Any new brand or new model requires modifying existing code.

This breaks the Open–Closed Principle and increases bugs. Here's how the code looks without Abstract Factory.

```
class PhoneService {  
  
    public Phone getPhone(String brand, String model) {  
  
        if (brand.equals("Apple")) {  
            if (model.equals("iphone14")) {  
                return new Phone("iPhone 14");  
            }  
        }  
    }  
}
```

```

        } else if (model.equals("iphone14-pro")) {
            return new Phone("iPhone 14 Pro");
        }
    }

    if (brand.equals("Samsung")) {
        if (model.equals("s23")) {
            return new Phone("S23");
        } else if (model.equals("s23-ultra")) {
            return new Phone("S23 Ultra");
        }
    }

    if (brand.equals("OnePlus")) {
        if (model.equals("oneplus8")) {
            return new Phone("OnePlus 8");
        } else if (model.equals("oneplus9")) {
            return new Phone("OnePlus 9");
        }
    }

    return null;
}
}

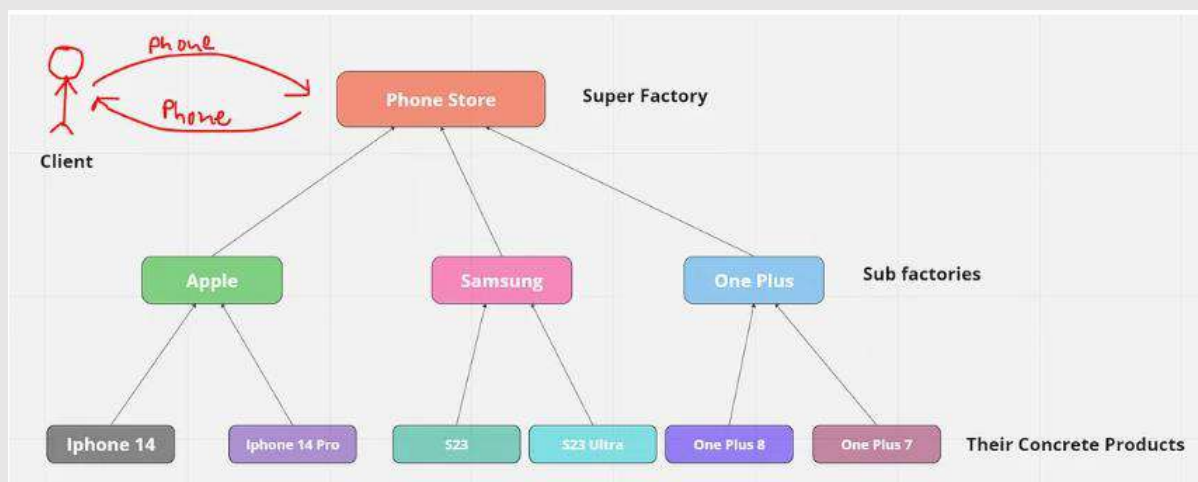
```

Why this is a problem

- Code becomes tightly coupled to every brand and model
- Hard to extend (adding a new brand means editing this class)
- Business logic mixes with object creation logic
- Violates Open–Closed Principle

3. The Abstract Factory Pattern Solution.

It mainly contains 4 components. 1.Product Interface, 2. Concrete Product, 3. Factory and Concrete Factory



a. Product (common contract for all phones)


```
interface Phone {  
    void getDescription();  
}
```

b. Concrete Products (We are implementing the Phone interface creating 3 products, Apple, Samsung, OnePlus models)

```
class ApplePhone implements Phone {  
    private String model;  
  
    ApplePhone(String model) {  
        this.model = model;  
    }  
  
    @Override  
    public void getDescription() {  
        System.out.println("Apple " + model);  
    }  
}  
  
class SamsungPhone implements Phone {  
    private String model;  
  
    SamsungPhone(String model) {  
        this.model = model;  
    }  
  
    @Override  
    public void getDescription() {  
        System.out.println("Samsung " + model);  
    }  
}  
  
class OnePlusPhone implements Phone {  
    private String model;  
  
    OnePlusPhone(String model) {  
        this.model = model;  
    }  
  
    @Override  
    public void getDescription() {  
        System.out.println("OnePlus " + model);  
    }  
}  
  
//To avoid null pointer issues  
class NullPhone implements Phone {  
    @Override  
    public void getDescription() {
```

```

        System.out.println("Invalid model");
    }
}

```

c. Abstract Factory (one method to create phones)

```

interface PhoneFactory {
    Phone createPhoneInstance (String model);
}

```

d. Concrete Factories (We are implementing PhoneFactory to create factory for each brand)

```

class AppleFactory implements PhoneFactory {
    @Override
    public Phone createPhoneInstance (String model) {
        switch (model) {
            case "iphone14": return new ApplePhone("iPhone 14");
            case "iphone14-pro": return new ApplePhone("iPhone 14 Pro");
            default: return new NullPhone();
        }
    }
}

class SamsungFactory implements PhoneFactory {
    @Override
    public Phone createPhoneInstance (String model) {
        switch (model) {
            case "s23": return new SamsungPhone("S23");
            case "s23-ultra": return new SamsungPhone("S23 Ultra");
            default: return new NullPhone();
        }
    }
}

class OnePlusFactory implements PhoneFactory {
    @Override
    public Phone createPhoneInstance(String model) {
        switch (model) {
            case "oneplus8": return new OnePlusPhone("OnePlus 8");
            case "oneplus9": return new OnePlusPhone("OnePlus 9");
            default: return new NullPhone();
        }
    }
}

```

e. Super Factory (selects the brand factory)

```

class PhoneStore {

```

```

private PhoneStore() {}

public static PhoneFactory getBrandFactory(String brand) {

    switch (brand) {
        case "Apple": return new AppleFactory();
        case "Samsung": return new SamsungFactory();
        case "OnePlus": return new OnePlusFactory();
        default: return null;
    }
}

```

f. main

```

public class AbstractFactoryPattern_Phone {
    public static void main(String[] args) {

        PhoneFactory appleFactory = PhoneStore.getBrandFactory("Apple");
        Phone p1 = appleFactory.createPhoneInstance ("iphone14");
        p1.getDescription();

        System.out.println();

        PhoneFactory samsungFactory = PhoneStore.getBrandFactory("Samsung");
        Phone p2 = samsungFactory.createPhoneInstance ("s23");
        p2.getDescription();

        System.out.println();

        // invalid model handled safely
        Phone p3 = appleFactory.createPhoneInstance ("iphone10");
        p3.getDescription();
    }
}

```

Components of Abstract Factory Design Pattern

1. Abstract Product

This is the main interface that defines what every product must support.

In our code, the **Phone interface** is the abstract product.

It acts as a common Java contract so every phone model implements the same method `getDescription()`.

2. Concrete Product

These are the actual classes that implement the abstract product.

In our code, **ApplePhone, SamsungPhone, OnePlusPhone, and NullPhone** are concrete products.

Each one provides its own version of `getDescription()` and represents a real phone model returned by a factory.

3. Abstract Factory

This is an interface that declares the creation method.

In our code, the **PhoneFactory interface** is the abstract factory. It exposes one method, `createPhone(String model)`, which explains *what* needs to be created but not *how*.

4. Concrete Factory

These are the real factory classes that implement the abstract factory and build the correct products. In our code, **AppleFactory**, **SamsungFactory**, and **OnePlusFactory** are concrete factories. Each one uses Java logic like switch or if to create the right phone model for its brand.

4.JDK Examples

- 1. DocumentBuilderFactory:** Used to create different DocumentBuilder implementations for XML parsing.
- 2. SSLContext:** Creates related objects like SSLEngine, key managers, and trust managers, all from one factory.
- 3. SocketFactory (javax.net / javax.net.ssl):** Returns different socket implementations such as secure, plain, or custom sockets.

5. When To Use Abstract Factory Pattern

Use this pattern when:

- A module needs to create a family of related objects. like different implementations of the same interface (Phone, Button, Parser), and all objects must come from the same factory.
- We want to follow the Open–Closed Principle by adding new product families (new factories) without modifying existing client logic.
- Object creation logic must stay centralized instead of being scattered across the application.

6. Pros and Cons

Pros

Reduces coupling: Client code depends only on interfaces, not concrete classes.

Easier to extend: Adding a new product family means adding a new factory without changing existing code.

Cons

More classes to maintain: Every product family requires multiple classes: factory + products.

7.Code & Reference link:

[DesignPatterns/DesignPatterns/src/com/designpatterns/AbstractFactoryPattern_Phone.java at main · CodingLyf-Fullstack/DesignPatterns](#)

[Abstract Factory Design Pattern in Java | by Akshat Sharma | Medium](#)

332. Design Patterns 11: What is builder pattern and when to use it?

1. Real-World Analogy

- Think of ordering a burger at a cafe.
- We start with the base burger. Then we choose cheese, veggies, sauces, and sides. Some items are required and some are optional.
- If the cafe asked us to choose everything in one long form, it would be confusing.
- Instead, they guide us step by step so we build the burger the way we want. At the end, they prepare the final burger as one complete item.
- The Builder pattern works the same way. It helps us build a complex object in small steps and gives the final product only when everything is ready.

2. The Problem

Think of a registration screen in an app.

We collect many details: name, email, password, phone, age, address, gender, city, country and more. Some fields are required and some are optional.

If we try to create a User object using constructors, the code becomes messy because we keep adding new parameters.

We end up with several constructors that look similar and are hard to read.

```
//telescoping constructors
class User {
    User(String name) { }
    User(String name, String email) { }
    User(String name, String email, String phone) { }
    User(String name, String email, String phone, int age) { }
    User(String name, String email, String phone, int age, String address) { }
    // and the list keeps growing
}
```

This class contains multiple constructors we can fix this by Builder pattern.

3. The Builder Pattern Solution

The Builder Pattern separates the construction of a complex object to a separate helper class. The helper class sets each part of the object step by step. When all details are ready, the builder creates the final object in one go. This keeps the code readable

- The Builder pattern helps us create a User object step by step.
- We keep required fields in the Builder constructor and add optional fields through clear methods.
- At the end, build() creates a complete User object that cannot be changed later.
- The creation becomes readable and safe.

Updated User class with builder pattern

```
class User {
    private final String name;    // required
    private final String email;   // required
    private final String phone;   // optional
    private final int age;        // optional
    private final String address; // optional

    private User(Builder builder) {
        this.name = builder.name;
        this.email = builder.email;
        this.phone = builder.phone;
        this.age = builder.age;
        this.address = builder.address;
    }

    public static class Builder {
        private final String name;
        private final String email;

        private String phone;
        private int age;
        private String address;

        public Builder(String name, String email) {
            this.name = name;
            this.email = email;
        }

        public Builder phone(String phone) {
            this.phone = phone;
            return this;
        }

        public Builder age(int age) {
            this.age = age;
            return this;
        }

        public Builder address(String address) {
            this.address = address;
            return this;
        }

        public User build() {
            return new User(this);
        }
    }
}
```

- The User class keeps all fields final, so once the object is built it cannot change.

- The Builder class takes required fields in its constructor and then allows optional fields like phone, age, and address to be added through readable methods.
- When all information is ready, the build() method creates a complete User. This avoids long constructors and keeps the registration logic simple.
- Here is the main class that uses it:

Main

```
User user = new User.Builder("Rahul", "rahul@example.com")
    .phone("9876543210")
    .age(25)
    .address("Hyderabad")
    .build();
```

4. JDK Examples

StringBuilder helps us build a string step by step before converting it to a final string.

ProcessBuilder lets us set commands, environment, directory and then start the process.

DateTimeFormatterBuilder helps us build complex date formats before creating the formatter.

5. When To Use Builder Pattern

Use the Builder pattern when a class has many parameters and most of them are optional.

Use it when we want to avoid long constructors that are hard to understand.

6. Pros and Cons

Pros. Builder makes complex object creation easy to read and understand. The pattern also keeps the final object safe by creating it only when all required information is ready.

Cons. Builder needs extra code for the helper class. It can feel heavy for very small objects that have only one or two fields.

7. Code link:

[DesignPatterns/DesignPatterns/src/com/designpatterns/BuilderPattern_UserRegistration.java at main · CodingLyf-Fullstack/DesignPatterns](#)

333. Design Patterns 12: What is prototype pattern and when to use it?

1. Real-World Analogy

- Think about preparing a resume.
- The first resume takes the longest because we choose fonts, layout, spacing, and sections.
- Once it is ready, creating more resumes becomes easy.
- We simply copy the first one and change only the name and details.
- The first resume works like a prototype that saves time each time we create a new version.

2. The Problem

In real enterprise applications we often send API requests to multiple services. A request usually contains many common fields like authentication tokens, correlation IDs, timestamps, user preferences, default headers, and region codes. Setting these fields for every request object becomes repetitive and slow. Many developers create the same request again and again using long initialization blocks:

```
ApiRequest req = new ApiRequest();
req.setAuthToken(loadToken());
req.setCorrelationId(UUID.randomUUID().toString());
req.setLocale("en_IN");
req.setTimeZone("Asia/Kolkata");
req.setClientVersion("v3");
// repeated in many modules
```

- This code appears in several microservices and leads to duplication.
- If a header format changes, we must update dozens of places.
- The system becomes hard to maintain and slower because each request repeats heavy setup.

3. The Prototype Pattern Solution

The Prototype pattern allows us create one base request with all defaults already filled. Whenever a module needs a new request, we simply clone this base version. Then we only change the fields specific to that module, like API path or payload. This brings consistency, reduces duplication, and improves performance.

Example:

```
class ApiRequest implements Cloneable {
    private String authToken;
    private String correlationId;
    private String locale;
    private String timeZone;
    private String clientVersion;
    private String payload;

    public ApiRequest(String authToken, String locale, String timeZone, String
clientVersion) {
        this.authToken = authToken;
        this.correlationId = UUID.randomUUID().toString();
        this.locale = locale;
        this.timeZone = timeZone;
        this.clientVersion = clientVersion;
    }

    @Override
    public ApiRequest clone() {
        try {
            ApiRequest copy = (ApiRequest) super.clone();
            copy.correlationId = UUID.randomUUID().toString();
            return copy;
        }
    }
}
```



```

        } catch (CloneNotSupportedException e) {
            throw new RuntimeException(e);
        }
    }

    public void setPayload(String payload) {
        this.payload = payload;
    }
}

```

Using it in Service class:

```

ApiRequest base = new ApiRequest("AUTH-123", "en_IN", "Asia/Kolkata", "v3");

ApiRequest orderReq = base.clone();
orderReq.setPayload("{orderId: 101}");

ApiRequest paymentReq = base.clone();
paymentReq.setPayload("{paymentId: 555}");

```

4. JDK Examples

- Object.clone() is a core example of the pattern.
- ArrayList.clone() gives a copy of the same list structure.
- HashMap.clone() copies the map quickly without rebuilding it.
- Calendar.clone() duplicates calendar settings instantly.

5. When To Use Prototype Pattern

It is used to when we make copy/clone from the existing object.

6. Code link: [DesignPatterns/DesignPatterns/src/com/designpatterns/Prototype_ApiRequest.java at main · CodingLyf-Fullstack/DesignPatterns](https://github.com/CodingLyf-Fullstack/DesignPatterns/blob/main/src/com/designpatterns/Prototype_ApiRequest.java)

334. Design Patterns 13: Differences between all the creational design patterns?

This table helps to remember them easily

Pattern	Core Idea	What It Solves	How It Works
Singleton	Only one object	Avoid multiple instances	One shared object is created and reused
Factory	Decide object at runtime	Remove tight coupling	A factory method returns required object

Pattern	Core Idea	What It Solves	How It Works
Abstract Factory	Factory of factories	Create related objects together	Factory creates other factories
Builder	Build step by step	Handle complex object creation	Object is created in parts
Prototype	Copy existing object	Avoid costly object creation	Clone an existing object

Structural Design Patterns

335. Design Patterns 14: What is Adapter pattern and when to use it?

1. Real-World Analogy

- Think of travelling to another country where the power sockets are different.
- Our charger cannot fit into their socket, so we buy a travel adaptor.
- The adaptor sits between the wall socket and our charger, making both work together even though they were never designed for each other.
- The adaptor does not change the charger or the socket.
- It only converts one type of plug into another so everything works smoothly.

2. The Problem

In enterprise projects, teams move from one database to another due to cost, performance or company decisions.

For example, the old system used **Oracle** with one style of connection and query execution.

The new system uses **PostgreSQL** with different method names and connection rules.

Both expose completely different methods:

Oracle library might use `openConnection` and `runQuery`:

```
oracle.openConnection(url, user, pass);
oracle.runQuery(sql);
```

PostgreSQL library might use `startSession` and `execute`:

```
pg.startSession(url, user, pass);
pg.execute(sql);
```

Now imagine a large project where many services interact with the database.

If all the service classes call these database methods directly, the code becomes tightly tied to a specific

database.

When the company switches to PostgreSQL, all modules break and developers must update thousands of lines of code.

3. The Adapter Pattern Solution

The Adapter design pattern is a structural design pattern that allows two incompatible interfaces to work together

The Adapter Pattern involves three key components:

1. Target Interface (What our application understands)

This is the interface that our whole project depends on.

It defines the common methods we want to use for any database, such as connect and execute.

This is the **DatabaseClient** interface.

```
interface DatabaseClient {  
    void connect(String url, String user, String pass);  
    void execute(String sql);  
}
```

2. Adaptee (The existing vendor library with a different interface)

These are the real database classes provided by vendors.

They have their own method names and their own connection style.

In our example, **OracleDB** and **PostgresDB** are the adaptees.

Our application cannot use these directly because the method signatures do not match our standard interface.

```
class OracleDB {  
    void openConnection(String url, String user, String pass) { }  
    void runQuery(String sql) { }  
}  
  
class PostgresDB {  
    void startSession(String url, String user, String pass) { }  
    void executeSql(String sql) { }  
}
```

3. Adapter (The bridge that connects both sides)

This class sits between the application and the vendor library.

It implements the Target Interface and holds an instance of the Adaptee inside it.

Whenever the application makes a call, the adapter converts it into the vendor's actual method.

In our example, **OracleAdapter** and **PostgresAdapter** are the adapters.

The adapter converts our standard connect and execute calls into the vendor's methods so the application never sees the differences.

```
class OracleAdapter implements DatabaseClient {

    private OracleDB oracle;

    public OracleAdapter(OracleDB oracle) {
        this.oracle = oracle;
    }

    public void connect(String url, String user, String pass) {
        oracle.openConnection(url, user, pass);
    }

    public void execute(String sql) {
        oracle.runQuery(sql);
    }
}

class PostgresAdapter implements DatabaseClient {

    private PostgresDB postgres;

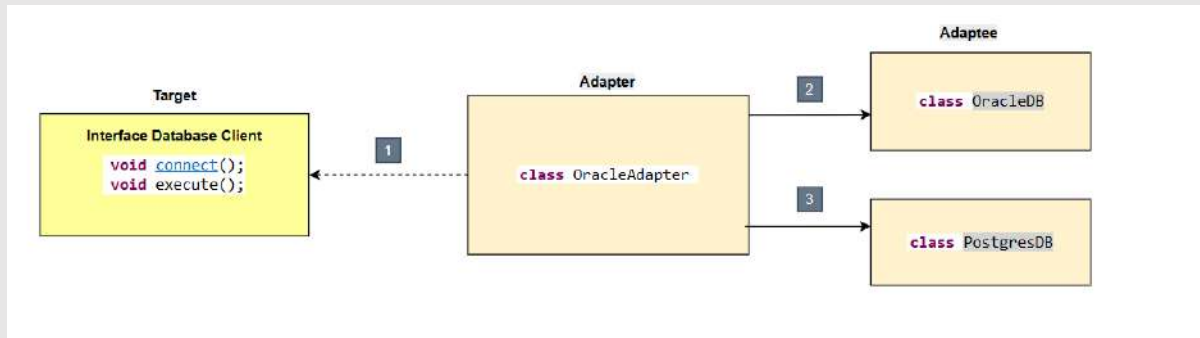
    public PostgresAdapter(PostgresDB postgres) {
        this.postgres = postgres;
    }

    public void connect(String url, String user, String pass) {
        postgres.startSession(url, user, pass);
    }

    public void execute(String sql) {
        postgres.executeSql(sql);
    }
}
```

- DatabaseClient is the common interface that the application expects. It has two actions: connect and execute.
- OracleDB and PostgresDB are old or third-party classes. They each have their own style of methods:
 - OracleDB uses **openConnection** and **runQuery**
 - PostgresDB uses **startSession** and **executeSql**
- But the client only understands the common methods defined in DatabaseClient: **connect** and **execute**.
- To solve this problem, we create adapters:
 - OracleAdapter converts *connect* into *openConnection* and *execute* into *runQuery*.

- PostgresAdapter converts *connect* into *startSession* and *execute* into *executeSql*.
- In the main method, the client uses both Oracle and Postgres through the same DatabaseClient interface. The client does not care about different method names. The adapters translate the calls behind the scenes.



1. OracleAdapter implements DatabaseClient

The dotted line shows an **implements** relationship.

The OracleAdapter class follows the DatabaseClient interface and provides the methods connect() and execute() that the client expects.

2. OracleAdapter uses OracleDB

The solid line shows a **usage or dependency**.

Inside OracleAdapter, when the client calls connect() or execute(), the adapter translates those calls into openConnection() and runQuery() of OracleDB.

3. OracleAdapter uses PostgresDB

This solid line also represents **usage**.

If used with Postgres, the adapter converts the calls into startSession() and executeSql() of PostgresDB.

4. JDK Examples

- InputStreamReader
- Collections.enumeration(list)
- Arrays.asList
- java.sql.Driver

5. When To Use Adapter Pattern

When you need to integrate two components or systems that have different interfaces but similar functionalities, and direct communication is not possible, an Adapter can bridge the gap.

In our database example. We had different database providers that offered similar features but exposed completely different method names. Direct communication was not possible because our application expected one style of interface while each database had its own. By using adapters like OracleAdapter and PostgresAdapter, we created a bridge that allowed our application to work with any database provider without changing the existing code.

6. Code link:

[DesignPatterns/DesignPatterns/src/com/designpatterns/AdapterPattern_DBConnection.java at main · CodingLyf-Fullstack/DesignPatterns](#)

7. Reference link: [Adapter Design Pattern Java | The Code Bean | Medium](#)

336. Design Patterns 15: What is Composite pattern and when to use it?

1. Real-World Analogy

Imagine a folder on a computer. A folder can contain files. A folder can also contain other folders. When we click "Delete," the system deletes both files and folders the same way. We do not need to worry whether an item is a file or another folder. The system treats them both in a uniform way.

This idea of treating individual items and groups of items in the same manner is the heart of the Composite pattern.

Composite Design Pattern is a structural design pattern which is used when we have to represent a Tree Like structure. The composite pattern is meant to allow treating individual objects and compositions of objects, or "composites" in the same way.

2. The Problem

Let's say we are working on Employee Management System where employees are arranged in levels. For example, a Manager has Team Leads under them, and each Team Lead has Developers under them. Every level can perform actions like showDetails(), calculateSalary() etc.

If we try to build this using normal classes, we write lot of if-else blocks to check if the node is a Manager or a Developer.

```
if(emp.isManager()) {  
    for(Employee e : emp.getTeam()) {  
        e.printDetails();  
    }  
} else {  
    emp.printDetails();  
}
```

When the hierarchy grows (CEO -> Directors -> Managers -> Leads -> Developers), this code becomes long, difficult to read, and hard to maintain.

Every new role forces us to add more conditions.

We need one structure where the system treats a Manager and a Developer in the same way.

3. The Composite Pattern Solution

The Composite pattern gives us a simple idea:

Create **one common interface** for everyone in the hierarchy.

A Developer is a leaf.

A Manager is a composite node that holds other employees inside it.
Both implement the same interface, so the client does not check who is who.

There are 3 main components in Composite Pattern: Component, Leaf and Composite

1. Component (Common Interface for Everyone)

This is the basic interface that all employees follow.
Managers and Developers both use the same method, like showDetails().
In our example, **Employee** is the component.

2. Leaf (Simple Object with No Children)

A leaf is an object that does not manage anyone else.
It represents the lowest level in the structure.
In our example, a **Developer** is a leaf because a developer does not have employees under them.

```
class Developer implements Employee {  
    private final String name;  
  
    public Developer(String name) {  
        this.name = name;  
    }  
  
    public void showDetails() {  
        System.out.println("Developer: " + name);  
    }  
}
```

3. Composite (Object That Holds Other Components)

A composite is an object that can contain other employees.
It performs its own work and also manages its child elements.
In our example, a **Manager** is a composite because a manager can have Developers or other Managers under them.

```
class Manager implements Employee {  
    private final String name;  
    private final List<Employee> team = new ArrayList<>();  
  
    public Manager(String name) {  
        this.name = name;  
    }  
  
    public void add(Employee e) {  
        team.add(e);  
    }  
  
    public void showDetails() {
```

```

        System.out.println("Manager: " + name);
        for(Employee e : team) {
            e.showDetails();
        }
    }
}

```

The Manager calls showDetails() on itself and then on each team member.

4.Main or Client

```

public static void main(String[] args) {

    Employee dev1 = new Developer("Rajesh");
    Employee dev2 = new Developer("Srinivas");

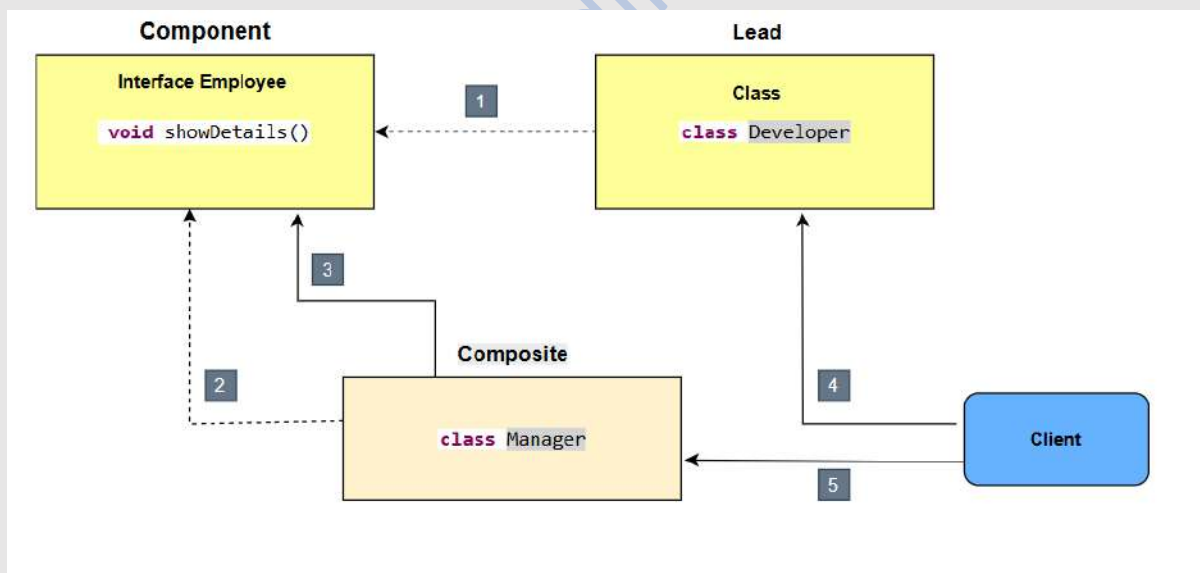
    Manager lead = new Manager("Laxmi");
    lead.add(dev1);
    lead.add(dev2);

    Manager manager = new Manager("Moshe");
    manager.add(lead);

    manager.showDetails();
}

```

Now the company hierarchy behaves as a tree where every element responds to showDetails(). The client does not care whether it is a Manager or Developer.



1. The Developer class **implements** the Employee interface.

This means a developer **is-a** type of employee and must provide the showDetails() method.

2. The Manager class also **implements** the Employee interface, so a manager is also treated as an employee.

3. The Manager class **contains** other employees, such as developers.

This means a manager **has-a** list of employees, which creates the composite structure.

4. The client can directly call *showDetails()* on a single developer. Even though it is a leaf object, it behaves like an employee.

5. The client can also call *showDetails()* on the manager. The manager internally calls *showDetails()* on each employee it manages, giving a combined output.

4. JDK Examples

- java.awt.Container (a container holds components or other containers)
- File and Directory APIs in Java NIO (Files and directories behave through a similar structure)

5. When To Use Composite Pattern

We use the Composite pattern when our project deals with tree-like structures such as:

- Company employee hierarchy
- UI component trees
- File-system structures

6. Code link:

[DesignPatterns/DesignPatterns/src/com/designpatterns/CompositePattern_EmployeeManagement.java at main · CodingLyf-Fullstack/DesignPatterns](#)

7. Reference link: [Composite Design Pattern. In this blog, we will explore the... | by Akshat Sharma | Medium](#)

337. Design Patterns 16: What is Proxy pattern and when to use it?

1. Real-World Analogy

Imagine a celebrity. People cannot meet the celebrity directly. They must go through a manager. The manager checks who is allowed to meet, manages schedules, filters requests, and protects the celebrity's time.

This is the Proxy Pattern in simple terms.

Definition: The Proxy Pattern provides an object that controls access to another object. The proxy performs checks, caching, security, or logging before allowing the real object to run.

The Proxy acts as an intermediary between the client and the real object, allowing you to add additional functionality or control access without modifying the actual object

2. The Problem

In an online banking application, the BankAccountService communicates with a central banking server. Each withdraw or deposit request involves identity checks, network calls, and audit logs.

Calling the real banking service directly from the UI and multiple modules can cause problems:

```
BankAccountService service = new RealBankAccountService();
```

```
service.withdraw("ACC1001", 2000);
service.withdraw("ACC1001", 2000); // repeated authentication
service.deposit("ACC1001", 5000);
```

Here,

The core service becomes overloaded and security becomes hard to manage.

We need a protective layer that filters requests before they hit the real banking service.

3. The Proxy Pattern Solution

The Proxy Pattern introduces a **proxy class** that stands between the client code and the real banking object.

The client calls the proxy instead of touching the real object.

The proxy performs security checks, logging, and validation, and only then forwards the request.

There are 3 components in Proxy pattern. 1. Subject, 2. RealSubject 3. Proxy

1. Subject Interface

This is the common interface used by both the real banking service and the proxy.

It defines the operations the client can perform, such as withdraw and deposit.

BankAccount is the Subject.

```
interface BankAccount {
    void withdraw(String accNo, double amount);
    void deposit(String accNo, double amount);
}
```

2. Real Subject

This is the actual banking service that performs the real work like updating account balance.

It implements the BankAccount interface and contains the main business logic.

RealBankAccountService is the Real Subject.

```
class RealBankAccountService implements BankAccount {
    public void withdraw(String accNo, double amount) {
        System.out.println("Withdrawing " + amount + " from " + accNo);
    }
    public void deposit(String accNo, double amount) {
        System.out.println("Depositing " + amount + " into " + accNo);
    }
}
```

3. Proxy

The proxy stands in front of the real banking service.

It also implements the BankAccount interface and decides when and how the real service is accessed.

It can add security checks, logs, or caching before calling the real service.

In our example, **BankAccountProxy** is the Proxy.

```
class BankAccountProxy implements BankAccount {
    private final RealBankAccountService service = new RealBankAccountService();

    public void withdraw(String accNo, double amount) {
        if (authenticate(accNo)) {
            System.out.println("Security check passed");
            service.withdraw(accNo, amount);
        } else {
            System.out.println("Access denied");
        }
    }

    public void deposit(String accNo, double amount) {
        if (authenticate(accNo)) {
            System.out.println("Security check passed");
            service.deposit(accNo, amount);
        } else {
            System.out.println("Access denied");
        }
    }

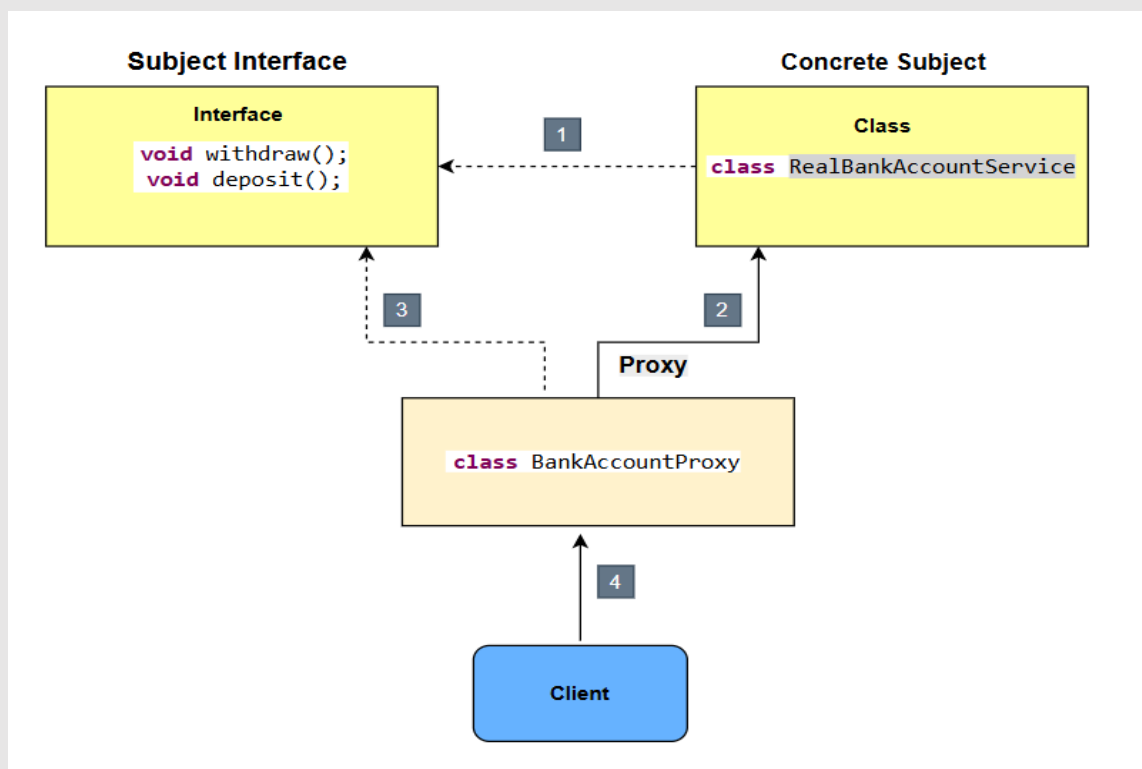
    private boolean authenticate(String accNo) {
        return accNo.startsWith("ACC"); // mock validation
    }
}
```

Main

```
public static void main(String[] args) {
    BankAccount account = new BankAccountProxy();
    account.withdraw("ACC1001", 2000);
    account.deposit("ACC1001", 5000);
}
```

- BankAccount is the main interface. It defines actions like withdraw and deposit.
- RealBankAccountService is the actual class that performs the real work. It prints messages for withdrawing and depositing money.
- BankAccountProxy acts like a security guard. Before calling the real service, it checks if the account number is valid. Only accounts starting with "ACC" are allowed.
- If the account passes the check, the proxy prints "Security check passed" and then calls the real service. If not, it prints "Access denied".

In the main method, we use the proxy instead of calling the real service directly. The proxy makes sure only valid accounts can perform actions. This adds an extra layer of security without changing the original service.



1. This dotted line shows that **RealBankAccountService** *is-a* **BankAccount**. It provides the real code for `withdraw()` and `deposit()` methods.

2. This solid line shows that **BankAccountProxy** has a **RealBankAccountService** object inside it.

3. This dotted line means **BankAccountProxy** also implements the same interface.

4. This solid line shows that the **Client** depends only on the proxy, not on the real bank service.

In simple words:

The client requests the proxy, and the proxy decides when to call the real bank service.

This helps add extra features like authentication without changing existing code.

4. JDK Examples

- `java.lang.reflect.Proxy`
- `java.sql.DriverManager` returning connection proxies
- Hibernate lazy loading uses proxies for database entities

Spring uses the Proxy pattern mainly to add extra behavior around methods without changing the actual class. When we enable features like AOP, transaction management, or security, Spring creates a proxy object instead of giving the real object directly. This proxy intercepts method calls and decides

what to do before and after the real method runs. For example, with transactions, the proxy starts a transaction, calls the real method, and then commits or rolls back automatically.

5. When To Use Proxy Pattern

1. Lazy Loading

If creating or connecting to the real banking service is costly, we can delay it until the first withdraw or deposit request happens. The proxy creates the real service only when needed, saving system resources.

2. Access Control

In banking, not everyone should perform withdraw or deposit operations. The proxy can check whether an account number or user is valid before calling the real service.

3. Remote Communication

If the banking service runs on another server, the proxy acts as a local representative. The client interacts with the proxy, and the proxy handles the communication with the remote banking system.

4. Additional operations

If account details or frequent transactions are requested multiple times, the proxy can store the results and return them without calling the real banking service again, improving performance.

5. Code link:

[DesignPatterns/DesignPatterns/src/com/designpatterns/ProxyPattern_BankingApplication.java at main · CodingLyf-Fullstack/DesignPatterns](#)

6. Reference link: [Proxy Design Pattern: A Comprehensive Guide | by Kalana De Silva | Medium](#)

338. Design Patterns 17: What is Flyweight pattern and when to use it?

1. Real-World Analogy

Imagine a public library. Many people borrow the same book, like "Harry Potter." The library does not create a new book every time someone wants to read it. The story, author, and pages are always the same. Only the reader changes, along with the date the book was borrowed. So, the library keeps one main copy of the book's details and reuses it again and again for different readers. This saves space and avoids storing the same information repeatedly.

This is how Flyweight pattern works.

Definition: Flyweight is a **memory-saving** design pattern. It helps when we need to create **many similar objects**. Instead of creating new objects every time, it **reuses existing ones**.

The Flyweight Design Pattern is a way to save memory in applications that create a large number of similar objects. Instead of creating a new object for each instance, the Flyweight pattern reuses existing ones wherever possible, sharing common parts between objects.

It separates the **intrinsic state** (shared, immutable state) from the **extrinsic state** (unique, contextual state) of an object.

What is intrinsic state and extrinsic state in the Flyweight Pattern?

Please note that these are the most important key concepts & backbone of the Flyweight design pattern. Therefore, it becomes crucial to understand them clearly.

The words intrinsic & extrinsic are made by applying prefixes 'in' & 'ex' in the word 'trinsic'. However, it is not a word by itself, we can interpret "trinsic" as relating to the qualities or attributes of an object, with a prefix (like "in-" or "ex-") determining whether those attributes are **internal** (intrinsic) or **external** (extrinsic).

- **Intrinsic State:** Shared, constant data that is stored in the flyweight object.
- **Extrinsic State:** Context-specific, dynamic data that is supplied externally.

Intrinsic State

- **Definition:**
 - Intrinsic state is the **shared, constant, or unchanging information** stored inside a flyweight object.
 - It is **independent of the context** where the object is used and remains **same across all instances**.
- **Example:** Imagine we are designing a library system. Each book in the library has fixed details like its **title, author, and ISBN number**. A book titled "*Effective Java*" by Joshua Bloch has an ISBN of "9780134685991". These details are constant, no matter who borrows the book.
 - **Intrinsic State:** Title, author, ISBN number.
- **In Programming:** Intrinsic state is stored within the flyweight object and is shared across multiple instances.

Extrinsic State

- **Definition:**
 - Extrinsic state is the **context-dependent, dynamic information** that varies depending on how or where the flyweight object is used.
 - It is **passed or provided by the client** rather than stored within the flyweight object.
- **Example:** Continuing with the library system analogy, the borrower's name, due date, and return status of a book will not have the fixed values.
 - **Extrinsic State:** Borrower's name, due date, and return status
- **In Programming:** Extrinsic state is passed to the flyweight object as a parameter during runtime.

Example#1: Hotel Room Management

Imagine a hotel booking system. Each room has fixed attributes like its **room type** (e.g., “Deluxe Room”, “Standard Room”), **bed count**, and **daily rate**.

For example: A “Deluxe Room” with two queen-sized beds costs \$100 per night. These details remain the same, no matter who books the room.

- **Intrinsic State:** Room type, bed count, daily rate
- **Extrinsic State:** Guest name, booking dates, and any special requests (e.g., extra pillows)

2. The Problem

In a hotel booking system, every time someone books a room, the system creates a new Room object. Imagine 500 guests book the same “Deluxe” room. Even though the room has the same type, same bed, and same view, the system creates 500 separate Room objects.

We are storing the same information again and again. The only details that actually change are the guest’s name and the date of booking. Over time, thousands of duplicate Room objects fill up memory. This slows down the system and makes it harder to maintain.

3. The Flyweight Pattern Solution

The Flyweight Pattern fixes the problem by separating what stays the same and what changes.

Intrinsic State

This is the information that never changes for a room type.

For example:

- Room type (Deluxe)
- Bed type (King Bed)
- Room view (City View)

We store this only once and reuse it.

Extrinsic State

This is the information that changes for each booking.

For example:

- Guest name
- Booking date

This data is provided when the booking happens.

So instead of creating a full Room object every time, we create **one shared Room object** for each room type and use it for all bookings. Only the guest details are different.

Components of the Flyweight Pattern

1. **Flyweight Interface:** The Flyweight Interface defines common methods for all flyweight objects and handles extrinsic state passed by the client.

2. **Concrete Flyweight:** Concrete Flyweight Classes implement the Flyweight interface and store the intrinsic (shared) state that is common across multiple objects.
3. **Flyweight Factory:** The **Flyweight Factory** manages the creation and reuse of flyweight objects, ensuring that shared instances are used whenever possible. It checks if an existing flyweight can be reused or creates a new one if necessary.

Here are the key steps to implement to implement Flyweight Design Pattern:

- Create a Flyweight Interface or Abstract Class: This will declare a method that flyweight objects can use.
- Create Concrete Flyweight Classes: These classes implement the flyweight interface and store the intrinsic state that can be shared.
- Create a Flyweight Factory: This factory will manage the flyweight objects and ensure that objects are shared instead of being instantiated repeatedly.
- Client Class: The client code will provide the extrinsic state and interact with the flyweight objects.

1. Flyweight Interface

The Flyweight Interface defines the common action that all shared room objects must support. It does not store any data itself. Instead, it receives the changing information (guest name and booking date) from the client when needed. This makes the shared room object reusable without holding any booking-specific details.

```
interface RoomFlyweight {  
    void book(String guest, String date); // extrinsic  
}
```

2. Concrete Flyweight

This class represents the shared room object. It holds the intrinsic data, which never changes for that room type, such as the room type, bed type, and view. These details are stored once and reused for every booking. When a booking happens, the client provides the extrinsic information like the guest name and the booking date. The shared Room object uses this data to complete the action, without storing it internally.

```
class Room implements RoomFlyweight {  
    private final String type; // intrinsic  
    private final String bed; // intrinsic  
    private final String view; // intrinsic  
  
    public Room(String type, String bed, String view) {  
        this.type = type;  
        this.bed = bed;  
        this.view = view;  
    }  
}
```



```

    public void book(String guest, String date) { // extrinsic
        System.out.println(type + " booked by " + guest + " on " + date);
    }
}

```

3. Flyweight Factory

This class is responsible for creating and managing shared Room objects. It checks whether a room of the given type already exists. If it does, the factory returns the existing Room object. If not, it creates a new one, stores it, and returns it. This ensures that only one Room object is created for each room type, so the same shared object is reused for multiple bookings, saving memory.

```

class RoomFactory {
    private static final Map<String, Room> rooms = new HashMap<>();

    public static Room getRoom(String type) {
        if (!rooms.containsKey(type)) {
            rooms.put(type, new Room("Deluxe", "King Bed", "City View"));
        }
        return rooms.get(type);
    }
}

```

Main

```

public class Flyweight_HotelBooking {
    public static void main(String[] args) {
        RoomFlyweight r1 = RoomFactory.getRoom("Deluxe");
        r1.book("Ravi", "12-June");

        RoomFlyweight r2 = RoomFactory.getRoom("Deluxe");
        r2.book("Sneha", "13-June");
    }
}

```

Explanation:

In this example, we are booking hotel rooms using the Flyweight Pattern. Instead of creating a new Room object for every booking, we reuse the same room details again and again.

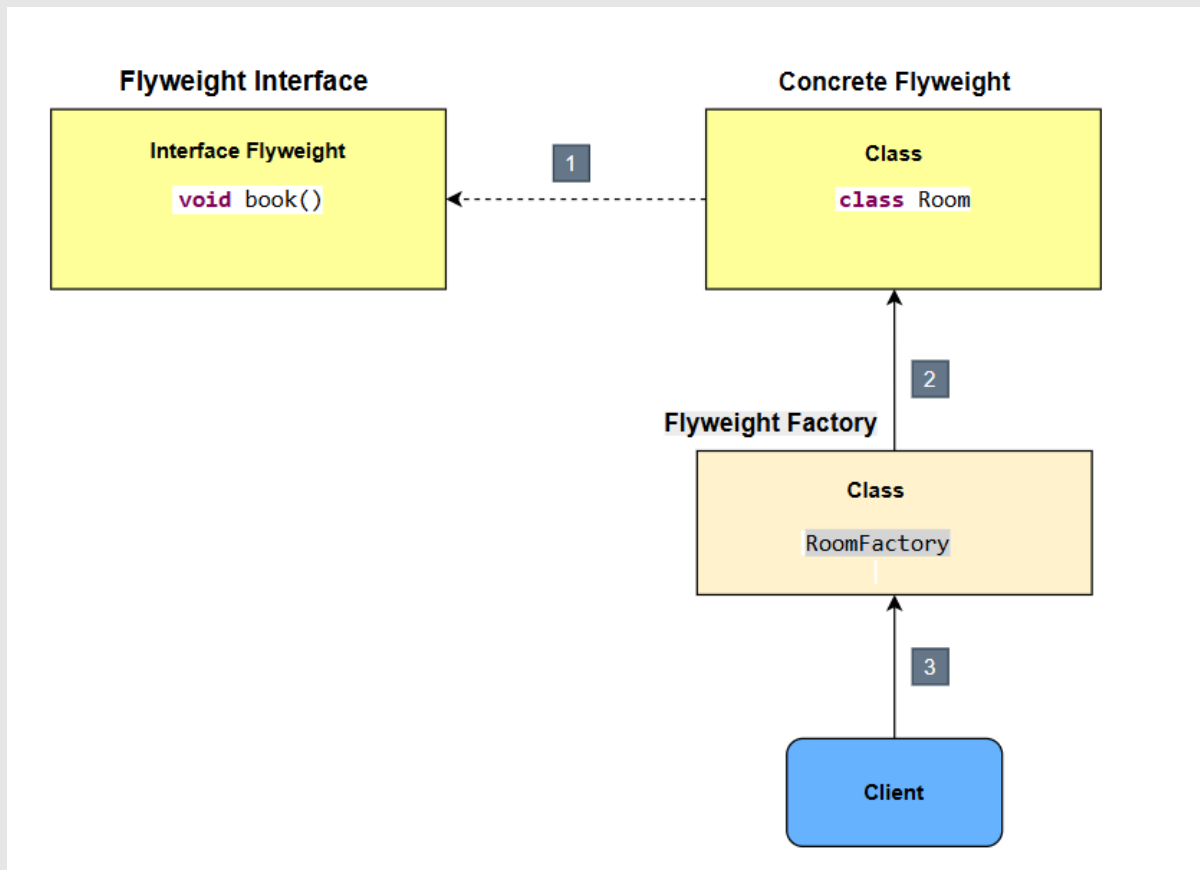
RoomFlyweight is an interface that has one method, `book()`. This method takes the guest name and booking date. These are the details that change each time someone makes a booking. This changing information is called **extrinsic state**.

The **Room** class is the actual shared room object. It stores the room type, bed type, and view. These details never change and are the same for all bookings of that room type. This is called **intrinsic state**. Since these values do not change, we store them only once.

The **RoomFactory** checks if a Room of a particular type already exists. If it exists, it returns the same Room object. If not, it creates one and stores it for future use. This avoids creating duplicate Room objects with the same data.

In the main() method, both bookings request a "Deluxe" room. The factory gives the same Room object for both Ravi and Sneha. Only the booking details change.

This approach saves memory because we reuse the room object instead of creating a new one for each booking.



1. Room implements the RoomFlyweight interface. The dotted line means **is-a relationship** (implementation). So, Room **is a** Flyweight and provides the actual booking logic.

2. RoomFactory creates the Room object only once and stores it.

If the same room type is requested again, it returns the **existing shared object** instead of creating a new one.

The solid arrow shows **has-a relationship** — the factory **has and manages** Room objects.

3. The **Client** does not create rooms directly. It calls `RoomFactory.getRoom("Deluxe")` to get a room object. This ensures that all clients share the same room object for the same room type.

4. JDK Examples

- String Pool

- Integer.valueOf() caching
- Boolean.TRUE and Boolean.FALSE reused

5. When To Use Flyweight Pattern

- Large Number of Similar Objects: When an application needs to manage a massive number of objects that share similar characteristics (e.g., characters in a text editor, icons in a UI).
- Memory Constraints: When memory usage is a concern, objects can be shared to reduce memory consumption.
- Object Creation Overhead: When creating or initializing objects, it is costly, and sharing] them optimizes performance.

5. Code link: [DesignPatterns/DesignPatterns/src/com/designpatterns/Flyweight HotelBooking.java at main · CodingLyf-Fullstack/DesignPatterns](#)

6. Reference link: [Flyweight Design Pattern: An Easy Way](#)

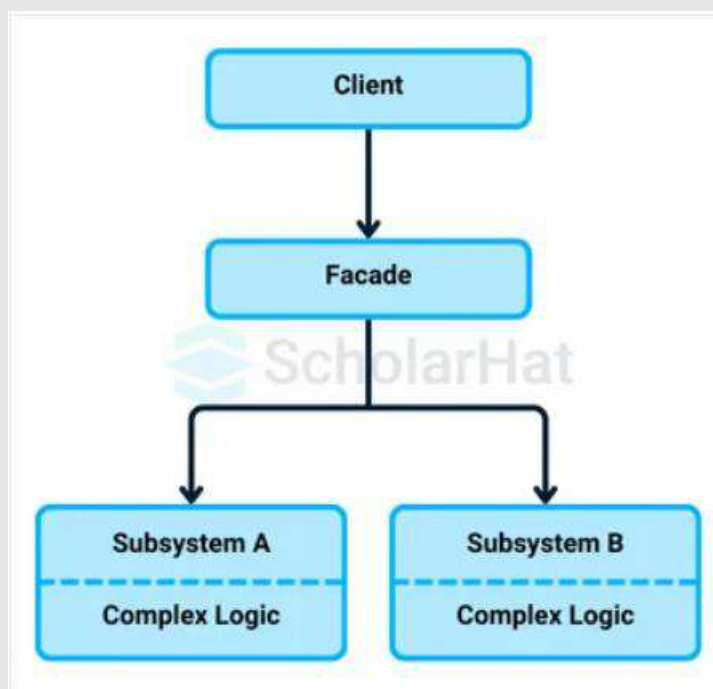
339. Design Patterns 18: What is Facade pattern and when to use it?

1. Real-World Analogy

Think of a travel package service. Instead of booking a flight, hotel, cab, and sightseeing separately, we call one travel agency. The agency handles all the complicated steps behind the scenes and gives us a single, simple service.

Definition

The Facade Pattern provides a single-entry point to a complex system. Instead of interacting directly with multiple subsystems, clients communicate with the facade, which handles and organizes the underlying processes.



2. The Problem

Imagine a banking application where a user performs a money transfer. The transfer process is not simple. Before the money moves, the system must:

1. Validate the account
2. Check the balance
3. Deduct the amount
4. Update the transaction history
5. Send an SMS or email notification

Without a Facade, the client code would look like this:

```
AccountValidator validator = new AccountValidator();
BalanceChecker checker = new BalanceChecker();
TransferService transfer = new TransferService();
NotificationService notify = new NotificationService();

validator.validate(account);
checker.checkBalance(account, amount);
transfer.send(amount, fromAcc, toAcc);
notify.sendMessage(account);
```

The client must understand and call these classes in the correct order.

This leads to:

- Tight coupling with internal classes
- Repeated code everywhere
- Difficult maintenance
- Harder onboarding for new developers

We need one simple class that handles everything internally.

3. The Facade Pattern Solution

The Facade Pattern creates a single class that exposes one simple method, like `transfer(amount)`, and hides all the complex steps inside it. The client now interacts with just one class.

Main Components

Component Purpose

Subsystems Classes that perform the actual work (validator, transfer, notification)

Facade A wrapper class that uses these subsystems internally

Component Purpose

Client Uses only the facade, without knowing the internal details

Step 1 : Subsystems

```
class AccountValidator {
    void validate(String acc) {
        System.out.println("Account validated");
    }
}

class BalanceChecker {
    void check(String acc, double amt) {
        System.out.println("Balance checked");
    }
}

class TransferService {
    void transfer(String from, String to, double amt) {
        System.out.println("Amount transferred");
    }
}

class NotifyService {
    void notifyUser(String acc) {
        System.out.println("Notification sent");
    }
}
```

Step 2: Façade

```
class BankFacade {
    private final AccountValidator validator = new AccountValidator();
    private final BalanceChecker checker = new BalanceChecker();
    private final TransferService transfer = new TransferService();
    private final NotifyService notify = new NotifyService();

    void transferMoney(String from, String to, double amt) {
        validator.validate(from);
        checker.check(from, amt);
        transfer.transfer(from, to, amt);
        notify.notifyUser(from);
    }
}
```

Main

```
public class FacadePattern_BankingApp {
    public static void main(String[] args) {
        BankFacade facade = new BankFacade();
        facade.transferMoney("ACC101", "ACC202", 5000);
    }
}
```

```

    }
}

```

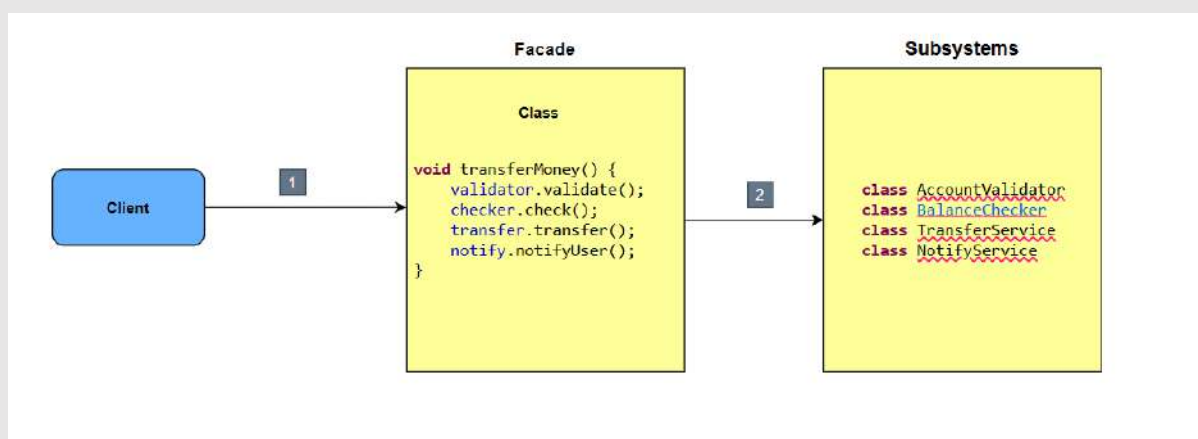
Explanation:

This example shows how the Facade Pattern simplifies a complex banking operation. In a real banking system, transferring money is not a single action. Before money moves, the system must verify the account, check if there is enough balance, perform the transfer, and finally notify the user. These tasks are done by different classes: *AccountValidator*, *BalanceChecker*, *TransferService*, and *NotifyService*.

Without a facade, the client would need to create objects of all these classes and call their methods in the correct order. This makes the code harder to read and maintain.

The *BankFacade* class hides all these details. It combines all the necessary steps inside one method, *transferMoney()*. The client calls only this method, and the facade handles the rest internally.

In the *main()* method, we create a *BankFacade* object and call *transferMoney()*. With just one line, the entire transfer process happens behind the scenes. This makes the application easier to use and keeps the client code clean.



1. The **Client** calls only one method: *transferMoney()*. It does not know how the money is validated, checked, transferred, or notified. The solid line shows a direct usage relationship. **Client uses Facade.**

2. Inside the *transferMoney()* method, the Facade talks to multiple subsystem classes. These subsystems do real work, but the client never deals with them directly. The solid line shows the **Facade has and uses subsystem objects.**

4. JDK Examples

`java.sql.Connection` hides JDBC complexity

5. When To Use Facade Pattern

Use this pattern when:

- The system has multiple classes and too many steps to perform one task
- The client should not deal with complex internal logic
- We want to reduce dependency and simplify method calls

- We want one place to manage workflows

5. Code link: [DesignPatterns/DesignPatterns/src/com/designpatterns/FacadePattern_BankingApp.java](https://github.com/DesignPatterns/DesignPatterns/src/com/designpatterns/FacadePattern_BankingApp.java) at main · CodingLyf-Fullstack/DesignPatterns

6. Reference link: [Mastering the Facade Design Pattern in Java: A Comprehensive Guide with Practical Examples](#) | by Hiten Pratap Singh | hprog99 | Medium

340. Design Patterns 19: What is Bridge pattern and when to use it?

1. Real-World Analogy

The Bridge Design Pattern is a structural pattern that is used to "decouple" an abstraction from its implementation so that the two can vary independently. In simpler terms, it separates the **Abstraction** (what you do) from the **implementation** (how it's done). This makes the system more flexible, extensible, and easy to maintain.

The Bridge Pattern helps us break a large and complicated class into two smaller parts that can change independently. One part decides **what** something should do, and the other part decides **how** it should do it. Because they are separated, we can add new features to one side without touching the other, making the system easier to maintain and extend.

How to Identify Abstraction and Implementation in the Bridge Pattern?

In the Bridge Pattern, the abstraction represents high-level control, while the implementation represents low-level details.

Simple Rule to Remember:

- Abstraction = "What" (High-Level Concept)
- Implementation = "How" (Low-Level Concrete Work)

Let's Break It Down with the Payment System Example:

1. Abstraction (What the user interacts with) is "WHAT" we do
 - Example: Payment Methods like UPI, Credit Card, PayPal
 - Users just select a payment method without worrying about how it's processed.
2. Implementation (How it works under the hood) is "HOW" it's done
 - Example: Payment Gateways like Razorpay, Stripe, Paytm
 - Different gateways process payments differently, but the user doesn't care.

2. The Problem

Let's use the example of a **remote-control system** for a variety of electronic devices to illustrate the Bridge pattern. Imagine we have different kinds of devices (e.g., TV, Radio) and different remote controls (e.g., BasicRemote, AdvancedRemote).

Without the Bridge Pattern, if we were to combine these variations, we'd need a subclass for each combination, leading to a huge class hierarchy:

- BasicRemoteTV, AdvancedRemoteTV, BasicRemoteRadio, AdvancedRemoteRadio, and so on.

This would quickly become unmanageable. Instead, by applying the Bridge pattern, we can decouple the **remote-control interface** from the **device interface**, making the system more flexible and scalable.

3. The Bridge Pattern Solution

The Bridge Pattern fixes this problem by separating the **remote's actions** from the **device's working logic**. Instead of mixing both in one class, we create two independent hierarchies:

- One set of classes for different types of **Remotes** (what the remote can do)
- Another set of classes for different **Devices** (how the device behaves)

We then connect these two using a bridge. Because of this, any remote can work with any device, without creating new classes for every combination.

Step-by-Step Implementation

Step 1: Implementor (Device Interface)

Defines what every device should be able to do.

```
interface Device {  
    void turnOn();  
    void turnOff();  
}
```

Step 2: Concrete Implementors (Different Devices)

```
class TV implements Device {  
    public void turnOn() { System.out.println("TV turned ON"); }  
    public void turnOff() { System.out.println("TV turned OFF"); }  
}  
  
class Radio implements Device {  
    public void turnOn() { System.out.println("Radio turned ON"); }  
    public void turnOff() { System.out.println("Radio turned OFF"); }  
}
```

Step 3: Abstraction (Remote Control)

The remote defines what actions are available. Notice it holds a reference to a Device.

```
abstract class Remote {  
    protected Device device;  
  
    public Remote(Device device) {  
        this.device = device;  
    }  
}
```



```

    }

    abstract void togglePower();
}

```

Step 4: Refined Abstraction (Different Remotes)

```

class BasicRemote extends Remote {
    public BasicRemote(Device device) {
        super(device);
    }

    void togglePower() {
        device.turnOn();
    }
}

class AdvancedRemote extends Remote {
    public AdvancedRemote(Device device) {
        super(device);
    }

    void togglePower() {
        device.turnOn();
        System.out.println("Advanced features enabled");
    }
}

```

Step 5: Main

```

public class BridgePattern_RemoteSystem {
    public static void main(String[] args) {
        Remote basicTVRemote = new BasicRemote(new TV());
        basicTVRemote.togglePower();

        Remote advancedRadioRemote = new AdvancedRemote(new Radio());
        advancedRadioRemote.togglePower();
    }
}

```

Code Explanation:

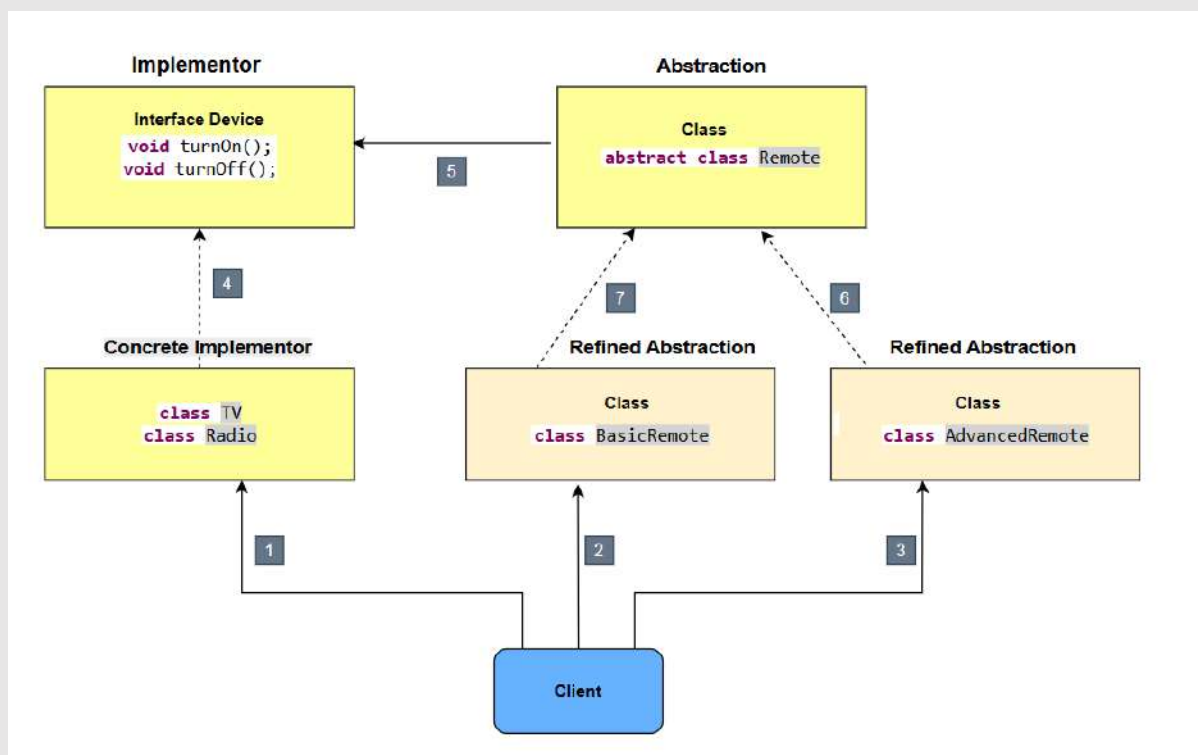
This example shows how the Bridge Pattern helps us connect different types of remotes with different devices without creating too many classes.

The Device interface represents anything that can be turned ON or OFF. TV and Radio are real devices that implement this behaviour. They know how to turn themselves ON and OFF.

The Remote class represents a remote control. It does not know how to operate the device directly. Instead, it holds a reference to a Device. This is the bridge. The remote sends commands, and the device performs the action.

BasicRemote and AdvancedRemote are two types of remotes. They extend the Remote class and add their own behavior. A basic remote only turns the device ON. An advanced remote turns the device ON and adds extra features.

In the main method, we create a BasicRemote for a TV and an AdvancedRemote for a Radio. Each remote works with any device because the bridge connects them. This means we can add more devices or remotes later without changing the existing code.



Bridge Pattern Diagram Explanation

1. The **Client** chooses which device to work with. It creates a **TV** or **Radio** object. A solid line means the client directly uses these classes.
2. The client also selects a type of remote such as **BasicRemote**. The solid line shows the client creates this remote and calls its methods.
3. Similarly, the client can choose **AdvancedRemote**, which has extra features.
4. The **dotted arrow** means **TV** and **Radio** *implement* the **Device** interface. This is an *is-a* relationship: TV **is-a** Device, Radio **is-a** Device.
5. The **Remote** class contains a reference to a **Device**. This is a *has-a* relationship: Remote **has-a** Device. This line shows the Remote depends on the Device interface, not on concrete classes. This makes the bridge possible.
6. A dotted arrow here shows inheritance. **AdvancedRemote is-a Remote** and adds more features.
7. This also shows inheritance. **BasicRemote is-a Remote** and provides basic operations.

How This Helps

With the Bridge Pattern:

- If a new device arrives (Projector), we just create one class.
- If a new type of remote arrives (VoiceRemote), we create one more class.
- Both work together instantly—no new combinations needed.

This makes the system flexible, scalable, and easy to extend.

Bridge Pattern Components Explained

Implementor

This is the interface that represents the implementation side. It tells what actions the actual device must support, like turning on or off, but doesn't care who is calling it. In our example, the *Device* interface defines these methods.

Concrete Implementor

These are the actual devices that implement the *Device* interface. Each device provides its real behavior. A TV turns on differently from a Radio, but both follow the same contract. This allows any remote to control any device.

Abstraction

This is the main class that represents what the system wants to do. It contains the core behavior and holds a reference to the implementation. In our remote example, the *Remote* class is the abstraction because it defines actions like turning a device on or off, without worrying about how the device actually performs these actions.

Refined Abstraction

These are the subclasses that add more specific features on top of the abstraction. They extend the *Remote* class and introduce extra behaviors. For example, an *AdvancedRemote* may add features like mute or voice control, while still using the same bridge to communicate with the device.

Together, the Abstraction (*Remote*) and Implementor (*Device*) work independently but stay connected through a bridge, allowing both sides to grow without affecting each other.

4. Code link:

[DesignPatterns/DesignPatterns/src/com/designpatterns/BridgePattern_RemoteSystem.java at main · CodingLyf-Fullstack/DesignPatterns](#)

5. Reference link: [Understanding the Bridge Design Pattern: A Comprehensive Guide - DEV Community](#)

341. Design Patterns 20: What is Decorator pattern and when to use it?

1. Real-World Analogy

Think of buying a product online. The base price is fixed, but at checkout we can add things like gift wrap, fast delivery, extended warranty, or discount coupons. The product itself never changes; we keep adding extra features around it.

Definition:

The Decorator Pattern allows us to add new behaviours to an object without changing its original code. We wrap the object inside other objects that add extra functionality step by step.

2. The Problem

In an e-commerce system, an order starts with the product price. Later, business teams request new features:

- Add GST
- Add packaging charge
- Add fast delivery fee
- Add festive discount
- Add membership discount

If we try handling this using inheritance, we end up with classes like:

```
OrderWithGST
OrderWithGSTAndPackaging
OrderWithFastDeliveryAndGST
OrderWithFastDeliveryAndPackagingAndDiscount
```

Every new combination means a new class. This becomes impossible to maintain. Even a small change forces modifications in multiple classes, increasing bugs and effort.

3. The Decorator Pattern Solution

The Decorator Pattern solves this by wrapping the base object with decorators. Each decorator adds one extra responsibility. We can stack multiple decorators as needed.

Main Components

Component: The basic interface (Order) that defines the core operation.

Concrete Component: The original object (BasicOrder) with the base price.

Decorator: A wrapper class that implements the same interface and holds a reference to the component.

Concrete Decorators: Extra features like GST, Packaging, Discount, Fast Delivery, each adding something new.

Step by Step Implementation**Step 1: Component**

```
interface Order {
    double cost();
}
```

```
}
```

Step 2: Concrete Component

```
class BasicOrder implements Order {  
    public double cost() {  
        return 500; // base price  
    }  
}
```

Step 3: Decorator

```
abstract class OrderDecorator implements Order {  
    protected Order order;  
  
    public OrderDecorator(Order order) {  
        this.order = order;  
    }  
}
```

Step 4: Concrete Decorators

```
class GSTDecorator extends OrderDecorator {  
    public GSTDecorator(Order order) {  
        super(order);  
    }  
  
    public double cost() {  
        return order.cost() + 50; // GST added  
    }  
}  
  
class FastDeliveryDecorator extends OrderDecorator {  
    public FastDeliveryDecorator(Order order) {  
        super(order);  
    }  
  
    public double cost() {  
        return order.cost() + 100; // delivery fee  
    }  
}
```

Step 5: Main

```
public class Decorator_Order {  
    public static void main(String[] args) {  
        Order order = new BasicOrder();  
        order = new GSTDecorator(order);  
        order = new FastDeliveryDecorator(order);  
  
        System.out.println("Total Cost: " + order.cost());  
    }  
}
```

Explanation:

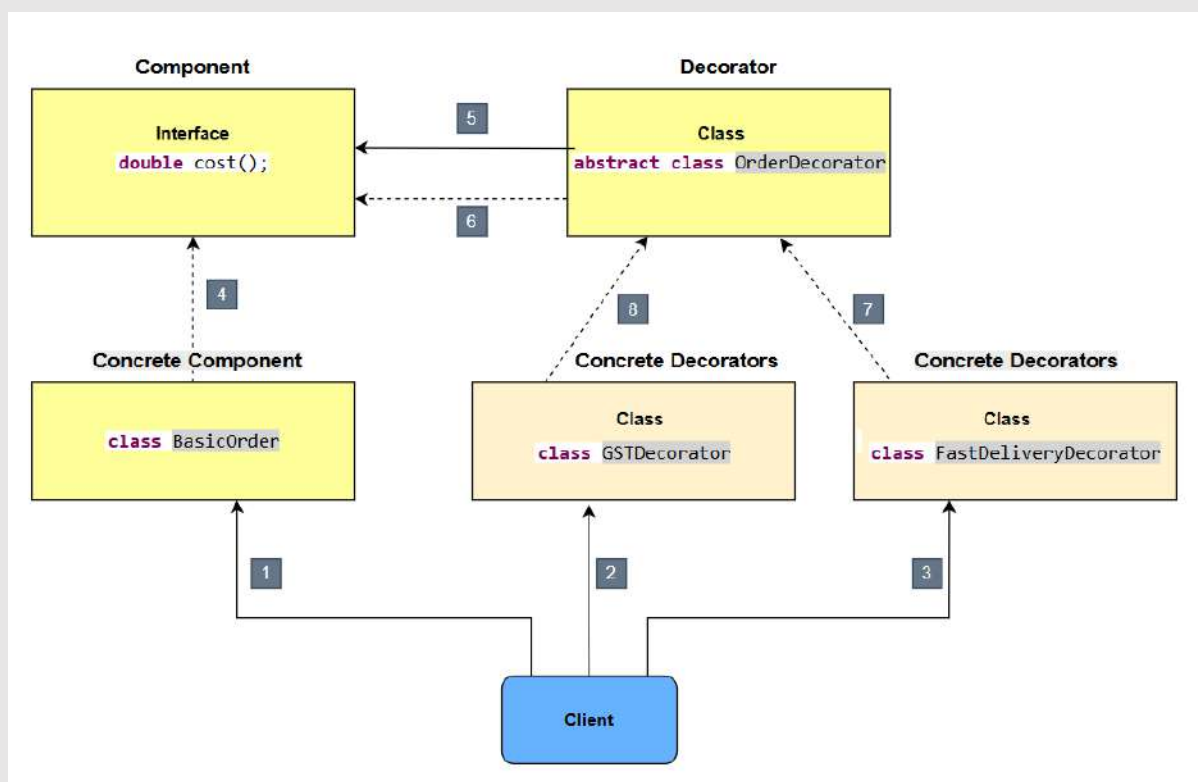
As This example shows how the Decorator Pattern helps us add extra charges to an order without changing the original order class.

The Order interface defines a method `cost()` that returns the price. `BasicOrder` is the normal order that costs 500. This class never changes.

To add new features like GST or fast delivery, we create decorator classes instead of modifying `BasicOrder`. Each decorator wraps the existing order and adds its own charge. `GSTDecorator` adds 50, and `FastDeliveryDecorator` adds 100.

In the main method, we start with a `BasicOrder`. Then we wrap it with `GSTDecorator` and later with `FastDeliveryDecorator`. Each wrapper adds its own cost, and the final total becomes 650.

This allows the system to keep adding features to an order one by one, without changing the original code or creating many subclasses.

**Explanations: Follow the numbers**

1. The Client first creates a `BasicOrder`. This is the original object that contains only the base price of the order. No extra features like GST or delivery charges are included yet.
2. The Client then wraps the basic order with `GSTDecorator`. This decorator adds GST to the cost without modifying the original `BasicOrder` class.
3. Next, the Client wraps the GST-added order with `FastDeliveryDecorator`. This decorator increases the price further by adding the fast delivery charges.

Interface and Class Relationships

4. (dotted line) BasicOrder implements the Order interface. This means **BasicOrder *is-a* Order**.

5. (solid line) OrderDecorator holds a reference to an Order object. This shows **OrderDecorator *has-a* Order**.

6. (dotted line) OrderDecorator implements the Order interface, so it behaves like an Order. This allows decorators to be used wherever a normal Order is expected.

7. (dotted line) FastDeliveryDecorator extends OrderDecorator. This means it inherits the and adds new functionality.

9. (dotted line) GSTDecorator also extends OrderDecorator. It adds GST charges on top of the wrapped order.

4. JDK Examples

These Java classes use the Decorator pattern:

- BufferedInputStream wrapping InputStream
- Collections.synchronizedList(list)

5. Code link: [DesignPatterns/DesignPatterns/src/com/designpatterns/Decorator_Order.java at main · CodingLyf-Fullstack/DesignPatterns](#)

6. Reference link: [Decorator Design Pattern Java | The Code Bean | Medium](#)

342. **Design Patterns 21: Differences between all the structural design patterns?**

This table helps to remember them easily

Pattern	Core Idea	What It Solves	How It Works	Real-World Analogy
Adapter	Convert one interface into another	Old system and new system can't talk to each other	Add a middle layer that translates calls	Charger plug adapter converts 2-pin to 3-pin
Bridge	Separate abstraction from implementation	Too many combinations lead to class explosion	Two hierarchies connected using a bridge	TV + Remote work independently but still connected
Composite	Work with single and group objects uniformly	Need tree-like structures	Treat leaf and composite in the same way	Folder with files and folders inside
Decorator	Add features without changing original code	Need dynamic and optional features	Wrap the object and add behavior	Adding cheese, toppings to pizza

Pattern	Core Idea	What It Solves	How It Works	Real-World Analogy
Facade	Provide a simple entry point to a complex system	Too many steps or subsystems	Create one class that hides complex interactions	Travel agent books flight + hotel + cab
Flyweight	Share objects to save memory	Too many similar objects waste memory	Divide data: shared (intrinsic) + unique (extrinsic)	Library keeps one book copy, many borrow it
Proxy	Control access to another object	Add security, caching, lazy loading	Stand-in object controls the original	Security guard checks before someone enters

If it's about making incompatible things work together - Adapter

If it's about separating abstraction from implementation- Bridge

If it's about treating part and whole the same way - Composite

If it's about adding features without modifying existing code - Decorator

If it's about hiding complex subsystems behind one simple interface - Facade

If it's about sharing objects to save memory - Flyweight

If it's about controlling access to another object - Proxy

Behavioural Design Pattern

In this section, we will see the important design patterns. Remaining we will cover in the future releases.

Important behavioral design patterns include **Observer**, **Strategy**, and **Command**, which are frequently used for communication and task management between objects. Other key patterns are **State** (altering an object's behavior when its state changes), **Iterator** (traversing a collection), and **Chain of Responsibility** (passing a request along a chain of handlers) and **Template**.

343. Design Patterns 22: What is Observer Pattern and when to use it?

Imagine a stock market app. When a company's stock price changes, every registered user who follows that stock instantly sees the update on their phone. The user doesn't repeatedly check the price; the system notifies them whenever something changes.

Definition:

The Observer design pattern is a behavioural design pattern, used to create a one-to-many dependency

between objects so that when one object (the subject) changes its state, all its dependents (observers) are notified and updated automatically.

2. The Problem

Consider an e-commerce application. When a payment is processed, several systems must react:

- The order service must mark the order as completed
- The inventory service must reduce stock
- The notification service must send an email or SMS
- The analytics system must log the transaction

If we call each of these services manually, the payment module ends up like this:

```
payment.updateOrder();  
payment.updateInventory();  
payment.sendNotification();  
payment.logAnalytics();
```

This creates **tight coupling**.

If a new listener is added tomorrow, we must modify the payment module again. The module becomes harder to maintain and test.

We need a way where the payment service just announces the event once, and whoever is interested responds automatically.

3. Observer Pattern Solution

The Observer Pattern solves this by separating the object that generates changes (Subject) from the objects that react to those changes (Observers).

Main Components

Subject (interface):

Defines methods to add observers and send updates. Any class that wants to be observed must implement this.

Concrete Subject (PaymentService class):

Implements the Subject interface. Its state or action changes. When `makePayment()` is called, it notifies all registered observers.

Observer (interface):

Defines the update method. Any class that wants to react to changes must implement this.

Concrete Observers (EmailService, InventoryService classes):

Implement the Observer interface. They react when the subject notifies them.

EmailService sends an email, InventoryService updates stock, both based on the payment status.

Implementation Steps

Step 1: Define the Observer Interface

```
interface Observer {  
    void update(String status);  
}
```

Step 2: Create the Subject (Observable) Interface

```
interface Subject {  
    void addObserver(Observer observer);  
  
    void notifyObservers(String message);  
}
```

Step 3: Implement the Concrete Subject

```
class PaymentService implements Subject {  
    private List<Observer> observers = new ArrayList<>();  
  
    public void addObserver(Observer o) {  
        observers.add(o);  
    }  
  
    public void notifyObservers(String status) {  
        for (Observer o : observers) {  
            o.update(status);  
        }  
    }  
  
    public void makePayment() {  
        System.out.println("Payment has been completed");  
        notifyObservers("SUCCESS");  
    }  
}
```

Step 4: Implement the Concrete Observers

```
class EmailService implements Observer {  
    public void update(String status) {  
        System.out.println("Email sent with status: " + status);  
    }  
}  
  
class InventoryService implements Observer {  
    public void update(String status) {  
        System.out.println("Inventory updated for status: " + status);  
    }  
}
```

Step 5.Main

```
public class ObserverDemo {  
    public static void main(String[] args) {  
        PaymentService payment = new PaymentService();  
  
        payment.addObserver(new EmailService());  
        payment.addObserver(new InventoryService());  
  
        payment.makePayment();  
    }  
}
```

Explanation:

This example shows how the Observer Pattern helps different parts of a system react automatically when a payment is completed.

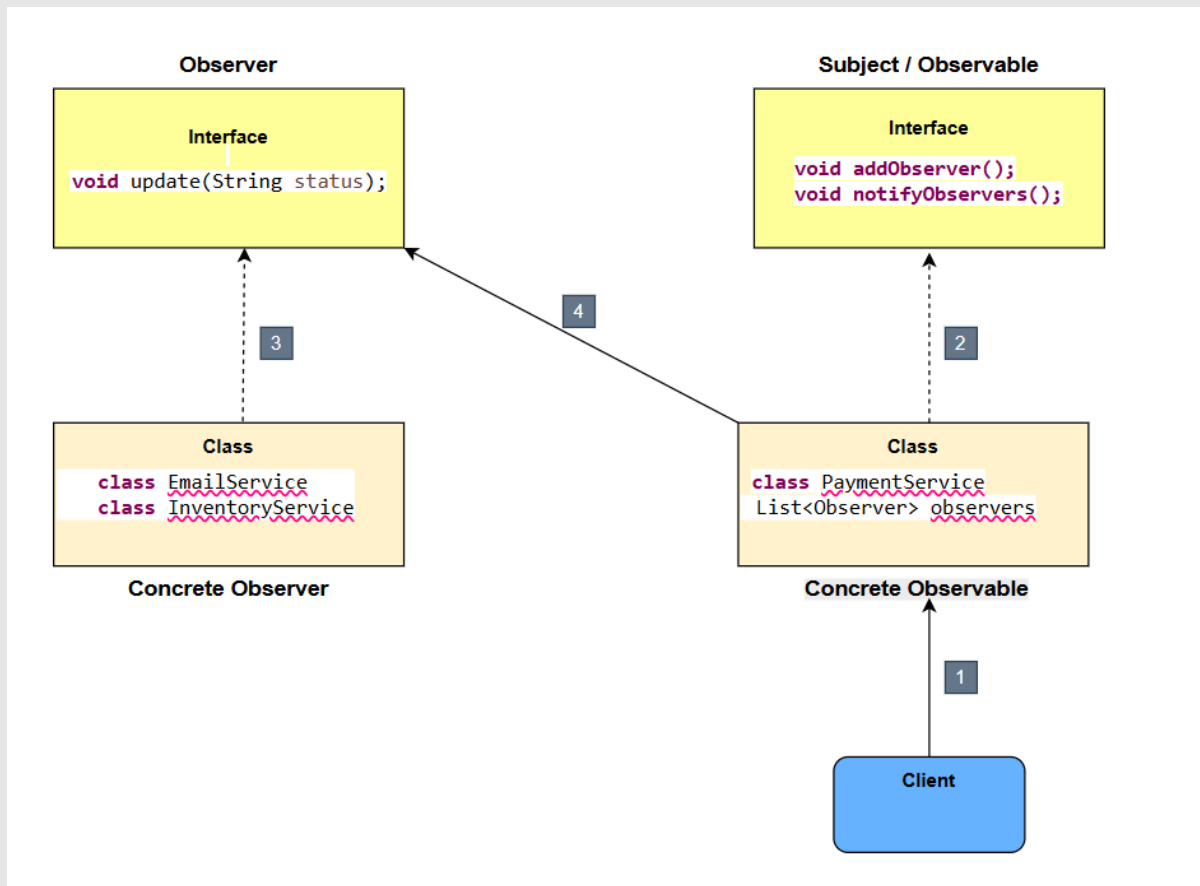
The *PaymentService* is the **Subject**. It keeps a list of observers and informs them whenever a payment is done. It does not know what each observer does; it simply sends a message.

Observer is an interface that defines the method *update()*. Any class that wants updates must implement this method.

EmailService and *InventoryService* are **Concrete Observers**. They react to the payment status by sending an email and updating the inventory, respectively.

In the main method, we register both observers with *PaymentService*. When *makePayment()* is called, the payment service prints a message and then notifies both observers. Each observer receives the update and performs its own action.

This keeps the system flexible. If we add a new observer, like an analytics logger, we don't need to modify the payment code. It just subscribes and starts receiving updates.



Explanation of the Flow

1. The client creates the `PaymentService` object and starts the payment execution. This means the client **uses** the concrete observable.
2. `PaymentService` implements the `Subject` interface. This allows it to add observers and notify them. The dotted line shows that `PaymentService` **is-a Subject**.
3. `EmailService` and `InventoryService` implement the `Observer` interface. They provide their own behavior for the `update()` method. The dotted line shows each service **is-an Observer**.
4. The solid line from `PaymentService` to the `Observer` interface shows that the payment service **has-a list of Observer** objects. When the payment is successful, it calls `update()` on each registered observer.

4. JDK Examples

Observer Pattern is used heavily in Java:

- `java.util.Observer` and `Observable` classes
- `EventListener` in Swing/AWT event handling
- Spring's `ApplicationEventPublisher` for event-driven programming

5. When To Use Observer Pattern

Use it when:

- One change should automatically update multiple parts of the system
- New reactions may be added without modifying existing code

6. Code Link: [DesignPatterns/DesignPatterns/src/com/designpatterns/ObserverDemo.java at main · CodingLyf-Fullstack/DesignPatterns](https://github.com/designpatterns/ObserverDemo.java)

7.Reference Link: [Observer Design Pattern in Java - DEV Community](#)

[Observer Design Pattern Java | The Code Bean | Medium](#)

344. **Design Patterns 23: What is Strategy Pattern and when to use it?**

1.Real-World Analogy

Think of choosing a route in Google Maps. We want to go from point A to point B. The app allows different ways: fastest route, no-toll route, bike route, or scenic route. The destination stays the same, but the *strategy* to reach it changes based on our preference.

Definition:

The Strategy Pattern allows us to select an algorithm at runtime. Instead of writing multiple if/else conditions to decide which logic to use, each algorithm is placed in a separate class, and we pick the one we want when executing the action.

2. The Problem

A payment system in an eCommerce application must support multiple payment modes:

- Credit card payment
- UPI payment
- Net banking
- Wallet payment

Initially, we might write code like:

```
if (mode.equals("CARD")) {  
    processCard();  
} else if (mode.equals("UPI")) {  
    processUPI();  
} else if (mode.equals("WALLET")) {  
    processWallet();  
}
```

As more payment options are added, this class becomes huge. Every new feature forces us to modify existing code. Testing becomes painful, and adding a new payment type involves touching multiple places in the code. This violates the Open/Closed Principle.

We need a design where payment methods can be plugged in without changing the payment code.

3. Strategy Pattern Solution

The Strategy Pattern moves each payment method into its own class. The main payment processor does not care how the payment is handled. It only calls the selected strategy.

Main Components

Strategy Interface

Defines a common interface for all concrete strategies. This interface declares the method(s) to be called.

Concrete Strategies

Implement the Strategy interface, providing specific implementations of the algorithm. Each concrete strategy represents a different way of performing the same operation. Separate classes for credit card, UPI, wallet, each implementing the same interface.

Context

Uses a strategy and executes it. The context decides *which* strategy to use at runtime.

Implementation Steps

Step 1: Strategy

```
interface PaymentStrategy {  
    void pay(double amount);  
}
```

Step 2: Concrete Strategies

```
class CardPayment implements PaymentStrategy {  
    public void pay(double amount) {  
        System.out.println("Paid " + amount + " using Card");  
    }  
}  
  
class UpiPayment implements PaymentStrategy {  
    public void pay(double amount) {  
        System.out.println("Paid " + amount + " using UPI");  
    }  
}
```

Step 3: Context

```
class PaymentProcessor {  
    private PaymentStrategy strategy;  
  
    public PaymentProcessor(PaymentStrategy strategy) {  
        this.strategy = strategy;  
    }  
  
    public void process(double amount) {
```

```
        strategy.pay(amount);  
    }  
}
```

Step 4: Main

```
public class StrategyPattern_Payment {  
    public static void main(String[] args) {  
        PaymentProcessor processor = new PaymentProcessor(new CardPayment());  
        processor.process(1200);  
  
        processor = new PaymentProcessor(new UpiPayment());  
        processor.process(600);  
    }  
}
```

Explanation:

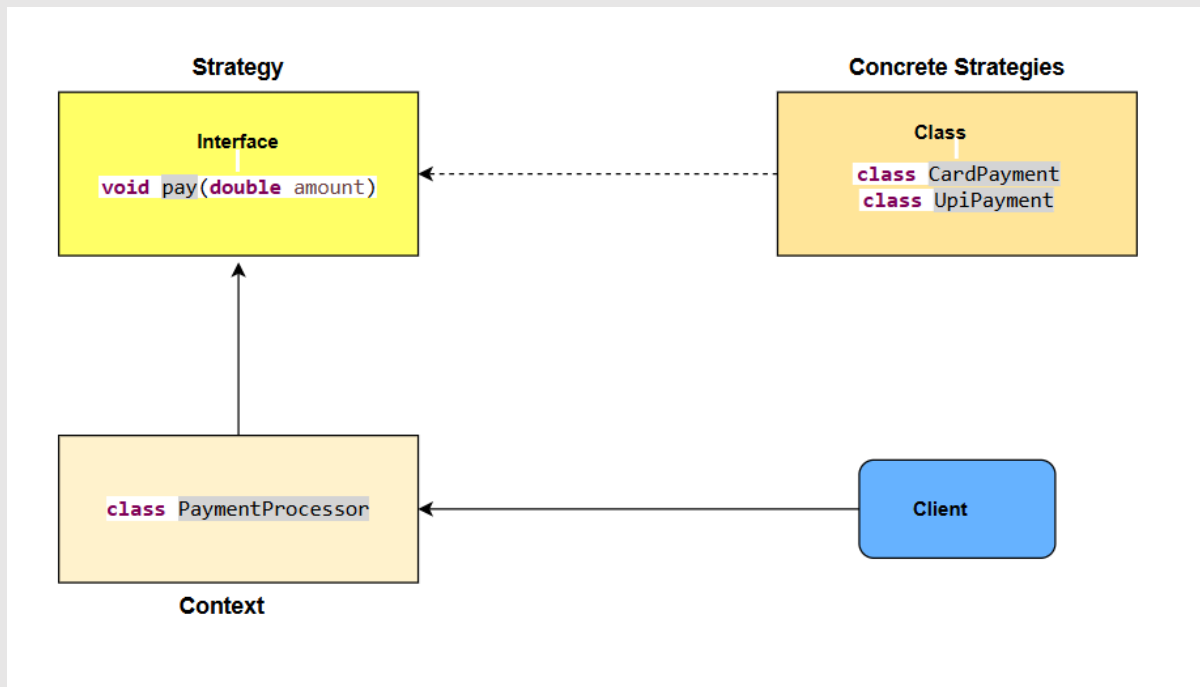
This program shows how different payment methods can be used without changing the main payment code. The PaymentStrategy interface defines a common method called pay(). Any payment method must implement this.

CardPayment and UpiPayment are two separate classes that implement the strategy. Each one has its own way of paying. One prints that payment happened using a card, and the other prints payment using UPI.

PaymentProcessor is the class that performs the payment. It doesn't know how the payment works internally. It only calls the pay() method of whatever strategy it receives. This makes it easy to switch payment methods.

In the main method, we first pass a CardPayment strategy to the processor, so the payment happens using a card. Then we replace it with a UpiPayment strategy, and the payment happens using UPI. The processor stays the same, only the strategy changes.

This removes the need for if/else conditions and makes the code easy to extend when a new payment method is added.



The PaymentProcessor (Context) doesn't know how the payment is done; it only calls a method defined in the PaymentStrategy interface. Different classes like CardPayment and UpiPayment implement this interface and provide their own way of paying. The Client chooses which payment method to use, and the processor follows that strategy without changing its code.

The **solid arrow** from Client to PaymentProcessor shows that the client uses the processor. The **solid arrow** from PaymentProcessor to the Strategy interface means the processor has a reference to a payment strategy. The **dotted arrow** from CardPayment and UpiPayment to the Strategy interface means these classes implement the strategy and provide their own way of paying.

PaymentProcessor **has-a** PaymentStrategy, while CardPayment **is-a** PaymentStrategy.

4. JDK Examples

Strategy Pattern is used widely in Java:

- Comparator interface for sorting with different comparison logic
- Collections.sort() where we pass different strategies
- ThreadPoolExecutor uses different Rejection policies
- javax.crypto.Cipher supports multiple encryption strategies

5. When To Use Strategy Pattern

Use it when:

- The behavior of a class must change based on context
- You have many conditional statements selecting different logic
- You want to add new behavior without modifying existing code

6. Code Link:

7.Reference Link:

345. Design Patterns 24: What is Command Pattern and when to use it?

1.Real-World Analogy

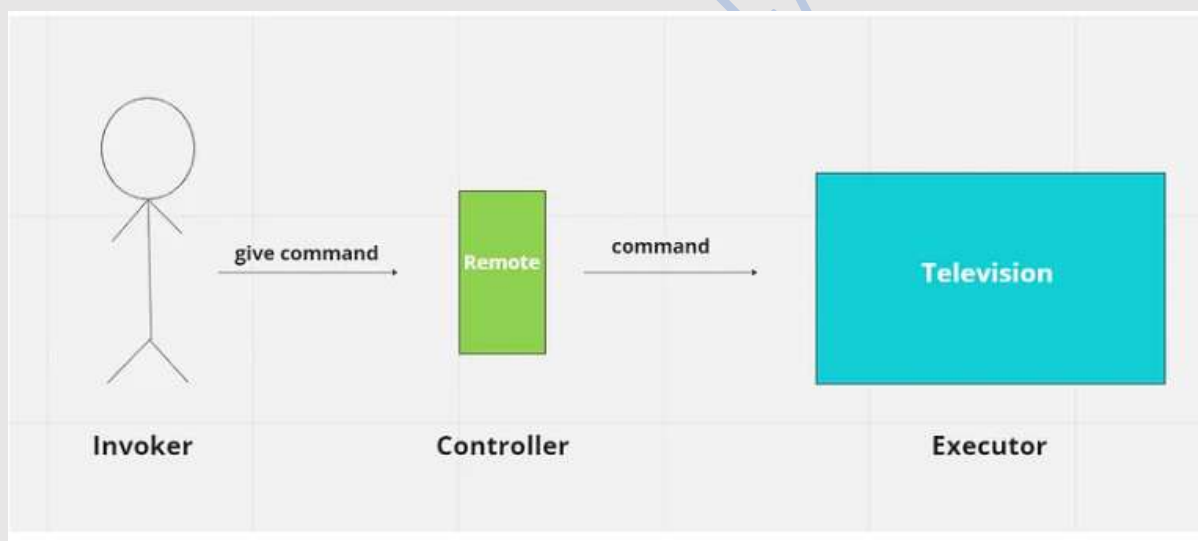
Think about a TV remote. When we press a button to turn on the TV or change the channel, we do not know how the TV actually performs those actions. We only trigger a request.

- **We** are the one giving the request (sender)
- **Remote** takes our request and decides which button/command to run (invoker)
- **TV** performs the action (receiver)

The important part is this:

We never talk to the TV directly. We talk to the remote, and the remote sends the correct command to the TV.

This is exactly what the **Command Pattern** does. It separates the person who requests an action from the object that performs it. Because of this, we can add new commands or change how things work without modifying the sender. It keeps the system flexible and easy to maintain.



Definition:

The Command Pattern turns a request into a separate object, so we can store it, pass it, or execute it whenever we want. It separates the code that asks for an action from the code that performs the action.

2. Another example

A **real-life example** of the Command Pattern can be found in a restaurant ordering system. In this system, a waiter takes orders from customers and sends them to the kitchen for preparation. The waiter doesn't prepare the food himself but delegates the responsibility to the kitchen staff.

In a Java implementation of the Command Pattern for the restaurant ordering system.

- The waiter would act as an *invoker* object
- While the kitchen staff would act as *receiver* objects.
- Each order would be encapsulated as a command object that contains information about the food to be prepared and the receiver object responsible for preparing it.

3. Command Pattern Solution

Let's implement Loan processing application using Command pattern.

In a banking system, many actions are performed on a loan such as approving it, rejecting it, verifying documents, or sending notifications. The system that triggers these actions doesn't need to know how each action works internally; it simply requests that the action be performed.

In the Command Pattern:

- The **LoanWorkflowEngine** acts like the waiter (Invoker)
- The **LoanService** acts like the kitchen staff (Receiver)
- Each command class, such as **ApproveLoanCommand** or **RejectLoanCommand**, represents a separate loan action (Concrete Commands)

Implementation

Step 1. Command Interface: It contains all the basic operations of particular command

```
interface LoanCommand {  
    void execute();  
}
```

Step 2. Concrete Commands: Each command by implementing command interface

```
class ApproveLoanCommand implements LoanCommand {  
    private LoanService service;  
    private String loanId;  
  
    public ApproveLoanCommand(LoanService service, String loanId) {  
        this.service = service;  
        this.loanId = loanId;  
    }  
  
    public void execute() {  
        service.approveLoan(loanId);  
    }  
}  
  
class RejectLoanCommand implements LoanCommand {  
    private LoanService service;  
    private String loanId;  
  
    public RejectLoanCommand(LoanService service, String loanId) {  
        this.service = service;  
    }  
}
```

```

        this.loanId = loanId;
    }

    public void execute() {
        service.rejectLoan(loanId);
    }
}

```

Step 3: Receiver: Class that receives and process the commands from invoker

```

class LoanService {
    public void approveLoan(String loanId) {
        System.out.println("Loan Approved: " + loanId);
    }

    public void rejectLoan(String loanId) {
        System.out.println("Loan Rejected: " + loanId);
    }
}

```

Step 4: Invoker: Who calls the command

```

class LoanWorkflowEngine {
    private List<LoanCommand> commands = new ArrayList<>();

    public void addCommand(LoanCommand command) {
        commands.add(command);
    }

    public void processCommands() {
        for (LoanCommand command : commands) {
            command.execute();
        }
        commands.clear();
    }
}

```

5. Main

```

public class CommandPattern_LoadApp {
    public static void main(String[] args) {

        // Usage example
        LoanService service = new LoanService();
        LoanWorkflowEngine engine = new LoanWorkflowEngine();

        engine.addCommand(new ApproveLoanCommand(service, "LN001"));
        engine.addCommand(new RejectLoanCommand(service, "LN002"));

        engine.processCommands();
    }
}

```

Explanation:

LoanService is the **Receiver**.

It contains the real business logic. When someone asks to approve or reject a loan, this class actually performs that work.

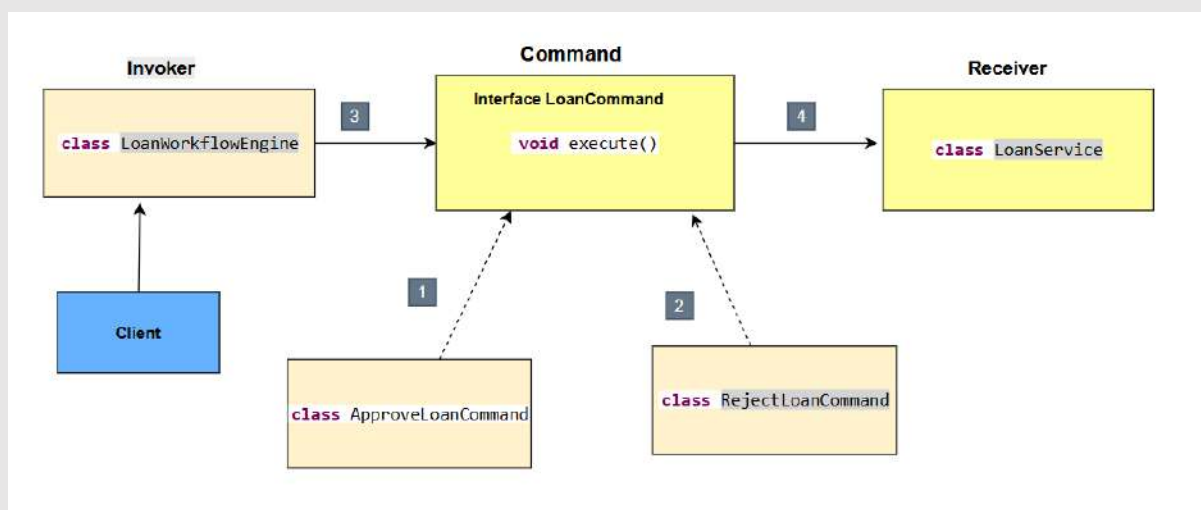
ApproveLoanCommand and **RejectLoanCommand** are **Command** objects.

Each one represents a single action. Instead of directly calling LoanService methods, the action is wrapped inside a command. When executed, the command tells LoanService what to do.

LoanWorkflowEngine is the **Invoker**.

It does not know how a loan is approved or rejected. It only knows how to trigger commands. It stores commands, and when asked, runs them one by one.

The **Client** creates the appropriate command objects (like ApproveLoanCommand) and hands them over to the LoanWorkflowEngine. The client decides *what* needs to be done, but not *how* it is done.

**1. Concrete Commands Implement the Command Interface (Dotted Lines)**

- ApproveLoanCommand implements LoanCommand
- This dotted line shows "implements" relationship, meaning the command class follows the structure of the execute() method defined in the interface.

2. Another Concrete Command Also Implements the Same Command Interface (Dotted Line)

- RejectLoanCommand also implements LoanCommand
- Since both commands implement the same interface, they provide different behaviors for the same execute() action.

3. Invoker Uses the Command Interface (Solid Line)

- LoanWorkflowEngine does not know the concrete command
- It only knows the LoanCommand interface
- The solid line shows "uses/depends on" relationship.

- The invoker calls `execute()` without caring which command is actually running.

4. Command Calls the Receiver (*Solid Line*)

- Inside the each command's `execute()` method, it calls methods on `LoanService`
- This solid line shows the "**has a**" relationship
- The receiver contains the real business logic like approving or rejecting a loan

4. JDK Examples

When to use the Command Pattern

1. To separate the requester from the executor

Use it when the object that asks for something to be done should not know who actually does it or how it is done. This keeps the code flexible, because the invoker can work with any command without changing itself.

2. To support undo/redo features

Since each action is a command object, we can store the commands in a history list. To undo, we reverse the command; to redo, we run it again.

3. To queue or schedule tasks

When operations need to be executed later or in a specific sequence, commands can be stored in a queue and executed whenever required. This is useful in workflow systems, job schedulers, and batch processing.

5. Code Link: [DesignPatterns/DesignPatterns/src/com/designpatterns/CommandPattern_LoadApp.java at main · CodingLyf-Fullstack/DesignPatterns](#)

6.Reference Link: [Command design patterns in Behavioral patterns with real life examples using Java. | by Gagan Jain | Medium](#)

346. **Design Patterns 25: What is State Pattern and when to use it?**

1.Real-World Analogy

Think about a **traffic signal**. It changes from **Red > Yellow > Green**, and each light controls how vehicles behave. The same signal box behaves differently based on its current state. We don't replace the signal box; only its **state** changes.

Definition:

The **State Pattern** does the same thing in software. It allows an object to change its behaviour when its internal state changes. Instead of writing long if-else or switch cases, each state is represented as a separate class.

2. The Problem

Consider an **Order Management System** in an e-commerce application.

An order goes through different stages:

- New
- Packed
- Shipped
- Delivered
- Cancelled

Initially, developers create code like this:

```
public void updateOrder(String status) {  
    if(status.equals("NEW")) { /* allow cancel */ }  
    else if(status.equals("PACKED")) { /* allow ship */ }  
    else if(status.equals("SHIPPED")) { /* allow track */ }  
    else if(status.equals("DELIVERED")) { /* no changes allowed */ }  
    else if(status.equals("CANCELLED")) { /* refund */ }  
}
```

Why this is a problem

- Too many if/else conditions
- Hard to test, debug, and extend
- Developers end up modifying existing code again and again

As the business grows, more states are added, like ReturnRequested, Returned, Refundedm which makes the code a maintenance difficult.

3. State Pattern Solution

The State Pattern removes these conditional checks by moving logic into separate state classes.

So:

- **Each state** knows what actions are allowed and which state should come next
- The **main object** (Order) doesn't decide anything. It just asks its current state to handle the request

This makes the code clean and easy to change.

Component Meaning

State Interface	A common behaviour for all states. Every state class must follow this and implement the required methods.
Concrete States	Actual classes that represent different states (New, Packed, Shipped, Delivered). Each one has its own behaviour and decides what the next state can be.

Component Meaning

Context The main object whose behaviour changes based on its current state. It stores the
(Order) current state and forwards work to that state instead of handling it itself.

Implementation:**Step1: State Interface**

```
interface OrderState {  
    void next(OrderContext ctx);  
    void printStatus();  
}
```

Step2: Concrete States

```
class NewState implements OrderState {  
  
    public void next(OrderContext ctx) {  
        ctx.setState(new PackedState());  
    }  
  
    public void printStatus() {  
        System.out.println("Order placed, waiting for packing.");  
    }  
}  
  
class PackedState implements OrderState {  
  
    public void next(OrderContext ctx) {  
        ctx.setState(new ShippedState());  
    }  
  
    public void printStatus() {  
        System.out.println("Order packed and ready for shipment.");  
    }  
}
```

Step3: Context

```
class OrderContext {  
    private OrderState state;  
  
    public OrderContext(OrderState state) {  
        this.state = state;  
    }  
  
    public void setState(OrderState state) {
```

```

        this.state = state;
    }

    public void nextStates() {
        state.next(this);
    }

    public void printStatus() {
        state.printStatus();
    }
}

```

Step 4: Main

```

public class StatePattern_OrderApp {
    public static void main(String[] args) {
        OrderContext order = new OrderContext(new NewState());
        order.printStatus();

        order.nextState();
        order.printStatus();

        order.nextState();
        order.printStatus();
    }
}

```

Explanation:

OrderContext represents an order. The order behaves differently based on what stage it is in. Instead of writing multiple if/else checks, we give each state its own class to define how the order should act.

- When an order is created, it starts in **NewState**
- next() moves the order to the next stage
- prev() moves it back to the previous stage
- printStatus() displays what is happening in the current stage

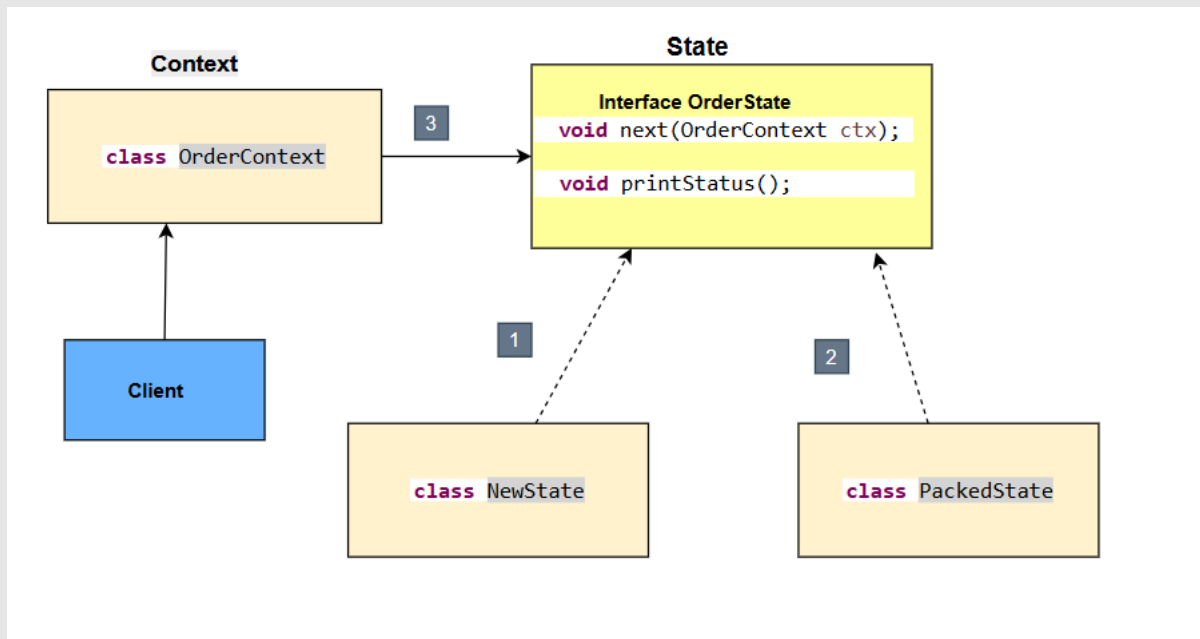
Each state class (NewState, PackedState, ShippedState, DeliveredState) knows what it should do and where it can move next.

Simple Flow

1. **NewState**
The order is placed and waiting to be packed, calling next() changes state to **PackedState**
2. **PackedState**
The order is packed and ready to ship, calling next() moves to **ShippedState**
3. **ShippedState**
The order is on the way, calling next() moves to **DeliveredState**

4. DeliveredState

Order is delivered and cannot move forward



1. The NewState Class Implements the OrderState Interface (Dotted Line)

The dotted line means **implementation**.

NewState follows the rules defined by OrderState, so it provides its own version of `next()` and `printStatus()`.

2. The PackedState Class Also Implements the OrderState Interface (Dotted Line)

Again, the dotted line shows implementation.

PackedState represents another stage of the order and has its own behavior for `next()` and `printStatus()`.

3. The OrderContext Uses OrderState (Solid Line)

The solid line means **dependency or usage or has-a relation**.

OrderContext holds a reference to the current state. Instead of writing conditions like:

```
if(new) do this; else if(packed) do that;
```

it delegates work to the current state through OrderState.

4. JDK Examples

Java uses State Pattern concepts in many places:

- Thread class (NEW, RUNNABLE, BLOCKED, TERMINATED)

5. When to use

Use it when:

- An object must behave differently based on its current state

- We want to avoid long if/else or switch-case blocks
- The system has clear state transitions (Order, Booking, Workflow)
- New states may be added in the future

6. Code Link: [DesignPatterns/DesignPatterns/src/com/designpatterns/StatePattern_OrderApp.java at main · CodingLyf-Fullstack/DesignPatterns](#)

7.Reference Link: [Mastering the State Design Pattern in Java: A Detailed Guide with Examples | by Ahmet Temel Kundupoglu | Medium](#)

Video link: [State Design Pattern in detail | Interview Question](#)

347. Design Patterns 26: What is Iterator Pattern and when to use it?

1.Real-World Analogy

Imagine we have a bookshelf with many books. We want to read one book after another, without opening the shelf and digging into how books are stored. We just need a simple way to move from the first book to the next, check if books are left, and stop when done.

Definition

Iterator Pattern gives a step-by-step way to access items inside a collection without exposing how that collection is built on the inside.

2. The Problem

In a library management system, a BookCollection class stores hundreds of books, Users want to browse through a collection of books one by one.

Without Iterator, we have to access the Internal structure of the collection and traverse through the list.

Problematic Code:

```
class Book {
    private String title;
    public Book(String title) {
        this.title = title;
    }
    public String getTitle() {
        return title;
    }
}

class BookCollection {
    // Internal storage exposed
    public Book[] books = new Book[5];
    private int size = 0;

    public void add(Book book) {
        books[size++] = book;
    }
}
```

```

public class LibraryClient {
    public static void main(String[] args) {
        BookCollection collection = new BookCollection();
        collection.add(new Book("Clean Code"));
        collection.add(new Book("Effective Java"));
        collection.add(new Book("Design Patterns"));

        // Client code must handle array, null checks and indexes
        for (int i = 0; i < collection.books.length; i++) {
            if (collection.books[i] != null) {
                System.out.println(collection.books[i].getTitle());
            }
        }
    }
}

```

The problem starts when the BookCollection exposes how it stores its books. Right now, the internal storage is an array, and everyone who wants to read the books must directly use that array. This forces the client code to understand how the books are arranged inside the collection. It must loop through the array by worrying about indexes, and null values.

3. Iterator Pattern Solution

Iterator Pattern, where a dedicated iterator will provide controlled access to the book collection. Iterator Pattern hides the internal structure of a collection and gives a clean way to go through elements.

Components and Implementation:

1. Iterator (Interface)

The Iterator interface provides methods to traverse through a collection. It includes:

- hasNext(): Ensures iteration stops when no elements are left.
- next(): Retrieves the next available element.

```

interface Iterator<T> {
    boolean hasNext();
    T next();
}

```

2. Concrete Iterator (BookIterator)

This is the actual class that implements the iterator logic for a specific collection. It maintains a position index to track the current element.

```

class BookIterator implements Iterator<Book> {
    private List<Book> books;
    private int position = 0;

    public BookIterator(List<Book> books) {
        this.books = books;
    }
}

```

```

    }
    @Override
    public boolean hasNext() {
        return position < books.size();
    }
    @Override
    public Book next() {
        return books.get(position++);
    }
}

```

3. Aggregate / Collection

```

interface BookCollection {
    Iterator<Book> createIterator();
}

```

4. Concrete Aggregate

The **Concrete Aggregate** implements the **Aggregate** interface and holds a list of books. It provides an iterator to traverse the books. This is the class that holds the items. It does not allow outsiders to touch its internal structure. Instead, it creates and returns an iterator object whenever someone wants to go through its items.

```

class Library implements BookCollection {
    private List<Book> books = new ArrayList<>();

    public void addBook(Book book) {
        books.add(book);
    }
    @Override
    public Iterator<Book> createIterator() {
        return new BookIterator(books);
    }
}

```

5.Main

```

public class Iterator_BookCollection {
    public static void main(String[] args) {
        Library library = new Library();
        library.addBook(new Book("Design Patterns"));
        library.addBook(new Book("Clean Code"));
        library.addBook(new Book("Effective Java"));

        Iterator<Book> iterator = library.createIterator();

        System.out.println("Books in the library:");
        while (iterator.hasNext()) {
            System.out.println(iterator.next().getTitle());
        }
    }
}

```

```
}
```

Explanation:

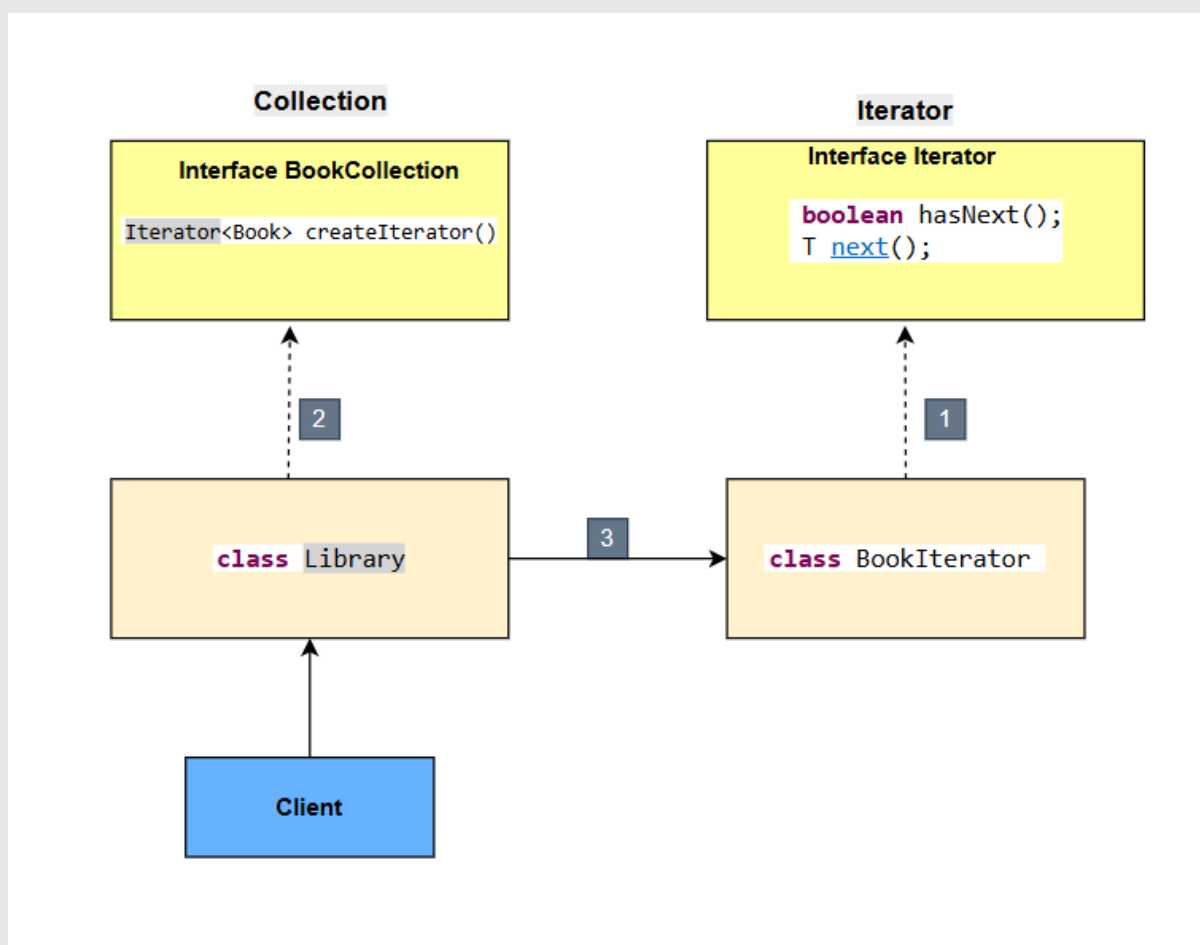
The *Library* stores books internally but does not allow direct access to its list.

Instead, it provides an iterator through the *createIterator()* method.

The *BookIterator* knows how to move through the list using *hasNext()* and *next()*.

The client uses the iterator to read book titles one by one without knowing how books are stored.

If the internal storage changes later, the client code remains the same because it depends only on the iterator.

**1. BookIterator implements Iterator Interface**

The dotted line means **implementation**.

BookIterator provides the logic for `hasNext()` and `next()` methods defined in the Iterator interface. It knows how to walk through the book collection, one book at a time.

2. Library implements BookCollection Interface

This dotted line also shows **implementation**.

Library provides the `createIterator()` method defined in BookCollection.

It decides how the books are stored internally but does not expose that structure.

3. Library creates BookIterator

The solid line shows a **dependency or has-a relationship**.

When the client asks for an iterator, the Library creates a BookIterator and returns it.

This allows the client to iterate over books without knowing how they are stored.

4. JDK Examples

Java uses Iterator Pattern everywhere in collections

- `java.util.Iterator`
- `List.iterator()`
- `Set.iterator()`
- `Map.keySet().iterator()`

5. When to use

Use Iterator Pattern when

- We want to access items of a collection without knowing its internal structure
- To support multiple traversal methods: When different data structures require different traversal strategies but you still want a single interface for the client to use.

6. Code Link: [DesignPatterns/DesignPatterns/src/com/designpatterns/Iterator/BookCollection.java at main · CodingLyf-Fullstack/DesignPatterns](https://github.com/CodingLyf/DesignPatterns/blob/main/src/com/designpatterns/Iterator/BookCollection.java)

7.Reference Link: [Iterator Design Pattern in Java. Introduction | by Kalana De Silva | Medium](#)

348. **Design Patterns 27: What is Chain of responsibility Pattern and when to use it?**

1.Real-World Analogy

A company receives a leave request. The employee sends it to their team lead. If the team lead has the authority, the request is approved. If not, the request moves to the project manager. If the project manager cannot approve, it goes to HR. The request travels step by step until someone handles it.

Definition

Chain of Responsibility Pattern allows a request to move through a chain of handlers. Each handler decides either to process the request or pass it to the next handler. The sender does not know who will actually handle the request.

2. The Problem

In an enterprise application, imagine a logging system where different log levels exist: INFO, DEBUG, ERROR. The code for logging messages becomes messy because every log call needs multiple checks. Developers end up writing code like this everywhere:

```
public class LoggerService {  
    public void log(String level, String message) {
```

```

    if (level.equals("INFO")) {
        System.out.println("INFO: " + message);
    } else if (level.equals("DEBUG")) {
        System.out.println("DEBUG: " + message);
    } else if (level.equals("ERROR")) {
        System.out.println("ERROR: " + message);
    }
    // More levels keep coming
    // Code grows forever
}
}

```

Problems with this approach

- The service becomes difficult to maintain
- Adding a new log level requires modifying existing code
- All conditions sit in one place, creating a long chain of if-else checks
- Hard to extend and test individually

3. Chain of Responsibility Pattern Solution

Chain of Responsibility divides the work into multiple handlers. Each handler processes the request it understands and sends the rest to the next handler.

Components and Implementation:

1. **Handler:** The Handler is an interface or abstract class that defines the common interface for handling requests. It typically includes a method like *handleRequest()*.
2. **ConcreteHandler:** ConcreteHandler classes implement the Handler interface and provide specific implementations for handling requests. Each ConcreteHandler has a reference to the next handler in the chain. It decides whether to handle the request or pass it to the next handler in the chain.
3. **Client:** The Client initiates the request and starts the chain of handling. It is responsible for creating the chain of handlers and linking them together.

Implementations

Step 1: Let's create our request handler first that is going to handle our requests and flow.

```

interface LoggerHandler {
    void setNextHandler(LoggerHandler nextHandler);

    void handleRequest(String level);
}

```

Step 2: Now it is time to make our concrete handles and, in this example, our concrete handles are InfoLogger and DebugLogger.

```

class InfoLogger implements LoggerHandler {

```

```

    LoggerHandler nextHandler;

    @Override
    public void setNextHandler(LoggerHandler nextHandler) {
        this.nextHandler = nextHandler;
    }

    @Override
    public void handleRequest(String level) {
        if("INFO".equals(level)) {
            System.out.println("Info logger");
        } else {
            this.nextHandler.handleRequest(level);
        }
    }
}

class DebugLogger implements LoggerHandler {

    LoggerHandler nextHandler;

    @Override
    public void setNextHandler(LoggerHandler nextHandler) {
        this.nextHandler = nextHandler;
    }

    @Override
    public void handleRequest(String level) {
        if("DEBUG".equals(level)) {
            System.out.println("DEBUG logger");
        } else {
            this.nextHandler.handleRequest(level);
        }
    }
}

```

Step 3: Let's create Main class that is responsible for generating requests and giving them to request handler.

```

public class ChainOfResponsibility {

    public static void main(String[] args) {
        LoggerHandler info = new InfoLogger();
        LoggerHandler debug = new DebugLogger();

        info.setNextHandler(debug);

        info.handleRequest("ERROR");
    }
}

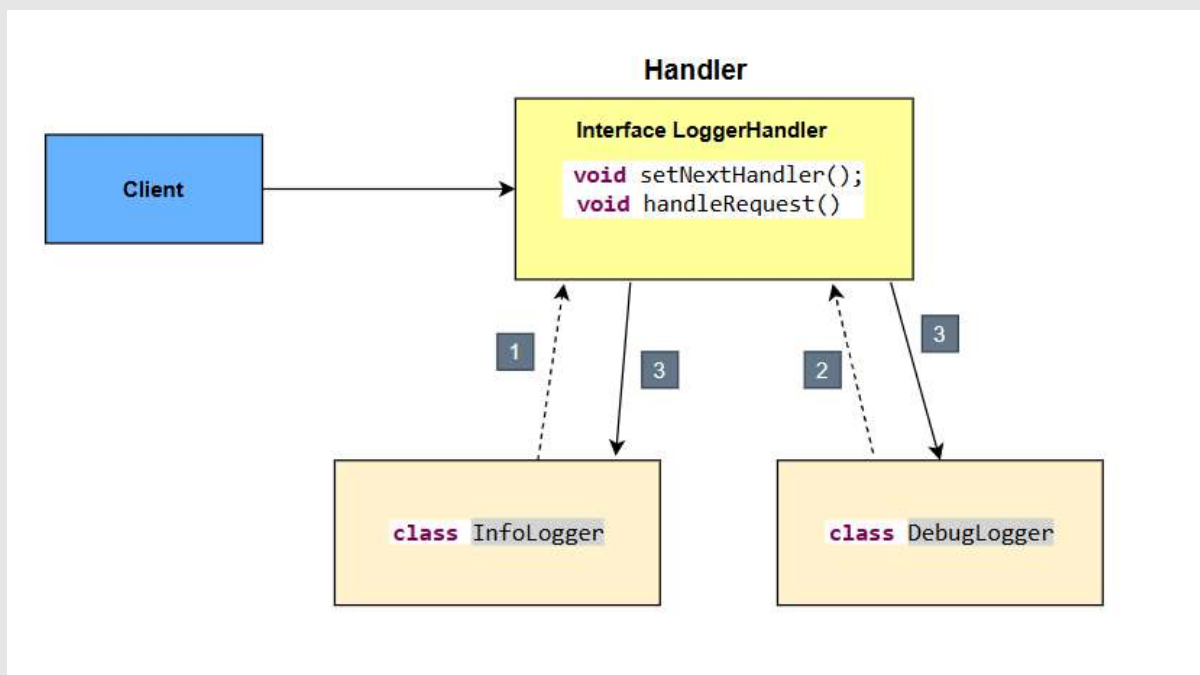
```



```
}
```

Explanation:

- This example shows how a request moves through a chain of handlers until one of them deals with it.
- The InfoLogger checks only for INFO logs. If the request is not INFO, it passes the request to the next handler.
- The DebugLogger handles DEBUG logs and, if it cannot process the request, it either forwards it to the next handler or prints a message saying no handler is available.
- In the main method, the INFO handler is linked to the DEBUG handler, creating a chain.
- When a DEBUG request is sent, the INFO handler ignores it and forwards it to DEBUG, which processes it.
- When an ERROR request is sent, INFO rejects it and passes it to DEBUG, but DEBUG also cannot handle it, so the request reaches the end and prints “No Handler found.”

**1. InfoLogger implements LoggerHandler**

The dotted line shows an **implements relationship**.

InfoLogger follows the LoggerHandler interface and provides its own version of handleRequest() and setNextHandler().

2. DebugLogger implements LoggerHandler

This dotted line also represents **implementation**.

DebugLogger implements the same interface and decides how to handle requests meant for debugging.

3. Handlers are linked together using setNextHandler()

The solid lines show a **dependency** or **chaining relationship**.

- InfoLogger holds a reference to the next handler, which is DebugLogger
- If InfoLogger cannot handle the request, it forwards it to DebugLogger

The request flows along these solid lines until a handler processes it.

4. JDK Examples

- `java.util.logging.Logger`
- Servlet Filters in Java Web Applications
- Exception handlers in frameworks that move through a chain until one resolves the error

5. When to use

Use Chain of Responsibility when

- Multiple handlers can process the same request
- The request should move step by step through a pipeline
- We want to avoid long if-else blocks
- We want to add new handlers without touching existing code

6. Use Cases:

- **Logging Systems:** Different log levels (info, warning, error) can be handled by different handlers in a chain.
- **Request Processing Pipelines:** In web applications, requests can pass through a chain of filters for authentication, authorization, logging, etc.
- **Event Handling:** Events can be processed by a chain of event handlers.
- **Validation Systems:** Input data can be validated sequentially by a chain of validators.

7. Code Link: [DesignPatterns/DesignPatterns/src/com/designpatterns/ChainOfResponsibility.java at main · CodingLyf-Fullstack/DesignPatterns](#)

8.Reference Link: [Beginner's guide to Chain Responsibility Design Pattern in Java | by Neha Gupta | Medium](#)

[Chain of Responsibility Design Pattern Use Case: Authentication and Authorization Middleware | by Mehar Chand | Medium](#)

349. **Design Patterns 28: What is Template Pattern and when to use it?**

1.Real-World Analogy

Imagine a bank cheque clearing process. Every bank follows the same sequence:

1. Receive cheque
2. Verify signature

3. Check account balance
4. Debit amount
5. Send confirmation

This order never changes. That is the fixed algorithm. All banks follow the same steps to clear a cheque. The steps stay in the same order. Only the signature checking step is different in each bank.

Template Method pattern works the same way. It keeps the main steps fixed and allows only one or two steps to be different. Those different steps are written by subclasses, while the main steps stay in the parent class.

Definition

The Template Method Pattern is a behavioural design pattern that defines the skeleton of an algorithm in a base class while allowing subclasses to provide specific implementations for some steps. This pattern helps to enforce a step-by-step workflow while letting subclasses customize individual steps.

2. The Problem

In an enterprise Java project, suppose we have multiple ways of uploading files: to AWS S3, to Google Cloud Storage, and to local server storage.

The common steps are:

1. Validate file
2. Convert file to the correct format
3. Upload to a storage provider
4. Log and send success response

Developers start writing separate classes for each provider, and soon we see repeated code everywhere. Each class writes the same validation, conversion, and logging steps, differing only in the exact upload logic. This leads to duplicate code and inconsistent workflow.

```
class S3Uploader {
    void upload(File f) {
        // validate
        // convert
        // uploadToS3
        // log
    }
}

class GCPUploader {
    void upload(File f) {
        // validate
        // convert
        // uploadToGCP
        // log
    }
}
```

Every uploader repeats most of the steps. Any change in validation or logging must be updated in all uploaders. This becomes difficult to manage and test.

3. Template Pattern Solution

Template Method pattern solves this by placing the fixed steps in a base class. The base class defines the workflow. The subclasses only override the parts that change.

Main Components

- Abstract class: defines the template method with steps in order
- Concrete subclasses: implement the variable steps

Step1: Write Abstract class with template method

```
abstract class FileUploader {  
  
    public final void upload(File file) { // Template Method  
        validate(file);  
        convert(file);  
        doUpload(file); // subclass specific  
        log(file);  
    }  
  
    void validate(File file) {  
        System.out.println("Validating file");  
    }  
  
    void convert(File file) {  
        System.out.println("Converting file");  
    }  
  
    abstract void doUpload(File file); // step that varies  
  
    void log(File file) {  
        System.out.println("Upload logged");  
    }  
}
```

Step 2: Concrete class

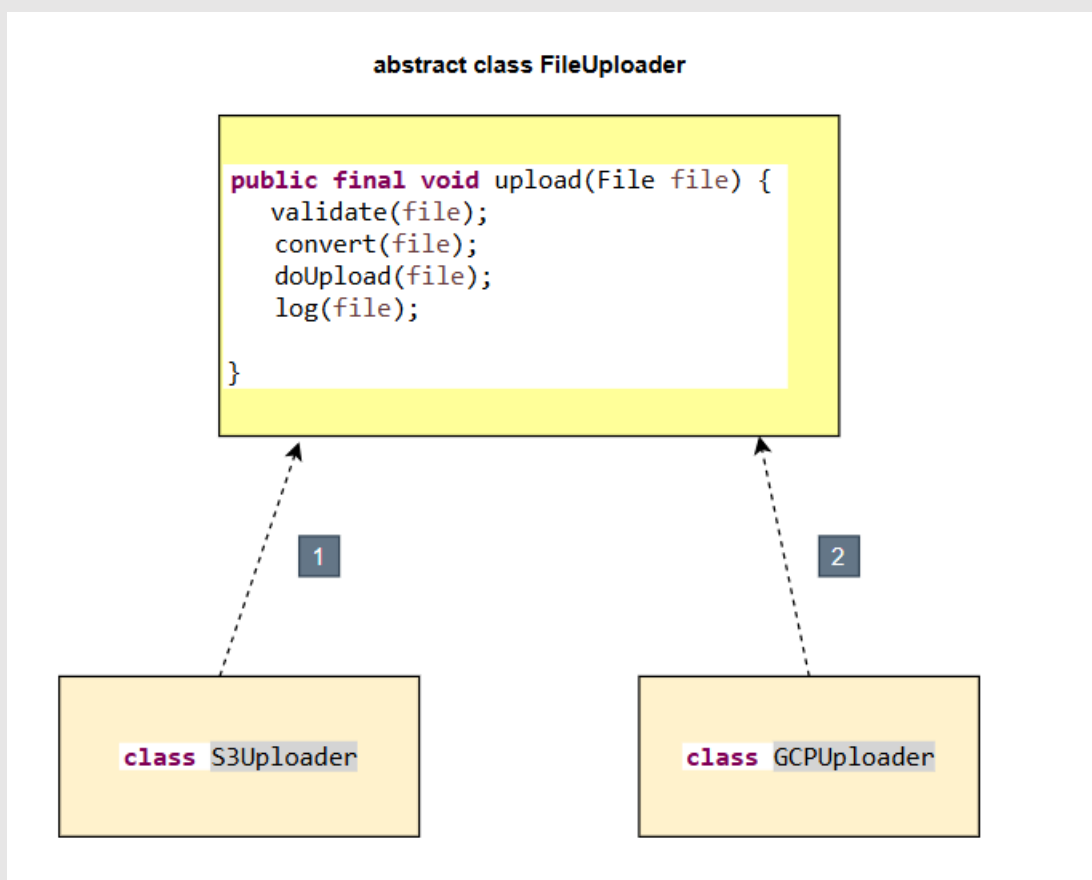
```
class S3Uploader extends FileUploader {  
    @Override  
    void doUpload(File file) {  
        System.out.println("Uploading to AWS S3");  
    }  
}  
  
class GCPUploader extends FileUploader {  
    @Override  
    void doUpload(File file) {  
        System.out.println("Uploading to Google Cloud");  
    }  
}
```

```
}
```

Step 3: Main

```
class TemplateMethodPattern_Uploader {
    public static void main(String[] args) {
        FileUploader uploader = new S3Uploader();
        uploader.upload(new File("test.txt"));

        uploader = new GCPUploader();
        uploader.upload(new File("test.txt"));
    }
}
```



1. S3Uploader extends FileUploader

The dotted line means inheritance.

S3Uploader gets the upload() method from FileUploader and only overrides the required steps like doUpload().

The overall upload sequence remains the same, but the details are S3-specific.

2. GCPUploader extends FileUploader

Again, the dotted line shows another subclass inheriting the base class.

GCPUploader provides its own version of uploading files to Google Cloud, but still uses the same fixed steps defined in upload().

Explanation:**JDK Examples**

This pattern exists in several places in Java

- **java.util.ArrayList**
 - ArrayList provides a fixed structure for list operations. It defines methods like add, remove and get in a set order. Concrete classes such as ArrayList or LinkedList implement only the parts that differ, while the general behavior stays in ArrayList.
- **java.util.AbstractMap**
 - AbstractMap defines the basic workflow for a Map, such as how entries are handled and how key lookups work. It fixes the common logic so all maps follow the same rules. Specific maps like HashMap and TreeMap only implement the parts that are different.
- **javax.servlet.http.HttpServlet**
 - The service() method defines the flow and calls doGet() or doPost() which are overridden by subclasses

5. When to use

Use this pattern when:

- We have a fixed algorithm but some steps vary
- We want to avoid duplicate code across different implementations

6. Code Link:

[DesignPatterns/DesignPatterns/src/com/designpatterns/TemplateMethodPattern_Uploader.java at main · CodingLyf-Fullstack/DesignPatterns](#)

7.Reference Link: [Template Method Design Pattern in Java | by Kalana De Silva | Medium](#)

[Template Design Pattern in Java. This blog is complete tutorial of... | by Akshat Sharma | Medium](#)

350. Design Patterns 29: Difference between behavioural design patterns?

Pattern	Core Idea	What It Solves	How It Works	Real-World Analogy
Observer	One-to-many notification	Many objects must react to a change	Observers subscribe to a subject and get notified automatically	YouTube subscribers get notified when a video is uploaded
Strategy	Choose behavior at runtime	Multiple algorithms for the same task	Encapsulate algorithms and switch them dynamically	Choosing payment method: card, UPI, wallet

Pattern	Core Idea	What It Solves	How It Works	Real-World Analogy
Command	It separates the person who requests an action from the object that performs it	Decouple sender from action	Caller triggers execute() on a command	Loan processing by various commands
State	Change behaviour based on internal state	Object behaviour changes with state	Each state is a separate class handling behaviour	Traffic light changing behaviour by color
Iterator	Traverse a collection safely	Access elements without exposing structure	Iterator provides next(), hasNext() methods	TV remote navigating channels
Chain of Responsibility	Pass request through handlers	Multiple handlers may process a request	Request moves through chain until handled	Customer support escalation flow
Template Method	Fix algorithm structure, vary steps	Common flow with customizable steps	Base class defines steps, subclasses fill details	Cooking recipe with fixed steps and variations

Questions on Design Patterns

351. Design Question 1

Suppose you are creating a payment system and it should support different payments like UPI, CreditCard, Netbanking.

Which design pattern you would use to create the object based on the Payment Type and why?

Solution

We can use the **Factory Method** Pattern for this requirement. In a payment system, the payment type can change at runtime. If we create objects using new everywhere, the code becomes hard to modify. Factory Method hides the object creation and decides which payment class to return based on the payment type. This helps us add new payment modes later without touching existing code.

Pseudo Code:

```
interface Payment {
```

```
    void pay();
}

class UpiPayment implements Payment { void pay(){} }
class CreditCardPayment implements Payment { void pay(){} }
class NetBankingPayment implements Payment { void pay(){} }

class PaymentFactory {
    static Payment getPayment(String type) {
        if(type.equals("UPI")) return new UpiPayment();
        if(type.equals("CARD")) return new CreditCardPayment();
        return new NetBankingPayment();
    }
}
```

352. Design Question 2

Why can you use Abstract factory method for the above scenario and when exactly Abstract factory pattern is preferred?

Solution

Think of the difference like this:

Factory Method picks *one* product from a family of similar objects.

Abstract Factory creates *multiple related products* that must work together as a group.

Why Abstract Factory is not suitable here

Our payment scenario only needs **one object at a time** based on the selected payment type: UPI, CreditCard, Netbanking. These payment types are not related as a family, and we are not creating multiple dependent objects together.

When Abstract Factory is preferred in a Payment System

Abstract Factory is useful when selecting a payment option is not just about creating one object, but creating a set of related objects

Example: Online Payments vs Offline Payments

Each category has its own rules.

Online payments require digital security checks, OTP verifications, and email notifications.

Offline payments work differently. They need manual verification, physical receipts, and SMS notifications.

These are two complete families.

Online Payment Family

- OnlinePayment
- OnlineSecurityCheck

- OnlineNotification

Offline Payment Family

- OfflinePayment
- OfflineSecurityCheck
- OfflineNotification

Once the payment mode is chosen, all related objects must come from the same family.

Pseudo Code

```
//Product interfaces
interface Payment { void pay(); }
interface SecurityCheck { void verify(); }
interface Notification { void notifyUser(); }

//Online implementations
class OnlinePayment implements Payment { ... }
class OnlineSecurityCheck implements SecurityCheck { ... }
class OnlineNotification implements Notification { ... }

//Offline implementations
class OfflinePayment implements Payment { ... }
class OfflineSecurityCheck implements SecurityCheck { ... }
class OfflineNotification implements Notification { ... }

//Abstract Factory
interface PaymentFactory {
Payment createPayment();
SecurityCheck createSecurityCheck();
Notification createNotification();
}

//Concrete Factories
class OnlinePaymentFactory implements PaymentFactory {
Payment createPayment() { return new OnlinePayment(); }
SecurityCheck createSecurityCheck() { return new OnlineSecurityCheck(); }
Notification createNotification() { return new OnlineNotification(); }
}

class OfflinePaymentFactory implements PaymentFactory {
Payment createPayment() { return new OfflinePayment(); }
SecurityCheck createSecurityCheck() { return new OfflineSecurityCheck(); }
Notification createNotification() { return new OfflineNotification(); }
}

//Client usage
PaymentFactory factory = getFactory("ONLINE");

Payment payment = factory.createPayment();
SecurityCheck check = factory.createSecurityCheck();
```

```
Notification notify = factory.createNotification();
```

353. Design Question 3

A company is building an e-commerce platform that must support two kinds of checkout experiences: Web Checkout and Mobile Checkout. Each checkout process requires a set of related components:

- A payment processor
- A receipt generator
- A notification service

For Web Checkout, payments go through the web gateway, receipts are generated in PDF format, and notifications are sent by email.

For Mobile Checkout, payments use an app-based gateway, receipts are generated as a mobile-friendly ticket, and notifications are sent as push messages.

The platform should allow us switch the entire checkout flow based on the device type without changing business logic.

Which design pattern should be used to create these related objects as a family, and why?

Solution

Use the **Abstract Factory Pattern** because each checkout environment (Web and Mobile) needs a group of related objects that must always match. Selecting one environment should build all the components of that family. This prevents mixing WebReceipt with MobilePayment accidentally and keeps the object creation logic in one place.

Pseudo Code:

```
//Common product interfaces
interface Payment {
void pay();
}

interface Receipt {
void generate();
}

interface Notification {
void notifyUser();
}

//Web family implementations
class WebPayment implements Payment { ... }
class WebReceipt implements Receipt { ... }
```

```

class WebEmailNotification implements Notification { ... }

//Mobile family implementations
class MobilePayment implements Payment { ... }
class MobileReceipt implements Receipt { ... }
class MobilePushNotification implements Notification { ... }

//Abstract Factory
interface CheckoutFactory {
Payment createPayment();
Receipt createReceipt();
Notification createNotification();
}

//Concrete Factories
class WebCheckoutFactory implements CheckoutFactory {
Payment createPayment() { return new WebPayment(); }
Receipt createReceipt() { return new WebReceipt(); }
Notification createNotification() { return new WebEmailNotification(); }
}

class MobileCheckoutFactory implements CheckoutFactory {
Payment createPayment() { return new MobilePayment(); }
Receipt createReceipt() { return new MobileReceipt(); }
Notification createNotification() { return new MobilePushNotification(); }
}

//Client usage
CheckoutFactory factory = getFactory("MOBILE"); // Runtime choice

Payment payment = factory.createPayment();
Receipt receipt = factory.createReceipt();
Notification notification = factory.createNotification();

//These objects will always match the selected family: MOBILE
payment.pay();
receipt.generate();
notification.notifyUser();

```

354. Design Question 4

Suppose You have a class in Travel booking App, Let's say HolidayPackage is the class name. And this class contains fields like flight, hotel, food plan, local transport, and outdoor activities. Some customers only want flight and hotel. Some want everything. The platform should let us build different combinations of a holiday package without forcing all fields.

Which design pattern you would use to create the object based on the Payment Type and why?

Solution

This scenario fits the **Builder Pattern**. A holiday package has many optional fields. Builder helps construct the object in steps and produces a final complete object only when required. It keeps the object creation clean and easy to read. Builder Pattern is ideal when an object has many optional fields and must be created step by step

Pseudo Code:

```
class HolidayPackage {
    String flight;
    String hotel;
    String foodPlan;
    String transport;
    String activities;

    HolidayPackage(String flight, String hotel, String foodPlan, String transport,
String activities) {
        this.flight = flight;
        this.hotel = hotel;
        this.foodPlan = foodPlan;
        this.transport = transport;
        this.activities = activities;
    }
}

//Builder
class HolidayPackageBuilder {
    String flight;
    String hotel;
    String foodPlan;
    String transport;
    String activities;

    HolidayPackageBuilder setFlight(String flight) {
        this.flight = flight;
        return this;
    }

    HolidayPackageBuilder setHotel(String hotel) {
        this.hotel = hotel;
        return this;
    }

    HolidayPackageBuilder setFoodPlan(String foodPlan) {
        this.foodPlan = foodPlan;
        return this;
    }

    HolidayPackageBuilder setTransport(String transport) {
        this.transport = transport;
```

```

        return this;
    }

    HolidayPackageBuilder setActivities(String activities) {
        this.activities = activities;
        return this;
    }

    HolidayPackage build() {
        return new HolidayPackage(flight, hotel, foodPlan, transport, activities);
    }
}

//Usage
HolidayPackage pkg = new HolidayPackageBuilder()
    .setFlight("Flight from Delhi to Goa")
    .setHotel("Beach Resort")
    .setActivities("Water Sports")
    .build();

```

355. Design Question 5

In a Java application, we frequently execute database queries for different modules like Orders, Customers, and Payments. Every query requires such as tenant_id, schema name and sql query. Where tenant_id and schema name be unique and initializing these values, every time is repetitive and wasteful.

Which design pattern helps us create a new database query object by copying an existing configured object and changing only the dynamic values?

Solution

Only the SQL statement differs from one query to another. We want a simple way to reuse the same base configuration and generate new query objects with different SQLs without rebuilding the entire object again.

This problem is solved using the **Prototype Pattern**. Prototype allows us to clone an already configured object and modify only what changes. This avoids repeated setup code and improves performance, especially when object creation has a cost.

Pseudo Code:

```

class DbQuery implements Cloneable {
    private String tenantId;
    private String schema;

    private String sql;

    public DbQuery(String tenantId, String schema) {

```

```

        this.tenantId = tenantId;
        this.schema = schema;
    }

    @Override
    public DbQuery clone() {
        try {
            return (DbQuery) super.clone();
        } catch (CloneNotSupportedException e) {
            throw new RuntimeException(e);
        }
    }

    public void setSql(String sql) {
        this.sql = sql;
    }

    public String getSql() {
        return sql;
    }
}

// Client
public class Prototype_DbQuery {

    public static void main(String[] args) {

        // Base query with default metadata
        DbQuery baseQuery = new DbQuery("TENANT-01", "sales_schema");

        // Clone and modify SQL only
        DbQuery orderQuery = baseQuery.clone();
        orderQuery.setSql("SELECT * FROM orders WHERE order_id = 101");

        DbQuery customerQuery = baseQuery.clone();
        customerQuery.setSql("SELECT * FROM customers WHERE cust_id = 22");

        DbExecutor executor = new DbExecutor();
        executor.execute(orderQuery);
        executor.execute(customerQuery);
    }
}

```

Here, we keep one DbQuery object with default metadata and clone it whenever a new query is needed. We only set the SQL string in the cloned object. All other fields remain same. Prototype Pattern is ideal when many similar objects are needed, and creating each from scratch is unnecessary.

356. Design Question 6

A Java application stores user data in UserEntity objects fetched from the database. Another module in the same system expects user information in a UserDTO format to send it across services. The two

classes have different field names and structure. Changing either class is not allowed because both are used in many places.

We need a way to convert UserEntity into UserDTO without modifying existing classes.

Which design pattern helps to connect incompatible classes by creating a middle class that adapts one to the other?

Solution

Adapter Pattern fits here. The adapter takes a UserEntity object and converts it into a compatible UserDTO object. This keeps existing classes untouched and still allows both to work together.

Code:

```
/** Target */
interface UserDTOClient {
    void setUserData(UserDTO dto);
    void sendData();
}

/** Adaptee */
class UserEntity {
    String fullName;
    String emailId;
    String role;

    public UserEntity(String fullName, String emailId, String role) {
        this.fullName = fullName;
        this.emailId = emailId;
        this.role = role;
    }
}

/** Target Object */
class UserDTO {
    String name;
    String mail;
    String designation;

    public UserDTO(String name, String mail, String designation) {
        this.name = name;
        this.mail = mail;
        this.designation = designation;
    }
}

/** Adapter */
class UserAdapter implements UserDTOClient {

    private UserEntity entity;
    private UserDTO dto;
```

```

    public UserAdapter(UserEntity entity) {
        this.entity = entity;
    }

    public void setUserData(UserDTO dto) {
        this.dto = dto;
    }

    public void sendData() {
        System.out.println("Sending Data");
        System.out.println("Name: " + dto.name);
        System.out.println("Email: " + dto.mail);
        System.out.println("Role: " + dto.designation);
    }

    /** Converts entity to DTO */
    public UserDTO convert() {
        return new UserDTO(entity.fullName, entity.emailId, entity.role);
    }
}

/** Client */
public class AdapterPattern_UserConversion {

    public static void main(String[] args) {

        // Existing class object (Adaptee)
        UserEntity entity = new UserEntity("Ravi Kumar", "ravi@mail.com", "Developer");

        // Adapter converts Entity to DTO
        UserAdapter adapter = new UserAdapter(entity);
        UserDTO dto = adapter.convert();

        // Pass converted DTO to the target client
        adapter.setUserData(dto);
        adapter.sendData();
    }
}

```

- UserEntity and UserDTO cannot talk directly.
- The **UserAdapter** sits in the middle.
- It **converts** the entity fields to the DTO format.
- No changes are required in existing classes.

357. Design Question 7

Scenario: A company is modernizing its billing system. The new application generates customer invoices using a modern *InvoiceService* interface that expects data in JSON format. However, the old legacy billing system stores and processes invoice data in XML. The legacy system cannot be changed because it is used by multiple departments and has been in production for years.

To make the new application work with the legacy system, we need something that converts JSON-based requests into the XML format understood by the old system, and also converts XML responses back into JSON.

Which design pattern helps two incompatible systems communicate by introducing a middle component that translates one system's requests into another system's expected format?

Solution

This situation is perfectly suited for the **Adapter Pattern**. The adapter acts as a translator between the new invoice service and the legacy billing system. It converts data formats and method calls so both systems can work together without modifying existing code.

```
/** Target: New system expects JSON format */
interface InvoiceService {
    void generateInvoice(String jsonData);
}

/** Adaptee: Legacy system works only with XML */
class LegacyBillingSystem {
    void createInvoice(String xmlData) {
        System.out.println("Processing invoice in XML format: " + xmlData);
    }
}

/** Adapter converts JSON to XML */
class BillingAdapter implements InvoiceService {

    private LegacyBillingSystem legacySystem;

    public BillingAdapter(LegacyBillingSystem legacySystem) {
        this.legacySystem = legacySystem;
    }

    public void generateInvoice(String jsonData) {
        String xmlData = convertJsonToXml(jsonData);
        legacySystem.createInvoice(xmlData);
    }

    private String convertJsonToXml(String json) {
        return "<invoice>" + json + "</invoice>"; // Simple representation
    }
}

/** Client */
public class AdapterPattern_BillingExample {

    public static void main(String[] args) {

        InvoiceService service =
            new BillingAdapter(new LegacyBillingSystem());
    }
}
```

```

        service.generateInvoice("{id:101, amount:5000}");
    }
}

```

358. Design Question 8

A Java notification system sends alerts to millions of users. Each alert contains common details like company header, footer text, and branding style. Only the user-specific information such as name, phone number, or personalized message changes for each alert.

If the system creates a separate object storing the header, footer, and branding for every alert, memory usage becomes very high. We need a way to store these common details once and reuse them while passing only the user-specific details at runtime.

Which design pattern helps achieve this memory-efficient structure?

Solution:

This requirement fits the **Flyweight Pattern**. The shared notification template acts as the **intrinsic state**, and the user-specific information is passed as **extrinsic state** during processing. This approach reduces the number of objects created and optimizes memory usage.

```

//Flyweight (stores intrinsic state)
class AlbumCover {
    private String image; // intrinsic (shared for the album)

    AlbumCover(String image) {
        this.image = image;
    }

    // extrinsic state is passed from outside
    public void show(String songName, int duration) {
        System.out.println("Song: " + songName + " duration " + duration + " cover "
            + image);
    }
}

//Flyweight Factory
class AlbumFactory {
    private static Map<String, AlbumCover> covers = new HashMap<>();

    public static AlbumCover getCover(String albumName) {
        // intrinsic state created once
        if (!covers.containsKey(albumName)) {
            covers.put(albumName, new AlbumCover(albumName + "_cover.png"));
        }

        return covers.get(albumName);
    }
}

```

```

    }
}

//Client
public class Flyweight_MusicAppExample {
    public static void main(String[] args) {

        // Same intrinsic object reused
        AlbumCover cover = AlbumFactory.getCover("MelodyHits");
        cover.show("Love Tune", 180); // extrinsic state

        AlbumCover reusedCover = AlbumFactory.getCover("MelodyHits"); // same
object
        reusedCover.show("Dream Song", 200); // different extrinsic state
    }
}

```

Point to note here

It may look like **Prototype** can work here, because we are reusing an existing object. But **this is not a Prototype scenario**. Here's why:

Why Flyweight fits, not Prototype

Flyweight goal

Share **one common object** across thousands or millions of uses

Intrinsic state is reused, **Extrinsic state** is supplied from outside

No cloning. No new objects. Just one shared instance reused everywhere.

In our notification example:

- Header, footer, branding: shared **intrinsic**
- User name, phone number, message: **extrinsic**

One template object is reused for all users. Memory is saved because we don't create copies.

What would happen if we use Prototype?

Prototype creates a **clone of an existing object**. That means:

`baseTemplate.clone()` -> new object created. Each alert becomes a separate object.

Even if cloning is faster than constructing from scratch, we STILL create:

- One new object per alert
- Millions of objects in memory

That defeats the purpose. Prototype solves **object creation complexity**, not **memory sharing**.

When Prototype is useful

Use Prototype when:

- Creating an object is expensive
- We want **multiple copies** with small changes
- Each object must be independent

Example: Cloning a base EmployeeProfile for new employees , Every clone is a new object.

When Flyweight is useful

Use Flyweight when:

- Many objects share most of their data
- We want **one shared instance**
- Only small, external values change

Example: Notification template reused for millions of users

359. Design Question 9

A music app shows thousands of **song thumbnails**. All songs from the same album share the same album cover image. Only the details such as song title, duration, and play count are different. If the cover image is stored separately for every song, the app uses a lot of memory.

We need a smarter way to store the album cover once and allow all songs from the same album to use it. Each song should keep only its changing properties.

Which design pattern helps reuse the shared album image across multiple song objects?

Solution:

The **Flyweight Pattern** solves this problem. The album cover is stored once and shared with every song in that album. Only the song-specific details are stored separately.

Intrinsic State: The shared part of the object that does not change. Here, it is the **album cover image**.

Extrinsic State: The data supplied at runtime that changes for every usage. Here, it is the **song name** and **duration**.

```
//Flyweight (stores intrinsic state)
class AlbumCover {
    private String image; // intrinsic (shared for the album)

    AlbumCover(String image) {
        this.image = image;
    }

    // extrinsic state is passed from outside
    public void show(String songName, int duration) {
```

```

        System.out.println("Song: " + songName + " duration " + duration + " cover
" + image);
    }
}

//Flyweight Factory
class AlbumFactory {
    private static Map<String, AlbumCover> covers = new HashMap<>();

    public static AlbumCover getCover(String albumName) {

        // intrinsic state created once
        if (!covers.containsKey(albumName)) {
            covers.put(albumName, new AlbumCover(albumName + "_cover.png"));
        }

        return covers.get(albumName);
    }
}

//Client
public class Flyweight_MusicAppExample {
    public static void main(String[] args) {

        // Same intrinsic object reused
        AlbumCover cover = AlbumFactory.getCover("MelodyHits");
        cover.show("Love Tune", 180); // extrinsic state

        AlbumCover reusedCover = AlbumFactory.getCover("MelodyHits"); // same
object
        reusedCover.show("Dream Song", 200); // different extrinsic state
    }
}

```

360. Design Question 10

A Java application accesses customer records stored in a remote microservice. The *CustomerService* interface provides a method *getCustomerDetails(id)*. Calling the real service every time is slow and increases network traffic, especially when the same customer data is requested repeatedly.

We need a way to introduce a layer that behaves like the real service but decides when to call the remote API. If the data is already present in memory, it should return the cached result instead of making another network call. This helps reduce response time and improves performance.

Which design pattern allows us to create an object that controls access to the real service and optionally adds caching without modifying the original code?

Solution:

This scenario is solved using the **Proxy Pattern**. The proxy object implements the same interface as the actual service. It decides whether to call the real service or return a cached value. The client does not

know whether it is talking to the proxy or the actual service. The proxy adds an extra layer of optimization.

```
//Subject
interface CustomerService {
    String getCustomerDetails(String id);
}

//Real Object
class RealCustomerService implements CustomerService {
    public String getCustomerDetails(String id) {
        System.out.println("Calling remote service for id: " + id);
        return "Customer-" + id; // imagine this comes from remote API
    }
}

//Proxy
class CustomerServiceProxy implements CustomerService {
    private RealCustomerService realService;
    private Map<String, String> cache = new HashMap<>();

    public CustomerServiceProxy() {
        this.realService = new RealCustomerService();
    }

    public String getCustomerDetails(String id) {
        if (cache.containsKey(id)) {
            System.out.println("Returning cached data for id: " + id);
            return cache.get(id);
        }
        String result = realService.getCustomerDetails(id);
        cache.put(id, result);
        return result;
    }
}

//Client
public class Proxy_CustomerServiceExample {
    public static void main(String[] args) {
        CustomerService service = new CustomerServiceProxy();

        System.out.println(service.getCustomerDetails("101")); // remote call
        System.out.println(service.getCustomerDetails("101")); // cached result
    }
}
```

Proxy helps us to add extra behavior such as caching, access control, auditing, or lazy initialization, without changing the real object's code. The proxy controls how and when the actual object is accessed.

361. Design Question 11

A Java application compresses files before storing them. The base implementation simply saves the file. Later, new requirements come in. Some files must be encrypted before saving, some must be logged, and some must even be audited. We cannot modify the existing FileStore class every time a new feature is required. We need a flexible approach that allows us to add extra behaviours around the save operation at runtime.

Which design pattern helps us wrap the existing file storage process with additional features like encryption, compression, or logging without changing the original class?

Solution

This scenario uses the **Decorator Pattern**. The decorator wraps the original file storage object and adds extra features on top of it. Each decorator adds one responsibility, and we can stack multiple decorators based on the requirement.

```
//Component
interface FileStore {
    void save(String data);
}

//Concrete Component
class BasicFileStore implements FileStore {
    public void save(String data) {
        System.out.println("Saving file: " + data);
    }
}

//Decorator
abstract class FileStoreDecorator implements FileStore {
    protected FileStore fileStore;

    FileStoreDecorator(FileStore fileStore) {
        this.fileStore = fileStore;
    }

    public void save(String data) {
        fileStore.save(data);
    }
}

//Concrete Decorators
class EncryptionDecorator extends FileStoreDecorator {
    EncryptionDecorator(FileStore fileStore) {
        super(fileStore);
    }

    public void save(String data) {
        data = "Encrypted(" + data + ")";
        super.save(data);
    }
}
```

```

    }
}

class LoggingDecorator extends FileStoreDecorator {
    LoggingDecorator(FileStore fileStore) {
        super(fileStore);
    }

    public void save(String data) {
        System.out.println("Logging: " + data);
        super.save(data);
    }
}

//Client
public class Decorator_FileStoreExample {
    public static void main(String[] args) {

        FileStore store = new LoggingDecorator(new EncryptionDecorator(new
BasicFileStore()));

        store.save("Report.pdf");
    }
}

```

Decorator allows us to add responsibilities step by step without changing existing code. The base class stays clean, and new features are added through wrappers.

362. Design Question 12

A coffee shop sells plain coffee. Customers can choose to add milk, sugar, chocolate, or whipped cream. Each addition changes the final taste and price. The shop should not create separate classes for every combination like *CoffeeWithMilkAndChocolate*. Instead, the shop needs a way to start with a basic coffee and wrap extra ingredients based on customer choice.

Which design pattern allows us to add extra features to an existing object step by step without creating unnecessary subclasses?

Solution

This is a good example of the **Decorator Pattern**. Each ingredient acts as a decorator. We start with a plain coffee and wrap it with milk, sugar, and any other add-ons the customer requests.

```

//Component
interface Coffee {
    void prepare();
}

//Concrete Component
class BasicCoffee implements Coffee {
    public void prepare() {

```



```

        System.out.println("Basic Coffee");
    }
}

//Decorator
abstract class CoffeeDecorator implements Coffee {
    protected Coffee coffee;

    CoffeeDecorator(Coffee coffee) {
        this.coffee = coffee;
    }

    public void prepare() {
        coffee.prepare();
    }
}

//Concrete Decorators
class MilkDecorator extends CoffeeDecorator {
    MilkDecorator(Coffee coffee) {
        super(coffee);
    }

    public void prepare() {
        super.prepare();
        System.out.println("Adding Milk");
    }
}

class SugarDecorator extends CoffeeDecorator {
    SugarDecorator(Coffee coffee) {
        super(coffee);
    }

    public void prepare() {
        super.prepare();
        System.out.println("Adding Sugar");
    }
}

//Client
public class Decorator_CoffeeExample {
    public static void main(String[] args) {
        Coffee coffee = new SugarDecorator(new MilkDecorator(new BasicCoffee()));
        coffee.prepare();
    }
}

```

Decorator helps us add features one by one without modifying the base object. It avoids class explosion and supports custom combinations at runtime.

363. Design Question 13

A Java-based file explorer displays files and folders. A file is a single item, but a folder can contain multiple files and other folders inside it. Both files and folders should be treated in the same way. For example, when we click "show details", the system should handle files and folders using the same method without checking their type each time.

We need a design that allows us to represent individual objects (files) and groups of objects (folders) uniformly, so that client code interacts with both in the same manner.

Which design pattern allows us to build tree-like structures where individual objects and groups of objects can be treated identically?

Solution

This is a classic use case of the **Composite Pattern**. Composite allows us to combine objects into a tree structure. Files act as leaf nodes, and folders act as composite nodes that hold other components. Both implement a common interface, so client code works the same way for both.

```
//Component
interface FileSystemComponent {
    void showDetails();
}

//Leaf
class FileItem implements FileSystemComponent {
    private String name;

    FileItem(String name) {
        this.name = name;
    }

    public void showDetails() {
        System.out.println("File: " + name);
    }
}

//Composite
class Folder implements FileSystemComponent {
    private String name;
    private List<FileSystemComponent> items = new ArrayList<>();

    Folder(String name) {
        this.name = name;
    }

    public void add(FileSystemComponent component) {
        items.add(component);
    }

    public void showDetails() {
```

```

        System.out.println("Folder: " + name);
        for (FileSystemComponent item : items) {
            item.showDetails();
        }
    }
}

//Client
public class Composite_FileExplorerExample {
    public static void main(String[] args) {

        FileSystemComponent file1 = new FileItem("Resume.pdf");
        FileSystemComponent file2 = new FileItem("Photo.png");

        Folder docs = new Folder("Documents");
        docs.add(file1);
        docs.add(file2);

        Folder root = new Folder("Root");
        root.add(docs);
        root.showDetails();
    }
}

```

Composite Pattern helps create tree structures. It allows us to treat a single object and a collection of objects in the same way.

364. Design Question 14

A Java reporting system generates different types of reports such as **PDF reports**, **Excel reports**, and **HTML reports**. Each report can be for various modules like **Sales**, **Inventory**, or **Customer data**. If we directly combine report type and module type, we end up with too many classes like SalesPDFReport, SalesExcelReport, InventoryPDFReport, InventoryExcelReport, and so on.

This increases the number of classes and makes the system difficult to extend. We need a way to keep the report type and report content separate so that they can change independently.

Which design pattern helps separate abstraction from implementation so both can evolve without affecting each other?

Solution

This is a perfect use case of the **Bridge Pattern**. Bridge separates the main abstraction (report generation) from its implementation (output format). We can add new report formats or new modules without modifying existing code.

```

//Implementor
interface ReportFormat {
    void generate(String data);
}

```

```
//Concrete Implementors
class PDFFormat implements ReportFormat {
    public void generate(String data) {
        System.out.println("PDF Report: " + data);
    }
}

class ExcelFormat implements ReportFormat {
    public void generate(String data) {
        System.out.println("Excel Report: " + data);
    }
}

//Abstraction
abstract class Report {
    protected ReportFormat format;

    Report(ReportFormat format) {
        this.format = format;
    }

    abstract void createReport();
}

//Refined Abstraction
class SalesReport extends Report {
    SalesReport(ReportFormat format) {
        super(format);
    }

    public void createReport() {
        format.generate("Sales Data");
    }
}

class InventoryReport extends Report {
    InventoryReport(ReportFormat format) {
        super(format);
    }

    public void createReport() {
        format.generate("Inventory Data");
    }
}

//Client
public class Bridge_ReportExample {
    public static void main(String[] args) {
        Report report1 = new SalesReport(new PDFFormat());
        report1.createReport();
    }
}
```

```
        Report report2 = new InventoryReport(new ExcelFormat());
        report2.createReport();
    }
}
```

In this example, the **abstraction** is the report itself, because it deals with *what* data the report contains, like Sales, Inventory, or Customer information. The **implementation** is the report format, because it defines *how* the report is generated, such as PDF, Excel, or HTML. If these are mixed directly, we get many unnecessary classes. By separating them, the abstraction can focus on report content, and the implementation can handle the output format. This makes both parts independent, so we can add new report types or new formats without changing existing code. That is the core reason to use the Bridge Pattern here.

Bridge removes tight coupling between abstraction and implementation. Both can grow independently, which keeps the code clean and scalable.

365. Design Question 15

A Java application needs to process an online order. To place an order, several steps must run in sequence: validate user details, check inventory, process payment, update shipping information, and send a confirmation message. Each step is handled by a different class like UserService, InventoryService, PaymentService, and NotificationService. The client code must call all these services in the correct order, making it complex and hard to maintain.

We need a single class that hides this complexity and provides a simple method like placeOrder() which internally calls all required services.

Which design pattern helps provide a simple interface over a group of classes so the client deals with only one entry point?

Solution

This is a good use case of the **Facade Pattern**. The facade class acts as a single-entry point and handles communication with multiple services behind the scenes. Client code becomes cleaner and easier to understand.

```
//Complex subsystem classes
class UserService {
    void validateUser() {
        System.out.println("User validated");
    }
}

class InventoryService {
    void checkStock() {
        System.out.println("Stock checked");
    }
}
```

```

}

class PaymentService {
    void makePayment() {
        System.out.println("Payment processed");
    }
}

class NotificationService {
    void sendNotification() {
        System.out.println("Notification sent");
    }
}

//Facade
class OrderFacade {
    private UserService user = new UserService();
    private InventoryService inventory = new InventoryService();
    private PaymentService payment = new PaymentService();
    private NotificationService notify = new NotificationService();

    void placeOrder() {
        user.validateUser();
        inventory.checkStock();
        payment.makePayment();
        notify.sendNotification();
    }
}

//Client
public class Facade_OrderExample {
    public static void main(String[] args) {
        OrderFacade facade = new OrderFacade();
        facade.placeOrder();
    }
}

```

Facade hides system complexity by exposing a single method to the client. Internally, it manages multiple classes and workflows but keeps client interaction simple.

366. Design Question 16

A Java authentication module supports different login mechanisms. Some users log in with a **password**, some with an **OTP**, and some using **fingerprint** through a device. The `LoginService` currently checks all these conditions inside one method. As new login methods get added, this class keeps growing, making it hard to maintain and test.

We need a design where the login method can change without modifying the `LoginService`. The service should just execute a login process, and the actual authentication method should be selected at runtime based on the user's preference.

Which design pattern helps define multiple authentication mechanisms and allows us switch between them without changing the main service?

Solution

This is solved using the **Strategy Pattern**. Each authentication approach becomes a separate class, and the service uses one of them at runtime. The authentication logic stays isolated, and the main service remains clean.

```
//Strategy
interface AuthMethod {
    void authenticate(String user);
}

//Concrete Strategies
class PasswordAuth implements AuthMethod {
    public void authenticate(String user) {
        System.out.println("Password authentication for " + user);
    }
}

class OtpAuth implements AuthMethod {
    public void authenticate(String user) {
        System.out.println("OTP authentication for " + user);
    }
}

class FingerprintAuth implements AuthMethod {
    public void authenticate(String user) {
        System.out.println("Fingerprint authentication for " + user);
    }
}

//Context
class LoginService {
    private AuthMethod method;

    LoginService(AuthMethod method) {
        this.method = method;
    }

    void login(String user) {
        method.authenticate(user);
    }
}

//Client
public class Strategy_LoginExample {
    public static void main(String[] args) {
        LoginService login1 = new LoginService(new PasswordAuth());
        login1.login("Ravi");
    }
}
```

```

        LoginService login2 = new LoginService(new OtpAuth());
        login2.login("Kiran");
    }
}

```

We create separate classes for each algorithm. The main class uses one of them through an interface. No changes are required when a new algorithm is introduced, add a new implementation.

367. Design Question 17

A Java application displays stock prices on multiple screens. The stock value changes throughout the day, and whenever there is a price update, all screens should show the latest value. Currently, each screen repeatedly asks the server for updates, which wastes resources and makes the system slow.

We need a design where the stock server notifies all registered displays automatically whenever the price changes. Each display should update itself without asking again and again.

Which design pattern helps achieve a publish-subscribe mechanism where one object notifies many dependent objects when its state changes?

Solution

We can use **Observer Pattern**. The stock server is the subject, and the displays are observers. When the stock price changes, the server notifies all observers.

```

//Subject
interface StockServer {
    void register(Display d);

    void notifyDisplays();

    void updatePrice(int price);
}

//Concrete Subject
class StockMarket implements StockServer {
    private List<Display> displays = new ArrayList<>();
    private int price;

    public void register(Display d) {
        displays.add(d);
    }

    public void updatePrice(int price) {
        this.price = price;
        notifyDisplays();
    }

    public void notifyDisplays() {
        for (Display d : displays) {
            d.refresh(price);
        }
    }
}

```



```

    }
}

//Observer
interface Display {
    void refresh(int price);
}

//Concrete Observers
class MobileDisplay implements Display {
    public void refresh(int price) {
        System.out.println("Mobile View: Price updated to " + price);
    }
}

class WebDisplay implements Display {
    public void refresh(int price) {
        System.out.println("Web View: Price updated to " + price);
    }
}

//Client
public class Observer_StockExample {
    public static void main(String[] args) {

        StockMarket market = new StockMarket();
        market.register(new MobileDisplay());
        market.register(new WebDisplay());

        market.updatePrice(150);
        market.updatePrice(175);
    }
}

```

Observer allows one object to broadcast changes to many others. Observers react automatically when the subject's state changes, reducing polling and improving performance.

368. Design Question 18

A restaurant waiter takes an order and gives it to the kitchen. The waiter does not cook the food. The order acts like an instruction for the chef. Once the chef receives it, they prepare the dish. The waiter only passes the request.

We need a system where the customer request is separated from how the kitchen fulfills it. The waiter just triggers the action without knowing the cooking process.

Which design pattern matches this behavior where a request is wrapped as an object and later executed by someone else?

Solution

This is a **Command Pattern**. The waiter is the invoker, the order is the command, and the chef is the receiver who executes the request.

```
//Receiver
class Chef {
    void makePizza() {
        System.out.println("Pizza is being cooked");
    }
}

//Concrete Command
class PizzaOrder implements Order {
    private Chef chef;

    PizzaOrder(Chef chef) {
        this.chef = chef;
    }

    public void cook() {
        chef.makePizza();
    }
}

//Invoker
class Waiter {
    private Order order;

    void takeOrder(Order order) {
        this.order = order;
    }

    void placeOrder() {
        order.cook();
    }
}

//Client
public class Command_RestaurantExample {
    public static void main(String[] args) {
        Chef chef = new Chef();
        Waiter waiter = new Waiter();

        waiter.takeOrder(new PizzaOrder(chef));
        waiter.placeOrder();
    }
}
```

Command Pattern separates *request creation* from *request execution*. The invoker just triggers the command, and the receiver performs the action. This gives better control, supports undo operations, queues, and scheduling of commands.

369. Design Question 19

A Java banking application allows users to perform actions like deposit money, withdraw money, and transfer funds. Each action requires different business rules and interacts with different backend services. If the UI directly calls these operations, adding new actions or modifying existing ones becomes risky and complex.

We need a design where each banking operation is treated as an independent object. The UI should only trigger these actions without knowing how the bank account processes them.

Which design pattern helps wrap each request as a separate object so the UI can execute banking operations uniformly?

Solution

This scenario uses the **Command Pattern**. Each banking operation becomes a command. The UI is the invoker, the bank account is the receiver, and the command object performs the task.

```
//Command
interface BankCommand {
    void execute();
}

//Receiver
class BankAccount {
    void deposit(int amount) {
        System.out.println("Deposited: " + amount);
    }

    void withdraw(int amount) {
        System.out.println("Withdrawn: " + amount);
    }
}

//Concrete Commands
class DepositCommand implements BankCommand {
    private BankAccount account;
    private int amount;

    DepositCommand(BankAccount account, int amount) {
        this.account = account;
        this.amount = amount;
    }

    public void execute() {
        account.deposit(amount);
    }
}

class WithdrawCommand implements BankCommand {
    private BankAccount account;
    private int amount;
```

```

        WithdrawCommand(BankAccount account, int amount) {
            this.account = account;
            this.amount = amount;
        }

        public void execute() {
            account.withdraw(amount);
        }
    }

//Invoker
class BankAppInvoker {
    private BankCommand command;

    void setCommand(BankCommand command) {
        this.command = command;
    }

    void clickButton() {
        command.execute();
    }
}

//Client
public class Command_BankExample {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();
        BankAppInvoker app = new BankAppInvoker();

        app.setCommand(new DepositCommand(account, 5000));
        app.clickButton();

        app.setCommand(new WithdrawCommand(account, 2000));
        app.clickButton();
    }
}

```

Command separates the **request** from the **execution**. The invoker triggers commands without knowing how they work. Each operation becomes reusable, testable, and easy to extend.

370. Design Question 20

A Java application manages a collection of online training courses. Some courses are stored in an ArrayList while archived courses are kept in a LinkedList. The dashboard module needs to display courses one by one, but it should not know how the courses are stored internally. If the dashboard checks for data structure types, the code becomes messy and tightly coupled.

We need a common way to loop through courses regardless of whether they are stored in an ArrayList or LinkedList.

Which design pattern provides a uniform way to access elements of a collection without exposing its internal structure?

Solution

The **Iterator Pattern** helps here. It separates the traversal logic from the collection. Each collection provides its own iterator, and the dashboard only interacts with the iterator to move item by item.

```
class Course {
    private String name;

    Course(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}

// Iterator
interface Iterator1<T> {
    boolean hasNext();

    T next();
}

// Concrete Iterator for ArrayList
class CurrentCourseIterator implements Iterator<Course> {
    private List<Course> list;
    private int index = 0;

    CurrentCourseIterator(List<Course> list) {
        this.list = list;
    }

    public boolean hasNext() {
        return index < list.size();
    }

    public Course next() {
        return list.get(index++);
    }
}

// Collection
interface CourseCollection {
    Iterator<Course> createIterator();
}

// Concrete Collection
class CurrentCourses implements CourseCollection {
```

```

    private List<Course> list = new ArrayList<>();

    void add(Course c) {
        list.add(c);
    }

    public Iterator<Course> createIterator() {
        return new CurrentCourseIterator(list);
    }
}

// Client
public class Iterator_DesignQuestion {
    public static void main(String[] args) {
        CurrentCourses courseList = new CurrentCourses();
        courseList.add(new Course("Java Basics"));
        courseList.add(new Course("Spring Boot"));
        courseList.add(new Course("Microservices"));

        Iterator<Course> it = courseList.createIterator();

        while (it.hasNext()) {
            System.out.println(it.next().getName());
        }
    }
}

```

Iterator provides a clean way to walk through elements without caring how they are stored. The client only focuses on the next element, not on the data structure.

371. Design Question 21

A Java document editor allows a user to edit a file. The editor behaves differently depending on the file's current state: **Read-Only**, **Editing**, or **Saving**. In Read-Only mode, typing is not allowed. In Editing mode, typing updates the document. In Saving mode, the editor temporarily blocks user input. If all these behaviours are written inside one class using if-else conditions, the code grows complex and becomes harder to maintain.

We need a way where the editor changes its behaviour automatically based on the current state of the document, without using condition checks everywhere.

Which design pattern helps an object alter its behaviour when its internal state changes?

Solution:

The **State Pattern** fits here. Each mode becomes a separate class. The editor switches states, and its behaviour changes accordingly. No conditions needed.

```

//State Interface
interface EditorState {
    void type(DocumentEditor editor, String text);
}

```

```

}

//Concrete States
class ReadOnlyState implements EditorState {
    public void type(DocumentEditor editor, String text) {
        System.out.println("File is read-only. Cannot type.");
    }
}

class EditingState implements EditorState {
    public void type(DocumentEditor editor, String text) {
        System.out.println("Typing: " + text);
    }
}

class SavingState implements EditorState {
    public void type(DocumentEditor editor, String text) {
        System.out.println("Saving in progress. Try later.");
    }
}

//Context
class DocumentEditor {
    private EditorState state;

    DocumentEditor(EditorState state) {
        this.state = state;
    }

    void setState(EditorState state) {
        this.state = state;
    }

    void pressKey(String text) {
        state.type(this, text);
    }
}

//Client
public class State_DocumentEditorExample {
    public static void main(String[] args) {

        DocumentEditor editor = new DocumentEditor(new ReadOnlyState());
        editor.pressKey("Hello"); // Cannot type

        editor.setState(new EditingState());
        editor.pressKey("Hello World"); // Typing allowed

        editor.setState(new SavingState());
        editor.pressKey("New Data"); // Saving in progress
    }
}

```

```
}
```

372. Design Question 22

A Java web application processes incoming HTTP requests. Before the request reaches the controller, it must pass through several checks: **Authentication**, **Authorization**, and **Input Validation**. Each check is done by a different component. If authentication fails, the request should not move forward. If it passes, the authorization should verify user roles. Finally, the input validation checks request data.

If we put all these checks inside one method, the code becomes long and difficult to extend. We need a design where each validator performs its task and passes the request to the next validator only if its own check passes.

Which design pattern allows a request to move through a chain of objects until one of them processes or stops it?

Solution

This can be solved using the **Chain of Responsibility Pattern**. Each check is a separate handler. The request flows through the chain until one rejects it or the chain finishes successfully.

```
abstract class RequestHandler {
    protected RequestHandler next;

    public void setNext(RequestHandler next) {
        this.next = next;
    }

    public abstract void handle(String request);
}

// Concrete Handlers
class AuthHandler extends RequestHandler {
    public void handle(String request) {
        if (request.contains("token")) {
            System.out.println("Authentication Passed");
            if (next != null)
                next.handle(request);
        } else {
            System.out.println("Authentication Failed");
        }
    }
}

class RoleHandler extends RequestHandler {
    public void handle(String request) {
        if (request.contains("ADMIN")) {
            System.out.println("Authorization Passed");
            if (next != null)
                next.handle(request);
        }
    }
}
```



```

        } else {
            System.out.println("Authorization Failed");
        }
    }
}

class ValidationHandler extends RequestHandler {
    public void handle(String request) {
        if (request.length() > 10) {
            System.out.println("Validation Passed");
            if (next != null)
                next.handle(request);
        } else {
            System.out.println("Validation Failed");
        }
    }
}

// Client
public class Chain_RequestExample {
    public static void main(String[] args) {
        RequestHandler auth = new AuthHandler();
        RequestHandler role = new RoleHandler();
        RequestHandler validation = new ValidationHandler();

        auth.setNext(role);
        role.setNext(validation);

        System.out.println("Processing request:");
        auth.handle("token-ADMIN-access");
    }
}

```

Chain of Responsibility avoids long condition blocks. Each handler works independently and passes the request along the chain if it succeeds.

373. Design Question 23

A Java application exports reports in different formats such as PDF and Excel. The export process always follows the same steps:

1. Fetch data
2. Format data
3. Write file

Only the formatting step changes based on the file type. If we place all these steps in each report class, we duplicate logic and make the system harder to maintain.

We need a design where the common algorithm stays in one place, and only the specific formatting logic is provided by subclasses.

Which design pattern allows us to define a fixed algorithm structure in a parent class while letting subclasses override only selected steps?

Solution

This is done using the Template Method Pattern. The base class defines the skeleton of the export process, and subclasses override only the formatting step.

```
//Template
abstract class ReportExporter {

    public final void export() {
        fetchData();
        formatData();
        writeFile();
    }

    void fetchData() {
        System.out.println("Fetching data from DB");
    }

    abstract void formatData(); // Varies based on report type

    void writeFile() {
        System.out.println("Writing file to disk");
    }
}

//Concrete Templates
class PdfExporter extends ReportExporter {
    void formatData() {
        System.out.println("Formatting data to PDF");
    }
}

class ExcelExporter extends ReportExporter {
    void formatData() {
        System.out.println("Formatting data to Excel");
    }
}

//Client
public class Template_ReportExample {
    public static void main(String[] args) {
        ReportExporter pdf = new PdfExporter();
        pdf.export();

        ReportExporter excel = new ExcelExporter();
        excel.export();
    }
}
```

Template keeps the core algorithm fixed and lets subclasses change only the parts that differ. This avoids code duplication and ensures consistent workflow.

374. Questions on Kafka

I have not personally worked on Kafka. These questions are collected from trusted and well-known resources. If Kafka experience is mentioned in the project, it is good to review these topics. Otherwise, going through them is optional.

1. What is Apache Kafka and why is it used?

Apache Kafka is a system used to move data from one place to another in real time. We can think of it like a strong pipeline where data keeps flowing without stopping. Applications send data into Kafka, and other applications read that data when needed. It is used when systems need fast, reliable, and continuous data sharing. For example, when an order is placed, Kafka can pass that order data to billing, inventory, and notification systems at the same time without delay.

2. How does Kafka differ from traditional messaging systems?

Traditional messaging systems usually delete messages once they are read. Kafka works differently. It keeps messages for a defined time, even after they are read. This allows many consumers to read the same data at different times. Kafka is also built for high data volume and speed, while traditional systems struggle at scale. For example, logs sent to Kafka can be replayed again for debugging, which is not possible in most old messaging systems.

3. What are Kafka producers and consumers?

Producers are applications that send data to Kafka. Consumers are applications that read data from Kafka. We can imagine producers as writers and consumers as readers. A producer sends messages like user activity or payment details. Consumers read those messages and process them. For example, one producer can send order events, while multiple consumers read them for analytics, email alerts, and database storage at the same time.

4. What is a Kafka topic?

A Kafka topic is like a named channel where messages are stored. All messages related to one purpose go into the same topic. For example, all order events can go into an orders topic. Producers write messages to a topic, and consumers read from that topic. Topics help organize data so systems know where to send and read messages. Each topic can hold a very large amount of data safely.

5. How are Kafka topics partitioned and why does it matter?

A topic is split into smaller parts called partitions. **Each partition stores messages in order.** Partitioning allows Kafka to handle large data by spreading it across servers. It also helps multiple consumers read data in parallel. For example, if a topic has three partitions, three consumers can read at the same time. This improves speed and performance while still keeping message order within each partition.

6. What is replication in Kafka and why is it important?

Replication means keeping copies of data on multiple brokers. Each partition has one main copy and one or more backup copies. This is important because if one broker fails, data is not lost. Kafka can continue working using the backup copies. For example, if a server crashes during order processing, Kafka still has the data on another server, so the system keeps running without interruption.

7. What is the role of ZooKeeper in Kafka?

ZooKeeper helps Kafka manage its cluster. It keeps track of brokers, topics, and which broker is the leader for each partition. It also helps detect failures. When a broker goes down, ZooKeeper helps Kafka choose a new leader. Without ZooKeeper, coordination between brokers would be difficult. It acts like a manager that ensures all Kafka parts stay in sync and work together.

8. How does Kafka ensure fault tolerance?

Kafka ensures fault tolerance using replication and leader election. Data is copied to multiple brokers. If the leader broker fails, another broker becomes the leader automatically. Consumers and producers continue working without knowing a failure happened. For example, if one machine goes offline, Kafka switches to another copy of the data. This design helps systems stay available even during hardware or network issues.

9. How does Kafka guarantee message ordering?

Kafka guarantees message order within a single partition. Messages are written and read in the same order they arrive. If all related messages go to the same partition, order is preserved. For example, all updates for one user can be sent to one partition, so consumers always read them in sequence. Across different partitions, ordering is not guaranteed, which is why partition choice matters.

10. What is a Kafka broker?

A Kafka broker is a server that stores data and serves clients. A Kafka cluster is made up of multiple brokers. Each broker handles some partitions and responds to producer and consumer requests. For example, one broker may store order data while another stores payment data. Brokers work together to balance load, store data safely, and keep Kafka running smoothly.

11. What is an offset in Kafka?

An offset is a unique number assigned to each message within a partition. It shows the position of a message. Consumers use offsets to track how much data they have read. For example, if a consumer last read offset 50, it knows the next message is 51. Offsets allow consumers to restart and continue from where they left off without losing data.

12. What is a consumer group?

A consumer group is a set of consumers working together to read a topic. Each partition is read by only one consumer in the group. This helps scale processing. For example, if a topic has four partitions and four consumers in a group, each consumer reads one partition. If a new consumer joins, Kafka redistributes partitions among the group automatically.

13. How does Kafka handle consumer lag?

Consumer lag happens when consumers read messages slower than producers write them. Kafka handles this by keeping data for a set time. Consumers can catch up later. We can monitor lag to see if consumers are slow. For example, if a consumer crashes, data stays in Kafka. When it restarts, it continues from the last offset without losing messages.

14. Explain partition leader and follower roles.

Each partition has one leader and one or more followers. The leader handles all read and write requests. Followers copy data from the leader. If the leader fails, one follower becomes the new leader. This setup ensures data safety and availability. For example, if a leader broker stops, Kafka quickly switches leadership so applications continue working.

15. What are delivery guarantees at most once, at least once, exactly once?

At most once means messages may be lost but never duplicated. At least once means messages are never lost but may be duplicated. Exactly once means messages are delivered once and only once. Kafka supports all three based on configuration. For example, logging systems may allow duplicates, while payment systems often need exactly once to avoid double charges.

16. What is Kafka Connect and why is it useful?

Kafka Connect is a tool to move data between Kafka and other systems without writing much code. It uses connectors to read from databases or write to storage systems. For example, we can pull data from a database into Kafka or push Kafka data into a data warehouse. This saves time and keeps data pipelines simple and consistent.

17. What is Kafka Streams and where is it used?

Kafka Streams is a library used to process data inside Kafka. It allows us to read data from topics, transform it, and write results back to topics. It is used for real time processing like counting events or filtering data. For example, we can read user clicks and calculate live statistics without using a separate big processing system.

18. How does Kafka handle large messages?

Kafka can handle large messages, but it is better to be careful. Large messages use more memory and network time. Kafka allows configuration to increase message size limits. A common approach is to store large data like files in external storage and send only references through Kafka. For example, we can send a file location instead of the file itself.

19. What strategies can be used for consumer group rebalancing?

Rebalancing happens when consumers join or leave a group. Kafka pauses consumption, redistributes partitions, and resumes work. To reduce impact, we can use cooperative rebalancing, keep consumer count stable, and avoid frequent restarts. For example, cooperative rebalancing allows consumers to keep some partitions while others are reassigned, reducing downtime.

20. How do producers decide which partition to send messages to?

Producers choose partitions using a key or a round-robin method. If a key is provided, messages with the same key go to the same partition. This helps maintain order. If no key is given, Kafka spreads messages evenly. For example, using user ID as a key ensures all user events go to the same partition.

21. What is the high-water mark in Kafka?

The high-water mark is the last offset that is safely replicated to all required replicas. Consumers can only read messages up to this point. This ensures they do not read uncommitted or unsafe data. For example, if a message is written but not fully copied, it stays invisible to consumers until replication is complete.

22. How can Kafka be secured SSL, SASL, ACLs?

Kafka security includes encryption, authentication, and authorization. SSL encrypts data in transit. SASL verifies client identity. ACLs control who can read or write topics. Together, they protect data from unauthorized access. For example, only approved applications can produce or consume sensitive data like user information.

23. How do you monitor Kafka cluster health?

Kafka health is monitored using metrics like broker status, consumer lag, disk usage, and request rates. Tools and dashboards help visualize this data. Monitoring helps detect issues early. For example,

high consumer lag may indicate slow processing. Regular monitoring ensures Kafka runs smoothly and avoids unexpected failures.

24. What happens if a Kafka broker fails?

When a broker fails, Kafka detects it quickly. Partition leaders on that broker are replaced by followers from other brokers. Producers and consumers reconnect automatically. Data is not lost because replicas exist. For example, if one server goes down during peak load, Kafka shifts work to other servers and continues processing normally.

25. How does Kafka handle schema evolution with Schema Registry?

Schema Registry manages message structure changes over time. It stores schemas and checks compatibility. This ensures producers and consumers understand each other even when schemas change. For example, adding a new optional field does not break old consumers. Schema Registry helps teams evolve data safely without stopping systems or causing errors.

26. How does Kafka handle message ordering and consistency?

Inside a partition, Kafka always keeps messages in the exact order in which they are written. Each message gets a number called an offset, and this number increases step by step. When producers send messages to the same partition, consumers will always read them in that same order.

Across partitions, Kafka does not promise ordering. Topics are split into partitions to handle more data at the same time. Because partitions work independently, a message in one partition can be processed before or after a message in another partition.

Consistency is handled using replication. Each partition has one leader and other follower replicas. All reads and writes go through the leader. When `acks=all` is used, Kafka confirms a write only after all in-sync replicas have the data. Producers can preserve order by using a key so related messages go to the same partition.

27. What strategies can be used for Kafka consumer group management?

Kafka consumer groups help us read data in parallel and share the load. When many consumers read from the same topic, Kafka decides how work is divided.

First, Kafka uses partition assignment. Partitions are automatically given to consumers inside a group. When a consumer joins or leaves, Kafka rebalances and assigns partitions again.

With range assignment, partitions are given in order. For example, if there are four partitions and two consumers, the first consumer gets the first two partitions and the second consumer gets the remaining two.

With round-robin assignment, partitions are shared evenly so load is balanced across consumers.

Consumers also manage offsets, which tell Kafka which message was read last. Offsets are stored safely so messages are not read again.

28. Kafka achieves high throughput and low latency

Kafka handles a large amount of data while staying fast because of a few simple design ideas. We do not send messages one by one. Producers collect many messages and send them together as a batch.

Message Batching

Kafka stays fast by sending messages in groups instead of one by one. Producers collect multiple messages and send them together as a batch. This reduces the number of network calls and disk writes, which improves throughput.

Data Compression

Before sending data, producers compress messages. This reduces message size, so less data travels over the network and less space is used on disk. This helps Kafka process data faster.

Zero-Copy Transfer

Kafka sends data directly from disk to the network without extra copying inside memory. This saves CPU time and speeds up data transfer.

Efficient Disk Writes

Kafka writes data to disk in a continuous order. This is much faster than random writes and helps reduce delay.

Partitioning and Parallel Work

Topics are split into partitions. This allows many producers and consumers to work at the same time. For example, different consumers can read different partitions together.

29. How would you monitor and troubleshoot Kafka performance issues in production?

Monitoring Kafka in production means keeping an eye on a few important metrics to understand how well the system is working.

Consumer lag shows the gap between the latest message written to Kafka and the message a consumer has processed. If this gap keeps increasing, it means consumers are not able to keep up with incoming data.

Throughput tells us how much data Kafka is handling. We check how fast producers send messages and how fast consumers read them. This helps us know whether Kafka is overloaded or working smoothly.

Broker health is also critical. We monitor disk usage, CPU load, and network input and output on each broker. If disk space is low or CPU usage is high, performance can drop.

Replication lag shows how far follower replicas are behind the leader. High replication lag can reduce fault tolerance and increase the risk of data issues.

375. February - 2026